

ProductionFood — Documentación de Preparación Técnica

Documentación de Preparación Técnica

Corporación Universitaria Antonio José de Sucre — Electiva Profesional II

18 de septiembre de 2026

Presentación

Qué contiene

Tres hallazgos de la revisión previa

Parte I · Base del Proyecto

00 — Stack Tecnológico y Versiones

01 — Convenciones del Proyecto

02 — Correcciones al Esquema de Base de Datos

03 — Matriz de Roles y Permisos

04 — Contrato de API

05 — Estándares de QA

06 — Arquitectura Backend

07 — Arquitectura Frontend

08 — Desviaciones frente a la Entrega 1

09 — Puesta en Marcha del Entorno

10 — Roadmap y Dependencias

11 — Despliegue de la Base de Datos en AWS RDS

Parte II · Historias de Usuario

Fase 1 · Infraestructura y Seguridad

HU-01 · Registro de usuarios

HU-02 · Gestión de usuarios

HU-03 · Inicio de sesión

HU-04 · Bitácora de auditoría

Fase 2 · Datos Maestros

HU-05 · Gestión de clientes

HU-06 · Gestión de proveedores

HU-07 · Unidades de medida

Fase 3 · Inventario de Materias Primas

HU-08 · Gestión de materias primas

HU-09 · Lotes de materia prima

HU-10 · Consulta de inventario de MP

Fase 4 · Catálogo y Recetas

HU-11 · Gestión de productos

HU-12 · Gestión de recetas

Fase 5 · Compras

HU-13 · Registro de compras

HU-14 · Detalle de compra

HU-15 · Historial de compras

Fase 6 · Pedidos y Ventas

HU-16 · Creación de pedidos

HU-17 · Productos en el pedido

HU-18 · Registro de devoluciones

HU-19 · Lotes de producto terminado

Fase 7 · Producción

HU-20 · Plan de producción

HU-21 · Productos del plan

HU-22 · Consumo de materia prima

HU-23 · Inventario automático por producción

Fase 8 · Reportes y Consultas

HU-24 · Dashboard resumen

HU-25 · Consulta de pedidos

HU-26 · Eficiencia de producción

HU-27 · Alerta de stock bajo

Parte III · Anexos

Scripts de migración

Presentación

ProductionFood — Sistema Web para la Planificación y Gestión de la Producción en Pymes de Alimentos.

Este documento consolida la preparación técnica previa al desarrollo: las decisiones transversales del proyecto y la especificación completa de las 27 historias de usuario, con sus tareas de backend, frontend y QA.

Qué contiene

Parte	Contenido
I. Base	Stack y versiones, convenciones, correcciones al esquema de datos, matriz de roles, contrato de API, estándares de QA, arquitectura, desviaciones frente a la Entrega 1, puesta en marcha, roadmap y despliegue en AWS RDS
II. Historias	27 historias x 5 documentos: especificación, diseño, tarea de backend, tarea de frontend y casos de prueba
III. Anexos	Scripts SQL de migración

Tres hallazgos de la revisión previa

- 1. El esquema tenía tres huecos bloqueantes.** No existía el precio en el detalle del pedido, no había forma de consultar el stock de producto terminado por producto, y **nada sumaba materia prima al inventario** — el sistema solo descontaba. Las tres correcciones están en la Parte I y en las migraciones del anexo.
- 2. Varias versiones del stack estaban fuera de soporte**, incluida la base de datos: Amazon RDS para MySQL 8.0 terminó su soporte estándar el 31 de julio de 2026 y pasa a facturarse como Extended Support.
- 3. El documento de Entrega 1 y las historias de usuario no coincidían** en alcance ni en actores. Las diferencias están registradas con lo que hay que corregir para la Entrega 2.

Parte I · Base del Proyecto

00 — Stack Tecnológico y Versiones

Revisión de versiones a **septiembre de 2026**. Varias de las declaradas originalmente están fuera de soporte; aquí está el stack corregido y por qué.

1. Auditoría del stack original

Componente	Versión original	Estado a sep-2026	Acción
Java	21 (LTS)	✓ LTS con soporte, pero ya no es el LTS actual	Actualizar a 25
Spring Boot	3.3.x	✗ Fuera de soporte OSS (3.3 terminó en 2025)	Actualizar a 4.1.x
Base de datos	MySQL 8.0 en AWS RDS	Fin de soporte estándar en RDS: 31-jul-2026	Actualizar a MySQL 8.4 LTS
Angular	17	✗ Fuera de soporte (nov 2023, ciclo de 18 meses)	Actualizar a 22
SpringDoc OpenAPI	2.6.0	✗ Incompatible con Spring Boot 4	Actualizar a 3.1.1
jjwt	0.12.6	⚠ Funciona, hay versión más reciente	Actualizar a 0.13.0
Maven	3.8.x	⚠ Antiguo	Actualizar a 3.9.x
Angular Material	—	Sigue la versión de Angular	22.x

El caso urgente: MySQL 8.0 en RDS

Amazon RDS para MySQL 8.0 alcanzó el fin de soporte estándar el 31 de julio de 2026, mes y medio antes del inicio de este proyecto. La comunidad MySQL lo declaró EOL en abril de 2026.

Qué implica en la práctica:

- Una instancia RDS que siga en 8.0 pasa automáticamente a **Extended Support**, que **se factura aparte y encarece la instancia**. En una cuenta de estudiante con crédito limitado, eso consume el presupuesto sin que nadie se dé cuenta hasta ver la factura.
- AWS puede forzar la actualización mayor en una ventana de mantenimiento.
- Crear una instancia nueva en 8.0 hoy es empezar pagando soporte extendido por una versión obsoleta.

Se crea la instancia directamente en MySQL 8.4 LTS. No es una preferencia técnica: es la única versión que no incurre en cargos de soporte extendido.

Nota sobre MariaDB. La tabla de stack de las historias de usuario mencionaba MariaDB 10.4 local. El documento de arquitectura (Entrega 1) declara MySQL en AWS RDS, y esa es la decisión vigente confirmada por el equipo. Toda la documentación, las migraciones y el driver apuntan a **MySQL 8.4 en AWS RDS**.

2. Stack adoptado

Backend

Capa	Tecnología	Versión	Soporte hasta
Lenguaje	Java (OpenJDK / Temurin)	25 LTS	Sep 2030 (Premier)
Framework	Spring Boot	4.1.x	Jul 2027 (OSS)
Seguridad	Spring Security (incluido en Boot 4.1)	7.x	Con Boot 4.1
JWT	<code>io.jsonwebtoken:jjwt</code>	0.13.0	Activo
Acceso a datos	Spring <code>JdbcTemplate</code> (sin ORM)	Con Boot 4.1	—
Base de datos	MySQL en AWS RDS	8.4 LTS	Abr 2032 (comunidad)
Driver	<code>com.mysql:mysql-connector-j</code>	9.x	Activo
Build	Maven	3.9.x	Activo
Documentación API	SpringDoc OpenAPI	3.1.1	Activo
Validación	Jakarta Validation	Con Boot 4.1	—
Pruebas	JUnit 5 + Mockito + MockMvc	Con Boot 4.1	—

Frontend

Capa	Tecnología	Versión
Framework	Angular	22.x
Lenguaje	TypeScript	5.9.x (la que fije Angular 22)
UI	Angular Material	22.x
Node.js	Node	22 LTS o 24 LTS
Gestor de paquetes	npm	10+

Herramientas

Uso	Herramienta
Pruebas de API	Postman
Control de versiones	Git + GitHub
IDE backend	IntelliJ IDEA Community 2026.x
IDE frontend	VS Code

3. Justificación de cada cambio

Java 21 → 25 LTS

Java 21 **sigue soportado** y no es una elección incorrecta. Se actualiza porque:

- Java 25 es el LTS vigente desde septiembre de 2025, con Premier Support hasta 2030.
- El período de solapamiento de licencia permisiva de Java 21 termina en **octubre de 2026**, el mes siguiente al inicio de este proyecto.
- Spring Boot 4.1 soporta Java 17–26; no hay fricción.

Aporta al proyecto, sin ser determinante: *pattern matching* maduro para los `switch` de transición de estado, y `record` para los DTO —que se usan en todo el proyecto— ya plenamente asentados.

Spring Boot 3.3 → 4.1

Spring Boot 3.3 terminó su soporte OSS en 2025, y **3.5 lo terminó en junio de 2026**. Arrancar un proyecto nuevo sobre una rama sin soporte no tiene defensa razonable cuando no hay código heredado que migrar: el costo de elegir bien es cero hoy y creciente después.

⚠ **Riesgo asumido, y cómo se mitiga.** Spring Boot 4 trae Spring Framework 7 y Spring Security 7. La mayoría de tutoriales de JWT con Spring que se encuentran en línea son para Boot 3.x, y la configuración de seguridad cambió. El equipo va a encontrar ejemplos que no compilan.

Por eso `06-ARQUITECTURA-BACKEND.md` incluye la configuración de seguridad **escrita para Spring Security 7**, no un enlace a un tutorial. Ese documento es la referencia; si un ejemplo de internet lo contradice, el ejemplo está desactualizado.

Plan B: si el equipo se bloquea más de dos días con la configuración de seguridad, es aceptable retroceder a **Spring Boot 3.5.x**, que tiene soporte comercial hasta 2032 y material de aprendizaje abundante. Se documentaría como deuda técnica. Es preferible un proyecto que avanza en 3.5 a uno detenido en 4.1.

MySQL 8.0 → 8.4 LTS (AWS RDS)

Explicado arriba: 8.0 ya está fuera de soporte estándar en RDS y entra en facturación extendida. MySQL 8.4 es LTS, con soporte de la comunidad hasta abril de 2032 — cubre el proyecto con margen sobrado.

Lo que cambia entre 8.0 y 8.4 **no afecta a este proyecto**: las diferencias están en replicación, plugins de autenticación por defecto y variables de sistema retiradas. El SQL de las migraciones es idéntico en ambas.

Sí hay dos detalles de sintaxis MySQL que importan y que difieren de MariaDB (ver [02-CORRECCIONES-DB.md](#) §Notas MySQL):

- Las expresiones en `DEFAULT` van **entre paréntesis**: `DEFAULT (curdate())`. Sin los paréntesis, MySQL rechaza la sentencia.
- El ancho de visualización `int(10)` está **obsoleto desde MySQL 8.0.17**. El esquema usa `int unsigned`.

Driver: `mariadb-java-client` → `mysql-connector-j`

Consecuencia directa del cambio de motor. Cambia también la URL JDBC:

```
jdbc:mysql://<endpoint-rds>:3306/productionfood
```

Angular 17 → 22

Angular 17 es de noviembre de 2023 y quedó fuera del ciclo de soporte (18 meses) a mediados de 2025. Angular 22 (junio de 2026) es el actual.

Lo que cambia para este proyecto, en concreto:

- **Signals estables** — el estado local por componente que pide [01-CONVENCIONES.md](#) §4 se resuelve sin `BehaviorSubject`.
- **Componentes standalone por defecto** — desde Angular 19 no hay que declarar `standalone: true`; los `NgModule` por feature ya no se usan.
- **Nueva sintaxis de control de flujo** (`@if`, `@for`, `@switch`) en lugar de `*ngIf` / `*ngFor`. Es la forma actual y la que se usa en los ejemplos de estos documentos.
- **Angular ARIA** (estable en 22) — accesibilidad sin trabajo extra.

Igual que con la base de datos: el proyecto no tiene código todavía, así que el costo de migración es exactamente cero.

SpringDoc 2.6.0 → 3.1.1

Obligado: SpringDoc 2.x **no funciona con Spring Boot 4**. La rama 3.x existe precisamente para dar soporte a Boot 4. No es una preferencia, es la única versión que arranca.

4. Dependencias Maven

```

<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>4.1.1</version>
  <relativePath/>
</parent>

<properties>
  <java.version>25</java.version>
  <springdoc.version>3.1.1</springdoc.version>
  <jjwt.version>0.13.0</jjwt.version>
</properties>

<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-jdbc</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-aop</artifactId>
  </dependency>

  <dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <scope>runtime</scope>
  </dependency>

  <dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
    <version>${springdoc.version}</version>
  </dependency>

  <!-- jjwt: api en compile, impl y jackson en runtime (es el patrón del proyecto) -->
  <dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>${jjwt.version}</version>
  </dependency>
  <dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-impl</artifactId>
    <version>${jjwt.version}</version>
    <scope>runtime</scope>
  </dependency>
  <dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-jackson</artifactId>
    <version>${jjwt.version}</version>
    <scope>runtime</scope>
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.springframework.security</groupId>
    <artifactId>spring-security-test</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>

```

5. Dependencias npm

```

npm install -g @angular/cli@22
ng new productionfood-frontend --routing --style=scss --ssr=false
cd productionfood-frontend
ng add @angular/material@22

```

`--ssr=false` : es una aplicación interna tras autenticación, no un sitio público. El renderizado del lado del servidor añade complejidad sin beneficio aquí.

6. Compromiso de mantenimiento

1. **Nadie actualiza una versión mayor sin acuerdo del equipo.** Las versiones de este documento se fijan en `pom.xml` y `package.json` , y se respetan.
2. **`package-lock.json` y las versiones exactas del `pom.xml` se versionan en Git.** Sin esto, cada integrante compila contra dependencias distintas y aparecen fallos que solo le ocurren a una persona.
3. **Se revisa este documento al inicio de cada fase.** Si sale una versión de parche con arreglo de seguridad, se aplica; las versiones mayores esperan al final del proyecto.

7. Fuentes

Versiones verificadas en septiembre de 2026:

- Spring Boot — [endoflife.date/spring-boot](#)
- Angular — [endoflife.date/angular](#)
- MySQL en RDS — [Versiones de MySQL en Amazon RDS](#) · [Estrategias de actualización 8.0 → 8.4](#) · [RDS Extended Support](#)
- Java — [OpenJDK 25](#) · [Oracle Java SE Support Roadmap](#)
- SpringDoc — [springdoc.org/v4](#)
- jjwt — [github.com/jwt/jwt](#)

01 — Convenciones del Proyecto

Documento normativo. Cualquier código que no siga estas convenciones se rechaza en revisión.

1. Idioma

Elemento	Idioma	Ejemplo
Tablas y columnas de BD	Español (tal como está en <code>db.sql</code>)	<code>materias_primas.stock_minimo</code>
Rutas de API	Español	<code>/api/v1/materias-primas</code>
Clases, métodos, variables Java	Español para el dominio, inglés para lo técnico	<code>MateriaPrimaService</code> , <code>findById</code>
Componentes y servicios Angular	Español para el dominio	<code>materia-prima-list.component.ts</code>
Mensajes al usuario final	Español	<code>"El correo ya está registrado"</code>
Comentarios y documentación	Español	—
Códigos de error internos	Inglés, SCREAMING_SNAKE	<code>EMAIL_ALREADY_EXISTS</code>

Regla anti-ambigüedad: nunca mezclar `materiaPrima` y `rawMaterial` en el mismo flujo. El dominio es español, punto.

2. Backend — estructura de paquetes

Paquete raíz: `co.edu.corposucre.productionfood`

```

co.edu.corposucre.productionfood
├── ProductionFoodApplication.java
├── config/
│   ├── OpenApiConfig.java          Configuración de Swagger/SpringDoc
│   ├── DataSourceConfig.java       JdbcTemplate + NamedParameterJdbcTemplate
│   └── WebConfig.java              CORS
├── security/
│   ├── SecurityConfig.java         SecurityFilterChain, reglas por ruta
│   ├── JwtService.java             Firma y validación de tokens
│   ├── JwtAuthenticationFilter.java Filtro OncePerRequest
│   ├── UsuarioDetailsService.java  Carga de usuario para autenticación
│   └── SecurityUtils.java          Acceso al usuario autenticado actual
├── common/
│   ├── error/
│   │   ├── ApiException.java       Excepción base con código y estado HTTP
│   │   ├── RecursoNoEncontradoException.java
│   │   ├── ConflictoNegocioException.java
│   │   ├── ErrorResponse.java      DTO de error (ver 04-CONTRATO-API.md)
│   │   └── GlobalExceptionHandler.java @RestControllerAdvice
│   ├── pagination/
│   │   ├── PageRequest.java        page, size, sort – validado
│   │   └── PageResponse.java        content, page, size, totalElements, totalPages
│   ├── audit/
│   │   ├── Auditable.java          Anotación de método
│   │   ├── AuditoriaAspect.java    AOP que escribe en bitacora
│   │   └── BitacoraService.java
│   └── <modulo>/                  Un paquete por agregado de negocio
│       ├── UsuarioController.java
│       ├── UsuarioService.java
│       ├── UsuarioRepository.java
│       ├── UsuarioRowMapper.java
│       ├── Usuario.java             Entidad (POJO plano, sin anotaciones JPA)
│       └── dto/
│           ├── CrearUsuarioRequest.java
│           ├── ActualizarUsuarioRequest.java
│           └── UsuarioResponse.java

```

Módulos previstos

usuario, rol, auth, bitacora, cliente, proveedor, unidadmedida, materiaprima, lote, inventario, producto, receta, compra, pedido, devolucion, produccion, reporte.

Responsabilidad de cada capa

Capa	Sí hace	No hace
Controller	Mapear HTTP ↔ DTO, validar con <code>@Valid</code> , declarar <code>@PreAuthorize</code>	Lógica de negocio, SQL, transacciones
Service	Reglas de negocio, orquestación, <code>@Transactional</code> , auditoría	Construir SQL, conocer <code>HttpServletRequest</code>
Repository	SQL con <code>JdbcTemplate</code> , <code>RowMapper</code> , paginación	Reglas de negocio, lanzar excepciones de dominio
RowMapper	<code>ResultSet</code> → entidad	Cualquier otra cosa

Sin ORM. Todo el SQL es explícito y vive en el repositorio. Nunca en el servicio, nunca en el controlador.

3. Backend — reglas de código

SQL

- **Siempre parametrizado.** Cero concatenación de valores. Ni siquiera “porque es un int”.

```

// MAL
jdbc.query("SELECT * FROM usuarios WHERE id_usuario = " + id, mapper);
// BIEN
jdbc.query("SELECT * FROM usuarios WHERE id_usuario = ?", mapper, id);

```

- **Filtros dinámicos** con `NamedParameterJdbcTemplate` y `StringBuilder` sobre el `WHERE`, nunca interpolando el valor:

```

var sql = new StringBuilder("SELECT ... FROM clientes WHERE 1=1");
var params = new MapSqlParameterSource();
if (nombre != null) { sql.append(" AND nombre LIKE :nombre"); params.addValue("nombre", "%" + nombre + "%"); }

```

- **ORDER BY dinámico:** el campo de ordenamiento se valida contra una lista blanca (`Set.of("nombre", "correo", "id_usuario")`) antes de concatenarse. Nunca se acepta el valor crudo del cliente.
- **Nunca `SELECT *`** en producción: se listan las columnas, para que un `ALTER TABLE` no rompa el `RowMapper` en silencio.

Transacciones

- `@Transactional` va en el **servicio**, nunca en el repositorio ni en el controlador.
- Toda operación que toque más de una tabla es transaccional. Sin excepciones.
- `@Transactional(readOnly = true)` en consultas: es documentación ejecutable.

Validación

- Bean Validation (`jakarta.validation`) en los DTO de request para lo sintáctico: obligatoriedad, longitud, rango, formato.
- Reglas que requieren consultar la base (unicidad, existencia de FK, stock suficiente) van en el **servicio**, no en anotaciones.

Nulos

- Los repositorios devuelven `Optional<T>` para búsquedas por id.
- El servicio convierte `Optional.empty()` en `RecursoNoEncontradoException`.
- Los controladores nunca ven un `null` de dominio.

4. Frontend — estructura

Angular 17+ con **componentes standalone** (sin `NgModule` por feature).

```
src/app/
├── app.config.ts           Providers raíz, interceptores, router
├── app.routes.ts          Rutas de primer nivel con lazy loading
├── core/
│   ├── auth/
│   │   ├── auth.service.ts  login, logout, token, usuario actual
│   │   ├── auth.guard.ts    canActivate – sesión válida
│   │   ├── rol.guard.ts     canActivate – rol autorizado
│   │   └── auth.interceptor.ts Adjunta Authorization: Bearer
│   ├── http/
│   │   ├── error.interceptor.ts 401→logout, 403→aviso, 5xx→snackbar
│   │   └── api.config.ts       baseUrl desde environment
│   └── ui/
│       ├── notificacion.service.ts Wrapper de MatSnackBar
│       └── confirmacion.service.ts Wrapper de MatDialog para confirmar
├── shared/
│   ├── components/
│   │   ├── tabla-paginada/    Tabla genérica con MatPaginator + MatSort
│   │   ├── barra-filtros/
│   │   ├── estado-chip/      Chip de color según estado
│   │   └── confirmar-dialog/
│   ├── pipes/                moneda, cantidad, estado-activo
│   └── models/               Interfaces compartidas (ApiResponse, ApiError)
├── features/
│   ├── <modulo>/
│   │   ├── <modulo>.routes.ts
│   │   ├── services/<modulo>.service.ts
│   │   └── models/<modulo>.model.ts
│   └── pages/
│       ├── <modulo>-lista/
│       └── <modulo>-formulario/
```

Reglas frontend

- **Reactive Forms siempre.** Nada de `ngModel` en formularios de dominio.
- **Un servicio HTTP por módulo**, tipado con interfaces. Nunca `any` en la respuesta.
- **Los componentes no llaman a `HttpClient` directamente**, solo a su servicio.
- **Estado local por componente** con signals. No se introduce NgRx en este alcance.
- **Modelos TypeScript espejo de los DTO del backend.** Si cambia el DTO, cambia la interfaz en el mismo PR.
- **Sin lógica de negocio duplicada:** la validación del cliente es para experiencia de usuario; la autoridad es el backend. Que el formulario deshabilite el botón no exime al servidor de validar.

5. Nomenclatura

Rutas de API

- Plural, kebab-case: `/api/v1/materias-primas`, `/api/v1/planes-produccion`
- Subrecursos anidados cuando el hijo no existe sin el padre: `/api/v1/compras/{idCompra}/detalles`, `/api/v1/recetas/{idReceta}/detalles`
- Acciones que no son CRUD, como subrecurso de estado: `PATCH /api/v1/pedidos/{id}/estado`, `POST /api/v1/compras/{id}/recepcion`

Java

Elemento	Convención	Ejemplo
Clase	PascalCase	MateriaPrimaService
Método	camelCase, verbo primero	buscarPorNombre , registrarConsumo
Constante	SCREAMING_SNAKE	MAX_PAGE_SIZE
DTO de entrada	<Accion><Entidad>Request	CrearClienteRequest
DTO de salida	<Entidad>Response	ClienteResponse

Angular

Elemento	Convención	Ejemplo
Archivo	kebab-case	materia-prima-lista.component.ts
Clase	PascalCase + sufijo	MateriaPrimaListaComponent
Interface	PascalCase, sin prefijo I	MateriaPrima , PageResponse<T>
Observable	sufijo \$	materiasPrimas\$

Base de datos

Se respeta lo existente en `db.sql`: `snake_case`, tablas en plural, PK `id_<singular>`. Las tablas nuevas siguen la misma convención.

6. Git

- **Rama por HU:** `feat/HU-08-materias-primas`, `fix/HU-12-receta-duplicada`
- **Commits atómicos**, bajo ~50 líneas, con el prefijo de la HU:

```
feat(HU-08): agregar MateriaPrimaRepository con filtro por nombre
Implementa búsqueda paginada con lista blanca de ordenamiento.
```

- **Un PR por capa de HU** (backend / frontend), bajo ~250 líneas. Una HU grande se parte en varios PR; no se acumula.
- Nunca se hace push directo a `main`.

7. Definición de Terminado (DoD)

Una HU está terminada cuando **todas** se cumplen:

- ☐ Todos los criterios de aceptación de la HU están implementados y verificados.
- ☐ Los endpoints están documentados en Swagger con ejemplos de request y response.
- ☐ Los casos de prueba de `task-qa.md` pasan, incluidos los negativos.
- ☐ El frontend maneja los tres estados: cargando, vacío y error.
- ☐ Las acciones de escritura quedan registradas en bitácora (ver HU-04).
- ☐ La autorización por rol está aplicada y probada con un rol no autorizado.
- ☐ No hay `System.out.println`, `console.log`, ni código comentado.
- ☐ El PR fue revisado y aprobado por alguien distinto de quien lo escribió.

02 — Correcciones al Esquema de Base de Datos

Auditoría del `db.sql` original contra los criterios de aceptación de las 27 historias. Cada hallazgo indica qué HU se rompe y cómo se corrige.

Todo lo aquí descrito está implementado en `migraciones/V2__correcciones.sql`. Nadie modifica la base con SQL suelto: se agrega una migración.

Resumen

#	Hallazgo	Severidad	HU afectadas
C-01	<code>detalle_pedido</code> sin <code>precio_unitario</code>	Bloqueante	HU-17, HU-25
C-02	No existe stock de producto terminado por producto	Bloqueante	HU-17, HU-19, HU-24
C-03	No hay entrada de materia prima al inventario	Bloqueante	HU-10, HU-13, HU-14, HU-23, HU-27
C-04	Sin trazabilidad de movimientos de stock (kardex)	Alta	HU-10, HU-17, HU-18, HU-23
C-05	<code>detalle_receta</code> sin UNIQUE(receta, materia prima)	Alta	HU-12
C-06	<code>estado</code> sin <code>DEFAULT 1</code> en 4 tablas	Media	HU-01, HU-08, HU-11, HU-12
C-07	Fechas “automáticas” sin <code>DEFAULT</code>	Media	HU-16, HU-18, HU-10
C-08	<code>consumo_materia_prima.cantidad_real</code> nullable	Media	HU-22, HU-26
C-09	División por cero en el cálculo de eficiencia	Media	HU-22, HU-26
C-10	<code>lotes</code> permite estados imposibles	Media	HU-09, HU-19
C-11	<code>bitacora.id_usuario</code> NOT NULL bloquea auditoría del sistema	Media	HU-04
C-12	Sin restricción de estados válidos	Media	HU-13, HU-16, HU-18, HU-20
C-13	Sin UNIQUE en líneas de detalle	Media	HU-14, HU-21, HU-22
C-14	Sin CHECK de cantidades y precios positivos	Media	Transversal
C-15	Faltan índices para los filtros que piden las HU	Baja	HU-02, HU-05, HU-06, HU-15, HU-25
C-16	Sin datos semilla (roles, unidades, admin inicial)	Bloqueante operativo	HU-01, HU-03, HU-07

C-01 · `detalle_pedido` no guarda el precio — Bloqueante

Problema. La tabla solo tiene `id_pedido`, `id_producto`, `cantidad`. No hay precio.

Dos consecuencias, y la segunda es la grave:

1. No se puede calcular el total de un pedido. HU-25 CA-04 pide mostrar el detalle del pedido; un detalle de venta sin importe no sirve.
2. **Se pierde el histórico.** Si el precio se lee de `productos.precio` al consultar, un pedido de hace tres meses se re-valoriza con el precio de hoy. Los reportes cambian solos cada vez que alguien edita el catálogo.

Corrección. `detalle_pedido` gana `precio_unitario DECIMAL(10,2) NOT NULL`, que se copia de `productos.precio` **en el momento de agregar la línea** y nunca se recalcula.

Mismo razonamiento aplica a `detalle_compra`, que sí tiene `precio_unitario` — correcto de origen.

C-02 · No existe el stock de producto terminado por producto — Bloqueante

Problema. HU-17 CA-04 exige “verificar que haya stock suficiente en `inventario_producto_terminado`”. Pero esa tabla está indexada por **lote** (`id_lote` UNIQUE), no por producto:

```
inventario_producto_terminado(id_inv_producto, id_lote, cantidad_disponible, fecha_ingreso_estante)
lotes(id_lote, codigo_lote, id_producto, id_materia_prima, ...)
```

Para saber cuántos panes hay disponibles hay que agregar todos los lotes de ese producto. No es una consulta directa, y peor: **la HU no dice de qué lote descontar** cuando hay varios.

Corrección. Dos partes:

1. **Vista** `v_stock_producto_terminado` que agrega por producto. El backend consulta la vista para validar disponibilidad.
2. **Política de salida FEFO** (*First Expired, First Out* — primero el que vence antes), declarada como regla de negocio. Es lo correcto para alimentos perecederos: si se descontara del lote más nuevo, el viejo caduca en el estante.

La vista excluye lotes vencidos: producto caducado no es stock vendible.

C-03 · Nada suma materia prima al inventario — Bloqueante

Problema. El flujo completo de inventario de materias primas está incompleto:

- HU-23 **descuenta** de `inventario` al registrar consumo de producción. ✓
- HU-10 **consulta** `inventario`. ✓
- HU-27 **alerta** cuando `inventario` está bajo. ✓
- **Nadie suma**. Ni HU-09 (registrar lote de MP) ni HU-13/HU-14 (compras) dicen que se incremente `inventario`.

Resultado: la tabla `inventario` arranca vacía, nunca crece, y toda la Fase 3, 7 y 8 opera sobre ceros. Tres historias completas quedan sin sentido.

Corrección — decisión de negocio. Se definen **dos puntos de entrada**, ambos transaccionales y ambos generan movimiento de kardex:

Punto de entrada	Cuándo	Qué hace
Recepción de compra (principal)	<code>PATCH /compras/{id}/estado → RECIBIDA</code>	Suma al inventario cada línea de <code>detalle_compra</code> . Opcionalmente crea lotes.
Registro de lote de MP (secundario)	<code>POST /lotes-materia-prima</code> (HU-09)	Suma <code>cantidad</code> al inventario de esa materia prima.

Importante: si una compra se recibe y *además* se registran lotes por esas mismas cantidades, el stock se duplica. Regla: **un lote registrado contra una compra recibida no vuelve a sumar**; se marca con `id_detalle_compra`. Ver `V2_correcciones.sql`.

La transición `RECIBIDA → PENDIENTE` (revertir recepción) se prohíbe: una vez que el stock entró, deshacerlo requiere un movimiento de ajuste explícito, no un cambio de estado.

C-04 · Sin kardex: el stock muta sin dejar rastro

Problema. Cuatro operaciones distintas modifican `inventario.cantidad_disponible` (recepción de compra, registro de lote, consumo de producción, ajuste) y tres modifican `inventario_producto_terminado` (alta de lote, venta, devolución aprobada). Pero solo se guarda el **saldo actual**.

Cuando el contador diga “aquí faltan 40 kg de harina”, no habrá forma de saber qué pasó. La bitácora de HU-04 no sustituye esto: registra “alguien actualizó la tabla inventario”, con un `varchar(255)` de detalle. No es un libro de movimientos consultable.

Corrección. Dos tablas de kardex, `movimientos_inventario_mp` y `movimientos_inventario_pt`, con:

- Tipo de movimiento (`ENTRADA_COMPRA`, `ENTRADA_LOTE`, `SALIDA_PRODUCCION`, `SALIDA_VENTA`, `ENTRADA_DEVOLUCION`, `AJUSTE_POSITIVO`, `AJUSTE_NEGATIVO`, `BAJA_VENCIMIENTO`)
- Cantidad, **saldo anterior y saldo posterior** (permite detectar inconsistencias)
- Referencia al documento que lo originó (`tabla_origen` + `id_origen`)
- Usuario y fecha

Regla: `inventario` **nunca se actualiza sin insertar el movimiento correspondiente, en la misma transacción**. Se centraliza en un único `InventarioService` con métodos `registrarEntrada()` / `registrarSalida()`; ningún otro servicio hace `UPDATE inventario`.

Esto convierte HU-10 y HU-27 en consultas auditables y da base real a HU-26.

C-05 · `detalle_receta` admite ingredientes duplicados

Problema. HU-12 CA-05: “No se permite duplicar la misma materia prima en una receta”. La tabla no tiene esa restricción. Validarlo solo en el servicio deja la puerta abierta a inserciones concurrentes: dos peticiones simultáneas pasan ambas la comprobación `SELECT ... WHERE` y ambas insertan.

Corrección. `UNIQUE KEY uk_detalle_receta (id_receta, id_materia_prima)`.

El servicio sigue validando para devolver un `409` legible, pero la base es la que garantiza. Es la diferencia entre un mensaje amable y una regla real.

C-06 · `estado` sin valor por defecto

Problema. Cuatro HU dicen “estado por defecto activo (`1`)”:

Tabla	Columna	HU	Estado en <code>db.sql</code>
<code>usuarios</code>	<code>estado</code>	HU-01 CA-05	<code>tinyint(1) NOT NULL</code> sin default
<code>materias_primas</code>	<code>estado</code>	HU-08 CA-04	<code>tinyint(1) NOT NULL</code> sin default
<code>productos</code>	<code>estado</code>	HU-11 CA-05	<code>tinyint(1) NOT NULL</code> sin default
<code>recetas</code>	<code>estado</code>	HU-12	<code>tinyint(1) NOT NULL</code> sin default

`clientes` y `proveedores` sí lo tienen. La inconsistencia es fuente segura de bugs: un `INSERT` que omita la columna falla en unas tablas y no en otras.

Corrección. `DEFAULT 1` en las cuatro.

C-07 · Fechas “automáticas” que no lo son

Tabla.columna	La HU dice	<code>db.sql</code> dice
<code>pedidos.fecha_pedido</code>	HU-16 CA-02: “se registra automáticamente con la fecha actual”	<code>date NOT NULL</code> sin default
<code>devoluciones.fecha_devolucion</code>	HU-18 CA-05: “se registra automáticamente”	<code>date NOT NULL</code> sin default
<code>inventario.fecha_actualizacion</code>	HU-10 CA-02 / HU-23 CA-02	<code>datetime NOT NULL</code> sin default

Corrección. `DEFAULT curdate()` en las dos primeras; `DEFAULT current_timestamp() ON UPDATE current_timestamp()` en la tercera — así se actualiza sola en cada `UPDATE` y se elimina la posibilidad de que alguien olvide tocarla.

C-08 · `cantidad_real` nullable contra su propio criterio

Problema. HU-22 CA-04: “`cantidad_real` es obligatoria (lo que realmente se usó)”. La columna es `decimal(10,3) DEFAULT NULL`.

Además rompe HU-23 CA-01, que descuenta `cantidad_real` del inventario: descontar `NULL` propaga `NULL` a `cantidad_disponible` y corrompe el saldo.

Corrección. `NOT NULL`. Si el flujo real necesita registrar el consumo teórico antes de conocer el real, eso es un estado distinto del detalle del plan, no un `NULL` en una columna de cantidad.

C-09 · La fórmula de eficiencia divide por cero

Problema. HU-22 CA-06 define `eficiencia = (cantidad_teorica / cantidad_real) * 100`.

Si `cantidad_real = 0` — perfectamente posible: se planificó un producto y no se consumió nada de ese ingrediente — la división revienta. MySQL devuelve `NULL` en lugar de lanzar error, así que **falla en silencio** y el reporte de HU-26 muestra huecos sin explicación aparente.

Hay un segundo problema, conceptual: con esa fórmula, consumir *menos* de lo teórico da más de 100%. Un 200% de eficiencia no significa el doble de bueno, significa que se usó la mitad del ingrediente — lo cual probablemente sea un error de registro, no una hazaña.

Corrección. - En SQL: `CASE WHEN cantidad_real = 0 THEN NULL ELSE (cantidad_teorica / cantidad_real) * 100 END`. - En la capa de servicio: se calcula con `BigDecimal`, se devuelve `null` cuando `cantidad_real = 0`, y el frontend muestra `-` en vez de `NaN` o `Infinity`. - Se documenta que **la desviación es la métrica interpretable**: `desviacion% = ((real - teorica) / teorica) * 100`. Positiva = se gastó de más. HU-26 muestra ambas.

C-10 · `lotes` acepta estados imposibles

Problema. `id_producto` e `id_materia_prima` son ambos nullable y no hay restricción. La tabla permite:

- Un lote con los dos nulos (no es de nada).
- Un lote con los dos llenos (¿es harina o es pan?).
- `fecha_vencimiento` anterior a `fecha_produccion`.

Corrección. Dos CHECK (MySQL los aplica desde 8.0.16; el proyecto usa 8.4):

```
CHECK ((id_producto IS NOT NULL AND id_materia_prima IS NULL)
OR (id_producto IS NULL AND id_materia_prima IS NOT NULL))
CHECK (fecha_vencimiento >= fecha_produccion)
```

C-11 · La bitácora no puede registrar acciones del sistema

Problema. `bitacora.id_usuario` es `NOT NULL` con FK a `usuarios`. Pero HU-04 CA-01 dice “se registra automáticamente al crear, editar o eliminar”. Hay acciones sin usuario autenticado:

- La creación del primer administrador (semilla).
- Intentos de login fallidos (HU-03 CA-04) — no hay usuario válido que registrar.
- Procesos programados (baja de lotes vencidos).

Corrección. `id_usuario` pasa a nullable. `NULL` significa “sistema”, y se documenta así. El frontend muestra “Sistema” cuando viene nulo.

C-12 · Estados libres: `varchar(30)` sin restricción

Problema. Cuatro tablas guardan el estado como texto libre. Las HU dan ejemplos (“Pendiente”, “Recibida”, “Cancelada”) pero nada impide `"pendiente"`, `"PENDIENTE"`, `"Pendinte"`. Cada filtro por estado se vuelve una lotería, y HU-15 CA-03, HU-24 CA-02 y HU-25 CA-03 filtran exactamente por ahí.

Corrección. Valores canónicos en `SCREAMING_SNAKE`, con `CHECK` en la base y `enum` en Java y TypeScript:

Tabla	Valores permitidos
<code>compras.estado</code>	<code>PENDIENTE</code> , <code>RECIBIDA</code> , <code>CANCELADA</code>
<code>pedidos.estado</code>	<code>PENDIENTE</code> , <code>EN_PREPARACION</code> , <code>ENTREGADO</code> , <code>CANCELADO</code>
<code>devoluciones.estado</code>	<code>PENDIENTE</code> , <code>APROBADA</code> , <code>RECHAZADA</code>
<code>planes_produccion.estado</code>	<code>PLANIFICADO</code> , <code>EN_PROCESO</code> , <code>COMPLETADO</code> , <code>CANCELADO</code>

La etiqueta legible (“En preparación”) se resuelve en el frontend. La base guarda el código.

Las **transiciones válidas** están en `04-CONTRATO-API.md §7`.

C-13 · Líneas de detalle duplicables

Además de C-05, faltan restricciones equivalentes en:

Tabla	UNIQUE necesario	Por qué
<code>detalle_compra</code>	<code>(id_compra, id_materia_prima)</code>	Dos líneas de la misma MP en una compra: el total es ambiguo y la recepción suma doble
<code>detalle_pedido</code>	<code>(id_pedido, id_producto)</code>	Idem para HU-17
<code>detalle_plan_produccion</code>	<code>(id_plan, id_producto)</code>	HU-21: dos cantidades planificadas para el mismo producto
<code>consumo_materia_prima</code>	<code>(id_detalle_plan, id_materia_prima)</code>	HU-22/HU-23: descontaría el inventario dos veces

En todos los casos, agregar la misma línea otra vez debe **sumar a la existente** o devolver `409`, decisión que se documenta por HU. Nunca crear un duplicado silencioso.

C-14 · Sin CHECK de valores positivos

Ninguna columna de cantidad, precio o stock impide negativos. Un `precio = -5000` o una `cantidad = -10` pasan sin protesta, y esta última corrompe el inventario al descontarse.

Corrección. `CHECK` en las columnas de dominio: cantidades `> 0` en líneas de detalle, saldos `>= 0` en inventarios, precios y costos `>= 0`, `stock_minimo >= 0`.

El `>= 0` en `inventario.cantidad_disponible` es la red de seguridad contra el bug clásico de vender más de lo que hay: aunque la validación del servicio falle, la base rechaza.

C-15 · Faltan índices para los filtros que piden las HU

Las HU piden explícitamente filtrar por campos sin índice. Con pocos registros no se nota; con datos reales, cada listado es un *full table scan*.

Índice	Lo pide
<code>clientes(nombre)</code>	HU-05 CA-04
<code>proveedores(nombre)</code>	HU-06 CA-04
<code>materias_primas(nombre)</code>	HU-08 CA-05
<code>productos(nombre)</code>	HU-11 CA-06
<code>usuarios(nombre)</code>	HU-02 CA-02
<code>compras(fecha)</code>	HU-15 CA-02
<code>pedidos(fecha_pedido)</code> , <code>pedidos(estado)</code>	HU-25 CA-02, CA-03
<code>planes_produccion(fecha_produccion)</code>	HU-24 CA-04, HU-26 CA-04
<code>lotes(fecha_vencimiento)</code>	HU-24 CA-05
<code>bitacora(tabla_afectada, fecha)</code>	HU-04

Nota: un índice B-tree **no** acelera `LIKE '%texto%'` (comodín inicial). Sí acelera `LIKE 'texto%'`. Los filtros por nombre se implementan como **prefijo** por defecto (`texto%`), que además es lo que el usuario espera al escribir en un buscador. Se documenta en `04-CONTRATO-API.md §4`.

C-16 · Sin datos semilla

El esquema no trae ni un registro. Sin esto el sistema es inarrancable:

- `roles` : HU-01 CA-04 exige que el rol exista. Sin filas, no se puede crear ningún usuario.
- `usuarios` : sin un administrador inicial nadie puede entrar a crear el primero. Problema del huevo y la gallina.
- `unidades_medida` : HU-08 CA-02 y HU-11 CA-04 exigen que la unidad exista.

Corrección. `V3_datos_semilla.sql` con los 5 roles, un catálogo básico de unidades y un administrador inicial.

⚠ **El administrador semilla tiene contraseña conocida y debe cambiarse antes de cualquier despliegue fuera de desarrollo local.** Está documentado en `09-SETUP-ENTORNO.md` y marcado como tarea de cierre en HU-01.

Notas de MySQL 8.4 y AWS RDS

El motor es **MySQL 8.4 LTS en Amazon RDS** (ver `00-STACK-VERSIONES.md`). Puntos que afectan directamente a estas correcciones:

Sintaxis que difiere de MariaDB

La tabla de stack de las historias mencionaba MariaDB, y es probable que alguien copie SQL de un tutorial de MariaDB. Esto es lo que va a fallar:

Construcción	MySQL 8.4	Consecuencia si se escribe como MariaDB
Expresión en <code>DEFAULT</code>	<code>DEFAULT (curdate())</code>	Error de sintaxis. Falla al ejecutar, fácil de ver
Ancho de entero	<code>int unsigned</code>	<code>int(10)</code> funciona pero avisa obsolescencia
<code>ORDER BY</code> en una vista	Se ignora	⚠ La vista se crea sin protestar y devuelve orden arbitrario

La tercera es la peligrosa. Por eso `v_lotes_disponibles_fefo` **no** confía en su propio `ORDER BY`: el orden FEFO se aplica en la consulta que la consume. Un FEFO que silenciosamente deja de ordenar por vencimiento hace que el producto viejo caduque en el estante mientras se vende el nuevo — y nada en el sistema lo señala.

Collation

Se adopta `utf8mb4_0900_ai_ci`, la nativa de MySQL 8. Es **accent-insensitive**, lo que significa que buscar `"panaderia"` encuentra `"Panadería"`. Para los filtros por nombre de HU-05, HU-06, HU-08 y HU-11 es exactamente el comportamiento que el usuario espera.

Efecto secundario a tener presente: también aplica a los `UNIQUE`. Dos unidades de medida llamadas `"Kilogramo"` y `"kilogrómo"` se consideran **la misma** y la segunda es rechazada. En este dominio es correcto.

Restricciones propias de RDS

Limitación	Impacto en este esquema
Sin privilegio <code>SUPER</code>	No se usan triggers ni funciones almacenadas. Las reglas viven en el servicio. Ninguna corrección de este documento las necesita
Variables solo vía <i>Parameter Group</i>	<code>time_zone</code> se configura ahí — ver abajo
<code>DEFINER</code> en vistas	Las tres vistas se crean con <code>SQL SECURITY INVOKER</code> para que sobrevivan a la restauración de una snapshot con otro usuario
<code>sql_mode</code> estricto por defecto	Un <code>INSERT</code> con datos que no caben falla en vez de truncar. Es lo correcto, pero rompe código escrito asumiendo MySQL permisivo

Zona horaria — el error que va a aparecer

Una instancia RDS nueva está en UTC. Colombia es UTC-5.

Consecuencia concreta: un pedido registrado a las 7:30 p.m. del 18 de septiembre se guarda con fecha **19 de septiembre**. HU-24 CA-01 (“total de pedidos del día actual”) y HU-25 dejan de cuadrar con lo que el usuario ve en pantalla, y el error solo se manifiesta después de las 7 p.m. — lo que hace que en las pruebas de la mañana todo parezca correcto.

Dos medidas, ambas necesarias:

- En el *Parameter Group* de RDS: `time_zone = America/Bogota`.
- En la aplicación: `spring.jackson.time-zone=America/Bogota` y arrancar la JVM con `-Duser.timezone=America/Bogota`.

La columna `fecha_pedido` es `DATE` con `DEFAULT (curdate())`, y `curdate()` la evalúa **el servidor de base de datos**. Sin el punto 1, el punto 2 no salva la situación.

03 — Matriz de Roles y Permisos

Las historias de usuario hablan de cinco perfiles distintos: *administrador*, *usuario*, *comprador*, *vendedor* y *productor*. El PDF de Entrega 1 define solo tres actores. Se adopta el modelo de **cinco roles**, que es el que las historias realmente exigen. La diferencia está registrada en `08-DESVIACIONES-PDF.md`.

1. Los cinco roles

id	Código	Nombre visible	Quién es	HU que lo mencionan
1	<code>ADMIN</code>	Administrador	Configura el sistema y sus catálogos	HU-01, 02, 05, 06, 07, 08, 09, 10, 11, 12, 19, 23, 24, 25, 26, 27
2	<code>PRODUCCION</code>	Producción	Planifica y ejecuta la fabricación	HU-20, 21, 22
3	<code>COMPRAS</code>	Compras	Adquiere materia prima	HU-13, 14, 15
4	<code>VENTAS</code>	Ventas	Atiende clientes y pedidos	HU-16, 17, 18
5	<code>CONSULTA</code>	Consulta	Solo lectura	PDF §6

Un usuario tiene **exactamente un rol** (`usuarios.id_rol` es una FK simple, no hay tabla puente). Si más adelante se requieren roles múltiples, es un cambio de esquema, no una configuración.

2. Principios

- `ADMIN` puede todo.** Es el superusuario del sistema; no se le restringe nada.
- `CONSULTA` solo lee.** Ningún `POST`, `PUT`, `PATCH` ni `DELETE`, en ningún módulo.
- Los roles operativos leen lo que necesitan para trabajar.** `VENTAS` necesita ver el catálogo de productos para armar un pedido, aunque no pueda editarlo. `PRODUCCION` necesita ver materias primas y recetas. Restringir la lectura transversal rompería los flujos sin ganar seguridad real.
- Escribir es lo que se restringe.** Cada rol escribe únicamente en su dominio.
- Nadie ve la bitácora salvo `ADMIN`.** Es el registro de auditoría; que el auditado pueda consultarlo íntegro no tiene sentido.

3. Matriz por módulo

`L` = leer · `E` = escribir (crear/editar) · `-` = sin acceso

Módulo	ADMIN	PRODUCCION	COMPRAS	VENTAS	CONSULTA
Usuarios y roles	L E	—	—	—	—
Bitácora	L	—	—	—	—
Clientes	L E	L	—	L E	L
Proveedores	L E	—	L E	—	L
Unidades de medida	L E	L	L	L	L
Materias primas	L E	L	L	—	L
Lotes de materia prima	L E	L E	L E	—	L
Inventario de MP (consulta)	L	L	L	—	L
Productos	L E	L	—	L	L
Recetas	L E	L E	—	—	L
Compras y su detalle	L E	L	L E	—	L
Recepción de compra	L E	—	L E	—	—
Pedidos y su detalle	L E	L	—	L E	L
Devoluciones	L E	—	—	L E	L
Aprobar devolución	L E	—	—	L E	—
Lotes de producto terminado	L E	L E	—	L	L
Planes de producción	L E	L E	—	—	L
Consumo de materia prima	L E	L E	—	—	L
Dashboard	L	L	L	L	L
Reportes	L	L	L	L	L

En **negrita**, el dominio propio de cada rol.

Decisiones que merecen justificación

- **PRODUCCION escribe lotes de materia prima.** Producción es quien físicamente recibe y abre los sacos de harina; obligar a que Compras registre cada lote genera un cuello de botella operativo.
- **PRODUCCION escribe lotes de producto terminado.** El lote de pan nace en el horno, no en la oficina. HU-19 lo atribuye a “administrador”, pero operativamente lo registra quien produce; **ADMIN** conserva el permiso.
- **VENTAS aprueba devoluciones.** HU-18 CA-07 hace que aprobar devuelva stock al inventario — una operación con efecto material. Se consideró reservarla a **ADMIN**, pero la devolución la recibe quien atiende al cliente. Queda en **VENTAS con registro obligatorio en bitácora y kardex**, que es el control que importa.
- **Solo COMPRAS y ADMIN reciben compras.** Es el punto donde entra stock al sistema (ver [02-CORRECCIONES-DB.md](#) §C-03). Es la operación de mayor impacto sobre el inventario y se mantiene acotada.

4. Implementación en Spring Security

Nombres de authority

Spring espera el prefijo **ROLE_**. El código del rol en base de datos **no lo lleva**; se agrega al construir el **UserDetails**:

```
// UsuarioDetailsService
new SimpleGrantedAuthority("ROLE_" + usuario.getRol().getNombre()) // ROLE_ADMIN
```

En las anotaciones se usa **hasRole**, que agrega el prefijo automáticamente:

```
@PreAuthorize("hasRole('ADMIN')")           // ✓
@PreAuthorize("hasAnyRole('ADMIN','COMPRAS')") // ✓
@PreAuthorize("hasAuthority('ADMIN')")       // ✗ no coincide con ROLE_ADMIN
```

Este desajuste entre **hasRole** y **hasAuthority** es la causa más común de un **403** que “no debería pasar”. Se usa **hasRole** en todo el proyecto.

Dónde se declara

En el **controlador**, sobre cada método. No en el servicio (se pierde el contexto HTTP y complica las pruebas) ni solo en **SecurityConfig** (queda lejos del endpoint y se desincroniza al refactorizar).

```

@RestController
@RequestMapping("/api/v1/materias-primas")
public class MateriaPrimaController {

    @GetMapping
    @PreAuthorize("hasAnyRole('ADMIN','PRODUCCION','COMPRAS','CONSULTA')")
    public PageResponse<MateriaPrimaResponse> listar(...) { ... }

    @PostMapping
    @PreAuthorize("hasRole('ADMIN')")
    public ResponseEntity<MateriaPrimaResponse> crear(@Valid @RequestBody CrearMateriaPrimaRequest req) { ... }
}

```

Requiere `@EnableMethodSecurity` en `SecurityConfig`. Sin esa anotación las `@PreAuthorize` se ignoran en silencio y todo queda abierto. Es un fallo que no produce ningún error visible: la aplicación arranca, los endpoints responden, y la autorización simplemente no existe. Hay un caso de prueba dedicado a esto en `05-ESTANDARES-QA.md`.

Rutas públicas

Solo dos, declaradas en `SecurityConfig`:

```

.requestMatchers("/api/v1/auth/login").permitAll()
.requestMatchers("/swagger-ui/**", "/v3/api-docs/**").permitAll() // solo perfil dev
.anyRequest().authenticated()

```

Swagger se cierra fuera de desarrollo mediante perfil de Spring.

5. Implementación en Angular

Guard de rol

```

export const rolGuard = (rolesPermitidos: Rol[]): CanActivateFn => () => {
  const auth = inject(AuthService);
  const router = inject(Router);
  if (auth.tieneAlgunRol(rolesPermitidos)) return true;
  router.navigate(['/sin-acceso']);
  return false;
};

// app.routes.ts
{
  path: 'usuarios',
  canActivate: [authGuard, rolGuard(['ADMIN'])],
  loadChildren: () => import('./features/usuario/usuario.routes')
}

```

Ocultar lo que no se puede usar

Un botón que siempre devuelve `403` es una mala experiencia. Se oculta con una directiva estructural:

```

<button mat-raised-button *siRol="['ADMIN']" (click)="crear()">Nuevo usuario</button>

```

Esto es cosmética, no seguridad. El menú y los botones ocultos mejoran la interfaz; cualquiera puede llamar al endpoint con Postman. **La autorización real está en el backend, siempre.** El frontend nunca es la barrera.

6. Respuesta ante falta de permisos

Situación	HTTP	Código	Mensaje
Sin token o token inválido/expirado	401	NO_AUTENTICADO	"Su sesión ha expirado. Inicie sesión nuevamente."
Token válido, rol insuficiente	403	SIN_PERMISO	"No tiene permisos para realizar esta acción."
Usuario desactivado (HU-03 CA-03)	403	USUARIO_INACTIVO	"Su usuario está desactivado. Contacte al administrador."

El interceptor de Angular distingue los tres: `401` cierra sesión y redirige al login; `403` muestra un aviso sin cerrar la sesión.

04 — Contrato de API

Reglas transversales a todos los endpoints. Cada `especificacion.md` de HU define su contrato específico; **este documento define lo que no se repite en cada uno.**

1. Base y versionado

```
http://localhost:8080/api/v1
```

- Todos los recursos cuelgan de `/api/v1`.
- Producción/JSON en `application/json; charset=UTF-8`.
- Swagger UI en `/swagger-ui.html`, OpenAPI en `/v3/api-docs`.

2. Autenticación

Todos los endpoints requieren JWT salvo `POST /api/v1/auth/login`.

```
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
```

Claims del token (HU-03 CA-05):

```
{
  "sub": "12",
  "correo": "maria@productionfood.local",
  "nombre": "María Gómez",
  "rol": "VENTAS",
  "iat": 1757980800,
  "exp": 1757984400
}
```

- `sub` es el `id_usuario` como string (es el estándar de JWT).
- Vigencia: **1 hora**. No hay refresh token en este alcance; al expirar se vuelve a iniciar sesión.
- El secreto de firma viene de `application.properties` vía variable de entorno. **Nunca se versiona en el repositorio.**

3. Métodos y códigos de estado

Método	Uso	Éxito	Cuerpo de respuesta
GET (colección)	Listar	200	PageResponse<T>
GET (recurso)	Obtener uno	200	T
POST	Crear	201 + cabecera Location	El recurso creado
PUT	Reemplazar completo	200	El recurso actualizado
PATCH	Modificar parcial (típicamente estado)	200	El recurso actualizado
DELETE	Eliminar línea de detalle	204	Vacío

No se usa DELETE sobre entidades principales. Las HU son explícitas: clientes, proveedores y usuarios se **desactivan** (`estado = 0`), no se borran. **DELETE** queda reservado a líneas de detalle (ítems de receta, de compra, de pedido).

Códigos de error

Código	Cuándo	Ejemplo típico
400	Petición malformada o validación sintáctica fallida	Falta un campo obligatorio, formato inválido
401	Sin token, token inválido o expirado	Sesión vencida
403	Autenticado pero sin permiso, o usuario inactivo	VENTAS intentando crear un producto
404	El recurso no existe	GET /clientes/9999
409	Conflicto con el estado actual	Correo duplicado, stock insuficiente, transición de estado inválida
422	Regla de negocio violada con datos sintácticamente válidos	Devolver más cantidad de la pedida
500	Error no controlado	—

409 vs 422 : 409 es conflicto con algo que ya existe en el sistema (duplicado, estado incompatible). 422 es una regla de negocio que los datos no satisfacen aunque el formato sea correcto. Ante la duda, 409. Lo que importa es ser consistente: cada `especificacion.md` fija el código de cada caso.

4. Listados: paginación, filtros y ordenamiento

Parámetros

Parámetro	Tipo	Por defecto	Restricción
page	int	0	>= 0
size	int	20	1..100 — por encima de 100 se devuelve 400
sort	string	Según el recurso	campo,asc o campo,desc

`size` se acota por arriba deliberadamente: sin ese límite, un `?size=999999` convierte cualquier listado en un ataque de denegación trivial.

Respuesta de colección

```
{
  "content": [ { "...": "..." } ],
  "page": 0,
  "size": 20,
  "totalElements": 143,
  "totalPages": 8,
  "first": true,
  "last": false
}
```

Ordenamiento seguro

El valor de `sort` **nunca se concatena crudo al SQL**. Cada repositorio declara su lista blanca:

```
private static final Map<String, String> CAMPOS_ORDEN = Map.of(
    "nombre", "c.nombre",
    "id", "c.id_cliente"
);
// Un campo fuera del mapa → 400 CAMPO_ORDEN_INVALIDO
```

Filtros de texto

Los filtros por nombre son **búsqueda por prefijo**: `LIKE 'texto%'`.

Dos razones: es lo que el usuario espera al teclear en un buscador, y es lo único que puede aprovechar un índice B-tree. `LIKE '%texto%'` fuerza recorrido completo de la tabla en cada pulsación (ver `02-CORRECCIONES-DB.md` §C-15).

Los caracteres `%` y `_` del valor recibido se escapan antes de armar el patrón; si no, un usuario que escriba `%` obtiene todos los registros.

Filtros de fecha

`fechaInicio` y `fechaFin` en formato `YYYY-MM-DD`, **ambos inclusivos**. Si `fechaInicio > fechaFin` → `400 RANGO_FECHAS_INVALIDO`.

5. Formato de error

Único para toda la API:

```
{
  "timestamp": "2026-09-18T14:32:11",
  "status": 409,
  "error": "Conflict",
  "code": "CORREO_DUPLICADO",
  "message": "Ya existe un usuario registrado con el correo indicado.",
  "path": "/api/v1/usuarios",
  "fieldErrors": []
}
```

Con errores de validación:

```
{
  "timestamp": "2026-09-18T14:35:02",
  "status": 400,
  "error": "Bad Request",
  "code": "VALIDACION_FALLIDA",
  "message": "Los datos enviados no son válidos.",
  "path": "/api/v1/usuarios",
  "fieldErrors": [
    { "field": "correo", "message": "Debe ser un correo electrónico válido" },
    { "field": "nombre", "message": "No puede estar vacío" }
  ]
}
```

Reglas:

- `code` es estable y en inglés SCREAMING_SNAKE: es para el código del frontend.
- `message` es para la persona: español, claro, sin jerga técnica.
- `message` **nunca expone detalles internos**. Nada de nombres de tabla, SQL, ni *stack traces*. Un `DataIntegrityViolationException` se traduce a un mensaje de dominio; el detalle técnico va al log del servidor, no a la respuesta.

Códigos de error transversales

code	HTTP	Significado
VALIDACION_FALLIDA	400	Bean Validation rechazó el cuerpo
PARAMETRO_INVALIDO	400	Parámetro de consulta mal formado
CAMPO_ORDEN_INVALIDO	400	<code>sort</code> fuera de la lista blanca
RANGO_FECHAS_INVALIDO	400	<code>fechaInicio</code> posterior a <code>fechaFin</code>
NO_AUTENTICADO	401	Token ausente, inválido o expirado
SIN_PERMISO	403	Rol insuficiente
USUARIO_INACTIVO	403	Usuario con <code>estado = 0</code>
RECURSO_NO_ENCONTRADO	404	El id no existe
REFERENCIA_INEXISTENTE	409	Una FK del cuerpo apunta a algo que no existe
ERROR_INTERNO	500	No controlado

Los códigos específicos de cada HU están en su `especificacion.md`.

6. Convenciones de datos

Tipo	Formato JSON	Ejemplo
Fecha	YYYY-MM-DD (ISO-8601)	"2026-09-18"
Fecha y hora	YYYY-MM-DDTHH:mm:ss	"2026-09-18T14:32:11"
Decimal	Número JSON, sin separador de miles	12500.50
Booleano de estado	<code>boolean</code> , no 0 / 1	"activo": true
Identificador	Número entero	42
Enumerado	String SCREAMING_SNAKE	"EN_PREPARACION"

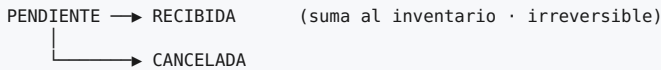
Notas:

- **Decimales.** En Java se usa `BigDecimal`, nunca `double` ni `float`. Un `double` no representa exactamente `0.1`; sumar precios con `double` produce centavos fantasma que aparecen como descuadres al cuadrar caja. En TypeScript se recibe como `number` y se formatea al mostrar; los cálculos de totales son autoridad del backend.
- **estado.** La base guarda `tinyint(1)`; la API expone `boolean`. La traducción se hace en el `RowMapper` y en el DTO. El frontend nunca ve un `1` cuando quiere decir "sí".
- **Zona horaria.** Las fechas se manejan como fecha local de Colombia (UTC-5). El servidor se configura con `-Duser.timezone=America/Bogota`. Un servidor en UTC hace que los pedidos registrados después de las 7 p.m. aparezcan con la fecha del día siguiente, y HU-24 CA-01 ("pedidos del día") deja de cuadrar con lo que ve el usuario.

7. Estados y transiciones

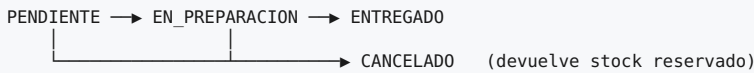
Los valores canónicos están en `02-CORRECCIONES-DB.md` §C-12. Las transiciones permitidas:

Compras (HU-13)



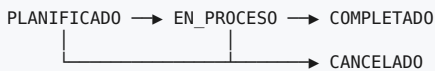
RECIBIDA es terminal: una vez que el stock entró, se corrige con un ajuste de inventario, nunca deshaciendo el estado.

Pedidos (HU-16)



ENTREGADO es terminal.

Planes de producción (HU-20)



Devoluciones (HU-18)



Toda transición no listada devuelve **409 TRANSICION_INVALIDA**, con un mensaje que indique el estado actual y los permitidos:

```

{
  "code": "TRANSICION_INVALIDA",
  "message": "No se puede pasar de ENTREGADO a PENDIENTE. Desde ENTREGADO no hay transiciones disponibles."
}
  
```

La validación vive en un único método por módulo (`validarTransicion(actual, nuevo)`), no dispersa en `if` por el servicio.

8. Idempotencia y concurrencia

- `GET`, `PUT` y `DELETE` son idempotentes por definición.
- `POST` no lo es: dos clics en “Guardar” crean dos registros. Mitigación: el frontend deshabilita el botón mientras la petición está en vuelo, **y** las restricciones `UNIQUE` de la base son la garantía real (`02-CORRECCIONES-DB.md` §C-05, §C-13).
- Descuentos de stock:** la comprobación de disponibilidad y el descuento ocurren en la **misma transacción**, con `SELECT ... FOR UPDATE` sobre la fila de inventario. Sin ese bloqueo, dos pedidos simultáneos del último pan disponible pasan ambos la validación y el stock queda negativo. El `CHECK (cantidad_disponible >= 0)` es la segunda línea de defensa.

9. CORS

Solo en desarrollo, y acotado:

```

configuration.setAllowedOrigins(List.of("http://localhost:4200"));
configuration.setAllowedMethods(List.of("GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS"));
configuration.setAllowedHeaders(List.of("Authorization", "Content-Type"));
  
```

`setAllowedOrigins(List.of("*"))` combinado con credenciales no funciona y, donde funciona, abre la API a cualquier sitio. No se usa.

10. Documentación en Swagger

Cada endpoint lleva:

```

@Operation(summary = "Registrar un cliente",
  description = "Crea un cliente activo. El nombre es obligatorio.")
@ApiResponses({
  @ApiResponse(responseCode = "201", description = "Cliente creado"),
  @ApiResponse(responseCode = "400", description = "Datos inválidos"),
  @ApiResponse(responseCode = "403", description = "Sin permisos")
})
  
```

Un endpoint sin documentar no cumple la Definición de Terminado (01-CONVENCIONES.md §7). Swagger es el contrato que consume el frontend y con el que QA arma las pruebas: si está incompleto, dos personas quedan bloqueadas.

05 — Estándares de QA

Cómo se prueba, qué se reporta y cuándo una historia se considera cerrada.

1. Alcance de las pruebas

El stack acordado incluye **Postman** como herramienta de prueba de API. Se organiza así:

Nivel	Herramienta	Quién	Cobertura mínima
Prueba de API (funcional)	Postman	QA	100% de los criterios de aceptación , casos positivos y negativos
Prueba unitaria de servicio	JUnit 5 + Mockito	Backend	Reglas de negocio con ramificación: cálculos, validaciones, transiciones
Prueba de integración	@SpringBootTest + MockMvc	Backend	Endpoints con efectos sobre varias tablas (HU-14, 17, 18, 23)
Prueba manual de interfaz	Navegador	QA	Flujos completos por HU, incluidos errores

No se exige cobertura total de pruebas unitarias. Se exige que toda regla con ramificación esté cubierta: los cálculos de HU-14, HU-22 y HU-26, las transiciones de estado, y las validaciones de stock. Un *getter* no necesita prueba; una fórmula de eficiencia que divide por cero, sí.

2. Colección de Postman

Estructura

ProductionFood API/ 00 - Setup/ - Login ADMIN - Login PRODUCCION - Login COMPRAS - Login VENTAS - Login CONSULTA - HU-01 - Registro de usuarios/ - CP-01 Crear usuario válido - CP-02 Crear con correo duplicado → 409 ... - HU-NN - .../

Variables de entorno

Variable	Ejemplo	Origen
baseUrl	http://localhost:8080/api/v1	Fija
tokenAdmin	eyJ...	Guardada por el script de "Login ADMIN"
tokenVentas	eyJ...	Guardada por el script de "Login VENTAS"
idClienteCreado	7	Encadenamiento entre peticiones

Script de la petición de login:

<pre>pm.test("Login exitoso", () => pm.response.to.have.status(200)); pm.environment.set("tokenAdmin", pm.response.json().token);</pre>
--

Ningún token se escribe a mano en la colección. Expiran en una hora; una colección con tokens pegados deja de funcionar al día siguiente y hace perder media mañana averiguando por qué.

Aserciones mínimas por petición

<pre>pm.test("Código de estado correcto", () => pm.response.to.have.status(201)); pm.test("Responde en menos de 2 segundos", () => pm.expect(pm.response.responseTime).to.be.below(2000)); pm.test("Devuelve el id generado", () => pm.expect(pm.response.json().idCliente).to.be.a('number'));</pre>
--

El umbral de 2 segundos es el RNF-02 del documento de requerimientos. Se verifica en cada petición, no solo cuando alguien se

acuerda.

3. Casos de prueba: formato

Cada `tarea-qa.md` de HU usa esta tabla:

ID	Criterio	Descripción	Datos de entrada	Resultado esperado	Tipo
CP-01	CA-01	Crear cliente con nombre válido	<code>{"nombre": "Panadería El Trigo"}</code>	201, cuerpo con <code>idCliente</code> y <code>activo: true</code>	Positivo
CP-02	CA-01	Crear cliente sin nombre	<code>{"nombre": ""}</code>	400, <code>code: VALIDACION_FALLIDA</code> , <code>fieldErrors[0].field = "nombre"</code>	Negativo

Todo criterio de aceptación tiene al menos un caso positivo y uno negativo. Un CA que solo se prueba por el camino feliz no está probado: los defectos viven en los bordes.

Tipos de caso

- **Positivo** — el flujo esperado funciona.
- **Negativo** — la entrada inválida se rechaza con el código correcto y mensaje útil.
- **Borde** — límites exactos: longitud máxima, cantidad = 0, stock exactamente igual al solicitado, fecha de vencimiento = hoy.
- **Seguridad** — sin token, con token de rol no autorizado, con token expirado.
- **Integridad** — el estado de la base después de la operación es el correcto (no basta con que la respuesta sea `200`).

4. Batería obligatoria de seguridad

Se aplica **a cada endpoint**, sin excepción:

ID	Caso	Esperado
SEC-01	Petición sin cabecera <code>Authorization</code>	401 NO_AUTENTICADO
SEC-02	Token malformado (<code>Bearer xxx</code>)	401 NO_AUTENTICADO
SEC-03	Token expirado	401 NO_AUTENTICADO
SEC-04	Token válido de un rol sin permiso	403 SIN_PERMISO
SEC-05	Token de un usuario con <code>estado = 0</code>	403 USUARIO_INACTIVO

SEC-04 no es opcional. Es la prueba que detecta la falta de `@EnableMethodSecurity` (ver `03-MATRIZ-ROLES.md` §4): sin esa anotación, las `@PreAuthorize` se ignoran en silencio, todo responde `200` y nada parece roto. Es el tipo de defecto que sobrevive hasta producción porque nadie lo busca. Si SEC-04 devuelve `200`, la autorización del sistema completo está desactivada.

Inyección SQL

En todo campo de filtro de texto se prueba con:

```
' OR '1'='1
'; DROP TABLE clientes; --
%
```

Esperado: se tratan como texto literal. Cero resultados o resultados coherentes, nunca un error 500 ni una lista completa. El `%` merece atención aparte: sin escapado devuelve **todos** los registros, y eso no lanza ningún error — simplemente filtra de más (ver `04-CONTRATO-API.md` §4).

5. Pruebas de integridad de datos

Para las HU que mueven inventario (HU-13, 14, 17, 18, 19, 22, 23) **no basta la respuesta HTTP**. Se verifica el estado de la base:

```
-- Antes de la operación
SELECT cantidad_disponible FROM inventario WHERE id_materia_prima = 3; -- 50.00

-- (ejecutar el endpoint que consume 12 kg)

-- Después
SELECT cantidad_disponible FROM inventario WHERE id_materia_prima = 3; -- 38.00 esperado

-- Y el kardex debe tener el movimiento correspondiente
SELECT tipo_movimiento, cantidad, saldo_anterior, saldo_posterior
FROM movimientos_inventario_mp
WHERE id_materia_prima = 3 ORDER BY id_movimiento DESC LIMIT 1;
-- SALIDA_PRODUCCION | 12.000 | 50.00 | 38.00
```

La regla de oro del kardex: **saldo_posterior del último movimiento debe ser igual a cantidad_disponible en inventario**. Si difieren, hay una ruta de código que actualiza el saldo sin registrar el movimiento — exactamente lo que el kardex existe para impedir.

Consulta de conciliación, ejecutada al cerrar cada fase:

```
SELECT i.id_materia_prima, i.cantidad_disponible, m.saldo_posterior
FROM inventario i
LEFT JOIN (
    SELECT id_materia_prima, saldo_posterior,
    ROW_NUMBER() OVER (PARTITION BY id_materia_prima ORDER BY id_movimiento DESC) rn
    FROM movimientos_inventario_mp
) m ON m.id_materia_prima = i.id_materia_prima AND m.rn = 1
WHERE i.cantidad_disponible <> COALESCE(m.saldo_posterior, 0);
-- Debe devolver 0 filas.
```

6. Prueba de transacciones

Las HU-14, HU-17, HU-18 y HU-23 declaran operaciones transaccionales. Se prueba que el *rollback* funciona, provocando un fallo en mitad de la operación:

1. Registrar consumo de materia prima para un plan con **dos** ingredientes.
2. El segundo ingrediente tiene stock insuficiente a propósito.
3. Esperado: **409 STOCK_INSUFICIENTE y el primer ingrediente no descontado**.

Un sistema que descuenta el primero y falla en el segundo deja el inventario corrupto de forma silenciosa. Este caso es obligatorio en toda HU marcada como transaccional.

7. Pruebas de interfaz

Cada pantalla se verifica en sus **tres estados**, no solo con datos:

Estado	Qué se espera
Cargando	Indicador visible; los controles deshabilitados
Vacío	Mensaje explicativo con acción sugerida, no una tabla en blanco
Error	Mensaje comprensible en español y opción de reintentar
Con datos	Paginación, ordenamiento y filtros funcionando

Además, por formulario:

- ☐ Los campos obligatorios se marcan antes de enviar.
- ☐ Los mensajes de error del servidor se muestran junto al campo correspondiente (**fieldErrors** del contrato).
- ☐ El botón de envío se deshabilita mientras la petición está en vuelo (evita el doble registro por doble clic).
- ☐ Al cancelar con cambios sin guardar se pide confirmación.
- ☐ La navegación por teclado (Tab) recorre los campos en orden lógico.

Se verifica en **Chrome y Firefox**, a 1366×768 como mínimo.

8. Reporte de defectos

```
### DEF-NNN · [HU-NN] Título breve y específico

**Severidad:** Crítica | Alta | Media | Baja
**Entorno:** Local · Backend commit `a1b2c3d` · Frontend commit `e4f5g6h`

**Pasos para reproducir**
1. Autenticarse como VENTAS
2. POST /api/v1/pedidos/5/detalles con {"idProducto": 3, "cantidad": 1000}
3. Observar la respuesta

**Resultado esperado:** 409 STOCK_INSUFICIENTE
**Resultado obtenido:** 201, el pedido se crea y el inventario queda en -970.00

**Evidencia:** captura + respuesta JSON + consulta SQL del estado posterior
```

Escala de severidad

Nivel	Definición	Ejemplos
Crítica	Pérdida o corrupción de datos, o brecha de seguridad	Inventario negativo, endpoint sin autorización, contraseñas en texto plano
Alta	Un criterio de aceptación no se cumple y no hay forma de sortearlo	No se puede crear un pedido
Media	CA incumplido con alternativa viable, o cálculo incorrecto en un reporte	El filtro por fecha excluye el último día
Baja	Cosmético, texto, alineación	Etiqueta mal redactada

Un defecto crítico bloquea el cierre de la fase completa, no solo de su HU.

9. Criterios de cierre

Por historia de usuario

- ☐ 100% de los casos de `tarea-qa.md` ejecutados.
- ☐ 100% de los positivos pasan.
- ☐ 100% de los negativos devuelven el código y `code` correctos.
- ☐ La batería SEC-01..SEC-05 pasa en todos los endpoints de la HU.
- ☐ Cero defectos críticos o altos abiertos.
- ☐ La colección de Postman está actualizada y versionada en el repositorio.

Por fase

- ☐ Todas las HU de la fase cerradas.
- ☐ Prueba de regresión de las fases anteriores ejecutada.
- ☐ Conciliación de inventario (§5) devuelve 0 filas.
- ☐ Ninguna operación principal supera los 2 segundos (RNF-02).

10. Datos de prueba

Se mantiene un script `datos_prueba.sql` fuera de las migraciones (las migraciones son esquema y semilla mínima; los datos de prueba son otra cosa y no deben viajar a un entorno real). Contiene:

- 3 proveedores, 5 clientes
- 8 materias primas con inventario variado: una **agotada**, una **bajo mínimo**, una normal
- 6 productos, 4 con receta completa
- 2 compras (una **PENDIENTE**, una **RECIBIDA**)
- 3 lotes de producto terminado: uno **vencido**, uno que vence **en 3 días**, uno vigente
- 2 planes de producción en estados distintos

Los casos incómodos —el stock en cero, el lote vencido, el que vence en 3 días— están puestos a propósito. Son los que ejercitan HU-24 CA-05 y HU-27, y los que nadie recuerda crear a mano cuando prueba.

Antes de cada ronda de pruebas se restaura la base desde cero: V1 → V2 → V3 → `datos_prueba.sql`. Probar sobre una base con residuos de la sesión anterior produce resultados que no se pueden reproducir ni explicar.

06 — Arquitectura Backend

Spring Boot 4.1 · Java 25 · JdbcTemplate · MySQL 8.4

Este documento contiene la configuración **escrita para Spring Security 7**. La mayoría de tutoriales de JWT con Spring que se encuentran en línea son para Spring Boot 3.x y no compilan aquí. **Si un ejemplo de internet contradice este documento, el ejemplo está desactualizado.**

1. Capas

```
HTTP
|
↓
Controller → valida el DTO (@Valid), autoriza (@PreAuthorize), mapea HTTP
|
↓
Service → reglas de negocio, @Transactional, auditoría, orquestación
|
↓
Repository → SQL con JdbcTemplate, RowMapper, paginación
|
↓
MySQL
```

Sin ORM: todo el SQL es explícito. Las responsabilidades de cada capa están en [01-CONVENCIONES.md](#) §2.

2. Seguridad

`SecurityConfig` (Spring Security 7)

```

@Configuration
@EnableWebSecurity
@EnableMethodSecurity // ⚠ SIN ESTO, TODAS LAS @PreAuthorize SE IGNORAN
public class SecurityConfig {

    private final JwtAuthenticationFilter jwtFilter;
    private final AuthenticationEntryPoint entryPoint;
    private final AccessDeniedHandler accessDeniedHandler;

    public SecurityConfig(JwtAuthenticationFilter jwtFilter,
                        AuthenticationEntryPoint entryPoint,
                        AccessDeniedHandler accessDeniedHandler) {
        this.jwtFilter = jwtFilter;
        this.entryPoint = entryPoint;
        this.accessDeniedHandler = accessDeniedHandler;
    }

    @Bean
    SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        return http
            // API stateless con JWT: CSRF no aplica (no hay cookies de sesión)
            .csrf(AbstractHttpConfigurer::disable)
            .cors(cors -> cors.configurationSource(corsConfigurationSource()))
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/v1/auth/login").permitAll()
                .requestMatchers("/swagger-ui/**", "/swagger-ui.html", "/v3/api-docs/**").permitAll()
                .anyRequest().authenticated())
            .exceptionHandling(e -> e
                .authenticationEntryPoint(entryPoint) // 401 en formato del contrato
                .accessDeniedHandler(accessDeniedHandler)) // 403 en formato del contrato
            .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)
            .build();
    }

    @Bean
    PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder(); // HU-01 CA-03
    }

    @Bean
    AuthenticationManager authenticationManager(AuthenticationConfiguration cfg) throws Exception {
        return cfg.getAuthenticationManager();
    }

    @Bean
    CorsConfigurationSource corsConfigurationSource() {
        var c = new CorsConfiguration();
        c.setAllowedOrigins(List.of("http://localhost:4200"));
        c.setAllowedMethods(List.of("GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS"));
        c.setAllowedHeaders(List.of("Authorization", "Content-Type"));
        var source = new UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/api/**", c);
        return source;
    }
}

```

@EnableMethodSecurity es la línea más importante del archivo. Sin ella la aplicación arranca igual, los endpoints responden igual, y las **@PreAuthorize** no se evalúan: la autorización del sistema completo queda desactivada sin un solo mensaje de error. El caso SEC-04 de [05-ESTANDARES-QA.md](#) existe para detectar exactamente esto.

Swagger se cierra fuera de desarrollo con **@Profile("dev")** sobre una configuración alternativa, o retirando esas rutas de **permitAll** en el perfil de producción.

JwtService

```

@Service
public class JwtService {

    private final SecretKey key;
    private final long expiracionMs;

    public JwtService(@Value("${app.jwt.secret}") String secret,
                     @Value("${app.jwt.expiration-ms}") long expiracionMs) {
        // La clave debe tener al menos 256 bits para HS256; jjwt lo verifica y falla si no
        this.key = Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
        this.expiracionMs = expiracionMs;
    }

    public String generar(Usuario u) { // HU-03 CA-05
        var ahora = new Date();
        return Jwts.builder()
            .subject(String.valueOf(u.getIdUsuario()))
            .claim("correo", u.getCorreo())
            .claim("nombre", u.getNombre())
            .claim("rol", u.getNombreRol())
            .issuedAt(ahora)
            .expiration(new Date(ahora.getTime() + expiracionMs))
            .signWith(key)
            .compact();
    }

    public Claims validarYExtraer(String token) {
        return Jwts.parser()
            .verifyWith(key)
            .build()
            .parseSignedClaims(token)
            .getPayload();
    }
}

```

Nota sobre jjwt 0.13: la API fluida cambió respecto a 0.11 (`setSubject` → `subject` , `parserBuilder()` → `parser()` , `signWith(key, alg)` → `signWith(key)`). Los ejemplos antiguos no compilan.

Configuración

```

app.jwt.secret=${JWT_SECRET}
app.jwt.expiration-ms=3600000

```

⚠ El secreto **nunca** se versiona. Un `JWT_SECRET` en el repositorio permite a cualquiera firmar tokens válidos con el rol que quiera: equivale a publicar la contraseña de administrador. Se genera con `openssl rand -base64 48` .

JwtAuthenticationFilter

```

@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res,
                                   FilterChain chain) throws ServletException, IOException {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            try {
                Claims claims = jwtService.validarYExtraer(header.substring(7));
                var authorities = List.of(new SimpleGrantedAuthority("ROLE_" + claims.get("rol")));
                var auth = new UsernamePasswordAuthenticationToken(
                    new UsuarioAutenticado(
                        Long.valueOf(claims.getSubject()),
                        claims.get("correo", String.class),
                        claims.get("rol", String.class)),
                    null, authorities);
                SecurityContextHolder.getContext().setAuthentication(auth);
            } catch (JwtException ex) {
                // Token inválido o expirado: se deja sin autenticar.
                // El EntryPoint devuelve 401 NO AUTENTICADO con el formato del contrato.
                SecurityContextHolder.clearContext();
            }
        }
        chain.doFilter(req, res);
    }
}

```

El prefijo `ROLE_` se agrega aquí porque `hasRole('ADMIN')` busca la authority `ROLE_ADMIN` . Ver `03-MATRIZ-ROLES.md` §4.

3. Manejo de errores

Un único `@RestControllerAdvice` produce el formato de `04-CONTRATO-API.md` §5:

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    ResponseEntity<ErrorResponse> validacion(MethodArgumentNotValidException ex, HttpServletRequest req) {
        var campos = ex.getBindingResult().getFieldErrors().stream()
            .map(f -> new FieldError(f.getField(), f.getDefaultMessage()))
            .toList();
        return build(HttpStatus.BAD_REQUEST, "VALIDACION_FALLIDA",
            "Los datos enviados no son válidos.", req, campos);
    }

    @ExceptionHandler(RecursoNoEncontradoException.class)
    ResponseEntity<ErrorResponse> noEncontrado(...) { /* 404 */ }

    @ExceptionHandler(ConflictoNegocioException.class)
    ResponseEntity<ErrorResponse> conflicto(...) { /* 409 con el code de la excepción */ }

    @ExceptionHandler(DataIntegrityViolationException.class)
    ResponseEntity<ErrorResponse> integridad(DataIntegrityViolationException ex, HttpServletRequest req) {
        // Traduce la violación de constraint a un mensaje de dominio.
        // NUNCA se devuelve ex.getMessage(): expone nombres de tabla, columnas y SQL.
        log.warn("Violación de integridad en {}", req.getRequestURI(), ex);
        return build(HttpStatus.CONFLICT, "CONFLICTO_INTEGRIDAD",
            "La operación entra en conflicto con datos existentes.", req, List.of());
    }

    @ExceptionHandler(Exception.class)
    ResponseEntity<ErrorResponse> general(Exception ex, HttpServletRequest req) {
        log.error("Error no controlado en {}", req.getRequestURI(), ex);
        return build(HttpStatus.INTERNAL_SERVER_ERROR, "ERROR_INTERNO",
            "Ocurrió un error inesperado. Intente nuevamente.", req, List.of());
    }
}
```

El detalle técnico va al log, el mensaje de dominio a la respuesta. Devolver el `getMessage()` de una excepción de base de datos filtra la estructura interna del sistema a cualquiera que provoque un error.

4. Repositorios con JdbcTemplate

Patrón base

```

@Repository
public class ClienteRepository {

    private static final String COLUMNAS =
        "id_cliente, nombre, contacto, telefono, estado";

    // Lista blanca de ordenamiento: el valor del cliente nunca llega crudo al SQL
    private static final Map<String,String> ORDEN = Map.of(
        "nombre", "nombre",
        "id", "id_cliente");

    private final NamedParameterJdbcTemplate jdbc;
    private final ClienteRowMapper mapper;

    public Optional<Cliente> buscarPorId(Long id) {
        var sql = "SELECT " + COLUMNAS + " FROM clientes WHERE id_cliente = :id";
        return jdbc.query(sql, Map.of("id", id), mapper).stream().findFirst();
    }

    public List<Cliente> buscar(String nombre, Boolean activo, PageRequest page) {
        var sql = new StringBuilder("SELECT " + COLUMNAS + " FROM clientes WHERE 1=1");
        var params = new MapSqlParameterSource();

        if (nombre != null && !nombre.isBlank()) {
            sql.append(" AND nombre LIKE :nombre");
            params.addValue("nombre", escaparLike(nombre) + "%"); // prefijo, usa el índice
        }
        if (activo != null) {
            sql.append(" AND estado = :estado");
            params.addValue("estado", activo ? 1 : 0);
        }

        sql.append(" ORDER BY ").append(ORDEN.get(page.campo())) // ya validado
            .append(page.ascendente() ? " ASC" : " DESC")
            .append(" LIMIT :limit OFFSET :offset");
        params.addValue("limit", page.size());
        params.addValue("offset", page.offset());

        return jdbc.query(sql.toString(), params, mapper);
    }

    /** Escapa los comodines de LIKE. Sin esto, buscar "%" devuelve todos los registros. */
    private String escaparLike(String v) {
        return v.replace("\\", "\\\").replace("%", "\\%").replace("_", "\\_");
    }
}

```

Insertar y recuperar el id generado

```

public Long insertar(Cliente c) {
    var sql = "INSERT INTO clientes (nombre, contacto, telefono, estado) " +
        "VALUES (:nombre, :contacto, :telefono, :estado)";
    var params = new MapSqlParameterSource()
        .addValue("nombre", c.getNombre())
        .addValue("contacto", c.getContacto())
        .addValue("telefono", c.getTelefono())
        .addValue("estado", c.isActivo() ? 1 : 0);

    var keyHolder = new GeneratedKeyHolder();
    jdbc.update(sql, params, keyHolder, new String[]{"id_cliente"});
    return keyHolder.getKey().longValue();
}

```

RowMapper

```

@Component
public class ClienteRowMapper implements RowMapper<Cliente> {
    @Override
    public Cliente mapRow(ResultSet rs, int rowNum) throws SQLException {
        var c = new Cliente();
        c.setIdCliente(rs.getLong("id_cliente"));
        c.setNombre(rs.getString("nombre"));
        c.setContacto(rs.getString("contacto"));
        c.setTelefono(rs.getString("telefono"));
        c.setActivo(rs.getBoolean("estado")); // tinyint(1) → boolean, según el contrato
        return c;
    }
}

```

Conteo para la paginación

```
public long contar(String nombre, Boolean activo) {
    // Mismo WHERE que buscar(). Si divergen, totalElements miente.
    ...
}
```

Mantener sincronizados el `WHERE` de `buscar()` y el de `contar()` es la fuente de error más frecuente en paginación manual: la lista filtra y el contador no, y la última página aparece vacía.

5. Transacciones y bloqueo de inventario

Toda operación que toca más de una tabla es transaccional:

```
@Service
public class ConsumoService {

    @Transactional // HU-23 CA-04
    public ConsumoResponse registrar(Long idDetallePlan, List<ConsumoRequest> consumos) {
        for (var c : consumos) {
            // SELECT ... FOR UPDATE: bloquea la fila hasta el fin de la transacción.
            // Sin esto, dos producciones simultáneas leen el mismo saldo, ambas validan
            // y el inventario queda negativo.
            var saldo = inventarioRepository.bloquearYObtener(c.idMateriaPrima());

            if (saldo.compareTo(c.cantidadReal()) < 0) {
                throw new ConflictoNegocioException("STOCK_INSUFICIENTE", "...");
            }
            inventarioService.registrarSalida(
                c.idMateriaPrima(), c.cantidadReal(),
                TipoMovimiento.SALIDA_PRODUCCION, "consumo_materia_prima", idConsumo);
        }
        return ...;
    }
}
```

```
// InventarioRepository
public BigDecimal bloquearYObtener(Long idMateriaPrima) {
    var sql = "SELECT cantidad_disponible FROM inventario " +
        "WHERE id_materia_prima = :id FOR UPDATE";
    ...
}
```

Regla del inventario: `InventarioService` es el **único** componente que ejecuta `UPDATE inventario`, y siempre inserta el movimiento de kardex en la misma transacción (`02-CORRECCIONES-DB.md` §C-04). Ningún otro servicio toca esas tablas directamente.

6. Auditoría (HU-04)

Aspecto AOP sobre los métodos anotados:

```
@Auditable(accion = "CREAR", tabla = "clientes")
public ClienteResponse crear(CrearClienteRequest req) { ... }
```

```
@Aspect
@Component
public class AuditoriaAspect {

    @AfterReturning(pointcut = "@annotation(auditable)", returning = "resultado")
    public void registrar(JoinPoint jp, Auditable auditable, Object resultado) {
        bitacoraService.registrar(
            SecurityUtils.idUsuarioActual(), // null si es acción del sistema (§C-11)
            auditable.accion(),
            auditable.tabla(),
            resumen(resultado)); // recortado a 255 caracteres
    }
}
```

`@AfterReturning`, no `@Before`: solo se audita lo que efectivamente ocurrió. Auditar antes registra intentos fallidos como si fueran cambios reales.

El fallo al escribir en bitácora no debe tumbar la operación de negocio. `BitacoraService.registrar` captura sus propias excepciones y las registra en el log. Que la auditoría falle es un problema; que impida vender, es peor.

7. Estructura de un endpoint completo

```

@RestController
@RequestMapping("/api/v1/clientes")
@Tag(name = "Clientes", description = "Gestión de clientes (HU-05)")
public class ClienteController {

    @GetMapping
    @PreAuthorize("hasAnyRole('ADMIN','VENTAS','PRODUCCION','CONSULTA')")
    @Operation(summary = "Listar clientes con filtro y paginación")
    public PageResponse<ClienteResponse> listar(
        @RequestParam(required = false) String nombre,
        @RequestParam(required = false) Boolean activo,
        @RequestParam(defaultValue = "0") @Min(0) int page,
        @RequestParam(defaultValue = "20") @Min(1) @Max(100) int size,
        @RequestParam(defaultValue = "nombre,asc") String sort) {
        return clienteService.listar(nombre, activo, PageRequest.of(page, size, sort));
    }

    @PostMapping
    @PreAuthorize("hasAnyRole('ADMIN','VENTAS')")
    @Operation(summary = "Registrar un cliente")
    @ApiResponse({
        @ApiResponse(responseCode = "201", description = "Cliente creado"),
        @ApiResponse(responseCode = "400", description = "Datos inválidos"),
        @ApiResponse(responseCode = "403", description = "Sin permisos")
    })
    public ResponseEntity<ClienteResponse> crear(@Valid @RequestBody CrearClienteRequest req) {
        var creado = clienteService.crear(req);
        return ResponseEntity
            .created(URI.create("/api/v1/clientes/" + creado.idCliente()))
            .body(creado);
    }
}

```

`@Max(100)` en `size` implementa el tope de `04-CONTRATO-API.md` \$4.

8. DTO con records

Java 25; los DTO son `record` :

```

public record CrearClienteRequest(
    @NotBlank(message = "El nombre es obligatorio")
    @Size(max = 100, message = "El nombre no puede superar 100 caracteres")
    String nombre,

    @Size(max = 100) String contacto,
    @Size(max = 20) String telefono
) {}

public record ClienteResponse(
    Long idCliente, String nombre, String contacto, String telefono, boolean activo
) {}

```

Los mensajes de validación van **en español**: llegan al usuario tal cual a través de `fieldErrors`.

9. Pruebas

```

@Test
void noPermiteConsumirMasStockDelDisponible() {
    when(inventarioRepository.bloquearYObtener(3L)).thenReturn(new BigDecimal("10.00"));

    var ex = assertThrows(ConflictoNegocioException.class,
        () -> consumoService.registrar(1L, List.of(new ConsumoRequest(3L, ..., new BigDecimal("15.00")))));

    assertEquals("STOCK_INSUFICIENTE", ex.getCodigo());
    verify(inventarioService, never()).registrarSalida(any(), any(), any(), any(), any());
}

```

Lo que se prueba obligatoriamente (`05-ESTANDARES-QA.md` §1): cálculos, validaciones de stock, transiciones de estado y la fórmula de eficiencia con `cantidad_real = 0`.

07 — Arquitectura Frontend

Angular 22 · TypeScript 5.9 · Angular Material 22 · componentes standalone

Angular cambió bastante entre la versión 17 (la del stack original) y la 22. Los ejemplos de este documento usan la sintaxis actual: `sin NgModule`, con `@if / @for` en las plantillas y `signals` para el estado. Un tutorial con `*ngIf` y `standalone: true` explícito es de una versión anterior.

1. Arranque

```
npm install -g @angular/cli@22
ng new productionfood-frontend --routing --style=scss --ssr=false
cd productionfood-frontend
ng add @angular/material@22
```

`--ssr=false`: es una aplicación interna tras autenticación, no un sitio público. El renderizado en servidor añade complejidad sin beneficio.

La estructura de carpetas está en `01-CONVENCIONES.md` §4.

2. Configuración raíz

```
// app.config.ts
export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes, withComponentInputBinding()),
    provideHttpClient(withInterceptors([authInterceptor, errorInterceptor])),
    provideAnimationsAsync(),
    { provide: MAT_DATE_LOCALE, useValue: 'es-CO' },
    { provide: LOCALE_ID, useValue: 'es-CO' }
  ]
};
```

`LOCALE_ID = 'es-CO'` hace que los pipes `date`, `number` y `currency` usen el formato colombiano: `$ 12.500,50` en lugar de `$12,500.50`. Sin esto, los precios se muestran con la coma y el punto invertidos respecto a lo que el usuario espera, y nadie lo nota hasta la sustentación.

Registrar el locale en `main.ts`:

```
import { registerLocaleData } from '@angular/common';
import localeEsCO from '@angular/common/locales/es-CO';
registerLocaleData(localeEsCO);
```

3. Autenticación

`AuthService`

Interceptores

Interceptores

Interceptores

La distinción entre 401 y 403 importa: cerrar la sesión ante un 403 expulsa al usuario por pulsar un botón que no le correspondía.

Guards

```
export const authGuard: CanActivateFn = () => {
  const auth = inject(AuthService);
  const router = inject(Router);
  return auth.autenticado() ? true : router.createUrlTree(['/login']);
};

export const rolGuard = (roles: Rol[]): CanActivateFn => () => {
  const auth = inject(AuthService);
  const router = inject(Router);
  return auth.tieneAlgunRol(roles) ? true : router.createUrlTree(['/sin-acceso']);
};
```

4. Servicios HTTP

Uno por módulo, tipado:

```
@Injectable({ providedIn: 'root' })
export class ClienteService {
  private readonly http = inject(HttpClient);
  private readonly url = `${environment.apiUrl}/clientes`;

  listar(filtro: ClienteFiltro): Observable<PageResponse<Cliente>> {
    let params = new HttpParams()
      .set('page', filtro.page)
      .set('size', filtro.size)
      .set('sort', filtro.sort);
    if (filtro.nombre) params = params.set('nombre', filtro.nombre);
    if (filtro.activo !== undefined) params = params.set('activo', filtro.activo);
    return this.http.get<PageResponse<Cliente>>(this.url, { params });
  }

  crear(dto: CrearClienteDto): Observable<Cliente> {
    return this.http.post<Cliente>(this.url, dto);
  }
}
```

Modelos espejo de los DTO del backend; nunca **any**:

```
export interface PageResponse<T> {
  content: T[];
  page: number; size: number;
  totalElements: number; totalPages: number;
  first: boolean; last: boolean;
}

export interface ApiError {
  timestamp: string; status: number; error: string;
  code: string; message: string; path: string;
  fieldErrors: { field: string; message: string }[];
}
```

5. Componente de listado

```

@Component({
  selector: 'app-cliente-lista',
  imports: [MatTableModule, MatPaginatorModule, MatButtonModule, ReactiveFormsModule],
  templateUrl: './cliente-lista.component.html'
})
export class ClienteListaComponent {
  private readonly service = inject(ClienteService);

  readonly cargando = signal(false);
  readonly error = signal<string | null>(null);
  readonly datos = signal<PageResponse<Cliente> | null>(null);

  readonly filtroNombre = new FormControl('');

  constructor() {
    // debounce: sin esto se dispara una petición por cada tecla pulsada
    this.filtroNombre.valueChanges
      .pipe(debounceTime(350), distinctUntilChanged(), takeUntilDestroyed())
      .subscribe(() => this.cargar(0));
    this.cargar(0);
  }

  cargar(page: number): void {
    this.cargando.set(true);
    this.error.set(null);
    this.service.listar({ page, size: 20, sort: 'nombre,asc', nombre: this.filtroNombre.value ?? '' })
      .subscribe({
        next: r => { this.datos.set(r); this.cargando.set(false); },
        error: (e: HttpErrorResponse) => {
          this.error.set(e.error?.message ?? 'No se pudieron cargar los clientes.');
```

Plantilla con los **tres estados obligatorios** (05-ESTANDARES-QA.md §7):

```

@if (cargando()) {
  <mat-progress-bar mode="indeterminate" />
} @else if (error()) {
  <div class="estado-error">
    <p>{{ error() }}</p>
    <button mat-stroked-button (click)="cargar(0)">Reintentar</button>
  </div>
} @else if (datos()?.content?.length === 0) {
  <div class="estado-vacio">
    <p>No hay clientes registrados.</p>
    <button mat-raised-button color="primary" *siRol="['ADMIN','VENTAS']" (click)="nuevo()">
      Registrar el primero
    </button>
  </div>
} @else {
  <table mat-table [dataSource]="datos()!.content">
    @for (col of columnas; track col) { <!-- ... --> }
  </table>
  <mat-paginator [length]="datos()!.totalElements"
    [pageSize]="datos()!.size"
    [pageIndex]="datos()!.page"
    (page)="cargar($event.pageIndex)" />
}
```

El estado vacío con una acción sugerida evita la pantalla en blanco que hace dudar al usuario de si la aplicación falló o si de verdad no hay datos.

6. Formularios y errores del servidor

```

export class ClienteFormularioComponent {
  readonly form = inject(FormBuilder).group({
    nombre: ['', Validators.required, Validators.maxLength(100)],
    contacto: ['', Validators.maxLength(100)],
    telefono: ['', Validators.maxLength(20)]
  });

  readonly guardando = signal(false);

  guardar(): void {
    if (this.form.invalid) { this.form.markAllAsTouched(); return; }
    this.guardando.set(true); // deshabilita el botón: evita doble envío

    this.service.crear(this.form.getRawValue() as CrearClienteDto).subscribe({
      next: () => { this.noti.exito('Cliente registrado.');
```

Mostrar el error del servidor junto al campo —y no solo en un aviso flotante— es lo que permite al usuario corregir sin adivinar cuál de los cinco campos estaba mal.

7. Directiva de rol

```

@Directive({ selector: '[siRol]' })
export class SiRolDirective {
  private readonly auth = inject(AuthService);
  private readonly vista = inject(ViewContainerRef);
  private readonly plantilla = inject(TemplateRef<unknown>);

  @Input() set siRol(roles: Rol[]) {
    this.vista.clear();
    if (this.auth.tieneAlgunRol(roles)) this.vista.createEmbeddedView(this.plantilla);
  }
}

```

Esto es **cosmética, no seguridad**. Ocultar un botón mejora la experiencia; cualquiera puede llamar el endpoint con Postman. La autorización real está en el backend ([03-MATRIZ-ROLES.md](#) §5).

8. Rutas

```

export const routes: Routes = [
  { path: 'login', loadComponent: () => import('./features/auth/login.component').then(m => m.LoginComponent) },
  {
    path: '',
    canActivate: [authGuard],
    loadComponent: () => import('./layout/layout.component').then(m => m.LayoutComponent),
    children: [
      { path: 'dashboard', loadChildren: () => import('./features/dashboard/dashboard.routes') },
      { path: 'clientes', canActivate: [rolGuard(['ADMIN', 'VENTAS', 'PRODUCCION', 'CONSULTA'])],
        loadChildren: () => import('./features/cliente/cliente.routes') },
      { path: 'usuarios', canActivate: [rolGuard(['ADMIN'])],
        loadChildren: () => import('./features/usuario/usuario.routes') },
      { path: '', redirectTo: 'dashboard', pathMatch: 'full' }
    ]
  },
  { path: 'sin-acceso', loadComponent: () => import('./shared/sin-acceso.component').then(m => m.SinAccesoComponent) },
  { path: '**', redirectTo: '' }
];

```

Carga diferida por módulo: el usuario de VENTAS no descarga el código de producción.

9. Formato de cantidades y precios

```
{{ producto.precio | currency:'COP':'symbol-narrow':'1.2-2' }} <!-- $ 12.500,00 -->
{{ inventario.cantidadDisponible | number:'1.2-2' }} {{ unidad }}
```

Con `LOCALE_ID = 'es-CO'`, el separador de miles es el punto y el decimal la coma.

Los totales no se calculan en el frontend. El total de un pedido o de una compra lo devuelve el backend. Si el cliente suma por su cuenta, dos capas calculan lo mismo con reglas de redondeo distintas y aparecen diferencias de centavos imposibles de explicar. El frontend muestra; el backend calcula.

10. Entornos

```
// environments/environment.ts
export const environment = { production: false, apiUrl: 'http://localhost:8080/api/v1' };

// environments/environment.prod.ts
export const environment = { production: true, apiUrl: '/api/v1' };
```

En producción se usa ruta relativa: el frontend se sirve desde el mismo origen que la API y desaparece el problema de CORS.

08 — Desviaciones frente a la Entrega 1

El documento *Plantilla Entrega 1 — Requerimientos y Arquitectura* (fecha de entrega 09/09/2026) ya fue presentado. Este trabajo de preparación encontró diferencias entre ese documento y las historias de usuario.

Decisión del equipo: las historias de usuario son la fuente de verdad para el alcance funcional. El documento de arquitectura lo es para la infraestructura de datos.

Este registro existe para corregir la Entrega 1 en su próxima versión, no para reprocharla: un desfase entre el documento de requerimientos y el backlog es normal cuando el segundo se detalla después. Lo que no sería normal es dejarlo sin registrar.

Resumen

#	Tema	Entrega 1	Historias de usuario	Decisión
D-01	Alcance: compras y proveedores	Excluido explícitamente	HU-06, 13, 14, 15	Se incluye
D-02	Alcance: pedidos, ventas, devoluciones	No mencionado	HU-16, 17, 18, 25	Se incluye
D-03	Alcance: lotes y trazabilidad	No mencionado	HU-09, 19	Se incluye
D-04	Número de requerimientos	10 RF	27 HU	27 HU
D-05	Actores	3	5 implícitos	5 roles
D-06	Base de datos	MySQL en AWS RDS	MariaDB 10.4 local	MySQL 8.4 en AWS RDS
D-07	Versiones del stack	Sin versiones	Versiones fuera de soporte	Actualizadas
D-08	Bitácora de auditoría	No mencionada	HU-04	Se incluye

D-01 · Compras y proveedores

Entrega 1, §3.2 Limitaciones: “El proyecto no incluirá en esta primera versión: [...] Gestión de proveedores y compras.”

Historias de usuario: HU-06 (proveedores), HU-13 (registro de compras), HU-14 (detalle de compra), HU-15 (historial de compras).

Decisión: se incluyen. No es solo que estén en el backlog — es que **sin compras el sistema no funciona**.

La razón es concreta: HU-23 descuenta materia prima del inventario al producir, HU-10 lo consulta y HU-27 alerta cuando está bajo. Pero ninguna historia definía cómo *entra* la materia prima. La recepción de compra es el punto de entrada principal del stock (ver [02-CORRECCIONES-DB.md](#) §C-03). Excluir compras deja el inventario permanentemente en cero y vacía de sentido tres historias más.

Acción en Entrega 2: retirar esa línea de §3.2 y agregar el módulo a §3.1.

D-02 · Pedidos, ventas y devoluciones

Entrega 1: no aparecen en el alcance, ni en los requerimientos funcionales, ni en los módulos de la sección 4.

Historias de usuario: HU-16 a HU-18 y HU-25.

Decisión: se incluyen. Son un módulo completo con impacto sobre el inventario de producto terminado. Su ausencia en la Entrega 1 parece un descuido de redacción más que una decisión de alcance: el modelo entidad-relación de §8 **sí incluye pedidos**, **detalle_pedido** y **devoluciones**. El esquema los contemplaba; el texto no los mencionó.

Acción en Entrega 2: agregar el módulo de pedidos a §3.1 y sus requerimientos a §4.

D-03 · Lotes y control de vencimientos

Entrega 1: no se menciona el manejo de lotes ni de fechas de vencimiento.

Historias: HU-09 (lotes de materia prima), HU-19 (lotes de producto terminado), HU-24 CA-05 (productos próximos a vencer).

Decisión: se incluyen. En una pyme de alimentos la trazabilidad por lote y el control de vencimientos no son un extra: son el requisito sanitario básico del sector. Su omisión en el documento debilita la justificación del proyecto.

La tabla **lotes** ya estaba en el modelo entidad-relación de §8.

D-04 · 10 requerimientos vs. 27 historias

Los 10 RF de la Entrega 1 son un subconjunto de las 27 historias. Correspondencia:

RF Entrega 1	Historia
RF-01 Registrar usuarios	HU-01
RF-02 Asignar roles	HU-01, HU-02
RF-03 Registrar productos	HU-11
RF-04 Registrar materias primas	HU-08
RF-05 Consultar inventario de MP	HU-10
RF-06 Registrar recetas	HU-12
RF-07 Asociar materias primas a receta	HU-12
RF-08 Crear planes de producción	HU-20
RF-09 Actualizar estado del plan	HU-20
RF-10 Registrar cantidad producida	HU-21, HU-22

Sin cobertura en la Entrega 1: HU-02 (gestión de usuarios), HU-03 (login), HU-04 (bitácora), HU-05 a HU-07 (maestros), HU-09, HU-13 a HU-19, HU-23 a HU-27.

Llama la atención que **HU-03, el inicio de sesión, no tenga un RF asociado**, aun cuando RNF-01 exige restringir el acceso según el rol. No se puede restringir por rol sin autenticar.

Acción en Entrega 2: ampliar la tabla de §4 con los requerimientos faltantes.

D-05 · Tres actores vs. cinco roles

Entrega 1, §6: Administrador, Encargado de producción, Usuario de consulta.

Historias de usuario: además de administrador y productor, aparecen *comprador* (HU-13, 14, 15) y *vendedor* (HU-16, 17, 18).

Decisión: cinco roles — **ADMIN**, **PRODUCCION**, **COMPRAS**, **VENTAS**, **CONSULTA**. Detalle completo en **03-MATRIZ-ROLES.md**.

Con tres actores, las operaciones de compras y ventas caerían todas en **ADMIN**, y el sistema perdería la separación de funciones que RNF-01 pretende.

Acción en Entrega 2: ampliar la tabla de actores de §6.

D-06 · Base de datos

Entrega 1, §7 y §9: MySQL alojado en AWS RDS. **Historias de usuario:** MariaDB 10.4 local.

Decisión: MySQL 8.4 LTS en AWS RDS, como indica la Entrega 1 y confirmó el equipo.

Se añade una precisión de versión que la Entrega 1 no daba: **RDS para MySQL 8.0 terminó su soporte estándar el 31 de julio de 2026** y pasa a facturarse como Extended Support. Crear la instancia en 8.0 hoy implica pagar por una versión obsoleta. Detalle en **00-STACK-VERSIONES.md** y **11-DESPLIEGUE-AWS-RDS.md**.

Acción en Entrega 2: especificar “MySQL 8.4 LTS” en §9, y corregir la tabla de stack de las historias de usuario, que decía MariaDB. (Ya corregida en [previo/Historias_de_Usuario.md](#).)

D-07 · Versiones del stack

La Entrega 1 nombra las tecnologías sin versión; las historias sí las daban, y varias estaban fuera de soporte a septiembre de 2026:

Componente	Declarado	Estado	Adoptado
Java	21	LTS vigente, pero ya no el actual	25 LTS
Spring Boot	3.3.x	Fuera de soporte OSS	4.1.x
Angular	17	Fuera de soporte	22
SpringDoc	2.6.0	Incompatible con Spring Boot 4	3.1.1
jjwt	0.12.6	Funcional, hay versión más reciente	0.13.0

Justificación de cada cambio en [00-STACK-VERSIONES.md](#).

Acción en Entrega 2: incluir la columna de versión en la tabla de §9.

D-08 · Bitácora de auditoría

Entrega 1: no la menciona en el alcance ni en los requerimientos. La tabla [bitacora](#) **sí aparece** en el modelo entidad-relación de §8, pero no se describe su propósito en la tabla de entidades.

Historias: HU-04 la define completa.

Decisión: se incluye. Es lo que da trazabilidad a las operaciones y sostiene RNF-01.

Requerimientos no funcionales: sin desviaciones

Los seis RNF de la Entrega 1 se mantienen y se trasladan a criterios verificables:

RNF	Cómo se verifica
RNF-01 Seguridad	Batería SEC-01..SEC-05 en cada endpoint (05-ESTANDARES-QA.md §4)
RNF-02 Rendimiento (< 2 s)	Aserción de tiempo en cada petición de Postman
RNF-03 Usabilidad	Tres estados obligatorios por pantalla (05-ESTANDARES-QA.md §7)
RNF-04 Escalabilidad	Estructura por módulos (01-CONVENCIONES.md §2)
RNF-05 Disponibilidad	Fuera del alcance de pruebas automatizadas
RNF-06 Mantenibilidad	Separación en capas, revisión de PR obligatoria

Qué llevar a la Entrega 2

1. Corregir §3.1 y §3.2: incluir compras, proveedores, pedidos, devoluciones y lotes.
2. Ampliar §4 con los 17 requerimientos faltantes.
3. Ampliar §6 a cinco actores.
4. Especificar versiones en §9, con MySQL 8.4 LTS.
5. Documentar la bitácora en la tabla de entidades de §8.
6. Incorporar las correcciones de esquema de [02-CORRECCIONES-DB.md](#) al modelo entidad-relación: [precio_unitario](#) en [detalle_pedido](#), las dos tablas de kardex y las restricciones de unicidad.

09 — Puesta en Marcha del Entorno

Pasos para dejar ProductionFood funcionando en una máquina nueva.

1. Requisitos

Herramienta	Versión	Verificar con
JDK (Temurin)	25	<code>java -version</code>
Maven	3.9.x	<code>mvn -version</code>
Node.js	22 o 24 LTS	<code>node -v</code>
npm	10+	<code>npm -v</code>
MySQL	8.4	<code>mysql --version</code>
Git	cualquiera	<code>git --version</code>
Postman	actual	—

`java -version` debe decir 25. Con varios JDK instalados, `JAVA_HOME` manda sobre el `java` del `PATH`, y Maven usa `JAVA_HOME`: es la causa habitual de que el proyecto compile en un equipo y no en otro.

2. Base de datos local

Con Docker (recomendado — la versión queda fijada y no interfiere con otras instalaciones):

```
docker run --name productionfood-mysql \
  -e MYSQL_ROOT_PASSWORD=local \
  -e MYSQL_DATABASE=productionfood \
  -e TZ=America/Bogota \
  -p 3306:3306 \
  -d mysql:8.4 \
  --character-set-server=utf8mb4 \
  --collation-server=utf8mb4_0900_ai_ci \
  --default-time-zone=-05:00
```

Sin Docker: instalar MySQL 8.4 y crear la base:

```
CREATE DATABASE productionfood
CHARACTER SET utf8mb4
COLLATE utf8mb4_0900_ai_ci;
```

`--default-time-zone=-05:00` no es opcional: sin ella, `curdate()` devuelve UTC y los pedidos registrados después de las 7 p.m. quedan con la fecha del día siguiente ([02-CORRECCIONES-DB.md](#), sección MySQL).

Migraciones

```
cd docs/00-base/migraciones
mysql -u root -p productionfood < V1_esquema_base.sql
mysql -u root -p productionfood < V2_correcciones.sql
mysql -u root -p productionfood < V3_datos_semilla.sql
```

Verificación:

```
SELECT VERSION(), @@time_zone;      -- 8.4.x -05:00
SELECT COUNT(*) FROM roles;         -- 5
SHOW TABLES LIKE 'movimientos_%';  -- 2 tablas
```

3. Backend

```
git clone <url-del-repositorio>
cd productionfood/backend
```

Variables de entorno. Crear `.env` local (está en `.gitignore`):

```
export DB_HOST=localhost
export DB_USER=root
export DB_PASSWORD=local
export JWT_SECRET=$(openssl rand -base64 48)
```

```
source .env
mvn clean install
mvn spring-boot:run
```

Comprobaciones:

- API en `http://localhost:8080`

- Swagger en `http://localhost:8080/swagger-ui.html`
- Prueba de login:

```
curl -X POST http://localhost:8080/api/v1/auth/login \
-H "Content-Type: application/json" \
-d '{"correo":"admin@productionfood.local","password":"Admin123*"}'
```

Debe devolver un token. Si devuelve `401`, la semilla V3 no se aplicó o el `PasswordEncoder` no es BCrypt.

⚠ **La contraseña `Admin123*` es pública:** está en el script de semilla, en este repositorio. Sirve para arrancar en local. Antes de exponer la aplicación fuera de `localhost`, crear un administrador real y desactivar este usuario (tarea de cierre en `HU-01/tarea-backend.md`).

4. Frontend

```
cd productionfood/frontend
npm ci # no 'npm install': ci respeta package-lock.json exactamente
ng serve
```

Aplicación en `http://localhost:4200`.

`npm ci` en lugar de `npm install`: instala las versiones exactas del *lock file*. Con `npm install`, cada integrante puede acabar con versiones menores distintas y aparecen fallos que solo le ocurren a una persona.

5. Postman

1. Importar `qa/ProductionFood.postman_collection.json`.
2. Importar `qa/local.postman_environment.json`.
3. Seleccionar el entorno **local**.
4. Ejecutar la carpeta `00 - Setup` para obtener los tokens de los cinco roles.

Los tokens duran una hora; al expirar se vuelve a ejecutar esa carpeta.

6. Problemas frecuentes

Síntoma	Causa	Solución
<code>Communications link failure</code> al arrancar	MySQL no está corriendo o el puerto difiere	<code>docker ps</code> / revisar <code>DB_HOST</code>
<code>Unknown database 'productionfood'</code>	La base no se creó	Ver §2
<code>Access denied for user 'root'</code>	Contraseña incorrecta	Revisar <code>DB_PASSWORD</code>
<code>WeakKeyException</code> de jjwt	<code>JWT_SECRET</code> con menos de 256 bits	Regenerar con <code>openssl rand -base64 48</code>
Todos los endpoints responden <code>403</code>	Falta el token, o falta <code>@EnableMethodSecurity</code>	Ver <code>06-ARQUITECTURA-BACKEND.md</code> §2
Todos los endpoints responden <code>200</code> incluso con rol incorrecto	Falta <code>@EnableMethodSecurity</code>	Agregarla. La autorización está desactivada
CORS bloqueado en el navegador	El origen del frontend no está permitido	Revisar <code>corsConfigurationSource</code>
Las fechas aparecen con un día de diferencia	Zona horaria en UTC	§2 y <code>11-DESPLIEGUE-AWS-RDS.md</code> §3
<code>Error 3819: Check constraint violated</code>	Un CHECK está funcionando	No es un fallo
El proyecto no compila tras actualizar	Versión de JDK distinta	<code>echo \$JAVA_HOME</code> debe apuntar al JDK 25

7. Estructura del repositorio

```
productionfood/
├── backend/      Proyecto Maven (Spring Boot)
├── frontend/     Proyecto Angular
├── docs/         Esta documentación
│   ├── 00-base/
│   ├── HU-NN/
│   └── previo/   Documentos originales de la Entrega 1
├── qa/          Colecciones de Postman y datos de prueba
└── README.md
```

`.gitignore` — lo que nunca se versiona

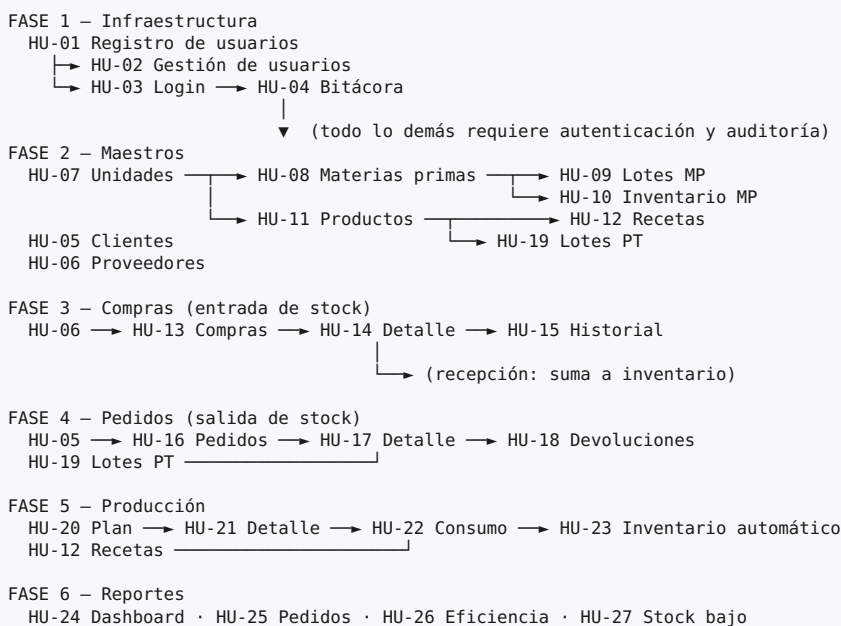
```
.env
*.env.local
application-local.properties
backend/target/
frontend/node_modules/
frontend/dist/
.idea/
.vscode/
*.log
```

.env en el repositorio es una fuga de credenciales. Los bots que rastrean GitHub buscando cadenas de conexión y secretos las encuentran en minutos, no en días. Si llega a subirse una contraseña, no basta con borrarla en un commit posterior: queda en el historial y **hay que rotarla**.

10 — Roadmap y Dependencias

Orden de ejecución de las 27 historias y qué bloquea qué.

1. Grafo de dependencias



2. Orden de ejecución

Sprint 0 — Preparación (antes de la primera historia)

Tareas sin las cuales todo lo demás se atasca:

- ☐ Repositorio creado, con ramas y `.gitignore`.
- ☐ Instancia RDS MySQL 8.4 creada (`11-DESPLIEGUE-AWS-RDS.md`).
- ☐ Migraciones V1, V2 y V3 aplicadas en local y en RDS.
- ☐ Proyecto Spring Boot 4.1 que arranca y conecta a la base.
- ☐ Proyecto Angular 22 que arranca.
- ☐ `GlobalExceptionHandler`, `PageRequest` / `PageResponse` y `SecurityConfig` en su sitio.
- ☐ Colección de Postman con el entorno configurado.

Esto no es una historia de usuario pero es trabajo real. Presupuestar una semana. Empezar HU-01 sin la infraestructura lista hace que la primera historia arrastre todo el andamiaje y parezca que “HU-01 tardó tres semanas”.

Sprint 1 — Seguridad (HU-01, 02, 03, 04)

Bloquea absolutamente todo lo demás: sin login no hay endpoint protegido que probar y sin bitácora hay que volver a tocar cada servicio después.

HU-04 va en el Sprint 1, no al final. La auditoría se implementa como aspecto AOP transversal; retrasarla obliga a revisar todos los servicios ya escritos.

Sprint 2 — Maestros (HU-07, 05, 06, 08, 11)

HU-07 (unidades) primero: HU-08 y HU-11 dependen de ella.

Son cinco CRUD de estructura casi idéntica. **Se implementa HU-05 completo —backend, frontend, pruebas— y se usa como plantilla** para los otros cuatro. Hacerlos en paralelo desde cero multiplica por cinco las decisiones de diseño y produce cinco estilos distintos.

Sprint 3 — Inventario y recetas (HU-09, 10, 12, 19)

Aquí aparece `InventarioService` con el kardex. Es el componente más delicado del sistema: lo tocan compras, pedidos, devoluciones y producción.

Se escribe una vez, bien, con pruebas unitarias, antes de que cuatro módulos dependan de él.

Sprint 4 — Compras (HU-13, 14, 15)

Primer flujo que suma stock. La recepción de compra (`PATCH /compras/{id}/estado` → `RECIBIDA`) es transaccional y genera movimientos de kardex.

Sprint 5 — Pedidos (HU-16, 17, 18)

Primer flujo que resta stock de producto terminado, con política FEFO. Requiere HU-19 hecha.

Sprint 6 — Producción (HU-20, 21, 22, 23)

HU-23 es la más compleja del proyecto: transaccional, con bloqueo de filas y alerta de stock mínimo.

Sprint 7 — Reportes (HU-24, 25, 26, 27)

Solo lectura. Dependen de que haya datos de los sprints anteriores: **son las últimas por necesidad**, no por prioridad.

3. Dependencias externas de cada historia

HU	Requiere cerradas	Nota
HU-01	—	Sprint 0 completo
HU-02	HU-01	
HU-03	HU-01	Bloquea todo lo demás
HU-04	HU-03	Transversal: se aplica a cada módulo posterior
HU-05	HU-03, HU-04	Plantilla de CRUD
HU-06	HU-05	Mismo patrón
HU-07	HU-03	Bloquea HU-08 y HU-11
HU-08	HU-07	
HU-09	HU-08	Suma inventario → requiere InventarioService
HU-10	HU-08	Consulta la vista v_inventario_materia_prima
HU-11	HU-07	
HU-12	HU-08, HU-11	UNIQUE de ingrediente por receta
HU-13	HU-06	
HU-14	HU-13, HU-08	Cálculo de totales
HU-15	HU-14	
HU-16	HU-05	
HU-17	HU-16, HU-19	Requiere stock de PT: sin HU-19 no hay nada que vender
HU-18	HU-17	Devuelve stock al aprobar
HU-19	HU-11	Crea inventario de PT
HU-20	HU-01	
HU-21	HU-20, HU-11	
HU-22	HU-21, HU-12	Teórico desde la receta
HU-23	HU-22, HU-10	La más compleja
HU-24	HU-13, HU-16, HU-20	Necesita datos de varios módulos
HU-25	HU-17	
HU-26	HU-22	
HU-27	HU-10	La más simple de la fase

4. Riesgos

Riesgo	Impacto	Mitigación
HU-17 planificada antes que HU-19	Alto	No hay stock de producto terminado que descontar: la historia no se puede probar. El roadmap ya las ordena
InventarioService escrito dos veces	Alto	Cuatro módulos lo consumen. Se escribe una vez en el Sprint 3, con pruebas
HU-04 dejada para el final	Medio	Obliga a revisitar todos los servicios. Va en el Sprint 1
Spring Boot 4 + Security 7 sin material de apoyo	Medio	06-ARQUITECTURA-BACKEND.md trae la configuración escrita. Plan B: retroceder a 3.5.x si el bloqueo supera dos días
Los cinco CRUD en paralelo desde cero	Medio	HU-05 primero como plantilla
Descuadres de inventario detectados tarde	Alto	Conciliación de kardex al cerrar cada fase (05-ESTANDARES-QA.md §5)
Instancia RDS olvidada facturando	Medio	Alarma de facturación y borrado al terminar el semestre

5. Reparto sugerido

Según el cronograma de la Entrega 1 (§10):

Sprint	Backend	Frontend	QA / BD
0	Greyliis, Carlos, Valeria	Ismael, Natalia	Greyliis (RDS)
1	Greyliis, Carlos	Ismael, Natalia	Natalia
2	Carlos, Valeria	Ismael, Natalia	Greyliis
3	Greyliis, Carlos	Ismael	Natalia
4	Valeria, Carlos	Natalia	Greyliis
5	Greyliis, Valeria	Ismael, Natalia	Natalia
6	Carlos, Greyliis	Ismael	Greyliis
7	Valeria	Ismael, Natalia	Natalia, Greyliis

El frontend de una historia va un sprint detrás de su backend. Mientras backend hace el Sprint N, frontend cierra el N-1 contra un contrato ya estable. Trabajar ambos a la vez sobre la misma historia obliga a rehacer el cliente cada vez que cambia una respuesta.

El contrato de `especificacion.md` es lo que permite a frontend empezar antes: se trabaja contra él, no contra la implementación.

11 — Despliegue de la Base de Datos en AWS RDS

Puesta en marcha de la instancia MySQL 8.4 que usa ProductionFood.

⚠ **Antes de empezar: esto cuesta dinero si se configura mal.** La capa gratuita de AWS cubre 750 horas/mes de `db.t4g.micro` durante 12 meses, pero solo si se eligen exactamente las opciones indicadas. Multi-AZ, una clase de instancia mayor o almacenamiento aprovisionado se facturan desde el primer minuto. Hay una lista de verificación de costos al final.

1. Creación de la instancia

Consola de AWS → RDS → **Create database.**

Opción	Valor	Por qué
Creation method	Standard create	<i>Easy create</i> elige opciones que no son las de capa gratuita
Engine	MySQL	—
Version	MySQL 8.4.x	8.0 salió de soporte estándar el 31-jul-2026 y factura Extended Support
Templates	Free tier	Fija automáticamente las opciones sin costo
DB instance identifier	<code>productionfood-db</code>	—
Master username	<code>admin</code>	No usar <code>root</code>
Master password	Generada, guardada en gestor de contraseñas	Nunca en el repositorio
Instance class	<code>db.t4g.micro</code>	Incluida en capa gratuita
Storage type	General Purpose SSD (gp3), 20 GB	Mínimo de capa gratuita
Storage autoscaling	Desactivado	Activo, puede crecer y facturar sin aviso
Multi-AZ	No	Duplica el costo. Innecesario para un proyecto académico
Public access	Yes (solo durante el desarrollo)	Ver \$2 antes de aceptar esto
VPC security group	Nuevo: <code>productionfood-sg</code>	—
Initial database name	<code>productionfood</code>	Crea la base; evita un paso manual
Backup retention	7 días	Incluido
Deletion protection	Activado	Evita borrar la instancia por accidente

2. Acceso público y grupo de seguridad

`Public access = Yes` hace la instancia alcanzable desde Internet. Es la forma práctica de que cinco integrantes trabajen desde sus casas sin montar una VPN, pero **la regla del security group es lo único que separa la base de datos del mundo.**

Lo que no se debe hacer

Tipo: MySQL/Aurora · Puerto: 3306 · Origen: 0.0.0.0/0 x

Esto expone el puerto 3306 a Internet entero. Los escáneres automáticos encuentran instancias RDS abiertas **en cuestión de horas**, no de días, y empiezan a probar credenciales. No es un riesgo teórico ni exagerado: es el motivo más común de bases de datos académicas comprometidas.

Lo correcto

Una regla por integrante, con su IP pública:

```
Tipo: MYSQL/Aurora · Puerto: 3306 · Origen: <IP pública>/32 · Descripción: "Carlos - casa"
```

Cada quien obtiene su IP en checkip.amazonaws.com.

Con IP dinámica del proveedor de Internet, la dirección cambia cada pocos días y hay que actualizar la regla. Es molesto. Es también el precio de no dejar la base abierta, y actualizar una regla toma treinta segundos.

3. Parameter Group — la zona horaria

Este paso no es opcional y se olvida siempre.

Una instancia RDS nueva está en **UTC**. Colombia es UTC-5. Un pedido registrado a las 7:30 p.m. se guarda con la fecha del día siguiente, porque `curdate()` lo evalúa el servidor de base de datos.

Consecuencia: HU-24 CA-01 ("pedidos del día actual") y los filtros por fecha de HU-25 dejan de coincidir con lo que el usuario ve. Y como solo ocurre después de las 7 p.m., las pruebas de la mañana pasan sin problema y el defecto llega hasta la sustentación.

Configuración

RDS → Parameter groups → **Create parameter group**

Campo	Valor
Family	mysql8.4
Name	productionfood-params

Parámetros a modificar:

Parámetro	Valor	Motivo
time_zone	America/Bogota	Lo anterior
character_set_server	utf8mb4	Acentos y ñ
collation_server	utf8mb4_0900_ai_ci	Coincide con las migraciones
max_connections	Dejar por defecto	El pool de HikariCP lo gobierna desde la aplicación

Luego: RDS → instancia → Modify → DB parameter group → `productionfood-params` → **Apply immediately** → reiniciar la instancia (`time_zone` es estático y requiere reinicio).

Verificación:

```
SELECT @@time_zone, @@system_time_zone, NOW(), CURDATE();
-- Se espera: America/Bogota · -05 · hora local de Colombia
```

En la aplicación, además:

```
spring.jackson.time-zone=America/Bogota
```

```
java -Duser.timezone=America/Bogota -jar productionfood-api.jar
```

Los dos lados. Ajustar solo la aplicación no cambia lo que `curdate()` devuelve en el servidor.

4. Conexión desde la aplicación

```
application.properties
```

```
spring.datasource.url=jdbc:mysql://${DB_HOST}:3306/productionfood?
useSSL=true&requireSSL=true&serverTimezone=America/Bogota&characterEncoding=utf8
spring.datasource.username=${DB_USER}
spring.datasource.password=${DB_PASSWORD}
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# Pool: la capa gratuita es pequeña, no tiene sentido pedir 50 conexiones
spring.datasource.hikari.maximum-pool-size=10
spring.datasource.hikari.minimum-idle=2
spring.datasource.hikari.connection-timeout=30000

spring.jackson.time-zone=America/Bogota
```

⚠ **Las credenciales van en variables de entorno, nunca en el archivo versionado.** Un `application.properties` con la contraseña de RDS subido a un repositorio público es una fuga de credenciales, y los bots que rastrean GitHub buscando cadenas de conexión encuentran ese patrón en minutos. `application.properties` se versiona con los `${...}`; los valores viven en el entorno local de cada integrante y `.env` va en `.gitignore`.

Verificación de conectividad

```
mysql -h productionfood-db.xxxxxxxx.us-east-1.rds.amazonaws.com \
-u admin -p --ssl-mode=REQUIRED productionfood -e "SELECT VERSION(), @@time_zone;"
```

5. Aplicación de las migraciones

En orden, desde `00-base/migraciones/`:

```
export RDS_HOST=productionfood-db.xxxxxxxx.us-east-1.rds.amazonaws.com

for f in V1_esquema_base.sql V2_correcciones.sql V3_datos_semilla.sql; do
  echo "Aplicando $f..."
  mysql -h "$RDS_HOST" -u admin -p --ssl-mode=REQUIRED productionfood < "$f" || break
done
```

Antes de ejecutar sobre una base con datos: tomar una snapshot. RDS → instancia → Actions → Take snapshot. Tarda un minuto y es la diferencia entre un susto y una pérdida. Recuerdese que el `ROLLBACK` no revierte sentencias DDL.

6. Problemas frecuentes

Síntoma	Causa	Solución
<code>Communications link failure</code> / cuelgue al conectar	El security group no permite tu IP	Agregar tu IP pública actual. Cambia sola con IP dinámica
<code>Access denied for user 'admin'</code>	Contraseña incorrecta, o se está conectando a la base equivocada	Revisar credenciales y nombre de base
<code>Public Key Retrieval is not allowed</code>	Driver MySQL 9 con la configuración SSL por defecto	Usar <code>useSSL=true</code> en la URL. No recurrir a <code>allowPublicKeyRetrieval=true</code> , que debilita la conexión
<code>Unknown database 'productionfood'</code>	No se indicó <i>Initial database name</i> al crear la instancia	<code>CREATE DATABASE productionfood CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;</code>
Las fechas aparecen con un día de diferencia	<code>time_zone</code> en UTC	§3 de este documento
<code>Error 3819: Check constraint violated</code>	Un CHECK está funcionando	No es un fallo: es la base rechazando un dato inválido
La factura llegó con cargos inesperados	Multi-AZ, autoscaling o Extended Support activos	§7

7. Control de costos

Lista de verificación al terminar de crear la instancia:

- ☐ Clase de instancia `db.t4g.micro` — no mayor.
- ☐ Multi-AZ **desactivado**.
- ☐ Storage autoscaling **desactivado**.
- ☐ Almacenamiento 20 GB gp3 — no aprovisionado (io1/io2 se facturan aparte).
- ☐ Versión **8.4**, no 8.0 — 8.0 incurre en cargos de Extended Support.
- ☐ **Una sola instancia**. Cinco integrantes comparten una, no crean una cada uno.

- ☐ Alarma de facturación configurada en CloudWatch (por ejemplo, USD 5).
- ☐ Recordatorio en el calendario: **la capa gratuita vence a los 12 meses.**

Al terminar el semestre

RDS → instancia → Actions → Take final snapshot → Delete

La snapshot conserva los datos por si hay que retomar el proyecto y cuesta una fracción de la instancia. **Una instancia RDS olvidada sigue facturando meses después de que el curso terminó** — es el final más común de los proyectos académicos en AWS.

Recuérdese desactivar *Deletion protection* antes de borrar.

8. Desarrollo local

No hace falta —ni conviene— que los cinco integrantes trabajen contra RDS todo el tiempo: consume la capa gratuita y cada prueba destructiva afecta a los demás.

MySQL 8.4 local para desarrollar; RDS para integración y para la sustentación.

```
docker run --name productionfood-mysql \  
-e MYSQL_ROOT_PASSWORD=local \  
-e MYSQL_DATABASE=productionfood \  
-e TZ=America/Bogota \  
-p 3306:3306 \  
-d mysql:8.4 \  
--character-set-server=utf8mb4 \  
--collation-server=utf8mb4_0900_ai_ci \  
--default-time-zone=-05:00
```

La misma versión mayor en ambos entornos. Desarrollar sobre MySQL 5.7 o MariaDB y desplegar en 8.4 hace que los CHECK se ignoren en local y se apliquen en el servidor: el código funciona en la máquina de cada quien y falla en la demostración.

Fase 1 · Infraestructura y Seguridad

HU-01 · Registro de usuarios

Especificación

Fase 1 · Fuente: `previo/Historias_de_Usuario.md`

Como administrador, **quiero** registrar usuarios del sistema con nombre, correo, contraseña y rol, **para que** controle quién tiene acceso al sistema.

Depende de: Ninguna. Es la primera historia del proyecto.

1. Criterios de aceptación

CA	Enunciado	Cómo se implementa
CA-01	Nombre obligatorio, máx. 100 caracteres	<code>@NotBlank</code> + <code>@Size(max=100)</code> en el DTO
CA-02	Correo obligatorio y único	<code>@NotBlank</code> + <code>@Email</code> ; unicidad verificada en el servicio y garantizada por <code>UNIQUE KEY correo</code>
CA-03	Contraseña obligatoria, máx. 200, almacenada como hash	<code>@NotBlank</code> + <code>@Size(min=8,max=72)</code> ; se guarda <code>BCryptPasswordEncoder.encode()</code>
CA-04	El rol debe existir en <code>roles</code>	El servicio valida contra <code>RoleRepository</code> antes de insertar
CA-05	Estado por defecto activo (<code>1</code>)	<code>DEFAULT 1</code> añadido en V2 (§C-06). El DTO no recibe <code>estado</code>
CA-06	Se retorna el usuario creado con su <code>id_usuario</code>	<code>201</code> + <code>Location</code> + <code>UsuarioResponse</code>

2. Endpoints

Método	Ruta	Descripción	Roles
<code>POST</code>	<code>/api/v1/usuarios</code>	Registrar un usuario	ADMIN
<code>GET</code>	<code>/api/v1/roles</code>	Listar roles disponibles	ADMIN

Reglas transversales (paginación, formato de error, autenticación) en `00-base/04-CONTRATO-API.md`.

3. Reglas de negocio

R-01 · La contraseña nunca se devuelve

`UsuarioResponse` **no tiene campo de contraseña**. Ni el hash. Un DTO de respuesta que arrastra el hash lo expone en cada listado, en los logs del navegador y en las capturas de pantalla que se pegan en los reportes de defectos.

R-02 · Hash con BCrypt, nunca texto plano ni MD5/SHA

`BCryptPasswordEncoder` incorpora *salt* automático y es deliberadamente lento, lo que encarece los ataques por fuerza bruta. Un hash SHA-256 de una contraseña corta se rompe con tablas precalculadas en segundos.

R-03 · Longitud de contraseña

Mínimo 8 caracteres. **Máximo 72**: BCrypt ignora todo lo que exceda ese límite, así que aceptar 200 caracteres daría una falsa sensación de seguridad — las contraseñas de 80 y de 150 caracteres con el mismo prefijo serían equivalentes. La columna es `varchar(200)` porque almacena el hash (60 caracteres), no la contraseña.

R-04 · Unicidad del correo: doble comprobación

El servicio consulta antes de insertar para devolver un `409` legible, pero **la garantía real es el `UNIQUE KEY`**: dos peticiones simultáneas con el mismo correo pasan ambas la consulta previa. Se captura `DuplicateKeyException` y se traduce a `CORREO_DUPLICADO`.

R-05 · Comparación de correos y acentos

La collation `utf8mb4_0900_ai_ci` es *accent-insensitive*: `maria@x.com` y `maría@x.com` se consideran el mismo correo. Es el comportamiento deseado. El correo se normaliza a minúsculas antes de guardar.

4. Modelo de datos

Tabla `usuarios`

Columna	Tipo	Notas
<code>id_usuario</code>	<code>int unsigned</code> PK AI	
<code>nombre</code>	<code>varchar(100)</code> NOT NULL	CA-01
<code>correo</code>	<code>varchar(100)</code> NOT NULL UNIQUE	CA-02
<code>password</code>	<code>varchar(200)</code> NOT NULL	Hash BCrypt (60 car.)
<code>estado</code>	<code>tinyint(1)</code> NOT NULL DEFAULT 1	CA-05 · corregido en V2 §C-06
<code>id_rol</code>	<code>int unsigned</code> NOT NULL FK → <code>roles</code>	CA-04

Roles disponibles (semilla V3): `ADMIN`, `PRODUCCION`, `COMPRAS`, `VENTAS`, `CONSULTA`. Ver `00-base/03-MATRIZ-ROLES.md`.

```
// POST /api/v1/usuarios
{ "nombre": "María Gómez", "correo": "maria@productionfood.local",
  "password": "Clave2026*", "idRol": 4 }

// 201 Created · Location: /api/v1/usuarios/12
{ "idUsuario": 12, "nombre": "María Gómez", "correo": "maria@productionfood.local",
  "activo": true, "rol": { "idRol": 4, "nombre": "VENTAS" } }
```

5. Códigos de error

code	HTTP	Cuándo
<code>CORREO_DUPLICADO</code>	409	Ya existe un usuario con ese correo
<code>ROL_INEXISTENTE</code>	409	El <code>idRol</code> no existe en la tabla <code>roles</code>
<code>PASSWORD_DEBIL</code>	400	Menos de 8 caracteres

Los transversales (`VALIDACION_FALLIDA`, `NO_AUTENTICADO`, `SIN_PERMISO`, `RECURSO_NO_ENCONTRADO`) están en `00-base/04-CONTRATO-API.md` §5.

6. Fuera de alcance

Lo que **no** cubre esta historia y no debe implementarse aquí: se limita estrictamente a los criterios de aceptación listados arriba. Cualquier funcionalidad adicional se propone como historia nueva, no se agrega en silencio.

7. Definición de Terminado

Se aplica la DoD de `00-base/01-CONVENCIONES.md` §7.

8. Notas

El administrador semilla es una credencial pública

`V3_datos_semilla.sql` crea `admin@productionfood.local` con contraseña `Admin123*`, que está escrita en el repositorio. Resuelve el problema del huevo y la gallina —sin un usuario inicial nadie puede entrar a crear el primero— pero **es una puerta abierta conocida**.

Tarea de cierre obligatoria antes de exponer la aplicación fuera de `localhost`: crear un administrador real y desactivar el semilla (`estado = 0`).

Diseño

1. Pantallas

Formulario de registro (diálogo modal sobre el listado de HU-02)

Registrar usuario
[X]

Nombre completo *

María Gómez
12 / 100

Correo electrónico *

maria@productionfood.local

Ya existe un usuario con este correo

Contraseña *

.....

Fortaleza: Buena

Confirmar contraseña *

.....

Rol *

Ventas

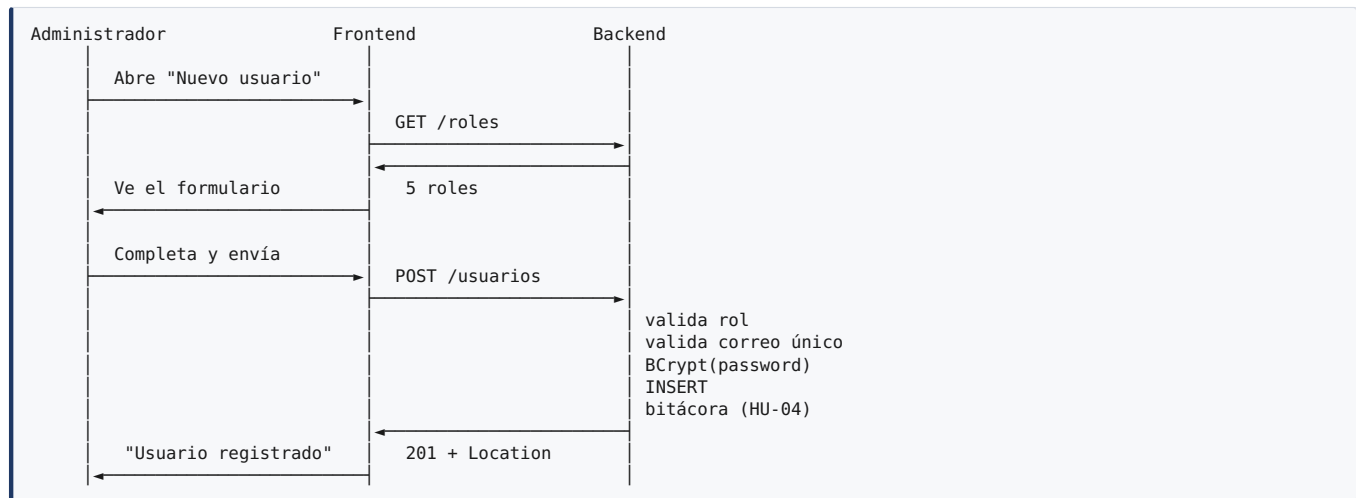
Atiende clientes, pedidos y devoluciones

[Cancelar]

[Registrar usuario]

El texto bajo el selector de rol muestra `roles.descripcion`. Sin él, quien registra tiene que adivinar qué implica cada rol — y asignar permisos a ciegas es cómo terminan todos los usuarios siendo administradores.

2. Flujo



Si el correo ya existe, el backend devuelve `409 CORREO_DUPLICADO` y el formulario marca el campo de correo sin perder lo ya escrito.

3. Componentes Angular Material

Elemento	Componente
Diálogo	<code>MatDialog</code>
Campos de texto	<code>MatFormField</code> + <code>MatInput</code>
Contraseña	<code>MatInput[type=password]</code> + <code>MatIconButton</code> para mostrar/ocultar
Fortaleza	<code>MatProgressBar</code> con color según nivel
Rol	<code>MatSelect</code> + <code>MatOption</code> (poblado desde la API)
Botones	<code>MatButton</code> (Cancelar) · <code>MatRaisedButton</code> color="primary" (Registrar)
Avisos	<code>MatSnackBar</code> vía <code>NotificacionService</code>

4. Validaciones en el formulario

Campo	Reglas de cliente	Mensaje
Nombre	requerido, máx. 100	"El nombre es obligatorio" / "Máximo 100 caracteres"
Correo	requerido, formato correo, máx. 100	"Ingrese un correo válido"
Contraseña	requerida, 8-72 caracteres	"Mínimo 8 caracteres"
Confirmar	requerida, igual a contraseña	"Las contraseñas no coinciden"
Rol	requerido	"Seleccione un rol"

La validación del cliente es para comodidad del usuario. **La autoridad es el backend**: el correo duplicado solo puede detectarse allí.

5. Estados de la pantalla

Estado	Qué se muestra
Cargando roles	Selector deshabilitado con spinner
Enviando	Botón deshabilitado, texto "Registrando..."
Error de validación	Mensaje bajo cada campo afectado
Error de servidor	<code>MatSnackBar</code> rojo, formulario conserva los datos
Éxito	Diálogo se cierra, <code>MatSnackBar</code> verde, listado se recarga

6. Accesibilidad y UX

- Todos los campos con `<mat-label>`; nunca solo *placeholder*.
- Navegación por teclado en orden lógico; `Enter` envía el formulario.
- Los mensajes de error del servidor se muestran **junto al campo** (`fieldErrors`).
- El botón de envío se deshabilita mientras la petición está en vuelo.
- Confirmación antes de descartar cambios sin guardar.
- Los tres estados obligatorios (cargando / vacío / error) según `00-base/05-ESTANDARES-QA.md` §7.

Tarea Backend

Primera historia del proyecto. Además del CRUD, aquí se montan piezas transversales que el resto reutiliza: `GlobalExceptionHandler`, `PageResponse` y el `PasswordEncoder`.

1. Pasos

Infraestructura transversal (se hace una vez, aquí)

1. `common/error/`: `ApiException`, `RecursoNoEncontradoException`, `ConflictoNegocioException`, `ErrorResponse`, `GlobalExceptionHandler` — formato de `04-CONTRATO-API.md` §5.
2. `common/pagination/`: `PageRequest` (con validación de `size` ≤ 100 y lista blanca de `sort`) y `PageResponse<T>`.
3. `SecurityConfig` con el bean `PasswordEncoder` (BCrypt) y `@EnableMethodSecurity`.

Módulo `rol`

4. `Rol` (POJO), `RolRowMapper`, `RolRepository.buscarPorId()` y `.listarTodos()`.
5. `RolController` con `GET /api/v1/roles`.

Módulo `usuario`

6. `Usuario` (POJO) y `UsuarioRowMapper`. **El mapper no expone `password` hacia el DTO.**
7. `UsuarioRepository`: `insertar()` con `GeneratedKeyHolder`, `existePorCorreo()`, `buscarPorId()`.
8. DTO: `CrearUsuarioRequest` (record con Bean Validation), `UsuarioResponse`, `RolResponse`.
9. `UsuarioService.crear()`:
 - normaliza el correo a minúsculas y recorta espacios;
 - valida que el rol exista $\rightarrow$ `ROL_INEXISTENTE`;
 - valida correo no duplicado $\rightarrow$ `CORREO_DUPLICADO`;

- codifica la contraseña con `passwordEncoder.encode()` ;
 - inserta y devuelve el `UsuarioResponse` .
10. `UserController.crear()` con `@PreAuthorize("hasRole('ADMIN')")` , devuelve `201` + `Location` .
 11. Capturar `DuplicateKeyException` en el servicio → `CORREO_DUPLICADO` (carrera).
 12. Documentar en Swagger con `@Operation` y `@ApiResponse` .

2. Archivos

```
backend/src/main/java/co/edu/corposucre/productionfood/
├── common/error/{ApiException, ErrorResponse, GlobalExceptionHandler, ...}.java
├── common/pagination/{PageRequest, PageResponse}.java
├── security/SecurityConfig.java
├── rol/{Rol, RolRepository, RolRowMapper, RolController}.java
├── usuario/
│   ├── {Usuario, UsuarioRepository, UsuarioRowMapper, UsuarioService, UsuarioController}.java
│   └── dto/{CrearUsuarioRequest, UsuarioResponse, RolResponse}.java
```

3. Puntos de cuidado

- `UsuarioResponse` **no lleva la contraseña**, ni siquiera el hash. Revisar el record antes del PR: es el error más fácil de cometer y el más difícil de detectar después.
- `@EnableMethodSecurity` en `SecurityConfig` . Sin ella todas las `@PreAuthorize` del proyecto se ignoran en silencio y la aplicación parece funcionar.
- Máximo 72 caracteres de contraseña: BCrypt trunca a partir de ahí (R-03).
- No confiar solo en `existePorCorreo()` : capturar también `DuplicateKeyException` .
- El DTO **no** recibe `estado` : lo pone el `DEFAULT 1` de la base (CA-05).

4. Checklist de cierre

- ☐ Todos los CA implementados y verificados manualmente.
- ☐ Endpoints documentados en Swagger con ejemplos.
- ☐ `@PreAuthorize` en cada método, según la matriz de roles.
- ☐ Operaciones de escritura auditadas en bitácora (HU-04).
- ☐ Pruebas unitarias de las reglas con ramificación.
- ☐ Sin `System.out.println` ni código comentado.
- ☐ PR bajo ~250 líneas, revisado y aprobado.
- ☐ **Tarea de cierre del proyecto:** crear administrador real y desactivar el semilla `admin@productionfood.local` .

Tarea Frontend

Formulario de alta de usuario. Es la primera pantalla del proyecto: aquí se establecen el `NotificacionService` , el manejo de `fieldErrors` y el layout base que reutilizan las demás.

1. Pasos

1. Estructura base: `core/` , `shared/` , `features/` según `01-CONVENCIONES.md` §4.
2. `shared/models/` : `PageResponse<T>` , `ApiError` .
3. `core/ui/notificacion.service.ts` — envoltorio de `MatSnackBar` (éxito / error / aviso).
4. `features/usuario/models/usuario.model.ts` : `Usuario` , `CrearUsuarioDto` , `Rol` .
5. `features/usuario/services/usuario.service.ts` : `crear()` , y `rol.service.ts` : `listar()` .
6. `usuario-formulario.component` : Reactive Form con los cuatro campos y confirmación de contraseña.
7. Selector de rol poblado desde `GET /api/v1/roles` (nunca una lista escrita a mano en el código: si se agrega un rol en la base, el formulario queda desactualizado).
8. Indicador de fuerza de contraseña y botón de mostrar/ocultar.
9. Mapear `fieldErrors` de la respuesta al control correspondiente.
10. Ruta con `rolGuard(['ADMIN'])` .

2. Archivos

```
frontend/src/app/
├── core/ui/notificacion.service.ts
├── shared/models/{page-response,api-error}.model.ts
├── features/usuario/
│   ├── models/usuario.model.ts
│   ├── services/{usuario,rol}.service.ts
│   ├── pages/usuario-formulario/
│   └── usuario.routes.ts
```

3. Puntos de cuidado

- **La confirmación de contraseña es solo del cliente:** no se envía al backend.
- El correo se envía en minúsculas y sin espacios, igual que lo normaliza el servidor.
- `maxLength(72)` en el campo de contraseña, coherente con R-03.
- Deshabilitar el botón mientras la petición está en vuelo: evita crear el usuario dos veces.
- El `409 CORREO_DUPLICADO` se muestra **junto al campo de correo**, no en un toast.

4. Checklist de cierre

- ☐ Todos los CA cubiertos desde la interfaz.
- ☐ Los tres estados (cargando / vacío / error) implementados.
- ☐ Errores del servidor mostrados junto al campo correspondiente.
- ☐ Guard de rol aplicado en la ruta.
- ☐ Modelos TypeScript espejo de los DTO del backend.
- ☐ Probado en Chrome y Firefox a 1366×768.
- ☐ Sin `console.log` ni `any`.

Casos de Prueba

Formato y criterios en `00-base/05-ESTANDARES-QA.md`.

1. Casos funcionales

ID	CA	Descripción	Entrada	Resultado esperado	Tipo
CP-01	CA-01, CA-06	Crear usuario válido	<code>{"nombre": "María Gómez", "correo": "maria@pf.local", "password": "Clave2026*", "idRol": 4}</code>	201 , cuerpo con <code>idUsuario</code> , <code>activo: true</code> , cabecera <code>Location</code>	Positivo
CP-02	CA-01	Nombre vacío	<code>{"nombre": "", ...}</code>	400 <code>VALIDACION_FALLIDA</code> , <code>fieldErrors</code> con <code>field: "nombre"</code>	Negativo
CP-03	CA-01	Nombre de 101 caracteres	<code>"A"×101</code>	400 , error en <code>nombre</code>	Borde
CP-04	CA-01	Nombre de exactamente 100	<code>"A"×100</code>	201	Borde
CP-05	CA-02	Correo duplicado	Correo de un usuario existente	409 <code>CORREO_DUPLICADO</code>	Negativo
CP-06	CA-02	Correo con formato inválido	<code>"maria.pf.local"</code>	400 , error en <code>correo</code>	Negativo
CP-07	CA-02	Correo duplicado con distinta capitalización	<code>"MARIA@pf.local"</code> existiendo <code>maria@pf.local</code>	409 <code>CORREO_DUPLICADO</code> (se normaliza a minúsculas)	Borde
CP-08	CA-03	La respuesta no debe contener la contraseña	Crear usuario válido	El JSON no incluye <code>password</code> ni el hash	Seguridad
CP-09	CA-03	Contraseña de 7 caracteres	<code>"Corta1*"</code>	400	Borde
CP-10	CA-03	La contraseña se guarda hasheada	Crear usuario y consultar la base	<code>password</code> empieza por <code>\$2a\$</code> y no es el texto plano	Integridad
CP-11	CA-04	Rol inexistente	<code>{"idRol": 999, ...}</code>	409 <code>ROL_INEXISTENTE</code>	Negativo
CP-12	CA-05	Estado por defecto	Crear sin enviar <code>estado</code>	<code>activo: true</code> ; en base <code>estado = 1</code>	Positivo
CP-13	—	Inyección SQL en el nombre	<code>"'; DROP TABLE usuarios; --"</code>	Se guarda como texto literal; la tabla sigue existiendo	Seguridad
CP-14	—	Crear usuario como rol VENTAS	Token de VENTAS	403 <code>SIN_PERMISO</code>	Seguridad
CP-15	—	Listar roles	<code>GET /api/v1/roles</code> con token ADMIN	200 con los 5 roles de la semilla	Positivo

2. Batería de seguridad

Obligatoria en **cada endpoint** de esta historia:

ID	Caso	Esperado
SEC-01	Sin cabecera <code>Authorization</code>	401 <code>NO_AUTENTICADO</code>
SEC-02	Token malformado	401 <code>NO_AUTENTICADO</code>
SEC-03	Token expirado	401 <code>NO_AUTENTICADO</code>
SEC-04	Token de rol sin permiso	403 <code>SIN_PERMISO</code>
SEC-05	Token de usuario con <code>estado = 0</code>	403 <code>USUARIO_INACTIVO</code>

SEC-04 es el caso crítico. Si devuelve 200 , falta `@EnableMethodSecurity` y la autorización del sistema completo está desactivada.

CP-08 merece atención especial. Es el defecto más común en este tipo de endpoint: el DTO de respuesta se construye copiando la entidad completa y arrastra el hash. No produce ningún error visible, y el hash termina en los logs del navegador y en las capturas de los reportes.

3. Verificación en base de datos

```
-- CP-10: la contraseña debe estar hasheada, nunca en texto plano
SELECT correo, password, LENGTH(password) AS largo, estado
FROM usuarios WHERE correo = 'maria@pf.local';
-- Se espera: password empieza por '$2a$', largo = 60, estado = 1

-- CP-12: el DEFAULT 1 de V2 debe estar aplicado
SHOW CREATE TABLE usuarios\G
-- Se espera: `estado` tinyint(1) NOT NULL DEFAULT '1'

-- CP-13: la tabla sigue intacta tras el intento de inyección
SELECT COUNT(*) FROM usuarios;
```

4. Criterios de cierre

- ☐ Todos los casos ejecutados; los positivos pasan.
- ☐ Los negativos devuelven el HTTP y el `code` esperados.
- ☐ SEC-01 a SEC-05 pasan en todos los endpoints.
- ☐ Ningún endpoint supera 2 s (RNF-02).
- ☐ Cero defectos críticos o altos abiertos.
- ☐ Colección de Postman actualizada y versionada.

HU-02 · Gestión de usuarios

Especificación

Fase 1 · Fuente: `previo/Historias_de_Usuario.md`

Como administrador, **quiero** listar, editar y desactivar usuarios, **para que** gestione quién tiene acceso al sistema.

Depende de: HU-01 (registro de usuarios)

1. Criterios de aceptación

CA	Enunciado	Cómo se implementa
CA-01	Listar todos los usuarios con paginación	<code>GET /usuarios</code> con <code>page</code> , <code>size</code> , <code>sort</code> — <code>04-CONTRATO-API.md</code> §4
CA-02	Filtrar por nombre o correo	Parámetro <code>busqueda</code> que aplica <code>LIKE prefijo%</code> sobre ambas columnas
CA-03	Editar nombre, correo, rol y estado	<code>PUT /usuarios/{id}</code> — la contraseña no se edita aquí
CA-04	Desactivar/activar cambiando <code>estado</code>	<code>PATCH /usuarios/{id}/estado</code>
CA-05	No se permite eliminar , solo desactivar	No existe endpoint <code>DELETE</code>

2. Endpoints

Método	Ruta	Descripción	Roles
<code>GET</code>	<code>/api/v1/usuarios</code>	Listar con filtro y paginación	ADMIN
<code>GET</code>	<code>/api/v1/usuarios/{id}</code>	Obtener un usuario	ADMIN
<code>PUT</code>	<code>/api/v1/usuarios/{id}</code>	Editar nombre, correo y rol	ADMIN
<code>PATCH</code>	<code>/api/v1/usuarios/{id}/estado</code>	Activar o desactivar	ADMIN

Reglas transversales (paginación, formato de error, autenticación) en `00-base/04-CONTRATO-API.md`.

3. Reglas de negocio

R-01 · Nunca se elimina un usuario (CA-05)

No hay endpoint `DELETE`, y no es solo una decisión de interfaz: `usuarios` está referenciada por `planes_produccion.id_usuario`, `bitacora.id_usuario` y las dos tablas de kardex. Borrar un usuario rompería la trazabilidad de todo lo que hizo — que es precisamente lo que la bitácora de HU-04 existe para conservar.

R-02 · Un administrador no puede desactivarse a sí mismo

Si el único administrador activo se desactiva, **nadie puede volver a entrar a reactivarlo**. El sistema queda inutilizable y solo se recupera con acceso directo a la base de datos.

Dos comprobaciones en el servicio: - El `idUsuario` del token no puede ser el mismo que se desactiva → `409 AUTO_DESACTIVACION`. - No se puede desactivar al último `ADMIN` activo → `409 ULTIMO_ADMIN`.

R-03 · La contraseña no se edita en este endpoint

`PUT /usuarios/{id}` no acepta `password`. Cambiar contraseña es una operación distinta, con reglas propias (contraseña actual, política de complejidad), y mezclarla con la edición de datos hace que un `PUT` parcial la sobrescriba sin querer. Queda fuera de alcance.

R-04 · El correo sigue siendo único al editar

La verificación de unicidad **excluye al propio usuario**: `SELECT ... WHERE correo = ? AND id_usuario <> ?`. Sin esa exclusión, guardar un usuario sin cambiarle el correo devuelve `CORREO_DUPLICADO` contra sí mismo.

R-05 · Efecto inmediato de la desactivación

El token ya emitido **sigue siendo criptográficamente válido hasta que expira** (máx. 1 hora). Por eso la validación de `estado` se hace **en cada petición**, no solo al iniciar sesión: un usuario desactivado recibe `403 USUARIO_INACTIVO` en su siguiente llamada, no una hora después.

4. Modelo de datos

Misma tabla `usuarios` de HU-01. Sin cambios de esquema.

```
// GET /api/v1/usuarios?busqueda=mar&page=0&size=20&sort=nombre,asc
{
  "content": [
    { "idUsuario": 12, "nombre": "María Gómez", "correo": "maria@productionfood.local",
      "activo": true, "rol": { "idRol": 4, "nombre": "VENTAS" } }
  ],
  "page": 0, "size": 20, "totalElements": 1, "totalPages": 1,
  "first": true, "last": true
}

// PUT /api/v1/usuarios/12
{ "nombre": "María Gómez Ruiz", "correo": "maria.gomez@productionfood.local", "idRol": 4 }

// PATCH /api/v1/usuarios/12/estado
{ "activo": false }
```

Campos ordenables (lista blanca): `nombre`, `correo`, `id`. Cualquier otro valor en `sort` → `400 CAMPO_ORDEN_INVALIDO`.

5. Códigos de error

code	HTTP	Cuándo
AUTO_DESACTIVACION	409	Un administrador intenta desactivarse a sí mismo
ULTIMO_ADMIN	409	Se intenta desactivar al último administrador activo
CORREO_DUPLICADO	409	El correo ya pertenece a otro usuario
ROL_INEXISTENTE	409	El <code>idRol</code> no existe

Los transversales (`VALIDACION_FALLIDA`, `NO_AUTENTICADO`, `SIN_PERMISO`, `RECURSO_NO_ENCONTRADO`) están en `00-base/04-CONTRATO-API.md` §5.

6. Fuera de alcance

Lo que **no** cubre esta historia y no debe implementarse aquí: se limita estrictamente a los criterios de aceptación listados arriba. Cualquier funcionalidad adicional se propone como historia nueva, no se agrega en silencio.

7. Definición de Terminado

Se aplica la DoD de `00-base/01-CONVENCIONES.md` §7.

8. Notas

Sobre la búsqueda por prefijo

CA-02 pide filtrar por nombre o correo. La implementación usa `WHERE (nombre LIKE :q OR correo LIKE :q)` con patrón `texto%`.

Buscar “gomez” **no** encontrará a “María Gómez” — el apellido no está al inicio del campo `nombre`. Es el compromiso consciente de `04-CONTRATO-API.md` §4: `%texto%` recorrería la tabla completa en cada pulsación de tecla. Para el volumen de usuarios de una pyme la diferencia es irrelevante, pero la convención se mantiene uniforme en todo el sistema.

Si en pruebas resulta incómodo, la alternativa correcta es un índice `FULLTEXT`, no cambiar a `%texto%`.

Diseño

1. Pantallas

Listado de usuarios

Usuarios [+ Nuevo usuario]			
Buscar nombre/correo	Rol Todos	Estado Todos	
NOMBRE ▲	CORREO	ROL	ESTADO ...
Administrador	admin@production...	ADMIN	●Activo :
Carlos Cochero	carlos@production...	COMPRAS	●Activo :
Ismael Batalla	ismael@production...	PRODUCCION	○Inactivo :
María Gómez	maria@production...	VENTAS	●Activo :
Elementos por página: 20 ▼ 1-4 de 4 ◀ ▶			

Menú  de cada fila: **Editar** · **Desactivar** (o **Activar**). **No hay opción de eliminar** (CA-05).

Confirmación de desactivación

Desactivar usuario

¿Desactivar a María Gómez?

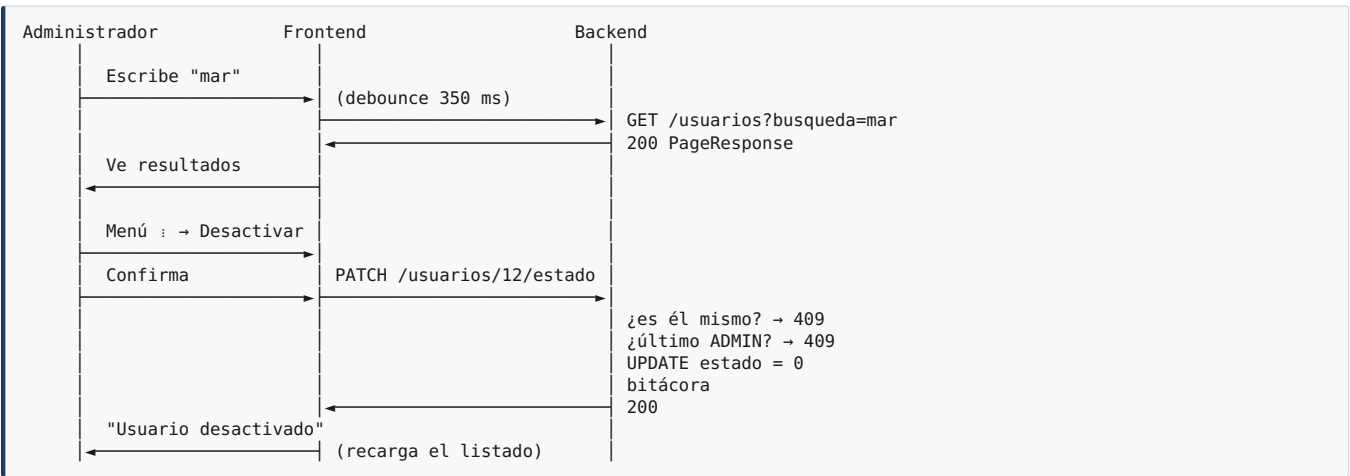
No podrá iniciar sesión ni continuar su sesión actual. Sus registros históricos se conservan.

[Cancelar]

[Desactivar]

El texto explica la consecuencia real. “¿Está seguro?” no informa nada: quien lo lee ya pulsó el botón a propósito.

2. Flujo



3. Componentes Angular Material

Elemento	Componente
Tabla	MatTable + MatSort + MatPaginator
Búsqueda	MatFormField + MatInput + MatIcon (lupa)
Filtros	MatSelect (rol, estado)
Estado	MatChip — verde activo / gris inactivo
Menú de fila	MatMenu + MatIconButton
Confirmación	MatDialog (ConfirmarDialogComponent)
Carga	MatProgressBar

4. Validaciones en el formulario

El formulario de edición reutiliza el de HU-01 **sin los campos de contraseña** (R-03).

Campo	Reglas
Nombre	requerido, máx. 100
Correo	requerido, formato correo, máx. 100
Rol	requerido

El campo de búsqueda no valida: cualquier texto es una búsqueda legítima, incluidos los caracteres especiales (que el backend escapa).

5. Estados de la pantalla

Estado	Qué se muestra
Cargando	<code>MatProgressBar</code> sobre la tabla; filtros activos
Vacío sin filtro	“No hay usuarios registrados” + botón “Registrar el primero”
Vacío con filtro	“Ningún usuario coincide con «mar»” + “Limpiar filtros”
Error	Mensaje + botón “Reintentar”
Con datos	Tabla + paginador

Distinguir el vacío por filtro del vacío real evita que el usuario crea que se perdieron los datos.

6. Accesibilidad y UX

- Todos los campos con `<mat-label>`; nunca solo *placeholder*.
- Navegación por teclado en orden lógico; `Enter` envía el formulario.
- Los mensajes de error del servidor se muestran **junto al campo** (`fieldErrors`).
- El botón de envío se deshabilita mientras la petición está en vuelo.
- Confirmación antes de descartar cambios sin guardar.
- Los tres estados obligatorios (cargando / vacío / error) según `00-base/05-ESTANDARES-QA.md` §7.

Tarea Backend

Amplía el módulo `usuario` de HU-01 con listado paginado, edición y cambio de estado. Aquí se estrena el patrón de listado con filtro que replicarán todos los CRUD posteriores.

1. Pasos

1. `UsuarioRepository.buscar(búsqueda, idRol, activo, PageRequest)` :
 - `WHERE 1=1` + condiciones opcionales;
 - `(nombre LIKE :q OR correo LIKE :q)` con `escaparLike(q) + "%"`;
 - `ORDER BY` desde la **lista blanca** `{nombre, correo, id}`;
 - `LIMIT :limit OFFSET :offset`.
2. `UsuarioRepository.contar(...)` — mismo **WHERE** que `buscar()`. Si divergen, `totalElements` miente y la última página aparece vacía.
3. `UsuarioRepository.actualizar()` y `.cambiarEstado()`.
4. `UsuarioRepository.existePorCorreoExcluyendo(correo, idUsuario)` — R-04.
5. `UsuarioRepository.contarAdminsActivos()` — R-02.
6. DTO: `ActualizarUsuarioRequest`, `CambiarEstadoRequest`.
7. `UsuarioService.listar()` → arma el `PageResponse<T>`.
8. `UsuarioService.actualizar()` : valida rol, valida correo único excluyendo al propio.
9. `UsuarioService.cambiarEstado()` :
 - `id == SecurityUtils.idUsuarioActual()` → `AUTO_DESACTIVACION`;
 - desactivando un `ADMIN` y `contarAdminsActivos() <= 1` → `ULTIMO_ADMIN`.
10. Controlador con `@PreAuthorize("hasRole('ADMIN')")` en los cuatro métodos.
11. Auditar `EDITAR` y `CAMBIAR_ESTADO` en bitácora (HU-04).
12. **Verificar que el filtro `estado` se consulta en cada petición** (R-05): el `JwtAuthenticationFilter` o el `UserDetailsService` debe rechazar usuarios inactivos.

2. Archivos

```

backend/src/main/java/co/edu/corposucre/productionfood/usuario/
├── UsuarioRepository.java      (+ buscar, contar, actualizar, cambiarEstado)
├── UsuarioService.java        (+ listar, actualizar, cambiarEstado)
├── UsuarioController.java     (+ GET, GET/{id}, PUT, PATCH)
└── dto/{ActualizarUsuarioRequest,CambiarEstadoRequest}.java

```

3. Puntos de cuidado

- `buscar()` y `contar()` deben compartir exactamente el mismo `WHERE`. Es el error más común de la paginación manual.
- `sort` nunca se concatena crudo. Lista blanca obligatoria.
- Escapar `%` y `_` en el término de búsqueda: sin eso, buscar `%` devuelve todo.
- R-02 no es hipotético: desactivar al último administrador deja el sistema sin acceso y solo se arregla con un `UPDATE` directo en la base.
- `PUT` no acepta `password` (R-03).
- El `LIMIT/OFFSET` va parametrizado, no interpolado.

4. Checklist de cierre

- ☐ Todos los CA implementados y verificados manualmente.
- ☐ Endpoints documentados en Swagger con ejemplos.
- ☐ `@PreAuthorize` en cada método, según la matriz de roles.
- ☐ Operaciones de escritura auditadas en bitácora (HU-04).
- ☐ Pruebas unitarias de las reglas con ramificación.
- ☐ Sin `System.out.println` ni código comentado.
- ☐ PR bajo ~250 líneas, revisado y aprobado.

Tarea Frontend

Listado paginado con filtro, edición y activación/desactivación. **Es la pantalla plantilla de todos los listados del proyecto:** se implementa con cuidado porque los demás CRUD la copian.

1. Pasos

1. `shared/components/tabla-paginada/` — tabla genérica reutilizable con `MatTable`, `MatPaginator` y `MatSort`, parametrizada por columnas.
2. `shared/components/confirmar-dialog/` — diálogo de confirmación reutilizable.
3. `usuario-lista.component`:
 - columnas: nombre, correo, rol, estado, acciones;
 - campo de búsqueda con `debounceTime(350)` — sin él se lanza una petición por tecla;
 - filtros por rol y por estado;
 - paginación conectada a `PageResponse`.
4. Chip de estado con color: verde “Activo” / gris “Inactivo”.
5. Acción de editar: abre el diálogo de HU-01 en modo edición (sin campos de contraseña).
6. Acción de activar/desactivar con confirmación explícita: “¿Desactivar a María Gómez? No podrá iniciar sesión.”
7. **No incluir acción de eliminar** (CA-05): que no exista el botón.
8. Manejar `409 AUTO_DESACTIVACION` y `409 ULTIMO_ADMIN` con el mensaje del servidor.
9. Los tres estados: cargando / vacío / error.

2. Archivos

```

frontend/src/app/
├── shared/components/tabla-paginada/
├── shared/components/confirmar-dialog/
└── features/usuario/pages/usuario-lista/

```

3. Puntos de cuidado

- El `debounceTime` no es opcional. Sin él, escribir “maria” lanza cinco peticiones y las respuestas pueden llegar desordenadas, mostrando resultados de una búsqueda anterior. Usar también `switchMap` para cancelar la petición en vuelo.
- Al desactivarse a sí mismo, el servidor responde `409`: mostrar el mensaje, no cerrar sesión.

- El `PageResponse` usa `page` base 0; `MatPaginator` también. No sumar ni restar 1.
- Sin botón de eliminar: si aparece en la interfaz, alguien lo va a pedir.

4. Checklist de cierre

- ☐ Todos los CA cubiertos desde la interfaz.
- ☐ Los tres estados (cargando / vacío / error) implementados.
- ☐ Errores del servidor mostrados junto al campo correspondiente.
- ☐ Guard de rol aplicado en la ruta.
- ☐ Modelos TypeScript espejo de los DTO del backend.
- ☐ Probado en Chrome y Firefox a 1366×768.
- ☐ Sin `console.log` ni `any`.

Casos de Prueba

Formato y criterios en `00-base/05-ESTANDARES-QA.md`.

1. Casos funcionales

ID	CA	Descripción	Entrada	Resultado esperado	Tipo
CP-01	CA-01	Listar con paginación	<code>GET /usuarios?page=0&size=5</code>	<code>200</code> , máx. 5 elementos, <code>totalElements</code> correcto	Positivo
CP-02	CA-01	Segunda página	<code>?page=1&size=5</code>	Elementos distintos a los de la página 0	Positivo
CP-03	CA-01	<code>size</code> fuera de rango	<code>?size=500</code>	<code>400 PARAMETRO_INVALIDO</code> (tope 100)	Borde
CP-04	CA-01	Campo de orden no permitido	<code>?sort=password,asc</code>	<code>400 CAMPO_ORDEN_INVALIDO</code>	Seguridad
CP-05	CA-02	Filtrar por nombre	<code>?busqueda=mar</code>	Solo usuarios cuyo nombre o correo empieza por "mar"	Positivo
CP-06	CA-02	Filtrar por correo	<code>?busqueda=admin@</code>	El administrador semilla	Positivo
CP-07	CA-02	Búsqueda sin resultados	<code>?busqueda=zzzzz</code>	<code>200</code> , <code>content: []</code> , <code>totalElements: 0</code>	Borde
CP-08	CA-02	Comodín <code>%</code> en la búsqueda	<code>?busqueda=%</code>	No devuelve todos: se trata como texto literal	Seguridad
CP-09	CA-03	Editar nombre y rol	<code>PUT</code> con datos válidos	<code>200</code> , cambios reflejados	Positivo
CP-10	CA-03	Guardar sin cambiar el correo	<code>PUT</code> con el correo actual	<code>200</code> — no debe dar <code>CORREO_DUPLICADO</code> contra sí mismo	Borde
CP-11	CA-03	Editar con correo de otro usuario	<code>PUT</code> con correo ajeno	<code>409 CORREO_DUPLICADO</code>	Negativo
CP-12	CA-03	Enviar <code>password</code> en el <code>PUT</code>	<code>{"password": "nueva"}</code>	Se ignora; la contraseña no cambia	Seguridad
CP-13	CA-04	Desactivar un usuario	<code>PATCH ../estado {"activo": false}</code>	<code>200</code> , <code>estado = 0</code> en base	Positivo
CP-14	CA-04	Usuario desactivado intenta usar su token vigente	Token emitido antes de desactivar	<code>403 USUARIO_INACTIVO inmediatamente</code> , no al expirar	Seguridad
CP-15	CA-04	Administrador se desactiva a sí mismo	<code>PATCH</code> sobre el propio id	<code>409 AUTO_DESACTIVACION</code>	Negativo
CP-16	CA-04	Desactivar al último administrador activo	Dejar un solo ADMIN y desactivarlo	<code>409 ULTIMO_ADMIN</code>	Negativo
CP-17	CA-04	Reactivar un usuario	<code>{"activo": true}</code>	<code>200</code> ; el usuario puede iniciar sesión	Positivo
CP-18	CA-05	No debe existir DELETE	<code>DELETE /usuarios/12</code>	<code>405 Method Not Allowed</code> o <code>404</code>	Negativo

2. Batería de seguridad

Obligatoria en **cada endpoint** de esta historia:

ID	Caso	Esperado
SEC-01	Sin cabecera <code>Authorization</code>	<code>401 NO_AUTENTICADO</code>
SEC-02	Token malformado	<code>401 NO_AUTENTICADO</code>
SEC-03	Token expirado	<code>401 NO_AUTENTICADO</code>
SEC-04	Token de rol <code>sin</code> permiso	<code>403 SIN_PERMISO</code>
SEC-05	Token de usuario con <code>estado = 0</code>	<code>403 USUARIO_INACTIVO</code>

SEC-04 es el caso crítico. Si devuelve `200`, falta `@EnableMethodSecurity` y la autorización del sistema completo está desactivada.

CP-14 y CP-16 son los casos que importan. CP-14 detecta si la validación de `estado` se hace solo al iniciar sesión: en ese caso un usuario despedido conserva acceso hasta una hora después de ser desactivado. CP-16 protege contra dejar el sistema sin ningún administrador, situación que solo se resuelve con acceso directo a la base de datos.

3. Verificación en base de datos

```
-- CP-13: la desactivación es lógica, el registro permanece
SELECT id_usuario, correo, estado FROM usuarios WHERE id_usuario = 12;
-- Se espera: estado = 0, la fila existe

-- CP-16: verificar cuántos administradores activos quedan
SELECT COUNT(*) FROM usuarios u JOIN roles r ON r.id_rol = u.id_rol
WHERE r.nombre = 'ADMIN' AND u.estado = 1;
-- Nunca debe poder llegar a 0 por esta vía

-- CP-18: ningún usuario se borra jamás
SELECT COUNT(*) FROM usuarios; -- comparar antes y después de las pruebas
```

4. Criterios de cierre

- ☐ Todos los casos ejecutados; los positivos pasan.
- ☐ Los negativos devuelven el HTTP y el `code` esperados.
- ☐ SEC-01 a SEC-05 pasan en todos los endpoints.
- ☐ Ningún endpoint supera 2 s (RNF-02).
- ☐ Cero defectos críticos o altos abiertos.
- ☐ Colección de Postman actualizada y versionada.

HU-03 · Inicio de sesión

Especificación

Fase 1 · Fuente: `previo/Historias_de_Usuario.md`

Como usuario, **quiero** iniciar sesión con correo y contraseña, **para que** acceda a las funcionalidades según mi rol.

Depende de: HU-01 (debe existir al menos un usuario)

1. Criterios de aceptación

CA	Enunciado	Cómo se implementa
CA-01	Se recibe correo y contraseña	<code>POST /auth/login</code> con <code>LoginRequest</code>
CA-02	Credenciales correctas → token JWT	<code>200</code> con <code>token</code> , <code>tipo</code> y datos del usuario
CA-03	Usuario desactivado → <code>403</code>	Se verifica <code>estado = 1</code> después de validar la contraseña
CA-04	Credenciales incorrectas → <code>401</code>	Mismo mensaje para correo inexistente y contraseña errónea
CA-05	El token contiene <code>id_usuario</code> , <code>correo</code> y <code>rol</code>	Claims <code>sub</code> , <code>correo</code> , <code>nombre</code> , <code>rol</code> — <code>04-CONTRATO-API.md</code> §2

2. Endpoints

Método	Ruta	Descripción	Roles
<code>POST</code>	<code>/api/v1/auth/login</code>	Autenticar y obtener token	Público
<code>GET</code>	<code>/api/v1/auth/perfil</code>	Datos del usuario autenticado	Autenticado

Reglas transversales (paginación, formato de error, autenticación) en `00-base/04-CONTRATO-API.md`.

3. Reglas de negocio

R-01 · Mensaje idéntico para correo inexistente y contraseña incorrecta

Ambos casos devuelven exactamente `401` con `"Correo o contraseña incorrectos."`

Si el sistema respondiera “el correo no existe” en un caso y “contraseña incorrecta” en el otro, cualquiera podría **enumerar los correos registrados** probando direcciones: el mensaje revela si la cuenta existe. Es una fuga de información que además facilita el ataque dirigido por fuerza bruta, porque permite descartar correos antes de empezar.

R-02 · El orden de las comprobaciones importa

- Buscar el usuario por correo.
- Verificar la contraseña.**
- Solo entonces, verificar `estado = 1` → `403 USUARIO_INACTIVO`.

Invertir 2 y 3 convierte el `403` en un oráculo: quien reciba “usuario inactivo” sabe que el correo existe, sin conocer la contraseña. Se responde `401` genérico hasta haber probado que quien pregunta conoce la credencial.

R-03 · Comparación en tiempo constante

La verificación se hace con `passwordEncoder.matches()`, que compara en tiempo constante. Comparar hashes con `equals()` permitiría deducir información por el tiempo de respuesta.

Cuando el correo no existe, se ejecuta igualmente un `matches()` contra un hash ficticio para que ambos caminos tarden lo mismo. Sin eso, la diferencia de tiempo entre “correo inexistente” (respuesta inmediata) y “contraseña incorrecta” (BCrypt tarda ~100 ms) delata qué correos están registrados, aunque el mensaje sea idéntico.

R-04 · Vigencia del token

1 hora (`app.jwt.expiration-ms=3600000`). **No hay refresh token** en este alcance: al expirar se vuelve a iniciar sesión.

R-05 · El estado se verifica en cada petición

El token es válido criptográficamente hasta que expira, incluso si el usuario se desactiva después. Por eso `JwtAuthenticationFilter` comprueba `estado` **en cada llamada**, no solo al autenticar (ver HU-02 R-05).

4. Modelo de datos

No requiere cambios de esquema. Consulta `usuarios` con `JOIN roles`.

```
// POST /api/v1/auth/login
{ "correo": "maria@productionfood.local", "password": "Clave2026*" }

// 200 OK
{
  "token": "eyJhbGciOiJIUzI1NiJ9...",
  "tipo": "Bearer",
  "expiraEn": 3600,
  "usuario": {
    "idUsuario": 12, "nombre": "María Gómez",
    "correo": "maria@productionfood.local",
    "rol": "VENTAS"
  }
}

// 401 Unauthorized
{ "code": "CREDENCIALES_INVALIDAS", "message": "Correo o contraseña incorrectos.", ... }

// 403 Forbidden
{ "code": "USUARIO_INACTIVO",
  "message": "Su usuario está desactivado. Contacte al administrador.", ... }
```

Claims del token (CA-05):

```
{ "sub": "12", "correo": "maria@productionfood.local",
  "nombre": "María Gómez", "rol": "VENTAS", "iat": ..., "exp": ... }
```

5. Códigos de error

code	HTTP	Cuándo
CREDENCIALES_INVALIDAS	401	Correo inexistente o contraseña incorrecta (mismo mensaje)
USUARIO_INACTIVO	403	Credenciales correctas pero <code>estado = 0</code>

Los transversales (`VALIDACION_FALLIDA` , `NO_AUTENTICADO` , `SIN_PERMISO` , `RECURSO_NO_ENCONTRADO`) están en `00-base/04-CONTRATO-API.md` §5.

6. Fuera de alcance

Lo que **no** cubre esta historia y no debe implementarse aquí: se limita estrictamente a los criterios de aceptación listados arriba. Cualquier funcionalidad adicional se propone como historia nueva, no se agrega en silencio.

7. Definición de Terminado

Se aplica la DoD de `00-base/01-CONVENCIONES.md` §7.

8. Notas

Lo que esta historia deja fuera, y conviene saberlo

No se implementa, por estar fuera del alcance acordado:

- **Bloqueo tras N intentos fallidos.** Sin él, un atacante puede probar contraseñas indefinidamente. BCrypt lo hace lento (unos 100 ms por intento), lo que limita el ritmo, pero no lo impide.
- **Refresh token.** La sesión caduca a la hora sin previo aviso.
- **Recuperación de contraseña.** Un usuario que olvide la suya necesita que un administrador se la cambie — y HU-02 R-03 tampoco implementa el cambio de contraseña. **Es un hueco real del backlog**, no de esta historia: conviene proponerlo como historia nueva.

Se registran aquí para que sean decisiones conscientes y no descubrimientos durante la sustentación.

Diseño

1. Pantallas

Pantalla de acceso

ProductionFood
Gestión de producción de alimentos

Correo electrónico
maria@productionfood.local

Contraseña
.....

△ Correo o contraseña incorrectos

Iniciar sesión

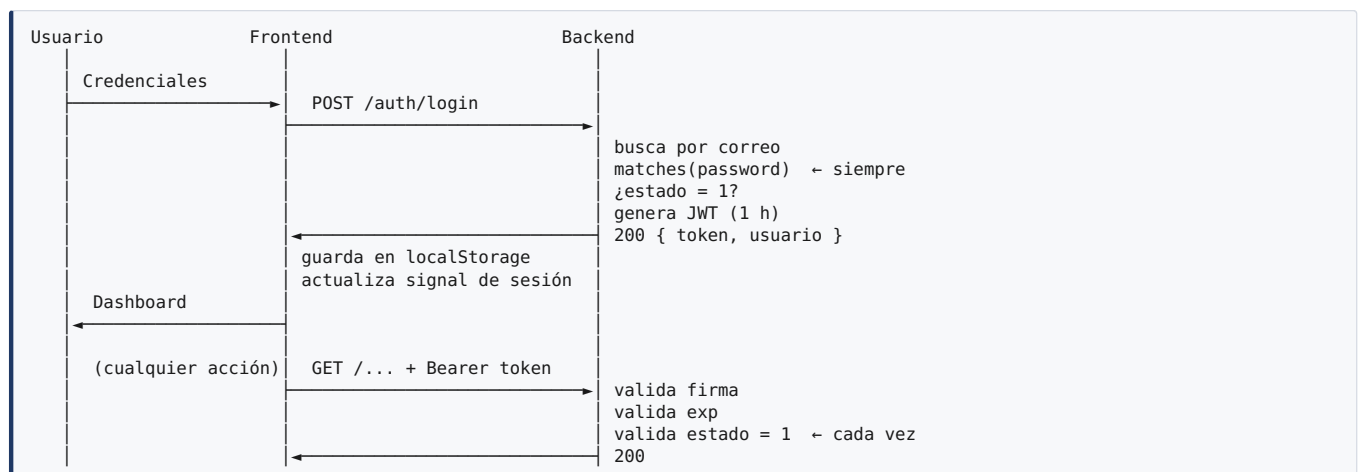
El mensaje de error es siempre el mismo, exista o no el correo (R-01).

Barra superior tras iniciar sesión

☰ ProductionFood María Gómez · VENTAS [🚪]

Mostrar el rol activo evita la confusión de “¿por qué no veo el módulo de compras?”.

2. Flujo



Al expirar el token: cualquier petición devuelve **401**; el interceptor cierra la sesión y redirige al login con el aviso “Su sesión expiró”.

3. Componentes Angular Material

Elemento	Componente
Tarjeta de login	<code>MatCard</code> centrada
Campos	<code>MatFormField</code> + <code>MatInput</code>
Ver contraseña	<code>MatIconButton</code> con <code>visibility</code> / <code>visibility_off</code>
Botón	<code>MatRaisedButton</code> <code>color="primary"</code> , ancho completo
Error	<code>MatError</code> sobre el formulario, no un toast
Cargando	<code>MatProgressSpinner</code> dentro del botón
Barra superior	<code>MatToolbar</code> + <code>MatMenu</code> de usuario
Menú lateral	<code>MatSidenav</code> + <code>MatNavList</code> filtrado por rol

4. Validaciones en el formulario

Campo	Reglas de cliente	Mensaje
Correo	requerido, formato correo	"Ingrese un correo válido"
Contraseña	requerida	"Ingrese su contraseña"

El cliente no valida la longitud de la contraseña en el login. Hacerlo revelaría la política de contraseñas a quien intenta adivinar, y además impediría entrar a usuarios creados antes de un cambio de política.

5. Estados de la pantalla

Estado	Qué se muestra
Inicial	Formulario vacío, foco en el correo
Enviando	Botón con spinner, campos deshabilitados
Credenciales incorrectas	<code>MatError</code> genérico; la contraseña se limpia, el correo se conserva
Usuario inactivo	"Su usuario está desactivado. Contacte al administrador."
Sin conexión	"No se pudo conectar con el servidor."
Éxito	Redirección según rol

Conservar el correo y limpiar solo la contraseña es lo que espera quien se equivocó al teclear: no lo obliga a reescribir todo.

6. Accesibilidad y UX

- Todos los campos con `<mat-label>` ; nunca solo *placeholder*.
- Navegación por teclado en orden lógico; `Enter` envía el formulario.
- Los mensajes de error del servidor se muestran **junto al campo** (`fieldErrors`).
- El botón de envío se deshabilita mientras la petición está en vuelo.
- Confirmación antes de descartar cambios sin guardar.
- Los tres estados obligatorios (cargando / vacío / error) según `00-base/05-ESTANDARES-QA.md` §7.

Tarea Backend

El corazón de la seguridad del sistema. **Bloquea todas las historias posteriores:** sin login no hay forma de probar ningún endpoint protegido. La configuración completa para Spring Security 7 está en `00-base/06-ARQUITECTURA-BACKEND.md` §2 — los tutoriales de Spring Boot 3.x que se encuentran en línea no compilan aquí.

1. Pasos

1. `JwtService` : `generar(Usuario)` y `validarYExtraer(token)` con jjwt 0.13 (API fluida nueva: `subject()` , `parser()` , `signWith(key)`).
2. `SecurityConfig` completo: `csrf` deshabilitado, `STATELESS` , `permitAll` en `/auth/login` y Swagger, `@EnableMethodSecurity` , `PasswordEncoder` .
3. `JwtAuthenticationFilter` `extends OncePerRequestFilter` :
 - extrae `Bearer` , valida, construye la autenticación con authority `ROLE_<rol>` ;
 - **consulta estado del usuario** (R-05) → si es 0, no autentica.
4. `UsuarioAutenticado` (record) como *principal*: `idUsuario` , `correo` , `rol` .
5. `SecurityUtils.idUsuarioActual()` — lo usan HU-04 y el kardex.
6. `AuthenticationEntryPoint` y `AccessDeniedHandler` que devuelven el `ErrorResponse` del contrato. Sin ellos, Spring responde HTML o un JSON con otro formato y el frontend no puede leer `code` .

7. `AuthService.login()` respetando el orden de R-02 y el `matches()` ficticio de R-03.
8. `AuthController`: `POST /login` (público) y `GET /perfil`.
9. Registrar en bitácora los intentos fallidos con `id_usuario = NULL` (posible gracias a §C-11).
10. Configurar `app.jwt.secret` desde variable de entorno.

2. Archivos

```
backend/src/main/java/co/edu/corposucre/productionfood/
├── security/
│   ├── SecurityConfig.java
│   ├── JwtService.java
│   ├── JwtAuthenticationFilter.java
│   ├── UsuarioAutenticado.java
│   ├── SecurityUtils.java
│   ├── ApiAuthenticationEntryPoint.java
│   └── ApiAccessDeniedHandler.java
├── auth/
│   ├── {AuthController,AuthService}.java
│   └── dto/{LoginRequest,LoginResponse,PerfilResponse}.java
```

3. Puntos de cuidado

- **JWT_SECRET nunca se versiona.** Quien lo tenga puede firmar tokens con el rol que quiera. Generar con `openssl rand -base64 48`; mínimo 256 bits o `jjwt` lanza `WeakKeyException`.
- `@EnableMethodSecurity`: sin ella, todas las `@PreAuthorize` del proyecto se ignoran y la aplicación parece funcionar perfectamente.
- **Prefijo `ROLE_`**: `hasRole('ADMIN')` busca la authority `ROLE_ADMIN`. Es la causa más común de un `403` inexplicable.
- **Mismo mensaje para correo inexistente y contraseña errónea** (R-01), y el `matches()` ficticio para igualar tiempos (R-03).
- El orden de R-02 no es un detalle de estilo: invertirlo filtra qué correos existen.
- **La respuesta de login no incluye el hash** de la contraseña.

4. Checklist de cierre

- ☐ Todos los CA implementados y verificados manualmente.
- ☐ Endpoints documentados en Swagger con ejemplos.
- ☐ `@PreAuthorize` en cada método, según la matriz de roles.
- ☐ Operaciones de escritura auditadas en bitácora (HU-04).
- ☐ Pruebas unitarias de las reglas con ramificación.
- ☐ Sin `System.out.println` ni código comentado.
- ☐ PR bajo ~250 líneas, revisado y aprobado.

Tarea Frontend

Pantalla de login, `AuthService`, interceptores y guards. Bloquea toda la navegación del resto de la aplicación.

1. Pasos

1. `core/auth/auth.service.ts` con signals (`usuario`, `autenticado`, `rol`), `login()`, `logout()`, `tieneAlgunRol()`.
2. Decodificación del JWT para leer los claims. **Solo para mostrar datos en la interfaz**, nunca para decidir permisos reales.
3. Validar `exp` al leer el token de `localStorage`: un token vencido produciría una sesión fantasma, con el menú visible y `401` en cada petición.
4. `core/auth/auth.interceptor.ts` — adjunta `Authorization: Bearer`.
5. `core/http/error.interceptor.ts` — `401` cierra sesión, `403` solo avisa.
6. `authGuard` y `rolGuard(roles)`.
7. `login.component`: formulario con correo y contraseña, mostrar/ocultar contraseña.
8. Redirección tras login según rol: ADMIN → dashboard, VENTAS → pedidos, etc.
9. Layout con barra superior: nombre del usuario, rol y “Cerrar sesión”.
10. Menú lateral que muestra solo los módulos del rol (directiva `*siRol`).

2. Archivos

```
frontend/src/app/
├── core/auth/{auth.service,auth.guard,rol.guard,auth.interceptor}.ts
├── core/http/error.interceptor.ts
├── features/auth/login.component.{ts,html,scss}
├── layout/layout.component.{ts,html,scss}
└── shared/directives/si-rol.directive.ts
```

3. Puntos de cuidado

- El token en `localStorage` es legible por cualquier script de la página. Es lo que especifica el stack; se compensa evitando `innerHTML` con datos del servidor. Documentado en `07-ARQUITECTURA-FRONTEND.md` §3.
- **403 no debe cerrar la sesión:** el usuario está autenticado, solo no autorizado para esa acción. Cerrarle la sesión por pulsar un botón equivocado es desconcertante.
- El menú filtrado por rol es **cosmética**: la seguridad real está en el backend.
- Mostrar el mismo mensaje genérico del servidor en el `401`; no inventar “el correo no existe” en el cliente (anularía R-01).
- Limpiar `localStorage` en `logout()`, incluido cualquier estado cacheado.

4. Checklist de cierre

- ☐ Todos los CA cubiertos desde la interfaz.
- ☐ Los tres estados (cargando / vacío / error) implementados.
- ☐ Errores del servidor mostrados junto al campo correspondiente.
- ☐ Guard de rol aplicado en la ruta.
- ☐ Modelos TypeScript espejo de los DTO del backend.
- ☐ Probado en Chrome y Firefox a 1366×768.
- ☐ Sin `console.log` ni `any`.

Casos de Prueba

Formato y criterios en `00-base/05-ESTANDARES-QA.md`.

1. Casos funcionales

ID	CA	Descripción	Entrada	Resultado esperado	Tipo
CP-01	CA-01, CA-02	Login válido	<code>{"correo": "admin@productionfood.local", "password": "Admin123*"}</code>	200 con token, tipo: "Bearer", datos del usuario	Positivo
CP-02	CA-05	El token contiene los claims exigidos	Decodificar el token de CP-01 en jwt.io	Contiene sub, correo, nombre, rol, exp	Positivo
CP-03	CA-04	Contraseña incorrecta	Correo válido + contraseña errónea	401 CREDENCIALES_INVALIDAS	Negativo
CP-04	CA-04	Correo inexistente	<code>"noexiste@x.local"</code>	401 con el mismo mensaje que CP-03	Seguridad
CP-05	CA-04	Los mensajes de CP-03 y CP-04 deben ser idénticos	Comparar ambas respuestas	code y message exactamente iguales	Seguridad
CP-06	CA-04	Tiempos de respuesta comparables	Medir CP-03 y CP-04 10 veces	Diferencia no sistemática (R-03)	Seguridad
CP-07	CA-03	Usuario desactivado	Desactivar un usuario e intentar login con credenciales correctas	403 USUARIO_INACTIVO	Negativo
CP-08	CA-03	Usuario desactivado con contraseña incorrecta	Desactivado + contraseña errónea	401, no 403 (R-02: no revela que existe)	Seguridad
CP-09	CA-01	Correo vacío	<code>{"correo": "", "password": "x"}</code>	400 VALIDACION_FALLIDA	Negativo
CP-10	CA-02	El token funciona	Usar el token en GET /usuarios	200	Positivo
CP-11	—	Token manipulado	Alterar un carácter de la firma	401 NO_AUTENTICADO	Seguridad
CP-12	—	Token con rol alterado	Cambiar "rol": "CONSULTA" por "ADMIN" en el payload	401 — la firma no valida	Seguridad
CP-13	—	Token expirado	Token con exp vencido	401 NO_AUTENTICADO	Seguridad
CP-14	—	La respuesta no expone el hash	Revisar el JSON de CP-01	No aparece password en ningún nivel	Seguridad
CP-15	—	Inyección SQL en el correo	<code>admin@x.local' OR '1'='1</code>	401, sin error 500	Seguridad
CP-16	—	Perfil del usuario autenticado	GET /auth/perfil con token	200 con los datos del token	Positivo

2. Batería de seguridad

Obligatoria en **cada endpoint** de esta historia:

ID	Caso	Esperado
SEC-01	Sin cabecera Authorization	401 NO_AUTENTICADO
SEC-02	Token malformado	401 NO_AUTENTICADO
SEC-03	Token expirado	401 NO_AUTENTICADO
SEC-04	Token de rol sin permiso	403 SIN_PERMISO
SEC-05	Token de usuario con estado = 0	403 USUARIO_INACTIVO

SEC-04 es el caso crítico. Si devuelve 200, falta @EnableMethodSecurity y la autorización del sistema completo está desactivada.

CP-05, CP-08 y CP-12 son los casos que de verdad importan en esta historia.

CP-05 y CP-08 verifican que el sistema no permita **enumerar correos registrados**: si el mensaje o el código HTTP cambia según si la cuenta existe, cualquiera puede averiguar quién tiene usuario en el sistema probando direcciones.

CP-12 verifica que la firma del token se valide de verdad. Un backend que decodifica el JWT sin verificar la firma acepta cualquier rol que el atacante escriba — y eso otorga acceso de administrador a cualquiera que sepa editar un JSON en base64.

3. Verificación en base de datos

```
-- CP-07: verificar el estado antes de la prueba
UPDATE usuarios SET estado = 0 WHERE correo = 'prueba@productionfood.local';

-- Los intentos fallidos se registran con id_usuario NULL (§C-11)
SELECT id_usuario, accion, tabla_afectada, detalle, fecha
FROM bitacora WHERE accion = 'LOGIN_FALLIDO' ORDER BY fecha DESC LIMIT 5;

-- Restaurar tras la prueba
UPDATE usuarios SET estado = 1 WHERE correo = 'prueba@productionfood.local';
```

4. Criterios de cierre

- ☐ Todos los casos ejecutados; los positivos pasan.
- ☐ Los negativos devuelven el HTTP y el `code` esperados.
- ☐ SEC-01 a SEC-05 pasan en todos los endpoints.
- ☐ Ningún endpoint supera 2 s (RNF-02).
- ☐ Cero defectos críticos o altos abiertos.
- ☐ Colección de Postman actualizada y versionada.

HU-04 · Bitácora de auditoría

Especificación

Fase 1 · Fuente: `previo/Historias_de_Usuario.md`

Como sistema, **quiero** registrar en bitácora cada acción crítica (crear, editar, eliminar), **para que** exista trazabilidad de las operaciones.

Depende de: HU-03 (se necesita saber quién actúa)

1. Criterios de aceptación

CA	Enunciado	Cómo se implementa
CA-01	Se registra automáticamente al crear, editar o eliminar	Aspecto AOP sobre los métodos anotados con <code>@Auditable</code>
CA-02	Se almacena <code>id_usuario</code> , <code>accion</code> , <code>tabla_afectada</code> , <code>detalle</code> , <code>fecha</code>	Tabla <code>bitacora</code> — sin cambios de columnas
CA-03	La fecha se registra con <code>current_timestamp()</code> por defecto	Ya está en el esquema original
CA-04	La bitácora es de solo lectura	Solo existe <code>GET</code> . No hay <code>POST</code> , <code>PUT</code> ni <code>DELETE</code> de bitácora

2. Endpoints

Método	Ruta	Descripción	Roles
<code>GET</code>	<code>/api/v1/bitacora</code>	Consultar con filtros y paginación	ADMIN
<code>GET</code>	<code>/api/v1/bitacora/acciones</code>	Catálogo de acciones registradas	ADMIN

Reglas transversales (paginación, formato de error, autenticación) en `00-base/04-CONTRATO-API.md`.

3. Reglas de negocio

R-01 · `id_usuario` puede ser nulo: significa “sistema”

El esquema original tenía `id_usuario NOT NULL`, lo que hacía **imposible auditar**: - la creación del administrador semilla, - los intentos de login fallidos (HU-03) — no hay usuario válido que registrar, - los procesos automáticos.

Corregido en V2 (`02-CORRECCIONES-DB.md` §C-11): la columna admite `NULL`, y `NULL` se muestra como “**Sistema**” en la interfaz.

R-02 · Se audita después del éxito, no antes

El aspecto usa `@AfterReturning`, no `@Before`. Auditar antes de ejecutar registraría como cambios reales operaciones que después fallaron y se revirtieron, llenando la bitácora de eventos que nunca ocurrieron.

R-03 · Un fallo de auditoría no debe tumbar la operación de negocio

`BitacoraService.registrar()` captura sus propias excepciones y las envía al log de la aplicación. Que la auditoría falle es un problema; que impida registrar una venta es peor.

Consecuencia asumida: la bitácora puede tener huecos si la base de datos falla al escribirla. Es el compromiso deliberado, y por eso la trazabilidad del **inventario** no descansa en la bitácora sino en el kardex (§C-04), que sí es transaccional.

R-04 · `detalle` está limitado a 255 caracteres

La columna es `varchar(255)`. El resumen se **recorta** antes de insertar. Sin recortar, con `sql_mode` estricto de MySQL 8 el `INSERT` falla en lugar de truncar, y por R-03 el fallo se tragaría en silencio: la acción no quedaría auditada.

R-05 · Nunca se auditan datos sensibles

`detalle` **no** incluye contraseñas, hashes ni tokens. El resumen de una creación de usuario registra el correo y el rol, nunca la

credencial. Una bitácora con contraseñas es peor que no tener bitácora: concentra en una tabla consultable lo que el resto del sistema protege.

R-06 · Acciones que se auditan

Acción	Cuándo
CREAR	Alta de cualquier entidad
EDITAR	Modificación
CAMBIAR_ESTADO	Activar/desactivar, transiciones de estado
ELIMINAR	Borrado de líneas de detalle
LOGIN / LOGIN_FALLIDO	HU-03
RECEPCION_COMPRA	HU-13 — entra stock
APROBAR_DEVOLUCION	HU-18 — vuelve stock
REGISTRAR_CONSUMO	HU-22 — sale stock

Las consultas (GET) no se auditan: generarían un volumen enorme sin aportar trazabilidad de cambios.

4. Modelo de datos

Tabla bitacora (columnas sin cambios; solo id_usuario pasa a nullable en V2)

Columna	Tipo	Notas
id_bitacora	int unsigned PK AI	
id_usuario	int unsigned NULL	NULL = sistema (R-01, §C-11)
accion	varchar(50) NOT NULL	R-06
tabla_afectada	varchar(50) NOT NULL	
detalle	varchar(255) NULL	Recortado (R-04)
fecha	datetime NOT NULL DEFAULT CURRENT_TIMESTAMP	CA-03

Índice añadido en V2: idx_bitacora_tabla (tabla_afectada, fecha) .

```
// GET /api/v1/bitacora?tabla=usuarios&accion=CREAR&fechaInicio=2026-09-01&fechaFin=2026-09-18
{
  "content": [
    { "idBitacora": 145, "usuario": { "idUsuario": 1, "nombre": "Administrador Inicial" },
      "accion": "CREAR", "tablaAfectada": "usuarios",
      "detalle": "Usuario creado: maria@productionfood.local, rol VENTAS",
      "fecha": "2026-09-18T14:32:11" },
    { "idBitacora": 144, "usuario": null,
      "accion": "LOGIN_FALLIDO", "tablaAfectada": "usuarios",
      "detalle": "Intento fallido para el correo: desconocido@x.local",
      "fecha": "2026-09-18T14:30:02" }
  ],
  "page": 0, "size": 20, "totalElements": 2, "totalPages": 1
}
```

usuario: null se muestra como “Sistema”.

5. Códigos de error

code	HTTP	Cuándo
ACCESO_DENEGADO_BITACORA	403	Un rol distinto de ADMIN intenta consultarla

Los transversales (VALIDACION_FALLIDA , NO_AUTENTICADO , SIN_PERMISO , RECURSO_NO_ENCONTRADO) están en 00-base/04-CONTRATO-API.md §5.

6. Fuera de alcance

Lo que no cubre esta historia y no debe implementarse aquí: se limita estrictamente a los criterios de aceptación listados arriba. Cualquier funcionalidad adicional se propone como historia nueva, no se agrega en silencio.

7. Definición de Terminado

Se aplica la DoD de `00-base/01-CONVENCIONES.md` §7.

8. Notas

Por qué esta historia va en el Sprint 1 y no al final

La auditoría es **transversal**: se implementa como aspecto AOP y se activa anotando los métodos de servicio de cada módulo.

Si se deja para el final hay que volver a abrir los veinte servicios ya escritos, anotarlos uno por uno y volver a probarlos. Implementada al principio, cada historia posterior solo agrega su `@Auditable` mientras se escribe.

Está en el Sprint 1 del `00-base/10-ROADMAP.md` por esta razón.

Diseño

1. Pantallas

Consulta de bitácora (solo ADMIN, solo lectura)

Bitácora de auditoría				
Desde	18/09/2026	Hasta	18/09/2026	Acción Todas
				Tabla Todas
FECHA ▼	USUARIO	ACCIÓN	TABLA	DETALLE
18/09 14:32:11	Admin Inicial	●CREAR	usuarios	Usuario crea...
18/09 14:30:02	⊗ Sistema	●LOGIN FALLIDO	usuarios	Intento falli...
18/09 14:28:45	Carlos Cochero	●EDITAR	proveedores	Proveedor act...
18/09 14:15:30	María Gómez	●CAMBIAR_ESTADO	pedidos	Pedido 45: PEN...
Elementos por página: 20 ▼ 1-4 de 128 ◀ ▶				

Sin botones de acción: la pantalla no tiene “Nuevo”, ni menú `:` por fila, ni posibilidad de editar. Es un registro, no un CRUD (CA-04).

El detalle se trunca en la celda; el texto completo aparece en un tooltip al pasar el cursor.

“**⊗ Sistema**” identifica los registros con `id_usuario = NULL`: acciones sin usuario autenticado, como los intentos de login fallidos o el script de semilla.

2. Flujo



Si el método lanza excepción, `@AfterReturning` no se ejecuta y no se audita nada (R-02). Si la auditoría falla, la operación de negocio ya terminó bien y no se revierte (R-03).

3. Componentes Angular Material

Elemento	Componente
Tabla	<code>MatTable</code> + <code>MatPaginator</code> (sin acciones de fila)
Rango de fechas	<code>MatDateRangePicker</code>
Filtros	<code>MatSelect</code> (acción, tabla)
Acción	<code>MatChip</code> con color por tipo
Detalle largo	<code>MatTooltip</code>
Usuario "Sistema"	<code>MatIcon</code> <code>settings</code> + texto en cursiva

4. Validaciones en el formulario

Esta pantalla **no tiene formularios de datos**. Solo los filtros:

Filtro	Regla
Rango de fechas	<code>fechaInicio <= fechaFin</code> , validado antes de llamar
Acción	Opcional, del catálogo de la API
Tabla afectada	Opcional

Por defecto se cargan los **últimos 7 días**. La bitácora crece rápido y cargarla completa es lento sin aportar nada: quien la consulta suele buscar algo reciente.

5. Estados de la pantalla

Estado	Qué se muestra
Cargando	<code>MatProgressBar</code>
Vacío	"No hay registros en el período seleccionado" + sugerencia de ampliar el rango
Error	Mensaje + "Reintentar"
Con datos	Tabla ordenada por fecha descendente

El estado vacío sugiere ampliar el rango porque la causa habitual es un filtro de fechas demasiado estrecho, no la ausencia de actividad.

6. Accesibilidad y UX

- Todos los campos con `<mat-label>` ; nunca solo *placeholder*.
- Navegación por teclado en orden lógico; `Enter` envía el formulario.
- Los mensajes de error del servidor se muestran **junto al campo** (`fieldErrors`).
- El botón de envío se deshabilita mientras la petición está en vuelo.
- Confirmación antes de descartar cambios sin guardar.
- Los tres estados obligatorios (cargando / vacío / error) según `00-base/05-ESTANDARES-QA.md` §7.

Tarea Backend

Componente transversal. Se implementa una vez y lo usan todas las historias posteriores. Requiere `spring-boot-starter-aop` .

1. Pasos

1. Agregar la dependencia `spring-boot-starter-aop` al `pom.xml` .
2. `@Auditable` — anotación de método con `accion()` y `tabla()` .
3. `Bitacora` (POJO), `BitacoraRowMapper` con `LEFT JOIN usuarios` para el nombre. `rs.getObject("id_usuario", Long.class)` , no `getLong()` : este último devuelve `0` para `NULL` y el registro del sistema aparecería atribuido al usuario 0.
4. `BitacoraRepository`: `insertar()` , `buscar(filtros, page)` , `contar(filtros)` , `listarAcciones()` (`SELECT DISTINCT accion`).
5. `BitacoraService.registrar(idUsuario, accion, tabla, detalle)` :
 - recorta `detalle` a 255 caracteres (R-04);
 - **envuelto en `try/catch` que solo registra en el log** (R-03).
6. `AuditoriaAspect` con `@AfterReturning` sobre `@annotation(auditable)` :
 - obtiene el usuario con `SecurityUtils.idUsuarioActual()` (puede ser `null`);
 - construye el resumen desde el valor retornado, **sin incluir campos sensibles** (R-05).
7. `BitacoraController` con **solo métodos `GET`** y `@PreAuthorize("hasRole('ADMIN')")` .

8. Filtros: `idUsuario`, `tabla`, `accion`, `fechaInicio`, `fechaFin`, paginación. Validar `fechaInicio <= fechaFin` → `400 RANGO_FECHAS_INVALIDO`.
9. Anotar los servicios ya escritos de HU-01 y HU-02.
10. En HU-03, registrar `LOGIN` y `LOGIN_FALLIDO` con `id_usuario = NULL` cuando corresponda.

2. Archivos

```
backend/src/main/java/co/edu/corposucre/productionfood/common/audit/
├── Auditable.java
├── AuditoriaAspect.java
├── BitacoraService.java
├── Bitacora.java
├── BitacoraRepository.java
├── BitacoraRowMapper.java
├── BitacoraController.java
└── dto/BitacoraResponse.java
```

3. Puntos de cuidado

- `getObject("id_usuario", Long.class)`, nunca `getLong()`: `getLong()` convierte `NULL` en `0` y el registro aparecería atribuido a un usuario inexistente.
- **Ningún endpoint de escritura** (CA-04). Si existe un `POST /bitacora`, la auditoría deja de ser confiable: cualquiera podría fabricar registros.
- **Recortar detalle a 255** antes de insertar (R-04). Con `sql_mode` estricto el `INSERT` falla, y por R-03 el fallo se traga en silencio.
- **Nunca auditar contraseñas ni tokens** (R-05). Revisar el método que construye el resumen antes del PR.
- `@AfterReturning`, no `@Before` (R-02).
- El `try/catch` de R-03 **no debe capturar y relanzar**: debe tragarse la excepción y registrarla en el log.

4. Checklist de cierre

- ☐ Todos los CA implementados y verificados manualmente.
- ☐ Endpoints documentados en Swagger con ejemplos.
- ☐ `@PreAuthorize` en cada método, según la matriz de roles.
- ☐ Operaciones de escritura auditadas en bitácora (HU-04).
- ☐ Pruebas unitarias de las reglas con ramificación.
- ☐ Sin `System.out.println` ni código comentado.
- ☐ PR bajo ~250 líneas, revisado y aprobado.

Tarea Frontend

Pantalla de consulta, solo lectura, accesible únicamente para ADMIN.

1. Pasos

1. `features/bitacora/models/bitacora.model.ts`.
2. `bitacora.service.ts` con `listar(filtros)` y `listarAcciones()`.
3. `bitacora-lista.component`: tabla con fecha, usuario, acción, tabla afectada, detalle.
4. Filtros: rango de fechas (`MatDatepicker`), acción (`MatSelect` poblado desde la API), tabla afectada y usuario.
5. Mostrar **"Sistema"** cuando `usuario` es `null` (R-01), con estilo diferenciado.
6. Chip de color por tipo de acción: verde `CREAR`, azul `EDITAR`, ámbar `CAMBIAR_ESTADO`, rojo `ELIMINAR` / `LOGIN_FALLIDO`.
7. Orden por defecto: fecha descendente (lo más reciente primero).
8. `detalle` truncado en la celda con tooltip para el texto completo.
9. **Sin botones de crear, editar ni eliminar** (CA-04).
10. Ruta con `rolGuard(['ADMIN'])`.

2. Archivos

```
frontend/src/app/features/bitacora/  
├── models/bitacora.model.ts  
├── services/bitacora.service.ts  
├── pages/bitacora-lista/  
└── bitacora.routes.ts
```

3. Puntos de cuidado

- **Ningún control de escritura en la pantalla** (CA-04): ni crear, ni editar, ni borrar.
- `usuario: null` → “Sistema”. Mostrar una celda vacía haría pensar que hubo un error de carga.
- El rango de fechas por defecto acota a los últimos 7 días: la bitácora crece rápido y cargarla entera es lento y poco útil.
- Validar en el cliente que `fechaInicio <= fechaFin` antes de llamar.
- Formatear la fecha con el locale `es-CO`.

4. Checklist de cierre

- ☐ Todos los CA cubiertos desde la interfaz.
- ☐ Los tres estados (cargando / vacío / error) implementados.
- ☐ Errores del servidor mostrados junto al campo correspondiente.
- ☐ Guard de rol aplicado en la ruta.
- ☐ Modelos TypeScript espejo de los DTO del backend.
- ☐ Probado en Chrome y Firefox a 1366×768.
- ☐ Sin `console.log` ni `any`.

Casos de Prueba

Formato y criterios en `00-base/05-ESTANDARES-QA.md`.

1. Casos funcionales

ID	CA	Descripción	Entrada	Resultado esperado	Tipo
CP-01	CA-01	Crear un usuario genera registro	POST /usuarios y luego consultar bitácora	Existe un registro CREAR / usuarios con el id del autor	Positivo
CP-02	CA-01	Editar genera registro	PUT /usuarios/{id}	Registro EDITAR	Positivo
CP-03	CA-01	Cambiar estado genera registro	PATCH /usuarios/{id}/estado	Registro CAMBIAR_ESTADO	Positivo
CP-04	CA-01	Una consulta NO genera registro	GET /usuarios	No aparece registro nuevo	Negativo
CP-05	CA-01	Operación fallida NO genera registro	POST /usuarios con correo duplicado (409)	No aparece registro (R-02)	Negativo
CP-06	CA-02	Se almacenan los cinco campos	Consultar cualquier registro	idUsuario , accion , tablaAfectada , detalle , fecha presentes	Positivo
CP-07	CA-03	La fecha se asigna sola	Insertar sin enviar fecha	fecha con la hora actual del servidor	Positivo
CP-08	CA-03	La fecha está en hora de Colombia	Crear un registro y comparar	Coincide con la hora local, no UTC	Integridad
CP-09	CA-04	No existe POST	POST /api/v1/bitacora	405 o 404	Seguridad
CP-10	CA-04	No existe PUT	PUT /api/v1/bitacora/1	405 o 404	Seguridad
CP-11	CA-04	No existe DELETE	DELETE /api/v1/bitacora/1	405 o 404	Seguridad
CP-12	—	Login fallido se audita como sistema	Login con contraseña errónea	Registro LOGIN_FALLIDO con usuario: null	Positivo
CP-13	—	La bitácora no contiene contraseñas	Crear usuario y revisar el detalle	No aparece la contraseña ni el hash (R-05)	Seguridad
CP-14	—	detalle largo no rompe la operación	Crear entidad con nombre de 100 caracteres	La operación tiene éxito; detalle recortado a 255	Borde
CP-15	—	Un rol distinto de ADMIN no accede	GET /bitacora con token de VENTAS	403 SIN_PERMISO	Seguridad
CP-16	—	Filtro por rango de fechas	?fechaInicio=2026-09-01&fechaFin=2026-09-18	Solo registros del rango, ambos días incluidos	Positivo
CP-17	—	Rango de fechas invertido	?fechaInicio=2026-09-18&fechaFin=2026-09-01	400 RANGO_FECHAS_INVALIDO	Negativo

2. Batería de seguridad

Obligatoria en **cada endpoint** de esta historia:

ID	Caso	Esperado
SEC-01	Sin cabecera Authorization	401 NO_AUTENTICADO
SEC-02	Token malformado	401 NO_AUTENTICADO
SEC-03	Token expirado	401 NO_AUTENTICADO
SEC-04	Token de rol sin permiso	403 SIN_PERMISO
SEC-05	Token de usuario con estado = 0	403 USUARIO_INACTIVO

SEC-04 es el caso crítico. Si devuelve **200**, falta `@EnableMethodSecurity` y la autorización del sistema completo está desactivada.

CP-13 es el caso crítico de esta historia. Una bitácora que registra contraseñas concentra en una sola tabla —consultable desde la interfaz por cualquier administrador— exactamente lo que el hash de HU-01 existe para proteger. El defecto aparece cuando el resumen se construye serializando el DTO completo en lugar de seleccionar campos.

CP-05 verifica R-02: auditar con `@Before` en lugar de `@AfterReturning` llena la bitácora de operaciones que fallaron, y deja de servir como registro de lo que efectivamente cambió.

3. Verificación en base de datos

```
-- CP-06: los cinco campos de CA-02
SELECT id_bitacora, id_usuario, accion, tabla_afectada, detalle, fecha
FROM bitacora ORDER BY id_bitacora DESC LIMIT 5;

-- CP-12: los registros del sistema llevan id_usuario NULL (§C-11)
SELECT COUNT(*) FROM bitacora WHERE id_usuario IS NULL;

-- CP-13: ninguna contraseña ni hash en la bitácora. DEBE devolver 0 filas.
SELECT id_bitacora, detalle FROM bitacora
WHERE detalle LIKE '%$2a$%' OR detalle LIKE '%password%' OR detalle LIKE '%token%';

-- CP-14: ningún detalle supera el límite de la columna
SELECT MAX(CHAR_LENGTH(detalle)) FROM bitacora; -- <= 255

-- CP-11: el conteo solo debe crecer, nunca disminuir
SELECT COUNT(*) FROM bitacora;
```

4. Criterios de cierre

- ☐ Todos los casos ejecutados; los positivos pasan.
- ☐ Los negativos devuelven el HTTP y el `code` esperados.
- ☐ SEC-01 a SEC-05 pasan en todos los endpoints.
- ☐ Ningún endpoint supera 2 s (RNF-02).
- ☐ Cero defectos críticos o altos abiertos.
- ☐ Colección de Postman actualizada y versionada.

Fase 2 · Datos Maestros

HU-05 · Gestión de clientes

Especificación

Fase 2 · Fuente: `previo/Historias_de_Usuario.md`

Como administrador, **quiero** crear, consultar, editar y desactivar clientes (nombre, contacto, teléfono, estado), **para que** lleve el registro de quiénes compran.

Depende de: HU-03, HU-04

1. Criterios de aceptación

CA	Enunciado	Cómo se implementa
CA-01	Nombre obligatorio, máx. 100	<code>@NotBlank</code> + <code>@Size(max=100)</code>
CA-02	Contacto y teléfono opcionales	Sin <code>@NotBlank</code> ; <code>@Size</code> para el largo
CA-03	Estado por defecto activo (<code>1</code>)	<code>DEFAULT 1</code> ya presente en el esquema original
CA-04	Listar con paginación y filtrar por nombre	<code>GET /clientes?nombre=&page=&size=&sort=</code>
CA-05	No se permite eliminar , solo desactivar	No existe endpoint <code>DELETE</code>

2. Endpoints

Método	Ruta	Descripción	Roles
<code>GET</code>	<code>/api/v1/clientes</code>	Listar con filtro y paginación	ADMIN, VENTAS, PRODUCCION, CONSULTA
<code>GET</code>	<code>/api/v1/clientes/{id}</code>	Obtener un cliente	ADMIN, VENTAS, PRODUCCION, CONSULTA
<code>POST</code>	<code>/api/v1/clientes</code>	Registrar	ADMIN, VENTAS
<code>PUT</code>	<code>/api/v1/clientes/{id}</code>	Editar	ADMIN, VENTAS
<code>PATCH</code>	<code>/api/v1/clientes/{id}/estado</code>	Activar o desactivar	ADMIN, VENTAS

Reglas transversales (paginación, formato de error, autenticación) en `00-base/04-CONTRATO-API.md`.

3. Reglas de negocio

R-01 · Nunca se elimina (CA-05)

`pedidos.id_cliente` es una FK sin `ON DELETE CASCADE` : borrar un cliente con pedidos fallaría por integridad referencial, y forzarlo destruiría el historial de ventas. La desactivación (`estado = 0`) conserva todo.

R-02 · Un cliente inactivo no puede recibir pedidos nuevos

HU-16 CA-01 exige que el cliente exista **y esté activo**. Desactivar es la forma de retirar a un cliente sin perder sus pedidos anteriores.

Los pedidos ya creados de un cliente que se desactiva **no se cancelan**: siguen su curso normal. Desactivar impide nuevas ventas, no anula compromisos adquiridos.

R-03 · Se permiten nombres duplicados

No hay `UNIQUE` sobre `nombre`, y es deliberado: dos clientes pueden llamarse igual (“Tienda La Esquina” hay varias). La interfaz muestra un aviso no bloqueante cuando detecta un nombre idéntico, para que quien registra decida.

R-04 · El teléfono se guarda tal como se escribe

`varchar(20)` sin validación de formato. Un teléfono puede ser fijo, celular, con indicativo o con extensión. Imponer un formato rígido obliga a falsear datos cuando el número real no encaja. Se valida solo la longitud.

4. Modelo de datos

Tabla `clientes` (sin cambios de esquema; V2 añade el índice `idx_clientes_nombre`)

Columna	Tipo	Notas
<code>id_cliente</code>	<code>int unsigned</code> PK AI	
<code>nombre</code>	<code>varchar(100)</code> NOT NULL	CA-01
<code>contacto</code>	<code>varchar(100)</code> NULL	CA-02
<code>telefono</code>	<code>varchar(20)</code> NULL	CA-02, R-04
<code>estado</code>	<code>tinyint(1)</code> NOT NULL DEFAULT 1	CA-03

```
// POST /api/v1/clientes
{ "nombre": "Tienda La Esquina", "contacto": "Pedro Ruiz", "telefono": "300 555 1234" }

// 201 Created • Location: /api/v1/clientes/7
{ "idCliente": 7, "nombre": "Tienda La Esquina",
  "contacto": "Pedro Ruiz", "telefono": "300 555 1234", "activo": true }
```

Campos ordenables: `nombre` , `id` .

5. Códigos de error

code	HTTP	Cuándo
<code>CLIENTE_CON_PEDIDOS_ACTIVOS</code>	409	Se intenta desactivar un cliente con pedidos en curso (advertencia configurable)

Los transversales (`VALIDACION_FALLIDA` , `NO_AUTENTICADO` , `SIN_PERMISO` , `RECURSO_NO_ENCONTRADO`) están en `00-base/04-CONTRATO-API.md` §5.

6. Fuera de alcance

Lo que **no** cubre esta historia y no debe implementarse aquí: se limita estrictamente a los criterios de aceptación listados arriba. Cualquier funcionalidad adicional se propone como historia nueva, no se agrega en silencio.

7. Definición de Terminado

Se aplica la DoD de `00-base/01-CONVENCIONES.md` §7.

8. Notas

Esta historia es la plantilla de todos los CRUD del proyecto

HU-06 (proveedores), HU-07 (unidades), HU-08 (materias primas) y HU-11 (productos) tienen la misma estructura. El `00-base/10-ROADMAP.md` indica implementar **esta primero y completa** —backend, frontend y pruebas— y usarla como molde.

Hacer los cinco en paralelo desde cero multiplica por cinco las decisiones de diseño y produce cinco estilos distintos que después hay que unificar.

Diseño

1. Pantallas

Listado de clientes

Clientes [+ Nuevo cliente]			
Buscar por nombre	Estado Activos		
NOMBRE ▲	CONTACTO	TELÉFONO	ESTADO ...
Panadería El Trigo	Ana Restrepo	310 444 2211	●Activo :
Supermercado Central	Luis Pérez	604 285 9900	●Activo :
Tienda La Esquina	Pedro Ruiz	300 555 1234	●Activo :
Elementos por página: 20 ▼ 1-3 de 3 ◀ ▶			

Formulario (diálogo)

Nuevo cliente [X]

Nombre *

Tienda La Esquina

18 / 100

i Ya existe un cliente con este nombre

Persona de contacto

Pedro Ruiz

Teléfono

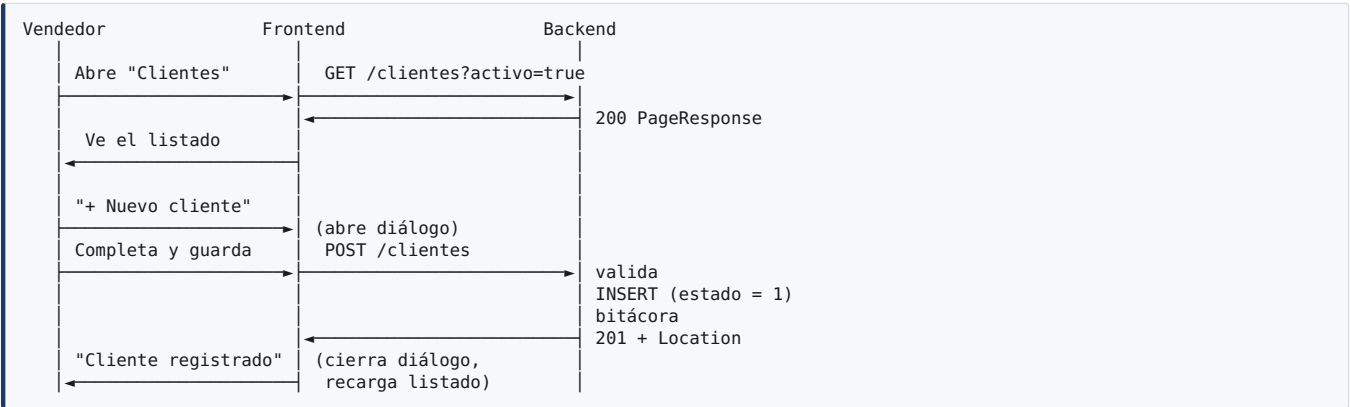
300 555 1234

[Cancelar]

[Guardar]

El aviso de nombre duplicado es **informativo, no bloqueante** (R-03): puede haber dos clientes con el mismo nombre legítimamente.

2. Flujo



3. Componentes Angular Material

Elemento	Componente
Tabla	<code>MatTable</code> + <code>MatSort</code> + <code>MatPaginator</code>
Búsqueda	<code>MatFormField</code> + <code>MatInput</code> + icono de lupa
Filtro de estado	<code>MatSelect</code>
Estado	<code>MatChip</code> verde/gris
Menú de fila	<code>MatMenu</code> (Editar · Desactivar)
Formulario	<code>MatDialog</code> + <code>MatFormField</code>
Confirmación	<code>ConfirmarDialogComponent</code> (de HU-02)
Avisos	<code>MatSnackBar</code>

4. Validaciones en el formulario

Campo	Reglas	Mensaje
Nombre	requerido, máx. 100	"El nombre es obligatorio"
Contacto	opcional, máx. 100	"Máximo 100 caracteres"
Teléfono	opcional, máx. 20	"Máximo 20 caracteres"

El teléfono no valida formato (R-04): puede ser fijo, celular, con indicativo o con extensión. Imponer un patrón obliga a falsear el dato cuando el número real no encaja.

5. Estados de la pantalla

Estado	Qué se muestra
Cargando	<code>MatProgressBar</code> sobre la tabla
Vacío sin filtro	"No hay clientes registrados" + "Registrar el primero"
Vacío con filtro	"Ningún cliente coincide con «xxx»" + "Limpiar filtros"
Error	Mensaje + "Reintentar"
Guardando	Botón deshabilitado con spinner

6. Accesibilidad y UX

- Todos los campos con `<mat-label>`; nunca solo *placeholder*.
- Navegación por teclado en orden lógico; `Enter` envía el formulario.
- Los mensajes de error del servidor se muestran **junto al campo** (`fieldErrors`).
- El botón de envío se deshabilita mientras la petición está en vuelo.
- Confirmación antes de descartar cambios sin guardar.
- Los tres estados obligatorios (cargando / vacío / error) según `00-base/05-ESTANDARES-QA.md` §7.

Tarea Backend

CRUD de referencia del proyecto. Se implementa con cuidado porque HU-06, HU-07, HU-08 y HU-11 lo copian. El patrón completo de repositorio está en `00-base/06-ARQUITECTURA-BACKEND.md` §4.

1. Pasos

1. `Cliente` (POJO) y `ClienteRowMapper` — `estado` → `boolean` con `rs.getBoolean()`.
2. `ClienteRepository`:
 - `buscar(nombre, activo, PageRequest)` con filtros opcionales y lista blanca de orden;
 - `contar(nombre, activo)` — **mismo** `WHERE`;
 - `buscarPorId()`, `insertar()` con `GeneratedKeyHolder`, `actualizar()`, `cambiarEstado()`;
 - `escaparLike()` para `%` y `_`.
3. DTO: `CrearClienteRequest`, `ActualizarClienteRequest`, `ClienteResponse`, `CambiarEstadoRequest`.
4. `ClienteService` con `@Transactional` en escrituras y `@Transactional(readonly = true)` en lecturas.
5. `ClienteController` con `@PreAuthorize` según la matriz de roles y `@Operation` de Swagger.
6. Anotar con `@Auditable` las operaciones de escritura (HU-04).
7. **No implementar** `DELETE` (CA-05).

2. Archivos

```
backend/src/main/java/co/edu/corposucre/productionfood/cliente/
├── {Cliente,ClienteRowMapper,ClienteRepository,ClienteService,ClienteController}.java
└── dto/{CrearClienteRequest,ActualizarClienteRequest,ClienteResponse}.java
```

3. Puntos de cuidado

- **buscar()** y **contar()** con el mismo **WHERE**, siempre.
- **sort** desde lista blanca; nunca concatenar el valor recibido.
- **Escapar % y _**: sin eso, buscar % devuelve la tabla entera.
- **estado** se expone como **boolean** en el JSON, no como **0 / 1** (04-CONTRATO-API.md §6).
- El DTO de creación **no** recibe **estado**: lo pone el **DEFAULT** (CA-03).
- Sin endpoint **DELETE**.
- Como es la plantilla: dejar el código limpio y comentado donde la decisión no sea obvia.

4. Checklist de cierre

- ☐ Todos los CA implementados y verificados manualmente.
- ☐ Endpoints documentados en Swagger con ejemplos.
- ☐ **@PreAuthorize** en cada método, según la matriz de roles.
- ☐ Operaciones de escritura auditadas en bitácora (HU-04).
- ☐ Pruebas unitarias de las reglas con ramificación.
- ☐ Sin **System.out.println** ni código comentado.
- ☐ PR bajo ~250 líneas, revisado y aprobado.

Tarea Frontend

Pantalla de referencia de los listados del proyecto. Reutiliza **tabla-paginada** y **confirmar-dialog** creados en HU-02.

1. Pasos

1. **cliente.model.ts**: **Cliente**, **CrearClienteDto**, **ActualizarClienteDto**.
2. **cliente.service.ts**: **listar**, **obtener**, **crear**, **actualizar**, **cambiarEstado**.
3. **cliente-lista.component**: tabla con nombre, contacto, teléfono, estado y menú **⋮**.
4. Búsqueda por nombre con **debounceTime(350)** + **switchMap**.
5. Filtro por estado (todos / activos / inactivos), por defecto **activos**.
6. **cliente-formulario.component** en **MatDialog**, para crear y editar.
7. Confirmación al desactivar, explicando la consecuencia.
8. Los tres estados: cargando / vacío / error, distinguiendo vacío-por-filtro.
9. Ruta con **rolGuard(['ADMIN', 'VENTAS', 'PRODUCCION', 'CONSULTA'])**; los botones de escritura con ***siRol=['ADMIN', 'VENTAS']**.

2. Archivos

```
frontend/src/app/features/cliente/
├── models/cliente.model.ts
├── services/cliente.service.ts
├── pages/{cliente-lista,cliente-formulario}/
└── cliente.routes.ts
```

3. Puntos de cuidado

- **debounceTime** + **switchMap** en la búsqueda: sin **switchMap**, las respuestas pueden llegar desordenadas y mostrar el resultado de una búsqueda anterior.
- Ocultar los botones de escritura para **CONSULTA** y **PRODUCCION** — cosmética, la autorización real es del backend.
- Filtro por defecto en “activos”: es lo que casi siempre se busca.
- Como es la plantilla: componentes reutilizables y sin lógica duplicada.

4. Checklist de cierre

- ☐ Todos los CA cubiertos desde la interfaz.
- ☐ Los tres estados (cargando / vacío / error) implementados.
- ☐ Errores del servidor mostrados junto al campo correspondiente.
- ☐ Guard de rol aplicado en la ruta.
- ☐ Modelos TypeScript espejo de los DTO del backend.
- ☐ Probado en Chrome y Firefox a 1366x768.
- ☐ Sin `console.log` ni `any`.

Casos de Prueba

Formato y criterios en `00-base/05-ESTANDARES-QA.md`.

1. Casos funcionales

ID	CA	Descripción	Entrada	Resultado esperado	Tipo
CP-01	CA-01, CA-03	Crear cliente con solo el nombre	<code>{"nombre":"Tienda La Esquina"}</code>	<code>201</code> , <code>activo: true</code> , contacto y teléfono <code>null</code>	Positivo
CP-02	CA-01	Nombre vacío	<code>{"nombre":""}</code>	<code>400 VALIDACION_FALLIDA</code> en nombre	Negativo
CP-03	CA-01	Nombre de 101 caracteres	<code>"A"×101</code>	<code>400</code>	Borde
CP-04	CA-01	Nombre de exactamente 100	<code>"A"×100</code>	<code>201</code>	Borde
CP-05	CA-02	Crear con todos los campos	nombre + contacto + teléfono	<code>201</code> con los tres valores	Positivo
CP-06	CA-02	Contacto de 101 caracteres	<code>contacto</code> largo	<code>400</code>	Borde
CP-07	CA-03	Estado por defecto	Crear sin <code>estado</code>	<code>activo: true</code> ; en base <code>estado = 1</code>	Positivo
CP-08	CA-04	Listar con paginación	<code>?page=0&size=5</code>	<code>200</code> , máx. 5, <code>totalElements</code> correcto	Positivo
CP-09	CA-04	Filtrar por nombre	<code>?nombre=Tien</code>	Solo los que empiezan por "Tien"	Positivo
CP-10	CA-04	Filtro sin coincidencias	<code>?nombre=zzzz</code>	<code>content: []</code> , <code>totalElements: 0</code>	Borde
CP-11	CA-04	Comodín en el filtro	<code>?nombre=%</code>	No devuelve todo: se trata literalmente	Seguridad
CP-12	CA-04	Orden no permitido	<code>?sort=estado,asc</code> (fuera de lista blanca)	<code>400 CAMPO_ORDEN_INVALIDO</code>	Seguridad
CP-13	CA-04	<code>size</code> excesivo	<code>?size=1000</code>	<code>400</code>	Borde
CP-14	—	Editar un cliente	<code>PUT</code> con datos nuevos	<code>200</code> con los cambios	Positivo
CP-15	—	Editar inexistente	<code>PUT /clientes/9999</code>	<code>404 RECURSO_NO_ENCONTRADO</code>	Negativo
CP-16	CA-05	Desactivar	<code>PATCH .../estado {"activo":false}</code>	<code>200</code> , <code>estado = 0</code> , la fila permanece	Positivo
CP-17	CA-05	No existe DELETE	<code>DELETE /clientes/7</code>	<code>405</code> o <code>404</code>	Negativo
CP-18	—	Inyección SQL en el filtro	<code>?nombre=' OR 1=1 - - Sin error 500; resultado coherente Seguridad CP-19 - Rol CONSULTA intenta crear POST con token CONSULTA 403 SIN_PERMISO Seguridad CP-20 - Rol CONSULTA puede listar GET con token CONSULTA 200`</code>	Positivo	

2. Batería de seguridad

Obligatoria en **cada endpoint** de esta historia:

ID	Caso	Esperado
SEC-01	Sin cabecera <code>Authorization</code>	<code>401 NO_AUTENTICADO</code>
SEC-02	Token malformado	<code>401 NO_AUTENTICADO</code>
SEC-03	Token expirado	<code>401 NO_AUTENTICADO</code>
SEC-04	Token de rol <code>sin</code> permiso	<code>403 SIN_PERMISO</code>
SEC-05	Token de usuario con <code>estado = 0</code>	<code>403 USUARIO_INACTIVO</code>

SEC-04 es el caso crítico. Si devuelve `200`, falta `@EnableMethodSecurity` y la autorización del sistema completo está desactivada.

3. Verificación en base de datos

```
-- CP-07: el DEFAULT de estado
SELECT id_cliente, nombre, estado FROM clientes ORDER BY id_cliente DESC LIMIT 1;

-- CP-16 y CP-17: desactivación lógica, nunca borrado
SELECT COUNT(*) FROM clientes;           -- comparar antes/después
SELECT estado FROM clientes WHERE id_cliente = 7;  -- 0 tras desactivar

-- CP-18: la tabla sigue intacta
SELECT COUNT(*) FROM clientes;

-- Verificar que el índice de V2 se usa en el filtro por nombre
EXPLAIN SELECT id_cliente, nombre FROM clientes WHERE nombre LIKE 'Tien%';
-- Se espera: key = idx_clientes_nombre
```

4. Criterios de cierre

- ☐ Todos los casos ejecutados; los positivos pasan.
- ☐ Los negativos devuelven el HTTP y el `code` esperados.
- ☐ SEC-01 a SEC-05 pasan en todos los endpoints.
- ☐ Ningún endpoint supera 2 s (RNF-02).
- ☐ Cero defectos críticos o altos abiertos.
- ☐ Colección de Postman actualizada y versionada.

HU-06 · Gestión de proveedores

Especificación

Fase 2 · Fuente: `previo/Historias_de_Usuario.md`

Como administrador, **quiero** CRUD de proveedores (nombre, contacto, teléfono, correo, estado), **para que** gestione quiénes nos suministran materia prima.

Depende de: HU-05 (mismo patrón de CRUD)

1. Criterios de aceptación

CA	Enunciado	Cómo se implementa
CA-01	Nombre obligatorio, máx. 100	<code>@NotBlank</code> + <code>@Size(max=100)</code>
CA-02	Contacto, teléfono y correo opcionales	Sin <code>@NotBlank</code> ; <code>@Email</code> solo si viene informado
CA-03	Estado por defecto activo (<code>1</code>)	<code>DEFAULT 1</code> ya presente en el esquema
CA-04	Listar con paginación y filtrar por nombre	<code>GET /proveedores?nombre=&page=&size=</code>
CA-05	No se permite eliminar , solo desactivar	No existe endpoint <code>DELETE</code>

2. Endpoints

Método	Ruta	Descripción	Roles
<code>GET</code>	<code>/api/v1/proveedores</code>	Listar con filtro y paginación	ADMIN, COMPRAS, CONSULTA
<code>GET</code>	<code>/api/v1/proveedores/{id}</code>	Obtener uno	ADMIN, COMPRAS, CONSULTA
<code>POST</code>	<code>/api/v1/proveedores</code>	Registrar	ADMIN, COMPRAS
<code>PUT</code>	<code>/api/v1/proveedores/{id}</code>	Editar	ADMIN, COMPRAS
<code>PATCH</code>	<code>/api/v1/proveedores/{id}/estado</code>	Activar o desactivar	ADMIN, COMPRAS

Reglas transversales (paginación, formato de error, autenticación) en `00-base/04-CONTRATO-API.md`.

3. Reglas de negocio

R-01 · Nunca se elimina (CA-05)

`compras.id_proveedor` es FK sin cascada. Borrar un proveedor con compras rompería el historial de gastos que HU-15 consulta.

R-02 · Un proveedor inactivo no admite compras nuevas

HU-13 CA-01 valida que el proveedor exista. Se añade la verificación de que esté **activo**: desactivar sirve para dejar de comprarle sin perder el historial.

Las compras ya registradas siguen su curso, incluidas las `PENDIENTE` de recibir. Desactivar un proveedor no cancela lo que ya se le pidió.

R-03 · El correo es opcional pero, si viene, debe ser válido

`@Email` en Bean Validation acepta `null` y cadena vacía sin protestar; solo valida cuando hay contenido. Es el comportamiento correcto para un campo opcional. **No hay** `UNIQUE`: dos sucursales del mismo proveedor pueden compartir correo.

R-04 · Diferencia con clientes

Proveedores tiene `correo` y clientes no. Es la única diferencia estructural entre ambas tablas; el resto del CRUD es idéntico a HU-05 y se implementa copiando ese patrón.

4. Modelo de datos

Tabla **proveedores** (sin cambios; V2 añade `idx_proveedores_nombre`)

Columna	Tipo	Notas
<code>id_proveedor</code>	<code>int unsigned</code> PK AI	
<code>nombre</code>	<code>varchar(100)</code> NOT NULL	CA-01
<code>contacto</code>	<code>varchar(100)</code> NULL	CA-02
<code>telefono</code>	<code>varchar(20)</code> NULL	CA-02
<code>correo</code>	<code>varchar(100)</code> NULL	CA-02, R-03
<code>estado</code>	<code>tinyint(1)</code> NOT NULL DEFAULT 1	CA-03

```
// POST /api/v1/proveedores
{ "nombre": "Harinas del Caribe S.A.S.", "contacto": "Jorge Mendoza",
  "telefono": "605 278 4410", "correo": "ventas@harinascaribe.com" }

// 201 Created
{ "idProveedor": 3, "nombre": "Harinas del Caribe S.A.S.", "contacto": "Jorge Mendoza",
  "telefono": "605 278 4410", "correo": "ventas@harinascaribe.com", "activo": true }
```

Campos ordenables: `nombre` , `id` .

5. Códigos de error

code	HTTP	Cuándo
<code>PROVEEDOR_INACTIVO</code>	409	Se intenta registrar una compra a un proveedor desactivado (validado en HU-13)

Los transversales (`VALIDACION_FALLIDA` , `NO_AUTENTICADO` , `SIN_PERMISO` , `RECURSO_NO_ENCONTRADO`) están en `00-base/04-CONTRATO-API.md` §5.

6. Fuera de alcance

Lo que **no** cubre esta historia y no debe implementarse aquí: se limita estrictamente a los criterios de aceptación listados arriba. Cualquier funcionalidad adicional se propone como historia nueva, no se agrega en silencio.

7. Definición de Terminado

Se aplica la DoD de `00-base/01-CONVENCIONES.md` §7.

8. Notas

Sobre el alcance

El documento de Entrega 1 excluía explícitamente la gestión de proveedores (§3.2 Limitaciones). Se incluye porque **sin compras no entra materia prima al inventario** y tres historias más quedarían sin sentido. Justificación completa en `00-base/08-DESVIACIONES-PDF.md` §D-01.

Diseño

1. Pantallas

Listado de proveedores

Proveedores [+ Nuevo proveedor]				
Buscar por nombre		Estado Activos ▾		
NOMBRE ▲	CONTACTO	TELÉFONO	CORREO	...
Distribuidora Lácteos	Ana Mejía	605 277 1100	✉ ventas@...	:
Harinas del Caribe S.A.S.	Jorge Mendoza	605 278 4410	✉ ventas@...	:
Insumos del Sinú	Marta Díaz	310 998 7712	–	:
Elementos por página: 20 ▾ 1–3 de 3 ◀ ▶				

El correo es un enlace `mailto:`. Cuando no hay, se muestra `–`, no una celda vacía: así se distingue “sin dato” de “falló la carga”.

Formulario

Nuevo proveedor [X]

Nombre *

Persona de contacto

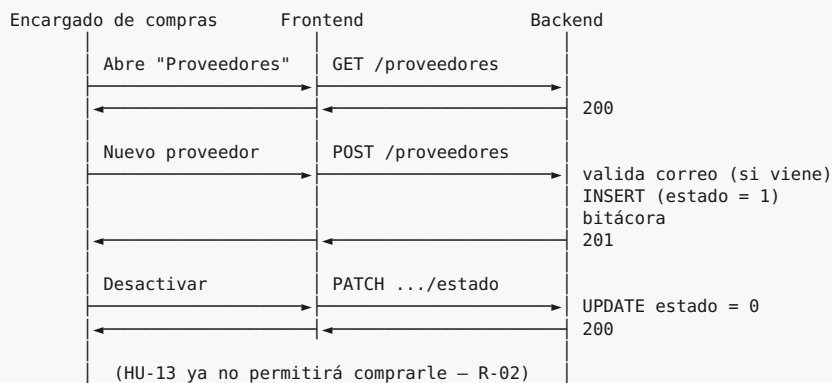
Teléfono

Correo electrónico

[Cancelar] [Guardar]

Solo el nombre lleva asterisco: los demás son opcionales (CA-02).

2. Flujo



3. Componentes Angular Material

Elemento	Componente
Tabla	<code>MatTable</code> + <code>MatSort</code> + <code>MatPaginator</code>
Búsqueda	<code>MatFormField</code> + <code>MatInput</code>
Estado	<code>MatChip</code> verde/gris
Correo	Enlace <code>mailto:</code> con <code>MatIcon</code>
Menú de fila	<code>MatMenu</code>
Formulario	<code>MatDialog</code>
Confirmación	<code>ConfirmarDialogComponent</code>

4. Validaciones en el formulario

Campo	Reglas	Mensaje
Nombre	requerido, máx. 100	"El nombre es obligatorio"
Contacto	opcional, máx. 100	—
Teléfono	opcional, máx. 20	—
Correo	opcional; si se escribe, formato válido, máx. 100	"Ingrese un correo válido"

El correo **no bloquea el guardado si está vacío**. Solo se valida cuando tiene contenido.

5. Estados de la pantalla

Estado	Qué se muestra
Cargando	<code>MatProgressBar</code>
Vacío sin filtro	"No hay proveedores registrados" + "Registrar el primero"
Vacío con filtro	"Ningún proveedor coincide con «xxx»"
Error	Mensaje + "Reintentar"
Guardando	Botón con spinner

6. Accesibilidad y UX

- Todos los campos con `<mat-label>` ; nunca solo *placeholder*.
- Navegación por teclado en orden lógico; `Enter` envía el formulario.
- Los mensajes de error del servidor se muestran **junto al campo** (`fieldErrors`).
- El botón de envío se deshabilita mientras la petición está en vuelo.
- Confirmación antes de descartar cambios sin guardar.
- Los tres estados obligatorios (cargando / vacío / error) según `00-base/05-ESTANDARES-QA.md` §7.

Tarea Backend

CRUD idéntico a HU-05 más el campo `correo` . **Se implementa copiando el patrón de HU-05**, no reinventándolo: si ambos difieren en estructura, mantenerlos cuesta el doble.

1. Pasos

1. Copiar la estructura del módulo `cliente` de HU-05.
2. `Proveedor` (POJO) y `ProveedorRowMapper` — incluye `correo` .
3. `ProveedorRepository` : `buscar` , `contar` , `buscarPorId` , `insertar` , `actualizar` , `cambiarEstado` . Lista blanca de orden: `{nombre, id}` .
4. DTO: `CrearProveedorRequest` (con `@Email` en `correo`), `ActualizarProveedorRequest` , `ProveedorResponse` .
5. `ProveedorService` con `@Transactional` y `@Auditable` .
6. `ProveedorController` con `@PreAuthorize` según la matriz: escritura para ADMIN y COMPRAS.
7. Método `existeActivo(id)` que usará HU-13 para validar R-02.
8. Sin **DELETE** (CA-05).

2. Archivos

```
backend/src/main/java/co/edu/corposucre/productionfood/proveedor/  
├── {Proveedor,ProveedorRowMapper,ProveedorRepository,ProveedorService,ProveedorController}.java  
└── dto/{CrearProveedorRequest,ActualizarProveedorRequest,ProveedorResponse}.java
```

3. Puntos de cuidado

- `@Email` acepta `null` y `""` . Si se quiere rechazar la cadena vacía, normalizarla a `null` en el servicio antes de guardar — así no se distingue entre “sin correo” y “correo vacío” en la base.
- Los mismos cuidados de HU-05: `buscar` / `contar` sincronizados, lista blanca de `sort` , escape de `%` y `_` .
- Exponer `existeActivo()` desde el repositorio: HU-13 lo necesita y no debe consultar la tabla por su cuenta.
- Sin endpoint **DELETE** .

4. Checklist de cierre

- ☐ Todos los CA implementados y verificados manualmente.
- ☐ Endpoints documentados en Swagger con ejemplos.
- ☐ `@PreAuthorize` en cada método, según la matriz de roles.
- ☐ Operaciones de escritura auditadas en bitácora (HU-04).
- ☐ Pruebas unitarias de las reglas con ramificación.
- ☐ Sin `System.out.println` ni código comentado.
- ☐ PR bajo ~250 líneas, revisado y aprobado.

Tarea Frontend

Copia de la pantalla de HU-05 con el campo de correo añadido. Reutiliza `tabla-paginada` y `confirmar-dialog`.

1. Pasos

- `proveedor.model.ts`, `proveedor.service.ts` siguiendo el patrón de HU-05.
- `proveedor-lista.component`: columnas nombre, contacto, teléfono, correo, estado, `:`.
- Búsqueda con `debounceTime(350)` + `switchMap`; filtro por estado (activos por defecto).
- `proveedor-formulario.component` con el campo de correo y validación de formato.
- El correo en la tabla como enlace `mailto:`.
- Confirmación al desactivar: "No se podrán registrar nuevas compras a este proveedor."
- Los tres estados de la pantalla.
- Ruta con `rolGuard(['ADMIN', 'COMPRAS', 'CONSULTA'])`; botones de escritura con `*siRol=['ADMIN', 'COMPRAS']`.

2. Archivos

```
frontend/src/app/features/proveedor/  
├── models/proveedor.model.ts  
├── services/proveedor.service.ts  
├── pages/{proveedor-lista,proveedor-formulario}/  
└── proveedor.routes.ts
```

3. Puntos de cuidado

- El correo es **opcional**: no marcarlo con asterisco ni bloquear el guardado si está vacío. Solo validar el formato cuando tiene contenido.
- Si el correo está vacío, enviar `null`, no `""`.
- En pantallas estrechas, la columna de correo es la primera que se oculta: es la menos crítica para identificar a un proveedor.

4. Checklist de cierre

- ☐ Todos los CA cubiertos desde la interfaz.
- ☐ Los tres estados (cargando / vacío / error) implementados.
- ☐ Errores del servidor mostrados junto al campo correspondiente.
- ☐ Guard de rol aplicado en la ruta.
- ☐ Modelos TypeScript espejo de los DTO del backend.
- ☐ Probado en Chrome y Firefox a 1366×768.
- ☐ Sin `console.log` ni `any`.

Casos de Prueba

Formato y criterios en `00-base/05-ESTANDARES-QA.md`.

1. Casos funcionales

ID	CA	Descripción	Entrada	Resultado esperado	Tipo
CP-01	CA-01, CA-03	Crear con solo el nombre	{"nombre": "Harinas del Caribe"}	201 , activo: true , resto null	Positivo
CP-02	CA-01	Nombre vacío	{"nombre": ""}	400 VALIDACION_FALLIDA	Negativo
CP-03	CA-01	Nombre de 101 caracteres	"A"×101	400	Borde
CP-04	CA-02	Crear con todos los campos	nombre, contacto, teléfono, correo	201 con los cuatro	Positivo
CP-05	CA-02	Correo con formato inválido	{"correo": "no-es-correo"}	400 , error en correo	Negativo
CP-06	CA-02	Correo omitido	Sin la clave correo	201 , correo: null	Borde
CP-07	CA-02	Correo como cadena vacía	{"correo": ""}	201 ; se guarda null (R-03)	Borde
CP-08	CA-02	Dos proveedores con el mismo correo	Repetir correo	201 — no hay UNIQUE (R-03)	Borde
CP-09	CA-03	Estado por defecto	Crear sin estado	activo: true	Positivo
CP-10	CA-04	Listar paginado	?page=0&size=5	200 , máx. 5	Positivo
CP-11	CA-04	Filtrar por nombre	?nombre=Hari	Solo los que empiezan por "Hari"	Positivo
CP-12	CA-04	Comodín en el filtro	?nombre=%	No devuelve todo	Seguridad
CP-13	CA-04	Orden fuera de lista blanca	?sort=correo,asc	400 CAMPO_ORDEN_INVALIDO	Seguridad
CP-14	—	Editar proveedor	PUT con datos nuevos	200	Positivo
CP-15	—	Editar inexistente	PUT /proveedores/9999	404	Negativo
CP-16	CA-05	Desactivar	PATCH .../estado	200 , estado = 0 , la fila permanece	Positivo
CP-17	CA-05	No existe DELETE	DELETE /proveedores/3	405 o 404	Negativo
CP-18	—	Rol VENTAS intenta crear	Token de VENTAS	403 SIN_PERMISO	Seguridad
CP-19	—	Rol COMPRAS puede crear	Token de COMPRAS	201	Positivo
CP-20	—	Inyección SQL en el filtro	?nombre=' OR 1=1 -	Sin error 500	Seguridad

2. Batería de seguridad

Obligatoria en **cada endpoint** de esta historia:

ID	Caso	Esperado
SEC-01	Sin cabecera Authorization	401 NO_AUTENTICADO
SEC-02	Token malformado	401 NO_AUTENTICADO
SEC-03	Token expirado	401 NO_AUTENTICADO
SEC-04	Token de rol sin permiso	403 SIN_PERMISO
SEC-05	Token de usuario con estado = 0	403 USUARIO_INACTIVO

SEC-04 es el caso crítico. Si devuelve 200 , falta @EnableMethodSecurity y la autorización del sistema completo está desactivada.

3. Verificación en base de datos

```
-- CP-07: el correo vacío se guarda como NULL, no como cadena vacía
SELECT id_proveedor, nombre, correo FROM proveedores WHERE correo = '';
-- Debe devolver 0 filas

-- CP-16/CP-17: desactivación lógica
SELECT COUNT(*) FROM proveedores; -- comparar antes/después

-- El índice de V2 se usa en el filtro
EXPLAIN SELECT id_proveedor, nombre FROM proveedores WHERE nombre LIKE 'Hari%';
-- Se espera: key = idx_proveedores_nombre
```

4. Criterios de cierre

- ☐ Todos los casos ejecutados; los positivos pasan.
- ☐ Los negativos devuelven el HTTP y el `code` esperados.
- ☐ SEC-01 a SEC-05 pasan en todos los endpoints.
- ☐ Ningún endpoint supera 2 s (RNF-02).
- ☐ Cero defectos críticos o altos abiertos.
- ☐ Colección de Postman actualizada y versionada.

HU-07 · Unidades de medida

Especificación

Fase 2 · Fuente: `previo/Historias_de_Usuario.md`

Como administrador, **quiero** CRUD de unidades de medida (nombre, abreviatura), **para que** estandarice las cantidades en todo el sistema.

Depende de: HU-03

1. Criterios de aceptación

CA	Enunciado	Cómo se implementa
CA-01	Nombre y abreviatura obligatorios	<code>@NotBlank</code> en ambos
CA-02	La abreviatura debe ser única	<code>UNIQUE KEY abreviatura</code> ya existe en el esquema; validación previa en el servicio
CA-03	Listar todas las unidades disponibles	<code>GET /unidades-medida</code> — sin paginación

2. Endpoints

Método	Ruta	Descripción	Roles
<code>GET</code>	<code>/api/v1/unidades-medida</code>	Listar todas	Todos los roles autenticados
<code>GET</code>	<code>/api/v1/unidades-medida/{id}</code>	Obtener una	Todos los roles autenticados
<code>POST</code>	<code>/api/v1/unidades-medida</code>	Crear	ADMIN
<code>PUT</code>	<code>/api/v1/unidades-medida/{id}</code>	Editar	ADMIN

Reglas transversales (paginación, formato de error, autenticación) en `00-base/04-CONTRATO-API.md`.

3. Reglas de negocio

R-01 · Sin paginación, y es deliberado (CA-03)

Una panadería maneja entre 5 y 15 unidades de medida. Paginar un catálogo de ese tamaño añade complejidad al cliente sin beneficio, y además **este endpoint alimenta los desplegables** de HU-08, HU-11 y HU-12: un selector paginado es una mala experiencia.

Devuelve la lista completa ordenada por nombre. Es la única excepción a la regla de paginación de `04-CONTRATO-API.md` §4, y por eso se declara explícitamente.

R-02 · La abreviatura se normaliza antes de comparar

Se recorta y se guarda tal como se escribe, respetando mayúsculas y minúsculas: `kg` y `KG` se muestran distinto.

Pero la collation `utf8mb4_0900_ai_ci` es *case-insensitive*: el `UNIQUE` considera `kg` y `KG` **la misma abreviatura**, y la segunda es rechazada. Es el comportamiento correcto —tener ambas sería confuso— pero conviene saberlo, porque el mensaje de error (“la abreviatura ya existe”) puede desconcertar a quien acaba de escribir algo que en pantalla se ve diferente.

R-03 · Lo que impide eliminar una unidad

`materias_primas.id_unidad`, `productos.id_unidad` y `recetas.id_unidad` la referencian. Borrar una unidad en uso rompería la integridad referencial.

Esta tabla no tiene columna `estado`, así que tampoco se puede desactivar. Por eso **no se implementa ningún `DELETE`**: una unidad creada por error se corrige editándola.

Se documenta como limitación consciente. Agregar una columna `estado` sería lo correcto si el catálogo creciera, pero no lo justifica un conjunto de quince filas.

R-04 · No se cambia el significado de una unidad en uso

Editar “Kilogramo/kg” para convertirlo en “Litro/L” **reinterpretaría silenciosamente todos los registros existentes**: las materias primas que decían 50 kg pasarían a decir 50 L, sin que nada cambie en sus datos.

El servicio verifica si la unidad está en uso antes de permitir editar la abreviatura, y la interfaz advierte: “Esta unidad se usa en 12 materias primas y 5 productos.”

R-05 · Catálogo inicial

V3_datos_semilla.sql crea nueve unidades: kg, g, L, mL, und, doc, paq, bol, bdj. Cubren el caso de una panadería sin necesidad de configuración previa.

4. Modelo de datos

Tabla unidades_medida (sin cambios)

Columna	Tipo	Notas
id_unidad	int unsigned PK AI	
nombre	varchar(50) NOT NULL	CA-01
abreviatura	varchar(50) NOT NULL UNIQUE	CA-01, CA-02

Sin columna estado — de ahí R-03.

```
// GET /api/v1/unidades-medida
[
  { "idUnidad": 9, "nombre": "Bandeja", "abreviatura": "bdj" },
  { "idUnidad": 8, "nombre": "Bolsa", "abreviatura": "bol" },
  { "idUnidad": 6, "nombre": "Docena", "abreviatura": "doc" },
  { "idUnidad": 2, "nombre": "Gramo", "abreviatura": "g" },
  { "idUnidad": 1, "nombre": "Kilogramo", "abreviatura": "kg" },
  { "idUnidad": 3, "nombre": "Litro", "abreviatura": "L" }
]
```

Respuesta como array plano, no PageResponse (R-01).

```
// POST /api/v1/unidades-medida
{ "nombre": "Media docena", "abreviatura": "1/2doc" }
```

5. Códigos de error

code	HTTP	Cuándo
ABREVIATURA_DUPLICADA	409	Ya existe una unidad con esa abreviatura (ignorando mayúsculas)
UNIDAD_EN_USO	409	Se intenta cambiar la abreviatura de una unidad referenciada

Los transversales (VALIDACION_FALLIDA , NO_AUTENTICADO , SIN_PERMISO , RECURSO_NO_ENCONTRADO) están en 00-base/04-CONTRATO-API.md §5.

6. Fuera de alcance

Lo que **no** cubre esta historia y no debe implementarse aquí: se limita estrictamente a los criterios de aceptación listados arriba. Cualquier funcionalidad adicional se propone como historia nueva, no se agrega en silencio.

7. Definición de Terminado

Se aplica la DoD de 00-base/01-CONVENCIONES.md §7.

8. Notas

Por qué esta historia va primero en la Fase 2

HU-08 CA-02 y HU-11 CA-04 exigen que id_unidad exista. Sin unidades registradas **no se puede crear ni una materia prima ni un producto**.

El 00-base/10-ROADMAP.md la pone antes que HU-08 y HU-11 por esta razón. La semilla V3 la deja resuelta desde el primer

arranque, pero la pantalla debe existir para poder agregar unidades que el negocio necesite.

Diseño

1. Pantallas

Catálogo de unidades

Unidades de medida [+ Nueva unidad]			
NOMBRE ▲	ABREVIATURA	EN USO	...
Bandeja	bdj	3 registros	:
Bolsa	bol	1 registro	:
Docena	doc	4 registros	:
Gramo	g	8 registros	:
Kilogramo	kg	17 registros	:
Litro	L	5 registros	:
Mililitro	mL	2 registros	:
Paquete	paq	– sin uso	:
Unidad	und	12 registros	:

Sin paginador (R-01): el catálogo completo cabe en pantalla. El menú  solo ofrece **Editar** — no hay eliminar (R-03).

La columna “En uso” hace visible qué unidades sostienen datos reales, y por tanto cuáles son delicadas de modificar.

Advertencia al editar una unidad en uso

▲ Esta unidad está en uso

«Kilogramo (kg)» se usa en:

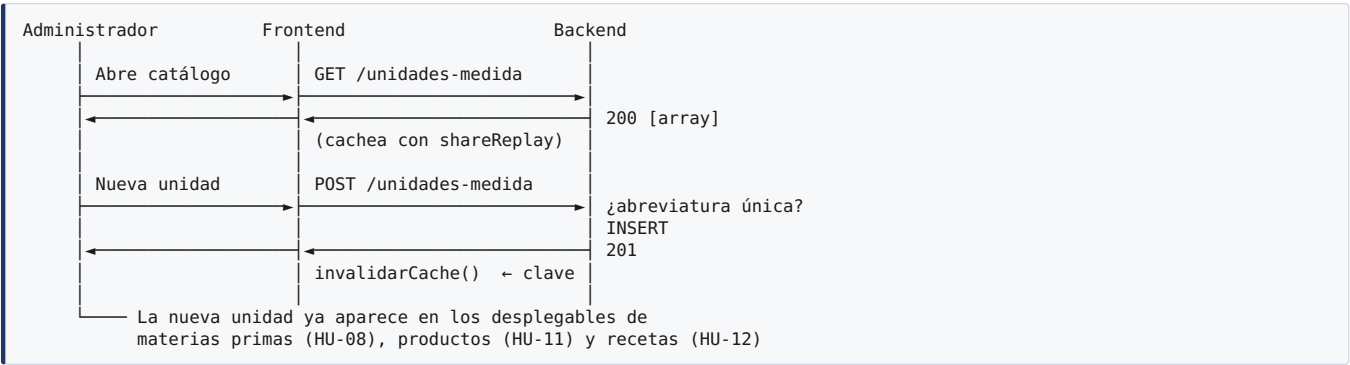
- 12 materias primas
- 5 productos

Cambiar la abreviatura modificará cómo se interpretan esas cantidades. Un registro de «50 kg» pasará a leerse como «50 L» sin que su valor cambie.

[Cancelar] [Entiendo, cambiar]

El texto explica la consecuencia concreta con un ejemplo. Es la diferencia entre una advertencia que se lee y una que se cierra por reflejo.

2. Flujo



Sin `invalidarCache()`, la unidad recién creada no aparece en los otros módulos hasta recargar la página, y el usuario concluye que la creación falló.

3. Componentes Angular Material

Elemento	Componente
Tabla	<code>MatTable</code> <code>sin</code> <code>MatPaginator</code>
Formulario	<code>MatDialog</code>
Campos	<code>MatFormField</code> + <code>MatInput</code>
Menú de fila	<code>MatMenu</code> (solo Editar)
Advertencia de uso	<code>MatDialog</code> con <code>MatIcon</code> <code>warning</code>
En uso	<code>MatChip</code> o texto atenuado si es “sin uso”

4. Validaciones en el formulario

Campo	Reglas	Mensaje
Nombre	requerido, máx. 50	“El nombre es obligatorio”
Abreviatura	requerida, máx. 50, sin espacios al inicio o final	“La abreviatura es obligatoria”

La unidad de la abreviatura **solo puede verificarla el backend** (CA-02). El cliente puede avisar si coincide con alguna de la lista ya cargada, pero la autoridad es el `409 ABREVIATURA_DUPLICADA`.

5. Estados de la pantalla

Estado	Qué se muestra
Cargando	<code>MatProgressBar</code>
Vacío	“No hay unidades registradas” + “Crear la primera”. Poco probable: la semilla V3 deja nueve
Error	Mensaje + “Reintentar”
Guardando	Botón con spinner

En los otros módulos, un desplegable de unidades vacío debe mostrar un enlace a esta pantalla: sin unidades no se puede crear una materia prima ni un producto, y conviene que el camino a la solución sea evidente.

6. Accesibilidad y UX

- Todos los campos con `<mat-label>`; nunca solo *placeholder*.
- Navegación por teclado en orden lógico; `Enter` envía el formulario.
- Los mensajes de error del servidor se muestran **junto al campo** (`fieldErrors`).
- El botón de envío se deshabilita mientras la petición está en vuelo.
- Confirmación antes de descartar cambios sin guardar.
- Los tres estados obligatorios (cargando / vacío / error) según `00-base/05-ESTANDARES-QA.md` §7.

Tarea Backend

El CRUD más simple del proyecto, con dos particularidades: no se pagina y no se elimina. Alimenta los desplegables de HU-08, HU-11 y HU-12.

1. Pasos

1. `UnidadMedida` (POJO) y `UnidadMedidaRowMapper`.
2. `UnidadMedidaRepository`:
 - `listarTodas()` → `ORDER BY nombre`, `sin LIMIT` (R-01);
 - `buscarPorId()`, `insertar()`, `actualizar()`;
 - `existePorAbreviatura(abrev)` y `existePorAbreviaturaExcluyendo(abrev, id)`;
 - `contarUsos(idUnidad)` → suma de referencias en `materias_primas`, `productos` y `recetas` (R-04).
3. DTO: `CrearUnidadRequest`, `ActualizarUnidadRequest`, `UnidadMedidaResponse` y `UnidadEnUsoResponse` (con el conteo por tabla).
4. `UnidadMedidaService`:
 - `crear()` valida abreviatura no duplicada → `ABREVIATURA_DUPLICADA`;
 - `actualizar()` valida excluyendo la propia; si cambia la abreviatura y `contarUsos() > 0`, exige confirmación explícita (parámetro `confirmar=true`) o devuelve `409 UNIDAD_EN_USO` (R-04);
 - captura `DuplicateKeyException` por si hay concurrencia.
5. `UnidadMedidaController`: `GET` para todos los roles autenticados, escritura solo ADMIN.

6. `@Auditable` en crear y editar.
7. Sin `DELETE` (R-03).

2. Archivos

```
backend/src/main/java/co/edu/corposucre/productionfood/unidadmedida/  
├── {UnidadMedida,UnidadMedidaRowMapper,UnidadMedidaRepository,...}.java  
└── dto/{CrearUnidadRequest,ActualizarUnidadRequest,UnidadMedidaResponse}.java
```

3. Puntos de cuidado

- **Devolver un array plano, no `PageResponse`** (R-01). Es la excepción documentada a la regla de paginación.
- **`GET` abierto a todos los roles autenticados:** PRODUCCION, COMPRAS y VENTAS necesitan el catálogo para sus formularios aunque no puedan editarlo.
- El `UNIQUE` con collation `_ai_ci` es *case-insensitive*: `kg` y `KG` colisionan (R-02). El mensaje de error debe ser claro al respecto.
- **R-04 no es un capricho:** cambiar la abreviatura de una unidad en uso reinterpreta todos los registros que la referencian sin tocar un solo dato de esas tablas.
- Sin `DELETE`. Si alguien lo pide, la respuesta está en R-03.

4. Checklist de cierre

- ☐ Todos los CA implementados y verificados manualmente.
- ☐ Endpoints documentados en Swagger con ejemplos.
- ☐ `@PreAuthorize` en cada método, según la matriz de roles.
- ☐ Operaciones de escritura auditadas en bitácora (HU-04).
- ☐ Pruebas unitarias de las reglas con ramificación.
- ☐ Sin `System.out.println` ni código comentado.
- ☐ PR bajo ~250 líneas, revisado y aprobado.

Tarea Frontend

Pantalla de catálogo simple, sin paginación. Además, el servicio se **cachea** porque lo consumen los formularios de varios módulos.

1. Pasos

1. `unidad-medida.model.ts` y `unidad-medida.service.ts`.
2. **Cachear el catálogo** con `shareReplay(1)`: HU-08, HU-11 y HU-12 lo piden en cada apertura de formulario. Sin caché se descarga la misma lista una y otra vez.
3. Método `invalidarCache()` que se llama tras crear o editar una unidad.
4. `unidad-lista.component`: tabla simple sin paginador, ordenada por nombre.
5. Columna “En uso” con el número de referencias, para que se vea qué unidades importan.
6. `unidad-formulario.component` en `MatDialog`.
7. Al editar una unidad en uso, mostrar advertencia clara: “Esta unidad se usa en 12 materias primas y 5 productos. Cambiar su abreviatura modificará cómo se interpretan esos registros.” — con confirmación explícita.
8. **Sin acción de eliminar** (R-03).
9. Ruta: lectura para todos los roles; botones de escritura con `*siRol=["ADMIN"]`.

2. Archivos

```
frontend/src/app/features/unidad-medida/  
├── models/unidad-medida.model.ts  
├── services/unidad-medida.service.ts ← con caché shareReplay(1)  
├── pages/{unidad-lista,unidad-formulario}/  
└── unidad-medida.routes.ts
```

3. Puntos de cuidado

- **La caché debe invalidarse al crear o editar.** Si no, el usuario agrega una unidad y no la ve en el desplegable de materias primas hasta recargar la página — y concluye que la creación falló.

- Sin `MatPaginator` en esta pantalla (R-01).
- La advertencia de R-04 debe decir **qué va a pasar**, no solo “¿está seguro?”.
- El desplegable en otros módulos muestra `"Kilogramo (kg)"`: solo el nombre es ambiguo entre unidades parecidas, y solo la abreviatura es críptica.

4. Checklist de cierre

- ☐ Todos los CA cubiertos desde la interfaz.
- ☐ Los tres estados (cargando / vacío / error) implementados.
- ☐ Errores del servidor mostrados junto al campo correspondiente.
- ☐ Guard de rol aplicado en la ruta.
- ☐ Modelos TypeScript espejo de los DTO del backend.
- ☐ Probado en Chrome y Firefox a 1366×768.
- ☐ Sin `console.log` ni `any`.

Casos de Prueba

Formato y criterios en `00-base/05-ESTANDARES-QA.md`.

1. Casos funcionales

ID	CA	Descripción	Entrada	Resultado esperado	Tipo
CP-01	CA-03	Listar todas	<code>GET /unidades-medida</code>	200 con array plano; incluye las 9 de la semilla	Positivo
CP-02	CA-03	La respuesta NO es paginada	Revisar el JSON de CP-01	Es un array, sin <code>content</code> ni <code>totalElements</code>	Positivo
CP-03	CA-03	Orden alfabético	Revisar CP-01	Ordenado por nombre ascendente	Positivo
CP-04	CA-01	Crear unidad válida	<code>{"nombre":"Media docena","abreviatura":"1/2doc"}</code>	201 con <code>idUnidad</code>	Positivo
CP-05	CA-01	Nombre vacío	<code>{"nombre":"","abreviatura":"x"}</code>	400 <code>VALIDACION_FALLIDA</code>	Negativo
CP-06	CA-01	Abreviatura vacía	<code>{"nombre":"Caja","abreviatura":""}</code>	400	Negativo
CP-07	CA-02	Abreviatura duplicada	<code>{"nombre":"Kilo","abreviatura":"kg"}</code>	409 <code>ABREVIATURA_DUPLICADA</code>	Negativo
CP-08	CA-02	Duplicada con otra capitalización	<code>{"abreviatura":"KG"}</code> existiendo <code>kg</code>	409 — la collation es case-insensitive (R-02)	Borde
CP-09	CA-02	Duplicada con espacios alrededor	<code>{"abreviatura":" kg "}</code>	409 tras recortar	Borde
CP-10	—	Editar sin cambiar la abreviatura	<code>PUT</code> con la misma abreviatura	200 — no debe dar duplicado contra sí misma	Borde
CP-11	—	Editar el nombre de una unidad en uso	Cambiar solo <code>nombre</code>	200	Positivo
CP-12	—	Cambiar la abreviatura de una unidad en uso	<code>PUT</code> sin <code>confirmar</code>	409 <code>UNIDAD_EN_USO</code> con el conteo de referencias	Negativo
CP-13	—	No existe <code>DELETE</code>	<code>DELETE /unidades-medida/1</code>	405 o 404 (R-03)	Negativo
CP-14	—	Rol PRODUCCION puede listar	<code>GET</code> con token PRODUCCION	200	Positivo
CP-15	—	Rol PRODUCCION no puede crear	<code>POST</code> con token PRODUCCION	403 <code>SIN_PERMISO</code>	Seguridad
CP-16	—	Rol CONSULTA puede listar	<code>GET</code> con token CONSULTA	200	Positivo
CP-17	—	Unidad inexistente	<code>GET /unidades-medida/9999</code>	404	Negativo

2. Batería de seguridad

Obligatoria en **cada endpoint** de esta historia:

ID	Caso	Esperado
SEC-01	Sin cabecera <code>Authorization</code>	<code>401 NO_AUTENTICADO</code>
SEC-02	Token malformado	<code>401 NO_AUTENTICADO</code>
SEC-03	Token expirado	<code>401 NO_AUTENTICADO</code>
SEC-04	Token de rol <code>sin</code> permiso	<code>403 SIN_PERMISO</code>
SEC-05	Token de usuario con <code>estado = 0</code>	<code>403 USUARIO_INACTIVO</code>

SEC-04 es el caso crítico. Si devuelve `200`, falta `@EnableMethodSecurity` y la autorización del sistema completo está desactivada.

CP-08 sorprende si no se conoce la collation. `utf8mb4_0900_ai_ci` es *accent-insensitive* y *case-insensitive*, así que el `UNIQUE` trata `kg` y `KG` como la misma abreviatura. El comportamiento es el deseado —tener ambas sería confuso— pero el mensaje de error debe explicarlo, porque en pantalla los dos valores se ven distintos.

3. Verificación en base de datos

```
-- CP-01: la semilla V3 debe haber dejado 9 unidades
SELECT COUNT(*) FROM unidades_medida; -- 9 (más las creadas en pruebas)

-- CP-08: el UNIQUE con collation _ai_ci es case-insensitive
SHOW CREATE TABLE unidades_medida\G
-- Se espera: UNIQUE KEY `abreviatura` (`abreviatura`)
SELECT 'kg' = 'KG' COLLATE utf8mb4_0900_ai_ci; -- devuelve 1

-- CP-12: contar las referencias antes de editar
SELECT
  (SELECT COUNT(*) FROM materias_primas WHERE id_unidad = 1) AS en_materias,
  (SELECT COUNT(*) FROM productos WHERE id_unidad = 1) AS en_productos,
  (SELECT COUNT(*) FROM recetas WHERE id_unidad = 1) AS en_recetas;

-- CP-13: ninguna unidad debe desaparecer
SELECT COUNT(*) FROM unidades_medida;
```

4. Criterios de cierre

- ☐ Todos los casos ejecutados; los positivos pasan.
- ☐ Los negativos devuelven el HTTP y el `code` esperados.
- ☐ SEC-01 a SEC-05 pasan en todos los endpoints.
- ☐ Ningún endpoint supera 2 s (RNF-02).
- ☐ Cero defectos críticos o altos abiertos.
- ☐ Colección de Postman actualizada y versionada.

Fase 3 · Inventario de Materias Primas

HU-08 · Gestión de materias primas

Especificación

Fase 3 · Fuente: `previo/Historias_de_Usuario.md`

Como administrador, **quiero** CRUD de materias primas (nombre, unidad, stock mínimo, costo unitario, estado), **para que** conozca los ingredientes disponibles.

Depende de: HU-07 (unidades de medida)

1. Criterios de aceptación

CA	Enunciado	Cómo se implementa
CA-01	Nombre obligatorio, máx. 100	<code>@NotBlank</code> + <code>@Size(max=100)</code>
CA-02	<code>id_unidad</code> debe existir en <code>unidades_medida</code>	Validado en el servicio contra <code>UnidadMedidaRepository</code>
CA-03	<code>stock_minimo</code> y <code>costo_unitario</code> obligatorios (decimales)	<code>@NotNull</code> + <code>@DecimalMin("0.00")</code> ; <code>BigDecimal</code>
CA-04	Estado por defecto activo (1)	<code>DEFAULT 1</code> añadido en V2 (§C-06) — el esquema original no lo tenía
CA-05	Listar con paginación y filtrar por nombre	<code>GET /materias-primas?nombre=&page=&size=</code>

2. Endpoints

Método	Ruta	Descripción	Roles
<code>GET</code>	<code>/api/v1/materias-primas</code>	Listar con filtro y paginación	ADMIN, PRODUCCION, COMPRAS, CONSULTA
<code>GET</code>	<code>/api/v1/materias-primas/{id}</code>	Obtener una	ADMIN, PRODUCCION, COMPRAS, CONSULTA
<code>POST</code>	<code>/api/v1/materias-primas</code>	Registrar	ADMIN
<code>PUT</code>	<code>/api/v1/materias-primas/{id}</code>	Editar	ADMIN
<code>PATCH</code>	<code>/api/v1/materias-primas/{id}/estado</code>	Activar o desactivar	ADMIN

Reglas transversales (paginación, formato de error, autenticación) en `00-base/04-CONTRATO-API.md` .

3. Reglas de negocio

R-01 · Al crear una materia prima se crea su fila de inventario

Esta es la regla más importante de la historia. La tabla `inventario` tiene `UNIQUE (id_materia_prima)` : hay exactamente una fila por materia prima, o ninguna.

Si no se crea al dar de alta la materia prima, ocurre lo siguiente: - HU-10 la muestra sin cantidad, o la omite del listado; - HU-27 no puede detectar que está bajo mínimo, porque no hay fila que comparar; - HU-23 falla al intentar descontar de una fila inexistente.

Al crear una materia prima, en la misma transacción, se inserta `inventario (id_materia_prima, cantidad_disponible = 0)` .

La vista `v_inventario_materia_prima` usa `LEFT JOIN` y `COALESCE(...,0)` como red de seguridad, pero eso corrige la *consulta*, no el problema: sin fila real no hay dónde sumar cuando llegue la primera compra.

R-02 · `stock_minimo` y `costo_unitario` con `BigDecimal` , nunca `double`

Un `double` no representa exactamente `0.1` . Acumular costos con `double` produce diferencias de centavos que aparecen como descuadres imposibles de explicar (`04-CONTRATO-API.md` §6).

R-03 · `costo_unitario` es un valor de referencia, no el costo real

El costo real de cada compra está en `detalle_compra.precio_unitario` (HU-14), que varía entre proveedores y fechas. Este campo

sirve para estimar el costo de una receta antes de comprar.

No se actualiza solo al registrar una compra: hacerlo sobrescribiría el valor de referencia con el precio puntual de la última compra, que puede ser atípico.

R-04 · Desactivar no borra el stock

Una materia prima inactiva conserva su inventario. Desactivar significa “no comprar más ni usar en recetas nuevas”, no “desapareció del almacén”. HU-10 debe poder mostrarla si aún tiene existencias.

R-05 · No se elimina

`inventario`, `lotes`, `detalle_compra`, `detalle_receta` y `consumo_materia_prima` la referencian. Solo se desactiva.

R-06 · `stock_minimo` puede ser cero

Significa “no alertar por esta materia prima”. Es válido para insumos que se compran bajo pedido. El `CHECK (stock_minimo >= 0)` de V2 impide negativos, que no tendrían sentido.

4. Modelo de datos

Tabla `materias_primas`

Columna	Tipo	Notas
<code>id_materia_prima</code>	<code>int unsigned</code> PK AI	
<code>nombre</code>	<code>varchar(100)</code> NOT NULL	CA-01
<code>id_unidad</code>	<code>int unsigned</code> NOT NULL FK	CA-02
<code>stock_minimo</code>	<code>decimal(10,2)</code> NOT NULL	CA-03 · <code>CHECK >= 0</code> (V2)
<code>costo_unitario</code>	<code>decimal(10,2)</code> NOT NULL	CA-03 · <code>CHECK >= 0</code> (V2)
<code>estado</code>	<code>tinyint(1)</code> NOT NULL DEFAULT 1	CA-04 · corregido en V2 §C-06

Tabla `inventario` — se crea una fila por cada materia prima (R-01)

Columna	Tipo	Notas
<code>id_inventario</code>	<code>int unsigned</code> PK AI	
<code>id_materia_prima</code>	<code>int unsigned</code> NOT NULL UNIQUE FK	Una fila por materia prima
<code>cantidad_disponible</code>	<code>decimal(10,2)</code> NOT NULL	<code>CHECK >= 0</code> (V2)
<code>fecha_actualizacion</code>	<code>datetime</code> NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE	V2 §C-07

```
// POST /api/v1/materias-primas
{
  "nombre": "Harina de trigo", "idUnidad": 1,
  "stockMinimo": 50.00, "costoUnitario": 3200.00 }

// 201 Created
{
  "idMateriaPrima": 5, "nombre": "Harina de trigo",
  "unidad": { "idUnidad": 1, "nombre": "Kilogramo", "abreviatura": "kg" },
  "stockMinimo": 50.00, "costoUnitario": 3200.00, "activo": true,
  "cantidadDisponible": 0.00 }
```

Campos ordenables: `nombre`, `stockMinimo`, `costoUnitario`, `id`.

5. Códigos de error

code	HTTP	Cuándo
<code>UNIDAD_INEXISTENTE</code>	409	El <code>idUnidad</code> no existe en <code>unidades_medida</code>
<code>MATERIA_PRIMA_EN_USO</code>	409	Se intenta desactivar una materia prima usada en recetas activas (advertencia)

Los transversales (`VALIDACION_FALLIDA`, `NO_AUTENTICADO`, `SIN_PERMISO`, `RECURSO_NO_ENCONTRADO`) están en `00-base/04-CONTRATO-API.md` §5.

6. Fuera de alcance

Lo que **no** cubre esta historia y no debe implementarse aquí: se limita estrictamente a los criterios de aceptación listados arriba. Cualquier funcionalidad adicional se propone como historia nueva, no se agrega en silencio.

7. Definición de Terminado

Se aplica la DoD de 00-base/01-CONVENCIONES.md §7.

8. Notas

El **DEFAULT 1** de **estado** es una corrección, no algo que ya estuviera

El esquema original declaraba `estado tinyint(1) NOT NULL` **sin valor por defecto**, pese a que CA-04 exige que las nuevas materias primas nazcan activas.

Sin la corrección de V2, un **INSERT** que omita la columna falla. Y como **clientes** y **proveedores** **sí** tenían el **DEFAULT**, el comportamiento era inconsistente entre tablas casi idénticas — el tipo de detalle que produce un error desconcertante en la cuarta pantalla, cuando las tres anteriores funcionaron.

Diseño

1. Pantallas

Listado

Materias primas

[+ Nueva materia prima]

Buscar por nombre

Estado

Activas ▾

NOMBRE ▲	UNIDAD	STOCK MÍN.	DISPONIBLE	COSTO UNIT.	ESTADO	...
Azúcar	kg	30,00	45,00	\$ 2.800,00	●Activa :	
Harina de trigo	kg	50,00	▲ 12,50	\$ 3.200,00	●Activa :	
Levadura	g	500,00	0,00	\$ 180,00	●Activa :	
Mantequilla	kg	20,00	35,00	\$ 18.500,00	●Activa :	

Elementos por página: 20 ▾ 1–4 de 4 ◀ ▶

- ▲ ámbar: `disponible < stock mínimo`
- rojo: `disponible = 0` (agotada)

Mostrar el disponible junto al mínimo convierte el listado en una herramienta de trabajo: se ve de un vistazo qué hay que comprar, sin ir a otra pantalla.

Formulario

Nueva materia prima

[X]

Nombre *

Harina de trigo

Unidad de medida *

Kilogramo (kg) ▾

Stock mínimo *

50,00 kg

Costo unitario *

\$ 3.200,00 /kg

Se alertará por debajo de esta cantidad

Valor de referencia para costear recetas

[Cancelar]

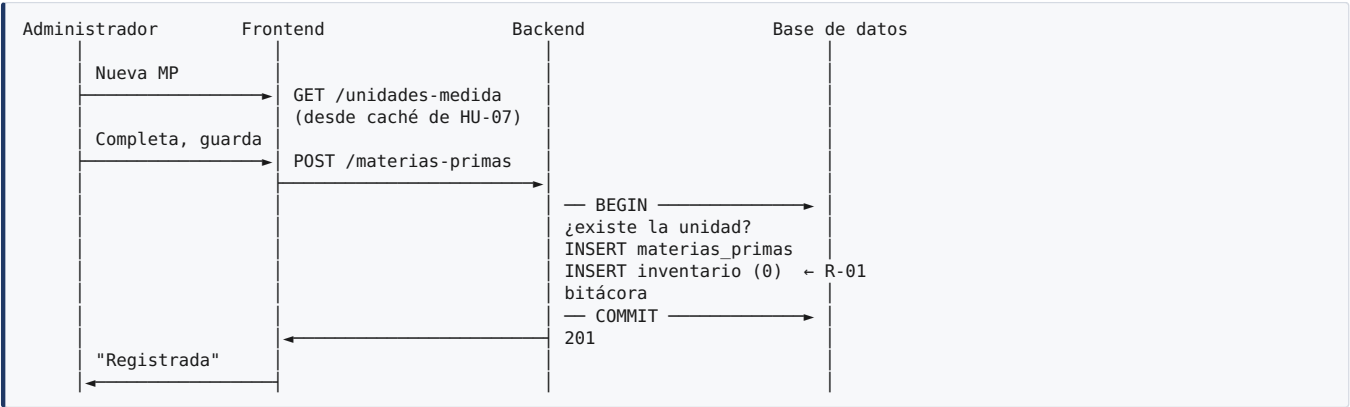
[Guardar]

El sufijo de unidad se actualiza al cambiar el selector. Sin él, es fácil teclear **50** pensando en kilos cuando la unidad configurada

es gramos.

El texto bajo “costo unitario” aclara R-03: no es el precio de compra, es una referencia.

2. Flujo



Las dos inserciones van en la misma transacción. Si falla la del inventario, tampoco se crea la materia prima: es preferible que el alta falle a que quede una materia prima inservible para el resto del sistema.

3. Componentes Angular Material

Elemento	Componente
Tabla	<code>MatTable</code> + <code>MatSort</code> + <code>MatPaginator</code>
Alerta de stock	<code>MatIcon</code> <code>warning</code> / <code>error</code> con color
Unidad	<code>MatSelect</code> (catálogo cacheado de HU-07)
Campos numéricos	<code>MatInput[type=number]</code> + <code>MatSuffix</code> con la unidad
Moneda	<code>MatPrefix</code> <code>\$</code> + pipe <code>currency: 'COP'</code>
Ayuda contextual	<code>MatHint</code> bajo cada campo numérico
Formulario	<code>MatDialog</code>

4. Validaciones en el formulario

Campo	Reglas	Mensaje
Nombre	requerido, máx. 100	“El nombre es obligatorio”
Unidad	requerida	“Seleccione una unidad de medida”
Stock mínimo	requerido, ≥ 0 , máx. 2 decimales	“Debe ser 0 o mayor”
Costo unitario	requerido, ≥ 0 , máx. 2 decimales	“Debe ser 0 o mayor”

Cero es válido en ambos campos numéricos (R-06). El validador debe ser `min(0)`, no `min(0.01)`.

5. Estados de la pantalla

Estado	Qué se muestra
Cargando	<code>MatProgressBar</code>
Vacío sin filtro	“No hay materias primas registradas” + “Registrar la primera”
Vacío con filtro	“Ninguna coincide con «xxx»”
Sin unidades disponibles	“Primero debe registrar unidades de medida” + enlace a HU-07
Error	Mensaje + “Reintentar”

El caso “sin unidades” importa: sin ellas el formulario no puede completarse, y el usuario necesita saber a dónde ir en lugar de quedarse ante un desplegable vacío.

6. Accesibilidad y UX

- Todos los campos con `<mat-label>`; nunca solo *placeholder*.

- Navegación por teclado en orden lógico; **Enter** envía el formulario.
- Los mensajes de error del servidor se muestran **junto al campo** (`fieldErrors`).
- El botón de envío se deshabilita mientras la petición está en vuelo.
- Confirmación antes de descartar cambios sin guardar.
- Los tres estados obligatorios (cargando / vacío / error) según `00-base/05-ESTANDARES-QA.md` §7.

Tarea Backend

CRUD siguiendo el patrón de HU-05, con dos añadidos importantes: validación de FK contra `unidades_medida` y **creación de la fila de inventario** en la misma transacción (R-01).

1. Pasos

1. `MateriaPrima` (POJO) y `MateriaPrimaRowMapper` con `JOIN unidades_medida` para devolver nombre y abreviatura.
2. `MateriaPrimaRepository`: `buscar`, `contar`, `buscarPorId`, `insertar`, `actualizar`, `cambiarEstado`. Lista blanca: `{nombre, stockMinimo, costoUnitario, id}`.
3. `InventarioRepository.crearFilaInicial(idMateriaPrima)` — `INSERT` con `cantidad_disponible = 0`.
4. DTO con `BigDecimal` en `stockMinimo` y `costoUnitario` (R-02), nunca `double`.
5. `MateriaPrimaService.crear()` `@Transactional`:
 - valida que la unidad exista → `UNIDAD_INEXISTENTE`;
 - inserta la materia prima;
 - **inserta la fila de inventario en cero** (R-01);
 - si algo falla, se revierte todo.
6. `MateriaPrimaService.cambiarEstado()`: al desactivar, consultar si está en recetas activas y devolver el conteo como advertencia (R-04).
7. Controlador con `@PreAuthorize`: lectura amplia, escritura solo ADMIN.
8. `@Auditable` en las escrituras.
9. Método `existeActiva(id)` para que lo usen HU-12, HU-14 y HU-22.

2. Archivos

```
backend/src/main/java/co/edu/corposucre/productionfood/
├── materiaprima/
│   ├── {MateriaPrima,MateriaPrimaRowMapper,MateriaPrimaRepository,...}.java
│   └── dto/{CrearMateriaPrimaRequest,ActualizarMateriaPrimaRequest,MateriaPrimaResponse}.java
└── inventario/
    └── InventarioRepository.java      ← crearFilaInicial()
```

3. Puntos de cuidado

- **R-01 es el punto crítico.** Sin la fila de inventario, HU-10, HU-23 y HU-27 fallan o muestran datos incompletos, y el síntoma aparece tres sprints después, lejos de la causa. Debe ir **en la misma transacción** que el alta.
- `BigDecimal`, no `double` (R-02). Revisar los DTO y el `RowMapper`: `rs.getBigDecimal()`, no `rs.getDouble()`.
- Validar la FK **en el servicio** para devolver `409 UNIDAD_INEXISTENTE` legible, no esperar al error de integridad de la base.
- `stock_minimo = 0` es válido (R-06): no rechazarlo.
- Sin `DELETE` (R-05).

4. Checklist de cierre

- ☐ Todos los CA implementados y verificados manualmente.
- ☐ Endpoints documentados en Swagger con ejemplos.
- ☐ `@PreAuthorize` en cada método, según la matriz de roles.
- ☐ Operaciones de escritura auditadas en bitácora (HU-04).
- ☐ Pruebas unitarias de las reglas con ramificación.
- ☐ Sin `System.out.println` ni código comentado.
- ☐ PR bajo ~250 líneas, revisado y aprobado.
- ☐ **Verificado: crear una materia prima deja su fila en `inventario` con cantidad 0.**

Tarea Frontend

Listado y formulario siguiendo el patrón de HU-05, con selector de unidad y campos numéricos.

1. Pasos

1. `materia-prima.model.ts` y `materia-prima.service.ts`.
 2. `materia-prima-lista.component`: columnas nombre, unidad, stock mínimo, costo unitario, **disponible**, estado, .
 3. Indicador visual cuando `cantidadDisponible < stockMinimo` (ícono de alerta ámbar) — anticipa HU-27 y hace útil el listado.
 4. Búsqueda con `debounceTime(350)` + `switchMap`; filtro por estado.
 5. `materia-prima-formulario.component`:
 - selector de unidad alimentado por el servicio cacheado de HU-07;
 - campos numéricos con sufijo de unidad dinámico (“kg” junto a stock mínimo);
 - costo con formato de moneda colombiana.
 6. Si el catálogo de unidades está vacío, mostrar enlace a la pantalla de HU-07.
 7. Los tres estados.
 8. Ruta: lectura ADMIN/PRODUCCION/COMPRAS/CONSULTA; escritura solo ADMIN.
-

2. Archivos

```
frontend/src/app/features/materia-prima/  
├── models/materia-prima.model.ts  
├── services/materia-prima.service.ts  
├── pages/{materia-prima-lista,materia-prima-formulario}/  
└── materia-prima.routes.ts
```

3. Puntos de cuidado

- **Los decimales se envían como `number` de JSON**, sin separador de miles. El formato `$ 3.200,00` es solo de presentación: lo que viaja es `3200.00`.
 - El sufijo de unidad junto a “stock mínimo” evita el error de teclear 50 pensando en kilos cuando la unidad configurada es gramos.
 - `stockMinimo = 0` es válido: no marcarlo como error (R-06).
 - El indicador de stock bajo se calcula con los datos que ya devuelve el listado; no hace falta una petición extra.
-

4. Checklist de cierre

- ☐ Todos los CA cubiertos desde la interfaz.
- ☐ Los tres estados (cargando / vacío / error) implementados.
- ☐ Errores del servidor mostrados junto al campo correspondiente.
- ☐ Guard de rol aplicado en la ruta.
- ☐ Modelos TypeScript espejo de los DTO del backend.
- ☐ Probado en Chrome y Firefox a 1366×768.
- ☐ Sin `console.log` ni `any`.

Casos de Prueba

Formato y criterios en `00-base/05-ESTANDARES-QA.md`.

1. Casos funcionales

ID	CA	Descripción	Entrada	Resultado esperado	Tipo
CP-01	CA-01, CA-04	Crear materia prima válida	<code>{"nombre":"Harina de trigo","idUnidad":1,"stockMinimo":50,"costoUnitario":3200}</code>	201, activo: true, cantidadDisponible: 0	Positivo
CP-02	—	Se crea la fila de inventario	Tras CP-01, consultar <code>inventario</code>	Existe fila con <code>cantidad_disponible = 0.00</code> (R-01)	Integridad
CP-03	CA-01	Nombre vacío	<code>{"nombre":""}</code>	400 VALIDACION_FALLIDA	Negativo
CP-04	CA-01	Nombre de 101 caracteres	<code>"A"×101</code>	400	Borde
CP-05	CA-02	Unidad inexistente	<code>{"idUnidad":9999}</code>	409 UNIDAD_INEXISTENTE	Negativo
CP-06	CA-02	Unidad omitida	Sin <code>idUnidad</code>	400	Negativo
CP-07	CA-03	Stock mínimo omitido	Sin <code>stockMinimo</code>	400	Negativo
CP-08	CA-03	Stock mínimo negativo	<code>{"stockMinimo":-5}</code>	400 o 409 — CHECK >= 0 de V2	Negativo
CP-09	CA-03	Stock mínimo cero	<code>{"stockMinimo":0}</code>	201 — es válido (R-06)	Borde
CP-10	CA-03	Costo con 2 decimales	<code>{"costoUnitario":3200.55}</code>	201 ; en base 3200.55 exacto	Positivo
CP-11	CA-03	Costo con 3 decimales	<code>{"costoUnitario":3200.555}</code>	Se redondea a 3200.56 (decimal(10,2))	Borde
CP-12	CA-04	Estado por defecto	Crear sin <code>estado</code>	activo: true ; en base estado = 1	Positivo
CP-13	CA-05	Listar paginado	<code>?page=0&size=5</code>	200 , máx. 5	Positivo
CP-14	CA-05	Filtrar por nombre	<code>?nombre=Hari</code>	Solo los que empiezan por "Hari"	Positivo
CP-15	CA-05	Orden por costo	<code>?sort=costoUnitario,desc</code>	Ordenado descendente	Positivo
CP-16	CA-05	Orden fuera de lista blanca	<code>?sort=estado,asc</code>	400 CAMPO_ORDEN_INVALIDO	Seguridad
CP-17	—	Editar materia prima	PUT con datos nuevos	200	Positivo
CP-18	—	Desactivar conserva el stock	Desactivar una con inventario > 0	200 ; cantidad_disponible no cambia (R-04)	Integridad
CP-19	—	No existe DELETE	<code>DELETE /materias-primas/5</code>	405 o 404	Negativo
CP-20	—	Rol COMPRAS puede listar	GET con token COMPRAS	200	Positivo
CP-21	—	Rol COMPRAS no puede crear	POST con token COMPRAS	403 SIN_PERMISO	Seguridad
CP-22	—	Rol PRODUCCION no puede crear	POST con token PRODUCCION	403 SIN_PERMISO	Seguridad

2. Batería de seguridad

Obligatoria en **cada endpoint** de esta historia:

ID	Caso	Esperado
SEC-01	Sin cabecera <code>Authorization</code>	401 NO_AUTENTICADO
SEC-02	Token malformado	401 NO_AUTENTICADO
SEC-03	Token expirado	401 NO_AUTENTICADO
SEC-04	Token de rol sin permiso	403 SIN_PERMISO
SEC-05	Token de usuario con <code>estado = 0</code>	403 USUARIO_INACTIVO

SEC-04 es el caso crítico. Si devuelve 200, falta `@EnableMethodSecurity` y la autorización del sistema completo está desactivada.

CP-02 es el caso crítico de esta historia y conviene ejecutarlo tras cada alta durante todo el proyecto. Una materia prima sin fila de inventario no da ningún error al crearse: el defecto aparece semanas después, cuando HU-23 intenta descontar de una fila que no existe o HU-27 no detecta un desabastecimiento. La consulta de conciliación de arriba lo detecta en un segundo.

3. Verificación en base de datos

```
-- CP-02: el caso que más importa de esta historia.
-- Toda materia prima DEBE tener su fila de inventario (R-01).
SELECT mp.id_materia_prima, mp.nombre, i.cantidad_disponible
FROM materias_primas mp
LEFT JOIN inventario i ON i.id_materia_prima = mp.id_materia_prima
WHERE i.id_inventario IS NULL;
-- DEBE devolver 0 filas. Cualquier fila aquí es un defecto crítico:
-- esa materia prima romperá HU-10, HU-23 y HU-27.

-- CP-08: el CHECK de V2 debe estar activo
INSERT INTO materias_primas (nombre, id_unidad, stock_minimo, costo_unitario)
VALUES ('Prueba', 1, -5, 100);
-- DEBE fallar con error 3819

-- CP-12: el DEFAULT 1 añadido en V2 §C-06
SHOW CREATE TABLE materias_primas\G
-- Se espera: `estado` tinyint(1) NOT NULL DEFAULT '1'

-- CP-18: desactivar no toca el inventario
SELECT mp.estado, i.cantidad_disponible
FROM materias_primas mp JOIN inventario i USING (id_materia_prima)
WHERE mp.id_materia_prima = 5;
```

4. Criterios de cierre

- ☐ Todos los casos ejecutados; los positivos pasan.
- ☐ Los negativos devuelven el HTTP y el `code` esperados.
- ☐ SEC-01 a SEC-05 pasan en todos los endpoints.
- ☐ Ningún endpoint supera 2 s (RNF-02).
- ☐ Cero defectos críticos o altos abiertos.
- ☐ Colección de Postman actualizada y versionada.

HU-09 · Lotes de materia prima

Especificación

Fase 3 · Fuente: `previo/Historias_de_Usuario.md`

Como administrador, **quiero** registrar lotes de materia prima con código, fechas de producción/vencimiento y cantidad, **para** que controle la vida útil de los ingredientes.

Depende de: HU-08

1. Criterios de aceptación

CA	Enunciado	Cómo se implementa
CA-01	<code>codigo_lote</code> obligatorio y único , máx. 30	<code>UNIQUE KEY codigo_lote</code> ya existe; validación previa en el servicio
CA-02	<code>id_materia_prima</code> debe existir	Validado contra <code>MateriaPrimaRepository</code>
CA-03	<code>fecha_produccion</code> y <code>fecha_vencimiento</code> obligatorias	<code>@NotNull</code> ; <code>CHECK (vencimiento >= produccion)</code> añadido en V2 §C-10
CA-04	<code>cantidad</code> obligatoria (decimal)	<code>@NotNull</code> + <code>@DecimalMin</code> positivo; <code>CHECK > 0</code> en V2
CA-05	Listar lotes por materia prima	<code>GET /lotes-materia-prima?idMateriaPrima=</code>

2. Endpoints

Método	Ruta	Descripción	Roles
GET	<code>/api/v1/lotes-materia-prima</code>	Listar con filtros y paginación	ADMIN, PRODUCCION, COMPRAS, CONSULTA
GET	<code>/api/v1/lotes-materia-prima/{id}</code>	Obtener uno	ADMIN, PRODUCCION, COMPRAS, CONSULTA
POST	<code>/api/v1/lotes-materia-prima</code>	Registrar un lote	ADMIN, PRODUCCION, COMPRAS

Reglas transversales (paginación, formato de error, autenticación) en `00-base/04-CONTRATO-API.md`.

3. Reglas de negocio

R-01 · Registrar un lote suma al inventario

Es uno de los dos puntos de entrada de stock del sistema (`02-CORRECCIONES-DB.md` §C-03). La operación es **transaccional** y hace tres cosas:

- `INSERT` en `lotes` ;
- `UPDATE inventario` sumando la cantidad;
- `INSERT` en `movimientos_inventario_mp` con tipo `ENTRADA_LOTE`.

Las tres o ninguna. Toda la suma pasa por `InventarioService.registrarEntrada()` ; ningún otro componente ejecuta `UPDATE inventario`.

R-02 · ⚠ El riesgo de la doble suma

El otro punto de entrada es la recepción de compra (HU-13). **Si se recibe una compra de 100 kg de harina y después se registra un lote por esos mismos 100 kg, el inventario marcará 200 kg.** El error no produce ningún mensaje: simplemente el sistema cree que hay el doble de harina de la que hay.

Mitigación, en dos partes:

- V2 añadió `lotes.id_detalle_compra` (§C-03). **Un lote creado desde la recepción de una compra lleva esa referencia y NO vuelve a sumar:** el stock ya entró con la recepción.
- Un lote registrado manualmente (sin `idDetalleCompra`) **sí suma**. Es la vía para inventario inicial o ajustes de entrada.

La interfaz debe dejar claro cuál de los dos casos se está usando. Es el punto donde más fácilmente se corrompe el inventario, y el más difícil de detectar después.

R-03 · Un lote es de materia prima o de producto terminado, nunca ambos

La tabla `lotes` es compartida con HU-19. El `CHECK ck_lotes_exclusividad` de V2 (§C-10) impide los estados imposibles que el esquema original permitía: ambos nulos o ambos llenos.

En esta historia `id_materia_prima` va informado e `id_producto` va `NULL`.

R-04 · La fecha de vencimiento no puede ser anterior a la de producción

`CHECK (fecha_vencimiento >= fecha_produccion)` en V2. Se permite que sean iguales: hay insumos de un solo día.

R-05 · Se permite registrar lotes ya vencidos

No se bloquea: puede ser necesario registrar inventario existente que ya caducó, para darlo de baja formalmente. **Pero se marca visualmente** y HU-24 CA-05 los reporta.

Un lote vencido de materia prima **sí suma al inventario**, porque físicamente está en la bodega. Darlo de baja es una decisión explícita, no automática.

R-06 · El código de lote se normaliza

Se recorta y se guarda en mayúsculas. La collation es *case-insensitive*, así que `h-2026-001` y `H-2026-001` colisionarían de todos modos; normalizar evita que la lista se vea desordenada.

4. Modelo de datos

Tabla `lotes` (compartida con HU-19; V2 añade `id_detalle_compra` y dos CHECK)

Columna	Tipo	Notas
<code>id_lote</code>	<code>int unsigned</code> PK AI	
<code>codigo_lote</code>	<code>varchar(30)</code> NOT NULL UNIQUE	CA-01
<code>id_producto</code>	<code>int unsigned</code> NULL	NULL en esta historia (R-03)
<code>id_materia_prima</code>	<code>int unsigned</code> NULL	Informado aquí
<code>id_detalle_compra</code>	<code>int unsigned</code> NULL	Añadido en V2 — evita la doble suma (R-02)
<code>fecha_produccion</code>	<code>date</code> NOT NULL	CA-03
<code>fecha_vencimiento</code>	<code>date</code> NOT NULL	CA-03 · <code>CHECK >= produccion</code>
<code>cantidad</code>	<code>decimal(10,2)</code> NOT NULL	CA-04 · <code>CHECK > 0</code>

Tabla `movimientos_inventario_mp` (creada en V2 §C-04) — recibe el movimiento `ENTRADA_LOTE`.

```
// POST /api/v1/lotes-materia-prima
{
  "codigoLote": "H-2026-0912", "idMateriaPrima": 5,
  "fechaProduccion": "2026-09-10", "fechaVencimiento": "2027-03-10",
  "cantidad": 100.00 }

// 201 Created
{
  "idLote": 14, "codigoLote": "H-2026-0912",
  "materiaPrima": { "idMateriaPrima": 5, "nombre": "Harina de trigo", "unidad": "kg" },
  "fechaProduccion": "2026-09-10", "fechaVencimiento": "2027-03-10",
  "cantidad": 100.00, "diasParaVencer": 173, "vencido": false,
  "inventarioActualizado": { "anterior": 12.50, "posterior": 112.50 } }
```

Devolver el saldo anterior y posterior hace visible el efecto sobre el inventario en el momento, en lugar de obligar a ir a otra pantalla a comprobarlo.

5. Códigos de error

code	HTTP	Cuándo
<code>CODIGO_LOTE_DUPLICADO</code>	409	Ya existe un lote con ese código
<code>MATERIA_PRIMA_INEXISTENTE</code>	409	El <code>idMateriaPrima</code> no existe
<code>FECHAS_INVALIDAS</code>	400	<code>fechaVencimiento</code> anterior a <code>fechaProduccion</code>
<code>CANTIDAD_INVALIDA</code>	400	<code>cantidad</code> menor o igual a cero

Los transversales (`VALIDACION_FALLIDA`, `NO_AUTENTICADO`, `SIN_PERMISO`, `RECURSO_NO_ENCONTRADO`) están en `00-base/04-CONTRATO-API.md`

6. Fuera de alcance

Lo que **no** cubre esta historia y no debe implementarse aquí: se limita estrictamente a los criterios de aceptación listados arriba. Cualquier funcionalidad adicional se propone como historia nueva, no se agrega en silencio.

7. Definición de Terminado

Se aplica la DoD de 00-base/01-CONVENCIONES.md §7.

8. Notas

Sobre cantidad : el lote no lleva saldo propio

lotes.cantidad es la cantidad **con la que el lote ingresó**, y no se modifica después. El saldo disponible de materia prima vive en inventario , agregado por materia prima.

Esto significa que **no hay trazabilidad por lote en el consumo** de materias primas: al producir (HU-22) se descuenta del total, no de un lote concreto. Para producto terminado sí la hay, porque inventario_producto_terminado es por lote.

Es una limitación del esquema original, no un descuido de esta historia. Se registra por si la trazabilidad completa se vuelve un requisito: implicaría llevar saldo por lote de materia prima y consumir en orden FEFO, como ya se hace con el producto terminado.

Diseño

1. Pantallas

Listado de lotes

Lotes de materia prima

[+ Nuevo lote]

Materia prima

Todas

☐ Solo vencidos

☐ Vencen en 7 días

CÓDIGO	MATERIA PRIMA	PRODUCCIÓN	VENCIMIENTO	CANTIDAD	ESTADO
H-2026-0912	Harina de trigo	10/09/2026	10/03/2027	100,00 kg	●
LV-2026-0905	Levadura	05/09/2026	25/09/2026	500,00 g	● 7d
MQ-2026-0820	Mantequilla	20/08/2026	15/09/2026	20,00 kg	
AZ-2026-0901	Azúcar	01/09/2026	01/09/2027	50,00 kg	●

vencido · ● vence en ≤ 7 días · ○ ≤ 30 días · ● vigente

Los lotes vencidos **se muestran** (R-05): siguen físicamente en la bodega hasta que alguien decida darlos de baja.

Formulario — con el impacto en inventario visible

Nuevo lote de materia prima

[X]

Código de lote *

H-2026-0912

Materia prima *

Harina de trigo (kg)

Fecha producción *

10/09/2026

Fecha vencimiento *

10/03/2027

Cantidad *

100,00

kg

i Impacto en el inventario

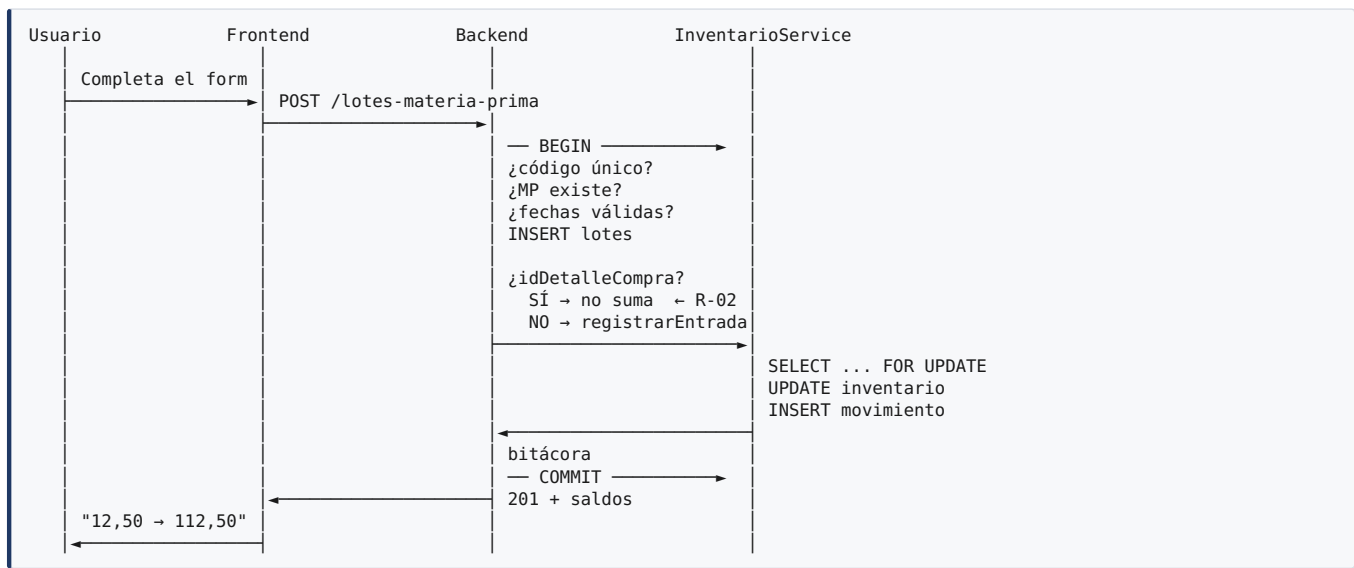
Harina de trigo: 12,50 kg → 112,50 kg

[Cancelar]

[Registrar lote]

El recuadro de impacto se actualiza en vivo. Es lo que evita que alguien registre un lote de una compra que ya sumó, sin darse cuenta (R-02).

2. Flujo



3. Componentes Angular Material

Elemento	Componente
Tabla	<code>MatTable</code> + <code>MatSort</code> + <code>MatPaginator</code>
Semáforo de vencimiento	<code>MatChip</code> con color + días restantes
Fechas	<code>MatDatepicker</code> (el de vencimiento con <code>min</code> atado a producción)
Materia prima	<code>MatSelect</code>
Cantidad	<code>MatInput[type=number]</code> + <code>MatSuffix</code> con la unidad
Impacto en inventario	<code>MatCard</code> destacada, actualizada en vivo
Filtros booleanos	<code>MatCheckbox</code>

4. Validaciones en el formulario

Campo	Reglas	Mensaje
Código de lote	requerido, máx. 30	"El código es obligatorio"
Materia prima	requerida	"Seleccione una materia prima"
Fecha producción	requerida	"Indique la fecha de producción"
Fecha vencimiento	requerida, ≥ producción	"No puede ser anterior a la producción"
Cantidad	requerida, > 0	"Debe ser mayor que cero"

La unicidad del código solo la verifica el backend. Una fecha de vencimiento en el pasado genera advertencia, no error (R-05).

5. Estados de la pantalla

Estado	Qué se muestra
Cargando	MatProgressBar
Vacío sin filtro	"No hay lotes registrados" + "Registrar el primero"
Vacío con filtro	"Ningún lote coincide con los filtros"
Vencimiento pasado	Advertencia ámbar en el formulario, sin bloquear
Error	Mensaje + "Reintentar"
Guardado	Confirmación con el saldo anterior y posterior

6. Accesibilidad y UX

- Todos los campos con `<mat-label>` ; nunca solo *placeholder*.
- Navegación por teclado en orden lógico; `Enter` envía el formulario.
- Los mensajes de error del servidor se muestran **junto al campo** (`fieldErrors`).
- El botón de envío se deshabilita mientras la petición está en vuelo.
- Confirmación antes de descartar cambios sin guardar.
- Los tres estados obligatorios (cargando / vacío / error) según `00-base/05-ESTANDARES-QA.md` §7.

Tarea Backend

Primera historia que suma stock. Aquí nace `InventarioService` , el componente más delicado del proyecto: lo consumirán HU-13, HU-17, HU-18, HU-22 y HU-23. Se escribe una vez, con pruebas, antes de que cinco módulos dependan de él.

1. Pasos

`InventarioService` (se crea aquí, lo usa todo el sistema)

1. `InventarioService.registrarEntrada(idMateriaPrima, cantidad, tipo, tablaOrigen, idOrigen, observacion)` :
 - `SELECT ... FOR UPDATE` sobre la fila de inventario (bloqueo);
 - calcula `saldoAnterior` y `saldoPosterior` ;
 - `UPDATE inventario` ;
 - `INSERT` en `movimientos_inventario_mp` ;
 - **todo dentro de la transacción del llamador.**
2. `InventarioService.registrarSalida(...)` — simétrico, con validación de saldo suficiente.
3. **Regla de oro:** ningún otro componente ejecuta `UPDATE inventario` . Si aparece un `UPDATE inventario` fuera de esta clase, es un defecto.

Módulo `lote`

4. `Lote` (POJO), `LoteRowMapper` con `JOIN materias_primas` y `unidades_medida` .
5. `LoteRepository` : `insertar` , `buscarPorId` , `existeCodigoLote(codigo)` , `buscarMateriaPrima(filtros, page)` , `contar(...)` .
6. DTO: `CrearLoteMateriaPrimaRequest` , `LoteMateriaPrimaResponse` (con `diasParaVencer` , `vencido` y los saldos).
7. `LoteService.crearLoteMateriaPrima()` `@Transactional` :
 - normaliza el código a mayúsculas (R-06);
 - valida código único → `CODIGO_LOTE_DUPLICADO` ;
 - valida materia prima existente → `MATERIA_PRIMA_INEXISTENTE` ;
 - valida fechas → `FECHAS_INVALIDAS` ;
 - `INSERT` en `lotes` con `id_producto = NULL` (R-03);

- si `idDetalleCompra` es `null`, llama a `registrarEntrada()` (R-01 y R-02); si viene informado, **no suma** — el stock ya entró con la recepción.
8. Filtros del listado: `idMateriaPrima`, `soloVencidos`, `proximosAVencer` (días).
 9. `@Auditable(accion="CREAR", tabla="lotes")`.

2. Archivos

```
backend/src/main/java/co/edu/corposucre/productionfood/
├── inventario/
│   ├── InventarioService.java           ← componente central
│   ├── InventarioRepository.java       (+ bloquearYObtener, actualizarSaldo)
│   ├── MovimientoInventarioRepository.java
│   └── TipoMovimiento.java             (enum)
├── lote/
│   ├── {Lote,LoteRowMapper,LoteRepository,LoteService,LoteController}.java
│   └── dto/{CrearLoteMateriaPrimaRequest,LoteMateriaPrimaResponse}.java
```

3. Puntos de cuidado

- **R-02, la doble suma, es el defecto más costoso posible aquí.** Un lote con `idDetalleCompra` informado **no debe sumar**: el stock ya entró con la recepción de la compra. Sumar dos veces no produce ningún error visible; el inventario simplemente miente.
- **`InventarioService` es el único que hace `UPDATE inventario`.** Establecer esta regla ahora evita que cinco módulos la incumplan después.
- **`SELECT ... FOR UPDATE` obligatorio:** sin bloqueo, dos entradas simultáneas sobre la misma materia prima pierden una de las dos sumas.
- **El movimiento de kardex va en la misma transacción que el `UPDATE`.** Si se separan, el saldo y el kardex divergen y la conciliación de `05-ESTANDARES-QA.md` §5 empieza a fallar sin causa aparente.
- El **`CHECK`** de fechas ya está en la base, pero validar en el servicio da un **400** legible en vez de un error de integridad.

4. Checklist de cierre

- ☐ Todos los CA implementados y verificados manualmente.
- ☐ Endpoints documentados en Swagger con ejemplos.
- ☐ `@PreAuthorize` en cada método, según la matriz de roles.
- ☐ Operaciones de escritura auditadas en bitácora (HU-04).
- ☐ Pruebas unitarias de las reglas con ramificación.
- ☐ Sin `System.out.println` ni código comentado.
- ☐ PR bajo ~250 líneas, revisado y aprobado.
- ☐ **`InventarioService` con pruebas unitarias de entrada, salida y saldo insuficiente.**
- ☐ **Verificado: un lote con `idDetalleCompra` NO suma al inventario.**

Tarea Frontend

Listado de lotes con indicadores de vencimiento y formulario de registro. La pantalla debe dejar claro que registrar un lote **modifica el inventario**.

1. Pasos

1. `lote.model.ts` y `lote-materia-prima.service.ts`.
2. `lote-mp-lista.component`: código, materia prima, producción, vencimiento, cantidad, estado de vencimiento.
3. Semáforo de vencimiento: rojo vencido · ámbar ≤ 7 días · gris ≤ 30 días · verde resto.
4. Filtros: materia prima (`MatSelect`), solo vencidos, próximos a vencer.
5. `lote-mp-formulario.component`:
 - `MatDatePicker` para ambas fechas, con `min` de vencimiento atado a producción;
 - **aviso visible**: “Registrar este lote sumará 100,00 kg al inventario de Harina de trigo” — calculado en vivo al completar el formulario;
 - advertencia si la fecha de vencimiento ya pasó (R-05), sin bloquear.
6. Tras guardar, mostrar el saldo resultante: “Inventario actualizado: 12,50 → 112,50 kg”.
7. Los tres estados.
8. Ruta: lectura ADMIN/PRODUCCION/COMPRAS/CONSULTA; escritura ADMIN/PRODUCCION/COMPRAS.

2. Archivos

```
frontend/src/app/features/lote/  
├── models/lote.model.ts  
├── services/lote-materia-prima.service.ts  
├── pages/{lote-mp-lista,lote-mp-formulario}/  
└── lote.routes.ts
```

3. Puntos de cuidado

- **El aviso de impacto en inventario no es decorativo.** Registrar un lote cambia el stock; si la pantalla no lo dice, el usuario no relaciona una acción con la otra y acaba duplicando entradas (R-02).
- El `min` del datepicker de vencimiento debe atarse a la fecha de producción seleccionada: evita el `400 FECHAS_INVALIDAS` antes de enviarlo.
- Los lotes vencidos se muestran, no se ocultan (R-05): están físicamente en la bodega.
- Mostrar el saldo anterior y posterior tras guardar cierra el ciclo: el usuario ve el efecto sin ir a otra pantalla.

4. Checklist de cierre

- ☐ Todos los CA cubiertos desde la interfaz.
- ☐ Los tres estados (cargando / vacío / error) implementados.
- ☐ Errores del servidor mostrados junto al campo correspondiente.
- ☐ Guard de rol aplicado en la ruta.
- ☐ Modelos TypeScript espejo de los DTO del backend.
- ☐ Probado en Chrome y Firefox a 1366×768.
- ☐ Sin `console.log` ni `any`.

Casos de Prueba

Formato y criterios en `00-base/05-ESTANDARES-QA.md`.

1. Casos funcionales

ID	CA	Descripción	Entrada	Resultado esperado	Tipo
CP-01	CA-01..04	Registrar lote válido	<code>{"codigoLote":"H-2026-0912","idMateriaPrima":5,"fechaProduccion":"2026-09-10","fechaVencimiento":"2027-03-10","cantidad":100}</code>	201 con <code>idLote</code> y saldos	Positivo
CP-02	—	El inventario sube exactamente la cantidad	Saldo antes 12,50 → registrar 100	Saldo después = 112,50 (R-01)	Integridad
CP-03	—	Se registra el movimiento de kardex	Tras CP-01	Fila <code>ENTRADA_LOTE</code> con saldo anterior 12,50 y posterior 112,50	Integridad
CP-04	—	Lote con <code>idDetalleCompra</code> NO suma	Registrar lote asociado a una compra recibida	El inventario no cambia (R-02)	Integridad
CP-05	CA-01	Código duplicado	Repetir <code>codigoLote</code>	409 <code>CODIGO_LOTE_DUPLICADO</code>	Negativo
CP-06	CA-01	Código duplicado con otra capitalización	<code>"h-2026-0912"</code> existiendo <code>"H-2026-0912"</code>	409 — se normaliza a mayúsculas (R-06)	Borde
CP-07	CA-01	Código de 31 caracteres	<code>"A"×31</code>	400	Borde
CP-08	CA-02	Materia prima inexistente	<code>{"idMateriaPrima":9999}</code>	409 <code>MATERIA_PRIMA_INEXISTENTE</code>	Negativo
CP-09	CA-03	Vencimiento anterior a producción	<code>produccion:"2026-09-10"</code> , <code>vencimiento:"2026-09-01"</code>	400 <code>FECHAS_INVALIDAS</code>	Negativo
CP-10	CA-03	Vencimiento igual a producción	Ambas <code>"2026-09-10"</code>	201 — permitido (R-04)	Borde
CP-11	CA-03	Fecha omitida	Sin <code>fechaVencimiento</code>	400	Negativo
CP-12	CA-04	Cantidad cero	<code>{"cantidad":0}</code>	400 <code>CANTIDAD_INVALIDA</code> — <code>CHECK > 0</code> de V2	Negativo
CP-13	CA-04	Cantidad negativa	<code>{"cantidad":-10}</code>	400	Negativo
CP-14	CA-05	Listar por materia prima	<code>?idMateriaPrima=5</code>	Solo lotes de esa materia prima	Positivo
CP-15	—	Lote ya vencido	<code>fechaVencimiento</code> en el pasado	201 con <code>vencido: true</code> (R-05); sí suma al inventario	Borde
CP-16	—	Filtro de próximos a vencer	<code>?proximosAVencer=7</code>	Solo los que vencen en ≤ 7 días	Positivo
CP-17	—	El lote no puede tener producto y materia prima a la vez	Intentar insertar ambos por SQL directo	La base lo rechaza — <code>ck_lotes_exclusividad</code> (R-03)	Integridad
CP-18	—	Transacción: si falla el kardex, no sube el inventario	Provocar fallo en el <code>INSERT</code> del movimiento	El saldo de <code>inventario</code> no cambia	Integridad
CP-19	—	Rol VENTAS no puede registrar	Token de VENTAS	403 <code>SIN_PERMISO</code>	Seguridad
CP-20	—	Rol PRODUCCION puede registrar	Token de PRODUCCION	201	Positivo

2. Batería de seguridad

Obligatoria en **cada endpoint** de esta historia:

ID	Caso	Esperado
SEC-01	Sin cabecera <code>Authorization</code>	401 <code>NO_AUTENTICADO</code>
SEC-02	Token malformado	401 <code>NO_AUTENTICADO</code>
SEC-03	Token expirado	401 <code>NO_AUTENTICADO</code>
SEC-04	Token de rol sin permiso	403 <code>SIN_PERMISO</code>
SEC-05	Token de usuario con <code>estado = 0</code>	403 <code>USUARIO_INACTIVO</code>

SEC-04 es el caso crítico. Si devuelve `200`, falta `@EnableMethodSecurity` y la autorización del sistema completo está desactivada.

CP-04 es el caso más importante de esta historia. Verifica la protección contra la doble suma (R-02): si un lote asociado a una compra ya recibida vuelve a sumar, el inventario marca el doble de existencias y **nada en el sistema lo señala**. El error se descubre cuando alguien va a la bodega a buscar 200 kg de harina y encuentra 100.

3. Verificación en base de datos

```
-- CP-02 y CP-03: el saldo y el kardex deben coincidir SIEMPRE
SELECT cantidad_disponible FROM inventario WHERE id_materia_prima = 5;

SELECT tipo_movimiento, cantidad, saldo_anterior, saldo_posterior, tabla_origen, id_origen
FROM movimientos_inventario_mp
WHERE id_materia_prima = 5 ORDER BY id_movimiento DESC LIMIT 1;
-- ENTRADA_LOTE | 100.000 | 12.50 | 112.50 | lotes | 14
-- El saldo_posterior DEBE ser igual a inventario.cantidad_disponible

-- Conciliación general (05-ESTANDARES-QA.md §5). DEBE devolver 0 filas.
SELECT i.id_materia_prima, i.cantidad_disponible, m.saldo_posterior
FROM inventario i
LEFT JOIN (
    SELECT id_materia_prima, saldo_posterior,
           ROW_NUMBER() OVER (PARTITION BY id_materia_prima ORDER BY id_movimiento DESC) rn
    FROM movimientos_inventario_mp
) m ON m.id_materia_prima = i.id_materia_prima AND m.rn = 1
WHERE i.cantidad_disponible <> COALESCE(m.saldo_posterior, 0);

-- CP-17: el CHECK de exclusividad de V2 §C-10
INSERT INTO lotes (codigo_lote, id_producto, id_materia_prima, fecha_produccion, fecha_vencimiento, cantidad)
VALUES ('X-1', 1, 5, curdate(), curdate(), 10);
-- DEBE fallar con error 3819
```

4. Criterios de cierre

- ☐ Todos los casos ejecutados; los positivos pasan.
- ☐ Los negativos devuelven el HTTP y el `code` esperados.
- ☐ SEC-01 a SEC-05 pasan en todos los endpoints.
- ☐ Ningún endpoint supera 2 s (RNF-02).
- ☐ Cero defectos críticos o altos abiertos.
- ☐ Colección de Postman actualizada y versionada.

HU-10 · Consulta de inventario de MP

Especificación

Fase 3 · Fuente: `previo/Historias_de_Usuario.md`

Como administrador, **quiero** consultar el inventario de materia prima (cantidad disponible, última actualización), **para que** sepa qué tenemos en bodega.

Depende de: HU-08

1. Criterios de aceptación

CA	Enunciado	Cómo se implementa
CA-01	Mostrar cada materia prima con su <code>cantidad_disponible</code>	Vista <code>v_inventario_materia_prima</code> (V2)
CA-02	Mostrar la <code>fecha_actualizacion</code> de cada registro	<code>DEFAULT CURRENT_TIMESTAMP ON UPDATE</code> añadido en V2 §C-07
CA-03	Filtrar por materia prima	<code>?idMateriaPrima=</code>
CA-04	Filtrar materia prima con stock por debajo del mínimo	<code>?soloStockBajo=true</code> — usa <code>estado_stock</code> de la vista

2. Endpoints

Método	Ruta	Descripción	Roles
GET	<code>/api/v1/inventario</code>	Consultar inventario con filtros	ADMIN, PRODUCCION, COMPRAS, CONSULTA
GET	<code>/api/v1/inventario/{idMateriaPrima}</code>	Detalle de una materia prima	ADMIN, PRODUCCION, COMPRAS, CONSULTA
GET	<code>/api/v1/inventario/{idMateriaPrima}/movimientos</code>	Kardex: historial de movimientos	ADMIN, PRODUCCION, COMPRAS

Reglas transversales (paginación, formato de error, autenticación) en `00-base/04-CONTRATO-API.md`.

3. Reglas de negocio

R-01 · Solo lectura

Esta historia **no modifica nada**. El inventario se mueve desde HU-09 (lotes), HU-13 (recepción de compras) y HU-23 (consumo de producción), siempre a través de `InventarioService`. Aquí solo se consulta.

R-02 · Se consulta la vista, no la tabla

`v_inventario_materia_prima` (creada en V2) resuelve en una sola consulta lo que si no requeriría varios `JOIN` y un `CASE` repetido en cada endpoint:

- `LEFT JOIN` con `inventario` y `COALESCE(...,0)`: una materia prima sin fila aparece con cero en lugar de desaparecer del listado;
- `estado_stock` calculado: `AGOTADO · BAJO · OK`;
- `faltante`: cuánto falta para alcanzar el mínimo.

El `LEFT JOIN` es una red de seguridad, **no sustituye a HU-08 R-01**: sin fila real de inventario no hay dónde sumar cuando llegue la primera compra.

R-03 · El kardex es lo que da sentido al saldo

`GET /inventario/{id}/movimientos` devuelve el historial de `movimientos_inventario_mp` (V2 §C-04): qué entró, qué salió, cuándo, quién y por qué documento.

Sin esto, cuando el saldo no cuadre con la bodega no habrá forma de saber qué pasó. La bitácora de HU-04 no sirve para eso: registra “alguien tocó la tabla inventario”, no un libro de movimientos consultable.

R-04 · Se muestran también las materias primas inactivas con stock

Una materia prima desactivada (HU-08 R-04) conserva sus existencias. Ocultarla del inventario haría que ese stock desapareciera del sistema aunque siga en la bodega. Se muestra marcada como inactiva; el filtro por estado permite excluirla.

R-05 • estado_stock y su significado

Valor	Condición	Interpretación
AGOTADO	disponible = 0	No hay nada. Bloquea producción
BAJO	0 < disponible < stock_minimo	Hay que reabastecer (alimenta HU-27)
OK	disponible >= stock_minimo	Normal

Con stock_minimo = 0 (HU-08 R-06), nunca se marca BAJO : es la forma de decir “no alertar por este insumo”.

4. Modelo de datos

Vista v_inventario_materia_prima (creada en V2)

Campo	Origen
id_materia_prima, nombre	materias_primas
unidad	unidades_medida.abreviatura
stock_minimo, costo_unitario, estado	materias_primas
cantidad_disponible	COALESCE(inventario.cantidad_disponible, 0)
fecha_actualizacion	inventario
estado_stock	Calculado: AGOTADO / BAJO / OK
faltante	GREATEST(stock_minimo - disponible, 0)

```
// GET /api/v1/inventario?soloStockBajo=true
{
  "content": [
    {
      "idMateriaPrima": 7, "nombre": "Levadura", "unidad": "g",
      "cantidadDisponible": 0.00, "stockMinimo": 500.00, "faltante": 500.00,
      "estadoStock": "AGOTADO", "activo": true,
      "fechaActualizacion": "2026-09-17T16:40:02" },
    {
      "idMateriaPrima": 5, "nombre": "Harina de trigo", "unidad": "kg",
      "cantidadDisponible": 12.50, "stockMinimo": 50.00, "faltante": 37.50,
      "estadoStock": "BAJO", "activo": true,
      "fechaActualizacion": "2026-09-18T09:12:44" }
  ],
  "page": 0, "size": 20, "totalElements": 2, "totalPages": 1
}

// GET /api/v1/inventario/5/movimientos
{
  "content": [
    {
      "fecha": "2026-09-18T09:12:44", "tipo": "SALIDA_PRODUCCION",
      "cantidad": 100.000, "saldoAnterior": 112.50, "saldoPosterior": 12.50,
      "origen": { "tabla": "consumo_materia_prima", "id": 31 },
      "usuario": "Ismael Batalla" },
    {
      "fecha": "2026-09-12T08:05:11", "tipo": "ENTRADA_LOTE",
      "cantidad": 100.000, "saldoAnterior": 12.50, "saldoPosterior": 112.50,
      "origen": { "tabla": "lotas", "id": 14 },
      "usuario": "Carlos Cochero" }
  ]
}
```

Campos ordenables: nombre, cantidadDisponible, faltante, fechaActualizacion .

5. Códigos de error

code	HTTP	Cuándo
------	------	--------

Los transversales (VALIDACION_FALLIDA , NO_AUTENTICADO , SIN_PERMISO , RECURSO_NO_ENCONTRADO) están en 00-base/04-CONTRATO-API.md \$5.

6. Fuera de alcance

Lo que **no** cubre esta historia y no debe implementarse aquí: se limita estrictamente a los criterios de aceptación listados arriba. Cualquier funcionalidad adicional se propone como historia nueva, no se agrega en silencio.

7. Definición de Terminado

Se aplica la DoD de 00-base/01-CONVENCIONES.md §7.

8. Notas

fecha_actualizacion se mantiene sola

El esquema original declaraba datetime NOT NULL sin valor por defecto, aunque CA-02 pida mostrarla y HU-23 CA-02 exija actualizarla.

V2 (§C-07) la dejó como DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP : se actualiza sola en cada UPDATE , sin que ningún servicio tenga que acordarse. Una fecha que depende de que el programador la asigne en cada ruta de código acaba desactualizada en la ruta que alguien olvidó.

Diseño

1. Pantallas

Inventario de materias primas

Inventario de materias primas

8 materias primas

2 bajo mínimo

1 agotadas

Materia prima

Todas

☒ Solo stock bajo ☐ Incluir inactivas

MATERIA PRIMA	DISPONIBLE	MÍNIMO	NIVEL	FALTA	ACTUALIZADO
Levadura	0,00 g	500,00 g	[]	500,00 g	ayer 16:40
● Harina de trigo	12,50 kg	50,00 kg	[]	37,50 kg	hoy 09:12
● Azúcar	45,00 kg	30,00 kg	[]	—	12/09 08:30
● Mantequilla	35,00 kg	20,00 kg	[]	—	10/09 11:15

La barra muestra disponible / mínimo . Un vistazo basta para saber qué pedir y cuánto: la columna “FALTA” ya trae la resta hecha.

Sin botones de acción: la pantalla es de consulta (R-01).

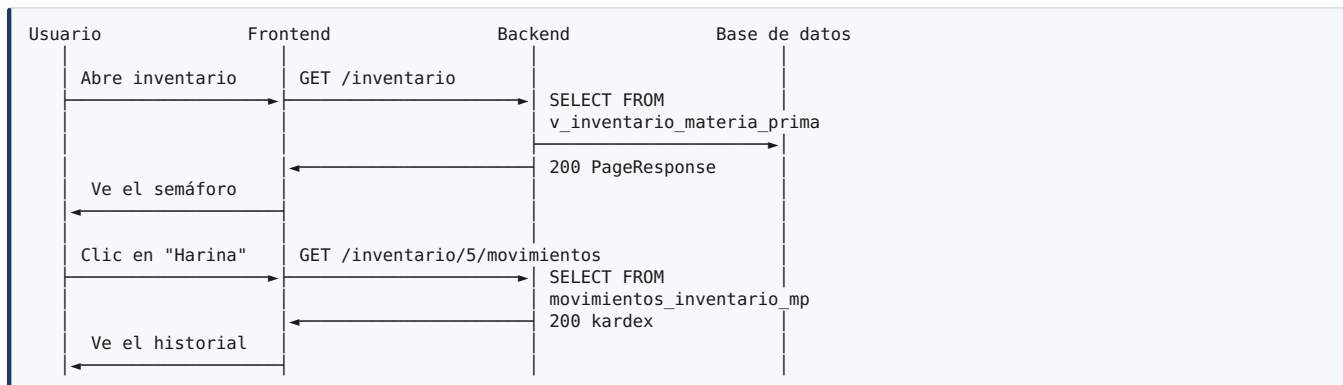
Kardex de una materia prima

← Harina de trigo · Movimientos			Saldo actual: 12,50 kg		
FECHA	MOVIMIENTO	CANT.	ANTES	DESPUÉS	USUARIO
18/09 09:12	↘ Salida producción Consumo #31	100,00 kg	112,50	12,50	Ismael B.
12/09 08:05	↗ Entrada lote Lote H-2026-0912	100,00 kg	12,50	112,50	Carlos C.
05/09 14:22	↗ Entrada compra Compra #8	50,00 kg	0,00	12,50	Carlos C.

↗ verde entradas · ↘ rojo salidas. El documento de origen es un enlace a la compra, el lote o el consumo que generó el movimiento.

Mostrar el saldo antes y después de cada movimiento es lo que permite encontrar **dónde** se descuadró cuando el conteo físico no coincide.

2. Flujo



Esta pantalla nunca escribe. El inventario cambia únicamente desde HU-09 (lotes), HU-13 (recepción de compra) y HU-23 (consumo de producción).

3. Componentes Angular Material

Elemento	Componente
Tarjetas de resumen	<code>MatCard</code> con conteos
Tabla	<code>MatTable</code> + <code>MatSort</code> + <code>MatPaginator</code>
Nivel de stock	<code>MatProgressBar</code> con color por <code>estadoStock</code>
Semáforo	<code>MatIcon</code> con color
Filtros	<code>MatSelect</code> + <code>MatCheckbox</code>
Kardex	<code>MatTable</code> con icono de dirección por movimiento
Enlaces al origen	<code>RouterLink</code> al documento correspondiente

4. Validaciones en el formulario

Esta pantalla **no tiene formularios de datos** (R-01). Solo filtros:

Filtro	Regla
Materia prima	Opcional
Solo stock bajo	Booleano
Incluir inactivas	Booleano, desactivado por defecto

En el kardex, el rango de fechas por defecto son los últimos 30 días: el historial crece rápido y quien lo consulta suele buscar algo reciente.

5. Estados de la pantalla

Estado	Qué se muestra
Cargando	<code>MatProgressBar</code> + tarjetas en esqueleto
Vacío	"No hay materias primas registradas" + enlace a HU-08
Vacío con filtro de stock bajo	✓ "Todas las materias primas están por encima del mínimo"
Kardex vacío	"Esta materia prima no registra movimientos"
Error	Mensaje + "Reintentar"

El vacío del filtro de stock bajo es una **buena noticia**, no una ausencia de datos: se muestra en verde y con ese texto, no con el mensaje genérico de "sin resultados".

6. Accesibilidad y UX

- Todos los campos con `<mat-label>` ; nunca solo *placeholder*.
- Navegación por teclado en orden lógico; `Enter` envía el formulario.
- Los mensajes de error del servidor se muestran **junto al campo** (`fieldErrors`).
- El botón de envío se deshabilita mientras la petición está en vuelo.
- Confirmación antes de descartar cambios sin guardar.
- Los tres estados obligatorios (cargando / vacío / error) según `00-base/05-ESTANDARES-QA.md` §7.

Tarea Backend

Historia de solo lectura. Se apoya en la vista de V2 y expone el kardex creado en HU-09.

1. Pasos

- `InventarioConsultaRowMapper` sobre `v_inventario_materia_prima`.
- `InventarioRepository.consultar(idMateriaPrima, soloStockBajo, incluirInactivas, page)`:
 - `SELECT ... FROM v_inventario_materia_prima WHERE 1=1` + filtros opcionales;
 - `soloStockBajo=true` → `AND estado_stock IN ('BAJO','AGOTADO')`;
 - lista blanca de orden: `{nombre, cantidadDisponible, faltante, fechaActualizacion}`.
- `InventarioRepository.contarConsulta(...)` — mismo `WHERE`.
- `MovimientoInventarioRepository.buscarPorMateriaPrima(id, fechaInicio, fechaFin, page)` con `LEFT JOIN usuarios` para el nombre (`getObject(..., Long.class)`, el usuario puede ser `NULL`).
- DTO: `InventarioResponse`, `MovimientoResponse`.
- `InventarioService` con `@Transactional(readonly = true)` en los tres métodos.
- `InventarioController` con `@PreAuthorize` de lectura amplia.
- Sin endpoints de escritura** (R-01).
- Endpoint de conciliación para QA (opcional, solo perfil `dev`): compara el saldo de `inventario` con el último `saldo_posterior` del kardex.

2. Archivos

```
backend/src/main/java/co/edu/corposucre/productionfood/inventario/  
├── InventarioController.java      ← solo GET  
├── InventarioService.java        (+ métodos de consulta)  
├── InventarioRepository.java     (+ consultar, contarConsulta)  
├── InventarioConsultaRowMapper.java  
└── dto/{InventarioResponse,MovimientoResponse}.java
```

3. Puntos de cuidado

- Sin endpoints de escritura aquí** (R-01). El inventario solo se mueve desde `InventarioService.registrarEntrada/registrarSalida`, invocados por HU-09, HU-13 y HU-23.
- Consultar **la vista**, no reconstruir los `JOIN` en cada endpoint (R-02).
- En el kardex, `id_usuario` puede ser `NULL`: usar `getObject(..., Long.class)` y mostrar “Sistema”.
- El kardex crece rápido: paginar siempre y ordenar por `id_movimiento DESC`, que usa el índice `idx_mov_mp_materia`.
- No ocultar las materias primas inactivas con stock (R-04).

4. Checklist de cierre

- ☐ Todos los CA implementados y verificados manualmente.
- ☐ Endpoints documentados en Swagger con ejemplos.
- ☐ `@PreAuthorize` en cada método, según la matriz de roles.
- ☐ Operaciones de escritura auditadas en bitácora (HU-04).
- ☐ Pruebas unitarias de las reglas con ramificación.
- ☐ Sin `System.out.println` ni código comentado.
- ☐ PR bajo ~250 líneas, revisado y aprobado.

Tarea Frontend

Pantalla de consulta con semáforo de stock y vista de kardex por materia prima.

1. Pasos

- `inventario.model.ts` y `inventario.service.ts`.
- `inventario-lista.component`: nombre, unidad, disponible, mínimo, faltante, estado, última actualización.
- Semáforo por estadoStock**: `AGOTADO` · ● `BAJO` · ● `OK`.
- Barra de progreso por fila: `disponible / stockMinimo`, en color según el estado.
- Filtros: materia prima, solo stock bajo (`MatCheckbox`), incluir inactivas.

6. Tarjetas de resumen arriba: total de materias primas, cuántas bajo mínimo, cuántas agotadas.
 7. `inventario-movimientos.component` : kardex de una materia prima, con tipo de movimiento, cantidad, saldos, documento de origen y usuario.
 8. Icono de entrada/salida con color en cada movimiento.
 9. Enlace desde cada fila del listado al kardex correspondiente.
 10. Ruta: lectura para ADMIN/PRODUCCION/COMPRAS/CONSULTA. **Sin botones de escritura.**
-

2. Archivos

```
frontend/src/app/features/inventario/  
├── models/inventario.model.ts  
├── services/inventario.service.ts  
├── pages/{inventario-lista, inventario-movimientos}/  
└── inventario.routes.ts
```

3. Puntos de cuidado

- **Ningún control de escritura en esta pantalla** (R-01): ni ajustar, ni editar saldos.
 - Mostrar el `faltante` en la misma fila evita la resta mental: es el dato que se necesita para hacer el pedido.
 - El kardex debe mostrar **saldo anterior y posterior** de cada movimiento: es lo que permite localizar dónde se descuadró.
 - Formatear `fechaActualizacion` con el locale `es-CO` ; si es de hoy, mostrar solo la hora.
 - Las materias primas inactivas con stock se muestran atenuadas, no ocultas (R-04).
-

4. Checklist de cierre

- ☐ Todos los CA cubiertos desde la interfaz.
- ☐ Los tres estados (cargando / vacío / error) implementados.
- ☐ Errores del servidor mostrados junto al campo correspondiente.
- ☐ Guard de rol aplicado en la ruta.
- ☐ Modelos TypeScript espejo de los DTO del backend.
- ☐ Probado en Chrome y Firefox a 1366×768.
- ☐ Sin `console.log` ni `any`.

Casos de Prueba

Formato y criterios en `00-base/05-ESTANDARES-QA.md`.

1. Casos funcionales

ID	CA	Descripción	Entrada	Resultado esperado	Tipo
CP-01	CA-01	Listar el inventario	<code>GET /inventario</code>	<code>200</code> con todas las materias primas y su disponible	Positivo
CP-02	CA-01	Materia prima sin movimientos aparece en cero	MP recién creada	Aparece con <code>cantidadDisponible: 0.00</code> (R-02)	Borde
CP-03	CA-02	Se muestra la fecha de actualización	Revisar CP-01	<code>fechaActualizacion</code> presente y con formato ISO	Positivo
CP-04	CA-02	La fecha cambia al mover el stock	Registrar un lote (HU-09) y volver a consultar	<code>fechaActualizacion</code> es posterior	Integridad
CP-05	CA-03	Filtrar por materia prima	<code>?idMateriaPrima=5</code>	Solo esa materia prima	Positivo
CP-06	CA-04	Filtrar solo stock bajo	<code>?soloStockBajo=true</code>	Solo <code>BAJO</code> y <code>AGOTADO</code>	Positivo
CP-07	CA-04	Stock exactamente igual al mínimo	disponible = 50, mínimo = 50	<code>estadoStock: "OK"</code> , <code>no</code> aparece en stock bajo	Borde
CP-08	CA-04	Stock una unidad por debajo	disponible = 49,99, mínimo = 50	<code>estadoStock: "BAJO"</code> , <code>faltante: 0.01</code>	Borde
CP-09	CA-04	Stock en cero	disponible = 0	<code>estadoStock: "AGOTADO"</code>	Borde
CP-10	CA-04	Stock mínimo en cero	mínimo = 0, disponible = 0	<code>estadoStock: "AGOTADO"</code> ; con disponible > 0 → <code>OK</code> (R-05)	Borde
CP-11	—	Cálculo del faltante	disponible = 12,50, mínimo = 50	<code>faltante: 37.50</code>	Positivo
CP-12	—	Materia prima inactiva con stock	Desactivar una con stock > 0	Sigue apareciendo, marcada inactiva (R-04)	Borde
CP-13	—	Kardex de una materia prima	<code>GET /inventario/5/movimientos</code>	Movimientos ordenados de más reciente a más antiguo	Positivo
CP-14	—	Coherencia saldo ↔ kardex	Comparar saldo y último movimiento	<code>saldo_posterior = cantidad_disponible</code> (R-03)	Integridad
CP-15	—	Kardex de MP sin movimientos	MP recién creada	<code>content: []</code> , sin error	Borde
CP-16	—	Movimiento del sistema	Movimiento con <code>id_usuario = NULL</code>	Se muestra "Sistema", no error ni usuario 0	Borde
CP-17	—	No existe endpoint de escritura	<code>POST / PUT / PATCH /inventario</code>	<code>405</code> o <code>404</code> (R-01)	Seguridad
CP-18	—	Rol CONSULTA puede ver	<code>GET</code> con token CONSULTA	<code>200</code>	Positivo
CP-19	—	Rol VENTAS no accede al inventario de MP	<code>GET</code> con token VENTAS	<code>403 SIN_PERMISO</code>	Seguridad
CP-20	—	Orden fuera de lista blanca	<code>?sort=costo_unitario,asc</code>	<code>400 CAMPO_ORDEN_INVALIDO</code>	Seguridad

2. Batería de seguridad

Obligatoria en **cada endpoint** de esta historia:

ID	Caso	Esperado
SEC-01	Sin cabecera <code>Authorization</code>	<code>401 NO_AUTENTICADO</code>
SEC-02	Token malformado	<code>401 NO_AUTENTICADO</code>
SEC-03	Token expirado	<code>401 NO_AUTENTICADO</code>
SEC-04	Token de rol sin permiso	<code>403 SIN_PERMISO</code>
SEC-05	Token de usuario con <code>estado = 0</code>	<code>403 USUARIO_INACTIVO</code>

SEC-04 es el caso crítico. Si devuelve `200`, falta `@EnableMethodSecurity` y la autorización del sistema completo está desactivada.

CP-14 debe ejecutarse al cerrar cada fase, no solo en esta historia. Es la prueba que detecta si algún módulo posterior actualiza el inventario sin pasar por `InventarioService`. Mientras devuelva 0 filas, el kardex es confiable; en cuanto devuelva una, el saldo dejó de ser explicable.

3. Verificación en base de datos

```
-- CP-14: la conciliación. DEBE devolver 0 filas, siempre.
SELECT i.id_materia_prima, i.cantidad_disponible, m.saldo_posterior
FROM inventario i
LEFT JOIN (
    SELECT id_materia_prima, saldo_posterior,
           ROW_NUMBER() OVER (PARTITION BY id_materia_prima ORDER BY id_movimiento DESC) rn
    FROM movimientos_inventario mp
) m ON m.id_materia_prima = i.id_materia_prima AND m.rn = 1
WHERE i.cantidad_disponible <> COALESCE(m.saldo_posterior, 0);
-- Una fila aquí significa que alguna ruta de código actualizó el saldo sin registrar
-- el movimiento: exactamente lo que el kardex existe para impedir.

-- CP-02: toda materia prima debe tener fila de inventario (HU-08 R-01)
SELECT mp.nombre FROM materias_primas mp
LEFT JOIN inventario i USING (id_materia_prima)
WHERE i.id_inventario IS NULL; -- 0 filas

-- CP-07/CP-08: verificar el cálculo de la vista
SELECT nombre, cantidad_disponible, stock_minimo, estado_stock, faltante
FROM v_inventario_materia_prima ORDER BY estado_stock, faltante DESC;

-- CP-04: el ON UPDATE de V2 §C-07 mantiene la fecha sola
SHOW CREATE TABLE inventario\G
-- `fecha_actualizacion` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP
-- ON UPDATE CURRENT_TIMESTAMP
```

4. Criterios de cierre

- ☐ Todos los casos ejecutados; los positivos pasan.
- ☐ Los negativos devuelven el HTTP y el `code` esperados.
- ☐ SEC-01 a SEC-05 pasan en todos los endpoints.
- ☐ Ningún endpoint supera 2 s (RNF-02).
- ☐ Cero defectos críticos o altos abiertos.
- ☐ Colección de Postman actualizada y versionada.

HU-11 · Gestión de productos

Especificación

Fase 4 · Fuente: `previo/Historias_de_Usuario.md`

Como administrador, **quiero** CRUD de productos terminados (nombre, descripción, precio, unidad, estado), **para que** mantenga el catálogo de lo que vende la panadería.

Depende de: HU-07 (unidades de medida)

1. Criterios de aceptación

CA	Enunciado	Cómo se implementa
CA-01	Nombre obligatorio, máx. 100	<code>@NotBlank</code> + <code>@Size(max=100)</code>
CA-02	Descripción opcional, máx. 150	<code>@Size(max=150)</code>
CA-03	Precio obligatorio (decimal positivo)	<code>@NotNull</code> + <code>@DecimalMin</code> ; <code>CHECK (precio >= 0)</code> en V2
CA-04	<code>id_unidad</code> debe existir	Validado contra <code>UnidadMedidaRepository</code>
CA-05	Estado por defecto activo (<code>1</code>)	<code>DEFAULT 1</code> añadido en V2 §C-06
CA-06	Listar con paginación y filtrar por nombre	<code>GET /productos?nombre=&page=&size=</code>

2. Endpoints

Método	Ruta	Descripción	Roles
<code>GET</code>	<code>/api/v1/productos</code>	Listar con filtro y paginación	ADMIN, VENTAS, PRODUCCION, CONSULTA
<code>GET</code>	<code>/api/v1/productos/{id}</code>	Obtener uno	ADMIN, VENTAS, PRODUCCION, CONSULTA
<code>POST</code>	<code>/api/v1/productos</code>	Registrar	ADMIN
<code>PUT</code>	<code>/api/v1/productos/{id}</code>	Editar	ADMIN
<code>PATCH</code>	<code>/api/v1/productos/{id}/estado</code>	Activar o desactivar	ADMIN

Reglas transversales (paginación, formato de error, autenticación) en `00-base/04-CONTRATO-API.md`.

3. Reglas de negocio

R-01 · Cambiar el precio no afecta a los pedidos ya registrados

Gracias a `detalle_pedido.precio_unitario`, añadido en V2 (§C-01), el precio se copia en el momento de agregar la línea al pedido (HU-17) y nunca se recalcula.

Sin esa columna, subir el precio del pan hoy cambiaría retroactivamente el valor de todos los pedidos del último año, y los reportes darían un número distinto cada vez que alguien edita el catálogo.

Editar el precio aquí solo afecta a las ventas futuras.

R-02 · Un producto sin receta no se puede planificar

HU-12 asocia la receta; HU-21 y HU-22 la necesitan para calcular el consumo teórico. Un producto sin receta puede venderse (si hay stock de lotes), pero **no puede entrar en un plan de producción** con consumo teórico calculado.

El listado indica qué productos tienen receta, para que el hueco sea visible.

R-03 · Desactivar un producto no retira su stock

HU-19 puede tener lotes de ese producto con existencias. Desactivar significa “no vender más ni planificar producción nueva”, no “desapareció del estante”. HU-17 CA-02 exige producto activo para agregarlo a un pedido.

R-04 · La unidad del producto y la de la receta pueden diferir

`productos.id_unidad` es la unidad de **venta** (unidad, docena, bandeja). `recetas.id_unidad` es la unidad de **producción** (HU-12 CA-02).

Un pan se vende por unidad pero la receta produce 30 unidades por tanda. Son dos conceptos distintos y el esquema los separa correctamente.

R-05 · No se elimina

`detalle_pedido`, `lotes`, `recetas` y `detalle_plan_produccion` lo referencian. Solo se desactiva.

R-06 · Precio cero permitido

`CHECK (precio >= 0)`, no `> 0`. Permite registrar productos promocionales o muestras gratuitas. CA-03 dice “positivo”, que se interpreta como “no negativo”.

4. Modelo de datos

Tabla `productos`

Columna	Tipo	Notas
<code>id_producto</code>	<code>int unsigned</code> PK AI	
<code>nombre</code>	<code>varchar(100)</code> NOT NULL	CA-01
<code>descripcion</code>	<code>varchar(150)</code> NULL	CA-02
<code>precio</code>	<code>decimal(10,2)</code> NOT NULL	CA-03 · <code>CHECK >= 0</code> (V2)
<code>id_unidad</code>	<code>int unsigned</code> NOT NULL FK	CA-04 · unidad de venta (R-04)
<code>estado</code>	<code>tinyint(1)</code> NOT NULL DEFAULT 1	CA-05 · corregido en V2 §C-06

```
// POST /api/v1/productos
{ "nombre": "Pan francés", "descripcion": "Pan de corteza crujiente, 80 g",
  "precio": 1200.00, "idUnidad": 5 }

// 201 Created
{ "idProducto": 3, "nombre": "Pan francés",
  "descripcion": "Pan de corteza crujiente, 80 g", "precio": 1200.00,
  "unidad": { "idUnidad": 5, "nombre": "Unidad", "abreviatura": "und" },
  "activo": true, "tieneReceta": false, "stockDisponible": 0.00 }
```

`tieneReceta` y `stockDisponible` se calculan con `LEFT JOIN` a `recetas` y a `v_stock_producto_terminado` (V2 §C-02). Son de solo lectura y hacen el listado útil sin navegar a otras pantallas.

Campos ordenables: `nombre`, `precio`, `id`.

5. Códigos de error

code	HTTP	Cuándo
<code>UNIDAD_INEXISTENTE</code>	409	El <code>idUnidad</code> no existe
<code>PRODUCTO_CON_STOCK</code>	409	Advertencia al desactivar un producto con existencias (no bloqueante)

Los transversales (`VALIDACION_FALLIDA`, `NO_AUTENTICADO`, `SIN_PERMISO`, `RECURSO_NO_ENCONTRADO`) están en `00-base/04-CONTRATO-API.md` §5.

6. Fuera de alcance

Lo que **no** cubre esta historia y no debe implementarse aquí: se limita estrictamente a los criterios de aceptación listados arriba. Cualquier funcionalidad adicional se propone como historia nueva, no se agrega en silencio.

7. Definición de Terminado

Se aplica la DoD de `00-base/01-CONVENCIONES.md` §7.

8. Notas

Esta historia es gemela de HU-08

`productos` y `materias_primas` tienen casi la misma estructura y el mismo CRUD. Se implementa copiando el patrón de HU-08, con dos diferencias:

- Productos tiene `descripcion`; materias primas no.
- **Productos NO crea fila de inventario al darse de alta.** El stock de producto terminado vive en `inventario_producto_terminado`, que es **por lote** (HU-19), no por producto. Es la asimetría del esquema que obligó a crear la vista `v_stock_producto_terminado` (§C-02).

Diseño

1. Pantallas

Catálogo de productos

Productos

[+ Nuevo producto]

Buscar por nombre

Estado

Activos

☐

Solo sin receta

PRODUCTO	DESCRIPCIÓN	PRECIO	UNIDAD	RECETA	STOCK	...
Croissant	Hojaldre con mant...	\$ 2.500,00	und		24	:
Pan francés	Corteza crujiente...	\$ 1.200,00	und		120	:
Pan integral	Con salvado, 500 g	\$ 4.800,00	und		0	:
Torta de zana...	Torta húmeda 1 kg	\$ 32.000,00	und		3	:

marca los productos **sin receta**: se pueden vender si hay stock, pero no se pueden planificar (R-02). El icono enlaza directamente a crear la receta en HU-12.

La columna STOCK viene de `v_stock_producto_terminado` (V2 §C-02), que agrega los lotes vigentes de HU-19.

Formulario

Nuevo producto

[X]

Nombre *

Pan francés

Descripción

Pan de corteza crujiente, 80 g

31 / 150

Precio de venta *

\$ 1.200,00

Unidad de venta *

Unidad (und)

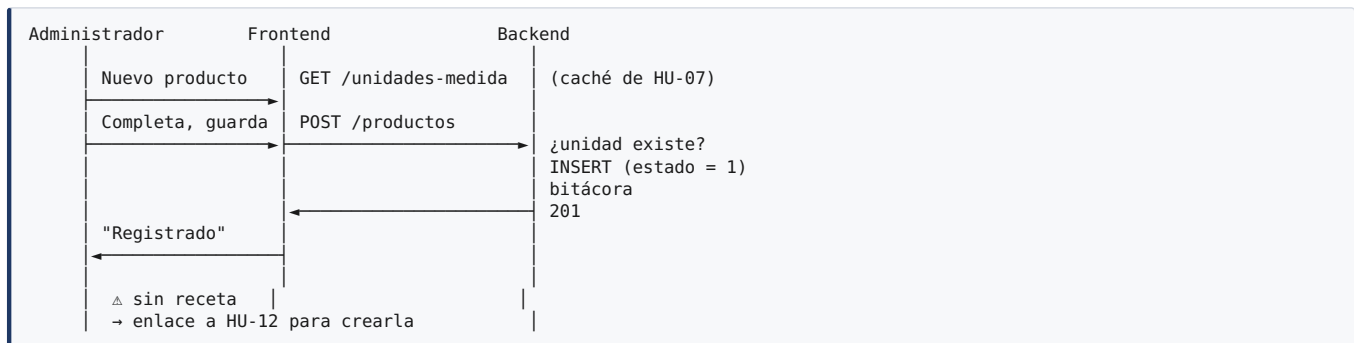
Cómo se vende, no cómo se produce

[Cancelar]

[Guardar]

La nota bajo el selector aclara R-04: esta unidad es la de venta. La receta (HU-12) tiene la suya, que puede ser distinta — se vende por unidad, se produce por tandas de 30.

2. Flujo



A diferencia de HU-08, no se crea ninguna fila de inventario: el stock de producto terminado entra con los lotes de HU-19.

3. Componentes Angular Material

Elemento	Componente
Tabla	<code>MatTable</code> + <code>MatSort</code> + <code>MatPaginator</code>
Precio	Pipe <code>currency: 'COP': 'symbol-narrow': '1.2-2'</code>
Receta	<code>MatIcon</code> ✓/△ con <code>MatTooltip</code> y <code>RouterLink</code> a HU-12
Stock	Texto con color según cantidad
Unidad	<code>MatSelect</code> (catálogo cacheado)
Descripción	<code>MatInput</code> + <code>MatHint</code> con contador
Formulario	<code>MatDialog</code>

4. Validaciones en el formulario

Campo	Reglas	Mensaje
Nombre	requerido, máx. 100	"El nombre es obligatorio"
Descripción	opcional, máx. 150	"Máximo 150 caracteres"
Precio	requerido, ≥ 0 , máx. 2 decimales	"Debe ser 0 o mayor"
Unidad	requerida	"Seleccione una unidad de venta"

Precio cero es válido (R-06): sirve para muestras y promociones.

5. Estados de la pantalla

Estado	Qué se muestra
Cargando	<code>MatProgressBar</code>
Vacío sin filtro	"No hay productos registrados" + "Registrar el primero"
Vacío con filtro	"Ningún producto coincide con «xxx»"
Vacío filtrando sin receta	✓ "Todos los productos tienen receta"
Sin unidades disponibles	Enlace a HU-07
Error	Mensaje + "Reintentar"

6. Accesibilidad y UX

- Todos los campos con `<mat-label>`; nunca solo *placeholder*.
- Navegación por teclado en orden lógico; `Enter` envía el formulario.
- Los mensajes de error del servidor se muestran **junto al campo** (`fieldErrors`).
- El botón de envío se deshabilita mientras la petición está en vuelo.
- Confirmación antes de descartar cambios sin guardar.
- Los tres estados obligatorios (cargando / vacío / error) según `00-base/05-ESTANDARES-QA.md` §7.

Tarea Backend

CRUD gemelo de HU-08. **No** crea fila de inventario: el stock de producto terminado es por lote (HU-19), no por producto.

1. Pasos

1. `Producto` (POJO) y `ProductoRowMapper` con `JOIN unidades_medida`.
2. `ProductoRepository`:
 - `buscar`, `contar`, `buscarPorId`, `insertar`, `actualizar`, `cambiarEstado`;
 - el `SELECT` del listado incluye `LEFT JOIN recetas` (para `tieneReceta`) y `LEFT JOIN v_stock_producto_terminado` (para `stockDisponible`);
 - lista blanca: `{nombre, precio, id}`.
3. DTO con `BigDecimal` en `precio`.
4. `ProductoService.crear()`: valida unidad existente → `UNIDAD_INEXISTENTE`. **No crea fila de inventario** (a diferencia de HU-08 R-01).
5. `ProductoService.cambiarEstado()`: al desactivar, consultar `v_stock_producto_terminado` y devolver el stock como advertencia (R-03).
6. Método `existeActivo(id)` para HU-12, HU-17, HU-19 y HU-21.
7. Controlador: lectura amplia, escritura solo ADMIN.
8. `@Auditable` en escrituras. Sin `DELETE` (R-05).

2. Archivos

```
backend/src/main/java/co/edu/corposucre/productionfood/producto/  
├── {Producto,ProductoRowMapper,ProductoRepository,ProductoService,ProductoController}.java  
└── dto/{CrearProductoRequest,ActualizarProductoRequest,ProductoResponse}.java
```

3. Puntos de cuidado

- **A diferencia de HU-08, aquí NO se crea fila de inventario.** El stock de producto terminado entra con los lotes de HU-19.
- `BigDecimal` en el precio, nunca `double`.
- `precio = 0` es válido (R-06): el `CHECK` es `>= 0`.
- **Cambiar el precio no debe tocar `detalle_pedido`** (R-01). Si alguien propone “actualizar los pedidos al nuevo precio”, la respuesta está en R-01.
- La unidad del producto es la de **venta**, distinta de la de la receta (R-04).

4. Checklist de cierre

- ☐ Todos los CA implementados y verificados manualmente.
- ☐ Endpoints documentados en Swagger con ejemplos.
- ☐ `@PreAuthorize` en cada método, según la matriz de roles.
- ☐ Operaciones de escritura auditadas en bitácora (HU-04).
- ☐ Pruebas unitarias de las reglas con ramificación.
- ☐ Sin `System.out.println` ni código comentado.
- ☐ PR bajo ~250 líneas, revisado y aprobado.

Tarea Frontend

Listado y formulario gemelos de HU-08, con indicadores de receta y stock.

1. Pasos

1. `producto.model.ts` y `producto.service.ts`.
2. `producto-lista.component`: nombre, descripción, precio, unidad, **receta**, **stock**, estado, `:`.
3. Indicador de receta: ✓ tiene ◀ sin receta (con enlace a HU-12 para crearla).
4. Indicador de stock desde `stockDisponible`.
5. Búsqueda con `debounceTime(350)`; filtros por estado y por “sin receta”.
6. `producto-formulario.component`:
 - precio con formato de moneda colombiana (\$ 1.200,00);
 - selector de unidad desde el servicio cacheado de HU-07;
 - contador de caracteres en la descripción (máx. 150).
7. Al desactivar un producto con stock, advertir: “Este producto tiene 45 unidades en estante. Desactivarlo impedirá nuevas ventas, pero el stock se conserva.”

- 8. Los tres estados.
- 9. Ruta: lectura ADMIN/VENTAS/PRODUCCION/CONSULTA; escritura solo ADMIN.

2. Archivos

```
frontend/src/app/features/producto/  
├── models/producto.model.ts  
├── services/producto.service.ts  
├── pages/{producto-lista,producto-formulario}/  
└── producto.routes.ts
```

3. Puntos de cuidado

- El precio se **envía** como `1200.00`; el formato `$ 1.200,00` es solo presentación. Con `LOCALE_ID = 'es-CO'` el pipe `currency` lo resuelve.
- El indicador “sin receta” con enlace directo a HU-12 evita que el hueco pase inadvertido: un producto sin receta no se puede planificar (R-02).
- Filtrar por “sin receta” es lo que permite completar el catálogo sin revisarlo entero.
- La advertencia al desactivar con stock explica la consecuencia real (R-03).

4. Checklist de cierre

- ☐ Todos los CA cubiertos desde la interfaz.
- ☐ Los tres estados (cargando / vacío / error) implementados.
- ☐ Errores del servidor mostrados junto al campo correspondiente.
- ☐ Guard de rol aplicado en la ruta.
- ☐ Modelos TypeScript espejo de los DTO del backend.
- ☐ Probado en Chrome y Firefox a 1366×768.
- ☐ Sin `console.log` ni `any`.

Casos de Prueba

Formato y criterios en `00-base/05-ESTANDARES-QA.md`.

1. Casos funcionales

ID	CA	Descripción	Entrada	Resultado esperado	Tipo
CP-01	CA-01, CA-05	Crear producto válido	<code>{"nombre": "Pan francés", "precio": 1200, "idUnidad": 5}</code>	<code>201</code> , <code>activo: true</code> , <code>tieneReceta: false</code>	Positivo
CP-02	CA-01	Nombre vacío	<code>{"nombre": ""}</code>	<code>400 VALIDACION_FALLIDA</code>	Negativo
CP-03	CA-01	Nombre de 101 caracteres	<code>"A"×101</code>	<code>400</code>	Borde
CP-04	CA-02	Sin descripción	Omitir <code>descripcion</code>	<code>201</code> , <code>descripcion: null</code>	Positivo
CP-05	CA-02	Descripción de 151 caracteres	<code>"A"×151</code>	<code>400</code>	Borde
CP-06	CA-02	Descripción de exactamente 150	<code>"A"×150</code>	<code>201</code>	Borde
CP-07	CA-03	Precio omitido	Sin <code>precio</code>	<code>400</code>	Negativo
CP-08	CA-03	Precio negativo	<code>{"precio": -100}</code>	<code>400</code> o <code>409</code> — <code>CHECK >= 0</code> de V2	Negativo
CP-09	CA-03	Precio cero	<code>{"precio": 0}</code>	<code>201</code> — válido (R-06)	Borde
CP-10	CA-03	Precio con 3 decimales	<code>{"precio": 1200.555}</code>	Se redondea a <code>1200.56</code>	Borde
CP-11	CA-04	Unidad inexistente	<code>{"idUnidad": 9999}</code>	<code>409 UNIDAD_INEXISTENTE</code>	Negativo
CP-12	CA-05	Estado por defecto	Crear sin <code>estado</code>	<code>activo: true</code> ; <code>estado = 1</code> en base	Positivo
CP-13	CA-06	Listar paginado	<code>?page=0&size=5</code>	<code>200</code> , máx. 5	Positivo
CP-14	CA-06	Filtrar por nombre	<code>?nombre=Pan</code>	Solo los que empiezan por “Pan”	Positivo
CP-15	CA-06	Orden por precio	<code>?sort=precio,desc</code>	Ordenado descendente	Positivo
CP-16	—	Cambiar el precio no altera pedidos existentes	Crear pedido con producto a \$1.200, subir a \$1.500, consultar el pedido	El detalle sigue en <code>\$1.200</code> (R-01)	Integridad
CP-17	—	Desactivar con stock	Producto con lotes disponibles	<code>200</code> + advertencia; el stock se conserva (R-03)	Borde
CP-18	—	No existe DELETE	<code>DELETE /productos/3</code>	<code>405</code> o <code>404</code>	Negativo
CP-19	—	Rol VENTAS puede listar	<code>GET</code> con token VENTAS	<code>200</code>	Positivo
CP-20	—	Rol VENTAS no puede crear	<code>POST</code> con token VENTAS	<code>403 SIN_PERMISO</code>	Seguridad
CP-21	—	<code>tieneReceta</code> refleja la realidad	Crear receta en HU-12 y volver a consultar	<code>tieneReceta: true</code>	Positivo

2. Batería de seguridad

Obligatoria en **cada endpoint** de esta historia:

ID	Caso	Esperado
SEC-01	Sin cabecera <code>Authorization</code>	<code>401 NO_AUTENTICADO</code>
SEC-02	Token malformado	<code>401 NO_AUTENTICADO</code>
SEC-03	Token expirado	<code>401 NO_AUTENTICADO</code>
SEC-04	Token de rol sin permiso	<code>403 SIN_PERMISO</code>
SEC-05	Token de usuario con <code>estado = 0</code>	<code>403 USUARIO_INACTIVO</code>

SEC-04 es el caso crítico. Si devuelve `200`, falta `@EnableMethodSecurity` y la autorización del sistema completo está desactivada.

CP-16 verifica la corrección más importante del esquema (02-CORRECCIONES-DB.md §C-01). Si el precio del detalle del pedido se lee del catálogo en lugar de guardarse, cada cambio de precio reescribe silenciosamente el valor de todas las ventas anteriores, y los reportes de HU-25 dan un número distinto cada mes sin que nadie toque un pedido.

3. Verificación en base de datos

```
-- CP-16: el histórico de precios debe quedar congelado en el pedido
SELECT dp.id_detalle_pedido, dp.cantidad, dp.precio_unitario AS precio_del_pedido,
       p.precio AS precio_actual_catalogo
FROM detalle_pedido dp JOIN productos p USING (id_producto)
WHERE dp.id_pedido = 1;
-- precio_del_pedido NO debe cambiar aunque precio_actual_catalogo suba.
-- Si coinciden siempre, es que se está leyendo el precio del catálogo: defecto de SC-01.

-- CP-08: el CHECK de V2
INSERT INTO productos (nombre, precio, id_unidad) VALUES ('X', -100, 5);
-- DEBE fallar con error 3819

-- CP-12: el DEFAULT añadido en V2 SC-06
SHOW CREATE TABLE productos\G

-- CP-21: productos sin receta (R-02)
SELECT p.id_producto, p.nombre FROM productos p
LEFT JOIN recetas r USING (id_producto)
WHERE r.id_receta IS NULL AND p.estado = 1;
```

4. Criterios de cierre

- ☐ Todos los casos ejecutados; los positivos pasan.
- ☐ Los negativos devuelven el HTTP y el `code` esperados.
- ☐ SEC-01 a SEC-05 pasan en todos los endpoints.
- ☐ Ningún endpoint supera 2 s (RNF-02).
- ☐ Cero defectos críticos o altos abiertos.
- ☐ Colección de Postman actualizada y versionada.

HU-12 · Gestión de recetas

Especificación

Fase 4 · Fuente: `previo/Historias_de_Usuario.md`

Como administrador, **quiero** asociar una receta a cada producto indicando materias primas y cantidades, **para que** sepa qué ingredientes se necesitan por unidad.

Depende de: HU-08, HU-11

1. Criterios de aceptación

CA	Enunciado	Cómo se implementa
CA-01	Cada producto puede tener una sola receta	<code>UNIQUE KEY id_producto</code> ya existe en el esquema
CA-02	La receta tiene <code>nombre</code> , <code>cantidad_producir</code> , <code>id_unidad</code> y <code>estado</code>	<code>@NotBlank</code> , <code>@NotNull</code> , FK validada; <code>DEFAULT 1</code> añadido en V2 §C-06
CA-03	El detalle lista materias primas con sus cantidades	Subrecurso <code>/recetas/{id}/detalles</code>
CA-04	Agregar, editar y eliminar materias primas del detalle	<code>POST</code> , <code>PUT</code> , <code>DELETE</code> sobre el subrecurso
CA-05	No se permite duplicar la misma materia prima en una receta	<code>UNIQUE (id_receta, id_materia_prima)</code> añadido en V2 §C-05

2. Endpoints

Método	Ruta	Descripción	Roles
<code>GET</code>	<code>/api/v1/recetas</code>	Listar recetas con paginación	ADMIN, PRODUCCION, CONSULTA
<code>GET</code>	<code>/api/v1/recetas/{id}</code>	Obtener receta con su detalle completo	ADMIN, PRODUCCION, CONSULTA
<code>GET</code>	<code>/api/v1/productos/{idProducto}/receta</code>	Receta de un producto	ADMIN, PRODUCCION, CONSULTA
<code>POST</code>	<code>/api/v1/recetas</code>	Crear receta (cabecera + detalle inicial)	ADMIN, PRODUCCION
<code>PUT</code>	<code>/api/v1/recetas/{id}</code>	Editar la cabecera	ADMIN, PRODUCCION
<code>PATCH</code>	<code>/api/v1/recetas/{id}/estado</code>	Activar o desactivar	ADMIN, PRODUCCION
<code>POST</code>	<code>/api/v1/recetas/{id}/detalles</code>	Agregar un ingrediente	ADMIN, PRODUCCION
<code>PUT</code>	<code>/api/v1/recetas/{id}/detalles/{idDetalle}</code>	Editar la cantidad de un ingrediente	ADMIN, PRODUCCION
<code>DELETE</code>	<code>/api/v1/recetas/{id}/detalles/{idDetalle}</code>	Quitar un ingrediente	ADMIN, PRODUCCION

Reglas transversales (paginación, formato de error, autenticación) en `00-base/04-CONTRATO-API.md`.

3. Reglas de negocio

R-01 · Sin duplicados: la garantía es de la base (CA-05)

El esquema original **no tenía** la restricción, aunque CA-05 la exige. Validarla solo en el servicio no basta: dos peticiones simultáneas pasan ambas el `SELECT` previo y ambas insertan.

V2 añadió `UNIQUE KEY uk_detalle_receta (id_receta, id_materia_prima)` (§C-05). El servicio sigue comprobando para devolver un `409` legible, pero **la base es la que garantiza**.

Un ingrediente duplicado haría que HU-22 calcule el doble de consumo teórico y HU-23 descuente el doble de inventario.

R-02 · `cantidad_producir` es el rendimiento de la receta

Es la cantidad de producto que rinde **una tanda completa**, no una unidad. Una receta que produce 30 panes con 5 kg de harina tiene `cantidad_producir = 30`.

El consumo teórico de HU-22 depende de esto:

```
consumo_teórico = (cantidad_planificada / cantidad_producir) × cantidad_del_ingredient
```

Planificar 90 panes con esa receta consume $(90 / 30) \times 5 = 15$ kg de harina.

Si `cantidad_producir` se interpreta como “por unidad” cuando en realidad es por tanda, todos los cálculos de producción salen multiplicados por 30. Es el malentendido más probable de esta historia, y no produce ningún error: solo números absurdos en HU-22 y HU-26. La interfaz debe hacer explícito el significado.

R-03 · `cantidad_producir` nunca puede ser cero

Dividir por ella es el núcleo del cálculo de R-02. El `CHECK (cantidad_producir > 0)` y la validación del servicio lo impiden. Es la misma clase de problema que el divisor cero de HU-22 (§C-09), pero aquí sí se puede bloquear en el origen.

R-04 · La unidad de la receta es la de producción

`recetas.id_unidad` es la unidad en que se mide `cantidad_producir`, y puede diferir de la unidad de venta del producto (HU-11 R-04). Lo habitual es que coincidan, pero el esquema permite que no.

R-05 · Una receta sin ingredientes es inútil pero se permite crearla

Se puede guardar la cabecera y agregar el detalle después, porque obligar a completar todo de una vez hace incómodo el trabajo real.

Pero una receta sin detalle no sirve para planificar: HU-22 calcularía consumo teórico cero. La interfaz la marca como incompleta y HU-21 advierte al usarla.

R-06 · Las cantidades del detalle llevan 3 decimales

`detalle_receta.cantidad` es `decimal(10,3)`, a diferencia de casi todas las demás cantidades del sistema, que son `(10,2)`.

Es correcto: en repostería se usan gramos de sal o levadura donde el tercer decimal importa. **Al calcular el consumo teórico, el resultado se guarda también con 3 decimales** (`consumo_materia_prima.cantidad_teorica` es `(10,3)`), pero el inventario es `(10,2)`. Ese redondeo al descontar debe hacerse de forma consistente — ver HU-23.

R-07 · Se puede eliminar una línea de detalle

Es la única excepción del proyecto a “no se elimina”: un ingrediente quitado de una receta no tiene valor histórico, y `detalle_receta` no es referenciada por ninguna otra tabla. La cabecera de la receta, en cambio, solo se desactiva.

4. Modelo de datos

Tabla `recetas`

Columna	Tipo	Notas
<code>id_receta</code>	<code>int unsigned</code> PK AI	
<code>id_producto</code>	<code>int unsigned</code> NOT NULL UNIQUE FK	CA-01
<code>nombre</code>	<code>varchar(100)</code> NOT NULL	CA-02
<code>cantidad_producir</code>	<code>decimal(10,2)</code> NOT NULL	CA-02 · R-02 · <code>CHECK > 0</code>
<code>id_unidad</code>	<code>int unsigned</code> NOT NULL FK	CA-02 · unidad de producción (R-04)
<code>estado</code>	<code>tinyint(1)</code> NOT NULL DEFAULT 1	V2 §C-06

Tabla `detalle_receta`

Columna	Tipo	Notas
<code>id_detalle_receta</code>	<code>int unsigned</code> PK AI	
<code>id_receta</code>	<code>int unsigned</code> NOT NULL FK	
<code>id_materia_prima</code>	<code>int unsigned</code> NOT NULL FK	
<code>cantidad</code>	<code>decimal(10,3)</code> NOT NULL	R-06 · <code>CHECK > 0</code>

`UNIQUE KEY uk_detalle_receta (id_receta, id_materia_prima)` — añadido en V2 §C-05.

```
// POST /api/v1/recetas
{
  "idProducto": 3, "nombre": "Masa de pan francés",
  "cantidadProducir": 30.00, "idUnidad": 5,
  "detalles": [
    { "idMateriaPrima": 5, "cantidad": 5.000 },
    { "idMateriaPrima": 7, "cantidad": 0.050 },
    { "idMateriaPrima": 9, "cantidad": 0.100 }
  ]
}

// 201 Created
{
  "idReceta": 2, "producto": { "idProducto": 3, "nombre": "Pan francés" },
  "nombre": "Masa de pan francés", "cantidadProducir": 30.00,
  "unidad": { "abreviatura": "und" }, "activo": true,
  "detalles": [
    { "idDetalleReceta": 11, "materiaPrima": { "idMateriaPrima": 5, "nombre": "Harina de trigo", "unidad": "kg" },
      "cantidad": 5.000, "costoEstimado": 16000.00 },
    { "idDetalleReceta": 12, "materiaPrima": { "idMateriaPrima": 7, "nombre": "Levadura", "unidad": "g" },
      "cantidad": 0.050, "costoEstimado": 9.00 }
  ],
  "costoTotalEstimado": 16009.00,
  "costoPorUnidad": 533.63
}
```

`costoEstimado` usa `materias_primas.costo_unitario` (HU-08 R-03), que es un valor de referencia. **No es el costo real**, que depende de cada compra.

5. Códigos de error

code	HTTP	Cuándo
PRODUCTO_YA_TIENE_RECETA	409	El producto ya tiene una receta (CA-01)
MATERIA_PRIMA_DUPLICADA	409	Ese ingrediente ya está en la receta (CA-05)
PRODUCTO_INEXISTENTE	409	El <code>idProducto</code> no existe o está inactivo
MATERIA_PRIMA_INEXISTENTE	409	El <code>idMateriaPrima</code> no existe o está inactiva
CANTIDAD_PRODUCIR_INVALIDA	400	<code>cantidadProducir</code> menor o igual a cero (R-03)

Los transversales (`VALIDACION_FALLIDA` , `NO_AUTENTICADO` , `SIN_PERMISO` , `RECURSO_NO_ENCONTRADO`) están en `00-base/04-CONTRATO-API.md` §5.

6. Fuera de alcance

Lo que **no** cubre esta historia y no debe implementarse aquí: se limita estrictamente a los criterios de aceptación listados arriba. Cualquier funcionalidad adicional se propone como historia nueva, no se agrega en silencio.

7. Definición de Terminado

Se aplica la DoD de `00-base/01-CONVENCIONES.md` §7.

8. Notas

Por qué esta historia sostiene toda la Fase 7

HU-22 calcula el consumo teórico a partir de la receta, y HU-23 descuenta el inventario con ese cálculo. Un error aquí —un ingrediente duplicado, una `cantidad_producir` mal interpretada— **no se manifiesta en esta historia**: aparece tres sprints después como un inventario que se descuenta mal y una eficiencia de producción sin sentido.

Por eso CA-05 tiene respaldo en la base (R-01) y por eso R-02 insiste en dejar explícito el significado de `cantidad_producir` en la interfaz.

Diseño

1. Pantallas

Editor de receta

← Receta: Masa de pan francés

●Activa [:]

Producto

Pan francés ▾

Nombre de la receta

Masa de pan francés

Cantidad que produce *

30,00

Unidad *

Unidad (und) ▾

i Cantidad que rinde UNA TANDA completa de esta receta.
Ejemplo: si con estos ingredientes salen 30 panes, escriba 30.

INGREDIENTES

[+ Agregar]

MATERIA PRIMA	CANTIDAD	POR UNIDAD	COSTO EST.	...
Harina de trigo	5,000 kg	0,167 kg/und	\$ 16.000,00	⇒ 🗑
Levadura	0,050 kg	0,002 kg/und	\$ 9,00	⇒ 🗑
Sal	0,100 kg	0,003 kg/und	\$ 85,00	⇒ 🗑
COSTO TOTAL ESTIMADO POR TANDA:			\$ 16.094,00	
COSTO ESTIMADO POR UNIDAD:			\$ 536,47	
Precio de venta actual:			\$ 1.200,00	
Margen estimado:			55,3 %	

La columna “POR UNIDAD” es la defensa contra el error de R-02. Si alguien escribe 1 en “cantidad que produce” pensando que es por unidad, esa columna mostrará 5,000 kg/und de harina por pan — un absurdo evidente que se corrige al instante, en lugar de descubrirse tres sprints después en HU-23.

El margen estimado es informativo: usa el costo de referencia de HU-08, no el costo real.

Agregar ingrediente

Agregar ingrediente

[X]

Materia prima *

Mantequilla (kg) ▾

Solo se muestran las que aún no están en esta receta

Cantidad *

0,250 kg

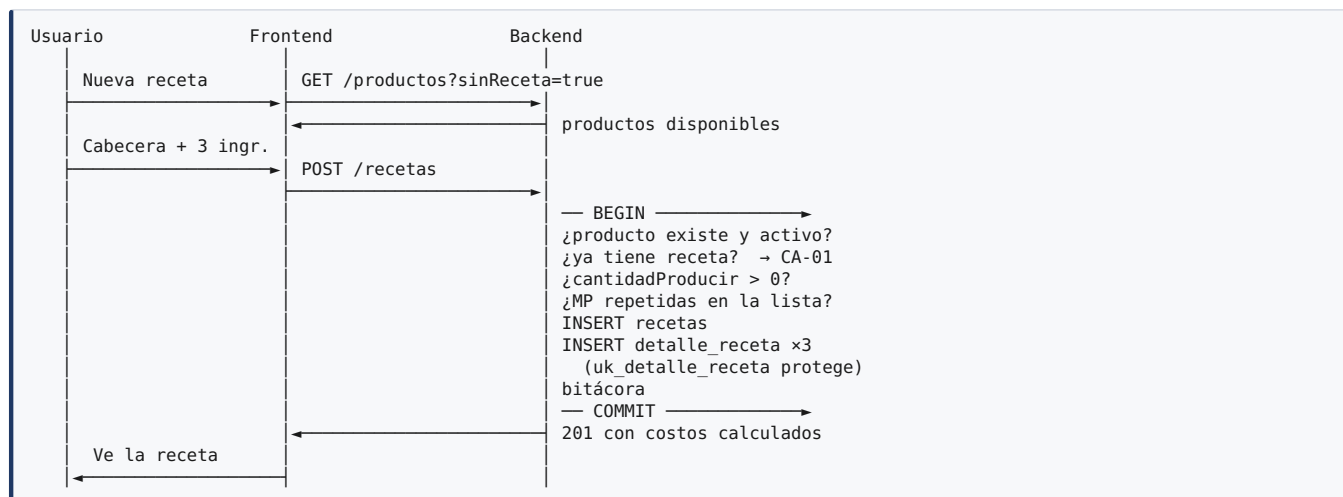
Admite hasta 3 decimales

Costo estimado: \$ 4.625,00

[Cancelar] [Agregar]

El selector **excluye los ingredientes ya presentes** (CA-05): el duplicado no llega ni a intentarse.

2. Flujo



Todo en una transacción: si falla el tercer ingrediente, no queda una receta a medias.

3. Componentes Angular Material

Elemento	Componente
Cabecera	<code>MatFormField</code> + <code>MatSelect</code>
Ayuda de rendimiento	<code>MatHint</code> destacado con ejemplo
Tabla de ingredientes	<code>MatTable</code> con acciones por fila
Agregar ingrediente	<code>MatDialog</code>
Cantidad	<code>MatInput[type=number]</code> con <code>step="0.001"</code>
Resumen de costos	<code>MatCard</code> al pie de la tabla
Quitar	<code>MatIconButton</code> + <code>ConfirmarDialogComponent</code>
Estado	<code>MatChip</code>

4. Validaciones en el formulario

Cabecera

Campo	Reglas	Mensaje
Producto	requerido, sin receta previa	"Seleccione un producto"
Nombre	requerido, máx. 100	"El nombre es obligatorio"
Cantidad a producir	requerida, > 0	"Debe ser mayor que cero"
Unidad	requerida	"Seleccione una unidad"

Ingrediente

Campo	Reglas	Mensaje
Materia prima	requerida, no repetida	"Seleccione una materia prima"
Cantidad	requerida, > 0, 3 decimales	"Debe ser mayor que cero"

El `step` del input de cantidad debe ser `0.001` (R-06). Con `0.01` no se puede registrar 0,005 kg de sal, y en repostería esa cifra es real.

5. Estados de la pantalla

Estado	Qué se muestra
Cargando	<code>MatProgressBar</code>
Receta sin ingredientes	⚠ "Esta receta no tiene ingredientes. No se podrá usar para planificar producción."
Sin productos disponibles	"Todos los productos ya tienen receta" o enlace a HU-11
Error	Mensaje + "Reintentar"
Guardando	Botón con spinner

El aviso de receta sin ingredientes dice **la consecuencia** (R-05), no solo que falta algo: es lo que hace que alguien la complete en lugar de dejarla así.

6. Accesibilidad y UX

- Todos los campos con `<mat-label>`; nunca solo *placeholder*.
- Navegación por teclado en orden lógico; `Enter` envía el formulario.
- Los mensajes de error del servidor se muestran **junto al campo** (`fieldErrors`).
- El botón de envío se deshabilita mientras la petición está en vuelo.
- Confirmación antes de descartar cambios sin guardar.
- Los tres estados obligatorios (cargando / vacío / error) según `00-base/05-ESTANDARES-QA.md` §7.

Tarea Backend

Primera historia con **cabecera y detalle**. El patrón (subrecurso anidado, transacción que crea ambos, restricción de unicidad en el detalle) se repite en HU-14, HU-17 y HU-21.

1. Pasos

1. `Receta` y `DetalleReceta` (POJO), con sus `RowMapper`. El de detalle hace `JOIN materias_primas` y `unidades_medida`.
2. `RecetaRepository`:
 - `buscarPorId`, `buscarPorProducto`, `existePorProducto(idProducto)`;
 - `insertar`, `actualizar`, `cambiarEstado`;
 - `buscar(filtros, page)`, `contar(...)`.
3. `DetalleRecetaRepository`:
 - `listarPorReceta`, `insertar`, `actualizarCantidad`, `eliminar`;
 - `existeMateriaPrima(idReceta, idMateriaPrima)` → CA-05.
4. DTO: `CrearRecetaRequest` (con lista de detalles), `ActualizarRecetaRequest`, `AgregarDetalleRequest`, `RecetaResponse`, `DetalleRecetaResponse`.
5. `RecetaService.crear()` `@Transactional`:
 - valida producto existente y activo → `PRODUCTO_INEXISTENTE`;
 - valida que no tenga receta → `PRODUCTO_YA_TIENE_RECETA` (CA-01);
 - valida `cantidadProducir > 0` → `CANTIDAD_PRODUCIR_INVALIDA` (R-03);
 - **valida que no haya materias primas repetidas en la lista recibida** antes de insertar (el `UNIQUE` lo impediría, pero con un error menos claro);
 - inserta cabecera y detalles; si algo falla, se revierte todo.
6. `RecetaService.agregarDetalle()`: valida duplicado → `MATERIA_PRIMA_DUPLICADA`; captura `DuplicateKeyException` por concurrencia (R-01).
7. Cálculo de costo estimado con `BigDecimal` y `RoundingMode.HALF_UP`: $\text{costoTotal} = \sum (\text{cantidad} \times \text{costo_unitario})$, $\text{costoPorUnidad} = \text{costoTotal} / \text{cantidadProducir}$.
8. `RecetaController` con los nueve endpoints y `@PreAuthorize`.
9. **DELETE permitido solo sobre el detalle** (R-07), nunca sobre la receta.
10. `@Auditable` en todas las escrituras, incluido el borrado de detalle.

2. Archivos

```
backend/src/main/java/co/edu/corposucre/productionfood/receta/
├── {Receta,DetalleReceta}.java
├── {RecetaRowMapper,DetalleRecetaRowMapper}.java
├── {RecetaRepository,DetalleRecetaRepository}.java
├── {RecetaService,RecetaController}.java
└── dto/{CrearRecetaRequest,ActualizarRecetaRequest,AgregarDetalleRequest,
    RecetaResponse,DetalleRecetaResponse}.java
```

3. Puntos de cuidado

- El `UNIQUE` de V2 es la garantía de CA-05 (R-01). Validar solo en el servicio deja pasar duplicados en peticiones simultáneas, y un ingrediente duplicado hace que HU-23 descuente el doble de inventario.
- Documentar en el código qué significa `cantidad_producir` (R-02): es el rendimiento de una tanda, no de una unidad. El cálculo de HU-22 depende de interpretarlo bien.
- Validar también los duplicados **dentro de la lista recibida** en `POST /recetas`: el `UNIQUE` los rechazaría, pero con un mensaje

menos útil.

- `cantidad` del detalle es `decimal(10,3)`, no `(10,2)` (R-06). Usar `BigDecimal` con la escala correcta.
- `cantidadProducir > 0` estrictamente (R-03): es divisor.
- `DELETE` solo sobre el detalle (R-07).

4. Checklist de cierre

- ☐ Todos los CA implementados y verificados manualmente.
- ☐ Endpoints documentados en Swagger con ejemplos.
- ☐ `@PreAuthorize` en cada método, según la matriz de roles.
- ☐ Operaciones de escritura auditadas en bitácora (HU-04).
- ☐ Pruebas unitarias de las reglas con ramificación.
- ☐ Sin `System.out.println` ni código comentado.
- ☐ PR bajo ~250 líneas, revisado y aprobado.

Tarea Frontend

Pantalla de cabecera más tabla de ingredientes editable. **La interfaz debe dejar explícito el significado de `cantidad_producir` (R-02):** es donde más fácilmente se introduce un error que no se detecta hasta la Fase 7.

1. Pasos

1. `receta.model.ts` y `receta.service.ts`.
2. `receta-lista.component`: producto, nombre de receta, rendimiento, nº de ingredientes, costo estimado, estado.
3. Marcar las recetas **sin ingredientes** como incompletas (R-05).
4. `receta-formulario.component` en dos secciones:
 - **Cabecera**: producto (solo los que no tienen receta), nombre, cantidad a producir, unidad;
 - **Ingredientes**: tabla editable con agregar, editar cantidad y quitar.
5. **Texto explícito junto a “cantidad a producir”**: “Cantidad que rinde una tanda completa de esta receta” con ejemplo. Es la mitigación de R-02.
6. Al agregar un ingrediente, **excluir del selector los ya incluidos** (CA-05): así el duplicado no llega ni a intentarse.
7. Costo estimado por ingrediente y total, recalculado en vivo.
8. **Mostrar el consumo por unidad**: “Esta receta usa 0,167 kg de harina por pan”, calculado como `cantidad / cantidadProducir`. Hace visible si el rendimiento se interpretó mal.
9. Confirmación al quitar un ingrediente.
10. Ruta: lectura ADMIN/PRODUCCION/CONSULTA; escritura ADMIN/PRODUCCION.

2. Archivos

```
frontend/src/app/features/receta/  
├── models/receta.model.ts  
├── services/receta.service.ts  
├── pages/{receta-lista, receta-formulario}/  
├── components/detalle-receta-tabla/  
└── receta.routes.ts
```

3. Puntos de cuidado

- **El punto 5 y el punto 8 son la defensa contra R-02.** Mostrar el consumo por unidad calculado hace evidente el error: si alguien pone “1” en cantidad a producir pensando en “por unidad”, verá que cada pan lleva 5 kg de harina y lo corregirá al instante.
- **Excluir del selector los ingredientes ya agregados** evita el `409` por completo.
- Las cantidades admiten **3 decimales** (R-06): el `step` del input debe ser `0.001`, no `0.01`. Con `0.01` no se puede registrar 0,005 kg de sal.
- El selector de producto solo muestra los que **no tienen receta** (CA-01).
- El costo estimado es referencial: indicarlo en la interfaz para que nadie lo tome como costo real.

4. Checklist de cierre

- ☐ Todos los CA cubiertos desde la interfaz.

- ☐ Los tres estados (cargando / vacío / error) implementados.
- ☐ Errores del servidor mostrados junto al campo correspondiente.
- ☐ Guard de rol aplicado en la ruta.
- ☐ Modelos TypeScript espejo de los DTO del backend.
- ☐ Probado en Chrome y Firefox a 1366×768.
- ☐ Sin `console.log` ni `any`.

Casos de Prueba

Formato y criterios en `00-base/05-ESTANDARES-QA.md`.

1. Casos funcionales

ID	CA	Descripción	Entrada	Resultado esperado	Tipo
CP-01	CA-01..03	Crear receta con detalle	Cabecera + 3 ingredientes	201 con receta y detalles	Positivo
CP-02	CA-01	Segunda receta para el mismo producto	POST con un idProducto que ya tiene receta	409 PRODUCTO_YA_TIENE_RECETA	Negativo
CP-03	CA-02	Nombre vacío	{"nombre":""}	400	Negativo
CP-04	CA-02	cantidadProducir en cero	{"cantidadProducir":0}	400 CANTIDAD_PRODUCIR_INVALIDA (R-03)	Negativo
CP-05	CA-02	cantidadProducir negativa	{"cantidadProducir":-5}	400	Negativo
CP-06	CA-02	Unidad inexistente	{"idUnidad":9999}	409	Negativo
CP-07	CA-02	Estado por defecto	Crear sin estado	activo: true	Positivo
CP-08	CA-03	Listar el detalle	GET /recetas/2	Detalles con nombre y unidad de cada materia prima	Positivo
CP-09	CA-04	Agregar un ingrediente	POST /recetas/2/detalles	201 ; el detalle crece	Positivo
CP-10	CA-04	Editar la cantidad	PUT /recetas/2/detalles/11 {"cantidad":6.5}	200 ; costo recalculado	Positivo
CP-11	CA-04	Quitar un ingrediente	DELETE /recetas/2/detalles/11	204 ; el detalle decrece (R-07)	Positivo
CP-12	CA-05	Ingrediente duplicado	POST con una materia prima ya presente	409 MATERIA_PRIMA_DUPLICADA	Negativo
CP-13	CA-05	Duplicado dentro del POST inicial	Crear receta con la misma MP dos veces en detalles[]	409 ; no se crea la receta	Negativo
CP-14	CA-05	Duplicado por SQL directo	INSERT directo en detalle_receta repitiendo la MP	La base lo rechaza — uk_detalle_receta (R-01)	Integridad
CP-15	—	Cantidad con 3 decimales	{"cantidad":0.005}	201 ; en base 0.005 exacto (R-06)	Borde
CP-16	—	Cantidad cero en el detalle	{"cantidad":0}	400 — CHECK > 0	Negativo
CP-17	—	Producto inexistente	{"idProducto":9999}	409 PRODUCTO_INEXISTENTE	Negativo
CP-18	—	Materia prima inexistente	{"idMateriaPrima":9999}	409 MATERIA_PRIMA_INEXISTENTE	Negativo
CP-19	—	Receta sin ingredientes	Crear solo la cabecera	201 ; marcada como incompleta (R-05)	Borde
CP-20	—	Cálculo del costo estimado	5 kg × \$3.200 + 0,05 kg × \$180	costoTotalEstimado = 16009.00	Positivo
CP-21	—	Costo por unidad	costoTotal 16.009 / cantidadProducir 30	533.63 (redondeo HALF_UP)	Positivo
CP-22	—	Transacción al crear	Provocar fallo en el tercer detalle	No se crea ni la receta ni los dos primeros detalles	Integridad
CP-23	—	Rol CONSULTA no puede editar	POST con token CONSULTA	403 SIN_PERMISO	Seguridad
CP-24	—	Rol PRODUCCION puede editar	POST con token PRODUCCION	201	Positivo

2. Batería de seguridad

Obligatoria en **cada endpoint** de esta historia:

ID	Caso	Esperado
SEC-01	Sin cabecera <code>Authorization</code>	<code>401 NO_AUTENTICADO</code>
SEC-02	Token malformado	<code>401 NO_AUTENTICADO</code>
SEC-03	Token expirado	<code>401 NO_AUTENTICADO</code>
SEC-04	Token de rol <code>sin</code> permiso	<code>403 SIN_PERMISO</code>
SEC-05	Token de usuario con <code>estado = 0</code>	<code>403 USUARIO_INACTIVO</code>

SEC-04 es el caso crítico. Si devuelve `200`, falta `@EnableMethodSecurity` y la autorización del sistema completo está desactivada.

CP-14 y la consulta final son las pruebas que evitan un desastre tardío.

Un ingrediente duplicado hace que HU-22 calcule el doble de consumo teórico y HU-23 descuente el doble de inventario. Una `cantidad_producir` mal interpretada multiplica todos los cálculos de producción por el rendimiento de la receta.

Ninguno de los dos errores produce un mensaje. Ambos se manifiestan en la Fase 7, lejos de su causa, como “el inventario no cuadra” y “la eficiencia da números raros”.

3. Verificación en base de datos

```
-- CP-14: la restricción de V2 §C-05. Esta sentencia DEBE fallar.
INSERT INTO detalle_receta (id_receta, id_materia_prima, cantidad) VALUES (2, 5, 1.000);
INSERT INTO detalle_receta (id_receta, id_materia_prima, cantidad) VALUES (2, 5, 2.000);
-- La segunda debe dar error 1062 (Duplicate entry).
-- Si ambas pasan, falta el UNIQUE y HU-23 descontará el doble de inventario.

SHOW CREATE TABLE detalle_receta\G
-- Se espera: UNIQUE KEY `uk_detalle_receta` (`id_receta`,`id_materia_prima`)

-- CP-02: un producto, una receta
SHOW CREATE TABLE recetas\G
-- Se espera: UNIQUE KEY `id_producto` (`id_producto`)

-- CP-15: los 3 decimales se conservan
SELECT id_detalle_receta, cantidad FROM detalle_receta WHERE id_receta = 2;
-- 0.005 debe verse como 0.005, no 0.01

-- CP-22: tras el fallo, ni receta ni detalles
SELECT COUNT(*) FROM recetas WHERE id_producto = 3;
SELECT COUNT(*) FROM detalle_receta WHERE id_receta = <la que no debió crearse>;

-- Verificación de R-02: el consumo teórico que usará HU-22
SELECT r.nombre, r.cantidad_producir, mp.nombre AS ingrediente, dr.cantidad,
       ROUND(dr.cantidad / r.cantidad_producir, 4) AS por_unidad
FROM recetas r
     JOIN detalle_receta dr USING (id_receta)
     JOIN materias_primas mp USING (id_materia_prima)
WHERE r.id_receta = 2;
-- "por_unidad" debe tener sentido físico: 0.1667 kg de harina por pan es razonable;
-- 5 kg de harina por pan indica que cantidad_producir se interpretó mal.
```

4. Criterios de cierre

- ☐ Todos los casos ejecutados; los positivos pasan.
- ☐ Los negativos devuelven el HTTP y el `code` esperados.
- ☐ SEC-01 a SEC-05 pasan en todos los endpoints.
- ☐ Ningún endpoint supera 2 s (RNF-02).
- ☐ Cero defectos críticos o altos abiertos.
- ☐ Colección de Postman actualizada y versionada.

HU-13 · Registro de compras

Especificación

Fase 5 · Fuente: `previo/Historias_de_Usuario.md`

Como comprador, **quiero** registrar una compra a proveedor con fecha y estado, **para que** documente las adquisiciones de materia prima.

Depende de: HU-06 (proveedores)

1. Criterios de aceptación

CA	Enunciado	Cómo se implementa
CA-01	<code>id_proveedor</code> debe existir	Validado contra <code>ProveedorRepository</code> ; se exige además que esté activo
CA-02	<code>fecha</code> obligatoria	<code>@NotNull</code> ; por defecto la fecha actual
CA-03	<code>estado</code> obligatorio	Valores canónicos: <code>PENDIENTE</code> , <code>RECIBIDA</code> , <code>CANCELADA</code> — <code>CHECK</code> en V2 §C-12
CA-04	Se retorna la compra creada con su <code>id_compra</code>	<code>201</code> + <code>Location</code> + <code>CompraResponse</code>

2. Endpoints

Método	Ruta	Descripción	Roles
GET	<code>/api/v1/compras</code>	Listar con filtros y paginación	ADMIN, COMPRAS, PRODUCCION, CONSULTA
GET	<code>/api/v1/compras/{id}</code>	Obtener compra con su detalle	ADMIN, COMPRAS, PRODUCCION, CONSULTA
POST	<code>/api/v1/compras</code>	Registrar una compra	ADMIN, COMPRAS
PUT	<code>/api/v1/compras/{id}</code>	Editar cabecera (solo si está PENDIENTE)	ADMIN, COMPRAS
PATCH	<code>/api/v1/compras/{id}/estado</code>	Cambiar estado · la recepción suma stock	ADMIN, COMPRAS

Reglas transversales (paginación, formato de error, autenticación) en `00-base/04-CONTRATO-API.md`.

3. Reglas de negocio

R-01 · Recibir una compra es el punto principal de entrada de stock

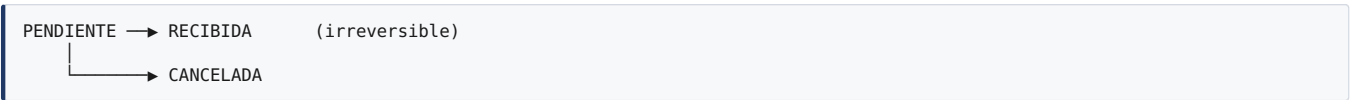
Es la corrección de fondo que hizo falta al esquema (`02-CORRECCIONES-DB.md` §C-03): el sistema descontaba materia prima pero **nada la sumaba**.

Al pasar una compra a `RECIBIDA`, en una única transacción:

- Se valida que la compra tenga al menos una línea de detalle.
- Por **cada línea** de `detalle_compra` se llama a `InventarioService.registrarEntrada(idMateriaPrima, cantidad, ENTRADA_COMPRA, "detalle_compra", idDetalle)`.
- Se actualiza `compras.estado = 'RECIBIDA'`.
- Se registra en bitácora con acción `RECEPCION_COMPRA`.

Si falla cualquier línea, **no entra ninguna**. Una recepción a medias deja un inventario que nadie puede explicar.

R-02 · `RECIBIDA` es un estado terminal



No se permite volver de `RECIBIDA` a `PENDIENTE`. Una vez que el stock entró, deshacerlo con un cambio de estado obligaría a restar de un inventario que quizá ya se consumió en producción, y el saldo podría quedar negativo.

Una recepción equivocada se corrige con un **movimiento de ajuste** explícito, que queda en el kardex con su motivo. La corrección es visible; deshacer el estado sería invisible.

Cualquier transición no permitida devuelve `409 TRANSICION_INVALIDA` (`04-CONTRATO-API.md` §7).

R-03 · Una compra sin detalle no se puede recibir

Recibir una compra vacía no sumaría nada y dejaría el estado en `RECIBIDA` sin efecto, imposibilitando después agregarle líneas (R-04). Se devuelve `409 COMPRA_SIN_DETALLE`.

R-04 · El detalle solo se modifica mientras está `PENDIENTE`

Agregar, editar o quitar líneas (HU-14) de una compra ya recibida alteraría el inventario sin dejar rastro del cambio. Una vez `RECIBIDA` o `CANCELADA`, el detalle queda congelado.

R-05 · Cancelar una compra recibida no devuelve el stock

`RECIBIDA` es terminal (R-02), así que no puede cancelarse. Solo se cancela desde `PENDIENTE`, cuando aún no ha entrado nada.

R-06 · Se puede comprar a un proveedor y que luego se desactive

La validación de proveedor activo (CA-01) es **solo al crear**. Las compras ya registradas de un proveedor desactivado siguen su curso normal, incluida su recepción: la mercancía ya fue pedida.

R-07 · La fecha puede ser pasada, no futura

Se registran compras de días anteriores. Una fecha futura carece de sentido para una adquisición ya realizada → `400 FECHA_FUTURA`.

4. Modelo de datos

Tabla `compras` (V2 añade el `CHECK` de estado y el índice `idx_compras_fecha`)

Columna	Tipo	Notas
<code>id_compra</code>	<code>int unsigned</code> PK AI	
<code>id_proveedor</code>	<code>int unsigned</code> NOT NULL FK	CA-01
<code>fecha</code>	<code>date</code> NOT NULL	CA-02, R-07
<code>estado</code>	<code>varchar(30)</code> NOT NULL	CA-03 · CHECK IN ('PENDIENTE','RECIBIDA','CANCELADA')

```
// POST /api/v1/compras
{ "idProveedor": 3, "fecha": "2026-09-18", "estado": "PENDIENTE" }

// 201 Created · Location: /api/v1/compras/12
{ "idCompra": 12,
  "proveedor": { "idProveedor": 3, "nombre": "Harinas del Caribe S.A.S." },
  "fecha": "2026-09-18", "estado": "PENDIENTE",
  "totalCompra": 0.00, "cantidadItems": 0, "editable": true }

// PATCH /api/v1/compras/12/estado
{ "estado": "RECIBIDA" }

// 200 OK – el stock ya entró
{ "idCompra": 12, "estado": "RECIBIDA", "totalCompra": 480000.00,
  "movimientosGenerados": [
    { "materiaPrima": "Harina de trigo", "cantidad": 100.00,
      "saldoAnterior": 12.50, "saldoPosterior": 112.50 },
    { "materiaPrima": "Azúcar", "cantidad": 50.00,
      "saldoAnterior": 45.00, "saldoPosterior": 95.00 }
  ]
}
```

Devolver los movimientos generados hace visible el efecto de la recepción en el momento mismo, sin ir a otra pantalla a comprobarlo.

Campos ordenables: `fecha`, `estado`, `id`.

5. Códigos de error

code	HTTP	Cuándo
PROVEEDOR_INEXISTENTE	409	El <code>idProveedor</code> no existe
PROVEEDOR_INACTIVO	409	El proveedor está desactivado (solo al crear)
TRANSICION_INVALIDA	409	Transición de estado no permitida (R-02)
COMPRA_SIN_DETALLE	409	Se intenta recibir una compra sin líneas (R-03)
COMPRA_NO_EDITABLE	409	Se intenta modificar una compra que no está <code>PENDIENTE</code> (R-04)
FECHA_FUTURA	400	La fecha de compra es posterior a hoy (R-07)

Los transversales (`VALIDACION_FALLIDA` , `NO_AUTENTICADO` , `SIN_PERMISO` , `RECURSO_NO_ENCONTRADO`) están en `00-base/04-CONTRATO-API.md` §5.

6. Fuera de alcance

Lo que **no** cubre esta historia y no debe implementarse aquí: se limita estrictamente a los criterios de aceptación listados arriba. Cualquier funcionalidad adicional se propone como historia nueva, no se agrega en silencio.

7. Definición de Terminado

Se aplica la DoD de `00-base/01-CONVENCIONES.md` §7.

8. Notas

La recepción es la operación de mayor impacto del sistema

Es el punto donde el inventario crece, y por eso está restringida a `ADMIN` y `COMPRAS` (`00-base/03-MATRIZ-ROLES.md`). Su contrapartida —el descuento por producción— está en HU-23.

Entre las dos sostienen todo el control de inventario: si cualquiera de las dos falla, los saldos dejan de significar algo y HU-10, HU-26 y HU-27 informan sobre datos falsos.

Diseño

1. Pantallas

Listado de compras

Compras

[+ Nueva compra]

Proveedor

Todos

Estado

Todos

Desde

01/09/26

Hasta

18/...

N.º	PROVEEDOR	FECHA	ÍTEMS	TOTAL	ESTADO
#12	Harinas del Caribe	18/09/2026	2	\$ 480.000,00	● Pendien
#11	Distribuidora Lácteos	15/09/2026	4	\$ 1.240.000,00	● Recibid
#10	Insumos del Sinú	12/09/2026	1	\$ 85.000,00	○ Cancela

Diálogo de recepción — la pantalla más importante de la historia

Este diálogo hace tres cosas que importan:

1. **Muestra el efecto antes de aplicarlo**, línea por línea.
2. **Advierte que es irreversible** (R-02), con el motivo.
3. Convierte una acción de menú en una decisión consciente.

Es la operación que más altera el estado del sistema; un clic distraído no debería poder dispararla.

Detalle de compra

Cuando la compra está **RECIBIDA** o **CANCELADA**, los botones de edición del detalle **desaparecen** (R-04), no se muestran deshabilitados: es más claro.

2. Flujo



3. Componentes Angular Material

Elemento	Componente
Tabla de compras	<code>MatTable</code> + <code>MatSort</code> + <code>MatPaginator</code>
Estado	<code>MatChip</code> con color
Filtros de fecha	<code>MatDateRangePicker</code>
Proveedor / estado	<code>MatSelect</code>
Diálogo de recepción	<code>MatDialog</code> con tabla de impacto y aviso destacado
Acciones de estado	<code>MatRaisedButton</code> (Recibir) · <code>MatButton</code> (Cancelar)
Total	Pipe <code>currency: 'COP'</code>

4. Validaciones en el formulario

Campo	Reglas	Mensaje
Proveedor	requerido, activo	“Seleccione un proveedor”
Fecha	requerida, no futura	“La fecha no puede ser posterior a hoy”
Estado	requerido, valor canónico	—

Al crear, el estado es siempre `PENDIENTE`: no se ofrece como opción editable. Los cambios posteriores pasan por el flujo de estados, no por el formulario.

5. Estados de la pantalla

Estado	Qué se muestra
Cargando	<code>MatProgressBar</code>
Vacío	“No hay compras registradas” + “Registrar la primera”
Vacío con filtro	“Ninguna compra coincide con los filtros”
Compra sin ítems	⚠ “Agregue al menos un ítem antes de recibir” (R-03)
Recibiendo	Botón con spinner; diálogo bloqueado
Recibida	Confirmación con los saldos resultantes
Error	Mensaje del servidor + “Reintentar”

6. Accesibilidad y UX

- Todos los campos con `<mat-label>`; nunca solo *placeholder*.
- Navegación por teclado en orden lógico; `Enter` envía el formulario.
- Los mensajes de error del servidor se muestran **junto al campo** (`fieldErrors`).
- El botón de envío se deshabilita mientras la petición está en vuelo.
- Confirmación antes de descartar cambios sin guardar.
- Los tres estados obligatorios (cargando / vacío / error) según `00-base/05-ESTANDARES-QA.md` §7.

Tarea Backend

La operación más delicada de la fase: **recibir una compra suma stock**. Usa el `InventarioService` creado en HU-09. Requiere una máquina de estados explícita.

1. Pasos

1. `EstadoCompra` (enum): `PENDIENTE`, `RECIBIDA`, `CANCELADA`, con `transicionesValidas()` — un único lugar donde vive la máquina de estados (R-02).
2. `Compra` (POJO) y `CompraRowMapper` con `JOIN proveedores`; el `SELECT` del listado calcula `totalCompra` con una subconsulta `SUM(cantidad * precio_unitario)` sobre `detalle_compra`.
3. `CompraRepository`: `buscar`, `contar`, `buscarPorId`, `insertar`, `actualizar`, `cambiarEstado`, `contarDetalles(idCompra)`.
4. DTO: `CrearCompraRequest`, `ActualizarCompraRequest`, `CambiarEstadoCompraRequest`, `CompraResponse`, `RecepcionResponse`.
5. `CompraService.crear()`: valida proveedor existente y activo, valida fecha no futura.
6. `CompraService.actualizar()`: solo si `estado == PENDIENTE` → `COMPRA_NO_EDITABLE` (R-04).
7. `CompraService.cambiarEstado()` `@Transactional` — el núcleo de la historia:
 - valida la transición con el enum → `TRANSICION_INVALIDA`;
 - si el nuevo estado es `RECIBIDA`:

- verifica que haya detalle → `COMPRA_SIN_DETALLE` (R-03);
 - **por cada línea**, `inventarioService.registrarEntrada(...)` con tipo `ENTRADA_COMPRA` y origen `detalle_compra / idDetalle`;
 - recoge los saldos anterior y posterior para la respuesta;
 - actualiza el estado;
 - `@Auditable(accion="RECEPCION_COMPRA")`.
8. Exponer `esEditable(idCompra)` para que HU-14 lo consulte antes de tocar el detalle.
9. Controlador con `@PreAuthorize`: escritura y recepción solo ADMIN y COMPRAS.

2. Archivos

```
backend/src/main/java/co/edu/corposucre/productionfood/compra/
├── {Compra, CompraRowMapper, CompraRepository, CompraService, CompraController}.java
├── EstadoCompra.java           ← máquina de estados en un solo lugar
└── dto/{CrearCompraRequest, ActualizarCompraRequest, CambiarEstadoCompraRequest,
        CompraResponse, RecepcionResponse}.java
```

3. Puntos de cuidado

- **Toda la recepción en UNA transacción** (R-01). Si la tercera línea falla, las dos primeras no deben haber entrado. Una recepción parcial produce un inventario que nadie puede explicar ni corregir con confianza.
- **RECIBIDA es terminal** (R-02). Si alguien pide “poder deshacer la recepción”, la respuesta está en R-02: se corrige con un ajuste de inventario, que queda registrado.
- **Las entradas pasan por `InventarioService`**, nunca con un `UPDATE inventario` directo. Cada línea genera su movimiento de kardex.
- La máquina de estados vive **en el enum**, no dispersa en `if` por el servicio: así una transición nueva se agrega en un solo sitio.
- Validar proveedor activo solo al crear (R-06).
- El **CHECK** de estado ya está en la base; validar antes da un mensaje mucho más útil.

4. Checklist de cierre

- ☐ Todos los CA implementados y verificados manualmente.
- ☐ Endpoints documentados en Swagger con ejemplos.
- ☐ `@PreAuthorize` en cada método, según la matriz de roles.
- ☐ Operaciones de escritura auditadas en bitácora (HU-04).
- ☐ Pruebas unitarias de las reglas con ramificación.
- ☐ Sin `System.out.println` ni código comentado.
- ☐ PR bajo ~250 líneas, revisado y aprobado.
- ☐ **Probado: recepción con fallo en una línea no deja ninguna entrada aplicada.**
- ☐ **Probado: `RECIBIDA → PENDIENTE` devuelve `409`.**

Tarea Frontend

Listado de compras y flujo de recepción. **La pantalla debe dejar claro que recibir una compra modifica el inventario de forma irreversible.**

1. Pasos

1. `compra.model.ts` con el enum `EstadoCompra`, y `compra.service.ts`.
2. `compra-lista.component`: número, proveedor, fecha, nº de ítems, total, estado, `:`.
3. Chip de estado con color: ámbar `PENDIENTE`, verde `RECIBIDA`, gris `CANCELADA`.
4. Filtros: proveedor, estado, rango de fechas.
5. `compra-formulario.component`: proveedor, fecha (por defecto hoy, `max` = hoy).
6. `compra-detalle.component`: cabecera + tabla de líneas (HU-14) + acciones de estado.
7. **Diálogo de recepción** que muestra, **antes de confirmar**, el efecto exacto sobre el inventario línea por línea (ver `diseño.md`).
8. Tras recibir, mostrar los saldos resultantes.
9. Los controles de edición solo aparecen si `editable === true` (R-04).
10. Ruta: lectura ADMIN/COMPRAS/PRODUCCION/CONSULTA; escritura ADMIN/COMPRAS.

2. Archivos

```
frontend/src/app/features/compra/  
├── models/compra.model.ts  
├── services/compra.service.ts  
├── pages/{compra-lista, compra-formulario, compra-detalle}/  
├── components/recepcion-dialog/  
└── compra.routes.ts
```

3. Puntos de cuidado

- **El diálogo de recepción debe decir “esta acción no se puede deshacer”** y mostrar qué va a pasar con cada materia prima. Es una operación irreversible que altera el inventario: no puede dispararse con un clic distraído en un menú.
- Ocultar los botones de edición cuando `editable === false`: es más claro que mostrarlos y devolver `409`.
- `max` del datepicker = hoy (R-07).
- El total de la compra lo calcula el backend; el frontend solo lo muestra.
- El chip de estado debe ser legible de un vistazo: es lo que se busca al abrir el listado.

4. Checklist de cierre

- ☐ Todos los CA cubiertos desde la interfaz.
- ☐ Los tres estados (cargando / vacío / error) implementados.
- ☐ Errores del servidor mostrados junto al campo correspondiente.
- ☐ Guard de rol aplicado en la ruta.
- ☐ Modelos TypeScript espejo de los DTO del backend.
- ☐ Probado en Chrome y Firefox a 1366×768.
- ☐ Sin `console.log` ni `any`.

Casos de Prueba

Formato y criterios en `00-base/05-ESTANDARES-QA.md`.

1. Casos funcionales

ID	CA	Descripción	Entrada	Resultado esperado	Tipo
CP-01	CA-01..04	Crear compra válida	<code>{"idProveedor":3,"fecha":"2026-09-18","estado":"PENDIENTE"}</code>	201 con <code>idCompra</code>	Positivo
CP-02	CA-01	Proveedor inexistente	<code>{"idProveedor":9999}</code>	409 <code>PROVEEDOR_INEXISTENTE</code>	Negativo
CP-03	CA-01	Proveedor desactivado	Proveedor con <code>estado = 0</code>	409 <code>PROVEEDOR_INACTIVO</code>	Negativo
CP-04	CA-02	Fecha omitida	Sin <code>fecha</code>	400 o se asigna hoy por defecto	Borde
CP-05	CA-02	Fecha futura	<code>"2027-01-01"</code>	400 <code>FECHA_FUTURA</code> (R-07)	Negativo
CP-06	CA-02	Fecha pasada	<code>"2026-08-01"</code>	201 — permitido (R-07)	Borde
CP-07	CA-03	Estado inválido	<code>{"estado":"ENVIADA"}</code>	400 — fuera de los valores canónicos	Negativo
CP-08	CA-03	Estado en minúsculas	<code>{"estado":"pendiente"}</code>	400 o se normaliza; nunca se guarda en minúsculas	Borde
CP-09	—	Recibir una compra suma al inventario	Compra con 2 líneas; <code>PATCH</code> a <code>RECIBIDA</code>	200 ; ambos inventarios suben exactamente (R-01)	Integridad
CP-10	—	Se generan los movimientos de kardex	Tras CP-09	2 movimientos <code>ENTRADA_COMPRA</code> con origen <code>detalle_compra</code>	Integridad
CP-11	—	RECIBIDA no vuelve a PENDIENTE	<code>PATCH</code> de <code>RECIBIDA</code> a <code>PENDIENTE</code>	409 <code>TRANSICION_INVALIDA</code> (R-02)	Negativo
CP-12	—	No se puede cancelar una compra recibida	<code>PATCH</code> de <code>RECIBIDA</code> a <code>CANCELADA</code>	409 <code>TRANSICION_INVALIDA</code> (R-05)	Negativo
CP-13	—	Cancelar desde PENDIENTE	<code>PATCH</code> a <code>CANCELADA</code>	200 ; el inventario no cambia	Positivo
CP-14	—	Recibir compra sin detalle	Compra sin líneas → <code>RECIBIDA</code>	409 <code>COMPRA_SIN_DETALLE</code> (R-03)	Negativo
CP-15	—	Recibir dos veces no suma dos veces	<code>PATCH</code> a <code>RECIBIDA</code> sobre una ya recibida	409 <code>TRANSICION_INVALIDA</code> ; el inventario no cambia	Integridad
CP-16	—	Transaccionalidad de la recepción	Compra de 3 líneas; provocar fallo en la tercera	Ninguna de las tres entra al inventario (R-01)	Integridad
CP-17	—	Editar compra PENDIENTE	<code>PUT</code> sobre una pendiente	200	Positivo
CP-18	—	Editar compra RECIBIDA	<code>PUT</code> sobre una recibida	409 <code>COMPRA_NO_EDITABLE</code> (R-04)	Negativo
CP-19	—	Compra de proveedor desactivado después	Crear, desactivar proveedor, recibir	200 — la recepción procede (R-06)	Borde
CP-20	—	Rol VENTAS no puede crear compras	Token de VENTAS	403 <code>SIN_PERMISO</code>	Seguridad
CP-21	—	Rol PRODUCCION no puede recibir	<code>PATCH</code> con token PRODUCCION	403 <code>SIN_PERMISO</code>	Seguridad
CP-22	—	Rol COMPRAS puede recibir	<code>PATCH</code> con token COMPRAS	200	Positivo

2. Batería de seguridad

Obligatoria en **cada endpoint** de esta historia:

ID	Caso	Esperado
SEC-01	Sin cabecera <code>Authorization</code>	401 <code>NO_AUTENTICADO</code>
SEC-02	Token malformado	401 <code>NO_AUTENTICADO</code>
SEC-03	Token expirado	401 <code>NO_AUTENTICADO</code>
SEC-04	Token de rol sin permiso	403 <code>SIN_PERMISO</code>
SEC-05	Token de usuario con <code>estado = 0</code>	403 <code>USUARIO_INACTIVO</code>

SEC-04 es el caso crítico. Si devuelve `200`, falta `@EnableMethodSecurity` y la autorización del sistema completo está desactivada.

CP-15 y CP-16 son los casos críticos de esta historia.

CP-15 protege contra la doble suma por doble recepción: si el estado no se valida antes de sumar, dos clics en “Recibir” duplican el stock sin ningún mensaje.

CP-16 verifica que la transacción envuelve **toda** la recepción. Una recepción parcial —tres líneas, dos aplicadas— deja el inventario en un estado que ni el operador ni el kardex pueden explicar, y que no se puede deshacer porque `RECIBIDA` es terminal.

3. Verificación en base de datos

```
-- CP-09 y CP-10: antes de recibir
SELECT id_materia_prima, cantidad_disponible FROM inventario WHERE id_materia_prima IN (5,6);
-- 5 → 12.50 ; 6 → 45.00

-- (ejecutar PATCH /compras/12/estado {"estado":"RECIBIDA"} con 100 kg y 50 kg)

-- después
SELECT id_materia_prima, cantidad_disponible FROM inventario WHERE id_materia_prima IN (5,6);
-- 5 → 112.50 ; 6 → 95.00

SELECT tipo_movimiento, cantidad, saldo_anterior, saldo_posterior, tabla_origen, id_origen
FROM movimientos_inventario_mp
WHERE tabla_origen = 'detalle_compra' ORDER BY id_movimiento DESC LIMIT 2;
-- ENTRADA_COMPRA con los saldos correctos y el id de cada línea

-- CP-16: tras el fallo forzado, NINGUNA entrada debe haberse aplicado
SELECT id_materia_prima, cantidad_disponible FROM inventario WHERE id_materia_prima IN (5,6,7);
-- Idénticos a los valores previos

-- CP-15: una compra recibida no puede generar movimientos nuevos
SELECT COUNT(*) FROM movimientos_inventario_mp
WHERE tabla_origen = 'detalle_compra'
AND id_origen IN (SELECT id_detalle_compra FROM detalle_compra WHERE id_compra = 12);
-- Debe ser igual al número de líneas, nunca el doble

-- Conciliación general (05-ESTANDARES-QA.md §5): 0 filas
```

4. Criterios de cierre

- ☐ Todos los casos ejecutados; los positivos pasan.
- ☐ Los negativos devuelven el HTTP y el `code` esperados.
- ☐ SEC-01 a SEC-05 pasan en todos los endpoints.
- ☐ Ningún endpoint supera 2 s (RNF-02).
- ☐ Cero defectos críticos o altos abiertos.
- ☐ Colección de Postman actualizada y versionada.

HU-14 · Detalle de compra

Especificación

Fase 5 · Fuente: `previo/Historias_de_Usuario.md`

Como comprador, **quiero** agregar detalles a la compra (materia prima, cantidad, precio unitario), **para que** el sistema registre los ítems y calcule el total.

Depende de: HU-13, HU-08

1. Criterios de aceptación

CA	Enunciado	Cómo se implementa
CA-01	<code>id_compra</code> debe existir	Validado; además debe estar <code>PENDIENTE</code> (HU-13 R-04)
CA-02	<code>id_materia_prima</code> debe existir	Validado contra <code>MateriaPrimaRepository</code>
CA-03	<code>cantidad</code> y <code>precio_unitario</code> obligatorios (decimales)	<code>@NotNull</code> ; <code>CHECK (cantidad > 0)</code> y <code>CHECK (precio >= 0)</code> en V2
CA-04	Calcular el total de la compra	<code>SUM(cantidad * precio_unitario)</code> — calculado por el backend, nunca por el cliente
CA-05	Listar el detalle de una compra	<code>GET /compras/{id}/detalles</code>

2. Endpoints

Método	Ruta	Descripción	Roles
<code>GET</code>	<code>/api/v1/compras/{idCompra}/detalles</code>	Listar el detalle	ADMIN, COMPRAS, PRODUCCION, CONSULTA
<code>POST</code>	<code>/api/v1/compras/{idCompra}/detalles</code>	Agregar un ítem	ADMIN, COMPRAS
<code>PUT</code>	<code>/api/v1/compras/{idCompra}/detalles/{id}</code>	Editar cantidad o precio	ADMIN, COMPRAS
<code>DELETE</code>	<code>/api/v1/compras/{idCompra}/detalles/{id}</code>	Quitar un ítem	ADMIN, COMPRAS

Reglas transversales (paginación, formato de error, autenticación) en `00-base/04-CONTRATO-API.md`.

3. Reglas de negocio

R-01 · El detalle solo se modifica mientras la compra está `PENDIENTE`

Es la regla de HU-13 R-04 aplicada aquí. Modificar una línea de una compra ya `RECIBIDA` alteraría el total sin que el inventario correspondiente cambie: el documento diría una cosa y el stock reflejaría otra, sin rastro del desajuste.

Los cuatro endpoints verifican el estado de la compra antes de actuar → `409 COMPRA_NO_EDITABLE`.

R-02 · Una materia prima no puede repetirse en la misma compra

V2 añadió `UNIQUE (id_compra, id_materia_prima)` (§C-13). Sin esa restricción, dos líneas de harina en la misma compra harían que la recepción de HU-13 sumara dos veces por caminos distintos y que el total fuera ambiguo de interpretar.

Al agregar una materia prima ya presente → `409 MATERIA_PRIMA_DUPLICADA`. **Lo correcto es editar la línea existente**, no crear otra.

R-03 · El total lo calcula el backend, siempre

```
subtotal_línea = cantidad × precio_unitario
total_compra  = Σ subtotales
```

Con `BigDecimal` y `RoundingMode.HALF_UP`. **El frontend nunca calcula el total que se guarda o se informa**; puede mostrar una previsualización mientras se escribe, pero la cifra oficial viene del servidor.

Si ambas capas calcularan, dos reglas de redondeo distintas producirían diferencias de centavos imposibles de explicar (`04-CONTRATO-API.md` §6).

R-04 · El precio de la compra es independiente del costo de referencia

`detalle_compra.precio_unitario` es lo que **realmente se pagó** en esta compra concreta. `materias_primas.costo_unitario` (HU-08 R-03) es un valor de referencia para costear recetas.

Registrar una compra no actualiza el costo de referencia: hacerlo lo sobrescribiría con el precio puntual de la última compra, que puede ser atípico por volumen o negociación.

La interfaz muestra ambos para que la diferencia sea visible y quien compra pueda juzgarla.

R-05 · El precio puede ser cero

`CHECK (precio_unitario >= 0)`, no `> 0`: hay muestras y bonificaciones. La cantidad, en cambio, debe ser estrictamente positiva (`CHECK > 0`): una línea de cero unidades no es una línea.

R-06 · Se permite eliminar líneas (mientras esté **PENDIENTE**)

Igual que en HU-12 R-07: una línea de detalle no tiene valor histórico propio mientras el documento no se haya ejecutado. Una vez **RECIBIDA**, nada del detalle se toca.

R-07 · Las cantidades del detalle son `decimal(10,2)`

A diferencia de `detalle_receta`, que usa 3 decimales (HU-12 R-06). Se compra en kilos y litros enteros o con dos decimales; no hace falta más precisión.

4. Modelo de datos

Tabla `detalle_compra` (V2 añade el `UNIQUE` y dos `CHECK`)

Columna	Tipo	Notas
<code>id_detalle_compra</code>	<code>int unsigned</code> PK AI	
<code>id_compra</code>	<code>int unsigned</code> NOT NULL FK	CA-01
<code>id_materia_prima</code>	<code>int unsigned</code> NOT NULL FK	CA-02
<code>cantidad</code>	<code>decimal(10,2)</code> NOT NULL	CA-03 · <code>CHECK > 0</code> · R-07
<code>precio_unitario</code>	<code>decimal(10,2)</code> NOT NULL	CA-03 · <code>CHECK >= 0</code> · R-05

`UNIQUE KEY uk_detalle_compra (id_compra, id_materia_prima)` — añadido en V2 §C-13.

```
// POST /api/v1/compras/12/detalles
{ "idMateriaPrima": 5, "cantidad": 100.00, "precioUnitario": 3200.00 }

// 201 Created
{ "idDetalleCompra": 28,
  "materiaPrima": { "idMateriaPrima": 5, "nombre": "Harina de trigo", "unidad": "kg" },
  "cantidad": 100.00, "precioUnitario": 3200.00, "subtotal": 320000.00,
  "costoReferencia": 3200.00, "diferenciaCosto": 0.00,
  "totalCompra": 320000.00 }

// GET /api/v1/compras/12/detalles
{
  "idCompra": 12, "estado": "PENDIENTE", "editable": true,
  "detalles": [
    { "idDetalleCompra": 28, "materiaPrima": {...}, "cantidad": 100.00,
      "precioUnitario": 3200.00, "subtotal": 320000.00 },
    { "idDetalleCompra": 29, "materiaPrima": {...}, "cantidad": 50.00,
      "precioUnitario": 3200.00, "subtotal": 160000.00 }
  ],
  "totalCompra": 480000.00, "cantidadItems": 2
}
```

`diferenciaCosto` compara el precio pagado con el costo de referencia (R-04): hace visible si esta compra salió más cara o más barata de lo previsto.

5. Códigos de error

code	HTTP	Cuándo
COMPRA_INEXISTENTE	404	La compra no existe
COMPRA_NO_EDITABLE	409	La compra no está PENDIENTE (R-01)
MATERIA_PRIMA_INEXISTENTE	409	El idMateriaPrima no existe o está inactiva
MATERIA_PRIMA_DUPLICADA	409	Esa materia prima ya está en la compra (R-02)
CANTIDAD_INVALIDA	400	cantidad menor o igual a cero

Los transversales (**VALIDACION_FALLIDA** , **NO_AUTENTICADO** , **SIN_PERMISO** , **RECURSO_NO_ENCONTRADO**) están en **00-base/04-CONTRATO-API.md** §5.

6. Fuera de alcance

Lo que **no** cubre esta historia y no debe implementarse aquí: se limita estrictamente a los criterios de aceptación listados arriba. Cualquier funcionalidad adicional se propone como historia nueva, no se agrega en silencio.

7. Definición de Terminado

Se aplica la DoD de **00-base/01-CONVENCIONES.md** §7.

8. Notas

Este detalle es lo que la recepción convierte en inventario

Cada línea de **detalle_compra** genera un movimiento **ENTRADA_COMPRA** cuando HU-13 recibe la compra. Por eso el **UNIQUE** de R-02 importa: una línea duplicada se traduciría directamente en stock duplicado.

El mismo patrón de cabecera-detalle aparece en HU-12 (recetas), HU-17 (pedidos) y HU-21 (planes de producción). Las restricciones de unicidad de V2 §C-13 cubren los cuatro casos.

Diseño

1. Pantallas

Tabla de ítems (dentro de la pantalla de HU-13)

ÍTEMS DE LA COMPRA #12				[+ Agregar ítem]	
MATERIA PRIMA	CANTIDAD	PRECIO UNIT.	vs. REF.	SUBTOTAL	...
Harina de trigo	100,00 kg	\$ 3.200,00	=	\$ 320.000,00	🗑
Azúcar	50,00 kg	\$ 3.500,00	↑ +\$300	\$ 175.000,00	🗑
Levadura	2,00 kg	\$ 150,00	↓ -\$30	\$ 300,00	🗑
TOTAL:				\$ 495.300,00	
3 ítems					

La columna **vs. REF.** compara con el costo de referencia de HU-08 (R-04): ↑ rojo más caro · ↓ verde más barato · = igual. Hace visible de inmediato si el proveedor subió precios, sin tener que recordar cuánto costaba antes.

El TOTAL es el que devuelve el backend (R-03), no una suma del navegador.

Agregar ítem

Agregar ítem a la compra
[X]

Materia prima *

Mantequilla (kg)

Solo se muestran las que aún no están en esta compra. Para cambiar una cantidad, edite la fila.

Cantidad *

20,00
kg

Precio unitario *

\$ 18.500,00

Costo de referencia:
\$ 18.500,00

Subtotal:
\$ 370.000,00

[Cancelar]
[Agregar]

El precio viene **prellenado con el costo de referencia** y es editable. Ahorra tecleo en el caso normal y hace evidente la desviación cuando el proveedor cobra otra cosa.

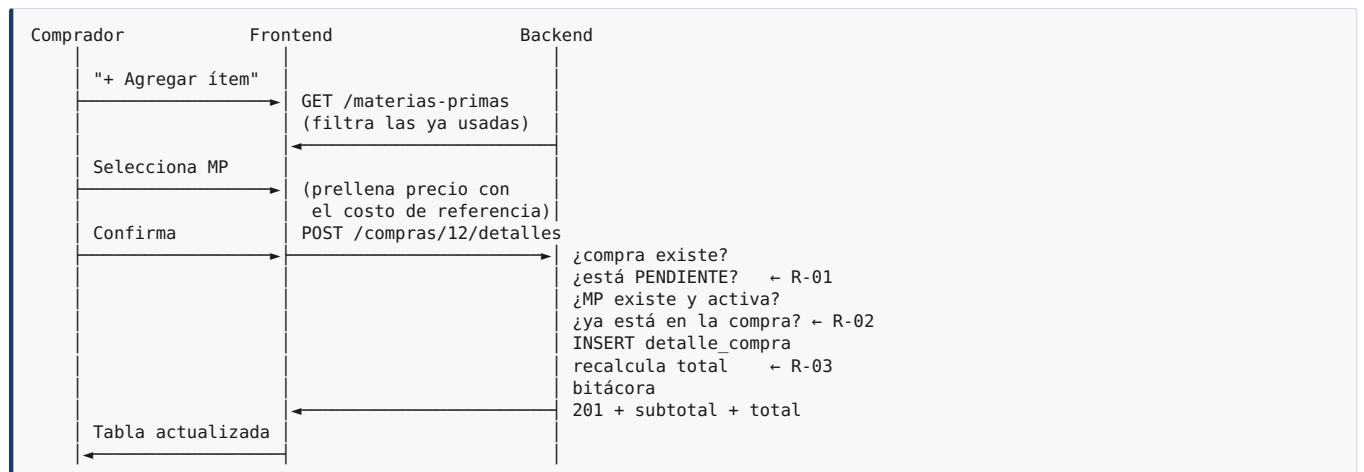
El selector excluye las materias primas ya presentes (R-02) y lo explica: sin esa nota, quien no encuentra la harina en la lista piensa que falta en el catálogo.

Compra ya recibida — modo lectura

ÍTEMS DE LA COMPRA #11			🔒 Compra recibida
MATERIA PRIMA	CANTIDAD	PRECIO UNIT.	SUBTOTAL
Leche entera	200,00 L	\$ 2.800,00	\$ 560.000,00
Mantequilla	40,00 kg	\$ 17.000,00	\$ 680.000,00
TOTAL:			\$ 1.240.000,00

Sin botón de agregar y sin acciones por fila (R-01). Los controles se ocultan, no se deshabilitan: un botón gris invita a preguntarse por qué no funciona.

2. Flujo



El total siempre viene del backend. El frontend lo muestra, no lo calcula.

3. Componentes Angular Material

Elemento	Componente
Tabla de ítems	<code>MatTable</code> con fila de totales (<code>matFooterRow</code>)
Agregar	<code>MatDialog</code>
Materia prima	<code>MatSelect</code> filtrado
Cantidad / precio	<code>MatInput[type=number]</code> con <code>step="0.01"</code>
Comparativo de costo	<code>MatIcon</code> ↑↓ con color + <code>MatTooltip</code>
Subtotal y total	Pipe <code>currency:'COP': 'symbol-narrow': '1.2-2'</code>
Eliminar	<code>MatIconButton</code> + <code>ConfirmarDialogComponent</code>
Modo lectura	<code>MatIcon</code> <code>lock</code> en la cabecera

4. Validaciones en el formulario

Campo	Reglas	Mensaje
Materia prima	requerida, activa, no repetida	"Seleccione una materia prima"
Cantidad	requerida, > 0, 2 decimales	"Debe ser mayor que cero"
Precio unitario	requerido, ≥ 0, 2 decimales	"No puede ser negativo"

La asimetría entre ambos campos es deliberada (R-05): una línea de cero unidades no tiene sentido, pero una de precio cero sí — es una bonificación o una muestra.

5. Estados de la pantalla

Estado	Qué se muestra
Cargando	<code>MatProgressBar</code> sobre la tabla
Sin ítems, compra editable	⚠ "Esta compra no tiene ítems. Agregue al menos uno antes de recibirla."
Sin ítems, compra cerrada	"Esta compra no registra ítems"
Compra no editable	🔒 Modo lectura, sin controles de escritura
Guardando	Fila con spinner; el resto de la tabla sigue usable
Error	Mensaje del servidor junto al campo, o <code>MatSnackBar</code>

El aviso de "sin ítems" enlaza con HU-13 R-03: una compra vacía no se puede recibir, y conviene saberlo antes de intentarlo.

6. Accesibilidad y UX

- Todos los campos con `<mat-label>` ; nunca solo *placeholder*.
- Navegación por teclado en orden lógico; `Enter` envía el formulario.
- Los mensajes de error del servidor se muestran **junto al campo** (`fieldErrors`).
- El botón de envío se deshabilita mientras la petición está en vuelo.
- Confirmación antes de descartar cambios sin guardar.
- Los tres estados obligatorios (cargando / vacío / error) según `00-base/05-ESTANDARES-QA.md` §7.

Tarea Backend

Subrecurso de compras con cálculo de totales. Repite el patrón de detalle de HU-12, añadiendo la verificación de estado del documento padre.

1. Pasos

1. `DetalleCompra` (POJO) y `DetalleCompraRowMapper` con `JOIN materias_primas` y `unidades_medida` ; incluye `costo_unitario` para el comparativo de R-04.
2. `DetalleCompraRepository` :
 - `listarPorCompra(idCompra)` , `insertar` , `actualizar` , `eliminar` ;
 - `existeMateriaPrima(idCompra, idMateriaPrima)` → R-02;
 - `calcularTotal(idCompra)` → `SELECT COALESCE(SUM(cantidad * precio_unitario), 0)` .
3. DTO: `AgregarDetalleCompraRequest` , `ActualizarDetalleCompraRequest` , `DetalleCompraResponse` , `DetalleCompraListaResponse` (con el total).
4. `DetalleCompraService` , todos los métodos con verificación previa:
 - la compra existe → `404 COMPRA_INEXISTENTE` ;

- la compra está **PENDIENTE** → **409 COMPRA_NO_EDITABLE** (R-01);
 - la materia prima existe y está activa;
 - no está duplicada → **409 MATERIA_PRIMA_DUPLICADA**, capturando también **DuplicateKeyException**.
5. **Cálculo del total con **BigDecimal** y **setScale(2, RoundingMode.HALF_UP)**** (R-03). Nunca **double**.
 6. Devolver el total recalculado en cada respuesta: evita que el cliente tenga que pedirlo aparte y garantiza que la cifra mostrada sea la del servidor.
 7. **@Auditable** en agregar, editar y eliminar.
 8. Controlador con **@PreAuthorize**: escritura solo ADMIN y COMPRAS.

2. Archivos

```
backend/src/main/java/co/edu/corposucre/productionfood/compra/  
├── {DetalleCompra,DetalleCompraRowMapper,DetalleCompraRepository}.java  
├── {DetalleCompraService,DetalleCompraController}.java  
└── dto/{AgregarDetalleCompraRequest,ActualizarDetalleCompraRequest,  
    DetalleCompraResponse,DetalleCompraListaResponse}.java
```

3. Puntos de cuidado

- **Verificar el estado de la compra en los cuatro endpoints** (R-01), incluido el **DELETE**. Es fácil olvidarlo justamente en el borrado.
- **BigDecimal en todo el cálculo** (R-03). Un **double** en **cantidad * precio** produce totales con centavos fantasma que no cuadran con la factura del proveedor.
- El **UNIQUE** de V2 respalda R-02, pero validar antes da un **409** con mensaje útil que sugiere editar la línea existente.
- **No actualizar **materias_primas.costo_unitario**** al registrar la compra (R-04).
- **precio_unitario = 0** es válido; **cantidad = 0** no (R-05).
- El total se recalcula desde la base en cada respuesta, no se acumula en memoria.

4. Checklist de cierre

- ☐ Todos los CA implementados y verificados manualmente.
- ☐ Endpoints documentados en Swagger con ejemplos.
- ☐ **@PreAuthorize** en cada método, según la matriz de roles.
- ☐ Operaciones de escritura auditadas en bitácora (HU-04).
- ☐ Pruebas unitarias de las reglas con ramificación.
- ☐ Sin **System.out.println** ni código comentado.
- ☐ PR bajo ~250 líneas, revisado y aprobado.

Tarea Frontend

Tabla de ítems dentro de la pantalla de detalle de compra (HU-13). Edición en línea y totales en vivo.

1. Pasos

1. **detalle-compra.model.ts** y **detalle-compra.service.ts**.
2. **detalle-compra-tabla.component** integrado en **compra-detalle**: materia prima, cantidad, precio unitario, subtotal, acciones.
3. Fila de totales al pie, con el valor **que devuelve el backend** (R-03).
4. **agregar-item-dialog.component**:
 - selector de materia prima **excluyendo las ya presentes** (R-02);
 - precio unitario **prellenado con el costo de referencia**, editable;
 - subtotal calculado en vivo como previsualización.
5. Mostrar la diferencia con el costo de referencia (R-04): ↑ rojo si es más caro, ↓ verde si es más barato.
6. Edición en línea de cantidad y precio con guardado inmediato.
7. Confirmación al quitar un ítem.
8. **Si **editable === false**, la tabla se muestra en modo lectura**: sin botón de agregar, sin acciones de fila (R-01).
9. Ruta heredada de HU-13.

2. Archivos

```
frontend/src/app/features/compra/  
├── models/detalle-compra.model.ts  
├── services/detalle-compra.service.ts  
└── components/{detalle-compra-tabla,agregar-item-dialog}/
```

3. Puntos de cuidado

- **El total mostrado debe ser el del backend** (R-03). La previsualización mientras se escribe está bien; la cifra final, no. Dos capas calculando con reglas de redondeo distintas producen descuadres que nadie sabe explicar.
- **Excluir del selector las materias primas ya presentes** (R-02): evita el `409` por completo y sugiere implícitamente editar la línea existente.
- Prellenar el precio con el costo de referencia ahorra tecleo y hace evidente la desviación cuando el proveedor cobra otra cosa.
- En modo no editable, **ocultar** los controles en lugar de deshabilitarlos.
- `step="0.01"` en cantidad y precio (R-07).

4. Checklist de cierre

- ☐ Todos los CA cubiertos desde la interfaz.
- ☐ Los tres estados (cargando / vacío / error) implementados.
- ☐ Errores del servidor mostrados junto al campo correspondiente.
- ☐ Guard de rol aplicado en la ruta.
- ☐ Modelos TypeScript espejo de los DTO del backend.
- ☐ Probado en Chrome y Firefox a 1366×768.
- ☐ Sin `console.log` ni `any`.

Casos de Prueba

Formato y criterios en `00-base/05-ESTANDARES-QA.md`.

1. Casos funcionales

ID	CA	Descripción	Entrada	Resultado esperado	Tipo
CP-01	CA-01..03	Agregar ítem válido	<code>{"idMateriaPrima":5,"cantidad":100,"precioUnitario":3200}</code>	201 con subtotal: 320000.00	Positivo
CP-02	CA-01	Compra inexistente	POST /compras/9999/detalles	404 COMPRA_INEXISTENTE	Negativo
CP-03	CA-01	Agregar ítem a compra RECIBIDA	POST sobre una recibida	409 COMPRA_NO_EDITABLE (R-01)	Negativo
CP-04	CA-01	Editar ítem de compra RECIBIDA	PUT sobre una recibida	409 COMPRA_NO_EDITABLE	Negativo
CP-05	CA-01	Eliminar ítem de compra RECIBIDA	DELETE sobre una recibida	409 COMPRA_NO_EDITABLE	Negativo
CP-06	CA-02	Materia prima inexistente	<code>{"idMateriaPrima":9999}</code>	409 MATERIA_PRIMA_INEXISTENTE	Negativo
CP-07	CA-03	Cantidad cero	<code>{"cantidad":0}</code>	400 CANTIDAD_INVALIDA — CHECK > 0	Negativo
CP-08	CA-03	Cantidad negativa	<code>{"cantidad":-5}</code>	400	Negativo
CP-09	CA-03	Precio cero	<code>{"precioUnitario":0}</code>	201 — válido (R-05)	Borde
CP-10	CA-03	Precio negativo	<code>{"precioUnitario":-100}</code>	400 — CHECK >= 0	Negativo
CP-11	CA-04	Cálculo del subtotal	cantidad 100 × precio 3.200	subtotal: 320000.00 exacto	Positivo
CP-12	CA-04	Cálculo del total	2 ítems: 320.000 + 160.000	totalCompra: 480000.00	Positivo
CP-13	CA-04	Total con decimales	33,33 × 3.333,33	111099.89 — redondeo HALF_UP, sin centavos fantasma	Borde
CP-14	CA-04	El total se actualiza al editar	Cambiar cantidad de 100 a 150	totalCompra refleja el cambio	Positivo
CP-15	CA-04	El total se actualiza al eliminar	Quitar un ítem	totalCompra disminuye	Positivo
CP-16	CA-05	Listar el detalle	GET /compras/12/detalles	Ítems con nombre, unidad y subtotales	Positivo
CP-17	CA-05	Detalle de compra vacía	Compra sin ítems	detalles: [], totalCompra: 0.00	Borde
CP-18	—	Materia prima duplicada	Agregar una MP ya presente	409 MATERIA_PRIMA_DUPLICADA (R-02)	Negativo
CP-19	—	Duplicado por SQL directo	INSERT repitiendo (id_compra, id_materia_prima)	La base lo rechaza — uk_detalle_compra	Integridad
CP-20	—	El costo de referencia NO se actualiza	Comprar a \$3.500 una MP con costo \$3.200	materias_primas.costo_unitario sigue en 3200.00 (R-04)	Integridad
CP-21	—	Rol VENTAS no puede agregar ítems	Token de VENTAS	403 SIN_PERMISO	Seguridad
CP-22	—	Rol CONSULTA puede listar	GET con token CONSULTA	200	Positivo

2. Batería de seguridad

Obligatoria en **cada endpoint** de esta historia:

ID	Caso	Esperado
SEC-01	Sin cabecera <code>Authorization</code>	<code>401 NO_AUTENTICADO</code>
SEC-02	Token malformado	<code>401 NO_AUTENTICADO</code>
SEC-03	Token expirado	<code>401 NO_AUTENTICADO</code>
SEC-04	Token de rol <code>sin</code> permiso	<code>403 SIN_PERMISO</code>
SEC-05	Token de usuario con <code>estado = 0</code>	<code>403 USUARIO_INACTIVO</code>

SEC-04 es el caso crítico. Si devuelve `200`, falta `@EnableMethodSecurity` y la autorización del sistema completo está desactivada.

CP-03 a CP-05 cubren el mismo riesgo desde tres ángulos. Es fácil recordar la validación de estado al agregar y olvidarla al eliminar. Una línea borrada de una compra ya recibida deja el total del documento sin correspondencia con el stock que entró, y sin ninguna señal de que eso ocurrió.

CP-13 verifica R-03. Si el total llega con decimales como `111099.88999999999`, se está calculando con `double` en algún punto de la cadena.

3. Verificación en base de datos

```
-- CP-19: la restricción de V2 §C-13. La segunda sentencia DEBE fallar.
INSERT INTO detalle_compra (id_compra, id_materia_prima, cantidad, precio_unitario)
VALUES (12, 5, 10, 1000);
INSERT INTO detalle_compra (id_compra, id_materia_prima, cantidad, precio_unitario)
VALUES (12, 5, 20, 1000);
-- Error 1062 esperado. Si ambas pasan, la recepción de HU-13 sumará dos veces.

SHOW CREATE TABLE detalle_compra\G
-- UNIQUE KEY `uk_detalle_compra` (`id_compra`,`id_materia_prima`)

-- CP-11 a CP-13: el total debe coincidir exactamente con el de la API
SELECT id_compra,
       SUM(cantidad * precio_unitario) AS total_calculado,
       COUNT(*) AS items
FROM detalle_compra WHERE id_compra = 12 GROUP BY id_compra;

-- CP-13: verificar que no hay error de coma flotante
SELECT cantidad, precio_unitario, cantidad * precio_unitario AS subtotal
FROM detalle_compra WHERE id_compra = 12;
-- Los decimales deben ser exactos, no 111099.88999999999

-- CP-20: el costo de referencia permanece
SELECT nombre, costo_unitario FROM materias_primas WHERE id_materia_prima = 5;

-- CP-07/CP-10: los CHECK de V2
INSERT INTO detalle_compra (id_compra, id_materia_prima, cantidad, precio_unitario)
VALUES (12, 6, 0, 100); -- DEBE fallar (cantidad > 0)
```

4. Criterios de cierre

- ☐ Todos los casos ejecutados; los positivos pasan.
- ☐ Los negativos devuelven el HTTP y el `code` esperados.
- ☐ SEC-01 a SEC-05 pasan en todos los endpoints.
- ☐ Ningún endpoint supera 2 s (RNF-02).
- ☐ Cero defectos críticos o altos abiertos.
- ☐ Colección de Postman actualizada y versionada.

HU-15 · Historial de compras

Especificación

Fase 5 · Fuente: `previo/Historias_de_Usuario.md`

Como comprador, **quiero** consultar el historial de compras por proveedor o rango de fechas, **para que** analice los gastos.

Depende de: HU-14

1. Criterios de aceptación

CA	Enunciado	Cómo se implementa
CA-01	Filtrar por <code>id_proveedor</code>	<code>?idProveedor=</code>
CA-02	Filtrar por rango de fechas	<code>?fechaInicio=&fechaFin=</code> — ambos inclusivos
CA-03	Filtrar por <code>estado</code>	<code>?estado=PENDIENTE RECIBIDA CANCELADA</code>
CA-04	Cada compra muestra sus detalles (materias primas, cantidades, subtotal)	<code>?incluirDetalle=true</code> o consulta individual

2. Endpoints

Método	Ruta	Descripción	Roles
GET	<code>/api/v1/compras</code>	Historial con filtros combinables	ADMIN, COMPRAS, PRODUCCION, CONSULTA
GET	<code>/api/v1/compras/resumen</code>	Totales agregados del período	ADMIN, COMPRAS
GET	<code>/api/v1/compras/por-proveedor</code>	Gasto agrupado por proveedor	ADMIN, COMPRAS

Reglas transversales (paginación, formato de error, autenticación) en `00-base/04-CONTRATO-API.md`.

3. Reglas de negocio

R-01 · Historia de solo lectura

No modifica nada. Amplía los filtros del listado de HU-13 y agrega dos endpoints de agregación. El estado de las compras se cambia en HU-13.

R-02 · Los filtros se combinan con `AND`

Proveedor, rango de fechas y estado se aplican simultáneamente. Todos opcionales: sin ninguno, devuelve el historial completo paginado.

R-03 · Rango de fechas inclusivo en ambos extremos

`fechaInicio = 2026-09-01` y `fechaFin = 2026-09-18` incluyen las compras de ambos días.

Como `compras.fecha` es `DATE` (sin hora), basta `fecha BETWEEN :inicio AND :fin`. Si la columna fuera `DATETIME`, una compra de las 15:00 del día final quedaría fuera al compararse contra la medianoche — el error clásico de los filtros por fecha.

`fechaInicio > fechaFin` → `400 RANGO_FECHAS_INVALIDO`.

R-04 · Las compras canceladas no cuentan como gasto

En los totales agregados, `CANCELADA` se **excluye**: nunca se pagó.

Las `PENDIENTE` se informan **aparte** de las `RECIBIDA`: son gasto comprometido pero no ejecutado. Sumarlas todas en una sola cifra daría un número que no corresponde ni a lo pagado ni a lo comprometido.

R-05 · El total se calcula sobre el detalle, siempre

No se guarda ningún total en `compras`: se recalcula con `SUM(cantidad * precio_unitario)` sobre `detalle_compra` en cada consulta.

Guardar un total denormalizado obligaría a mantenerlo sincronizado en cada alta, edición y borrado de línea — y bastaría un camino olvidado para que el documento mintiera. Con el volumen de una pyme, recalcularlo es instantáneo.

R-06 · Cargar el detalle de todas las compras es costoso

`?incluirDetalle=true` sobre un historial de un año dispara el problema N+1 o produce una respuesta enorme. Por eso:

- por defecto el listado **no** incluye detalles, solo el total y el número de ítems;
- con `incluirDetalle=true`, el `size` máximo se limita a **20**;
- la consulta usa un único `JOIN` con agrupación en memoria, no una consulta por compra.

4. Modelo de datos

Sin cambios de esquema. Consulta `compras`, `detalle_compra`, `proveedores` y `materias_primas`. V2 añadió `idx_compras_fecha` (`fecha`, `estado`), que cubre CA-02 y CA-03.

```
// GET /api/v1/compras?idProveedor=3&fechaInicio=2026-09-01&fechaFin=2026-09-18&estado=RECIBIDA
{
  "content": [
    { "idCompra": 11, "proveedor": { "idProveedor": 3, "nombre": "Harinas del Caribe" },
      "fecha": "2026-09-15", "estado": "RECIBIDA",
      "cantidadItems": 4, "totalCompra": 1240000.00 }
  ],
  "page": 0, "size": 20, "totalElements": 1, "totalPages": 1
}

// GET /api/v1/compras/resumen?fechaInicio=2026-09-01&fechaFin=2026-09-30
{
  "periodo": { "desde": "2026-09-01", "hasta": "2026-09-30" },
  "recibidas": { "cantidad": 8, "total": 4850000.00 },
  "pendientes": { "cantidad": 2, "total": 620000.00 },
  "canceladas": { "cantidad": 1, "total": 85000.00 },
  "gastoEjecutado": 4850000.00,
  "gastoComprometido": 620000.00
}

// GET /api/v1/compras/por-proveedor?fechaInicio=2026-09-01&fechaFin=2026-09-30
[
  { "proveedor": "Harinas del Caribe S.A.S.", "cantidadCompras": 4,
    "total": 2340000.00, "porcentaje": 48.25 },
  { "proveedor": "Distribuidora Lácteos", "cantidadCompras": 3,
    "total": 1810000.00, "porcentaje": 37.32 }
]
```

`gastoEjecutado` y `gastoComprometido` van separados por R-04.

Campos ordenables: `fecha`, `estado`, `total`, `id`.

5. Códigos de error

code	HTTP	Cuándo
RANGO_FECHAS_INVALIDO	400	<code>fechaInicio</code> posterior a <code>fechaFin</code>
PROVEEDOR_INEXISTENTE	404	El <code>idProveedor</code> del filtro no existe

Los transversales (`VALIDACION_FALLIDA`, `NO_AUTENTICADO`, `SIN_PERMISO`, `RECURSO_NO_ENCONTRADO`) están en `00-base/04-CONTRATO-API.md` §5.

6. Fuera de alcance

Lo que **no** cubre esta historia y no debe implementarse aquí: se limita estrictamente a los criterios de aceptación listados arriba. Cualquier funcionalidad adicional se propone como historia nueva, no se agrega en silencio.

7. Definición de Terminado

Se aplica la DoD de `00-base/01-CONVENCIONES.md` §7.

8. Notas

Qué queda fuera

No se implementa exportación a Excel ni PDF: no está en los criterios de aceptación. Si el equipo lo considera necesario, es una historia nueva.

Tampoco se comparan períodos entre sí (este mes contra el anterior). El endpoint de resumen acepta cualquier rango, así que la comparación puede hacerse con dos llamadas desde el cliente.

Diseño

1. Pantallas

Historial con resumen

Historial de compras

Proveedor

Todos

Estado

Todos

Período

01/09/2026 – 30/09/2026

GASTO EJECUTADO

\$ 4.850.000

8 recibidas

COMPROMETIDO

\$ 620.000

2 pendientes

COMPRAS

11

1 cancelada

GASTO POR PROVEEDOR

Harinas del Caribe

\$ 2.340.000

48,3 %

Distribuidora Lácteos

\$ 1.810.000

37,3 %

Insumos del Sinú

\$ 700.000

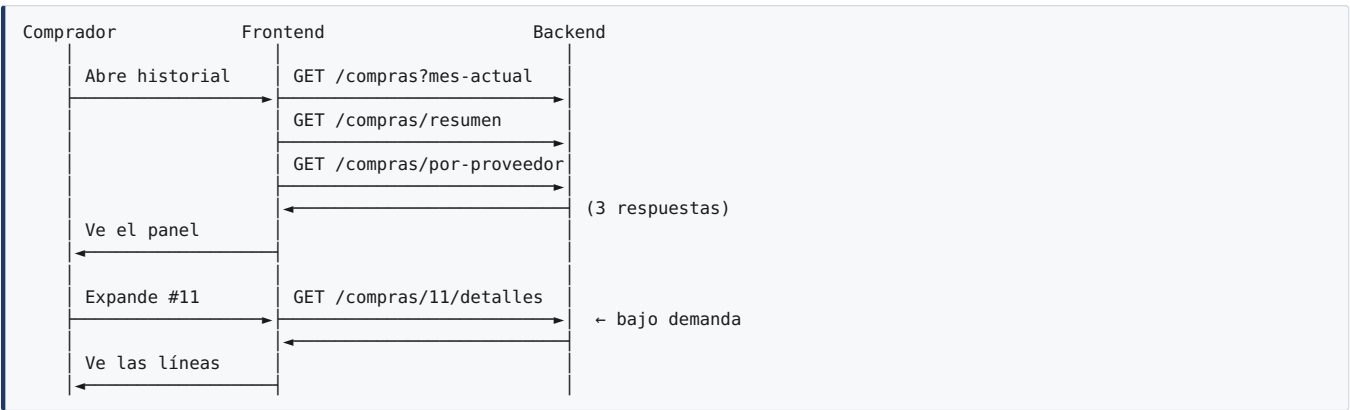
14,4 %

N.º	PROVEEDOR	FECHA	ÍTEM	TOTAL	ESTADO
▶ #12	Harinas del Caribe	18/09/2026	2	\$ 480.000,00	● Pendiente
▼ #11	Distribuidora Lácteos	15/09/2026	4	\$ 1.240.000,00	● Recibida
	└ Leche entera	200,00 L	\$ 2.800,00	\$ 560.000,00	
	└ Mantequilla	40,00 kg	\$17.000,00	\$ 680.000,00	
▶ #10	Insumos del Sinú	12/09/2026	1	\$ 85.000,00	○ Cancelada

Gasto ejecutado y comprometido van separados (R-04): lo primero es dinero que salió, lo segundo es dinero pedido pero no recibido. Una sola cifra que los sumara no correspondería a ninguna realidad contable.

El detalle se carga al expandir la fila (R-06), no de antemano.

2. Flujo



Las tres consultas iniciales son independientes y pueden lanzarse en paralelo.

3. Componentes Angular Material

Elemento	Componente
Tarjetas de resumen	<code>MatCard</code> con cifras grandes
Gasto por proveedor	Barras horizontales (ver la skill de visualización de datos)
Tabla expandible	<code>MatTable</code> con <code>multiTemplateDataRows</code>
Rango de fechas	<code>MatDateRangePicker</code>
Filtros	<code>MatSelect</code>
Estado	<code>MatChip</code> con color
Importes	Pipe <code>currency: 'COP'</code>

4. Validaciones en el formulario

Pantalla de consulta: **sin formularios de datos** (R-01). Solo filtros:

Filtro	Regla
Proveedor	Opcional
Estado	Opcional, valor canónico
Rango de fechas	<code>fechaInicio <= fechaFin</code> , validado antes de llamar

Por defecto: **mes en curso**. Cargar el historial completo en la primera apertura es lento y casi nunca es lo que se busca.

5. Estados de la pantalla

Estado	Qué se muestra
Cargando	Tarjetas en esqueleto + <code>MatProgressBar</code>
Vacío en el período	“No hay compras en el período seleccionado” + sugerencia de ampliar el rango
Vacío con filtros	“Ninguna compra coincide con los filtros” + “Limpiar filtros”
Detalle vacío al expandir	“Esta compra no registra ítems”
Error	Mensaje + “Reintentar”

6. Accesibilidad y UX

- Todos los campos con `<mat-label>`; nunca solo *placeholder*.
- Navegación por teclado en orden lógico; `Enter` envía el formulario.
- Los mensajes de error del servidor se muestran **junto al campo** (`fieldErrors`).
- El botón de envío se deshabilita mientras la petición está en vuelo.
- Confirmación antes de descartar cambios sin guardar.
- Los tres estados obligatorios (cargando / vacío / error) según `00-base/05-ESTANDARES-QA.md` §7.

Tarea Backend

Solo lectura. Amplía el listado de HU-13 con filtros combinables y agrega dos endpoints de agregación.

1. Pasos

1. Ampliar `CompraRepository.buscar()` con `idProveedor`, `fechaInicio`, `fechaFin`, `estado` — todos opcionales, combinados con `AND` (R-02).
2. `CompraRepository.resumenPorPeriodo(fechaInicio, fechaFin)`: agrupa por estado con `SUM(cantidad * precio_unitario)`, excluyendo `CANCELADA` del gasto (R-04).
3. `CompraRepository.gastoPorProveedor(fechaInicio, fechaFin)`: `GROUP BY proveedor` sobre compras `RECIBIDA`, ordenado por total descendente.
4. Carga del detalle con `incluirDetalle=true`:
 - **una sola consulta** con `JOIN` a `detalle_compra` y agrupación en memoria por `id_compra`. Nunca una consulta por compra (R-06);
 - forzar `size <= 20` cuando el parámetro esté activo.
5. Validar `fechaInicio <= fechaFin` → `400 RANGO_FECHAS_INVALIDO` (R-03).
6. DTO: `ResumenComprasResponse`, `GastoProveedorResponse`.
7. `@Transactional(readOnly = true)` en todos los métodos.

8. Sin endpoints de escritura (R-01).

2. Archivos

```
backend/src/main/java/co/edu/corposucre/productionfood/compra/
├── CompraRepository.java      (+ buscar ampliado, resumenPorPeriodo, gastoPorProveedor)
├── CompraService.java        (+ métodos de consulta)
├── CompraController.java     (+ /resumen, /por-proveedor)
└── dto/{ResumenComprasResponse,GastoProveedorResponse}.java
```

3. Puntos de cuidado

- **Evitar el N+1 en incluirDetalle** (R-06): un `JOIN` con agrupación, no un bucle de consultas. Con 200 compras, la versión ingenua hace 201 viajes a la base y supera de largo los 2 s del RNF-02.
- **Las canceladas fuera del gasto** (R-04), y las pendientes en una cifra aparte.
- `BETWEEN` funciona correctamente porque `fecha` es `DATE` (R-03). Si algún día pasara a `DATETIME`, habría que usar `< fechaFin + 1 día`.
- El total se recalcula siempre desde el detalle (R-05); no cachear ni denormalizar.
- Verificar con `EXPLAIN` que el filtro por fechas usa `idx_compras_fecha`.

4. Checklist de cierre

- ☐ Todos los CA implementados y verificados manualmente.
- ☐ Endpoints documentados en Swagger con ejemplos.
- ☐ `@PreAuthorize` en cada método, según la matriz de roles.
- ☐ Operaciones de escritura auditadas en bitácora (HU-04).
- ☐ Pruebas unitarias de las reglas con ramificación.
- ☐ Sin `System.out.println` ni código comentado.
- ☐ PR bajo ~250 líneas, revisado y aprobado.

Tarea Frontend

Pantalla de consulta con filtros, tarjetas de resumen y desglose por proveedor.

1. Pasos

1. Ampliar `compra.service.ts` con `resumen()` y `gastoPorProveedor()`.
2. `compra-historial.component`:
 - filtros: proveedor, estado, rango de fechas (`MatDateRangePicker`);
 - por defecto: **mes actual**, todos los estados.
3. Tarjetas de resumen: gasto ejecutado, gasto comprometido, nº de compras.
4. Gráfico de barras horizontal con el gasto por proveedor (los 5 mayores).
5. Tabla de compras con fila expandible que carga el detalle **bajo demanda** (R-06): al desplegar una fila, se pide el detalle de esa compra.
6. Validar en el cliente `fechaInicio <= fechaFin` antes de llamar.
7. Los tres estados.
8. Ruta: ADMIN/COMPRAS/PRODUCCION/CONSULTA. Los endpoints de agregación, solo ADMIN/COMPRAS.

2. Archivos

```
frontend/src/app/features/compra/
├── services/compra.service.ts      (+ resumen, gastoPorProveedor)
├── pages/compra-historial/
└── components/{resumen-compras-cards,gasto-proveedor-chart}/
```

3. Puntos de cuidado

- **Cargar el detalle al expandir la fila**, no todo de golpe (R-06): mantiene la pantalla rápida con cualquier volumen de historial.

- Mostrar gasto ejecutado y comprometido **por separado** (R-04): fundirlos en una cifra da un número que no significa nada.
 - El rango por defecto (mes actual) evita cargar el historial completo en la primera apertura.
 - Los importes se formatean con el locale `es-CO` ; los cálculos vienen del backend.
-

4. Checklist de cierre

- ☐ Todos los CA cubiertos desde la interfaz.
- ☐ Los tres estados (cargando / vacío / error) implementados.
- ☐ Errores del servidor mostrados junto al campo correspondiente.
- ☐ Guard de rol aplicado en la ruta.
- ☐ Modelos TypeScript espejo de los DTO del backend.
- ☐ Probado en Chrome y Firefox a 1366×768.
- ☐ Sin `console.log` ni `any` .

Casos de Prueba

Formato y criterios en `00-base/05-ESTANDARES-QA.md` .

1. Casos funcionales

ID	CA	Descripción	Entrada	Resultado esperado	Tipo
CP-01	CA-01	Filtrar por proveedor	?idProveedor=3	Solo compras de ese proveedor	Positivo
CP-02	CA-01	Proveedor sin compras	Proveedor recién creado	content: [], totalElements: 0	Borde
CP-03	CA-02	Filtrar por rango	?fechaInicio=2026-09-01&fechaFin=2026-09-18	Solo compras del rango	Positivo
CP-04	CA-02	Rango inclusivo en ambos extremos	Compras el 01/09 y el 18/09 exactos	Ambas aparecen (R-03)	Borde
CP-05	CA-02	Rango invertido	fechaInicio=18/09 , fechaFin=01/09	400 RANGO_FECHAS_INVALIDO	Negativo
CP-06	CA-02	Rango de un solo día	fechaInicio = fechaFin = 2026-09-18	Compras de ese día	Borde
CP-07	CA-03	Filtrar por estado	?estado=RECIBIDA	Solo recibidas	Positivo
CP-08	CA-03	Estado inválido	?estado=ENVIADA	400 PARAMETRO_INVALIDO	Negativo
CP-09	CA-01..03	Filtros combinados	proveedor + rango + estado	Se aplican los tres con AND (R-02)	Positivo
CP-10	CA-04	Detalle en el listado	?incluirDetalle=true	Cada compra trae sus líneas	Positivo
CP-11	CA-04	Subtotales correctos	Revisar CP-10	subtotal = cantidad × precio en cada línea	Positivo
CP-12	CA-04	Tope de size con detalle	? incluirDetalle=true&size=100	size limitado a 20 o 400 (R-06)	Borde
CP-13	—	Las canceladas no cuentan como gasto	Período con una compra cancelada	gastoEjecutado la excluye (R-04)	Integridad
CP-14	—	Pendientes separadas de recibidas	GET /compras/resumen	gastoEjecutado y gastoComprometido en cifras distintas	Positivo
CP-15	—	Gasto por proveedor	GET /compras/por-proveedor	Agrupado, ordenado por total descendente	Positivo
CP-16	—	Porcentajes suman 100	Revisar CP-15	La suma de porcentaje es 100,00 ± 0,01	Positivo
CP-17	—	Resumen de período sin compras	Rango sin datos	Totales en 0.00 , sin error	Borde
CP-18	—	⌚ Rendimiento con detalle	200 compras, incluirDetalle=true&size=20	Responde en < 2 s (RNF-02) — verifica que no hay N+1	Rendimiento
CP-19	—	Rol CONSULTA puede ver el historial	GET /compras con token CONSULTA	200	Positivo
CP-20	—	Rol CONSULTA no accede al resumen	GET /compras/resumen con token CONSULTA	403 SIN_PERMISO	Seguridad
CP-21	—	No hay endpoints de escritura aquí	POST /compras/resumen	405 o 404 (R-01)	Seguridad

2. Batería de seguridad

Obligatoria en **cada endpoint** de esta historia:

ID	Caso	Esperado
SEC-01	Sin cabecera Authorization	401 NO_AUTENTICADO
SEC-02	Token malformado	401 NO_AUTENTICADO
SEC-03	Token expirado	401 NO_AUTENTICADO
SEC-04	Token de rol sin permiso	403 SIN_PERMISO
SEC-05	Token de usuario con estado = 0	403 USUARIO_INACTIVO

SEC-04 es el caso crítico. Si devuelve 200 , falta @EnableMethodSecurity y la autorización del sistema completo está desactivada.

CP-18 es la prueba que detecta el problema N+1 (R-06). Si el historial con detalle tarda más de dos segundos con 200 compras, el backend está haciendo una consulta por compra en lugar de un **JOIN** con agrupación. El síntoma aparece solo con datos reales: con las cinco compras de prueba, ambas implementaciones parecen igual de rápidas.

3. Verificación en base de datos

```
-- CP-13: las canceladas fuera del gasto ejecutado
SELECT c.estado, COUNT(*) AS compras,
       COALESCE(SUM(d.cantidad * d.precio_unitario), 0) AS total
FROM compras c LEFT JOIN detalle_compra d USING (id_compra)
WHERE c.fecha BETWEEN '2026-09-01' AND '2026-09-30'
GROUP BY c.estado;
-- gastoEjecutado de la API = total de RECIBIDA, sin CANCELADA

-- CP-04: el rango debe ser inclusivo
SELECT id_compra, fecha FROM compras
WHERE fecha BETWEEN '2026-09-01' AND '2026-09-18' ORDER BY fecha;
-- Las compras del 01 y del 18 deben estar presentes

-- CP-15/CP-16: gasto por proveedor
SELECT p.nombre, COUNT(DISTINCT c.id_compra) AS compras,
       SUM(d.cantidad * d.precio_unitario) AS total
FROM compras c
     JOIN proveedores p USING (id_proveedor)
     JOIN detalle_compra d USING (id_compra)
WHERE c.estado = 'RECIBIDA' AND c.fecha BETWEEN '2026-09-01' AND '2026-09-30'
GROUP BY p.id_proveedor, p.nombre ORDER BY total DESC;

-- CP-18: el índice de V2 debe usarse
EXPLAIN SELECT id_compra FROM compras
WHERE fecha BETWEEN '2026-09-01' AND '2026-09-30' AND estado = 'RECIBIDA';
-- Se espera: key = idx_compras_fecha
```

4. Criterios de cierre

- ☐ Todos los casos ejecutados; los positivos pasan.
- ☐ Los negativos devuelven el HTTP y el `code` esperados.
- ☐ SEC-01 a SEC-05 pasan en todos los endpoints.
- ☐ Ningún endpoint supera 2 s (RNF-02).
- ☐ Cero defectos críticos o altos abiertos.
- ☐ Colección de Postman actualizada y versionada.

HU-16 · Creación de pedidos

Especificación

Fase 6 · Fuente: `previo/Historias_de_Usuario.md`

Como vendedor, **quiero** crear un pedido para un cliente con fecha de entrega y estado, **para que** gestione las ventas.

Depende de: HU-05 (clientes)

1. Criterios de aceptación

CA	Enunciado	Cómo se implementa
CA-01	<code>id_cliente</code> debe existir y estar activo	Validado contra <code>ClienteRepository.existeActivo()</code>
CA-02	<code>fecha_pedido</code> se registra automáticamente	<code>DEFAULT (curdate())</code> añadido en V2 §C-07
CA-03	<code>fecha_entrega</code> es opcional	Nullable; si se informa, no puede ser anterior al pedido
CA-04	<code>estado</code> obligatorio	Canónicos: <code>PENDIENTE</code> , <code>EN_PREPARACION</code> , <code>ENTREGADO</code> , <code>CANCELADO</code> — <code>CHECK</code> en V2 §C-12
CA-05	Se retorna el pedido creado con su <code>id_pedido</code>	<code>201 + Location + PedidoResponse</code>

2. Endpoints

Método	Ruta	Descripción	Roles
<code>GET</code>	<code>/api/v1/pedidos</code>	Listar con filtros y paginación	ADMIN, VENTAS, PRODUCCION, CONSULTA
<code>GET</code>	<code>/api/v1/pedidos/{id}</code>	Obtener pedido con su detalle	ADMIN, VENTAS, PRODUCCION, CONSULTA
<code>POST</code>	<code>/api/v1/pedidos</code>	Crear un pedido	ADMIN, VENTAS
<code>PUT</code>	<code>/api/v1/pedidos/{id}</code>	Editar cabecera (solo si está <code>PENDIENTE</code>)	ADMIN, VENTAS
<code>PATCH</code>	<code>/api/v1/pedidos/{id}/estado</code>	Cambiar estado	ADMIN, VENTAS

Reglas transversales (paginación, formato de error, autenticación) en `00-base/04-CONTRATO-API.md`.

3. Reglas de negocio

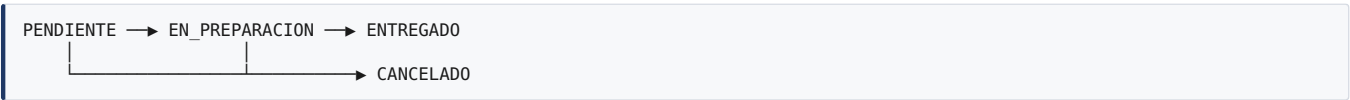
R-01 · La fecha del pedido la pone el servidor

CA-02 dice “automáticamente”. V2 añadió `DEFAULT (curdate())` (§C-07), y `curdate()` **lo evalúa el servidor de base de datos**, no la aplicación ni el navegador.

⚠ Esto hace crítica la zona horaria: una instancia RDS está en UTC por defecto, y Colombia es UTC-5. Un pedido registrado a las 7:30 p.m. se guardaría con la fecha del día siguiente, y HU-24 CA-01 (“pedidos del día”) dejaría de cuadrar con lo que ve el usuario — solo después de las 7 p.m., así que las pruebas de la mañana no lo detectan.

La configuración obligatoria está en `00-base/11-DESPLIEGUE-AWS-RDS.md` §3.

R-02 · Máquina de estados



`ENTREGADO` es terminal. Un pedido entregado no se cancela: si el cliente devuelve algo, eso es una devolución (HU-18), que sí repone stock y deja registro.

Transición no permitida → `409 TRANSICION_INVALIDA`.

R-03 · Cuándo sale el stock del inventario

Decisión de diseño que las historias no fijaban. El descuento ocurre **al agregar el producto al pedido** (HU-17 CA-04 verifica stock suficiente en ese momento), no al entregarlo.

Razón: si se descontara al entregar, dos vendedores podrían comprometer el mismo pan con dos clientes distintos y el conflicto aparecería al momento de la entrega, frente al cliente. Descontar al agregar reserva la mercancía.

Consecuencia: cancelar un pedido **devuelve el stock** a los lotes de los que salió. Lo gestiona HU-17; aquí solo se documenta porque el cambio de estado a **CANCELADO** es el disparador.

R-04 · Cancelar un pedido devuelve el stock

Al pasar a **CANCELADO** desde **PENDIENTE** o **EN_PREPARACION**, cada línea del detalle genera un movimiento **ENTRADA_DEVOLUCION** en **movimientos_inventario_pt** sobre el lote original. Transaccional.

Si el stock no volviera, cancelar un pedido haría desaparecer mercancía del sistema que sigue en el estante.

R-05 · La cabecera solo se edita mientras está PENDIENTE

Igual que en compras (HU-13 R-04). Una vez en preparación, cambiar el cliente del pedido no tiene sentido operativo.

R-06 · La fecha de entrega puede ser futura, la del pedido no

fecha_entrega es un compromiso a futuro: se permite cualquier fecha igual o posterior a la del pedido. **fecha_pedido** la pone el servidor con la fecha de hoy.

R-07 · Un cliente puede desactivarse después de tener pedidos

La validación de cliente activo (CA-01) es **solo al crear**. Los pedidos existentes de un cliente desactivado siguen su curso hasta entregarse: son compromisos adquiridos (HU-05 R-02).

4. Modelo de datos

Tabla pedidos (V2 añade el **DEFAULT** de fecha, el **CHECK** de estado y el índice)

Columna	Tipo	Notas
id_pedido	int unsigned PK AI	
id_cliente	int unsigned NOT NULL FK	CA-01
fecha_pedido	date NOT NULL DEFAULT (curdate())	CA-02 · V2 §C-07 · R-01
fecha_entrega	date NULL	CA-03, R-06
estado	varchar(30) NOT NULL	CA-04 · CHECK en V2 §C-12

```
// POST /api/v1/pedidos
{ "idCliente": 7, "fechaEntrega": "2026-09-20" }

// 201 Created · Location: /api/v1/pedidos/45
{ "idPedido": 45,
  "cliente": { "idCliente": 7, "nombre": "Tienda La Esquina" },
  "fechaPedido": "2026-09-18", "fechaEntrega": "2026-09-20",
  "estado": "PENDIENTE", "totalPedido": 0.00, "cantidadItems": 0,
  "editable": true }

// PATCH /api/v1/pedidos/45/estado
{ "estado": "EN_PREPARACION" }
```

El cuerpo del **POST** **no acepta fechaPedido ni estado**: la fecha la pone la base (R-01) y el estado inicial siempre es **PENDIENTE**.

Campos ordenables: **fechaPedido**, **fechaEntrega**, **estado**, **id**.

5. Códigos de error

code	HTTP	Cuándo
CLIENTE_INEXISTENTE	409	El <code>idCliente</code> no existe
CLIENTE_INACTIVO	409	El cliente está desactivado (solo al crear)
TRANSICION_INVALIDA	409	Transición de estado no permitida (R-02)
PEDIDO_NO_EDITABLE	409	Se intenta modificar un pedido que no está <code>PENDIENTE</code> (R-05)
FECHA_ENTREGA_INVALIDA	400	<code>fechaEntrega</code> anterior a <code>fechaPedido</code> (R-06)

Los transversales (`VALIDACION_FALLIDA` , `NO_AUTENTICADO` , `SIN_PERMISO` , `RECURSO_NO_ENCONTRADO`) están en `00-base/04-CONTRATO-API.md` §5.

6. Fuera de alcance

Lo que **no** cubre esta historia y no debe implementarse aquí: se limita estrictamente a los criterios de aceptación listados arriba. Cualquier funcionalidad adicional se propone como historia nueva, no se agrega en silencio.

7. Definición de Terminado

Se aplica la DoD de `00-base/01-CONVENCIONES.md` §7.

8. Notas

El pedido es el espejo de la compra

La estructura es la misma que HU-13: cabecera con estado, detalle en historia aparte (HU-17), máquina de estados, y una transición que mueve inventario.

La diferencia está en **cuándo** se mueve el stock:

	Compra (HU-13)	Pedido (HU-16/17)
Momento	Al recibir (cambio de estado)	Al agregar la línea (R-03)
Dirección	Entrada	Salida
Reversible	No (<code>RECIBIDA</code> es terminal)	Sí (cancelar devuelve stock, R-04)

La asimetría es deliberada: la mercancía comprada no existe hasta que llega, pero la mercancía vendida hay que apartarla en cuanto se compromete.

Diseño

1. Pantallas

Listado de pedidos

Pedidos

[+ Nuevo pedido]

Cliente

Todos

Estado

Todos

Período

01/09 – 30/09/2026

N.º	CLIENTE	PEDIDO	ENTREGA	ÍTEMS	TOTAL	ESTADO
#48	Panadería El Trigo	18/09/26	17/09/26	3	\$ 45.600	● Pendien
#47	Tienda La Esquina	18/09/26	● HOY	2	\$ 12.400	En prep
#46	Supermercado Central	17/09/26	20/09/26	5	\$ 98.000	En prep
#45	Tienda La Esquina	16/09/26	16/09/26	2	\$ 8.400	● Entrega

entrega vencida · ● entrega hoy. Es la información que se busca al abrir la pantalla por la mañana: qué hay que sacar hoy y qué ya se retrasó.

Formulario de pedido

Nuevo pedido [X]

Cliente *

Tienda La Esquina ▼

Solo se muestran clientes activos

Fecha de entrega

20/09/2026

Opcional. No puede ser anterior a hoy.

i La fecha del pedido se registra automáticamente: 18/09/2026

[Cancelar]

[Crear pedido]

No hay campo de fecha de pedido ni de estado: los pone el servidor (R-01). La nota informativa muestra cuál será, sin ofrecerla como editable.

Diálogo de cancelación — devuelve stock

△ Cancelar el pedido #47

Al cancelar, los productos volverán al inventario:

Pan francés24 und → lote PF-2026-0917

Croissant6 und → lote CR-2026-0916

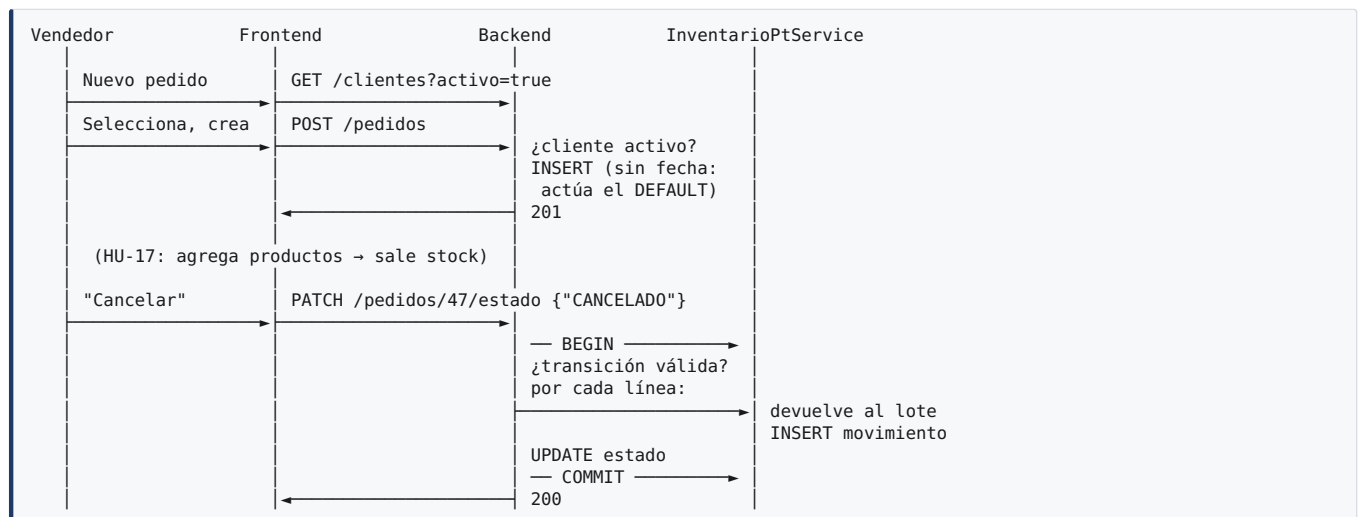
El pedido quedará cancelado y no podrá reactivarse.

[Volver]

[Cancelar el pedido]

Mostrar a qué lote vuelve cada producto (R-04) hace visible que la operación mueve inventario. Sin esa información, cancelar parece un cambio de etiqueta.

2. Flujo



3. Componentes Angular Material

Elemento	Componente
Tabla	<code>MatTable</code> + <code>MatSort</code> + <code>MatPaginator</code>
Estado	<code>MatChip</code> con color
Alerta de entrega	<code>MatIcon</code> con color según proximidad
Cliente	<code>MatSelect</code> (solo activos)
Fecha de entrega	<code>MatDatepicker</code> con <code>min</code>
Diálogo de cancelación	<code>MatDialog</code> con tabla de devolución
Acciones de estado	<code>MatRaisedButton</code> / <code>MatMenu</code>

4. Validaciones en el formulario

Campo	Reglas	Mensaje
Cliente	requerido, activo	“Seleccione un cliente”
Fecha de entrega	opcional, $\geq$ fecha del pedido	“No puede ser anterior a hoy”

No se validan `fechaPedido` ni `estado`: no están en el formulario (R-01).

5. Estados de la pantalla

Estado	Qué se muestra
Cargando	<code>MatProgressBar</code>
Vacío	“No hay pedidos registrados” + “Crear el primero”
Vacío con filtro	“Ningún pedido coincide con los filtros”
Pedido sin productos	△ “Agregue productos a este pedido”
Sin clientes activos	Enlace a HU-05 para registrar uno
Cancelando	Botón con spinner; diálogo bloqueado
Error	Mensaje del servidor + “Reintentar”

6. Accesibilidad y UX

- Todos los campos con `<mat-label>`; nunca solo *placeholder*.
- Navegación por teclado en orden lógico; `Enter` envía el formulario.
- Los mensajes de error del servidor se muestran **junto al campo** (`fieldErrors`).
- El botón de envío se deshabilita mientras la petición está en vuelo.
- Confirmación antes de descartar cambios sin guardar.
- Los tres estados obligatorios (cargando / vacío / error) según `00-base/05-ESTANDARES-QA.md` §7.

Tarea Backend

Cabecera del pedido con máquina de estados. La cancelación devuelve stock, así que **el cambio de estado es transaccional** aunque parezca una simple actualización.

1. Pasos

1. `EstadoPedido` (enum) con `transicionesValidas()` (R-02), igual patrón que `EstadoCompra`.
2. `Pedido` (POJO) y `PedidoRowMapper` con `JOIN clientes`; el listado calcula `totalPedido` con subconsulta `SUM(cantidad * precio_unitario)` sobre `detalle_pedido` — posible gracias a la columna añadida en V2 §C-01.
3. `PedidoRepository`: `buscar`, `contar`, `buscarPorId`, `insertar`, `actualizar`, `cambiarEstado`.
4. **El INSERT omite `fecha_pedido`** para que actúe el `DEFAULT (curdate())` (R-01). No enviar `LocalDate.now()` desde Java: la fecha debe ser la del servidor de base de datos, una sola fuente de verdad.
5. DTO: `CrearPedidoRequest` (solo `idCliente` y `fechaEntrega`), `ActualizarPedidoRequest`, `CambiarEstadoPedidoRequest`, `PedidoResponse`.
6. `PedidoService.crear()`: valida cliente existente y **activo** → `CLIENTE_INACTIVO`; valida `fechaEntrega >= hoy` si viene informada.
7. `PedidoService.cambiarEstado()` **@Transactional**:
 - valida la transición con el enum;
 - **si el nuevo estado es `CANCELADO`**, por cada línea del detalle llama a `InventarioPtService.registrarEntrada(idLote, cantidad, ENTRADA_DEVOLUCION, "pedidos", idPedido)` para devolver el stock al lote de origen (R-04);
 - actualiza el estado y audita.

8. `esEditable(idPedido)` para HU-17.
9. Controlador: escritura ADMIN y VENTAS.

2. Archivos

```
backend/src/main/java/co/edu/corposucre/productionfood/pedido/
├── {Pedido, PedidoRowMapper, PedidoRepository, PedidoService, PedidoController}.java
├── EstadoPedido.java
└── dto/{CrearPedidoRequest, ActualizarPedidoRequest, CambiarEstadoPedidoRequest,
    PedidoResponse}.java
```

3. Puntos de cuidado

- **No enviar la fecha desde Java** (R-01): omitir la columna en el `INSERT` y dejar que el `DEFAULT (curdate())` la ponga. Si la aplicación y la base están en zonas horarias distintas, tener dos fuentes de fecha produce inconsistencias que solo aparecen de noche.
- **Cancelar devuelve stock** (R-04) y es transaccional. Es fácil tratarlo como un simple `UPDATE estado` y olvidar la devolución: el resultado es mercancía que desaparece del sistema pero sigue en el estante.
- **ENTREGADO** es terminal (R-02): una devolución posterior se registra en HU-18.
- Validar cliente activo solo al crear (R-07).
- La cancelación debe registrar el movimiento **sobre el lote original** de cada línea, no sobre un lote cualquiera del producto.

4. Checklist de cierre

- ☐ Todos los CA implementados y verificados manualmente.
- ☐ Endpoints documentados en Swagger con ejemplos.
- ☐ `@PreAuthorize` en cada método, según la matriz de roles.
- ☐ Operaciones de escritura auditadas en bitácora (HU-04).
- ☐ Pruebas unitarias de las reglas con ramificación.
- ☐ Sin `System.out.println` ni código comentado.
- ☐ PR bajo ~250 líneas, revisado y aprobado.
- ☐ **Probado: cancelar un pedido con líneas devuelve el stock a los lotes de origen.**

Tarea Frontend

Listado de pedidos y formulario de cabecera. El flujo de estados debe advertir cuando la acción devuelve stock.

1. Pasos

1. `pedido.model.ts` con el enum `EstadoPedido`, y `pedido.service.ts`.
2. `pedido-lista.component`: número, cliente, fecha de pedido, fecha de entrega, ítems, total, estado, `:`.
3. Chip de estado: ámbar `PENDIENTE`, azul `EN_PREPARACION`, verde `ENTREGADO`, gris `CANCELADO`.
4. **Resaltar los pedidos con entrega para hoy o vencida**: es lo que se busca al abrir la pantalla por la mañana.
5. Filtros: cliente, estado, rango de fechas.
6. `pedido-formulario.component`: cliente (solo activos), fecha de entrega (`min` = hoy). **Sin campo de fecha de pedido ni de estado** (R-01).
7. `pedido-detalle.component`: cabecera + tabla de productos (HU-17) + acciones de estado.
8. **Diálogo de cancelación que advierte la devolución de stock** (R-04), mostrando qué vuelve a cada lote.
9. Ocultar controles de edición si `editable === false`.
10. Ruta: lectura ADMIN/VENTAS/PRODUCCION/CONSULTA; escritura ADMIN/VENTAS.

2. Archivos

```
frontend/src/app/features/pedido/
├── models/pedido.model.ts
├── services/pedido.service.ts
├── pages/{pedido-lista, pedido-formulario, pedido-detalle}/
├── components/cancelar-pedido-dialog/
└── pedido.routes.ts
```

3. Puntos de cuidado

- **El formulario no ofrece fecha de pedido ni estado** (R-01): los pone el servidor. Mostrarlos como editables invita a manipular algo que no se puede.
 - **El diálogo de cancelación debe decir que el stock vuelve al inventario** (R-04): sin esa información, cancelar parece una operación menor y se hace sin pensar.
 - `min` del datepicker de entrega = fecha del pedido (R-06).
 - El total viene del backend.
 - Resaltar las entregas de hoy convierte el listado en la pantalla de trabajo del día.
-

4. Checklist de cierre

- ☐ Todos los CA cubiertos desde la interfaz.
- ☐ Los tres estados (cargando / vacío / error) implementados.
- ☐ Errores del servidor mostrados junto al campo correspondiente.
- ☐ Guard de rol aplicado en la ruta.
- ☐ Modelos TypeScript espejo de los DTO del backend.
- ☐ Probado en Chrome y Firefox a 1366×768.
- ☐ Sin `console.log` ni `any`.

Casos de Prueba

Formato y criterios en `00-base/05-ESTANDARES-QA.md`.

1. Casos funcionales

ID	CA	Descripción	Entrada	Resultado esperado	Tipo
CP-01	CA-01, CA-05	Crear pedido válido	<code>{"idCliente":7,"fechaEntrega":"2026-09-20"}</code>	201 con <code>idPedido</code> , <code>estado: "PENDIENTE"</code>	Positivo
CP-02	CA-01	Cliente inexistente	<code>{"idCliente":9999}</code>	409 CLIENTE_INEXISTENTE	Negativo
CP-03	CA-01	Cliente desactivado	Cliente con <code>estado = 0</code>	409 CLIENTE_INACTIVO	Negativo
CP-04	CA-02	La fecha del pedido la pone el servidor	Crear sin enviar <code>fechaPedido</code>	<code>fechaPedido</code> = hoy, según el servidor	Positivo
CP-05	CA-02	Enviar <code>fechaPedido</code> no tiene efecto	<code>{"fechaPedido":"2020-01-01"}</code>	Se ignora; queda la fecha de hoy (R-01)	Seguridad
CP-06	CA-02	Zona horaria	Crear un pedido después de las 19:00 hora de Colombia	<code>fechaPedido</code> es hoy , no mañana (R-01)	Integridad
CP-07	CA-03	Sin fecha de entrega	Omitir <code>fechaEntrega</code>	201 , <code>fechaEntrega: null</code>	Positivo
CP-08	CA-03	Entrega anterior al pedido	<code>fechaEntrega</code> de ayer	400 FECHA_ENTREGA_INVALIDA	Negativo
CP-09	CA-03	Entrega el mismo día	<code>fechaEntrega</code> = hoy	201 — permitido (R-06)	Borde
CP-10	CA-04	Estado inicial	Crear pedido	Siempre PENDIENTE , aunque se envíe otro	Positivo
CP-11	CA-04	Transición válida	<code>PENDIENTE → EN_PREPARACION</code>	200	Positivo
CP-12	CA-04	<code>EN_PREPARACION → ENTREGADO</code>	<code>PATCH</code>	200	Positivo
CP-13	CA-04	ENTREGADO es terminal	<code>ENTREGADO → PENDIENTE</code>	409 TRANSICION_INVALIDA (R-02)	Negativo
CP-14	CA-04	No se cancela un pedido entregado	<code>ENTREGADO → CANCELADO</code>	409 TRANSICION_INVALIDA	Negativo
CP-15	CA-04	Salto de estado	<code>PENDIENTE → ENTREGADO</code>	409 TRANSICION_INVALIDA	Negativo
CP-16	—	Cancelar devuelve el stock	Pedido con 2 líneas → <code>CANCELADO</code>	El stock vuelve a los lotes de origen (R-04)	Integridad
CP-17	—	Se registran los movimientos de la devolución	Tras CP-16	Movimientos <code>ENTRADA_DEVOLUCION</code> en <code>movimientos_inventario_pt</code>	Integridad
CP-18	—	Cancelar pedido sin líneas	Pedido vacío → <code>CANCELADO</code>	200 ; ningún movimiento generado	Borde
CP-19	—	Editar pedido PENDIENTE	<code>PUT</code> sobre uno pendiente	200	Positivo
CP-20	—	Editar pedido EN_PREPARACION	<code>PUT</code> sobre uno en preparación	409 PEDIDO_NO_EDITABLE (R-05)	Negativo
CP-21	—	Cliente desactivado después	Crear, desactivar cliente, cambiar estado	200 — el pedido sigue su curso (R-07)	Borde
CP-22	—	Rol COMPRAS no puede crear pedidos	Token de COMPRAS	403 SIN_PERMISO	Seguridad
CP-23	—	Rol PRODUCCION puede consultar	<code>GET</code> con token PRODUCCION	200	Positivo

2. Batería de seguridad

Obligatoria en **cada endpoint** de esta historia:

ID	Caso	Esperado
SEC-01	Sin cabecera <code>Authorization</code>	401 NO_AUTENTICADO
SEC-02	Token malformado	401 NO_AUTENTICADO
SEC-03	Token expirado	401 NO_AUTENTICADO
SEC-04	Token de rol sin permiso	403 SIN_PERMISO
SEC-05	Token de usuario con <code>estado = 0</code>	403 USUARIO_INACTIVO

SEC-04 es el caso crítico. Si devuelve `200` , falta `@EnableMethodSecurity` y la autorización del sistema completo está desactivada.

CP-06 es el caso que se descubre tarde. Solo falla después de las 7 p.m. hora de Colombia, cuando el servidor está en UTC. Las pruebas de la mañana pasan sin problema y el defecto llega hasta la sustentación, donde aparece como “los pedidos de ayer salen con la fecha de hoy”. Ejecutarlo requiere probar de noche o cambiar la hora del sistema.

CP-16 y CP-17 verifican R-04. Es fácil implementar la cancelación como un simple `UPDATE estado` y olvidar la devolución de stock: el resultado es mercancía que el sistema da por vendida y que sigue en el estante.

3. Verificación en base de datos

```
-- CP-04 a CP-06: la fecha y la zona horaria
SELECT @@time_zone, @@system_time_zone, NOW(), CURDATE();
-- Se espera: America/Bogota · -05 · hora local de Colombia

SHOW CREATE TABLE pedidos\G
-- Se espera: `fecha_pedido` date NOT NULL DEFAULT (curdate())

SELECT id_pedido, fecha_pedido, fecha_entrega, estado
FROM pedidos ORDER BY id_pedido DESC LIMIT 1;

-- CP-16 y CP-17: cancelar debe devolver el stock a los lotes de origen
SELECT ipt.id_lote, l.codigo_lote, ipt.cantidad_disponible
FROM inventario_producto_terminado ipt JOIN lotes l USING (id_lote)
WHERE l.id_producto = 3;
-- (anotar antes de cancelar, comparar después: debe haber subido)

SELECT tipo_movimiento, cantidad, saldo_anterior, saldo_posterior, tabla_origen, id_origen
FROM movimientos_inventario_pt
WHERE tabla_origen = 'pedidos' AND id_origen = 45 ORDER BY id_movimiento DESC;
-- ENTRADA_DEVOLUCION por cada línea del pedido cancelado

-- CP-13 a CP-15: el CHECK de estados de V2 §C-12
SHOW CREATE TABLE pedidos\G
-- CHECK (`estado` in ('PENDIENTE', 'EN_PREPARACION', 'ENTREGADO', 'CANCELADO'))
```

4. Criterios de cierre

- ☐ Todos los casos ejecutados; los positivos pasan.
- ☐ Los negativos devuelven el HTTP y el `code` esperados.
- ☐ SEC-01 a SEC-05 pasan en todos los endpoints.
- ☐ Ningún endpoint supera 2 s (RNF-02).
- ☐ Cero defectos críticos o altos abiertos.
- ☐ Colección de Postman actualizada y versionada.

HU-17 · Productos en el pedido

Especificación

Fase 6 · Fuente: `previo/Historias_de_Usuario.md`

Como vendedor, **quiero** agregar productos al pedido con cantidad, **para que** se registren los ítems y se descuente del inventario de producto terminado.

Depende de: HU-16, HU-19 (sin lotes de producto terminado no hay stock que vender)

1. Criterios de aceptación

CA	Enunciado	Cómo se implementa
CA-01	<code>id_pedido</code> debe existir	Validado; además debe estar <code>PENDIENTE</code> o <code>EN_PREPARACION</code>
CA-02	<code>id_producto</code> debe existir y estar activo	Validado contra <code>ProductoRepository.existeActivo()</code>
CA-03	<code>cantidad</code> obligatoria (decimal positivo)	<code>@NotNull</code> + <code>CHECK (cantidad > 0)</code> en V2
CA-04	Verificar stock suficiente en <code>inventario_producto_terminado</code>	Vista <code>v_stock_producto_terminado</code> (V2 §C-02) + descuento FEFO
CA-05	Listar el detalle de un pedido	<code>GET /pedidos/{id}/detalles</code>

2. Endpoints

Método	Ruta	Descripción	Roles
<code>GET</code>	<code>/api/v1/pedidos/{idPedido}/detalles</code>	Listar el detalle	ADMIN, VENTAS, PRODUCCION, CONSULTA
<code>GET</code>	<code>/api/v1/productos/{id}/stock</code>	Stock disponible de un producto	ADMIN, VENTAS, PRODUCCION, CONSULTA
<code>POST</code>	<code>/api/v1/pedidos/{idPedido}/detalles</code>	Agregar un producto	ADMIN, VENTAS
<code>PUT</code>	<code>/api/v1/pedidos/{idPedido}/detalles/{id}</code>	Cambiar la cantidad	ADMIN, VENTAS
<code>DELETE</code>	<code>/api/v1/pedidos/{idPedido}/detalles/{id}</code>	Quitar un producto	ADMIN, VENTAS

Reglas transversales (paginación, formato de error, autenticación) en `00-base/04-CONTRATO-API.md`.

3. Reglas de negocio

R-01 · El esquema no permitía cumplir CA-04 tal como estaba escrito

CA-04 pide verificar el stock en `inventario_producto_terminado`. Pero esa tabla está indexada **por lote** (`id_lote` UNIQUE), no por producto:

```
inventario_producto_terminado(id_inv_producto, id_lote, cantidad_disponible, ...)
lotes(id_lote, codigo_lote, id_producto, ...)
```

Saber cuántos panes hay disponibles requiere agregar todos los lotes de ese producto. Y peor: **la historia no decía de qué lote descontar** cuando hay varios.

Resuelto en V2 (`02-CORRECCIONES-DB.md` §C-02) con dos vistas: `v_stock_producto_terminado` (agregado por producto, excluyendo lotes vencidos) y `v_lotes_disponibles_fefo` (lotes ordenables por vencimiento).

R-02 · Política FEFO — *First Expired, First Out*

Se descuenta primero del lote que vence antes. Es la política correcta para alimentos perecederos: descontar del lote más nuevo dejaría el viejo caducando en el estante.

```
Pedido: 30 panes
Lotes disponibles:
PF-2026-0915  vence 19/09   12 und   ← se toman los 12
PF-2026-0917  vence 21/09   40 und   ← se toman 18
PF-2026-0918  vence 22/09   25 und   ← no se toca
```

Una línea de pedido puede consumir **varios lotes**. Cada lote consumido genera su propio movimiento `SALIDA_VENTA` en `movimientos_inventario_pt`.

⚠ **La vista no puede ordenar por sí sola:** MySQL ignora el `ORDER BY` declarado dentro de una vista. El orden FEFO se aplica **en la consulta que la usa**, y está documentado en `V2_correcciones.sql`. Una implementación que confíe en el `ORDER BY` de la vista dejará de respetar FEFO sin dar ningún error.

R-03 · Los lotes vencidos no son stock vendible

`v_stock_producto_terminado` filtra `fecha_vencimiento >= curdate()`. Un producto con 100 unidades vencidas tiene stock **cero** a efectos de venta.

R-04 · El precio se copia al agregar la línea

Gracias a `detalle_pedido.precio_unitario`, añadido en V2 (§C-01), el precio se toma de `productos.precio` **en el momento de agregar** y no se recalcula nunca.

Sin esa columna, subir el precio del pan hoy cambiaría el valor de todos los pedidos históricos, y HU-25 daría un número distinto cada vez que alguien edita el catálogo.

R-05 · Concurrencia: el último pan

Dos vendedores agregan simultáneamente el último pan disponible. Sin protección, ambos leen `cantidad_disponible = 1`, ambos validan y ambos descuentan: el stock queda en `-1`.

Tres defensas, en orden: 1. `SELECT ... FOR UPDATE` sobre las filas de `inventario_producto_terminado` implicadas; 2. la validación de stock y el descuento **en la misma transacción**; 3. `CHECK (cantidad_disponible >= 0)` de V2 como última red.

Si el `CHECK` llega a dispararse, hay un fallo en 1 o 2.

R-06 · Un producto no puede repetirse en el mismo pedido

`UNIQUE (id_pedido, id_producto)` añadido en V2 §C-13. Agregar uno ya presente → `409 PRODUCTO_DUPLICADO`; lo correcto es **editar la cantidad** de la línea existente.

R-07 · Editar la cantidad ajusta el stock por diferencia

- Aumentar de 10 a 15 → se descuentan **5** más (validando stock).
- Reducir de 10 a 6 → se **devuelven 4** a los lotes de origen.

No se devuelve todo y se vuelve a descontar: eso generaría movimientos de kardex que no corresponden a ningún hecho real.

R-08 · Quitar una línea devuelve todo el stock

`DELETE` devuelve la cantidad completa a los lotes de los que salió, con movimientos `ENTRADA_DEVOLUCION`.

R-09 · El detalle solo se modifica si el pedido no está cerrado

`PENDIENTE` y `EN_PREPARACION` admiten cambios. `ENTREGADO` y `CANCELADO`, no: la mercancía ya salió o ya volvió. Una devolución posterior a la entrega es HU-18.

R-10 · Trazabilidad lote ↔ línea de pedido

Como una línea puede consumir varios lotes, la correspondencia queda registrada **en el kardex** (`movimientos_inventario_pt` con `tabla_origen = 'detalle_pedido'`), no en `detalle_pedido`. Es la única forma de saber después qué lote fue a qué cliente — dato necesario ante un retiro sanitario de producto.

4. Modelo de datos

Tabla `detalle_pedido` (V2 añade `precio_unitario`, el `UNIQUE` y el `CHECK`)

Columna	Tipo	Notas
id_detalle_pedido	int unsigned PK AI	
id_pedido	int unsigned NOT NULL FK	CA-01
id_producto	int unsigned NOT NULL FK	CA-02
cantidad	decimal(10,2) NOT NULL	CA-03 · CHECK > 0
precio_unitario	decimal(10,2) NOT NULL	Añadido en V2 §C-01 · R-04

UNIQUE KEY uk_detalle_pedido (id_pedido, id_producto) — V2 §C-13.

```
// GET /api/v1/productos/3/stock
{ "idProducto": 3, "nombre": "Pan francés", "cantidadDisponible": 77.00,
  "lotesActivos": 3, "proximoVencimiento": "2026-09-19",
  "lotes": [
    { "codigoLote": "PF-2026-0915", "vence": "2026-09-19", "disponible": 12.00 },
    { "codigoLote": "PF-2026-0917", "vence": "2026-09-21", "disponible": 40.00 },
    { "codigoLote": "PF-2026-0918", "vence": "2026-09-22", "disponible": 25.00 }
  ]
}

// POST /api/v1/pedidos/47/detalles
{ "idProducto": 3, "cantidad": 30.00 }

// 201 Created
{ "idDetallePedido": 88,
  "producto": { "idProducto": 3, "nombre": "Pan francés", "unidad": "und" },
  "cantidad": 30.00, "precioUnitario": 1200.00, "subtotal": 36000.00,
  "lotesConsumidos": [
    { "codigoLote": "PF-2026-0915", "cantidad": 12.00, "vence": "2026-09-19" },
    { "codigoLote": "PF-2026-0917", "cantidad": 18.00, "vence": "2026-09-21" }
  ],
  "totalPedido": 36000.00 }

// 409 STOCK_INSUFICIENTE
{ "code": "STOCK_INSUFICIENTE",
  "message": "Solo hay 77 unidades disponibles de Pan francés. Se solicitaron 100." }
```

lotesConsumidos hace visible la aplicación de FEFO y sirve de comprobante de trazabilidad.

5. Códigos de error

code	HTTP	Cuándo
PEDIDO_INEXISTENTE	404	El pedido no existe
PEDIDO_NO_EDITABLE	409	El pedido está ENTREGADO o CANCELADO (R-09)
PRODUCTO_INEXISTENTE	409	El producto no existe
PRODUCTO_INACTIVO	409	El producto está desactivado
PRODUCTO_DUPLICADO	409	Ese producto ya está en el pedido (R-06)
STOCK_INSUFICIENTE	409	No hay unidades disponibles suficientes (CA-04)
CANTIDAD_INVALIDA	400	cantidad menor o igual a cero

Los transversales (VALIDACION_FALLIDA , NO_AUTENTICADO , SIN_PERMISO , RECURSO_NO_ENCONTRADO) están en 00-base/04-CONTRATO-API.md §5.

6. Fuera de alcance

Lo que **no** cubre esta historia y no debe implementarse aquí: se limita estrictamente a los criterios de aceptación listados arriba. Cualquier funcionalidad adicional se propone como historia nueva, no se agrega en silencio.

7. Definición de Terminado

Se aplica la DoD de 00-base/01-CONVENCIONES.md §7.

8. Notas

Esta es la historia más compleja de la Fase 6

Combina cuatro cosas que pueden fallar de forma silenciosa: agregación de stock por lotes (R-01), política FEFO (R-02), concurrencia (R-05) y ajuste por diferencia al editar (R-07).

Ninguna produce un error visible cuando se implementa mal: - sin FEFO, el producto viejo caduca en el estante mientras se vende el nuevo; - sin bloqueo, el stock queda negativo bajo concurrencia; - sin ajuste por diferencia, el kardex registra movimientos que nunca ocurrieron.

Por eso `area-qa.md` dedica casos específicos a cada una.

Diseño

1. Pantallas

Detalle del pedido

PRODUCTOS DEL PEDIDO #47				[+ Agregar producto]	
PRODUCTO	CANTIDAD	PRECIO UNIT.	SUBTOTAL	LOTES CONSUMIDOS	...
Pan francés	30 und	\$ 1.200,00	\$ 36.000,00	PF-...0915 (12) ● PF-...0917 (18)	🗑️
Croissant	6 und	\$ 2.500,00	\$ 15.000,00	CR-...0916 (6)	🗑️
TOTAL:			\$ 51.000,00		

La columna **LOTES CONSUMIDOS** es la trazabilidad (R-10): permite saber después qué lote fue a qué cliente, dato indispensable ante un retiro sanitario. El ● marca un lote próximo a vencer.

Agregar producto — con previsualización FEFO

Agregar producto al pedido[X]

Producto *

Pan francés – 77 disponibles▼

Solo se muestran productos con stock disponible

Cantidad *

30und

Precio unitario
\$ 1.200,00
(precio actual)

Se tomará de estos lotes (el que vence primero):

● PF-2026-091512 undvence en 1 día

PF-2026-091718 undvence en 3 días

Quedarán 47 unidades disponibles

Subtotal:

\$ 36.000,00

[Cancelar]

[Agregar]

El recuadro de lotes se recalcula al cambiar la cantidad. Hace tres cosas útiles:

- Enseña que el sistema aplica FEFO y qué mercancía va a salir físicamente.
- Avisa si se está despachando producto próximo a vencer.
- Muestra el stock resultante, evitando la sorpresa del 409.

Sin stock suficiente

Cantidad *

100und▲

▲ Solo hay 77 unidades disponibles.
Faltan 23 unidades para completar el pedido.

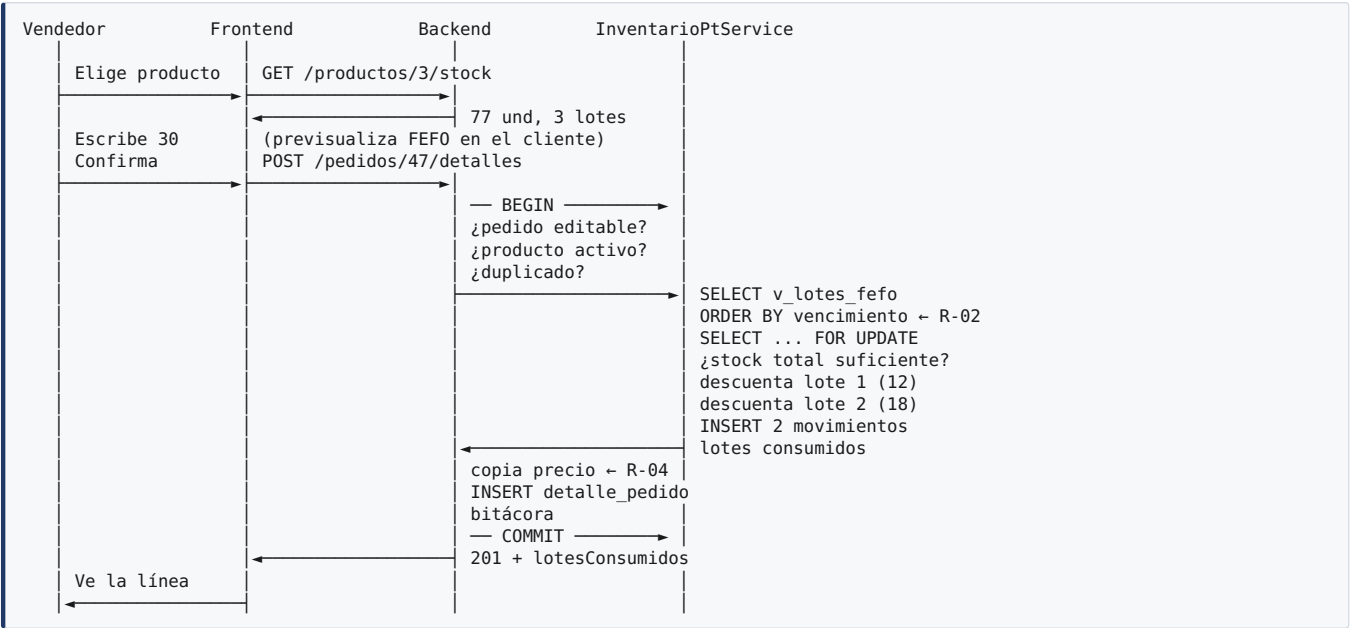
[Cancelar]

[Agregar]

← deshabilitado

El mensaje dice cuánto falta, no solo que no alcanza.

2. Flujo



Si falla cualquier paso, ningún lote queda descontado (CP-22).

3. Componentes Angular Material

Elemento	Componente
Tabla del detalle	<code>MatTable</code> con fila de totales
Lotes consumidos	<code>MatChip</code> compactos con <code>MatTooltip</code>
Agregar	<code>MatDialog</code>
Producto	<code>MatSelect</code> con stock en la etiqueta
Previsualización FEFO	<code>MatCard</code> destacada, recalculada en vivo
Alerta de vencimiento	<code>MatIcon</code> ámbar + días restantes
Cantidad	<code>MatInput[type=number]</code> con <code>max</code> = disponible
Importes	Pipe <code>currency: 'COP'</code>

4. Validaciones en el formulario

Campo	Reglas	Mensaje
Producto	requerido, activo, con stock, no repetido	“Seleccione un producto”
Cantidad	requerida, > 0, ≤ disponible	“Solo hay 77 unidades disponibles”

⚠ El `max` del cliente es orientativo. Entre la consulta de stock y el envío, otro vendedor puede haber tomado unidades. El `409 STOCK_INSUFICIENTE` del servidor es la autoridad (R-05), y hay que manejarlo mostrando el disponible actualizado.

5. Estados de la pantalla

Estado	Qué se muestra
Cargando stock	Spinner junto al selector
Sin productos con stock	“No hay productos disponibles en inventario” + enlace a HU-19
Cantidad excede el stock	⚠ con el disponible y el faltante; botón deshabilitado
Lote próximo a vencer	🕒 con los días restantes en la previsualización
Pedido no editable	🔒 Tabla en modo lectura, sin controles (R-09)
<code>409</code> al enviar	“El stock cambió mientras completaba el formulario. Disponible ahora: N”
Error	Mensaje del servidor + “Reintentar”

El mensaje del 409 explica **por qué** cambió: es lo que evita que el vendedor crea que el sistema falló.

6. Accesibilidad y UX

- Todos los campos con `<mat-label>`; nunca solo *placeholder*.
- Navegación por teclado en orden lógico; `Enter` envía el formulario.
- Los mensajes de error del servidor se muestran **junto al campo** (`fieldErrors`).
- El botón de envío se deshabilita mientras la petición está en vuelo.
- Confirmación antes de descartar cambios sin guardar.
- Los tres estados obligatorios (cargando / vacío / error) según `00-base/05-ESTANDARES-QA.md` §7.

Tarea Backend

La historia más delicada del proyecto junto con HU-23. Aquí nace `InventarioPtService`, el equivalente para producto terminado del `InventarioService` de HU-09. Requiere FEFO, bloqueo de filas y ajuste por diferencia.

1. Pasos

`InventarioPtService` (se crea aquí; lo usan HU-16, HU-18 y HU-19)

1. `registrarSalidaFefo(idProducto, cantidad, tablaOrigen, idOrigen)`:
 - consulta `v_lotes_disponibles_fefo` con `ORDER BY fecha_vencimiento ASC, id_lote ASC` en la propia consulta (R-02: la vista no ordena por sí sola);
 - `SELECT ... FOR UPDATE` sobre las filas de `inventario_producto_terminado` implicadas;
 - valida stock total suficiente → `STOCK_INSUFICIENTE`;
 - recorre los lotes descontando hasta cubrir la cantidad;
 - por cada lote consumido: `UPDATE` del saldo + `INSERT` en `movimientos_inventario_pt` con tipo `SALIDA_VENTA`;
 - devuelve la lista de lotes consumidos.
2. `registrarEntrada(idLote, cantidad, tipo, tablaOrigen, idOrigen)` — para devoluciones.
3. **Regla de oro:** ningún otro componente ejecuta `UPDATE inventario_producto_terminado`.

Módulo `detalle_pedido`

4. `DetallePedido` (POJO) y `DetallePedidoRowMapper` con `JOIN` productos.
5. `DetallePedidoRepository`: `listarPorPedido`, `insertar`, `actualizarCantidad`, `eliminar`, `existeProducto(idPedido, idProducto)`, `calcularTotal(idPedido)`.
6. DTO: `AgregarDetallePedidoRequest`, `ActualizarCantidadRequest`, `DetallePedidoResponse` (con `lotesConsumidos`), `StockProductoResponse`.
7. `DetallePedidoService.agregar()` `@Transactional`:
 - pedido existe y es editable → `PEDIDO_NO_EDITABLE`;
 - producto existe y activo;
 - no duplicado → `PRODUCTO_DUPLICADO`;
 - copia `productos.precio` en `precio_unitario` (R-04);
 - `registrarSalidaFefo(...)`;
 - `INSERT` en `detalle_pedido`.
8. `actualizarCantidad()` con **ajuste por diferencia** (R-07):
 - `nueva > actual` → salida FEFO por la diferencia;
 - `nueva < actual` → entrada a los lotes de origen por la diferencia, consultando el kardex para saber a cuáles volver;
 - **no** devolver todo y volver a descontar.
9. `eliminar()`: devuelve la cantidad completa a los lotes de origen (R-08).
10. Consulta de stock desde `v_stock_producto_terminado`.
11. `@Auditable` en las tres escrituras.

2. Archivos

```

backend/src/main/java/co/edu/corposucre/productionfood/
├── inventario/
│   ├── InventarioPtService.java           ← FEFO y bloqueo
│   ├── InventarioPtRepository.java
│   └── MovimientoInventarioPtRepository.java
└── pedido/
    ├── {DetallePedido, DetallePedidoRowMapper, DetallePedidoRepository}.java
    ├── {DetallePedidoService, DetallePedidoController}.java
    └── dto/{AgregarDetallePedidoRequest, ActualizarCantidadRequest,
        DetallePedidoResponse, StockProductoResponse, LoteConsumidoResponse}.java

```

3. Puntos de cuidado

- El **ORDER BY** va en la consulta, no en la vista (R-02). MySQL ignora el **ORDER BY** interno de una vista **sin dar ningún aviso**: la vista se crea, devuelve filas, y el orden es arbitrario. FEFO deja de aplicarse y nadie se entera hasta que caduca producto en el estante.
- **SELECT ... FOR UPDATE antes de validar el stock** (R-05). Validar y descontar en operaciones separadas permite que dos pedidos simultáneos vendan el mismo último pan.
- **Copiar el precio, no referenciarlo** (R-04). Si el **SELECT** del detalle lee **productos.precio** en lugar de **detalle_pedido.precio_unitario**, se pierde el histórico y la corrección §C-01 queda anulada.
- **Ajuste por diferencia al editar** (R-07), no devolver-y-redescontar: el kardex debe reflejar lo que realmente pasó.
- Excluir lotes vencidos del stock vendible (R-03).
- El **CHECK (cantidad_disponible >= 0)** es la última red; si salta, el bloqueo falló.

4. Checklist de cierre

- ☐ Todos los CA implementados y verificados manualmente.
- ☐ Endpoints documentados en Swagger con ejemplos.
- ☐ **@PreAuthorize** en cada método, según la matriz de roles.
- ☐ Operaciones de escritura auditadas en bitácora (HU-04).
- ☐ Pruebas unitarias de las reglas con ramificación.
- ☐ Sin **System.out.println** ni código comentado.
- ☐ PR bajo ~250 líneas, revisado y aprobado.
- ☐ **Pruebas unitarias de registrarSalidaFefo** : un lote, varios lotes, stock exacto, stock insuficiente.
- ☐ **Probado bajo concurrencia: dos peticiones simultáneas por el último ítem.**

Tarea Frontend

Tabla de productos dentro del pedido, con stock visible en tiempo real y los lotes que se van a consumir.

1. Pasos

1. **detalle-pedido.model.ts** y **detalle-pedido.service.ts**.
2. **detalle-pedido-tabla.component** : producto, cantidad, precio unitario, subtotal, lotes consumidos, acciones.
3. **agregar-producto-dialog.component** :
 - selector de producto **excluyendo los ya presentes** (R-06) y **los que no tienen stock**;
 - al seleccionar, consultar **GET /productos/{id}/stock** y mostrar el disponible;
 - **previsualización de los lotes que se consumirán** según FEFO, actualizada al cambiar la cantidad;
 - bloquear el botón si la cantidad supera el disponible, con mensaje claro.
4. Mostrar los lotes consumidos en cada fila del detalle (trazabilidad, R-10).
5. Advertir cuando se consuma un lote **próximo a vencer**: “Se tomarán 12 unidades del lote PF-2026-0915, que vence en 1 día”.
6. Edición de cantidad en línea, indicando el ajuste: “Se descontarán 5 unidades más”.
7. Confirmación al quitar, avisando que el stock vuelve al inventario (R-08).
8. Modo lectura si el pedido no es editable (R-09).
9. Total del pedido siempre desde el backend.

2. Archivos

```

frontend/src/app/features/pedido/
├── models/detalle-pedido.model.ts
├── services/detalle-pedido.service.ts
└── components/{detalle-pedido-tabla, agregar-producto-dialog, lotes-consumidos-chip}/

```

3. Puntos de cuidado

- **Consultar el stock al seleccionar el producto**, no al abrir el diálogo: el dato debe ser lo más fresco posible. Aun así, **el stock puede haber cambiado** entre la consulta y el envío: el `409 STOCK_INSUFICIENTE` del servidor es la autoridad y hay que manejarlo con un mensaje útil, no genérico.
- **La previsualización de lotes FEFO** enseña al usuario qué mercancía va a salir y avisa de vencimientos próximos.
- Excluir del selector los productos sin stock evita el error antes de cometerlo.
- Al editar la cantidad, indicar el **delta** (R-07), no solo el valor final.
- El subtotal y el total vienen del backend.

4. Checklist de cierre

- ☐ Todos los CA cubiertos desde la interfaz.
- ☐ Los tres estados (cargando / vacío / error) implementados.
- ☐ Errores del servidor mostrados junto al campo correspondiente.
- ☐ Guard de rol aplicado en la ruta.
- ☐ Modelos TypeScript espejo de los DTO del backend.
- ☐ Probado en Chrome y Firefox a 1366×768.
- ☐ Sin `console.log` ni `any`.

Casos de Prueba

Formato y criterios en `00-base/05-ESTANDARES-QA.md`.

1. Casos funcionales

ID	CA	Descripción	Entrada	Resultado esperado	Tipo
CP-01	CA-01..04	Agregar producto con stock suficiente	30 unidades, hay 77	201 con LotesConsumidos	Positivo
CP-02	CA-04	El stock se descuenta	Tras CP-01	Disponible baja de 77 a 47	Integridad
CP-03	CA-04	FEFO: se consume primero el que vence antes	Lotes: 12 und (19/09), 40 und (21/09); pedir 30	Se toman 12 del primero y 18 del segundo (R-02)	Integridad
CP-04	CA-04	FEFO con orden de inserción inverso	Crear primero el lote que vence después	Igual: manda el vencimiento, no el orden de alta	Integridad
CP-05	CA-04	Stock insuficiente	Pedir 100, hay 77	409 STOCK_INSUFICIENTE con las cifras en el mensaje	Negativo
CP-06	CA-04	Stock exacto	Pedir 77, hay 77	201 ; disponible queda en 0	Borde
CP-07	CA-04	Una unidad más que el stock	Pedir 78, hay 77	409 STOCK_INSUFICIENTE	Borde
CP-08	CA-04	Los lotes vencidos no cuentan	100 und vencidas, 0 vigentes	409 STOCK_INSUFICIENTE (R-03)	Integridad
CP-09	CA-04	Concurrencia por el último ítem	2 peticiones simultáneas, 1 unidad disponible	Una 201 , otra 409 . Nunca stock negativo (R-05)	Integridad
CP-10	CA-02	Producto inexistente	{"idProducto":9999}	409 PRODUCTO_INEXISTENTE	Negativo
CP-11	CA-02	Producto desactivado	Producto con estado = 0	409 PRODUCTO_INACTIVO	Negativo
CP-12	CA-03	Cantidad cero	{"cantidad":0}	400 CANTIDAD_INVALIDA	Negativo
CP-13	CA-03	Cantidad negativa	{"cantidad":-5}	400	Negativo
CP-14	—	El precio se copia, no se referencia	Agregar a \$1.200; subir el catálogo a \$1.500; consultar	El detalle sigue en 1200.00 (R-04)	Integridad
CP-15	—	Producto duplicado	Agregar uno ya presente	409 PRODUCTO_DUPLICADO (R-06)	Negativo
CP-16	—	Aumentar cantidad descuenta la diferencia	De 10 a 15	Se descuentan 5, no 15 (R-07)	Integridad
CP-17	—	Reducir cantidad devuelve la diferencia	De 10 a 6	Se devuelven 4 a los lotes de origen	Integridad
CP-18	—	Aumentar sin stock	De 10 a 200 sin disponibilidad	409 STOCK_INSUFICIENTE ; la línea no cambia	Negativo
CP-19	—	Quitar una línea devuelve todo el stock	DELETE de una línea de 30	Los 30 vuelven a los lotes de origen (R-08)	Integridad
CP-20	—	Agregar a pedido ENTREGADO	POST sobre uno entregado	409 PEDIDO_NO_EDITABLE (R-09)	Negativo
CP-21	—	Agregar a pedido CANCELADO	POST sobre uno cancelado	409 PEDIDO_NO_EDITABLE	Negativo
CP-22	—	Transaccionalidad	Provocar fallo tras descontar el primer lote	Ningún lote queda descontado	Integridad
CP-23	—	Trazabilidad lote → pedido	Consultar el kardex tras CP-01	Movimientos SALIDA_VENTA con tabla_origen = detalle_pedido (R-10)	Integridad
CP-24	—	RoI COMPRAS no puede agregar	Token de COMPRAS	403 SIN_PERMISO	Seguridad

2. Batería de seguridad

Obligatoria en **cada endpoint** de esta historia:

ID	Caso	Esperado
SEC-01	Sin cabecera <code>Authorization</code>	<code>401 NO_AUTENTICADO</code>
SEC-02	Token malformado	<code>401 NO_AUTENTICADO</code>
SEC-03	Token expirado	<code>401 NO_AUTENTICADO</code>
SEC-04	Token de rol <code>sin</code> permiso	<code>403 SIN_PERMISO</code>
SEC-05	Token de usuario con <code>estado = 0</code>	<code>403 USUARIO_INACTIVO</code>

SEC-04 es el caso crítico. Si devuelve `200`, falta `@EnableMethodSecurity` y la autorización del sistema completo está desactivada.

CP-03, CP-04 y CP-09 son los casos críticos del proyecto entero.

CP-03/CP-04 (FEFO): si se confía en el `ORDER BY` declarado dentro de la vista, MySQL lo ignora y el orden es arbitrario. Todo funciona, las cifras cuadran, y el producto viejo caduca en el estante mientras se vende el nuevo. No hay ningún mensaje de error.

CP-09 (conurrencia): requiere lanzar dos peticiones en paralelo, por ejemplo con `ab -n 2 -c 2` o el *runner* de Postman con dos hilos. Es la única forma de detectar la falta de `FOR UPDATE`, y el síntoma en producción es un inventario negativo que nadie sabe explicar.

3. Verificación en base de datos

```
-- CP-03 y CP-04: verificar que FEFO se aplicó
SELECT l.codigo_lote, l.fecha_vencimiento, ipt.cantidad_disponible
FROM lotes l JOIN inventario_producto_terminado ipt USING (id_lote)
WHERE l.id_producto = 3 ORDER BY l.fecha_vencimiento;
-- El lote que vence ANTES debe ser el que quedó en cero o más reducido.
-- Si se consumió el más nuevo, FEFO no se está aplicando: probablemente
-- se confió en el ORDER BY de la vista, que MySQL ignora (R-02).

SELECT m.tipo_movimiento, l.codigo_lote, l.fecha_vencimiento,
       m.cantidad, m.saldo_anterior, m.saldo_posterior
FROM movimientos_inventario_pt m JOIN lotes l USING (id_lote)
WHERE m.tabla_origen = 'detalle_pedido' AND m.id_origen = 88
ORDER BY l.fecha_vencimiento;

-- CP-09: el stock NUNCA debe quedar negativo
SELECT id_lote, cantidad_disponible FROM inventario_producto_terminado
WHERE cantidad_disponible < 0;
-- DEBE devolver 0 filas. Si devuelve alguna, el CHECK de V2 no está activo
-- y además el bloqueo FOR UPDATE no funcionó.

-- CP-14: el precio histórico se congela
SELECT dp.id_detalle_pedido, dp.cantidad, dp.precio_unitario AS precio_pedido,
       p.precio AS precio_catalogo
FROM detalle_pedido dp JOIN productos p USING (id_producto)
WHERE dp.id_pedido = 47;
-- precio_pedido NO debe seguir a precio_catalogo

-- CP-16/CP-17: el ajuste por diferencia deja UN movimiento, no dos
SELECT tipo_movimiento, cantidad, fecha FROM movimientos_inventario_pt
WHERE tabla_origen = 'detalle_pedido' AND id_origen = 88 ORDER BY id_movimiento;
-- Al subir de 10 a 15: un SALIDA_VENTA de 5.
-- Si aparece ENTRADA de 10 + SALIDA de 15, se implementó devolver-y-redescontar (R-07).

-- CP-08: la vista excluye vencidos
SELECT * FROM v_stock_producto_terminado WHERE id_producto = 3;
```

4. Criterios de cierre

- ☐ Todos los casos ejecutados; los positivos pasan.
- ☐ Los negativos devuelven el HTTP y el `code` esperados.
- ☐ SEC-01 a SEC-05 pasan en todos los endpoints.
- ☐ Ningún endpoint supera 2 s (RNF-02).
- ☐ Cero defectos críticos o altos abiertos.
- ☐ Colección de Postman actualizada y versionada.

HU-18 · Registro de devoluciones

Especificación

Fase 6 · Fuente: `previo/Historias_de_Usuario.md`

Como vendedor, **quiero** registrar devoluciones de un pedido con motivo, descripción y cantidad, **para que** controle las devoluciones de producto.

Depende de: HU-17

1. Criterios de aceptación

CA	Enunciado	Cómo se implementa
CA-01	<code>id_detalle_pedido</code> debe existir	Validado contra <code>DetallePedidoRepository</code>
CA-02	<code>motivo</code> obligatorio, máx. 50	<code>@NotBlank</code> + <code>@Size(max=50)</code>
CA-03	<code>descripcion</code> opcional, máx. 255	<code>@Size(max=255)</code>
CA-04	<code>cantidad_devuelta</code> obligatoria, no puede exceder la cantidad del pedido	Validado contra el acumulado de devoluciones previas
CA-05	<code>fecha_devolucion</code> se registra automáticamente	<code>DEFAULT (curdate())</code> añadido en V2 §C-07
CA-06	<code>estado</code> obligatorio	Canónicos: <code>PENDIENTE</code> , <code>APROBADA</code> , <code>RECHAZADA</code> — <code>CHECK</code> en V2 §C-12
CA-07	Al aprobarse, la cantidad vuelve al inventario de producto terminado	<code>InventarioPtService.registrarEntrada()</code> con tipo <code>ENTRADA_DEVOLUCION</code>

2. Endpoints

Método	Ruta	Descripción	Roles
GET	<code>/api/v1/devoluciones</code>	Listar con filtros y paginación	ADMIN, VENTAS, PRODUCCION, CONSULTA
GET	<code>/api/v1/devoluciones/{id}</code>	Obtener una devolución	ADMIN, VENTAS, PRODUCCION, CONSULTA
GET	<code>/api/v1/pedidos/{id}/devoluciones</code>	Devoluciones de un pedido	ADMIN, VENTAS, PRODUCCION, CONSULTA
POST	<code>/api/v1/devoluciones</code>	Registrar una devolución	ADMIN, VENTAS
PATCH	<code>/api/v1/devoluciones/{id}/estado</code>	Aprobar o rechazar · aprobar repone stock	ADMIN, VENTAS

Reglas transversales (paginación, formato de error, autenticación) en `00-base/04-CONTRATO-API.md`.

3. Reglas de negocio

R-01 · El stock vuelve al aprobar, no al registrar

Una devolución nace `PENDIENTE`: alguien la reporta, pero la mercancía todavía no se ha revisado. Reponer el stock en ese momento haría que producto en mal estado —que es el motivo habitual de una devolución— volviera a estar disponible para vender.

Al pasar a `APROBADA`, y solo entonces, se repone. Transaccional.

R-02 · A qué lote vuelve la mercancía

La devolución apunta a `detalle_pedido`, pero una línea de pedido pudo consumir **varios lotes** (HU-17 R-02). La historia no dice a cuál devolver.

Decisión: se consulta el kardex (`movimientos_inventario_pt` con `tabla_origen = 'detalle_pedido'`) para saber de qué lotes salió esa línea, y se devuelve **en orden inverso al consumo** — al último lote del que se tomó primero.

Razón: el último lote consumido es el de vencimiento más lejano, y es el que más probabilidades tiene de seguir siendo vendible. Devolver al lote más antiguo repondría mercancía que quizá ya venció, inflando un stock que no se puede vender.

Si un lote de destino ya venció, la devolución se aprueba igualmente pero **la unidad no vuelve como stock vendible**: `v_stock_producto_terminado` excluye vencidos (HU-17 R-03). Queda registrada en el kardex para la trazabilidad.

R-03 · No se puede devolver más de lo pedido (CA-04)

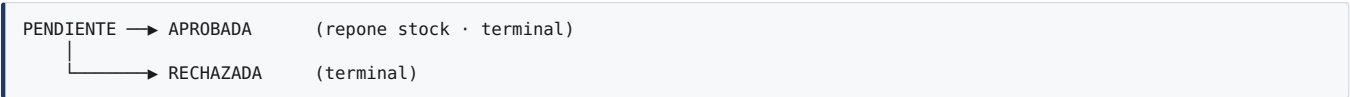
La validación es contra el **acumulado**, no contra cada devolución por separado:

```
Σ(devoluciones APROBADA + PENDIENTE de esa línea) + nueva ≤ detalle_pedido.cantidad
```

Sin contar las `PENDIENTE`, se podrían registrar tres devoluciones de 10 sobre una línea de 10, y al aprobarlas todas volverían 30 unidades al inventario de una venta de 10.

Las `RECHAZADA` **no** cuentan: liberan la cantidad para una devolución posterior.

R-04 · Máquina de estados



Ambos son terminales. Una devolución aprobada por error se corrige con un ajuste de inventario, no revirtiendo el estado — mismo criterio que la recepción de compra (HU-13 R-02).

R-05 · Solo se devuelve de pedidos entregados o en preparación

Devolver de un pedido `PENDIENTE` no tiene sentido: la mercancía no ha salido. De uno `CANCELADO` tampoco: el stock ya volvió entero (HU-16 R-04), y aprobar una devolución encima lo duplicaría.

R-06 · La fecha la pone el servidor

`DEFAULT (curdate())`, añadido en V2 §C-07: el esquema original la exigía `NOT NULL` sin valor por defecto, aunque CA-05 pidiera que fuera automática. Mismo cuidado con la zona horaria que en HU-16 R-01.

R-07 · El motivo es texto libre, con sugerencias

`varchar(50)` sin restricción. La interfaz ofrece motivos frecuentes —*Producto en mal estado, Error en el pedido, Cliente insatisfecho, Producto vencido*— pero permite escribir otro.

Normalizar el motivo en un catálogo sería mejor para reportes, pero no está en los criterios de aceptación y el esquema no lo contempla. Se registra como mejora posible.

4. Modelo de datos

Tabla `devoluciones` (V2 añade el `DEFAULT` de fecha, el `CHECK` de estado y un índice)

Columna	Tipo	Notas
<code>id_devolucion</code>	<code>int unsigned</code> PK AI	
<code>id_detalle_pedido</code>	<code>int unsigned</code> NOT NULL FK	CA-01
<code>motivo</code>	<code>varchar(50)</code> NOT NULL	CA-02, R-07
<code>descripcion</code>	<code>varchar(255)</code> NULL	CA-03
<code>cantidad_devuelta</code>	<code>decimal(10,2)</code> NOT NULL	CA-04 · <code>CHECK > 0</code>
<code>fecha_devolucion</code>	<code>date</code> NOT NULL DEFAULT (curdate())	CA-05 · V2 §C-07
<code>estado</code>	<code>varchar(30)</code> NOT NULL	CA-06 · <code>CHECK</code> en V2 §C-12

```
// POST /api/v1/devoluciones
{ "idDetallePedido": 88, "motivo": "Producto en mal estado",
  "descripcion": "Tres panes llegaron aplastados en el transporte",
  "cantidadDevuelta": 3.00 }

// 201 Created
{ "idDevolucion": 15,
  "pedido": { "idPedido": 47, "cliente": "Tienda La Esquina" },
  "producto": { "idProducto": 3, "nombre": "Pan francés" },
  "cantidadPedida": 30.00, "cantidadDevuelta": 3.00,
  "devueltoPreviamente": 0.00, "disponibleParaDevolver": 27.00,
  "motivo": "Producto en mal estado",
  "fechaDevolucion": "2026-09-18", "estado": "PENDIENTE" }

// PATCH /api/v1/devoluciones/15/estado
{ "estado": "APROBADA" }

// 200 OK – el stock volvió
{ "idDevolucion": 15, "estado": "APROBADA",
  "lotesRepuestos": [
    { "codigoLote": "PF-2026-0917", "cantidad": 3.00,
      "saldoAnterior": 22.00, "saldoPosterior": 25.00 }
  ]
}

// 422 CANTIDAD_EXCEDE_PEDIDO
{ "code": "CANTIDAD_EXCEDE_PEDIDO",
  "message": "Se pidieron 30 unidades y ya hay 28 devueltas o en trámite. Solo puede devolver 2." }
```

Campos ordenables: `fechaDevolucion`, `estado`, `id`.

5. Códigos de error

code	HTTP	Cuándo
DETALLE_PEDIDO_INEXISTENTE	409	El <code>idDetallePedido</code> no existe
CANTIDAD_EXCEDE_PEDIDO	422	La devolución supera lo pedido, contando las previas (R-03)
PEDIDO_NO_DEVOLVIBLE	409	El pedido está <code>PENDIENTE</code> o <code>CANCELADO</code> (R-05)
TRANSICION_INVALIDA	409	La devolución ya fue aprobada o rechazada (R-04)
CANTIDAD_INVALIDA	400	<code>cantidadDevuelta</code> menor o igual a cero

Los transversales (`VALIDACION_FALLIDA`, `NO_AUTENTICADO`, `SIN_PERMISO`, `RECURSO_NO_ENCONTRADO`) están en `00-base/04-CONTRATO-API.md` §5.

6. Fuera de alcance

Lo que **no** cubre esta historia y no debe implementarse aquí: se limita estrictamente a los criterios de aceptación listados arriba. Cualquier funcionalidad adicional se propone como historia nueva, no se agrega en silencio.

7. Definición de Terminado

Se aplica la DoD de `00-base/01-CONVENCIONES.md` §7.

8. Notas

Por qué el `422` y no un `409`

`CANTIDAD_EXCEDE_PEDIDO` usa `422 Unprocessable Entity`: los datos son sintácticamente correctos —un número positivo en un campo de cantidad— pero violan una regla de negocio que depende del estado acumulado del sistema.

Es la distinción de `00-base/04-CONTRATO-API.md` §3: `409` para conflictos con algo que ya existe (un duplicado, un estado incompatible), `422` para reglas de negocio que los datos no satisfacen.

Diseño

1. Pantallas

Bandeja de devoluciones

Devoluciones						
Estado		Período				
Pendientes ▾		01/09/2026 – 30/09/2026				
PEDIDO	CLIENTE	PRODUCTO	CANT.	MOTIVO	FECHA	...
#47	Tienda La Esquina	Pan francés	3	Producto en mal...	18/09	:
#46	Supermercado Cen...	Croissant	2	Error en el ped...	17/09	:
#45	Panadería El Trigo	Torta de zana...	1	Cliente insatis...	16/09	:

Filtrada por **PENDIENTE** de entrada: es la bandeja de trabajo de quien revisa devoluciones. Menú : **Aprobar** · **Rechazar** · **Ver pedido**.

Registrar devolución (desde el detalle del pedido)

Registrar devolución [X]

Pedido #47 · Tienda La Esquina
Producto: Pan francés

Cantidad pedida:30 und
Ya devuelta o en trámite:3 und
Disponible para devolver:27 und

Cantidad a devolver *

3und

Motivo *

Producto en mal estado▾

Descripción

Tres panes llegaron aplastados en el transporte

48 / 255

i La devolución quedará pendiente de aprobación.
El stock volverá al inventario solo al aprobarse.

[Cancelar] [Registrar]

Los tres números del recuadro son lo que hace comprensible el límite (R-03). Sin “ya devuelta o en trámite”, el 422 parece arbitrario.

La nota final explica R-01: registrar no repone.

Diálogo de aprobación

✓ Aprobar devolución

Pedido #47 · Pan francés · 3 unidades
Motivo: Producto en mal estado

Al aprobar, el stock volverá al inventario:

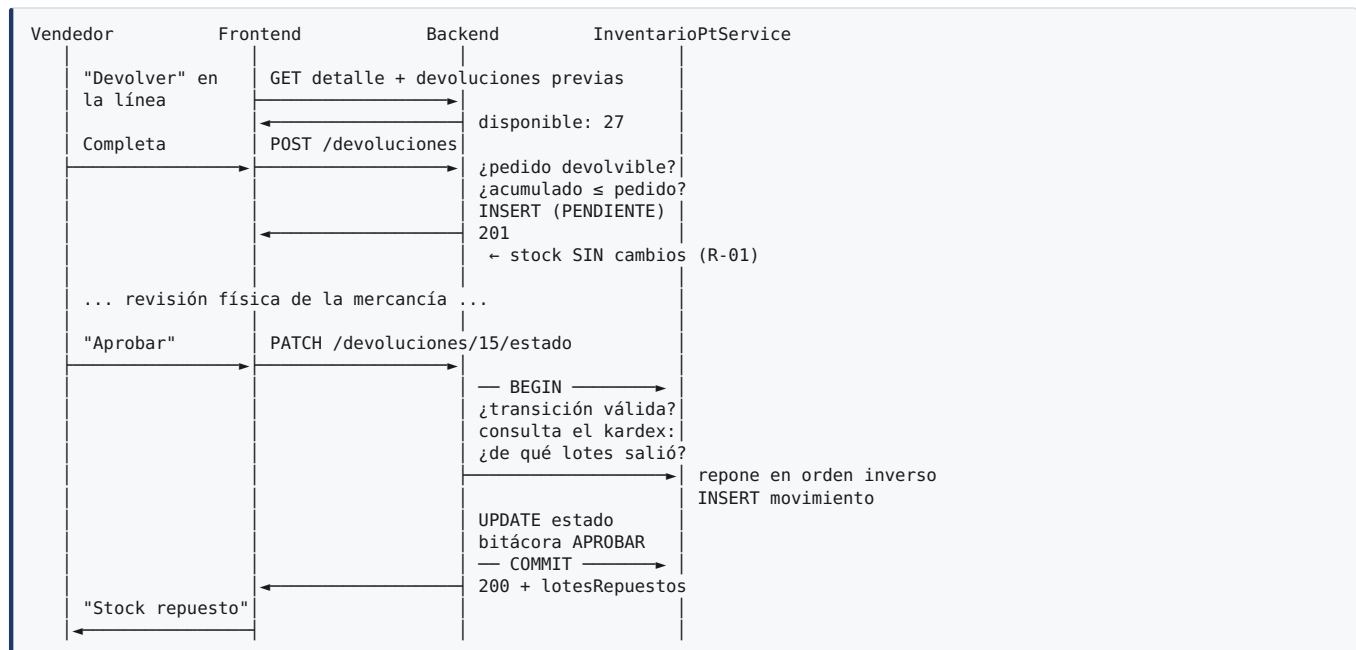
Lote PF-2026-0917+3 und22 → 25 und
Vence el 21/09/2026 (en 3 días)

⚠ Esta acción no se puede deshacer.

[Cancelar] [Confirmar aprobación]

Muestra a qué lote vuelve y su vencimiento (R-02). Si el lote está vencido, se advierte que la mercancía no volverá como stock vendible.

2. Flujo



3. Componentes Angular Material

Elemento	Componente
Bandeja	<code>MatTable</code> + <code>MatSort</code> + <code>MatPaginator</code>
Estado	<code>MatChip</code> — ámbar pendiente, verde aprobada, gris rechazada
Resumen de cantidades	<code>MatCard</code> con los tres números
Motivo	<code>MatSelect</code> con opción "Otro" + <code>MatInput</code>
Descripción	<code>MatInput</code> textarea con contador
Diálogos	<code>MatDialog</code> (aprobar / rechazar)
Rango de fechas	<code>MatDateRangePicker</code>

4. Validaciones en el formulario

Campo	Reglas	Mensaje
Cantidad	requerida, > 0, ≤ disponible	"Solo puede devolver 27 unidades"
Motivo	requerido, máx. 50	"Indique el motivo de la devolución"
Descripción	opcional, máx. 255	"Máximo 255 caracteres"

⚠ El `max` del cliente es orientativo: otra devolución pendiente puede registrarse mientras tanto. El `422` del servidor es la autoridad (R-03).

5. Estados de la pantalla

Estado	Qué se muestra
Cargando	<code>MatProgressBar</code>
Bandeja vacía	✓ "No hay devoluciones pendientes de revisión"
Vacío con filtro	"Ninguna devolución coincide con los filtros"
Sin cantidad disponible	⚠ "Esta línea ya fue devuelta por completo" — sin formulario
Lote de destino vencido	⚠ "El lote de destino venció. La mercancía no volverá como stock vendible."
Aprobando	Botón con spinner
<code>422</code> al enviar	Mensaje del servidor con el disponible actualizado

La bandeja vacía es **buena noticia**: se muestra en verde, no con el mensaje genérico de "sin resultados".

6. Accesibilidad y UX

- Todos los campos con `<mat-label>`; nunca solo *placeholder*.
- Navegación por teclado en orden lógico; `Enter` envía el formulario.
- Los mensajes de error del servidor se muestran **junto al campo** (`fieldErrors`).
- El botón de envío se deshabilita mientras la petición está en vuelo.
- Confirmación antes de descartar cambios sin guardar.
- Los tres estados obligatorios (cargando / vacío / error) según `00-base/05-ESTANDARES-QA.md` §7.

Tarea Backend

Aprobar una devolución repone stock, así que el cambio de estado es transaccional. Reutiliza `InventarioPtService` de HU-17 y consulta el kardex para saber a qué lote devolver.

1. Pasos

1. `EstadoDevolucion` (enum) con `transicionesValidas()` (R-04).
2. `Devolucion` (POJO) y `DevolucionRowMapper` con `JOIN detalle_pedido`, `pedidos`, `productos` y `clientes`.
3. `DevolucionRepository`:
 - `insertar`, `buscarPorId`, `cambiarEstado`;
 - `buscar(filtros, page)`, `contar(...)`;
 - `sumarDevueltoPorDetalle(idDetallePedido)` → suma de `APROBADA` + `PENDIENTE`, excluyendo `RECHAZADA` (R-03).
4. El `INSERT` omite `fecha_devolucion` para que actúe el `DEFAULT (curdate())` (R-06).
5. DTO: `CrearDevolucionRequest`, `CambiarEstadoDevolucionRequest`, `DevolucionResponse` (con `disponibleParaDevolver`), `AprobacionResponse` (con `lotesRepuestos`).
6. `DevolucionService.crear()`:
 - el detalle de pedido existe → `DETALLE_PEDIDO_INEXISTENTE`;
 - el pedido está `ENTREGADO` o `EN_PREPARACION` → `PEDIDO_NO_DEVOLVIBLE` (R-05);
 - `sumarDevueltoPorDetalle()` + `nueva <= cantidad` → `422 CANTIDAD_EXCEDE_PEDIDO` (R-03);
 - estado inicial `PENDIENTE`.
7. `DevolucionService.cambiarEstado()` `@Transactional`:
 - valida la transición;
 - **si es `APROBADA`**:
 - consulta `movimientos_inventario_pt` con `tabla_origen = 'detalle_pedido' AND id_origen = idDetallePedido` y tipo `SALIDA_VENTA`, **ordenado por `id_movimiento DESC`** (orden inverso al consumo, R-02);
 - reparte la cantidad devuelta entre esos lotes en ese orden;
 - `InventarioPtService.registrarEntrada(idLote, cant, ENTRADA_DEVOLUCION, "devoluciones", idDevolucion)`;
 - actualiza el estado;
 - `@Auditable(accion="APROBAR_DEVOLUCION")`.
8. Controlador con `@PreAuthorize`: registrar y aprobar, ADMIN y VENTAS.

2. Archivos

```
backend/src/main/java/co/edu/corposucre/productionfood/devolucion/  
├── {Devolucion,DevolucionRowMapper,DevolucionRepository}.java  
├── {DevolucionService,DevolucionController}.java  
├── EstadoDevolucion.java  
└── dto/{CrearDevolucionRequest,CambiarEstadoDevolucionRequest,  
    DevolucionResponse,AprobacionResponse}.java
```

3. Puntos de cuidado

- **El stock vuelve al aprobar, nunca al registrar** (R-01). Reponer al registrar devolvería al inventario producto en mal estado, listo para venderse otra vez.
- **Contar las `PENDIENTE` en el acumulado** (R-03). Si solo se suman las aprobadas, tres devoluciones pendientes de 10 sobre una línea de 10 repondrían 30 al aprobarse.
- **Consultar el kardex para saber a qué lote devolver** (R-02). Elegir un lote arbitrario del producto rompe la trazabilidad y puede reponer sobre un lote que ni siquiera participó en esa venta.
- Devolver en **orden inverso al consumo**: el último lote tomado es el de vencimiento más lejano.
- Omitir `fecha_devolucion` en el `INSERT` (R-06).

- Ambos estados finales son terminales (R-04).

4. Checklist de cierre

- ☐ Todos los CA implementados y verificados manualmente.
- ☐ Endpoints documentados en Swagger con ejemplos.
- ☐ `@PreAuthorize` en cada método, según la matriz de roles.
- ☐ Operaciones de escritura auditadas en bitácora (HU-04).
- ☐ Pruebas unitarias de las reglas con ramificación.
- ☐ Sin `System.out.println` ni código comentado.
- ☐ PR bajo ~250 líneas, revisado y aprobado.
- ☐ **Probado: aprobar repone exactamente en los lotes de los que salió la venta.**
- ☐ **Probado: el acumulado incluye las devoluciones pendientes.**

Tarea Frontend

Registro desde el detalle del pedido y bandeja de aprobación. El diálogo de aprobación debe dejar claro que repone stock.

1. Pasos

1. `devolucion.model.ts` y `devolucion.service.ts`.
2. Acción **“Registrar devolución”** en cada línea del detalle del pedido (HU-17), visible solo si el pedido es devolvablee (R-05).
3. `devolucion-formulario.component`:
 - muestra cantidad pedida, ya devuelta y **disponible para devolver**;
 - `max` de cantidad = disponible;
 - motivo como `MatSelect` con opción “Otro” que habilita texto libre (R-07);
 - descripción opcional con contador de caracteres.
4. `devolucion-lista.component`: bandeja con pedido, cliente, producto, cantidad, motivo, fecha, estado.
5. Filtro por defecto: **PENDIENTE** — es la bandeja de trabajo.
6. **Diálogo de aprobación** que advierte la reposición de stock y muestra a qué lote vuelve (R-01, R-02).
7. Diálogo de rechazo que solicita el motivo del rechazo.
8. Los tres estados.
9. Ruta: lectura amplia; registrar y aprobar, ADMIN y VENTAS.

2. Archivos

```
frontend/src/app/features/devolucion/  
├── models/devolucion.model.ts  
├── services/devolucion.service.ts  
├── pages/{devolucion-lista,devolucion-formulario}/  
├── components/{aprobar-devolucion-dialog,rechazar-devolucion-dialog}/  
└── devolucion.routes.ts
```

3. Puntos de cuidado

- **Mostrar siempre los tres números:** pedido, ya devuelto y disponible. Sin el segundo, el `422` resulta incomprensible para quien registra.
- **El diálogo de aprobación debe decir que el stock vuelve al inventario** (R-01): es una operación que altera existencias, no un simple cambio de etiqueta.
- La bandeja filtrada por **PENDIENTE** convierte la pantalla en una lista de tareas.
- El `max` del cliente es orientativo: otra devolución pendiente puede haberse registrado mientras tanto. El `422` del servidor manda.
- Advertir si el lote de destino está vencido: la unidad no volverá como stock vendible (R-02).

4. Checklist de cierre

- ☐ Todos los CA cubiertos desde la interfaz.
- ☐ Los tres estados (cargando / vacío / error) implementados.
- ☐ Errores del servidor mostrados junto al campo correspondiente.
- ☐ Guard de rol aplicado en la ruta.

- ☐ Modelos TypeScript espejo de los DTO del backend.
- ☐ Probado en Chrome y Firefox a 1366×768.
- ☐ Sin `console.log` ni `any`.

Casos de Prueba

Formato y criterios en `00-base/05-ESTANDARES-QA.md`.

1. Casos funcionales

ID	CA	Descripción	Entrada	Resultado esperado	Tipo
CP-01	CA-01..06	Registrar devolución válida	3 de una línea de 30	201 , estado: "PENDIENTE"	Positivo
CP-02	CA-01	Detalle de pedido inexistente	{"idDetallePedido":9999}	409 DETALLE_PEDIDO_INEXISTENTE	Negativo
CP-03	CA-02	Motivo vacío	{"motivo":""}	400 VALIDACION_FALLIDA	Negativo
CP-04	CA-02	Motivo de 51 caracteres	"A"×51	400	Borde
CP-05	CA-03	Sin descripción	Omitir <code>descripcion</code>	201 , <code>descripcion: null</code>	Positivo
CP-06	CA-03	Descripción de 256 caracteres	"A"×256	400	Borde
CP-07	CA-04	No se puede devolver más de lo pedido	Devolver 31 de una línea de 30	422 CANTIDAD_EXCEDE_PEDIDO	Negativo
CP-08	CA-04	Devolver exactamente lo pedido	30 de 30	201	Borde
CP-09	CA-04	El acumulado incluye las pendientes	Línea de 10; devolución pendiente de 8; registrar otra de 5	422 — solo quedan 2 disponibles (R-03)	Integridad
CP-10	CA-04	Las rechazadas liberan cantidad	Línea de 10; devolución de 8 rechazada; registrar 8	201 — las rechazadas no cuentan (R-03)	Borde
CP-11	CA-04	Cantidad cero	{"cantidadDevuelta":0}	400 CANTIDAD_INVALIDA	Negativo
CP-12	CA-05	La fecha la pone el servidor	Crear sin <code>fechaDevolucion</code>	Fecha de hoy según el servidor (R-06)	Positivo
CP-13	CA-05	Enviar fecha no tiene efecto	{"fechaDevolucion":"2020-01-01"}	Se ignora; queda hoy	Seguridad
CP-14	CA-06	Estado inicial	Crear devolución	Siempre PENDIENTE	Positivo
CP-15	CA-07	Aprobar repone el stock	PATCH a APROBADA con cantidad 3	El inventario del lote sube 3 (R-01)	Integridad
CP-16	CA-07	Registrar NO repone stock	Crear devolución y revisar inventario	El stock no cambia hasta aprobar (R-01)	Integridad
CP-17	CA-07	Vuelve al lote correcto	Línea que consumió 2 lotes; aprobar devolución	Repone en los lotes del kardex, en orden inverso (R-02)	Integridad
CP-18	CA-07	Se registra el movimiento	Tras CP-15	ENTRADA_DEVOLUCION con origen devoluciones	Integridad
CP-19	—	Rechazar no repone stock	PATCH a RECHAZADA	200 ; el inventario no cambia	Positivo
CP-20	—	Aprobar dos veces	PATCH a APROBADA sobre una ya aprobada	409 TRANSICION_INVALIDA ; el stock no sube otra vez	Integridad
CP-21	—	Aprobada no vuelve a pendiente	APROBADA → PENDIENTE	409 TRANSICION_INVALIDA (R-04)	Negativo
CP-22	—	Devolución sobre pedido PENDIENTE	Pedido sin entregar	409 PEDIDO_NO_DEVOLVIBLE (R-05)	Negativo
CP-23	—	Devolución sobre pedido CANCELADO	Pedido cancelado	409 PEDIDO_NO_DEVOLVIBLE — el stock ya volvió	Negativo
CP-24	—	Transaccionalidad de la aprobación	Provocar fallo al reponer	La devolución no queda aprobada	Integridad
CP-25	—	Rol COMPRAS no puede aprobar	Token de COMPRAS	403 SIN_PERMISO	Seguridad

2. Batería de seguridad

Obligatoria en **cada endpoint** de esta historia:

ID	Caso	Esperado
SEC-01	Sin cabecera <code>Authorization</code>	<code>401 NO_AUTENTICADO</code>
SEC-02	Token malformado	<code>401 NO_AUTENTICADO</code>
SEC-03	Token expirado	<code>401 NO_AUTENTICADO</code>
SEC-04	Token de rol <code>sin</code> permiso	<code>403 SIN_PERMISO</code>
SEC-05	Token de usuario con <code>estado = 0</code>	<code>403 USUARIO_INACTIVO</code>

SEC-04 es el caso crítico. Si devuelve `200`, falta `@EnableMethodSecurity` y la autorización del sistema completo está desactivada.

CP-09, CP-16 y CP-20 son los casos críticos.

CP-09: si el acumulado solo cuenta las aprobadas, se pueden registrar varias devoluciones pendientes que sumadas superan lo vendido; al aprobarlas todas, el inventario recibe más unidades de las que salieron.

CP-16: reponer al registrar en lugar de al aprobar devuelve al estante producto que todavía no se ha revisado — y el motivo más común de devolución es que está en mal estado.

CP-20: sin validar la transición antes de reponer, dos clics en “Aprobar” duplican la reposición.

3. Verificación en base de datos

```
-- CP-15 a CP-18: el stock vuelve al aprobar, y al lote correcto
-- Antes de aprobar:
SELECT l.codigo_lote, l.fecha_vencimiento, ipt.cantidad_disponible
FROM lotes l JOIN inventario_producto_terminado ipt USING (id_lote)
WHERE l.id_producto = 3 ORDER BY l.fecha_vencimiento;

-- De qué lotes salió esa línea de pedido (esto es lo que debe consultar el backend)
SELECT m.id_lote, l.codigo_lote, m.cantidad, m.id_movimiento
FROM movimientos_inventario_pt m JOIN lotes l USING (id_lote)
WHERE m.tabla_origen = 'detalle_pedido' AND m.id_origen = 88
  AND m.tipo_movimiento = 'SALIDA_VENTA'
ORDER BY m.id_movimiento DESC; -- orden inverso al consumo (R-02)

-- (ejecutar PATCH /devoluciones/15/estado {"estado":"APROBADA"})

-- Después: el saldo subió en el lote correcto
SELECT tipo_movimiento, cantidad, saldo_anterior, saldo_posterior, tabla_origen, id_origen
FROM movimientos_inventario_pt
WHERE tabla_origen = 'devoluciones' ORDER BY id_movimiento DESC LIMIT 3;

-- CP-09: el acumulado debe incluir las PENDIENTE
SELECT dp.id_detalle_pedido, dp.cantidad AS pedida,
  COALESCE(SUM(CASE WHEN d.estado IN ('APROBADA', 'PENDIENTE')
    THEN d.cantidad_devuelta END), 0) AS comprometida,
  COALESCE(SUM(CASE WHEN d.estado = 'RECHAZADA'
    THEN d.cantidad_devuelta END), 0) AS rechazada
FROM detalle_pedido dp LEFT JOIN devoluciones d USING (id_detalle_pedido)
WHERE dp.id_detalle_pedido = 88
GROUP BY dp.id_detalle_pedido, dp.cantidad;
-- "comprometida" NUNCA debe superar "pedida"

-- CP-20: aprobar dos veces no debe generar dos reposiciones
SELECT COUNT(*) FROM movimientos_inventario_pt
WHERE tabla_origen = 'devoluciones' AND id_origen = 15;
-- Debe ser igual al número de lotes repuestos, nunca el doble

-- CP-12: el DEFAULT de V2 §C-07
SHOW CREATE TABLE devoluciones\G
-- `fecha_devolucion` date NOT NULL DEFAULT (curdate())
```

4. Criterios de cierre

- ☐ Todos los casos ejecutados; los positivos pasan.
- ☐ Los negativos devuelven el HTTP y el `code` esperados.
- ☐ SEC-01 a SEC-05 pasan en todos los endpoints.
- ☐ Ningún endpoint supera 2 s (RNF-02).
- ☐ Cero defectos críticos o altos abiertos.
- ☐ Colección de Postman actualizada y versionada.

HU-19 · Lotes de producto terminado

Especificación

Fase 6 · Fuente: `previo/Historias_de_Usuario.md`

Como administrador, **quiero** registrar lotes de producto terminado (código, producto, fechas, cantidad), **para que** sepa qué hay en estante.

Depende de: HU-11 (productos)

1. Criterios de aceptación

CA	Enunciado	Cómo se implementa
CA-01	<code>codigo_lote</code> obligatorio y único	<code>UNIQUE KEY codigo_lote</code> ; compartido con los lotes de materia prima (HU-09)
CA-02	<code>id_producto</code> debe existir	Validado contra <code>ProductoRepository</code>
CA-03	<code>fecha_produccion</code> y <code>fecha_vencimiento</code> obligatorias	<code>CHECK (vencimiento >= produccion)</code> en V2 §C-10
CA-04	<code>cantidad</code> obligatoria (decimal)	<code>CHECK (cantidad > 0)</code> en V2
CA-05	Se crea automáticamente un registro en <code>inventario_producto_terminado</code>	Transaccional; es lo que convierte el lote en stock vendible
CA-06	Listar lotes por producto o por estado de vencimiento	<code>?idProducto=</code> , <code>?soloVencidos=</code> , <code>?proximosAVencer=</code>

2. Endpoints

Método	Ruta	Descripción	Roles
GET	<code>/api/v1/lotes-producto</code>	Listar con filtros y paginación	ADMIN, VENTAS, PRODUCCION, CONSULTA
GET	<code>/api/v1/lotes-producto/{id}</code>	Obtener un lote con su saldo	ADMIN, VENTAS, PRODUCCION, CONSULTA
POST	<code>/api/v1/lotes-producto</code>	Registrar un lote	ADMIN, PRODUCCION
GET	<code>/api/v1/lotes-producto/{id}/movimientos</code>	Kardex del lote	ADMIN, PRODUCCION, VENTAS

Reglas transversales (paginación, formato de error, autenticación) en `00-base/04-CONTRATO-API.md` .

3. Reglas de negocio

R-01 · Esta historia es la que crea el stock vendible

Sin lotes de producto terminado, `inventario_producto_terminado` está vacío y **HU-17 no puede vender nada**: cualquier intento devuelve `STOCK_INSUFICIENTE` .

Por eso el `00-base/10-ROADMAP.md` la sitúa **antes** de HU-17, aunque su número sea mayor. Planificar HU-17 antes que HU-19 hace que la historia no se pueda ni probar.

R-02 · El alta es transaccional: lote + inventario (CA-05)

Al registrar un lote, en la misma transacción:

- `INSERT` en `lotes` con `id_producto` informado e `id_materia_prima` a `NULL` ;
- `INSERT` en `inventario_producto_terminado` con `cantidad_disponible = cantidad` ;
- `INSERT` en `movimientos_inventario_pt` con tipo `ENTRADA_PRODUCCION` .

Las tres o ninguna. Un lote sin su fila de inventario es un registro que no sirve para nada: no aparece en el stock, no se puede vender, y nada indica que falte algo.

La relación es `1:1` — `inventario_producto_terminado.id_lote` es `UNIQUE` .

R-03 · El saldo vive en el inventario, no en el lote

`lotes.cantidad` es la cantidad con la que el lote **ingresó** y **nunca se modifica**.

`inventario_producto_terminado.cantidad_disponible` es lo que **queda**, y baja con cada venta (HU-17) y sube con cada devolución (HU-18).

Confundirlos es el error natural al leer el esquema: `lotes.cantidad` parece el saldo pero es el histórico de entrada. Consultar `lotes.cantidad` para saber qué hay disponible devuelve siempre la cantidad original, como si nunca se hubiera vendido nada.

R-04 · Un lote vencido deja de ser stock vendible

`v_stock_producto_terminado` filtra `fecha_vencimiento >= curdate()` (§C-02). El lote sigue existiendo con su saldo, pero no cuenta como disponible para HU-17.

Es intencionado: el producto sigue físicamente en el estante hasta que alguien lo retira, pero el sistema no permite venderlo.

R-05 · No se registran lotes con vencimiento pasado

A diferencia de los lotes de materia prima (HU-09 R-05), aquí **sí se bloquea**: registrar producto terminado ya vencido no tiene caso operativo — nacería sin poder venderse. → `400 LOTE_YA_VENCIDO`.

R-06 · Un lote es de producto o de materia prima

`CHECK ck_lotes_exclusividad` de V2 §C-10. En esta historia `id_producto` va informado e `id_materia_prima` va `NULL`. Al revés en HU-09.

R-07 · Relación con los planes de producción

Lo natural sería que HU-21, al registrar la cantidad producida, generara el lote automáticamente. **Las historias no lo piden y no se implementa**: el lote se registra aquí, a mano.

Se documenta como mejora evidente para una fase posterior. Por ahora, producir y registrar el lote son dos acciones separadas, y nada impide olvidar la segunda.

R-08 · El código de lote se normaliza a mayúsculas

Igual que en HU-09 R-06. El `UNIQUE` es *case-insensitive* por la collation, así que `pf-2026-001` y `PF-2026-001` colisionan de todos modos.

4. Modelo de datos

Tabla `lotes` (compartida con HU-09)

Columna	Tipo	Notas
<code>id_lote</code>	<code>int unsigned</code> PK AI	
<code>codigo_lote</code>	<code>varchar(30)</code> NOT NULL UNIQUE	CA-01
<code>id_producto</code>	<code>int unsigned</code> NULL	Informado aquí (R-06)
<code>id_materia_prima</code>	<code>int unsigned</code> NULL	<code>NULL</code> en esta historia
<code>fecha_produccion</code>	<code>date</code> NOT NULL	CA-03
<code>fecha_vencimiento</code>	<code>date</code> NOT NULL	CA-03 · <code>CHECK >= produccion</code>
<code>cantidad</code>	<code>decimal(10,2)</code> NOT NULL	CA-04 · cantidad de ingreso , no saldo (R-03)

Tabla `inventario_producto_terminado`

Columna	Tipo	Notas
<code>id_inv_producto</code>	<code>int unsigned</code> PK AI	
<code>id_lote</code>	<code>int unsigned</code> NOT NULL UNIQUE FK	Relación 1:1 (R-02)
<code>cantidad_disponible</code>	<code>decimal(10,2)</code> NOT NULL	Saldo actual · <code>CHECK >= 0</code> (V2)
<code>fecha_ingreso_estante</code>	<code>datetime</code> NOT NULL DEFAULT <code>CURRENT_TIMESTAMP</code>	

```
// POST /api/v1/lotes-producto
{ "codigoLote": "PF-2026-0918", "idProducto": 3,
  "fechaProduccion": "2026-09-18", "fechaVencimiento": "2026-09-22",
  "cantidad": 25.00 }

// 201 Created
{ "idLote": 31, "codigoLote": "PF-2026-0918",
  "producto": { "idProducto": 3, "nombre": "Pan francés", "unidad": "und" },
  "fechaProduccion": "2026-09-18", "fechaVencimiento": "2026-09-22",
  "cantidadIngresada": 25.00, "cantidadDisponible": 25.00,
  "diasParaVencer": 4, "vencido": false,
  "stockTotalProducto": 102.00 }
```

Se devuelven **cantidadIngresada** y **cantidadDisponible** por separado (R-03): al crear coinciden, pero después divergen y la distinción importa.

Campos ordenables: `fechaVencimiento`, `fechaProduccion`, `codigoLote`, `id`.

5. Códigos de error

code	HTTP	Cuándo
CODIGO_LOTE_DUPLICADO	409	Ya existe un lote con ese código (incluidos los de materia prima)
PRODUCTO_INEXISTENTE	409	El <code>idProducto</code> no existe
FECHAS_INVALIDAS	400	<code>fechaVencimiento</code> anterior a <code>fechaProduccion</code>
LOTE_YA_VENCIDO	400	La fecha de vencimiento ya pasó (R-05)
CANTIDAD_INVALIDA	400	<code>cantidad</code> menor o igual a cero

Los transversales (`VALIDACION_FALLIDA`, `NO_AUTENTICADO`, `SIN_PERMISO`, `RECURSO_NO_ENCONTRADO`) están en `00-base/04-CONTRATO-API.md` §5.

6. Fuera de alcance

Lo que **no** cubre esta historia y no debe implementarse aquí: se limita estrictamente a los criterios de aceptación listados arriba. Cualquier funcionalidad adicional se propone como historia nueva, no se agrega en silencio.

7. Definición de Terminado

Se aplica la DoD de `00-base/01-CONVENCIONES.md` §7.

8. Notas

Sobre la trazabilidad

Este lote es el que HU-17 consume por FEFO y el que HU-18 repone al aprobar una devolución. La cadena completa —qué lote se produjo, a qué cliente fue y qué volvió— queda en `movimientos_inventario_pt`.

Es la información que se necesita ante un retiro sanitario: saber qué clientes recibieron producto de un lote determinado. Por eso el kardex de esta historia expone el historial completo del lote, no solo su saldo.

Diseño

1. Pantallas

Lotes en estante

Lotes de producto terminado

[+ Nuevo lote]

12

lotes con stock

3

vencen en ≤ 3 días

1

vencido con stock

Producto

Todos

☒ Solo con stock

☐ Incluir vencidos

CÓDIGO	PRODUCTO	PRODUCCIÓN	VENCE	INGRESÓ	DISPONIBLE	EST.
CR-2026-0914	Croissant	14/09/26	16/09/26	30 und	8 und	
PF-2026-0915	Pan francés	15/09/26	19/09/26	120 und	0 und	
PF-2026-0917	Pan francés	17/09/26	21/09/26	60 und	40 und	● 3d
PF-2026-0918	Pan francés	18/09/26	22/09/26	25 und	25 und	● 4d
TZ-2026-0916	Torta zanah...	16/09/26	23/09/26	5 und	3 und	● 5d

INGRESÓ y DISPONIBLE son columnas distintas (R-03): la diferencia es lo vendido. **PF-2026-0915** ingresó con 120 y está agotado; **CR-2026-0914** está vencido y **aún tiene 8 unidades en el estante** — hay que retirarlas físicamente.

La tarjeta roja “vencido con stock” es la que exige acción inmediata.

Formulario

Nuevo lote de producto terminado

[X]

Código de lote *

PF-2026-0918

Sugerencia: prefijo del producto + fecha

Producto *

Pan francés (und)

Fecha producción *

18/09/2026

Fecha vencimiento *

22/09/2026

No puede ser anterior a hoy

Cantidad producida *

25 und

i Este lote quedará disponible para la venta.

Stock de Pan francés: 77 und → 102 und

[Cancelar]

[Registrar lote]

El recuadro final conecta esta historia con HU-17: lo que se registra aquí es lo que se puede vender allí.

Kardex de un lote

← Lote PF-2026-0917 · Pan francés

Vence 21/09/2026 (en 3 días)

Ingresó: 60 und

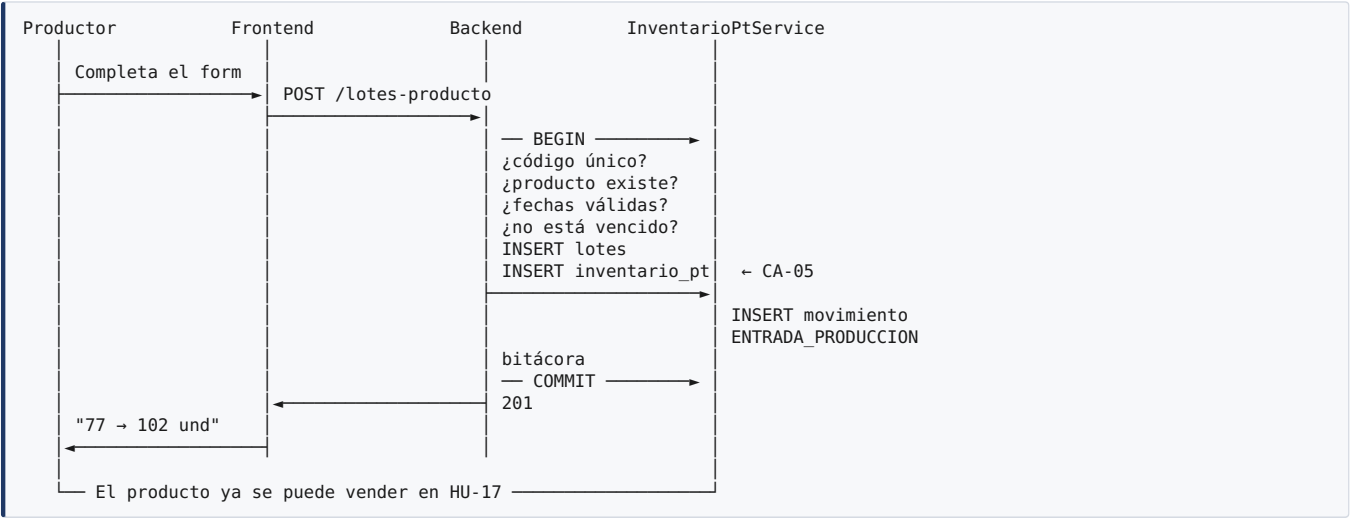
· Disponible: 40 und

· Vendido: 20 und

FECHA	MOVIMIENTO	CANT.	ANTES	DESPUÉS	ORIGEN
18/09 11:20	↗ Entrada devolución	3 und	37	40	Devol. #15
18/09 09:45	↘ Salida venta	18 und	55	37	Pedido #47
17/09 16:10	↘ Salida venta	5 und	60	55	Pedido #46
17/09 08:00	↗ Entrada producción	60 und	0	60	Lote

Esta pantalla es la trazabilidad completa del lote: **qué clientes recibieron producto de él**. Es la información que se necesita ante un retiro sanitario.

2. Flujo



Las tres inserciones son una sola operación (R-02): un lote sin inventario no existe para el resto del sistema.

3. Componentes Angular Material

Elemento	Componente
Tarjetas de resumen	MatCard con conteos por estado
Tabla	MatTable + MatSort + MatPaginator
Semáforo	MatChip con color y días restantes
Producto	MatSelect
Fechas	MatDatepicker con min
Impacto en stock	MatCard destacada, en vivo
Kardex	MatTable con iconos de dirección
Filtros	MatCheckbox + MatSelect

4. Validaciones en el formulario

Campo	Reglas	Mensaje
Código de lote	requerido, máx. 30, único	"El código es obligatorio"
Producto	requerido	"Seleccione un producto"
Fecha producción	requerida	"Indique la fecha de producción"
Fecha vencimiento	requerida, ≥ producción, ≥ hoy	"No puede registrar un lote ya vencido"
Cantidad	requerida, > 0	"Debe ser mayor que cero"

A diferencia de los lotes de materia prima (HU-09 R-05), aquí sí se bloquea el vencimiento pasado (R-05): un lote de producto terminado que nace vencido no se puede vender, así que registrarlo no tiene utilidad.

5. Estados de la pantalla

Estado	Qué se muestra
Cargando	Tarjetas en esqueleto + MatProgressBar
Vacío	"No hay lotes registrados" + "Registrar el primero. Sin lotes no se puede vender."
Vacío con filtro	"Ningún lote coincide con los filtros"
Vencidos con stock	Destacado: "N lotes vencidos aún tienen unidades en estante"
Vencimiento pasado en el formulario	Error bloqueante (R-05)
Guardado	Confirmación con el stock resultante del producto
Error	Mensaje del servidor + "Reintentar"

El estado vacío dice la consecuencia —sin lotes no hay nada que vender—, que es lo que conecta esta historia con HU-17.

6. Accesibilidad y UX

- Todos los campos con `<mat-label>`; nunca solo *placeholder*.
- Navegación por teclado en orden lógico; **Enter** envía el formulario.
- Los mensajes de error del servidor se muestran **junto al campo** (`fieldErrors`).
- El botón de envío se deshabilita mientras la petición está en vuelo.
- Confirmación antes de descartar cambios sin guardar.
- Los tres estados obligatorios (cargando / vacío / error) según `00-base/05-ESTANDARES-QA.md` §7.

Tarea Backend

Bloquea HU-17: sin esta historia no hay stock que vender. Amplía el módulo `lote` de HU-09 y usa `InventarioPtService` de HU-17 — conviene coordinar el orden de implementación con quien haga esa historia.

1. Pasos

1. Ampliar `LoteRepository` con:
 - `insertarLoteProducto()` — `id_producto` informado, `id_materia_prima` a `NULL`;
 - `buscarProducto(filtros, page)` y `contar(...)`;
 - el `SELECT` hace `JOIN inventario_producto_terminado` para traer el saldo actual.
2. `InventarioPtRepository.crearParaLote(idLote, cantidad)`.
3. DTO: `CrearLoteProductoRequest`, `LoteProductoResponse` (con `cantidadIngresada` y `cantidadDisponible` separadas, R-03).
4. `LoteService.crearLoteProducto()` `@Transactional` :
 - normaliza el código a mayúsculas (R-08);
 - valida código único → `CODIGO_LOTE_DUPLICADO`;
 - valida producto existente → `PRODUCTO_INEXISTENTE`;
 - valida fechas → `FECHAS_INVALIDAS`;
 - **valida que no esté vencido** → `LOTE_YA_VENCIDO` (R-05);
 - `INSERT` en `lotes`;
 - `INSERT` en `inventario_producto_terminado` (CA-05, R-02);
 - `INSERT` del movimiento `ENTRADA_PRODUCCION` en `movimientos_inventario_pt`.
5. Filtros: `idProducto`, `soloVencidos`, `proximosAVencer` (días), `conStock`.
6. Kardex del lote desde `movimientos_inventario_pt`.
7. `@Auditable(accion="CREAR", tabla="lotes")`.
8. Controlador: escritura ADMIN y PRODUCCION (`00-base/03-MATRIZ-ROLES.md`).

2. Archivos

```
backend/src/main/java/co/edu/corposucre/productionfood/
├── lote/
│   ├── LoteRepository.java          (+ insertarLoteProducto, buscarProducto)
│   ├── LoteService.java             (+ crearLoteProducto)
│   ├── LoteController.java          (+ endpoints /lotes-producto)
│   └── dto/{CrearLoteProductoRequest,LoteProductoResponse}.java
├── inventario/
│   └── InventarioPtRepository.java  (+ crearParaLote)
```

3. Puntos de cuidado

- **Las tres inserciones en una sola transacción** (R-02). Un lote sin su fila de inventario no aparece en el stock, no se puede vender y **no da ningún error**: parece que el registro funcionó.
- **No confundir `lotes.cantidad` con el saldo** (R-03). Es la cantidad de ingreso y no cambia nunca. El disponible está en `inventario_producto_terminado`. Un `SELECT` que lea `lotes.cantidad` para el stock mostrará siempre la cantidad original.
- El código de lote es único **en toda la tabla**, compartida con los lotes de materia prima de HU-09. Conviene una convención de prefijos (`PF-` producto, `H-` harina) para que no colisionen por accidente.
- `CHECK ck_lotes_exclusividad: id_materia_prima` debe ir `NULL` (R-06).
- Bloquear el registro de lotes ya vencidos (R-05), a diferencia de HU-09.

4. Checklist de cierre

- ☐ Todos los CA implementados y verificados manualmente.
- ☐ Endpoints documentados en Swagger con ejemplos.
- ☐ `@PreAuthorize` en cada método, según la matriz de roles.
- ☐ Operaciones de escritura auditadas en bitácora (HU-04).
- ☐ Pruebas unitarias de las reglas con ramificación.
- ☐ Sin `System.out.println` ni código comentado.
- ☐ PR bajo ~250 líneas, revisado y aprobado.
- ☐ **Verificado: todo lote creado tiene su fila en `inventario_producto_terminado`.**

Tarea Frontend

Listado de lotes en estante con semáforo de vencimiento y formulario de registro.

1. Pasos

1. Ampliar `lote.service.ts` con los métodos de producto terminado.
2. `lote-producto-lista.component` : código, producto, producción, vencimiento, **ingresado**, **disponible**, estado.
3. Mostrar **ingresado y disponible en columnas distintas** (R-03): la diferencia es lo vendido, y verla ahí ahorra ir al kardex.
4. Semáforo: vencido · ● ≤ 3 días · ● ≤ 7 días · ● resto. Los umbrales son más estrictos que en materia prima: el producto terminado dura menos.
5. Filtros: producto, solo con stock, próximos a vencer, incluir vencidos.
6. `lote-producto-formulario.component` :
 - `MatDatepicker` con `min` de vencimiento atado a producción;
 - **bloquear fechas de vencimiento pasadas** (R-05);
 - aviso del stock resultante del producto.
7. Tarjetas de resumen: lotes vigentes, próximos a vencer, vencidos con stock.
8. Vista de kardex del lote: qué entró, qué se vendió, qué volvió.
9. Ruta: lectura amplia; escritura ADMIN y PRODUCCION.

2. Archivos

```
frontend/src/app/features/lote/
├── services/lote-producto.service.ts
├── pages/{lote-producto-lista,lote-producto-formulario,lote-producto-kardex}/
├── components/semaforo-vencimiento/
```

3. Puntos de cuidado

- **Dos columnas, ingresado y disponible** (R-03). Mostrar solo una lleva a que alguien interprete la cantidad de ingreso como stock actual.
- **Los lotes vencidos con stock deben ser visibles y destacados**: son producto que hay que retirar físicamente del estante. Ocultarlos los hace invisibles al problema.
- Umbrales de vencimiento más cortos que en materia prima: un pan dura días, la harina meses.
- El datepicker de vencimiento con `min` = hoy (R-05) evita el `400` antes de enviarlo.

4. Checklist de cierre

- ☐ Todos los CA cubiertos desde la interfaz.
- ☐ Los tres estados (cargando / vacío / error) implementados.
- ☐ Errores del servidor mostrados junto al campo correspondiente.
- ☐ Guard de rol aplicado en la ruta.
- ☐ Modelos TypeScript espejo de los DTO del backend.
- ☐ Probado en Chrome y Firefox a 1366×768.
- ☐ Sin `console.log` ni `any`.

Casos de Prueba

Formato y criterios en `00-base/05-ESTANDARES-QA.md`.

1. Casos funcionales

ID	CA	Descripción	Entrada	Resultado esperado	Tipo
CP-01	CA-01..04	Registrar lote válido	Código, producto, fechas, 25 unidades	201 con cantidadDisponible: 25.00	Positivo
CP-02	CA-05	Se crea la fila de inventario	Tras CP-01	Existe fila en inventario_producto_terminado con 25.00 (R-02)	Integridad
CP-03	CA-05	Se registra el movimiento de entrada	Tras CP-01	ENTRADA_PRODUCCION en movimientos_inventario_pt	Integridad
CP-04	CA-05	El producto pasa a tener stock vendible	Tras CP-01, consultar GET /productos/3/stock	El disponible incluye las 25 unidades (R-01)	Integridad
CP-05	CA-01	Código duplicado	Repetir codigoLote	409 CODIGO_LOTE_DUPLICADO	Negativo
CP-06	CA-01	Código que ya usa un lote de materia prima	Usar un código de HU-09	409 — la tabla lotes es compartida	Borde
CP-07	CA-01	Código con otra capitalización	"pf-2026-0918" existiendo el mayúsculas	409 — se normaliza (R-08)	Borde
CP-08	CA-02	Producto inexistente	{"idProducto":9999}	409 PRODUCTO_INEXISTENTE	Negativo
CP-09	CA-03	Vencimiento anterior a producción	venc < prod	400 FECHAS_INVALIDAS	Negativo
CP-10	CA-03	Lote ya vencido	fechaVencimiento de ayer	400 LOTE_YA_VENCIDO (R-05)	Negativo
CP-11	CA-03	Vence hoy	fechaVencimiento = hoy	201 — todavía es vendible	Borde
CP-12	CA-04	Cantidad cero	{"cantidad":0}	400 CANTIDAD_INVALIDA	Negativo
CP-13	CA-04	Cantidad negativa	{"cantidad":-5}	400	Negativo
CP-14	CA-06	Filtrar por producto	?idProducto=3	Solo lotes de ese producto	Positivo
CP-15	CA-06	Filtrar próximos a vencer	?proximosAVencer=7	Solo los que vencen en ≤ 7 días	Positivo
CP-16	CA-06	Filtrar solo vencidos	?soloVencidos=true	Solo los vencidos	Positivo
CP-17	—	lotes.cantidad no cambia al vender	Vender 10 de un lote de 25	lotes.cantidad sigue en 25; el disponible baja a 15 (R-03)	Integridad
CP-18	—	Un lote vencido no cuenta como stock	Lote con 20 unidades vencido ayer	v_stock_producto_terminado lo excluye (R-04)	Integridad
CP-19	—	El lote vencido sigue existiendo	Tras CP-18	La fila y su saldo permanecen; solo deja de ser vendible	Borde
CP-20	—	Exclusividad producto/materia prima	INSERT directo con ambos informados	La base lo rechaza — ck_lotes_exclusividad (R-06)	Integridad
CP-21	—	Transaccionalidad	Provocar fallo en el INSERT de inventario	No se crea el lote tampoco (R-02)	Integridad
CP-22	—	Kardex del lote	GET /lotes-producto/31/movimientos	Entrada, ventas y devoluciones del lote	Positivo
CP-23	—	Rol VENTAS no puede registrar lotes	Token de VENTAS	403 SIN_PERMISO	Seguridad
CP-24	—	Rol PRODUCCION puede registrar	Token de PRODUCCION	201	Positivo

2. Batería de seguridad

Obligatoria en cada endpoint de esta historia:

ID	Caso	Esperado
SEC-01	Sin cabecera Authorization	401 NO_AUTENTICADO
SEC-02	Token malformado	401 NO_AUTENTICADO
SEC-03	Token expirado	401 NO_AUTENTICADO
SEC-04	Token de rol sin permiso	403 SIN_PERMISO
SEC-05	Token de usuario con estado = 0	403 USUARIO_INACTIVO

SEC-04 es el caso crítico. Si devuelve `200`, falta `@EnableMethodSecurity` y la autorización del sistema completo está desactivada.

CP-02 y CP-17 son los casos críticos.

CP-02: un lote sin fila de inventario se registra sin dar ningún error, aparece en el listado de lotes y es invisible para las ventas. El síntoma llega después, como “registré el lote pero no puedo vender el producto”.

CP-17: confundir `lotes.cantidad` con el saldo disponible es el error natural al leer el esquema. Si alguna ruta de código actualiza `lotes.cantidad` al vender, se pierde el dato de cuánto ingresó el lote y la trazabilidad deja de cuadrar.

3. Verificación en base de datos

```
-- CP-02 y CP-21: todo lote de producto DEBE tener su fila de inventario
SELECT l.id_lote, l.codigo_lote, l.cantidad AS ingresada, ipt.cantidad_disponible
FROM lotes l
LEFT JOIN inventario_producto_terminado ipt USING (id_lote)
WHERE l.id_producto IS NOT NULL AND ipt.id_inv_producto IS NULL;
-- DEBE devolver 0 filas. Un lote sin inventario es invisible para las ventas
-- y no produce ningún error: parece que se registró bien.

-- CP-17: la cantidad del lote NO es el saldo
SELECT l.codigo_lote, l.cantidad AS ingreso_original, ipt.cantidad_disponible AS saldo_actual,
       l.cantidad - ipt.cantidad_disponible AS vendido
FROM lotes l JOIN inventario_producto_terminado ipt USING (id_lote)
WHERE l.id_producto = 3;
-- ingreso_original debe permanecer constante; saldo_actual baja con cada venta.
-- Si ambos bajan juntos, se está actualizando lotes.cantidad: defecto contra R-03.

-- CP-18: la vista excluye vencidos
SELECT * FROM v_stock_producto_terminado WHERE id_producto = 3;
SELECT l.codigo_lote, l.fecha_vencimiento, ipt.cantidad_disponible
FROM lotes l JOIN inventario_producto_terminado ipt USING (id_lote)
WHERE l.id_producto = 3 ORDER BY l.fecha_vencimiento;
-- El total de la vista NO debe incluir los lotes con fecha_vencimiento < hoy

-- CP-20: el CHECK de exclusividad de V2 §C-10
INSERT INTO lotes (codigo_lote, id_producto, id_materia_prima,
                  fecha_produccion, fecha_vencimiento, cantidad)
VALUES ('X-99', 3, 5, curdate(), curdate(), 10);
-- DEBE fallar con error 3819

-- Lotes vencidos que todavía tienen stock: hay que retirarlos físicamente
SELECT l.codigo_lote, p.nombre, l.fecha_vencimiento, ipt.cantidad_disponible
FROM lotes l
JOIN productos p USING (id_producto)
JOIN inventario_producto_terminado ipt USING (id_lote)
WHERE l.fecha_vencimiento < curdate() AND ipt.cantidad_disponible > 0;
```

4. Criterios de cierre

- ☐ Todos los casos ejecutados; los positivos pasan.
- ☐ Los negativos devuelven el HTTP y el `code` esperados.
- ☐ SEC-01 a SEC-05 pasan en todos los endpoints.
- ☐ Ningún endpoint supera 2 s (RNF-02).
- ☐ Cero defectos críticos o altos abiertos.
- ☐ Colección de Postman actualizada y versionada.

HU-20 · Plan de producción

Especificación

Fase 7 · Fuente: `previo/Historias_de_Usuario.md`

Como productor, **quiero** crear un plan de producción con fechas, estado, observaciones y usuario responsable, **para que** organice la fabricación.

Depende de: HU-01 (usuarios)

1. Criterios de aceptación

CA	Enunciado	Cómo se implementa
CA-01	<code>fecha_planificacion</code> y <code>fecha_produccion</code> obligatorias	<code>@NotNull</code> ; producción no puede ser anterior a planificación
CA-02	<code>estado</code> obligatorio	Canónicos: <code>PLANIFICADO</code> , <code>EN_PROCESO</code> , <code>COMPLETADO</code> , <code>CANCELADO</code> — <code>CHECK</code> en V2 §C-12
CA-03	<code>observaciones</code> opcional, máx. 255	<code>@Size(max=255)</code>
CA-04	<code>id_usuario</code> debe existir	Validado; se asigna el usuario autenticado por defecto
CA-05	Se retorna el plan creado con su <code>id_plan</code>	<code>201</code> + <code>Location</code> + <code>PlanResponse</code>

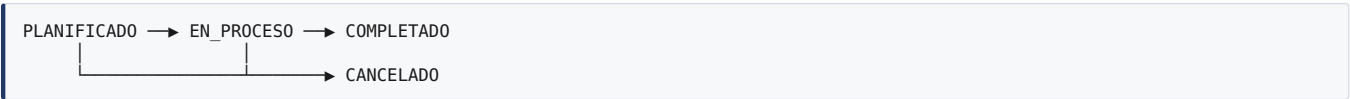
2. Endpoints

Método	Ruta	Descripción	Roles
<code>GET</code>	<code>/api/v1/planes-produccion</code>	Listar con filtros y paginación	ADMIN, PRODUCCION, CONSULTA
<code>GET</code>	<code>/api/v1/planes-produccion/{id}</code>	Obtener plan con su detalle	ADMIN, PRODUCCION, CONSULTA
<code>POST</code>	<code>/api/v1/planes-produccion</code>	Crear un plan	ADMIN, PRODUCCION
<code>PUT</code>	<code>/api/v1/planes-produccion/{id}</code>	Editar cabecera (solo si está <code>PLANIFICADO</code>)	ADMIN, PRODUCCION
<code>PATCH</code>	<code>/api/v1/planes-produccion/{id}/estado</code>	Cambiar estado	ADMIN, PRODUCCION

Reglas transversales (paginación, formato de error, autenticación) en `00-base/04-CONTRATO-API.md` .

3. Reglas de negocio

R-01 · Máquina de estados



`COMPLETADO` es terminal. A diferencia de compras y pedidos, **ningún cambio de estado aquí mueve inventario**: el movimiento lo genera el registro de consumo (HU-22 y HU-23), no la transición del plan.

Transición no permitida → `409 TRANSICION_INVALIDA` .

R-02 · No se puede pasar a `EN_PROCESO` sin productos

Un plan sin líneas de detalle (HU-21) no tiene nada que producir. Iniciallo dejaría un plan en proceso que nunca podrá completarse con sentido → `409 PLAN_SIN_PRODUCTOS` .

R-03 · `fecha_produccion` no puede ser anterior a `fecha_planificacion`

Se planifica antes de producir. Ambas pueden ser el mismo día. Se permite `fecha_produccion` **futura**: es lo normal, se planifica para mañana.

R-04 · El responsable es el usuario autenticado, por defecto

CA-04 exige `id_usuario`. Si el cuerpo no lo envía, se toma de `SecurityUtils.idUsuarioActual()`. Solo `ADMIN` puede asignar el plan a otro usuario; `PRODUCCION` crea planes a su nombre.

Así el responsable no queda a merced de lo que el cliente decida enviar.

R-05 · La cabecera solo se edita mientras está `PLANIFICADO`

Una vez `EN_PROCESO` hay consumos registrados contra las líneas del plan; cambiar la fecha o el responsable desvirtuaría los datos de eficiencia de HU-26.

R-06 · Cancelar un plan no revierte el consumo ya registrado

Si el plan estaba `EN_PROCESO` con consumos registrados (HU-22), esa materia prima **ya se usó físicamente**. Cancelar el plan no la devuelve al inventario.

Es deliberado: el material consumido no vuelve al saco. Cancelar significa “no se sigue produciendo”, no “no pasó nada”. Para corregir un consumo mal registrado hace falta un ajuste de inventario explícito.

R-07 · El plan no genera lotes automáticamente

Lo natural sería que al completar un plan se crearan los lotes de producto terminado (HU-19). **Las historias no lo piden y no se implementa**: producir y registrar el lote son acciones separadas.

Se documenta como mejora evidente, igual que en HU-19 R-07. Por ahora nada impide completar un plan y olvidar el registro del lote, con lo que el producto fabricado no aparecería como stock vendible.

4. Modelo de datos

Tabla `planes_produccion` (V2 añade el `CHECK` de estado y el índice)

Columna	Tipo	Notas
<code>id_plan</code>	<code>int unsigned</code> PK AI	
<code>fecha_planificacion</code>	<code>date</code> NOT NULL	CA-01
<code>fecha_produccion</code>	<code>date</code> NOT NULL	CA-01, R-03
<code>estado</code>	<code>varchar(30)</code> NOT NULL	CA-02 · <code>CHECK</code> en V2 §C-12
<code>observaciones</code>	<code>varchar(255)</code> NULL	CA-03
<code>id_usuario</code>	<code>int unsigned</code> NOT NULL FK	CA-04, R-04

```
// POST /api/v1/planes-produccion
{ "fechaPlanificacion": "2026-09-18", "fechaProduccion": "2026-09-19",
  "observaciones": "Producción para el pedido #47 y reposición de estante" }

// 201 Created · Location: /api/v1/planes-produccion/9
{ "idPlan": 9,
  "fechaPlanificacion": "2026-09-18", "fechaProduccion": "2026-09-19",
  "estado": "PLANIFICADO",
  "observaciones": "Producción para el pedido #47 y reposición de estante",
  "responsable": { "idUsuario": 4, "nombre": "Ismael Batalla" },
  "cantidadProductos": 0, "editable": true }
```

El cuerpo **no acepta** `estado`: siempre nace `PLANIFICADO`.

Campos ordenables: `fechaProduccion`, `fechaPlanificacion`, `estado`, `id`.

5. Códigos de error

code	HTTP	Cuándo
USUARIO_INEXISTENTE	409	El <code>idUsuario</code> no existe
TRANSICION_INVALIDA	409	Transición de estado no permitida (R-01)
PLAN_SIN_PRODUCTOS	409	Se intenta iniciar un plan sin líneas (R-02)
PLAN_NO_EDITABLE	409	Se intenta modificar un plan que no está <code>PLANIFICADO</code> (R-05)
FECHAS_INVALIDAS	400	<code>fechaProduccion</code> anterior a <code>fechaPlanificacion</code> (R-03)

Los transversales (`VALIDACION_FALLIDA` , `NO_AUTENTICADO` , `SIN_PERMISO` , `RECURSO_NO_ENCONTRADO`) están en `00-base/04-CONTRATO-API.md` §5.

6. Fuera de alcance

Lo que **no** cubre esta historia y no debe implementarse aquí: se limita estrictamente a los criterios de aceptación listados arriba. Cualquier funcionalidad adicional se propone como historia nueva, no se agrega en silencio.

7. Definición de Terminado

Se aplica la DoD de `00-base/01-CONVENCIONES.md` §7.

8. Notas

Cómo encaja con el resto de la fase

Historia	Qué aporta
HU-20 (esta)	Cabecera del plan: cuándo, quién, en qué estado
HU-21	Qué productos y cuántos: planificado vs. producido
HU-22	Consumo real de materia prima por producto del plan
HU-23	El consumo descuenta inventario automáticamente

La cabecera no mueve stock. Todo el efecto sobre el inventario ocurre en HU-22 y HU-23.

Diseño

1. Pantallas

Listado de planes

Planes de producción

[+ Nuevo plan]

Estado

Todos

Responsable

Todos

Período

15/09 – 30/09/26

N.º	PRODUCCIÓN	RESPONSABLE	PRODUCTOS	AVANCE	ESTADO
#10	● H0Y 19/09	Ismael Batalla	3		○ Planificado
#9	18/09/2026	Ismael Batalla	2		En proceso
#8	17/09/2026	Natalia Góngora	4		● Completado
#7	16/09/2026	Ismael Batalla	1		Cancelado

La barra de AVANCE es $\Sigma \text{ producido} / \Sigma \text{ planificado}$ de las líneas del plan (HU-21). ● marca la producción del día: es lo que se busca al abrir la pantalla por la mañana.

Detalle del plan

← Plan #9 En proceso |
Planificado: 17/09/2026 · Producción: 18/09/2026
Responsable: Ismael Batalla
Obs.: Producción para el pedido #47 y reposición de estante
[Cancelar] [✓ Completar plan]

PRODUCTOS DEL PLAN (HU-21) [+ Agregar producto]

PRODUCTO	PLANIFICADO	PRODUCIDO	AVANCE	...
----------	-------------	-----------	--------	-----

Pan francés	90 und	60 und		67% =
Croissant	30 und	30 und		100% =

CONSUMO DE MATERIA PRIMA (HU-22)
... (ver HU-22)

Formulario

Nuevo plan de producción [X]

Fecha planificación * Fecha producción *

18/09/2026 19/09/2026

No anterior a la
planificación

Responsable

Ismael Batalla (usted)

Observaciones

Producción para el pedido #47 y
reposición de estante

58 / 255

[Cancelar] [Crear plan]

El responsable aparece como campo **de solo lectura** para PRODUCCION (R-04); solo ADMIN ve un selector.

Cancelación — advertencia honesta

△ Cancelar el plan #9

El plan quedará cancelado y no podrá reactivarse.

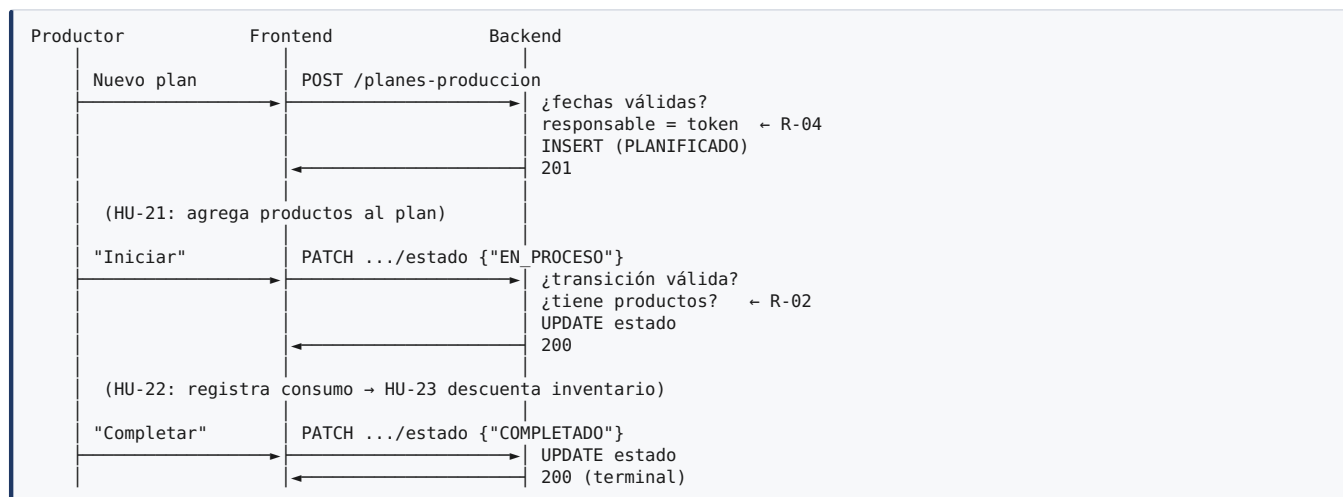
△ La materia prima ya consumida NO volverá al inventario. Se registraron 3 consumos por un total de 18,5 kg.

Para corregir un consumo mal registrado, use un ajuste de inventario.

[Volver] [Cancelar el plan]

Esta advertencia (R-06) es lo que evita la expectativa equivocada de que cancelar “deshace todo”. La harina ya se usó.

2. Flujo



Ninguna de estas transiciones toca el inventario. El stock se mueve en HU-22 y HU-23.

3. Componentes Angular Material

Elemento	Componente
Tabla	<code>MatTable</code> + <code>MatSort</code> + <code>MatPaginator</code>
Estado	<code>MatChip</code> con color
Avance	<code>MatProgressBar</code> con porcentaje
Fechas	<code>MatDatepicker</code> con <code>min</code>
Responsable	<code>MatSelect</code> (solo ADMIN) o texto de solo lectura
Observaciones	<code>MatInput</code> textarea con contador
Acciones de estado	<code>MatRaisedButton</code> con <code>MatTooltip</code> explicativo
Cancelación	<code>MatDialog</code> con advertencia de consumos

4. Validaciones en el formulario

Campo	Reglas	Mensaje
Fecha planificación	requerida	"Indique la fecha de planificación"
Fecha producción	requerida, $\geq$ planificación	"No puede ser anterior a la planificación"
Responsable	requerido (automático)	—
Observaciones	opcional, máx. 255	"Máximo 255 caracteres"

No hay campo de estado (R-01): nace `PLANIFICADO` y cambia por el flujo de acciones.

5. Estados de la pantalla

Estado	Qué se muestra
Cargando	<code>MatProgressBar</code>
Vacío	"No hay planes de producción" + "Crear el primero"
Vacío con filtro	"Ningún plan coincide con los filtros"
Plan sin productos	⚠ "Agregue productos antes de iniciar la producción"; botón "Iniciar" deshabilitado con tooltip (R-02)
Cancelando	Diálogo con el resumen de consumos ya registrados
Error	Mensaje del servidor + "Reintentar"

6. Accesibilidad y UX

- Todos los campos con `<mat-label>`; nunca solo *placeholder*.
- Navegación por teclado en orden lógico; `Enter` envía el formulario.
- Los mensajes de error del servidor se muestran **junto al campo** (`fieldErrors`).
- El botón de envío se deshabilita mientras la petición está en vuelo.

- Confirmación antes de descartar cambios sin guardar.
- Los tres estados obligatorios (cargando / vacío / error) según `00-base/05-ESTANDARES-QA.md` §7.

Tarea Backend

Cabecera con máquina de estados. Más simple que HU-13 y HU-16 porque **ninguna transición mueve inventario** (R-01): el movimiento vive en HU-22 y HU-23.

1. Pasos

1. `EstadoPlan` (enum) con `transicionesValidas()` (R-01), mismo patrón que `EstadoCompra` y `EstadoPedido`.
2. `PlanProduccion` (POJO) y `PlanProduccionRowMapper` con `JOIN usuarios` para el nombre del responsable; el listado cuenta las líneas con subconsulta sobre `detalle_plan_produccion`.
3. `PlanProduccionRepository`: `buscar`, `contar`, `buscarPorId`, `insertar`, `actualizar`, `cambiarEstado`, `contarDetalles(idPlan)`.
4. DTO: `CrearPlanRequest` (sin `estado`), `ActualizarPlanRequest`, `CambiarEstadoPlanRequest`, `PlanResponse`.
5. `PlanProduccionService.crear()`:
 - valida `fechaProduccion >= fechaPlanificacion` → `FECHAS_INVALIDAS`;
 - **asigna el usuario autenticado** si no viene `idUsuario`, y solo permite asignar otro si el solicitante es `ADMIN` (R-04);
 - estado inicial `PLANIFICADO`.
6. `PlanProduccionService.actualizar()`: solo si está `PLANIFICADO` → `PLAN_NO_EDITABLE`.
7. `PlanProduccionService.cambiarEstado()` `@Transactional`:
 - valida la transición;
 - **al pasar a `EN_PROCESO`, exige al menos una línea** → `PLAN_SIN_PRODUCTOS` (R-02);
 - **no toca inventario** (R-06).
8. `esEditable(idPlan)` para que HU-21 lo consulte.
9. Controlador: escritura ADMIN y PRODUCCION.

2. Archivos

```
backend/src/main/java/co/edu/corposucre/productionfood/produccion/  
├── {PlanProduccion,PlanProduccionRowMapper,PlanProduccionRepository}.java  
├── {PlanProduccionService,PlanProduccionController}.java  
├── EstadoPlan.java  
└── dto/{CrearPlanRequest,ActualizarPlanRequest,CambiarEstadoPlanRequest,PlanResponse}.java
```

3. Puntos de cuidado

- **No confiar en el `idUsuario` que envía el cliente** (R-04): un usuario de PRODUCCION podría asignar su plan a otra persona y distorsionar la trazabilidad. Se toma del token salvo que quien llama sea `ADMIN`.
- **Cancelar no revierte consumos** (R-06). Si alguien pide “deshacer todo al cancelar”, la respuesta es que la harina ya se usó: se corrige con un ajuste de inventario, no borrando el consumo.
- La validación de R-02 va en el cambio de estado, no en la creación: un plan recién creado no tiene líneas todavía, y eso es normal.
- Ninguna transición de esta historia llama a `InventarioService`.

4. Checklist de cierre

- ☐ Todos los CA implementados y verificados manualmente.
- ☐ Endpoints documentados en Swagger con ejemplos.
- ☐ `@PreAuthorize` en cada método, según la matriz de roles.
- ☐ Operaciones de escritura auditadas en bitácora (HU-04).
- ☐ Pruebas unitarias de las reglas con ramificación.
- ☐ Sin `System.out.println` ni código comentado.
- ☐ PR bajo ~250 líneas, revisado y aprobado.

Tarea Frontend

Listado y formulario de cabecera, más la pantalla de detalle que aloja HU-21 y HU-22.

1. Pasos

1. `plan-produccion.model.ts` con el enum `EstadoPlan`, y su servicio.
2. `plan-lista.component`: número, fecha de producción, responsable, nº de productos, estado, `!`.
3. Chip de estado: gris `PLANIFICADO`, azul `EN_PROCESO`, verde `COMPLETADO`, rojo tenue `CANCELADO`.
4. **Resaltar los planes cuya producción es hoy**: es la pantalla de trabajo de la jornada.
5. Filtros: estado, responsable, rango de fechas de producción.
6. `plan-formulario.component`: fechas (`min` de producción atado a planificación), observaciones con contador. **Sin campo de estado** (R-01).
7. `plan-detalle.component`: cabecera + productos (HU-21) + consumos (HU-22) + acciones de estado.
8. Botón “Iniciar producción” **deshabilitado si el plan no tiene productos** (R-02), con explicación en tooltip.
9. Diálogo de cancelación que advierte: “Los consumos ya registrados no se revertirán.”
10. Ruta: lectura ADMIN/PRODUCCION/CONSULTA; escritura ADMIN/PRODUCCION.

2. Archivos

```
frontend/src/app/features/produccion/  
├── models/plan-produccion.model.ts  
├── services/plan-produccion.service.ts  
├── pages/{plan-lista,plan-formulario,plan-detalle}/  
└── produccion.routes.ts
```

3. Puntos de cuidado

- **El formulario no ofrece el estado** (R-01): nace `PLANIFICADO` y cambia por el flujo.
- Deshabilitar “Iniciar producción” con tooltip explicativo es mejor que permitir el clic y devolver `409` (R-02).
- El diálogo de cancelación debe ser honesto sobre R-06: la materia prima consumida no vuelve. Sin esa advertencia, cancelar parece inocuo.
- Resaltar la producción del día conecta con el dashboard de HU-24.

4. Checklist de cierre

- ☐ Todos los CA cubiertos desde la interfaz.
- ☐ Los tres estados (cargando / vacío / error) implementados.
- ☐ Errores del servidor mostrados junto al campo correspondiente.
- ☐ Guard de rol aplicado en la ruta.
- ☐ Modelos TypeScript espejo de los DTO del backend.
- ☐ Probado en Chrome y Firefox a 1366×768.
- ☐ Sin `console.log` ni `any`.

Casos de Prueba

Formato y criterios en `00-base/05-ESTANDARES-QA.md`.

1. Casos funcionales

ID	CA	Descripción	Entrada	Resultado esperado	Tipo
CP-01	CA-01..05	Crear plan válido	Fechas válidas + observaciones	201, estado: "PLANIFICADO"	Positivo
CP-02	CA-01	Fecha de planificación omitida	Sin fechaPlanificacion	400 VALIDACION_FALLIDA	Negativo
CP-03	CA-01	Producción anterior a planificación	prod = ayer, plan = hoy	400 FECHAS_INVALIDAS (R-03)	Negativo
CP-04	CA-01	Ambas fechas iguales	prod = plan = hoy	201 — permitido (R-03)	Borde
CP-05	CA-01	Producción futura	prod = dentro de 7 días	201 — es lo normal	Positivo
CP-06	CA-02	Estado inicial	Crear plan	Siempre PLANIFICADO, aunque se envíe otro	Positivo
CP-07	CA-02	Estado inválido	{"estado": "EN_HORNO"}	400 o se ignora	Negativo
CP-08	CA-03	Sin observaciones	Omitir el campo	201, observaciones: null	Positivo
CP-09	CA-03	Observaciones de 256 caracteres	"A"×256	400	Borde
CP-10	CA-04	El responsable es el usuario del token	Crear sin idUsuario	responsable = usuario autenticado (R-04)	Positivo
CP-11	CA-04	PRODUCCION no puede asignar a otro	Token PRODUCCION con idUsuario ajeno	Se ignora; queda a su nombre (R-04)	Seguridad
CP-12	CA-04	ADMIN sí puede asignar a otro	Token ADMIN con idUsuario ajeno	201 con ese responsable	Positivo
CP-13	CA-04	Usuario inexistente	{"idUsuario": 9999} como ADMIN	409 USUARIO_INEXISTENTE	Negativo
CP-14	—	No se inicia un plan sin productos	PLANIFICADO → EN_PROCESO sin líneas	409 PLAN_SIN_PRODUCTOS (R-02)	Negativo
CP-15	—	Iniciar con productos	Plan con 2 líneas → EN_PROCESO	200	Positivo
CP-16	—	Completar plan	EN_PROCESO → COMPLETADO	200	Positivo
CP-17	—	COMPLETADO es terminal	COMPLETADO → EN_PROCESO	409 TRANSICION_INVALIDA (R-01)	Negativo
CP-18	—	Salto de estado	PLANIFICADO → COMPLETADO	409 TRANSICION_INVALIDA	Negativo
CP-19	—	Cancelar NO revierte consumos	Plan con consumos registrados → CANCELADO	200; el inventario no cambia (R-06)	Integridad
CP-20	—	Editar plan PLANIFICADO	PUT sobre uno planificado	200	Positivo
CP-21	—	Editar plan EN_PROCESO	PUT sobre uno en proceso	409 PLAN_NO_EDITABLE (R-05)	Negativo
CP-22	—	Rol VENTAS no puede crear planes	Token de VENTAS	403 SIN_PERMISO	Seguridad
CP-23	—	Rol CONSULTA puede listar	GET con token CONSULTA	200	Positivo

2. Batería de seguridad

Obligatoria en **cada endpoint** de esta historia:

ID	Caso	Esperado
SEC-01	Sin cabecera Authorization	401 NO_AUTENTICADO
SEC-02	Token malformado	401 NO_AUTENTICADO
SEC-03	Token expirado	401 NO_AUTENTICADO
SEC-04	Token de rol sin permiso	403 SIN_PERMISO
SEC-05	Token de usuario con estado = 0	403 USUARIO_INACTIVO

SEC-04 es el caso crítico. Si devuelve `200`, falta `@EnableMethodSecurity` y la autorización del sistema completo está desactivada.

CP-11 es una prueba de seguridad, no solo funcional. Si el backend acepta el `idUsuario` que envía el cliente sin verificar el rol, cualquier usuario de PRODUCCION puede crear planes a nombre de otra persona, y la trazabilidad de quién planificó qué deja de valer.

3. Verificación en base de datos

```
-- CP-10/CP-11: el responsable debe ser quien creó el plan
SELECT p.id_plan, p.fecha_produccion, p.estado, u.nombre AS responsable
FROM planes_produccion p JOIN usuarios u USING (id_usuario)
ORDER BY p.id_plan DESC LIMIT 5;

-- CP-19: cancelar no debe tocar el inventario
-- Antes de cancelar:
SELECT id_materia_prima, cantidad_disponible FROM inventario;
-- (ejecutar PATCH ../estado {"estado":"CANCELADO"})
-- Después: idénticos. Y ningún movimiento nuevo:
SELECT COUNT(*) FROM movimientos_inventario_mp
WHERE tabla_origen = 'planes_produccion';
-- Debe ser 0: el plan nunca genera movimientos por sí mismo

-- CP-14: contar líneas antes de iniciar
SELECT COUNT(*) FROM detalle_plan_produccion WHERE id_plan = 9;

-- CP-06 a CP-18: el CHECK de estados de V2 §C-12
SHOW CREATE TABLE planes_produccion\G
-- CHECK ('estado' in ('PLANIFICADO','EN_PROCESO','COMPLETADO','CANCELADO'))
```

4. Criterios de cierre

- ☐ Todos los casos ejecutados; los positivos pasan.
- ☐ Los negativos devuelven el HTTP y el `code` esperados.
- ☐ SEC-01 a SEC-05 pasan en todos los endpoints.
- ☐ Ningún endpoint supera 2 s (RNF-02).
- ☐ Cero defectos críticos o altos abiertos.
- ☐ Colección de Postman actualizada y versionada.

HU-21 · Productos del plan

Especificación

Fase 7 · Fuente: `previo/Historias_de_Usuario.md`

Como productor, **quiero** agregar productos al plan de producción con cantidad planificada vs. producida, **para que** controle la ejecución.

Depende de: HU-20, HU-11

1. Criterios de aceptación

CA	Enunciado	Cómo se implementa
CA-01	<code>id_plan</code> debe existir	Validado; además el plan no puede estar cerrado
CA-02	<code>id_producto</code> debe existir	Validado contra <code>ProductoRepository</code>
CA-03	<code>cantidad_planificada</code> obligatoria (decimal positivo)	<code>CHECK > 0</code> en V2
CA-04	<code>cantidad_producida</code> inicia en <code>0.00</code> y se actualiza durante la producción	<code>DEFAULT 0.00</code> ya en el esquema; endpoint propio para actualizarla
CA-05	Listar el detalle de un plan	<code>GET /planes-produccion/{id}/detalles</code>

2. Endpoints

Método	Ruta	Descripción	Roles
<code>GET</code>	<code>/api/v1/planes-produccion/{idPlan}/detalles</code>	Listar los productos del plan	ADMIN, PRODUCCION, CONSULTA
<code>POST</code>	<code>/api/v1/planes-produccion/{idPlan}/detalles</code>	Agregar un producto	ADMIN, PRODUCCION
<code>PUT</code>	<code>/api/v1/planes-produccion/{idPlan}/detalles/{id}</code>	Editar la cantidad planificada	ADMIN, PRODUCCION
<code>PATCH</code>	<code>/api/v1/planes-produccion/{idPlan}/detalles/{id}/producido</code>	Registrar cantidad producida	ADMIN, PRODUCCION
<code>DELETE</code>	<code>/api/v1/planes-produccion/{idPlan}/detalles/{id}</code>	Quitar un producto	ADMIN, PRODUCCION

Reglas transversales (paginación, formato de error, autenticación) en `00-base/04-CONTRATO-API.md`.

3. Reglas de negocio

R-01 · Dos operaciones distintas sobre la misma fila

Se separan deliberadamente en endpoints diferentes porque ocurren en momentos y con permisos distintos:

Operación	Endpoint	Cuándo	Estado del plan
Planificar	<code>PUT ../detalles/{id}</code>	Antes de producir	Solo <code>PLANIFICADO</code>
Registrar producido	<code>PATCH ../detalles/{id}/producido</code>	Durante la producción	<code>EN_PROCESO</code>

Un único `PUT` que aceptara ambos campos permitiría modificar lo planificado *después* de producir, y HU-26 compararía contra un objetivo reescrito a posteriori: la eficiencia siempre saldría perfecta.

R-02 · `cantidad_planificada` solo se edita mientras el plan está `PLANIFICADO`

Es la regla que da sentido a HU-26: el plan es el compromiso, y comparar lo producido contra un compromiso ajustado sobre la marcha no mide nada.

R-03 · `cantidad_producida` solo se registra con el plan `EN_PROCESO`

No antes —no se ha empezado— ni después de `COMPLETADO` o `CANCELADO`.

R-04 · Se puede producir más de lo planificado

No se bloquea. Una tanda puede rendir más de lo previsto, y eso es un dato real que HU-26 debe poder mostrar. La interfaz lo señala (avance > 100%) pero no lo impide. Bloquearlo obligaría a falsear el registro.

R-05 · Un producto no puede repetirse en el mismo plan

`UNIQUE (id_plan, id_producto)` añadido en V2 §C-13. Para producir más del mismo producto se **edita la cantidad planificada**, no se agrega otra línea.

Sin la restricción, HU-22 tendría dos líneas de detalle para el mismo producto y el consumo teórico se calcularía dos veces.

R-06 · Un producto sin receta se puede planificar, pero no calcular su consumo

HU-22 calcula el consumo teórico a partir de la receta (HU-12). Un producto sin receta puede añadirse al plan —quizá se produce artesanalmente— pero **no tendrá consumo teórico** y quedará fuera del análisis de eficiencia de HU-26. Se permite, y se advierte claramente en la interfaz. Ocultar el hueco haría que apareciera después como un consumo teórico de cero sin explicación.

R-07 · Registrar producción no crea el lote ni suma stock

Actualizar `cantidad_producida` es un registro de avance. **No genera el lote de producto terminado** (HU-19) ni suma a `inventario_producto_terminado`.

Es la consecuencia de HU-19 R-07 y HU-20 R-07: producir y registrar el lote son acciones separadas en este alcance. **Nada impide completar un plan y olvidar registrar el lote**, con lo que el producto fabricado no aparecería como stock vendible. Se documenta como el hueco más visible del flujo de producción y candidato claro a una historia futura.

R-08 · Quitar una línea con consumos registrados

Si HU-22 ya registró consumo contra esa línea, quitarla dejaría movimientos de inventario huérfanos. Se devuelve `409 DETALLE_CON_CONSUMOS`: primero hay que resolver el consumo.

4. Modelo de datos

Tabla `detalle_plan_produccion` (V2 añade el `UNIQUE` y dos `CHECK`)

Columna	Tipo	Notas
<code>id_detalle_plan</code>	<code>int unsigned</code> PK AI	
<code>id_plan</code>	<code>int unsigned</code> NOT NULL FK	CA-01
<code>id_producto</code>	<code>int unsigned</code> NOT NULL FK	CA-02
<code>cantidad_planificada</code>	<code>decimal(10,2)</code> NOT NULL	CA-03 · <code>CHECK > 0</code>
<code>cantidad_producida</code>	<code>decimal(10,2)</code> NOT NULL DEFAULT 0.00	CA-04 · <code>CHECK >= 0</code>

`UNIQUE KEY uk_detalle_plan (id_plan, id_producto)` — V2 §C-13.

```
// POST /api/v1/planes-produccion/9/detalles
{ "idProducto": 3, "cantidadPlanificada": 90.00 }

// 201 Created
{ "idDetallePlan": 21,
  "producto": { "idProducto": 3, "nombre": "Pan francés", "unidad": "und" },
  "cantidadPlanificada": 90.00, "cantidadProducida": 0.00,
  "avance": 0.00, "tieneReceta": true,
  "consumoTeoricoEstimado": [
    { "materiaPrima": "Harina de trigo", "cantidad": 15.000, "unidad": "kg" },
    { "materiaPrima": "Levadura", "cantidad": 0.150, "unidad": "kg" }
  ]
}

// PATCH /api/v1/planes-produccion/9/detalles/21/producido
{ "cantidadProducida": 60.00 }

// 200 OK
{ "idDetallePlan": 21, "cantidadPlanificada": 90.00,
  "cantidadProducida": 60.00, "avance": 66.67 }
```

Campos ordenables: producto , cantidadPlanificada , avance .

code	HTTP	Cuándo
PLAN_INEXISTENTE	404	El plan no existe
PLAN_NO_EDITABLE	409	El plan no está PLANIFICADO (para editar lo planificado)
PLAN_NO_EN_PROCESO	409	El plan no está EN_PROCESO (para registrar producción)
PRODUCTO_INEXISTENTE	409	El producto no existe
PRODUCTO_DUPLICADO	409	Ese producto ya está en el plan (R-05)
DETALLE_CON_CONSUMOS	409	La línea tiene consumos registrados (R-08)
CANTIDAD_INVALIDA	400	Cantidad planificada menor o igual a cero

Lo que **no** cubre esta historia y no debe implementarse aquí: se limita estrictamente a los criterios de aceptación listados arriba. Cualquier funcionalidad adicional se propone como historia nueva, no se agrega en silencio.

1. **Verificar que hay materia prima suficiente** antes de iniciar la producción, en lugar de descubrirlo a mitad de tanda.
2. **Detectar errores en la receta** (HU-12 R-02): si planificar 90 panes arroja un consumo teórico de 450 kg de harina, la **cantidad_producir** de la receta está mal interpretada. Es una segunda oportunidad de encontrar ese error antes de que llegue a HU-23.

Tabla de productos del plan

Los campos de **PRODUCIDO son editables en línea** cuando el plan está **EN_PROCESO** (R-03).

⚠ **El aviso del pie es importante** (R-07): registrar producción aquí no pone el producto a la venta. Sin ese recordatorio y su enlace a HU-19, es fácil completar un plan y que el producto fabricado nunca aparezca en el inventario vendible.

Pan integral no tiene receta: se puede planificar, pero quedará fuera del análisis de consumo y eficiencia (R-06).

Agregar producto al plan

Agregar producto al plan[X]

Producto *

Pan francés (und)▼

Cantidad a producir *

90und

Materia prima que se necesitará

Harina de trigo	15,000 kg	disponible	112,50	✓
Levadura	0,150 kg	disponible	0,00	
Sal	0,300 kg	disponible	8,20	✓

⚠ No hay levadura suficiente para esta producción

[Cancelar]

[Agregar]

El contraste con el inventario disponible es lo más útil de esta pantalla. Descubrir que falta levadura ahora, al planificar, es muy distinto a descubrirlo a mitad de tanda con la masa ya hecha.

También sirve de segunda verificación de la receta (HU-12 R-02): si planificar 90 panes pidiera 450 kg de harina, el error de **cantidad_producir** salta a la vista aquí.

Producto sin receta

⚠ Este producto no tiene receta

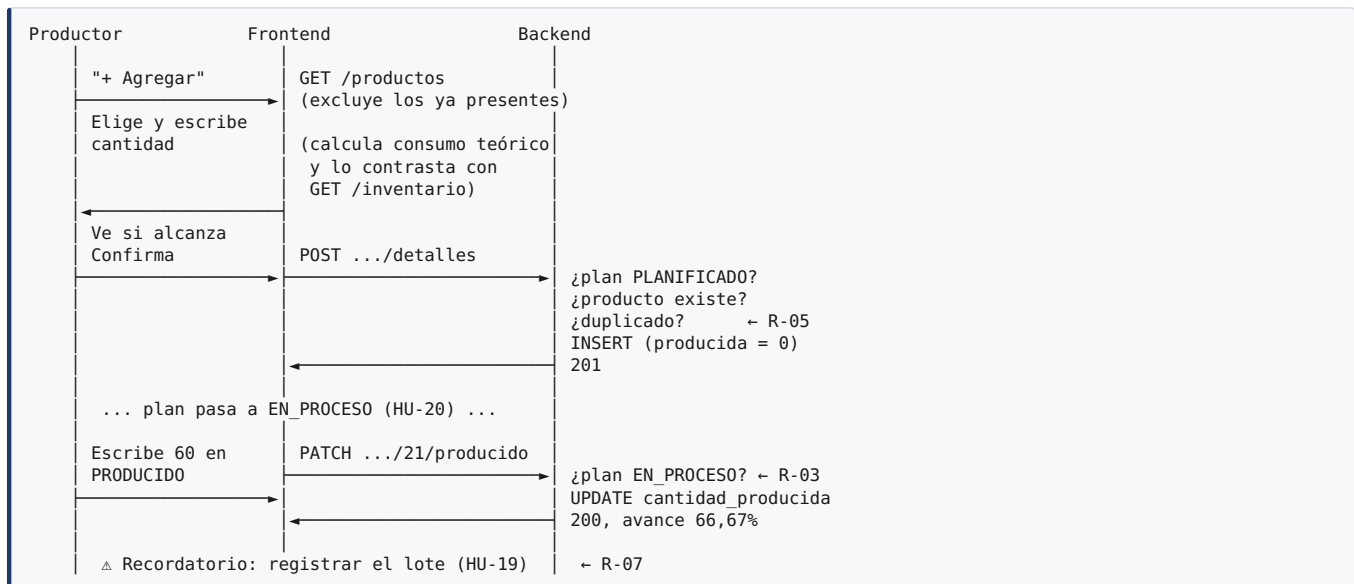
«Pan integral» se puede agregar al plan, pero:

- no se calculará su consumo teórico de materia prima
- quedará fuera del análisis de eficiencia (HU-26)

[Crear receta primero]

[Agregar de todos modos]

2. Flujo



3. Componentes Angular Material

Elemento	Componente
Tabla	<code>MatTable</code> con edición en línea
Avance	<code>MatProgressBar</code> con color por tramo
Producido	<code>MatInput[type=number]</code> en línea, con guardado al perder el foco
Receta	<code>MatIcon</code> ✓/Δ con <code>MatTooltip</code>
Consumo teórico	<code>MatCard</code> en el diálogo, con semáforo de disponibilidad
Agregar	<code>MatDialog</code>
Recordatorio del lote	<code>MatCard</code> informativa con <code>RouterLink</code> a HU-19

4. Validaciones en el formulario

Campo	Reglas	Mensaje
Producto	requerido, no repetido	"Seleccione un producto"
Cantidad planificada	requerida, > 0	"Debe ser mayor que cero"
Cantidad producida	requerida, ≥ 0, sin tope superior	"No puede ser negativa"

La cantidad producida no tiene límite superior (R-04): producir más de lo planificado es un resultado posible y un dato que HU-26 debe poder reflejar.

5. Estados de la pantalla

Estado	Qué se muestra
Cargando	<code>MatProgressBar</code>
Plan sin productos	Δ "Agregue productos antes de iniciar la producción"
Producto sin receta	Δ Advertencia con opción de crear la receta primero
Materia prima insuficiente	En el diálogo, junto al ingrediente que falta
Plan PLANIFICADO	Editable lo planificado; campo de producido deshabilitado
Plan EN_PROCESO	Producido editable; planificado bloqueado
Plan cerrado	■ Todo en modo lectura
Avance > 100%	Barra azul con el porcentaje, no como error

6. Accesibilidad y UX

- Todos los campos con `<mat-label>`; nunca solo *placeholder*.
- Navegación por teclado en orden lógico; `Enter` envía el formulario.
- Los mensajes de error del servidor se muestran **junto al campo** (`fieldErrors`).

- El botón de envío se deshabilita mientras la petición está en vuelo.
- Confirmación antes de descartar cambios sin guardar.
- Los tres estados obligatorios (cargando / vacío / error) según `00-base/05-ESTANDARES-QA.md` §7.

Tarea Backend

Detalle del plan con dos operaciones separadas: planificar y registrar producción (R-01). Calcula el consumo teórico estimado a partir de la receta de HU-12.

1. Pasos

1. `DetallePlanProduccion` (POJO) y `DetallePlanRowMapper` con `JOIN productos`, `unidades_medida` y `LEFT JOIN recetas` (para `tieneReceta`).
2. `DetallePlanRepository`: `listarPorPlan`, `insertar`, `actualizarPlanificada`, `actualizarProducida`, `eliminar`, `existeProducto(idPlan, idProducto)`, `tieneConsumos(idDetallePlan)`.
3. **Cálculo del consumo teórico** (`CalculadoraConsumoService`, reutilizable por HU-22):

```
consumoTeorico = (cantidadPlanificada / receta.cantidadProducir) × detalleReceta.cantidad
```

Con `BigDecimal`, escala 3 y `RoundingMode.HALF_UP` (`detalle_receta.cantidad` es `decimal(10,3)`, HU-12 R-06).

4. DTO: `AgregarDetallePlanRequest`, `ActualizarPlanificadaRequest`, `RegistrarProducidaRequest`, `DetallePlanResponse` (con `avance` y `consumoTeoricoEstimado`).
5. `DetallePlanService`:
 - `agregar()`: plan existe y **está PLANIFICADO**; producto existe; no duplicado → `PRODUCTO_DUPLICADO`;
 - `actualizarPlanificada()`: solo con plan `PLANIFICADO` → `PLAN_NO_EDITABLE` (R-02);
 - `registrarProducida()`: solo con plan `EN_PROCESO` → `PLAN_NO_EN_PROCESO` (R-03); **no valida un tope superior** (R-04);
 - `eliminar()`: si `tieneConsumos()` → `409 DETALLE_CON_CONSUMOS` (R-08).
6. `avance = (producida / planificada) × 100`, con `planificada > 0` garantizado por el `CHECK`.
7. `@Auditable` en las cuatro escrituras.
8. Controlador: escritura ADMIN y PRODUCCION.

2. Archivos

```
backend/src/main/java/co/edu/corposucre/productionfood/produccion/
├── {DetallePlanProduccion, DetallePlanRowMapper, DetallePlanRepository}.java
├── {DetallePlanService, DetallePlanController}.java
├── CalculadoraConsumoService.java      ← la reutiliza HU-22
└── dto/{AgregarDetallePlanRequest, ActualizarPlanificadaRequest,
    RegistrarProducidaRequest, DetallePlanResponse}.java
```

3. Puntos de cuidado

- **Endpoints separados para planificar y producir** (R-01). Un `PUT` que acepte ambos campos permite reescribir el objetivo después de conocer el resultado, y la eficiencia de HU-26 pierde todo significado.
- `CalculadoraConsumoService` **se escribe una vez aquí y la usa HU-22**. Si el cálculo se duplica, las dos versiones divergen y el consumo teórico estimado no coincidirá con el que HU-22 registre.
- **No limitar la cantidad producida** (R-04): producir de más es un dato real.
- **Registrar producción no crea el lote** (R-07). Es el hueco conocido del flujo; conviene que la interfaz lo recuerde.
- `BigDecimal` con escala 3 en el consumo teórico (HU-12 R-06).
- Verificar consumos antes de eliminar una línea (R-08).

4. Checklist de cierre

- ☐ Todos los CA implementados y verificados manualmente.
- ☐ Endpoints documentados en Swagger con ejemplos.
- ☐ `@PreAuthorize` en cada método, según la matriz de roles.
- ☐ Operaciones de escritura auditadas en bitácora (HU-04).
- ☐ Pruebas unitarias de las reglas con ramificación.

- ☐ Sin `System.out.println` ni código comentado.
- ☐ PR bajo ~250 líneas, revisado y aprobado.

Tarea Frontend

Tabla de productos del plan con avance, dentro de la pantalla de detalle de HU-20. Los controles cambian según el estado del plan.

1. Pasos

- `detalle-plan.model.ts` y su servicio.
- `detalle-plan-tabla.component`: producto, planificado, producido, avance, receta, `:`.
- Barra de avance por línea**, con color: gris < 100%, verde = 100%, azul > 100% (R-04).
- `agregar-producto-plan-dialog.component`:
 - selector **excluyendo los ya presentes** (R-05);
 - al elegir producto y cantidad, mostrar el **consumo teórico estimado**;
 - contrastar el consumo teórico con el inventario disponible** y advertir si no alcanza;
 - ⚠ avisar si el producto **no tiene receta** (R-06).
- Registrar producción en línea**, habilitado solo con el plan `EN_PROCESO` (R-03).
- Los controles de edición de lo planificado solo con el plan `PLANIFICADO` (R-02).
- Recordatorio tras registrar producción: “Registre el lote de producto terminado para que esté disponible para la venta” con enlace a HU-19 (R-07).
- Confirmación al quitar; manejar `409 DETALLE_CON_CONSUMOS` con mensaje claro.
- Ruta heredada de HU-20.

2. Archivos

```
frontend/src/app/features/produccion/  
├── models/detalle-plan.model.ts  
├── services/detalle-plan.service.ts  
└── components/{detalle-plan-tabla,agregar-producto-plan-dialog,registrar-producido-inline}/
```

3. Puntos de cuidado

- El recordatorio del lote (R-07) no es decorativo.** Sin él, se completa un plan y el producto fabricado nunca aparece como stock vendible, sin que nada lo señale. Es el hueco más fácil de sufrir de todo el flujo.
- Contrastar consumo teórico con inventario disponible** en el diálogo evita descubrir a mitad de tanda que falta harina.
- Advertir sobre los productos sin receta (R-06): quedarán fuera del análisis de HU-26.
- Mostrar avance > 100% en color distinto, no como error (R-04).
- Los controles deben responder al estado del plan: planificar con `PLANIFICADO`, producir con `EN_PROCESO`.

4. Checklist de cierre

- ☐ Todos los CA cubiertos desde la interfaz.
- ☐ Los tres estados (cargando / vacío / error) implementados.
- ☐ Errores del servidor mostrados junto al campo correspondiente.
- ☐ Guard de rol aplicado en la ruta.
- ☐ Modelos TypeScript espejo de los DTO del backend.
- ☐ Probado en Chrome y Firefox a 1366×768.
- ☐ Sin `console.log` ni `any`.

Casos de Prueba

Formato y criterios en `00-base/05-ESTANDARES-QA.md`.

1. Casos funcionales

ID	CA	Descripción	Entrada	Resultado esperado	Tipo
CP-01	CA-01..04	Agregar producto al plan	<code>{"idProducto":3,"cantidadPlanificada":90}</code>	<code>201</code> , <code>cantidadProducida: 0.00</code>	Positivo
CP-02	CA-04	cantidadProducida inicia en cero	Tras CP-01	<code>0.00</code> en la respuesta y en la base	Positivo
CP-03	CA-01	Plan inexistente	<code>POST /planes-produccion/9999/detalles</code>	<code>404 PLAN_INEXISTENTE</code>	Negativo
CP-04	CA-01	Agregar a plan EN_PROCESO	<code>POST</code> sobre uno en proceso	<code>409 PLAN_NO_EDITABLE</code>	Negativo
CP-05	CA-02	Producto inexistente	<code>{"idProducto":9999}</code>	<code>409 PRODUCTO_INEXISTENTE</code>	Negativo
CP-06	CA-03	Cantidad planificada cero	<code>{"cantidadPlanificada":0}</code>	<code>400 CANTIDAD_INVALIDA — CHECK > 0</code>	Negativo
CP-07	CA-03	Cantidad negativa	<code>{"cantidadPlanificada":-10}</code>	<code>400</code>	Negativo
CP-08	CA-04	Registrar producción con plan EN_PROCESO	<code>PATCH .../producido {"cantidadProducida":60}</code>	<code>200</code> , <code>avance: 66.67</code>	Positivo
CP-09	CA-04	No se registra producción con plan PLANIFICADO	<code>PATCH</code> sobre plan planificado	<code>409 PLAN_NO_EN_PROCESO</code> (R-03)	Negativo
CP-10	CA-04	No se registra producción con plan COMPLETADO	<code>PATCH</code> sobre plan completado	<code>409 PLAN_NO_EN_PROCESO</code>	Negativo
CP-11	CA-04	Producción cero	<code>{"cantidadProducida":0}</code>	<code>200</code> — válido, significa que no se produjo nada	Borde
CP-12	CA-04	Se puede producir más de lo planificado	Planificado 90, producido 100	<code>200</code> , <code>avance: 111.11</code> (R-04)	Borde
CP-13	—	No se edita lo planificado con el plan EN_PROCESO	<code>PUT .../detalles/21</code> en proceso	<code>409 PLAN_NO_EDITABLE</code> (R-02)	Negativo
CP-14	—	Editar lo planificado con plan PLANIFICADO	<code>PUT</code> sobre plan planificado	<code>200</code>	Positivo
CP-15	CA-05	Listar el detalle	<code>GET /planes-produccion/9/detalles</code>	Líneas con avance y consumo teórico	Positivo
CP-16	—	Producto duplicado en el plan	Agregar uno ya presente	<code>409 PRODUCTO_DUPLICADO</code> (R-05)	Negativo
CP-17	—	Duplicado por SQL directo	<code>INSERT</code> repitiendo <code>(id_plan, id_producto)</code>	La base lo rechaza — <code>uk_detalle_plan</code>	Integridad
CP-18	—	Consumo teórico correcto	Receta rinde 30 con 5 kg; planificar 90	Consumo teórico = 15,000 kg (HU-12 R-02)	Integridad
CP-19	—	Producto sin receta	Agregar uno sin receta	<code>201</code> , <code>tieneReceta: false</code> , sin consumo teórico (R-06)	Borde
CP-20	—	Registrar producción NO crea lote ni stock	Tras CP-08	<code>inventario_producto_terminado no cambia</code> (R-07)	Integridad
CP-21	—	Quitar línea sin consumos	<code>DELETE</code> de una línea limpia	<code>204</code>	Positivo
CP-22	—	Quitar líneas con consumos	<code>DELETE</code> tras registrar consumo en HU-22	<code>409 DETALLE_CON_CONSUMOS</code> (R-08)	Negativo
CP-23	—	Rol VENTAS no puede agregar	Token de VENTAS	<code>403 SIN_PERMISO</code>	Seguridad

2. Batería de seguridad

Obligatoria en **cada endpoint** de esta historia:

ID	Caso	Esperado
SEC-01	Sin cabecera <code>Authorization</code>	<code>401 NO_AUTENTICADO</code>
SEC-02	Token malformado	<code>401 NO_AUTENTICADO</code>
SEC-03	Token expirado	<code>401 NO_AUTENTICADO</code>
SEC-04	Token de rol sin permiso	<code>403 SIN_PERMISO</code>
SEC-05	Token de usuario con <code>estado = 0</code>	<code>403 USUARIO_INACTIVO</code>

SEC-04 es el caso crítico. Si devuelve `200`, falta `@EnableMethodSecurity` y la autorización del sistema completo está desactivada.

CP-09 y CP-13 protegen el sentido de HU-26. Si se puede registrar producción antes de iniciar, o editar lo planificado después de producir, la comparación planificado-vs-producido deja de medir nada: siempre se podría ajustar el objetivo al resultado.

CP-20 documenta el hueco de R-07. No es un defecto de esta historia: es el comportamiento acordado. Conviene tenerlo presente porque significa que completar un plan **no** pone el producto a la venta.

3. Verificación en base de datos

```
-- CP-02: el DEFAULT de cantidad_producida
SELECT id_detalle_plan, cantidad_planificada, cantidad_producida
FROM detalle_plan_produccion WHERE id_plan = 9;

-- CP-17: la restricción de V2 §C-13. La segunda DEBE fallar.
INSERT INTO detalle_plan_produccion (id_plan, id_producto, cantidad_planificada)
VALUES (9, 3, 50);
INSERT INTO detalle_plan_produccion (id_plan, id_producto, cantidad_planificada)
VALUES (9, 3, 40);
-- Error 1062. Sin el UNIQUE, HU-22 calcularía el consumo teórico dos veces.

-- CP-18: verificar el consumo teórico contra la receta
SELECT dpp.cantidad_planificada,
       r.cantidad_producir,
       mp.nombre AS ingrediente,
       dr.cantidad AS por_tanda,
       ROUND(dpp.cantidad_planificada / r.cantidad_producir * dr.cantidad, 3) AS teorico
FROM detalle_plan_produccion dpp
JOIN recetas r ON r.id_producto = dpp.id_producto
JOIN detalle_receta dr USING (id_receta)
JOIN materias_primas mp USING (id_materia_prima)
WHERE dpp.id_detalle_plan = 21;
-- 90 / 30 * 5,000 = 15,000 kg de harina.
-- Un resultado absurdo (450 kg) indica que cantidad_producir de la receta
-- se interpretó mal (HU-12 R-02).

-- CP-20: registrar producción no debe tocar el stock de producto terminado
SELECT COUNT(*) FROM movimientos_inventario_pt
WHERE tabla_origen = 'detalle_plan_produccion';
-- Debe ser 0; el lote se registra aparte, en HU-19 (R-07)

-- CP-12: producir de más es válido
SELECT cantidad_planificada, cantidad_producida,
       ROUND(cantidad_producida / cantidad_planificada * 100, 2) AS avance
FROM detalle_plan_produccion WHERE id_plan = 9;
```

4. Criterios de cierre

- ☐ Todos los casos ejecutados; los positivos pasan.
- ☐ Los negativos devuelven el HTTP y el `code` esperados.
- ☐ SEC-01 a SEC-05 pasan en todos los endpoints.
- ☐ Ningún endpoint supera 2 s (RNF-02).
- ☐ Cero defectos críticos o altos abiertos.
- ☐ Colección de Postman actualizada y versionada.

HU-22 · Consumo de materia prima

Especificación

Fase 7 · Fuente: `previo/Historias_de_Usuario.md`

Como productor, **quiero** registrar el consumo real de materia prima por cada producto del plan, **para que** compare con lo teórico y mida eficiencia.

Depende de: HU-21, HU-12

1. Criterios de aceptación

CA	Enunciado	Cómo se implementa
CA-01	<code>id_detalle_plan</code> debe existir	Validado; el plan debe estar <code>EN_PROCESO</code>
CA-02	<code>id_materia_prima</code> debe existir	Validado contra <code>MateriaPrimaRepository</code>
CA-03	<code>cantidad_teorica</code> obligatoria (viene de la receta)	La calcula el backend , no la envía el cliente
CA-04	<code>cantidad_real</code> obligatoria (lo que realmente se usó)	NOT NULL desde V2 §C-08 — era nullable
CA-05	<code>fecha_registro</code> se registra automáticamente	<code>DEFAULT CURRENT_TIMESTAMP</code> ya en el esquema
CA-06	Calcular la eficiencia $(teorica / real) \times 100$	⚠ Con protección contra división por cero — V2 §C-09

2. Endpoints

Método	Ruta	Descripción	Roles
GET	<code>/api/v1/planes-produccion/{idPlan}/consumos</code>	Consumos de un plan	ADMIN, PRODUCCION, CONSULTA
GET	<code>/api/v1/detalles-plan/{idDetallePlan}/consumo-teorico</code>	Consumo teórico calculado	ADMIN, PRODUCCION
POST	<code>/api/v1/detalles-plan/{idDetallePlan}/consumos</code>	Registrar consumo real · descuenta inventario (HU-23)	ADMIN, PRODUCCION
PUT	<code>/api/v1/consumos/{id}</code>	Corregir la cantidad real	ADMIN, PRODUCCION

Reglas transversales (paginación, formato de error, autenticación) en `00-base/04-CONTRATO-API.md`.

3. Reglas de negocio

R-01 · `cantidad_teorica` **la calcula el servidor, no el cliente**

CA-03 dice “viene de la receta”. Si el cliente la enviara, podría mandar cualquier número y la eficiencia de HU-26 sería una cifra inventada.

El backend la calcula con `CalculadoraConsumoService` (creado en HU-21):

```
cantidad_teorica = (cantidad_planificada / receta.cantidad_producir) * detalle_receta.cantidad
```

El cuerpo de la petición **solo trae** `idMateriaPrima` y `cantidadReal`.

R-02 · ⚠ `cantidad_teorica` **usa lo planificado, no lo producido**

Decisión que las historias no fijaban y que cambia por completo el significado del reporte.

- Contra lo **planificado**: mide desviación respecto al objetivo. Si se planificaron 90 panes y se hicieron 60, el consumo teórico de 90 panes será mucho mayor que el real y la eficiencia saldrá baja — aunque el consumo por pan haya sido perfecto.
- Contra lo **producido**: mide la eficiencia real del proceso.

Se calcula contra lo planificado, porque es lo que CA-03 dice literalmente (“viene de la receta”, y la receta se aplica sobre la cantidad planificada del detalle del plan).

HU-26 debe mostrar ambas cifras para que el dato sea interpretable: la desviación contra el plan y el rendimiento por unidad efectivamente producida. Sin esa distinción, una eficiencia del 66% no dice si se desperdició materia prima o simplemente se produjo menos.

R-03 · La fórmula de CA-06 divide por cero

$(\text{cantidad_teorica} / \text{cantidad_real}) \times 100$ revienta cuando $\text{cantidad_real} = 0$, y ese caso es perfectamente posible: se planificó un producto y no se consumió nada de ese ingrediente.

MySQL devuelve NULL en lugar de lanzar error (02-CORRECCIONES-DB.md §C-09), así que el fallo es silencioso: el reporte de HU-26 muestra huecos sin explicación.

Protección en tres capas: - SQL: `CASE WHEN cantidad_real = 0 THEN NULL ELSE ... END`; - servicio: `BigDecimal`, devolviendo `null` si el divisor es cero; - frontend: muestra `-`, nunca `NaN` ni `Infinity`.

R-04 · La fórmula de CA-06 es contraintuitiva

Con $(\text{teorica} / \text{real}) \times 100$, **consumir menos de lo teórico da más de 100%**. Un 200% no significa el doble de bueno: significa que se usó la mitad del ingrediente previsto, lo cual casi siempre indica un error de registro o una receta mal calibrada, no una hazaña.

Se implementa CA-06 tal como está escrito, **y además** se expone la métrica interpretable:

```
desviacion% = ((real - teorica) / teorica) * 100
```

Positiva = se gastó de más. Negativa = de menos. Cero = exacto.

HU-26 muestra las dos. La eficiencia porque la pide el criterio de aceptación; la desviación porque es la que se puede leer.

R-05 · Registrar consumo descuento inventario (HU-23)

Es el disparador de HU-23: al registrar, se descuenta `cantidad_real` del inventario de materia prima, en la misma transacción, con movimiento de kardex.

Las dos historias son inseparables: HU-22 es el registro, HU-23 el efecto. Se implementan juntas.

R-06 · Un ingrediente no se registra dos veces por línea de plan

`UNIQUE (id_detalle_plan, id_materia_prima)` añadido en V2 §C-13. Sin él, el inventario se descontaría dos veces por el mismo consumo.

Para corregir, se **edita** el consumo existente (`PUT /consumos/{id}`), que ajusta el inventario por diferencia.

R-07 · Corregir un consumo ajusta el inventario por diferencia

Igual criterio que HU-17 R-07: - de 15 a 18 kg → se descuentan **3** más; - de 15 a 12 kg → se **devuelven 3** al inventario.

No se revierte todo y se vuelve a descontar: el kardex debe reflejar hechos, no operaciones contables intermedias.

R-08 · Se permite registrar materia prima que no está en la receta

Puede haber consumos imprevistos. En ese caso $\text{cantidad_teorica} = 0$ y la eficiencia queda `NULL` (R-03) — con divisor válido pero dividiendo cero, la fórmula da 0%.

La interfaz lo marca como **consumo fuera de receta**, que es información valiosa: indica que la receta no refleja el proceso real.

R-09 · Precisión decimal

`cantidad_teorica` y `cantidad_real` son `decimal(10,3)` (3 decimales), pero `inventario.cantidad_disponible` es `decimal(10,2)`.

Al descontar hay que redondear, y el criterio debe ser uniforme: `HALF_UP` a 2 decimales. Un redondeo inconsistente acumula diferencias de gramos que con el tiempo descuadran el inventario sin causa aparente.

4. Modelo de datos

Tabla `consumo_materia_prima` (V2: `cantidad_real` pasa a `NOT NULL`, más `UNIQUE` y `CHECK`)

Columna	Tipo	Notas
id_consumo	int unsigned PK AI	
id_detalle_plan	int unsigned NOT NULL FK	CA-01
id_materia_prima	int unsigned NOT NULL FK	CA-02
cantidad_teorica	decimal(10,3) NOT NULL	CA-03 · calculada por el servidor (R-01)
cantidad_real	decimal(10,3) NOT NULL	CA-04 · corregido en V2 §C-08
fecha_registro	datetime NOT NULL DEFAULT CURRENT_TIMESTAMP	CA-05

UNIQUE KEY uk_consumo_mp (id_detalle_plan, id_materia_prima) — V2 §C-13.

```
// GET /api/v1/detalles-plan/21/consumo-teorico
{ "idDetallePlan": 21, "producto": "Pan francés",
  "cantidadPlanificada": 90.00, "recetaRinde": 30.00, "factor": 3.0,
  "ingredientes": [
    { "idMateriaPrima": 5, "nombre": "Harina de trigo", "unidad": "kg",
      "porTanda": 5.000, "teorico": 15.000, "disponible": 112.50 },
    { "idMateriaPrima": 7, "nombre": "Levadura", "unidad": "kg",
      "porTanda": 0.050, "teorico": 0.150, "disponible": 0.00 }
  ]
}

// POST /api/v1/detalles-plan/21/consumos
{ "idMateriaPrima": 5, "cantidadReal": 16.200 }

// 201 Created
{ "idConsumo": 31,
  "materiaPrima": { "idMateriaPrima": 5, "nombre": "Harina de trigo", "unidad": "kg" },
  "cantidadTeorica": 15.000, "cantidadReal": 16.200,
  "eficiencia": 92.59,
  "desviacionPorcentaje": 8.00,
  "fueraDeReceta": false,
  "fechaRegistro": "2026-09-18T10:22:41",
  "inventario": { "anterior": 112.50, "posterior": 96.30 }
}
```

Nótese que el cuerpo del **POST** **no incluye** **cantidadTeorica** (R-01).

$eficiencia = (15 / 16,2) \times 100 = 92,59$ · $desviacion = ((16,2 - 15) / 15) \times 100 = 8,00$. La segunda se lee sola: se gastó un 8% más de lo previsto.

5. Códigos de error

code	HTTP	Cuándo
DETALLE_PLAN_INEXISTENTE	404	El idDetallePlan no existe
PLAN_NO_EN_PROCESO	409	El plan no está EN_PROCESO
MATERIA_PRIMA_INEXISTENTE	409	La materia prima no existe
CONSUMO_DUPLICADO	409	Ese ingrediente ya tiene consumo en esta línea (R-06)
STOCK_INSUFICIENTE	409	No hay inventario suficiente (HU-23)
CANTIDAD_INVALIDA	400	cantidadReal negativa

Los transversales (**VALIDACION_FALLIDA** , **NO_AUTENTICADO** , **SIN_PERMISO** , **RECURSO_NO_ENCONTRADO**) están en **00-base/04-CONTRATO-API.md** §5.

6. Fuera de alcance

Lo que **no** cubre esta historia y no debe implementarse aquí: se limita estrictamente a los criterios de aceptación listados arriba. Cualquier funcionalidad adicional se propone como historia nueva, no se agrega en silencio.

7. Definición de Terminado

Se aplica la DoD de **00-base/01-CONVENCIONES.md** §7.

8. Notas

El caso de **cantidad_real = 0**

```
// POST con cantidadReal = 0
{ "idConsumo": 32, "cantidadTeorica": 0.150, "cantidadReal": 0.000,
  "eficiencia": null, // ← división por cero evitada (R-03)
  "desviacionPorcentaje": -100.00, // ← se consumió 100% menos de lo previsto
  "advertencia": "No se registró consumo de este ingrediente" }
```

`eficiencia: null` se muestra como `–` en la interfaz. `desviacion` sigue siendo calculable porque divide entre `cantidad_teorica`, que es distinta de cero.

Este ejemplo muestra por qué R-04 importa: la eficiencia no se puede calcular, pero la desviación dice exactamente lo que pasó.

Diseño

1. Pantallas

Tabla de consumos (dentro del plan)

CONSUMO DE MATERIA PRIMA · Pan francés (90 und planificadas) [Registrar consumo]					
MATERIA PRIMA	TEÓRICO	REAL	DESVIACIÓN	EFICIENCIA	REGISTRO
Harina de trigo	15,000 kg	16,200 kg	+8,0 %	92,6 %	10:22
Levadura	0,150 kg	0,150 kg	● 0,0 %	100,0 %	10:23
Sal	0,300 kg	0,000 kg	– 100,0 %	– ⓘ	10:23
Azúcar △ fuera de receta	0,000 kg	0,500 kg	–	0,0 %	10:25

La columna **DESVIACIÓN** es la que se lee (R-04): `+8,0%` dice “se gastó un 8% más”. La eficiencia de 92,6% para el mismo dato es correcta pero menos directa.

El `–` con ⓘ en la fila de Sal es `eficiencia: null` (R-03): no se puede calcular porque no se consumió nada. El tooltip lo explica. **Nunca debe aparecer NaN ni Infinity.**

El azúcar está marcado **fuera de receta** (R-08): se consumió algo que la receta no contempla, dato útil porque indica que la receta no refleja el proceso real.

Registrar consumo

Registrar consumo · Pan francés [X]

Planificado: 90 und · Receta rinde 30 und · Factor: x3

MATERIA PRIMA	TEÓRICO	REAL CONSUMIDO	DISPONIBLE
Harina de trigo	15,000 kg	16,200	112,50 kg ✓
Levadura	0,150 kg	0,150	0,00 kg △ sin stock
Sal	0,300 kg	0,300	8,20 kg ✓

[+ Agregar ingrediente fuera de receta]

△ Al registrar, se descontará del inventario:

Harina de trigo	–16,20 kg	112,50 → 96,30 kg
Sal	–0,30 kg	8,20 → 7,90 kg

Levadura: no hay stock suficiente (0,00 disponible)

[Cancelar] [Registrar consumo]

Los campos vienen prellenados con el teórico: en el caso normal se consume lo previsto y solo hay que corregir lo que difiere.

El recuadro de impacto (R-05) deja claro que esto **descuenta inventario**, no es un apunte. Y advierte del stock insuficiente antes de intentar la operación.

Corregir un consumo

Corregir consumo · Harina de trigo [X]

Registrado actualmente: 16,200 kg

Cantidad real corregida *

18,000

kg

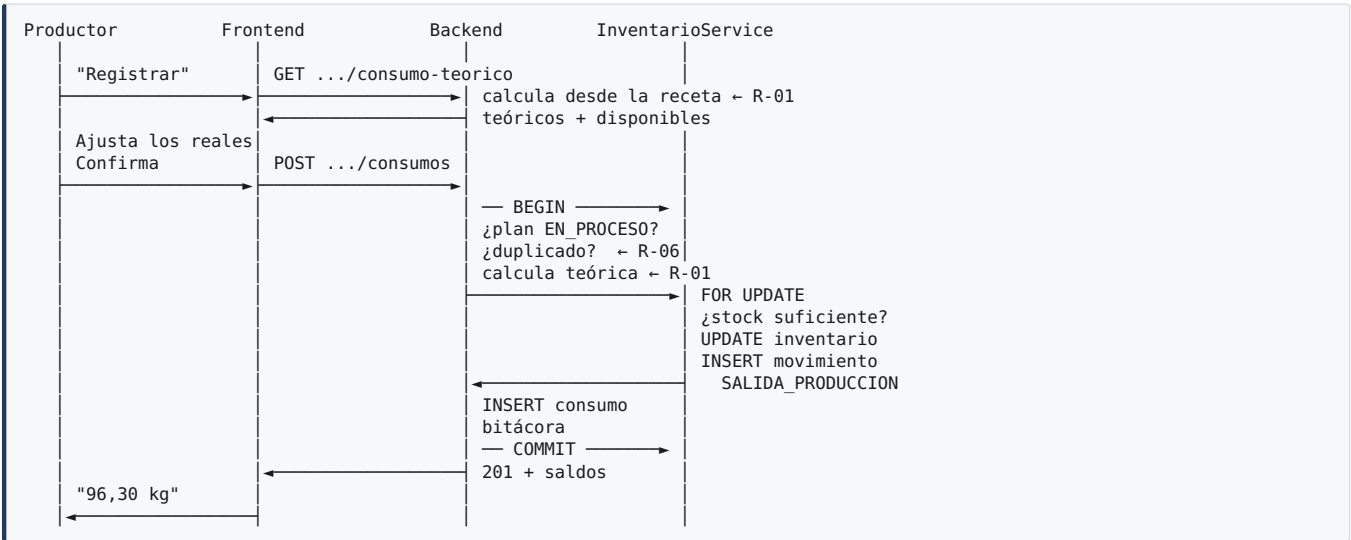
Se descontarán 1,800 kg adicionales
Inventario: 96,30 → 94,50 kg

[Cancelar]

[Corregir]

Indica el **ajuste**, no solo el valor final (R-07): es lo que hace comprensible qué va a pasar con el inventario.

2. Flujo



HU-22 y HU-23 son la misma transacción. Si el descuento falla, el consumo no se registra.

3. Componentes Angular Material

Elemento	Componente
Tabla de consumos	MatTable
Desviación	MatChip con color por signo
Eficiencia nula	Texto − + MatIcon info con MatTooltip
Fuera de receta	MatChip ámbar
Diálogo de registro	MatDialog con tabla editable
Cantidades	MatInput[type=number] con step="0.001"
Impacto en inventario	MatCard destacada, en vivo
Disponibilidad	MatIcon ✓/ por ingrediente

4. Validaciones en el formulario

Campo	Reglas	Mensaje
Materia prima	requerida, no duplicada en la línea	"Seleccione una materia prima"
Cantidad real	requerida, ≥ 0, 3 decimales	"No puede ser negativa"

Cero es válido (R-03): significa que no se consumió ese ingrediente, y es un dato legítimo. El `step` debe ser `0.001` (R-09).

⚠ `cantidadTeorica` no es un campo del formulario (R-01): la calcula el servidor.

5. Estados de la pantalla

Estado	Qué se muestra
Cargando teóricos	<code>MatProgressBar</code> en el diálogo
Producto sin receta	⚠ “Sin receta: no hay consumo teórico. Registre los ingredientes manualmente.”
Stock insuficiente	Junto al ingrediente, con el disponible; botón deshabilitado
Eficiencia nula	— con tooltip “No se puede calcular: no se registró consumo”
Consumo fuera de receta	⚠ Marcado en la tabla
Plan no <code>EN_PROCESO</code>	📖 Modo lectura
<code>409 STOCK_INSUFICIENTE</code>	Mensaje con el disponible actual
Registrando	Botón con spinner

6. Accesibilidad y UX

- Todos los campos con `<mat-label>`; nunca solo *placeholder*.
- Navegación por teclado en orden lógico; `Enter` envía el formulario.
- Los mensajes de error del servidor se muestran **junto al campo** (`fieldErrors`).
- El botón de envío se deshabilita mientras la petición está en vuelo.
- Confirmación antes de descartar cambios sin guardar.
- Los tres estados obligatorios (cargando / vacío / error) según `00-base/05-ESTANDARES-QA.md` §7.

Tarea Backend

Se implementa junto con HU-23: registrar el consumo descuento inventario en la misma transacción. Reutiliza `CalculadoraConsumoService` de HU-21 e `InventarioService` de HU-09.

1. Pasos

1. `ConsumoMateriaPrima` (POJO) y `ConsumoRowMapper` con `JOIN materias_primas` y `unidades_medida`.
2. `ConsumoRepository`: `listarPorPlan`, `listarPorDetallePlan`, `insertar`, `actualizarReal`, `buscarPorId`, `existeConsumo(idDetallePlan, idMateriaPrima)`.
3. **Reutilizar** `CalculadoraConsumoService` de HU-21 para `cantidad_teorica` (R-01). No duplicar la fórmula: si las dos versiones divergen, el teórico estimado que se muestra al planificar no coincidirá con el que se registra.
4. `CalculadoraEficienciaService`:

```
// R-03: el divisor puede ser cero
public BigDecimal eficiencia(BigDecimal teorica, BigDecimal real) {
    if (real == null || real.compareTo(BigDecimal.ZERO) == 0) return null;
    return teorica.divide(real, 4, RoundingMode.HALF_UP)
        .multiply(BigDecimal.valueOf(100))
        .setScale(2, RoundingMode.HALF_UP);
}

// R-04: la métrica interpretable
public BigDecimal desviacion(BigDecimal teorica, BigDecimal real) {
    if (teorica == null || teorica.compareTo(BigDecimal.ZERO) == 0) return null;
    return real.subtract(teorica).divide(teorica, 4, RoundingMode.HALF_UP)
        .multiply(BigDecimal.valueOf(100)).setScale(2, RoundingMode.HALF_UP);
}
```

5. DTO: `RegistrarConsumoRequest` (sin `cantidadTeorica`), `ActualizarConsumoRequest`, `ConsumoResponse`, `ConsumoTeoricoResponse`.
6. `ConsumoService.registrar()` `@Transactional` — junto con HU-23:
 - el detalle de plan existe y su plan está `EN_PROCESO`;
 - la materia prima existe;
 - no hay consumo previo de ese ingrediente → `CONSUMO_DUPLICADO` (R-06);
 - **calcula** `cantidad_teorica` con la receta (R-01); si el ingrediente no está en la receta, `0.000` y se marca `fueraDeReceta` (R-08);
 - `inventarioService.registrarSalida(...)` con `SALIDA_PRODUCCION`, redondeando a 2 decimales con `HALF_UP` (R-09);
 - `INSERT` en `consumo_materia_prima`.
7. `ConsumoService.actualizar()` con **ajuste por diferencia** (R-07).

8. `@Auditable(accion="REGISTRAR_CONSUMO")`.

2. Archivos

```
backend/src/main/java/co/edu/corposucre/productionfood/produccion/
├── {ConsumoMateriaPrima, ConsumoRowMapper, ConsumoRepository}.java
├── {ConsumoService, ConsumoController}.java
├── CalculadoraEficienciaService.java ← la reutiliza HU-26
└── dto/{RegistrarConsumoRequest, ActualizarConsumoRequest,
    ConsumoResponse, ConsumoTeoricoResponse}.java
```

3. Puntos de cuidado

- **cantidadTeorica nunca llega del cliente** (R-01). Un DTO que la acepte permite falsear la eficiencia con solo editar el JSON.
- **Proteger la división por cero** (R-03). MySQL devuelve **NULL** sin avisar: el reporte de HU-26 se llena de huecos y nadie sabe por qué.
- **Redondeo uniforme a 2 decimales al descontar** (R-09): el consumo tiene 3 decimales y el inventario 2. Un criterio inconsistente acumula gramos y descuadra el inventario sin causa aparente.
- **Ajuste por diferencia al corregir** (R-07), no revertir y volver a descontar.
- **Reutilizar la calculadora de HU-21**, no reescribir la fórmula.
- El **UNIQUE** de V2 evita el doble descuento (R-06).
- Exponer **desviacion** además de **eficiencia** (R-04): es la cifra que se puede leer.

4. Checklist de cierre

- ☐ Todos los CA implementados y verificados manualmente.
- ☐ Endpoints documentados en Swagger con ejemplos.
- ☐ `@PreAuthorize` en cada método, según la matriz de roles.
- ☐ Operaciones de escritura auditadas en bitácora (HU-04).
- ☐ Pruebas unitarias de las reglas con ramificación.
- ☐ Sin `System.out.println` ni código comentado.
- ☐ PR bajo ~250 líneas, revisado y aprobado.
- ☐ **Pruebas unitarias de la eficiencia: real = 0, teórica = 0, ambos = 0, valores normales.**
- ☐ **Verificado: el redondeo de 3 a 2 decimales no descuadra el inventario.**

Tarea Frontend

Tabla de consumos dentro del plan, con teórico prellenado y comparativa visible. La pantalla debe advertir que registrar consumo **descuenta inventario**.

1. Pasos

1. `consumo.model.ts` y `consumo.service.ts`.
2. `consumo-tabla.component` dentro del detalle del plan (HU-20): materia prima, teórico, real, desviación, eficiencia, fecha.
3. **Prellenar el campo real con el teórico**: en el caso normal se consume lo previsto, y así solo se corrige lo que difiere.
4. Mostrar **desviación y eficiencia** juntas (R-04), con la desviación destacada por ser la legible: **+8,0%** en rojo, **-5,0%** en azul, **0%** en verde.
5. **Mostrar – cuando la eficiencia sea null** (R-03), nunca **NaN** ni **Infinity**, con tooltip: “No se puede calcular: no se registró consumo”.
6. `registrar-consumo-dialog.component`:
 - lista los ingredientes de la receta con su teórico y el disponible en inventario;
 - permite registrar **todos de una vez** o uno a uno;
 - **advierte el impacto en el inventario** antes de confirmar;
 - permite agregar un ingrediente fuera de receta, marcándolo (R-08).
7. Marcar visualmente los consumos **fuera de receta**.
8. Al corregir, indicar el ajuste: “Se descontarán 3,000 kg adicionales” (R-07).
9. Solo habilitado con el plan **EN_PROCESO**.

2. Archivos

```
frontend/src/app/features/produccion/  
├── models/consumo.model.ts  
├── services/consumo.service.ts  
└── components/{consumo-tabla, registrar-consumo-dialog, indicador-desviacion}/
```

3. Puntos de cuidado

- **eficiencia: null se muestra como -** (R-03). Un `NaN` o un `Infinity` en pantalla es un defecto visible de inmediato; peor es un `0%` inventado por el cliente para disimularlo.
- **La desviación es la cifra que la gente entiende** (R-04). Una eficiencia del 200% se malinterpreta como algo bueno; un “-50% de consumo” se lee solo.
- **Prellenar con el teórico** reduce el trabajo y hace que la diferencia salte a la vista.
- **Advertir el impacto en inventario** antes de confirmar: es una operación que descuenta stock (R-05), no un simple apunte.
- Mostrar el disponible junto a cada teórico evita intentar consumir lo que no hay.

4. Checklist de cierre

- ☐ Todos los CA cubiertos desde la interfaz.
- ☐ Los tres estados (cargando / vacío / error) implementados.
- ☐ Errores del servidor mostrados junto al campo correspondiente.
- ☐ Guard de rol aplicado en la ruta.
- ☐ Modelos TypeScript espejo de los DTO del backend.
- ☐ Probado en Chrome y Firefox a 1366×768.
- ☐ Sin `console.log` ni `any`.

Casos de Prueba

Formato y criterios en `00-base/05-ESTANDARES-QA.md`.

1. Casos funcionales

ID	CA	Descripción	Entrada	Resultado esperado	Tipo
CP-01	CA-01..05	Registrar consumo válido	<code>{"idMateriaPrima":5,"cantidadReal":16.200}</code>	201 con teórica, real, eficiencia y desviación	Positivo
CP-02	CA-03	La teórica la calcula el servidor	Enviar <code>{"cantidadTeorica":999}</code> en el cuerpo	Se ignora; se usa la de la receta (R-01)	Seguridad
CP-03	CA-03	Cálculo de la teórica	Receta rinde 30 con 5 kg; planificado 90	<code>cantidadTeorica: 15.000</code>	Integridad
CP-04	CA-04	Cantidad real omitida	Sin <code>cantidadReal</code>	400 VALIDACION_FALLIDA	Negativo
CP-05	CA-04	<code>cantidad_real</code> no admite NULL	INSERT directo con <code>NULL</code>	La base lo rechaza — NOT NULL desde V2 §C-08	Integridad
CP-06	CA-04	Cantidad real negativa	<code>{"cantidadReal":-5}</code>	400 CANTIDAD_INVALIDA	Negativo
CP-07	CA-05	La fecha se asigna sola	Crear sin fecha	<code>fechaRegistro</code> con la hora del servidor	Positivo
CP-08	CA-06	Eficiencia normal	teórica 15, real 16,2	<code>eficiencia: 92.59</code>	Positivo
CP-09	CA-06	División por cero	teórica 15, real 0	<code>eficiencia: null</code> , no error ni <code>Infinity</code> (R-03)	Integridad
CP-10	CA-06	Teórica cero, real positiva	teórica 0, real 5	<code>eficiencia: 0</code> , <code>desviacion: null</code> (R-08)	Borde
CP-11	CA-06	Ambas cero	teórica 0, real 0	<code>eficiencia: null</code> , <code>desviacion: null</code>	Borde
CP-12	CA-06	Eficiencia mayor que 100%	teórica 15, real 7,5	<code>eficiencia: 200.00</code> , <code>desviacion: -50.00</code> (R-04)	Borde
CP-13	CA-06	Cálculo de la desviación	teórica 15, real 16,2	<code>desviacionPorcentaje: 8.00</code>	Positivo
CP-14	CA-01	Plan no EN_PROCESO	Registrar con plan <code>PLANIFICADO</code>	409 PLAN_NO_EN_PROCESO	Negativo
CP-15	CA-02	Materia prima inexistente	<code>{"idMateriaPrima":9999}</code>	409 MATERIA_PRIMA_INEXISTENTE	Negativo
CP-16	—	Consumo duplicado	Registrar dos veces el mismo ingrediente	409 CONSUMO_DUPLICADO (R-06)	Negativo
CP-17	—	Duplicado por SQL directo	INSERT repitiendo <code>(id_detalle_plan, id_materia_prima)</code>	La base lo rechaza — <code>uk_consumo_mp</code>	Integridad
CP-18	—	Ingrediente fuera de receta	Registrar una MP que no está en la receta	201, <code>cantidadTeorica: 0.000</code> , <code>fueraDeReceta: true</code> (R-08)	Borde
CP-19	—	El inventario se descuenta (HU-23)	Tras CP-01	El inventario baja 16,20 kg	Integridad
CP-20	—	Redondeo de 3 a 2 decimales	<code>cantidadReal: 16.205</code>	El inventario baja <code>16.21</code> (HALF_UP, R-09)	Integridad
CP-21	—	Stock insuficiente	Consumir más de lo disponible	409 STOCK_INSUFICIENTE ; no se registra el consumo	Negativo
CP-22	—	Corregir ajusta por diferencia	De 15 a 18 kg	Se descuentan 3 más, no 18 (R-07)	Integridad
CP-23	—	Corregir hacia abajo devuelve	De 15 a 12 kg	Se devuelven 3 al inventario	Integridad
CP-24	—	Rol VENTAS no puede registrar consumos	Token de VENTAS	403 SIN_PERMISO	Seguridad

2. Batería de seguridad

Obligatoria en **cada endpoint** de esta historia:

ID	Caso	Esperado
SEC-01	Sin cabecera <code>Authorization</code>	401 NO_AUTENTICADO
SEC-02	Token malformado	401 NO_AUTENTICADO
SEC-03	Token expirado	401 NO_AUTENTICADO
SEC-04	Token de rol sin permiso	403 SIN_PERMISO
SEC-05	Token de usuario con <code>estado = 0</code>	403 USUARIO_INACTIVO

SEC-04 es el caso crítico. Si devuelve `200`, falta `@EnableMethodSecurity` y la autorización del sistema completo está desactivada.

CP-02, CP-09 y CP-20 son los casos críticos.

CP-02: si el DTO acepta `cantidadTeorica`, cualquiera puede editar el JSON y fabricar una eficiencia del 100%. El reporte de HU-26 deja de medir nada.

CP-09: la división por cero no lanza error en MySQL, devuelve `NULL`. El reporte se llena de huecos y la causa es invisible desde la interfaz.

CP-20: el consumo tiene 3 decimales y el inventario 2. Un redondeo inconsistente acumula gramos en cada producción, y a los dos meses el inventario no cuadra con la bodega sin que ningún movimiento individual parezca incorrecto.

3. Verificación en base de datos

```
-- CP-09 a CP-12: la división por cero de §C-09
SELECT c.id_consumo, c.cantidad_teorica, c.cantidad_real,
       CASE WHEN c.cantidad_real = 0 THEN NULL
       ELSE ROUND(c.cantidad_teorica / c.cantidad_real * 100, 2) END AS eficiencia,
       CASE WHEN c.cantidad_teorica = 0 THEN NULL
       ELSE ROUND((c.cantidad_real - c.cantidad_teorica) / c.cantidad_teorica * 100, 2)
       END AS desviacion
FROM consumo_materia_prima c WHERE c.id_detalle_plan = 21;
-- Sin el CASE, MySQL devuelve NULL en silencio al dividir por cero.
-- Verificar que la API devuelve null y no 0, Infinity o un error.

-- CP-03: la teórica debe coincidir con la receta
SELECT dpp.cantidad_planificada, r.cantidad_producir, dr.cantidad AS por_tanda,
       ROUND(dpp.cantidad_planificada / r.cantidad_producir * dr.cantidad, 3) AS teorica_esperada,
       c.cantidad_teorica AS teorica_registrada
FROM consumo_materia_prima c
JOIN detalle_plan_produccion dpp USING (id_detalle_plan)
JOIN recetas r ON r.id_producto = dpp.id_producto
JOIN detalle_receta dr ON dr.id_receta = r.id_receta
AND dr.id_materia_prima = c.id_materia_prima
WHERE c.id_consumo = 31;
-- teorica_esperada y teorica_registrada DEBEN coincidir

-- CP-05: la corrección de V2 §C-08
SHOW CREATE TABLE consumo_materia_prima\G
-- `cantidad_real` decimal(10,3) NOT NULL
-- UNIQUE KEY `uk_consumo_mp` (`id_detalle_plan`,`id_materia_prima`)

-- CP-19 y CP-20: el descuento y su redondeo
SELECT tipo_movimiento, cantidad, saldo_anterior, saldo_posterior
FROM movimientos_inventario_mp
WHERE tabla_origen = 'consumo_materia_prima' ORDER BY id_movimiento DESC LIMIT 3;
-- SALIDA_PRODUCCION. Con cantidadReal = 16.205, el descuento debe ser 16.21

-- CP-22: el ajuste por diferencia deja UN movimiento
SELECT tipo_movimiento, cantidad, fecha FROM movimientos_inventario_mp
WHERE tabla_origen = 'consumo_materia_prima' AND id_origen = 31
ORDER BY id_movimiento;
-- Al corregir de 15 a 18: un SALIDA_PRODUCCION de 3.
-- Si aparece ENTRADA de 15 + SALIDA de 18, se implementó revertir-y-redescontar.

-- Conciliación (05-ESTANDARES-QA.md §5): 0 filas
```

4. Criterios de cierre

- ☐ Todos los casos ejecutados; los positivos pasan.
- ☐ Los negativos devuelven el HTTP y el `code` esperados.
- ☐ SEC-01 a SEC-05 pasan en todos los endpoints.
- ☐ Ningún endpoint supera 2 s (RNF-02).
- ☐ Cero defectos críticos o altos abiertos.
- ☐ Colección de Postman actualizada y versionada.

HU-23 · Inventario automático por producción

Especificación

Fase 7 · Fuente: `previo/Historias_de_Usuario.md`

Como administrador, **quiero** que al registrar consumo de materia prima se actualice automáticamente el inventario, **para** que los datos de stock sean consistentes.

Depende de: HU-22, HU-10

1. Criterios de aceptación

CA	Enunciado	Cómo se implementa
CA-01	Al registrar un <code>consumo_materia_prima</code> , se descuenta <code>cantidad_real</code> del <code>inventario</code>	<code>InventarioService.registrarSalida()</code> en la misma transacción
CA-02	Se actualiza la <code>fecha_actualizacion</code> en <code>inventario</code>	<code>ON UPDATE CURRENT_TIMESTAMP</code> añadido en V2 §C-07 — se actualiza sola
CA-03	Si el stock queda por debajo del <code>stock_minimo</code> , se genera una alerta (flag en la respuesta)	Campo <code>alertas[]</code> en la respuesta de HU-22
CA-04	La operación es transaccional (si falla, se revierte todo)	<code>@Transactional</code> sobre el método de servicio completo

2. Endpoints

Método	Ruta	Descripción	Roles
POST	<code>/api/v1/detalles-plan/{id}/consumos</code>	(HU-22) Registrar consumo · este es el disparador	ADMIN, PRODUCCION
PUT	<code>/api/v1/consumos/{id}</code>	(HU-22) Corregir consumo · ajusta el inventario	ADMIN, PRODUCCION
GET	<code>/api/v1/inventario/alertas</code>	Materias primas bajo mínimo tras la última producción	ADMIN, PRODUCCION, COMPRAS

Reglas transversales (paginación, formato de error, autenticación) en `00-base/04-CONTRATO-API.md`.

3. Reglas de negocio

R-01 · Esta historia no tiene endpoints propios

Es el **comportamiento** de HU-22, no una funcionalidad separada. Se documenta aparte porque concentra las garantías de consistencia del inventario, que son transversales.

Se implementa junto con HU-22, en la misma transacción y el mismo PR.

R-02 · La transaccionalidad es el corazón de la historia (CA-04)

El registro de consumo hace cinco cosas que deben ocurrir todas o ninguna:

- `SELECT ... FOR UPDATE` sobre la fila de inventario;
- validar stock suficiente;
- `UPDATE inventario` restando `cantidad_real`;
- `INSERT` en `movimientos_inventario_mp` con tipo `SALIDA_PRODUCCION`;
- `INSERT` en `consumo_materia_prima`.

Un fallo en el paso 5 debe revertir los pasos 3 y 4. Si no, el inventario baja pero el consumo no queda registrado: la materia prima desaparece del sistema sin justificación, y el kardex apunta a un consumo inexistente.

Con **varios ingredientes en una sola petición**, la transacción envuelve todos: si el tercero no tiene stock, los dos primeros **no** quedan descontados. Una producción a medias deja un inventario que nadie puede reconstruir.

R-03 · `SELECT ... FOR UPDATE` no es opcional

Dos producciones simultáneas que consuman la misma materia prima leen el mismo saldo, ambas validan contra él y ambas descuentan. El resultado es un saldo que no corresponde a ninguna de las dos operaciones.

El bloqueo se toma **antes** de validar y se libera al cerrar la transacción. El `CHECK (cantidad_disponible >= 0)` de V2 es la última red: si llega a dispararse, el bloqueo no está funcionando.

R-04 · La alerta de stock bajo es informativa, no bloqueante (CA-03)

Si tras el descuento `cantidad_disponible < stock_minimo`, la respuesta incluye una alerta. **No impide la operación**: la materia prima ya se consumió físicamente, negarse a registrarlo solo haría que el sistema se alejara de la realidad.

```
"alertas": [
  { "idMateriaPrima": 5, "nombre": "Harina de trigo",
    "cantidadDisponible": 46.30, "stockMinimo": 50.00, "faltante": 3.70,
    "nivel": "BAJO" }
]
```

Niveles: `BAJO` (por debajo del mínimo) y `AGOTADO` (en cero). Alimenta HU-27 y el dashboard de HU-24.

R-05 · Stock insuficiente: se rechaza el registro

Si `cantidad_real > cantidad_disponible` → `409 STOCK_INSUFICIENTE`, y **no se registra nada**.

Es una decisión discutible: físicamente, si el productor usó 20 kg, los usó, aunque el sistema creyera que había 15. Pero permitir el registro dejaría el inventario en negativo, que es un estado sin significado y que el `CHECK` de la base rechazaría de todos modos.

La discrepancia se resuelve con un ajuste de inventario explícito —reconociendo que había más de lo registrado— y luego se registra el consumo. Así queda constancia de que el inventario estaba mal, en lugar de disimularlo.

R-06 · `fecha_actualizacion` se mantiene sola (CA-02)

`DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP`, añadido en V2 §C-07. El servicio **no** la asigna: la base lo hace en cada `UPDATE`.

Una fecha que dependa de que el programador se acuerde en cada ruta de código acaba desactualizada justo en la ruta que alguien olvidó.

R-07 · Toda salida pasa por `InventarioService`

Ningún servicio ejecuta `UPDATE inventario` directamente. La regla se establece en HU-09 y esta historia es su principal consumidora.

Es lo que garantiza que **el kardex nunca se desincronice del saldo**: el movimiento y la actualización viven en el mismo método, y no hay forma de hacer uno sin el otro.

R-08 · Redondeo de 3 a 2 decimales

`consumo_materia_prima.cantidad_real` es `decimal(10,3)`; `inventario.cantidad_disponible` es `decimal(10,2)`.

El descuento redondea con `HALF_UP` a 2 decimales, siempre igual. Un criterio inconsistente acumula gramos en cada producción y descuadra el inventario a las semanas, sin que ningún movimiento individual parezca incorrecto.

4. Modelo de datos

No introduce tablas nuevas. Coordina tres:

Tabla	Operación
<code>consumo_materia_prima</code>	<code>INSERT</code> (HU-22)
<code>inventario</code>	<code>UPDATE</code> <code>cantidad_disponible</code> · <code>fecha_actualizacion</code> automática
<code>movimientos_inventario_mp</code>	<code>INSERT</code> con <code>SALIDA_PRODUCCION</code> (kardex, V2 §C-04)

```
// POST /api/v1/detalles-plan/21/consumos (varios ingredientes a la vez)
{
  "consumos": [
    { "idMateriaPrima": 5, "cantidadReal": 16.200 },
    { "idMateriaPrima": 7, "cantidadReal": 0.150 },
    { "idMateriaPrima": 9, "cantidadReal": 0.300 }
  ]
}

// 201 Created
{
  "consumosRegistrados": 3,
  "movimientos": [
    { "materiaPrima": "Harina de trigo", "descontado": 16.20,
      "saldoAnterior": 112.50, "saldoPosterior": 96.30 },
    { "materiaPrima": "Levadura", "descontado": 0.15,
      "saldoAnterior": 2.00, "saldoPosterior": 1.85 },
    { "materiaPrima": "Sal", "descontado": 0.30,
      "saldoAnterior": 8.20, "saldoPosterior": 7.90 }
  ],
  "alertas": [
    { "idMateriaPrima": 7, "nombre": "Levadura",
      "cantidadDisponible": 1.85, "stockMinimo": 5.00,
      "faltante": 3.15, "nivel": "BAJO" }
  ]
}

// 409 STOCK_INSUFICIENTE — nada se registró
{ "code": "STOCK_INSUFICIENTE",
  "message": "No hay suficiente Levadura. Disponible: 0,15 kg. Requerido: 2,00 kg.",
  "detalle": { "idMateriaPrima": 7, "disponible": 0.150, "requerido": 2.000 } }
```

5. Códigos de error

code	HTTP	Cuándo
STOCK_INSUFICIENTE	409	La cantidad real supera el inventario disponible (R-05)
INVENTARIO_NO_INICIALIZADO	500	La materia prima no tiene fila en inventario — defecto de HU-08 R-01

Los transversales (**VALIDACION_FALLIDA** , **NO_AUTENTICADO** , **SIN_PERMISO** , **RECURSO_NO_ENCONTRADO**) están en **00-base/04-CONTRATO-API.md** §5.

6. Fuera de alcance

Lo que **no** cubre esta historia y no debe implementarse aquí: se limita estrictamente a los criterios de aceptación listados arriba. Cualquier funcionalidad adicional se propone como historia nueva, no se agrega en silencio.

7. Definición de Terminado

Se aplica la DoD de **00-base/01-CONVENCIONES.md** §7.

8. Notas

Esta historia es donde se comprueba si el inventario significa algo

Es la contrapartida de la recepción de compra (HU-13 R-01): allí entra el stock, aquí sale. Entre las dos sostienen todo el control de materias primas.

La regla de conciliación de **00-base/05-ESTANDARES-QA.md** §5 —el **saldo_posterior** del último movimiento debe ser igual a **cantidad_disponible** — es la prueba de que ambas funcionan. En cuanto devuelve una fila, alguna ruta de código actualizó el saldo sin registrar el movimiento, y el inventario deja de ser explicable.

Diseño

1. Pantallas

Esta historia no tiene pantallas propias

Su interfaz es la de HU-22 más la presentación de las alertas. Lo que aporta visualmente es hacer **evidente el efecto sobre el inventario**.

Impacto antes de confirmar (en el diálogo de HU-22)

⚠ Al registrar, se descontará del inventario:

Harina de trigo	-16,20 kg	112,50 →	96,30 kg	✓
Levadura	-0,15 kg	2,00 →	1,85 kg	●
Sal	-0,30 kg	8,20 →	7,90 kg	✓

● Levadura quedará por debajo del mínimo (5,00 kg)

[Cancelar]

[Registrar consumo]

La advertencia de stock bajo aparece **antes**, pero **no bloquea** (R-04): la materia prima ya se usó físicamente.

Resultado y alertas

✓ Consumo registrado

Inventario actualizado:

Harina de trigo	112,50 →	96,30 kg
Levadura	2,00 →	1,85 kg
Sal	8,20 →	7,90 kg

● ALERTA DE STOCK

Levadura quedó en 1,85 kg (mínimo 5,00 kg)
Faltan 3,15 kg para alcanzar el nivel mínimo.

[Ver alertas de stock]

[Registrar compra]

Las acciones sugeridas conectan con HU-27 y HU-13: la alerta no es solo información, es el punto donde empieza la reposición.

Stock insuficiente

No se pudo registrar el consumo

No hay suficiente Levadura:

Disponible:	0,15 kg
Requerido:	2,00 kg
Faltan:	1,85 kg

i Ningún consumo de esta operación fue registrado.

Si la materia prima sí estaba disponible físicamente, el inventario del sistema está desactualizado. Registre un ajuste de inventario antes de continuar.

[Entendido]

Dos cosas importan en este mensaje:

1. **“Ningún consumo fue registrado”** confirma el rollback (R-02): el usuario sabe que no quedó nada a medias.

2. **La sugerencia del ajuste** (R-05) le da una salida cuando la materia prima sí estaba allí. Sin ella, el productor queda bloqueado ante un sistema que le dice que no hay algo que tiene delante.

Indicador global de alertas

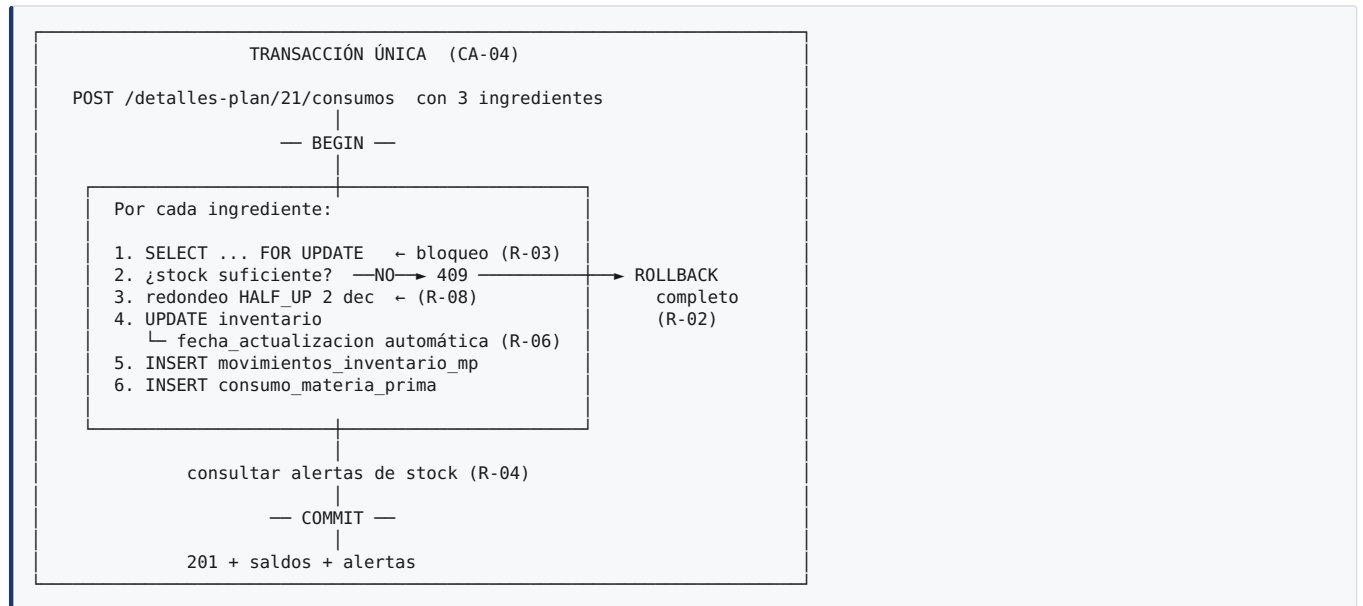
☰ ProductionFood

● 2 alertas

Ismael B. · PRODUCCION [🔔]

Visible desde cualquier pantalla para ADMIN, PRODUCCION y COMPRAS. Al pulsarlo lleva a HU-27.

2. Flujo



Un fallo en cualquier punto revierte todo. Es la garantía que hace que el inventario signifique algo: no existe el estado intermedio de “dos ingredientes descontados y el consumo sin registrar”.

3. Componentes Angular Material

Elemento	Componente
Impacto previsto	MatCard en el diálogo de HU-22
Saldos resultantes	MatList en la confirmación
Alerta de stock	MatCard ámbar/roja con acciones
Indicador global	MatBadge sobre MatIcon en MatToolbar
Error de stock	MatDialog con las tres cifras y la sugerencia
Nivel de alerta	MatChip — ámbar BAJO , rojo AGOTADO

4. Validaciones en el formulario

Esta historia **no aporta formularios propios**. Las validaciones son las de HU-22.

Lo que sí define es el **contenido de los mensajes**:

Situación	Qué debe decir
Stock insuficiente	Disponible, requerido, faltante, y que nada se registró
Alerta de stock bajo	Cantidad actual, mínimo, faltante, y qué hacer
Inventario no inicializado	Que es un problema de datos, no del usuario

Un mensaje que solo diga “stock insuficiente” deja al productor sin saber cuánto falta ni qué puede hacer.

5. Estados de la pantalla

Estado	Qué se muestra
Antes de confirmar	Impacto previsto con semáforo por ingrediente
Registrando	Botón con spinner; diálogo bloqueado
Registrado sin alertas	✓ Confirmación con los saldos resultantes
Registrado con alertas	✓ + ● tarjeta de alerta con acciones sugeridas
Stock insuficiente	Diálogo con cifras, confirmación de rollback y salida sugerida
Inventario no inicializado	“Esta materia prima no tiene inventario inicializado. Contacte al administrador.”

El caso de inventario no inicializado (R-01 de HU-08) no es culpa de quien registra: el mensaje debe dejarlo claro y dirigirlo a quien puede resolverlo.

6. Accesibilidad y UX

- Todos los campos con `<mat-label>`; nunca solo *placeholder*.
- Navegación por teclado en orden lógico; `Enter` envía el formulario.
- Los mensajes de error del servidor se muestran **junto al campo** (`fieldErrors`).
- El botón de envío se deshabilita mientras la petición está en vuelo.
- Confirmación antes de descartar cambios sin guardar.
- Los tres estados obligatorios (cargando / vacío / error) según `00-base/05-ESTANDARES-QA.md` §7.

Tarea Backend

No es una historia independiente: es el comportamiento transaccional de HU-22. Se implementa en el mismo PR. El foco está en las garantías de consistencia.

1. Pasos

Lo que se implementa dentro de `ConsumoService.registrar()`

1. Anotar el método con `@Transactional` — sin `readOnly`, sin propagación especial.
2. Aceptar **varios consumos en una sola petición** y envolverlos todos en la misma transacción (R-02).
3. Por cada consumo, en orden:
 - `inventarioRepository.bloquearYObtener(idMateriaPrima)` → `SELECT ... FOR UPDATE` (R-03);
 - si no existe la fila → `500 INVENTARIO_NO_INICIALIZADO` (defecto de HU-08 R-01);
 - validar `cantidadReal <= disponible` → `409 STOCK_INSUFICIENTE` (R-05);
 - `inventarioService.registrarSalida(idMateriaPrima, cantidadRedondeada, SALIDA_PRODUCCION, "consumo_materia_prima", idConsumo)`;
 - **redondear a 2 decimales con `HALF_UP`** antes de descontar (R-08).
4. **No asignar `fecha_actualizacion`**: la pone el `ON UPDATE` de V2 (R-06).
5. Tras completar los descuentos, consultar `v_inventario_materia_prima` para las materias primas afectadas y **construir el array de alertas** (R-04). Informativo, nunca bloquea.
6. Devolver los saldos anterior y posterior de cada movimiento.

Verificaciones que deben quedar escritas

7. Prueba de integración: consumo de 3 ingredientes con el tercero sin stock → **ningún descuento aplicado**.
8. Prueba de concurrencia: dos peticiones simultáneas sobre la misma materia prima → los saldos finales cuadran y no hay negativos.
9. Endpoint de conciliación (perfil `dev`) que ejecute la consulta de `00-base/05-ESTANDARES-QA.md` §5.

2. Archivos

```
backend/src/main/java/co/edu/corposucre/productionfood/
├── produccion/ConsumoService.java      ← @Transactional, orquesta todo
├── inventario/
│   ├── InventarioService.java          (registrarSalida – ya existe desde HU-09)
│   ├── InventarioRepository.java       (+ bloquearYObtener con FOR UPDATE)
│   └── dto/{AlertaStockResponse,MovimientoAplicadoResponse}.java
```

3. Puntos de cuidado

- `@Transactional` **sobre el método completo** (R-02), no sobre cada consumo por separado. Con varios ingredientes, un fallo en el tercero debe revertir los dos primeros.
- `SELECT ... FOR UPDATE` **antes de validar** (R-03). Validar y descontar en operaciones separadas permite que dos producciones consuman el mismo saldo.
- **No asignar `fecha_actualizacion` desde Java** (R-06): la pone la base. Asignarla a mano introduce una segunda fuente de verdad que se desincroniza.
- **Redondeo `HALF_UP` a 2 decimales, siempre igual** (R-08). Es el error que no se ve: cada producción pierde miligramos y a las semanas el inventario no cuadra con la bodega.
- **La alerta no bloquea** (R-04): la materia prima ya se consumió.
- **Todo pasa por `InventarioService`** (R-07). Si aparece un `UPDATE inventario` en este servicio, es un defecto.
- El `CHECK (cantidad_disponible >= 0)` es la última red, no la primera: si salta, hay un fallo en el bloqueo o en la validación.

4. Checklist de cierre

- ☐ Todos los CA implementados y verificados manualmente.
- ☐ Endpoints documentados en Swagger con ejemplos.
- ☐ `@PreAuthorize` en cada método, según la matriz de roles.
- ☐ Operaciones de escritura auditadas en bitácora (HU-04).
- ☐ Pruebas unitarias de las reglas con ramificación.
- ☐ Sin `System.out.println` ni código comentado.
- ☐ PR bajo ~250 líneas, revisado y aprobado.
- ☐ **Prueba de rollback: fallo en el tercer ingrediente no deja descuentos aplicados.**
- ☐ **Prueba de concurrencia: dos peticiones simultáneas, sin saldos negativos.**
- ☐ **Conciliación kardex ↔ saldo: 0 filas.**

Tarea Frontend

Sin pantallas propias: son las de HU-22 más la presentación de las alertas de stock. El foco está en que el usuario **vea el efecto sobre el inventario**.

1. Pasos

- En el diálogo de registro de consumo (HU-22), mostrar el **impacto previsto** sobre cada materia prima antes de confirmar.
- Tras registrar, mostrar los **saldos resultantes** de cada ingrediente.
- `alertas-stock.component`: banner con las materias primas que quedaron bajo mínimo tras la operación (R-04), con enlace a HU-27 y a crear una compra (HU-13).
- Distinguir visualmente **BAJO** (ámbar) de **AGOTADO** (rojo).
- Manejar **409 STOCK INSUFICIENTE** mostrando **disponible y requerido**, y sugiriendo el ajuste de inventario como camino (R-05).
- Indicador global de alertas en la barra superior, visible desde cualquier pantalla para ADMIN, PRODUCCION y COMPRAS.
- Si se produce `INVENTARIO_NO_INICIALIZADO`, mensaje explícito: es un defecto de datos, no un error del usuario.

2. Archivos

```
frontend/src/app/  
├── features/produccion/components/impacto-inventario/  
├── shared/components/alertas-stock/  
└── layout/components/indicador-alertas/
```

3. Puntos de cuidado

- **Mostrar el impacto antes y los saldos después** cierra el ciclo: el usuario entiende que registrar consumo mueve inventario, en lugar de verlo como un formulario más.
- **El mensaje del 409 debe traer las dos cifras** (disponible y requerido) y sugerir qué hacer. Un “stock insuficiente” a secas deja al productor sin salida cuando la materia prima sí estaba físicamente.
- La alerta de stock bajo **no debe bloquear ni interrumpir**: es información posterior a una operación ya consumada (R-04).
- El indicador global conecta esta historia con HU-27 y con la decisión de comprar.

4. Checklist de cierre

- ☐ Todos los CA cubiertos desde la interfaz.
- ☐ Los tres estados (cargando / vacío / error) implementados.
- ☐ Errores del servidor mostrados junto al campo correspondiente.
- ☐ Guard de rol aplicado en la ruta.
- ☐ Modelos TypeScript espejo de los DTO del backend.
- ☐ Probado en Chrome y Firefox a 1366×768.
- ☐ Sin `console.log` ni `any`.

Casos de Prueba

Formato y criterios en `00-base/05-ESTANDARES-QA.md`.

1. Casos funcionales

ID	CA	Descripción	Entrada	Resultado esperado	Tipo
CP-01	CA-01	El consumo descuenta el inventario	Consumir 16,2 kg de un saldo de 112,50	Saldo posterior 96,30	Integridad
CP-02	CA-01	Se registra el movimiento de kardex	Tras CP-01	<code>SALIDA_PRODUCCION</code> con origen <code>consumo_materia_prima</code>	Integridad
CP-03	CA-01	Descuento con varios ingredientes	3 consumos en una petición	Los 3 inventarios bajan correctamente	Integridad
CP-04	CA-02	fecha_actualizacion cambia sola	Anotar la fecha, consumir, volver a consultar	La fecha es posterior (R-06)	Integridad
CP-05	CA-02	La fecha la pone la base, no la aplicación	Revisar <code>SHOW CREATE TABLE</code>	<code>ON UPDATE CURRENT_TIMESTAMP</code> presente	Integridad
CP-06	CA-03	Alerta al quedar bajo mínimo	Consumo que deja 46,30 con mínimo 50	<code>alertas[]</code> con <code>nivel: "BAJO"</code> y <code>faltante: 3.70</code>	Positivo
CP-07	CA-03	Alerta al agotarse	Consumo que deja el saldo en cero	<code>nivel: "AGOTADO"</code>	Borde
CP-08	CA-03	Sin alerta si queda por encima	Consumo que deja 96,30 con mínimo 50	<code>alertas: []</code>	Positivo
CP-09	CA-03	La alerta NO bloquea el registro	Consumo que deja bajo mínimo	<code>201</code> — se registra igualmente (R-04)	Integridad
CP-10	CA-03	Stock exactamente en el mínimo	Queda 50,00 con mínimo 50,00	Sin alerta — no está por debajo	Borde
CP-11	CA-04	Rollback con varios ingredientes	3 consumos; el tercero sin stock	<code>409</code> ; ninguno de los 3 descontado (R-02)	Integridad
CP-12	CA-04	Rollback ante fallo posterior	Provocar error al insertar el consumo	El inventario no queda descontado	Integridad
CP-13	CA-04	El kardex no queda huérfano	Tras CP-12	No hay movimiento sin su consumo asociado	Integridad
CP-14	—	Concurrencia	2 consumos simultáneos de la misma MP, stock para uno	Uno <code>201</code> , otro <code>409</code> . Nunca negativo (R-03)	Integridad
CP-15	—	El stock nunca queda negativo	Cualquier escenario	<code>SELECT ... WHERE cantidad_disponible < 0</code> devuelve 0 filas	Integridad
CP-16	—	Stock insuficiente	Consumir 200 con 96,30 disponibles	<code>409 STOCK_INSUFICIENTE</code> con ambas cifras (R-05)	Negativo
CP-17	—	Redondeo de 3 a 2 decimales	<code>cantidadReal: 16.205</code>	El inventario baja <code>16.21</code> (HALF_UP, R-08)	Integridad
CP-18	—	Redondeo hacia abajo	<code>cantidadReal: 16.204</code>	El inventario baja <code>16.20</code>	Borde
CP-19	—	Redondeo acumulado	10 consumos de <code>0.005</code> cada uno	El total descontado es coherente, sin deriva	Integridad
CP-20	—	Materia prima sin fila de inventario	MP creada sin su fila (defecto de HU-08)	<code>500 INVENTARIO_NO_INICIALIZADO</code> , mensaje claro	Negativo
CP-21	—	Conciliación kardex ↔ saldo	Tras toda la batería	La consulta de conciliación devuelve 0 filas	Integridad
CP-22	—	Corregir consumo ajusta por diferencia	De 15 a 18 (HU-22 R-07)	Se descuentan 3 más; un solo movimiento	Integridad
CP-23	—	⌚ Rendimiento	Registro de 10 consumos en una petición	Responde en < 2 s (RNF-02)	Rendimiento

2. Batería de seguridad

Obligatoria en **cada endpoint** de esta historia:

ID	Caso	Esperado
SEC-01	Sin cabecera <code>Authorization</code>	<code>401 NO_AUTENTICADO</code>
SEC-02	Token malformado	<code>401 NO_AUTENTICADO</code>
SEC-03	Token expirado	<code>401 NO_AUTENTICADO</code>
SEC-04	Token de rol <code>sin</code> permiso	<code>403 SIN_PERMISO</code>
SEC-05	Token de usuario con <code>estado = 0</code>	<code>403 USUARIO_INACTIVO</code>

SEC-04 es el caso crítico. Si devuelve `200`, falta `@EnableMethodSecurity` y la autorización del sistema completo está desactivada.

Esta historia es la más importante de probar de todo el proyecto. Sus defectos no producen mensajes de error: producen un inventario que dice una cosa y una bodega que dice otra, y la diferencia se descubre semanas después contando sacos.

CP-11 (rollback) es el caso que más se olvida. Registrar tres ingredientes y que el tercero falle dejando los dos primeros descontados produce un inventario imposible de reconstruir: el consumo no quedó registrado, así que no hay forma de saber qué se descontó ni por qué.

CP-14 (conurrencia) requiere peticiones en paralelo — `ab -n 2 -c 2` o el *runner* de Postman con dos hilos—. Es la única forma de detectar la falta de `FOR UPDATE`.

CP-21 (conciliación) debe ejecutarse al cerrar **cada fase**, no solo aquí. Es la prueba de que el kardex sigue siendo confiable a medida que más módulos tocan el inventario.

3. Verificación en base de datos

```

-- === Preparación: anotar los saldos antes ===
SELECT id_materia_prima, cantidad_disponible, fecha_actualizacion
FROM inventario WHERE id_materia_prima IN (5, 7, 9);

-- === CP-01, CP-02: descuento y kardex ===
-- (ejecutar POST /detalles-plan/21/consumos con 16.200 kg de la MP 5)
SELECT cantidad_disponible, fecha_actualizacion FROM inventario WHERE id_materia_prima = 5;
-- 112.50 → 96.30, y la fecha debe haber cambiado sola (CP-04)

SELECT tipo_movimiento, cantidad, saldo_anterior, saldo_posterior, tabla_origen, id_origen
FROM movimientos_inventario_mp
WHERE id_materia_prima = 5 ORDER BY id_movimiento DESC LIMIT 1;
-- SALIDA_PRODUCCION | 16.200 | 112.50 | 96.30 | consumo_materia_prima | 31

-- === CP-11, CP-12: rollback ===
-- Preparar: MP 7 con stock insuficiente a propósito.
-- Enviar 3 consumos donde el tercero excede el stock.
SELECT id_materia_prima, cantidad_disponible FROM inventario WHERE id_materia_prima IN (5,7,9);
-- DEBEN ser idénticos a los valores previos: ningún descuento aplicado.

SELECT COUNT(*) FROM consumo_materia_prima WHERE id_detalle_plan = 21;
-- Sin incrementos

-- === CP-13: el kardex no debe tener movimientos huérfanos ===
SELECT m.id_movimiento, m.id_origen
FROM movimientos_inventario_mp m
LEFT JOIN consumo_materia_prima c ON c.id_consumo = m.id_origen
WHERE m.tabla_origen = 'consumo_materia_prima' AND c.id_consumo IS NULL;
-- DEBE devolver 0 filas

-- === CP-15: nunca negativo ===
SELECT id_materia_prima, cantidad_disponible FROM inventario WHERE cantidad_disponible < 0;
-- DEBE devolver 0 filas. El CHECK de V2 lo impide, pero si llegara a saltar
-- significa que el FOR UPDATE no está funcionando.

-- === CP-21: LA CONCILIACIÓN. Ejecutar al cerrar cada fase. ===
SELECT i.id_materia_prima, mp.nombre,
       i.cantidad_disponible AS saldo_inventario,
       m.saldo_posterior AS saldo_kardex,
       i.cantidad_disponible - COALESCE(m.saldo_posterior, 0) AS diferencia
FROM inventario i
JOIN materias_primas mp USING (id_materia_prima)
LEFT JOIN (
    SELECT id_materia_prima, saldo_posterior,
           ROW_NUMBER() OVER (PARTITION BY id_materia_prima ORDER BY id_movimiento DESC) rn
    FROM movimientos_inventario_mp
) m ON m.id_materia_prima = i.id_materia_prima AND m.rn = 1
WHERE i.cantidad_disponible <> COALESCE(m.saldo_posterior, 0);
-- DEBE devolver 0 filas SIEMPRE.
-- Una sola fila significa que alguna ruta de código actualizó el saldo
-- sin registrar el movimiento: el inventario dejó de ser explicable.

-- === CP-17 a CP-19: el redondeo ===
SELECT c.cantidad_real AS registrado_3_dec, m.cantidad AS descontado_2_dec
FROM consumo_materia_prima c
JOIN movimientos_inventario_mp m ON m.tabla_origen = 'consumo_materia_prima'
AND m.id_origen = c.id_consumo
WHERE c.id_detalle_plan = 21;
-- 16.205 → 16.21 ; 16.204 → 16.20 (HALF_UP)

-- === CP-20: toda MP debe tener fila de inventario (HU-08 R-01) ===
SELECT mp.nombre FROM materias_primas mp
LEFT JOIN inventario i USING (id_materia_prima) WHERE i.id_inventario IS NULL;

```

4. Criterios de cierre

- ☐ Todos los casos ejecutados; los positivos pasan.
- ☐ Los negativos devuelven el HTTP y el `code` esperados.
- ☐ SEC-01 a SEC-05 pasan en todos los endpoints.
- ☐ Ningún endpoint supera 2 s (RNF-02).
- ☐ Cero defectos críticos o altos abiertos.
- ☐ Colección de Postman actualizada y versionada.

HU-24 · Dashboard resumen

Especificación

Fase 8 · Fuente: `previo/Historias_de_Usuario.md`

Como administrador, **quiero** un dashboard con resumen del día (pedidos del día, compras pendientes, stock bajo, producción del día), **para que** tenga visión general de la operación.

Depende de: HU-13, HU-16, HU-20

1. Criterios de aceptación

CA	Enunciado	Cómo se implementa
CA-01	Total de pedidos del día actual	<code>WHERE fecha_pedido = curdate()</code>
CA-02	Compras con estado “Pendiente”	<code>WHERE estado = 'PENDIENTE'</code> — sin filtro de fecha
CA-03	Materias primas con stock por debajo del mínimo	Vista <code>v_inventario_materia_prima</code> (V2)
CA-04	Planes de producción del día actual	<code>WHERE fecha_produccion = curdate()</code>
CA-05	Productos próximos a vencer (próximos 7 días)	<code>fecha_vencimiento BETWEEN curdate() AND curdate() + 7 días</code>

2. Endpoints

Método	Ruta	Descripción	Roles
GET	<code>/api/v1/dashboard</code>	Resumen completo del día	Todos los roles autenticados
GET	<code>/api/v1/dashboard/pedidos-hoy</code>	Detalle de los pedidos del día	ADMIN, VENTAS, CONSULTA
GET	<code>/api/v1/dashboard/vencimientos</code>	Productos próximos a vencer	ADMIN, VENTAS, PRODUCCION, CONSULTA

Reglas transversales (paginación, formato de error, autenticación) en `00-base/04-CONTRATO-API.md`.

3. Reglas de negocio

R-01 · ⚠ “El día actual” depende de la zona horaria del servidor

CA-01 y CA-04 comparan contra `curdate()`, que evalúa **el servidor de base de datos**.

Una instancia RDS está en UTC por defecto; Colombia es UTC-5. Entre las 7 p.m. y la medianoche, `curdate()` devuelve **el día siguiente**, y el dashboard mostraría “0 pedidos hoy” mientras el vendedor acaba de registrar tres.

Configuración obligatoria en `00-base/11-DESPLIEGUE-AWS-RDS.md` §3. Es el mismo riesgo de HU-16 R-01, y aquí se hace visible de forma inmediata.

R-02 · Cada tarjeta responde a una pregunta operativa

Tarjeta	Pregunta	Filtro
Pedidos de hoy	¿Cuánto se vendió hoy?	<code>fecha_pedido = curdate()</code>
Compras pendientes	¿Qué mercancía está por llegar?	<code>estado = 'PENDIENTE'</code> , sin fecha
Stock bajo	¿Qué hay que comprar?	<code>estado_stock IN ('BAJO', 'AGOTADO')</code>
Producción de hoy	¿Qué se está fabricando?	<code>fecha_produccion = curdate()</code>
Por vencer	¿Qué hay que sacar ya?	Vence en ≤ 7 días y tiene stock

Compras pendientes no lleva filtro de fecha (CA-02): una compra pendiente de hace un mes es precisamente la que hay que revisar.

R-03 · Los productos por vencer solo cuentan si tienen stock

Un lote vencido sin existencias no requiere acción. La consulta exige `cantidad_disponible > 0` ; si no, el dashboard mostraría alarmas sobre lotes ya agotados.

Se incluyen también **los ya vencidos con stock**, en rojo: son los que hay que retirar del estante hoy mismo.

R-04 · El dashboard es de solo lectura y debe ser rápido

Cinco consultas independientes. Todas deben responder en conjunto por debajo de los 2 segundos del RNF-02.

- Las cinco se ejecutan **en paralelo** desde el cliente, o en un único endpoint que las agrupe.
- Todas se apoyan en índices añadidos en V2 §C-15: `idx_pedidos_fecha` , `idx_compras_fecha` , `idx_planes_fecha` , `idx_lotes_vencimiento` .
- **Sin caché**: un dashboard operativo que muestre datos de hace cinco minutos hace tomar decisiones sobre información falsa.

R-05 · El contenido se adapta al rol

Todos los roles acceden, pero ven lo que les concierne:

Rol	Tarjetas
ADMIN	Las cinco
VENTAS	Pedidos de hoy, por vencer
COMPRAS	Compras pendientes, stock bajo
PRODUCCION	Producción de hoy, stock bajo, por vencer
CONSULTA	Las cinco, solo lectura

Es filtrado de presentación; los endpoints subyacentes mantienen su propia autorización.

R-06 · Los importes se calculan sobre el detalle

El total de pedidos del día se calcula con `SUM(cantidad * precio_unitario)` sobre `detalle_pedido` — posible gracias a la columna añadida en V2 §C-01. Sin ella, el dashboard no podría mostrar ninguna cifra de ventas.

4. Modelo de datos

Sin cambios de esquema. Consulta `pedidos` , `compras` , `planes_produccion` , `lotes` y las vistas de V2.

```
// GET /api/v1/dashboard
{
  "fecha": "2026-09-18",
  "pedidosHoy": {
    "cantidad": 7, "total": 284500.00,
    "porEstado": { "PENDIENTE": 3, "EN_PREPARACION": 2, "ENTREGADO": 2, "CANCELADO": 0 }
  },
  "comprasPendientes": {
    "cantidad": 2, "total": 620000.00,
    "masAntigua": { "idCompra": 10, "fecha": "2026-09-02", "diasPendiente": 16 }
  },
  "stockBajo": {
    "cantidad": 3, "agotadas": 1,
    "items": [
      { "nombre": "Levadura", "disponible": 0.00, "minimo": 5.00,
        "faltante": 5.00, "nivel": "AGOTADO" },
      { "nombre": "Harina de trigo", "disponible": 46.30, "minimo": 50.00,
        "faltante": 3.70, "nivel": "BAJO" }
    ]
  },
  "produccionHoy": {
    "cantidad": 2,
    "porEstado": { "PLANIFICADO": 1, "EN_PROCESO": 1, "COMPLETADO": 0 },
    "avancePromedio": 45.50
  },
  "porVencer": {
    "cantidad": 4, "vencidos": 1,
    "items": [
      { "codigoLote": "CR-2026-0914", "producto": "Croissant",
        "vence": "2026-09-16", "diasParaVencer": -2, "disponible": 8.00 },
      { "codigoLote": "PF-2026-0917", "producto": "Pan francés",
        "vence": "2026-09-21", "diasParaVencer": 3, "disponible": 40.00 }
    ]
  }
}
```

`diasParaVencer` negativo indica lote ya vencido (R-03).

5. Códigos de error

code	HTTP	Cuándo
------	------	--------

Los transversales (`VALIDACION_FALLIDA` , `NO_AUTENTICADO` , `SIN_PERMISO` , `RECURSO_NO_ENCONTRADO`) están en `00-base/04-CONTRATO-API.md` §5.

6. Fuera de alcance

Lo que **no** cubre esta historia y no debe implementarse aquí: se limita estrictamente a los criterios de aceptación listados arriba. Cualquier funcionalidad adicional se propone como historia nueva, no se agrega en silencio.

7. Definición de Terminado

Se aplica la DoD de `00-base/01-CONVENCIONES.md` §7.

8. Notas

Este dashboard es la prueba de fuego de las correcciones del esquema

Cada tarjeta depende de algo que el `db.sql` original no permitía:

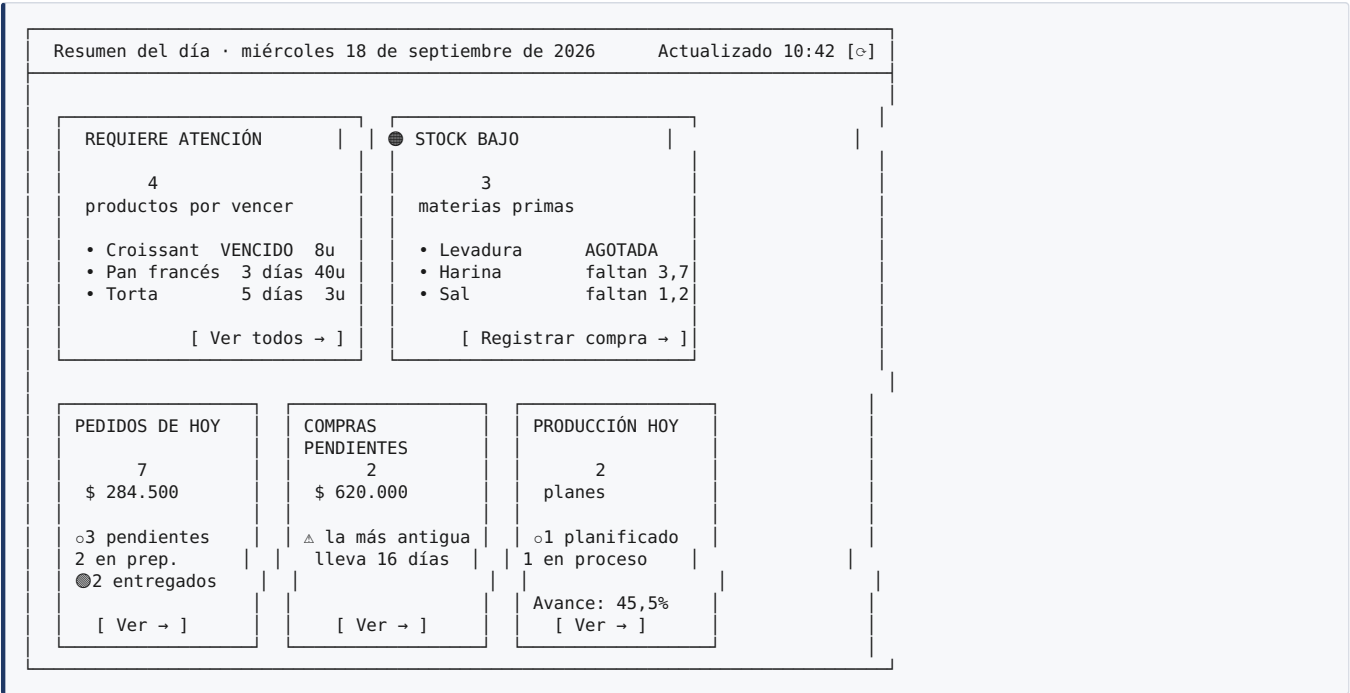
Tarjeta	Depende de
Total de pedidos de hoy	<code>detalle_pedido.precio_unitario</code> (§C-01)
Stock bajo	Que exista fila de inventario por materia prima (HU-08 R-01) y que las compras sumen (§C-03)
Por vencer con stock	<code>inventario_producto_terminado</code> poblado desde HU-19
Producción de hoy	Estados canónicos (§C-12)

Si alguna tarjeta muestra ceros persistentes, el problema no suele estar en esta historia sino en la corrección que la sostiene.

Diseño

1. Pantallas

Pantalla de inicio



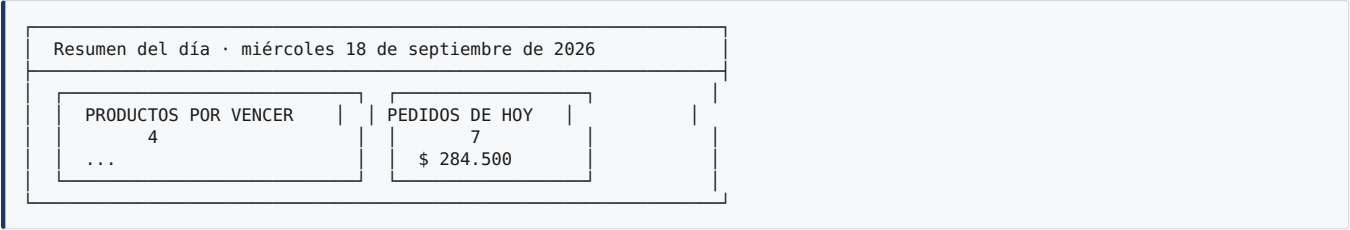
Las tarjetas están ordenadas por urgencia, no por número de criterio de aceptación. Lo vencido y lo agotado arriba, en

formato grande: son las dos cosas que exigen una decisión hoy. El resto es información de contexto.

Cada cifra es un enlace a la pantalla donde se actúa. El dashboard detecta; no resuelve.

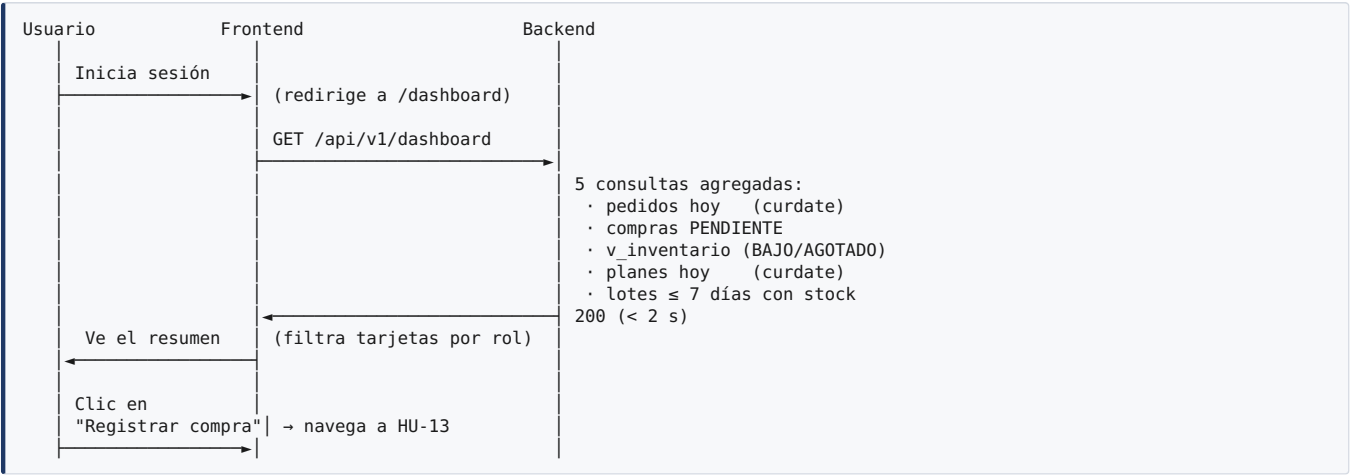
El aviso “la más antigua lleva 16 días” convierte un número neutro en algo accionable: dos compras pendientes no dicen nada, una compra pendiente de hace más de dos semanas sí.

Vista de un rol operativo (VENTAS)



Cada rol ve lo que le concierne (R-05). Mostrar a VENTAS el stock de materia prima solo añade ruido.

2. Flujo



Sin caché (R-04): cada apertura consulta datos frescos.

3. Componentes Angular Material

Elemento	Componente
Tarjetas	MatCard con jerarquía visual por urgencia
Cifra principal	Tipografía grande, con color según estado
Desglose por estado	MatChip pequeños
Semáforo	MatIcon con color
Enlaces	MatButton con RouterLink
Recarga	MatIconButton refresh + marca de hora
Carga por tarjeta	MatProgressSpinner individual

Si se añaden gráficos, cargar antes la guía de visualización de datos del proyecto para mantener la coherencia cromática con HU-26.

4. Validaciones en el formulario

El dashboard **no tiene formularios** (R-04): es solo lectura.

El único parámetro configurable es el horizonte de vencimiento, fijado en **7 días** por CA-05. Se expone como parámetro opcional (?diasVencimiento=) para que HU-27 y la pantalla de lotes puedan reutilizar el endpoint con otro horizonte.

5. Estados de la pantalla

Estado	Qué se muestra
Cargando	Spinner por tarjeta , no uno global
Tarjeta vacía sin incidencias	✓ “Sin productos próximos a vencer” en verde
Stock bajo vacío	✓ “Todas las materias primas por encima del mínimo”
Sin pedidos hoy	“Aún no hay pedidos registrados hoy”
Base vacía	Mensaje de bienvenida con enlaces a los maestros
Error en una tarjeta	Esa tarjeta muestra “No se pudo cargar” + reintentar; las demás siguen funcionando

Que una tarjeta falle no debe tumbar el dashboard (R-04). Y los estados vacíos de las tarjetas de alerta son buenas noticias: se muestran en verde, no con el mensaje genérico de “sin datos”.

6. Accesibilidad y UX

- Todos los campos con `<mat-label>`; nunca solo *placeholder*.
- Navegación por teclado en orden lógico; `Enter` envía el formulario.
- Los mensajes de error del servidor se muestran **junto al campo** (`fieldErrors`).
- El botón de envío se deshabilita mientras la petición está en vuelo.
- Confirmación antes de descartar cambios sin guardar.
- Los tres estados obligatorios (cargando / vacío / error) según `00-base/05-ESTANDARES-QA.md` §7.

Tarea Backend

Solo lectura. Cinco consultas agregadas sobre datos que ya existen. El reto es el rendimiento (R-04), no la lógica.

1. Pasos

1. `DashboardRepository` con cinco métodos, cada uno una consulta agregada:
 - `pedidosDelDia()` — cuenta y total con `SUM(cantidad * precio_unitario)`, agrupado por estado;
 - `comprasPendientes()` — **sin filtro de fecha** (R-02), incluye la más antigua;
 - `stockBajo()` — sobre `v_inventario_materia_prima`, `estado_stock IN ('BAJO','AGOTADO')`;
 - `produccionDelDia()` — planes con `fecha_produccion = curdate()`, con avance promedio;
 - `proximosAVencer(dias)` — lotes con stock que vencen en $\leq N$ días, **incluidos los ya vencidos** (R-03).
2. Usar `curdate()` del servidor, no `LocalDate.now()` de Java (R-01): una sola fuente de fecha.
3. `DashboardService` con `@Transactional(readOnly = true)`; ejecuta las cinco consultas y arma el DTO.
4. DTO: `DashboardResponse` con los cinco bloques anidados.
5. Endpoint único `GET /dashboard` que las agrupa, más dos endpoints de detalle.
6. **Verificar con EXPLAIN que las cinco usan los índices de V2 §C-15.**
7. Sin caché (R-04).
8. `@PreAuthorize("isAuthenticated()")` en el endpoint principal; el filtrado por rol es de presentación (R-05).

2. Archivos

```
backend/src/main/java/co/edu/corposucre/productionfood/reporte/
├── {DashboardRepository,DashboardService,DashboardController}.java
└── dto/{DashboardResponse,PedidosHoyResponse,ComprasPendientesResponse,
    StockBajoResponse,ProduccionHoyResponse,PorVencerResponse}.java
```

3. Puntos de cuidado

- `curdate()` del servidor, no `LocalDate.now()` de Java (R-01). Si la aplicación y la base están en zonas distintas, el dashboard y los datos discrepan a partir de las 7 p.m.
- **Compras pendientes sin filtro de fecha** (R-02): una compra pendiente de hace un mes es la más importante de todas.
- **Los lotes por vencer deben tener stock** (R-03), y los ya vencidos con existencias se incluyen: son acción inmediata.
- **Sin caché** (R-04): un dashboard operativo con datos rancios lleva a decisiones erróneas.
- Verificar el plan de ejecución de las cinco consultas: es la historia donde más fácil se incumple el RNF-02.
- El total de ventas depende de `detalle_pedido.precio_unitario` (§C-01, R-06).

4. Checklist de cierre

- ☐ Todos los CA implementados y verificados manualmente.
- ☐ Endpoints documentados en Swagger con ejemplos.
- ☐ `@PreAuthorize` en cada método, según la matriz de roles.
- ☐ Operaciones de escritura auditadas en bitácora (HU-04).
- ☐ Pruebas unitarias de las reglas con ramificación.
- ☐ Sin `System.out.println` ni código comentado.
- ☐ PR bajo ~250 líneas, revisado y aprobado.

Tarea Frontend

Pantalla de inicio de la aplicación. Debe cargar rápido y priorizar lo accionable.

1. Pasos

- `dashboard.model.ts` y `dashboard.service.ts`.
- `dashboard.component` con cinco tarjetas, **ordenadas por urgencia**, no por número de criterio: primero lo que exige acción hoy.
- Cada tarjeta muestra la cifra grande, el desglose y un enlace a la pantalla correspondiente.
- Filtrado por rol** (R-05) con la directiva `*siRol`.
- Semáforo en stock bajo y vencimientos.
- Botón de recarga manual y marca de hora de la última actualización.
- Ruta por defecto tras iniciar sesión.
- Los tres estados **por tarjeta**: una consulta lenta no debe bloquear las otras cuatro.
- Antes de escribir cualquier gráfico, cargar la guía de visualización de datos del proyecto: los colores y las formas deben ser consistentes con HU-26.

2. Archivos

```
frontend/src/app/features/dashboard/  
├── models/dashboard.model.ts  
├── services/dashboard.service.ts  
├── pages/dashboard/  
└── components/{tarjeta-pedidos,tarjeta-compras,tarjeta-stock,  
               tarjeta-produccion,tarjeta-vencimientos}/
```

3. Puntos de cuidado

- **Estados independientes por tarjeta**: si la consulta de vencimientos tarda, las otras cuatro ya deben estar visibles. Un dashboard que se bloquea entero por una consulta lenta es peor que uno incompleto.
- **Ordenar por urgencia, no por número de CA**: lo vencido y lo agotado arriba.
- Mostrar la hora de la última actualización: sin ella, nadie sabe si está viendo datos de ahora o de cuando abrió la pestaña por la mañana.
- Cada cifra debe ser un enlace: el dashboard sirve para detectar, y de ahí se va a actuar.
- Nada de auto-refresco agresivo: recarga manual y al volver a la pestaña.

4. Checklist de cierre

- ☐ Todos los CA cubiertos desde la interfaz.
- ☐ Los tres estados (cargando / vacío / error) implementados.
- ☐ Errores del servidor mostrados junto al campo correspondiente.
- ☐ Guard de rol aplicado en la ruta.
- ☐ Modelos TypeScript espejo de los DTO del backend.
- ☐ Probado en Chrome y Firefox a 1366×768.
- ☐ Sin `console.log` ni `any`.

Casos de Prueba

Formato y criterios en `00-base/05-ESTANDARES-QA.md`.

1. Casos funcionales

ID	CA	Descripción	Entrada	Resultado esperado	Tipo
CP-01	CA-01	Pedidos del día	3 pedidos hoy, 5 ayer	<code>pedidosHoy.cantidad = 3</code>	Positivo
CP-02	CA-01	Zona horaria	Crear un pedido a las 20:00 hora de Colombia	Cuenta como hoy , no mañana (R-01)	Integridad
CP-03	CA-01	Total de ventas del día	3 pedidos por 284.500	<code>pedidosHoy.total = 284500.00</code> (R-06)	Positivo
CP-04	CA-01	Sin pedidos hoy	Día sin registros	<code>cantidad: 0 , total: 0.00</code> , sin error	Borde
CP-05	CA-02	Compras pendientes	2 pendientes, 5 recibidas	<code>comprasPendientes.cantidad = 2</code>	Positivo
CP-06	CA-02	Sin filtro de fecha	Compra pendiente de hace un mes	Aparece en el conteo (R-02)	Integridad
CP-07	CA-02	Las canceladas no cuentan	1 pendiente, 1 cancelada	Solo cuenta la pendiente	Borde
CP-08	CA-03	Stock bajo	2 bajo mínimo, 1 agotada	<code>cantidad: 3 , agotadas: 1</code>	Positivo
CP-09	CA-03	Stock exactamente en el mínimo	disponible = mínimo	No aparece como bajo	Borde
CP-10	CA-04	Producción del día	2 planes para hoy	<code>produccionHoy.cantidad = 2</code>	Positivo
CP-11	CA-04	Los planes cancelados	Plan de hoy cancelado	Se cuenta aparte en <code>porEstado</code>	Borde
CP-12	CA-05	Próximos a vencer	Lotes que vencen en 3 y 5 días	Ambos en <code>porVencer</code>	Positivo
CP-13	CA-05	Vence exactamente en 7 días	<code>fecha_vencimiento = hoy + 7</code>	Incluido — rango inclusivo	Borde
CP-14	CA-05	Vence en 8 días	<code>hoy + 8</code>	No incluido	Borde
CP-15	CA-05	Solo cuentan los que tienen stock	Lote por vencer con 0 unidades	No aparece (R-03)	Integridad
CP-16	CA-05	Lote ya vencido con stock	Venció hace 2 días, quedan 8 unidades	Aparece con <code>diasParaVencer: -2</code> (R-03)	Borde
CP-17	—	🕒 Rendimiento del dashboard completo	Base con 500 pedidos, 200 compras, 100 lotes	<code>GET /dashboard</code> en < 2 s (RNF-02, R-04)	Rendimiento
CP-18	—	Base vacía	Sistema recién instalado	Todas las cifras en 0, sin errores ni nulos	Borde
CP-19	—	Rol VENTAS accede	<code>GET /dashboard</code> con token VENTAS	<code>200</code>	Positivo
CP-20	—	Rol CONSULTA accede	<code>GET /dashboard</code> con token CONSULTA	<code>200</code>	Positivo
CP-21	—	Sin endpoints de escritura	<code>POST /dashboard</code>	<code>405</code> o <code>404</code>	Seguridad

2. Batería de seguridad

Obligatoria en **cada endpoint** de esta historia:

ID	Caso	Esperado
SEC-01	Sin cabecera <code>Authorization</code>	<code>401 NO_AUTENTICADO</code>
SEC-02	Token malformado	<code>401 NO_AUTENTICADO</code>
SEC-03	Token expirado	<code>401 NO_AUTENTICADO</code>
SEC-04	Token de rol sin permiso	<code>403 SIN_PERMISO</code>
SEC-05	Token de usuario con <code>estado = 0</code>	<code>403 USUARIO_INACTIVO</code>

SEC-04 es el caso crítico. Si devuelve `200` , falta `@EnableMethodSecurity` y la autorización del sistema completo está desactivada.

CP-02 es el caso que hace visible el problema de zona horaria de todo el sistema. Si el servidor está en UTC, a partir de las 7 p.m. hora de Colombia el dashboard muestra “0 pedidos hoy” mientras el equipo acaba de registrar varios. Es el síntoma más inmediato del defecto que también afecta a HU-16 y HU-18.

CP-17 conviene ejecutarlo con datos realistas. Con las diez filas de prueba todo responde al instante; el problema aparece con volumen, y es justo la pantalla que todos abren al entrar.

3. Verificación en base de datos

```
-- CP-02: la zona horaria. Ejecutar antes de cualquier prueba del dashboard.
SELECT @@time_zone, @@system_time_zone, NOW(), CURDATE();
-- Se espera: America/Bogota · -05 · fecha y hora locales de Colombia

-- CP-01, CP-03: pedidos del día
SELECT COUNT(DISTINCT p.id_pedido) AS pedidos,
       COALESCE(SUM(dp.cantidad * dp.precio_unitario), 0) AS total
FROM pedidos p LEFT JOIN detalle_pedido dp USING (id_pedido)
WHERE p.fecha_pedido = curdate();

-- CP-06: compras pendientes, SIN filtro de fecha
SELECT COUNT(*) AS pendientes, MIN(fecha) AS mas_antigua,
       DATEDIFF(curdate(), MIN(fecha)) AS dias
FROM compras WHERE estado = 'PENDIENTE';

-- CP-08, CP-09: stock bajo desde la vista de V2
SELECT nombre, cantidad_disponible, stock_minimo, estado_stock, faltante
FROM v_inventario_materia_prima
WHERE estado_stock IN ('BAJO', 'AGOTADO') ORDER BY faltante DESC;

-- CP-10: producción del día
SELECT estado, COUNT(*) FROM planes_produccion
WHERE fecha_produccion = curdate() GROUP BY estado;

-- CP-12 a CP-16: por vencer, SOLO con stock
SELECT l.codigo_lote, p.nombre, l.fecha_vencimiento,
       DATEDIFF(l.fecha_vencimiento, curdate()) AS dias,
       ipt.cantidad_disponible
FROM lotes l
  JOIN productos p USING (id_producto)
  JOIN inventario_producto_terminado ipt USING (id_lote)
WHERE ipt.cantidad_disponible > 0
  AND l.fecha_vencimiento <= curdate() + INTERVAL 7 DAY
ORDER BY l.fecha_vencimiento;
-- Incluye los vencidos (días negativos) y excluye los agotados

-- CP-17: verificar que se usan los índices de V2 §C-15
EXPLAIN SELECT COUNT(*) FROM pedidos WHERE fecha_pedido = curdate();
EXPLAIN SELECT COUNT(*) FROM compras WHERE estado = 'PENDIENTE';
EXPLAIN SELECT COUNT(*) FROM planes_produccion WHERE fecha_produccion = curdate();
EXPLAIN SELECT * FROM lotes WHERE fecha_vencimiento <= curdate() + INTERVAL 7 DAY;
-- Se esperan: idx_pedidos_fecha, idx_compras_fecha, idx_planes_fecha, idx_lotes_vencimiento
```

4. Criterios de cierre

- ☐ Todos los casos ejecutados; los positivos pasan.
- ☐ Los negativos devuelven el HTTP y el `code` esperados.
- ☐ SEC-01 a SEC-05 pasan en todos los endpoints.
- ☐ Ningún endpoint supera 2 s (RNF-02).
- ☐ Cero defectos críticos o altos abiertos.
- ☐ Colección de Postman actualizada y versionada.

HU-25 · Consulta de pedidos

Especificación

Fase 8 · Fuente: `previo/Historias_de_Usuario.md`

Como administrador, **quiero** consultar pedidos por cliente, rango de fechas o estado, **para que** dé seguimiento a las ventas.

Depende de: HU-17

1. Criterios de aceptación

CA	Enunciado	Cómo se implementa
CA-01	Filtrar por <code>id_cliente</code>	<code>?idCliente=</code>
CA-02	Filtrar por rango de fechas	<code>?fechaInicio=&fechaFin=</code> — ambos inclusivos
CA-03	Filtrar por <code>estado</code>	<code>?estado=PENDIENTE EN_PREPARACION ENTREGADO CANCELADO</code>
CA-04	Cada pedido muestra su detalle (productos, cantidades)	<code>?incluirDetalle=true</code> o consulta individual
CA-05	Soporta paginación	<code>page</code> , <code>size</code> , <code>sort</code> — <code>04-CONTRATO-API.md</code> §4

2. Endpoints

Método	Ruta	Descripción	Roles
GET	<code>/api/v1/pedidos</code>	Consulta con filtros combinables	ADMIN, VENTAS, PRODUCCION, CONSULTA
GET	<code>/api/v1/pedidos/resumen</code>	Totales agregados del período	ADMIN, VENTAS
GET	<code>/api/v1/pedidos/por-cliente</code>	Ventas agrupadas por cliente	ADMIN, VENTAS
GET	<code>/api/v1/pedidos/productos-mas-vendidos</code>	Ranking de productos	ADMIN, VENTAS

Reglas transversales (paginación, formato de error, autenticación) en `00-base/04-CONTRATO-API.md`.

3. Reglas de negocio

R-01 · Solo lectura

Amplía los filtros del listado de HU-16 y agrega tres endpoints de agregación. El estado de los pedidos se cambia en HU-16.

R-02 · Los importes históricos son fiables gracias a §C-01

`detalle_pedido.precio_unitario`, añadido en V2, es lo que permite que este reporte signifique algo: **cada pedido se valora con el precio que tenía cuando se vendió**.

Sin esa columna, el total de un pedido de hace tres meses se recalcularía con el precio de hoy, y el reporte de ventas de agosto daría un número distinto cada vez que alguien edita el catálogo. Es la diferencia entre un informe y una cifra que cambia sola.

R-03 · Los pedidos cancelados no cuentan como venta

En los totales agregados, `CANCELADO` se **excluye**: no se vendió nada.

`ENTREGADO` es venta consumada; `PENDIENTE` y `EN_PREPARACION` son venta comprometida. Se informan **por separado**, igual que en HU-15 R-04: sumarlos todos daría una cifra que no corresponde ni a lo facturado ni a lo prometido.

R-04 · Las devoluciones aprobadas se descuentan del total

Un pedido de 30 panes con 3 devueltos y aprobados representa una venta neta de 27.

El reporte muestra **venta bruta, devoluciones y venta neta**. Informar solo la bruta sobrevalora las ventas; informar solo la neta oculta el volumen de devoluciones, que es un indicador de calidad.

R-05 · Rango de fechas inclusivo

`fecha_pedido` es `DATE`, así que `BETWEEN` funciona sin sorpresas (mismo razonamiento que HU-15 R-03). `fechaInicio > fechaFin` → 400 `RANGO_FECHAS_INVALIDO`.

R-06 · Cargar el detalle de todos los pedidos es costoso

Igual que en HU-15 R-06: con `incluirDetalle=true` el `size` se limita a **20** y la consulta usa un único `JOIN` con agrupación en memoria, nunca una consulta por pedido.

R-07 · El ranking de productos usa cantidades, no importes

“Más vendido” se mide por unidades despachadas. Medirlo por importe haría que un producto caro vendido tres veces superara a uno barato vendido doscientas, lo cual responde a otra pregunta.

Se exponen ambas columnas y se ordena por cantidad.

4. Modelo de datos

Sin cambios de esquema. V2 añadió `idx_pedidos_fecha (fecha_pedido, estado)`, que cubre CA-02 y CA-03.

```
// GET /api/v1/pedidos?idCliente=7&fechaInicio=2026-09-01&fechaFin=2026-09-18&estado=ENTREGADO
{
  "content": [
    { "idPedido": 45, "cliente": { "idCliente": 7, "nombre": "Tienda La Esquina" },
      "fechaPedido": "2026-09-16", "fechaEntrega": "2026-09-16",
      "estado": "ENTREGADO", "cantidadItems": 2,
      "totalBruto": 8400.00, "devoluciones": 0.00, "totalNeto": 8400.00 }
  ],
  "page": 0, "size": 20, "totalElements": 1, "totalPages": 1
}

// GET /api/v1/pedidos/resumen?fechaInicio=2026-09-01&fechaFin=2026-09-30
{
  "periodo": { "desde": "2026-09-01", "hasta": "2026-09-30" },
  "entregados": { "cantidad": 24, "total": 1840000.00 },
  "enCurso": { "cantidad": 6, "total": 320000.00 },
  "cancelados": { "cantidad": 2, "total": 45000.00 },
  "ventaBruta": 1840000.00,
  "devoluciones": 62000.00,
  "ventaNeta": 1778000.00,
  "ticketPromedio": 76666.67
}

// GET /api/v1/pedidos/productos-mas-vendidos?fechaInicio=2026-09-01&fechaFin=2026-09-30
[
  { "producto": "Pan francés", "unidadesVendidas": 1240.00,
    "importe": 1488000.00, "pedidos": 18 },
  { "producto": "Croissant", "unidadesVendidas": 320.00,
    "importe": 800000.00, "pedidos": 12 }
]
```

Campos ordenables: `fechaPedido`, `fechaEntrega`, `estado`, `total`, `id`.

5. Códigos de error

code	HTTP	Cuándo
<code>RANGO_FECHAS_INVALIDO</code>	400	<code>fechaInicio</code> posterior a <code>fechaFin</code>
<code>CLIENTE_INEXISTENTE</code>	404	El <code>idCliente</code> del filtro no existe

Los transversales (`VALIDACION_FALLIDA`, `NO_AUTENTICADO`, `SIN_PERMISO`, `RECURSO_NO_ENCONTRADO`) están en `00-base/04-CONTRATO-API.md` §5.

6. Fuera de alcance

Lo que **no** cubre esta historia y no debe implementarse aquí: se limita estrictamente a los criterios de aceptación listados arriba. Cualquier funcionalidad adicional se propone como historia nueva, no se agrega en silencio.

7. Definición de Terminado

Se aplica la DoD de `00-base/01-CONVENCIONES.md` §7.

8. Notas

Este reporte es el que valida la corrección §C-01

Si el total de un pedido cambia al editar el precio del producto en el catálogo, la columna `precio_unitario` no se está usando y toda la información de ventas es inestable.

Es el caso de prueba CP-14 de esta historia, y el mismo que CP-16 de HU-11 verifica desde el otro extremo.

Diseño

1. Pantallas

Consulta de pedidos con resumen

Consulta de pedidos

Cliente

Todos

Estado

Todos

Período

01/09/2026 – 30/09/2026

VENTA NETA

\$ 1.778.000

24 entregados

COMPROMETIDO

\$ 320.000

6 en curso

DEVOLUCIONES

\$ 62.000

3,4 %

TICKET PROM.

\$ 76.667

VENTAS POR CLIENTE

Supermercado Cen. \$842.000

Panadería El Trigo \$521.000

Tienda La Esquina \$415.000

PRODUCTOS MÁS VENDIDOS

Pan francés 1.240 und \$1.488.000

Croissant 320 und \$ 800.000

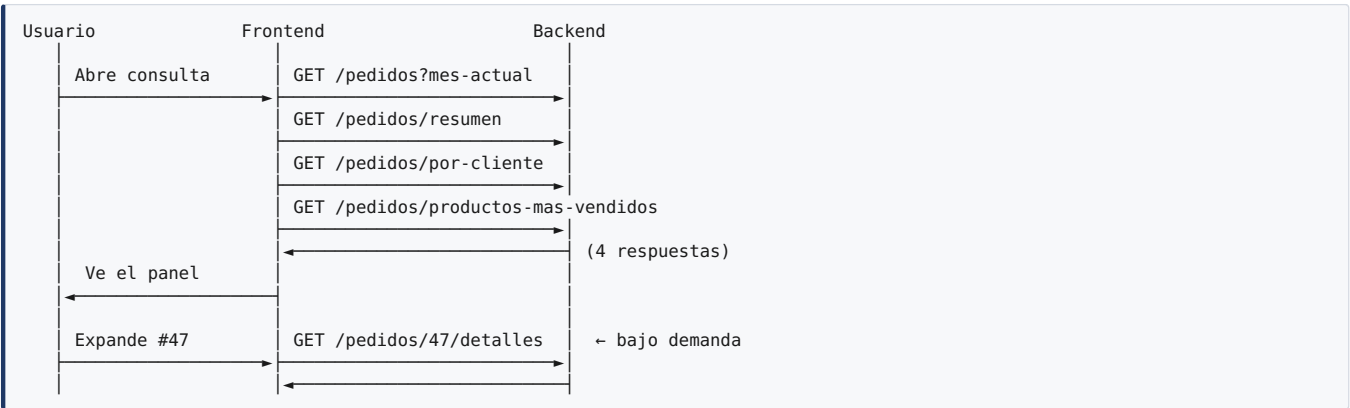
Torta de zanah. 28 und \$ 896.000

N.º	CLIENTE	PEDIDO	ENTREGA	BRUTO	DEVOL.	NETO	EST.
▶ #48	Panadería El Trigo	18/09/26	17/09/26	\$ 45.600	–	\$45.600	●
▼ #47	Tienda La Esquina	18/09/26	HOY	\$ 51.000	\$ 3.600	\$47.400	
	└ Pan francés	30 und × \$1.200 =		\$36.000	(3 devueltos)		
	└ Croissant	6 und × \$2.500 =		\$15.000			
▶ #45	Tienda La Esquina	16/09/26	16/09/26	\$ 8.400	–	\$ 8.400	●

Las tres columnas de importe (bruto, devoluciones, neto) hacen visible el porcentaje de devolución (R-04), que es un indicador de calidad. Un 3,4% de devoluciones merece atención aunque la venta neta sea buena.

El ranking de productos va **ordenado por unidades** (R-07): la torta de zanahoria factura más que el croissant, pero se venden 28 unidades frente a 320. Son dos preguntas distintas y la columna de importe permite responder ambas.

2. Flujo



Las cuatro consultas iniciales son independientes y se lanzan en paralelo.

3. Componentes Angular Material

Elemento	Componente
Tarjetas	<code>MatCard</code> con cifras grandes
Ventas por cliente	Barras horizontales
Productos más vendidos	<code>MatTable</code> compacta
Tabla expandible	<code>MatTable</code> con <code>multiTemplateDataRows</code>
Rango de fechas	<code>MatDateRangePicker</code>
Estado	<code>MatChip</code> con color
Importes	Pipe <code>currency: 'COP'</code>

Antes de escribir los gráficos, cargar la guía de visualización de datos del proyecto: la paleta debe coincidir con la de HU-24 y HU-26.

4. Validaciones en el formulario

Pantalla de consulta: **sin formularios de datos** (R-01). Solo filtros:

Filtro	Regla
Cliente	Opcional
Estado	Opcional, valor canónico
Rango de fechas	<code>fechaInicio <= fechaFin</code>

Por defecto, **mes en curso**.

5. Estados de la pantalla

Estado	Qué se muestra
Cargando	Tarjetas en esqueleto
Vacío en el período	“No hay pedidos en el período seleccionado”
Vacío con filtros	“Ningún pedido coincide” + “Limpiar filtros”
Sin devoluciones	– en la columna, no <code>\$ 0</code>
Error en un bloque	Ese bloque falla; los demás siguen

Mostrar – en lugar de `$ 0` en devoluciones distingue “no hubo” de “hubo por cero”.

6. Accesibilidad y UX

- Todos los campos con `<mat-label>`; nunca solo *placeholder*.
- Navegación por teclado en orden lógico; `Enter` envía el formulario.
- Los mensajes de error del servidor se muestran **junto al campo** (`fieldErrors`).
- El botón de envío se deshabilita mientras la petición está en vuelo.
- Confirmación antes de descartar cambios sin guardar.
- Los tres estados obligatorios (cargando / vacío / error) según `00-base/05-ESTANDARES-QA.md` §7.

Tarea Backend

Solo lectura. Amplía el listado de HU-16 con filtros combinables y agrega tres endpoints de agregación. Mismo patrón que HU-15.

1. Pasos

1. Ampliar `PedidoRepository.buscar()` con `idCliente`, `fechaInicio`, `fechaFin`, `estado`, combinados con `AND`.
2. El `SELECT` calcula tres importes por pedido:
 - `totalBruto` = `SUM(dp.cantidad * dp.precio_unitario)`;
 - `devoluciones` = `SUM(d.cantidad_devuelta * dp.precio_unitario)` sobre devoluciones `APROBADA`;
 - `totalNeto` = bruto – devoluciones (R-04).
3. `PedidoRepository.resumenPorPeriodo()` — agrupa por estado, excluye `CANCELADO` de la venta (R-03), calcula ticket promedio sobre los entregados.
4. `PedidoRepository.ventasPorCliente()` — `GROUP BY cliente`, ordenado por total.
5. `PedidoRepository.productosMasVendidos()` — `GROUP BY producto`, **ordenado por unidades** (R-07), con el importe como columna adicional.

6. `incluirDetalle=true` con un único `JOIN` y agrupación en memoria; `size` limitado a 20 (R-06).
7. Validar `fechaInicio <= fechaFin` → `400 RANGO_FECHAS_INVALIDO`.
8. DTO: `ResumenPedidosResponse`, `VentasClienteResponse`, `ProductoVendidoResponse`.
9. `@Transactional(readOnly = true)`; sin endpoints de escritura (R-01).

2. Archivos

```
backend/src/main/java/co/edu/corposucre/productionfood/
├── pedido/PedidoRepository.java      (+ buscar ampliado y 3 agregaciones)
├── pedido/PedidoService.java         (+ métodos de consulta)
├── pedido/PedidoController.java      (+ /resumen, /por-cliente, /productos-mas-vendidos)
└── reporte/dto/{ResumenPedidosResponse,VentasClienteResponse,ProductoVendidoResponse}.java
```

3. Puntos de cuidado

- **Leer `detalle_pedido.precio_unitario`, nunca `productos.precio`** (R-02). Si el `JOIN` trae el precio del catálogo, todo el histórico de ventas se recalcula solo y el reporte deja de ser reproducible.
- **Descontar las devoluciones aprobadas** (R-04), no las pendientes ni las rechazadas.
- **Las cancelaciones fuera de la venta** (R-03), y lo comprometido en cifra aparte.
- **Ranking por unidades, no por importe** (R-07).
- Evitar el N+1 en `incluirDetalle` (R-06).
- Verificar con `EXPLAIN` que se usa `idx_pedidos_fecha`.

4. Checklist de cierre

- ☐ Todos los CA implementados y verificados manualmente.
- ☐ Endpoints documentados en Swagger con ejemplos.
- ☐ `@PreAuthorize` en cada método, según la matriz de roles.
- ☐ Operaciones de escritura auditadas en bitácora (HU-04).
- ☐ Pruebas unitarias de las reglas con ramificación.
- ☐ Sin `System.out.println` ni código comentado.
- ☐ PR bajo ~250 líneas, revisado y aprobado.

Tarea Frontend

Pantalla de consulta con filtros, tarjetas de resumen y dos rankings.

1. Pasos

1. Ampliar `pedido.service.ts` con `resumen()`, `ventasPorCliente()` y `productosMasVendidos()`.
2. `pedido-consulta.component`:
 - filtros: cliente, estado, rango de fechas;
 - por defecto: **mes en curso**, todos los estados.
3. Tarjetas: venta neta, venta comprometida, devoluciones, ticket promedio.
4. **Mostrar bruto, devoluciones y neto juntos** (R-04): el porcentaje de devoluciones es un indicador de calidad que se pierde si solo se muestra el neto.
5. Gráfico de barras: ventas por cliente (los 5 mayores).
6. Tabla de productos más vendidos, ordenada por unidades.
7. Tabla de pedidos con fila expandible que carga el detalle bajo demanda (R-06).
8. Cargar antes la guía de visualización de datos del proyecto, para que los gráficos sean coherentes con HU-24 y HU-26.
9. Ruta: consulta ADMIN/VENTAS/PRODUCCION/CONSULTA; agregaciones ADMIN/VENTAS.

2. Archivos

```
frontend/src/app/features/pedido/
├── services/pedido.service.ts      (+ agregaciones)
├── pages/pedido-consulta/
└── components/{resumen-ventas-cards,ventas-cliente-chart,productos-vendidos-tabla}/
```

3. Puntos de cuidado

- **Las tres cifras juntas** (bruto, devoluciones, neto): mostrar solo el neto oculta cuánto se está devolviendo, que es justamente lo que conviene vigilar.
 - Cargar el detalle al expandir (R-06).
 - El rango por defecto (mes en curso) evita traer todo el histórico al abrir.
 - Los importes se formatean con locale `es-CO` ; los cálculos vienen del backend.
-

4. Checklist de cierre

- ☐ Todos los CA cubiertos desde la interfaz.
- ☐ Los tres estados (cargando / vacío / error) implementados.
- ☐ Errores del servidor mostrados junto al campo correspondiente.
- ☐ Guard de rol aplicado en la ruta.
- ☐ Modelos TypeScript espejo de los DTO del backend.
- ☐ Probado en Chrome y Firefox a 1366×768.
- ☐ Sin `console.log` ni `any` .

Casos de Prueba

Formato y criterios en `00-base/05-ESTANDARES-QA.md` .

1. Casos funcionales

ID	CA	Descripción	Entrada	Resultado esperado	Tipo
CP-01	CA-01	Filtrar por cliente	?idCliente=7	Solo pedidos de ese cliente	Positivo
CP-02	CA-01	Cliente sin pedidos	Cliente nuevo	content: []	Borde
CP-03	CA-02	Filtrar por rango	?fechaInicio=2026-09-01&fechaFin=2026-09-18	Solo del rango	Positivo
CP-04	CA-02	Rango inclusivo	Pedidos el 01/09 y el 18/09 exactos	Ambos aparecen (R-05)	Borde
CP-05	CA-02	Rango invertido	inicio > fin	400 RANGO_FECHAS_INVALIDO	Negativo
CP-06	CA-03	Filtrar por estado	?estado=ENTREGADO	Solo entregados	Positivo
CP-07	CA-03	Estado inválido	?estado=DESPACHADO	400 PARAMETRO_INVALIDO	Negativo
CP-08	CA-01..03	Filtros combinados	cliente + rango + estado	Los tres con AND	Positivo
CP-09	CA-04	Detalle del pedido	?incluirDetalle=true	Cada pedido con sus productos	Positivo
CP-10	CA-04	Tope de size con detalle	?incluirDetalle=true&size=100	Limitado a 20 o 400 (R-06)	Borde
CP-11	CA-05	Paginación	?page=1&size=5	Página 1 con máx. 5	Positivo
CP-12	CA-05	Orden por total	?sort=total,desc	Ordenado descendente	Positivo
CP-13	CA-05	Orden fuera de lista blanca	?sort=id_cliente,asc	400 CAMPO_ORDEN_INVALIDO	Seguridad
CP-14	—	El total usa el precio histórico	Pedido a \$1.200; subir catálogo a \$1.500; consultar	El total no cambia (R-02)	Integridad
CP-15	—	Las devoluciones se descuentan	Pedido de 30; devolución aprobada de 3	totalNeto = bruto – 3 unidades (R-04)	Integridad
CP-16	—	Las devoluciones pendientes no descuentan	Devolución en estado PENDIENTE	totalNeto = totalBruto	Borde
CP-17	—	Los cancelados no cuentan como venta	Período con pedidos cancelados	ventaNeta los excluye (R-03)	Integridad
CP-18	—	Comprometido separado de entregado	GET /pedidos/resumen	entregados y enCurso en cifras distintas	Positivo
CP-19	—	Ticket promedio	24 entregados por 1.840.000	76666.67	Positivo
CP-20	—	Ranking por unidades, no por importe	Producto caro con 3 ventas vs. barato con 200	El barato aparece primero (R-07)	Integridad
CP-21	—	Ventas por cliente	GET /pedidos/por-cliente	Agrupado y ordenado por total	Positivo
CP-22	—	Período sin pedidos	Rango vacío	Cifras en 0, sin error	Borde
CP-23	—	⌚ Rendimiento	500 pedidos, incluirDetalle=true&size=20	< 2 s (RNF-02)	Rendimiento
CP-24	—	Rol CONSULTA puede consultar	GET /pedidos con token CONSULTA	200	Positivo
CP-25	—	Rol PRODUCCION no accede al resumen	GET /pedidos/resumen con token PRODUCCION	403 SIN_PERMISO	Seguridad

2. Batería de seguridad

Obligatoria en **cada endpoint** de esta historia:

ID	Caso	Esperado
SEC-01	Sin cabecera Authorization	401 NO_AUTENTICADO
SEC-02	Token malformado	401 NO_AUTENTICADO
SEC-03	Token expirado	401 NO_AUTENTICADO
SEC-04	Token de rol sin permiso	403 SIN_PERMISO
SEC-05	Token de usuario con estado = 0	403 USUARIO_INACTIVO

SEC-04 es el caso crítico. Si devuelve `200`, falta `@EnableMethodSecurity` y la autorización del sistema completo está desactivada.

CP-14 es la verificación de que la corrección §C-01 se está usando. Es posible haber añadido la columna `precio_unitario` y seguir leyendo `productos.precio` en las consultas de reporte: la corrección estaría en el esquema pero no en el código, y el síntoma —un informe de ventas que cambia solo— aparecería meses después.

3. Verificación en base de datos

```
-- CP-14: el precio histórico. La prueba clave de §C-01.
SELECT p.id_pedido, dp.cantidad, dp.precio_unitario AS precio_venta,
       pr.precio AS precio_catalogo_hoy,
       dp.cantidad * dp.precio_unitario AS subtotal_historico
FROM pedidos p
     JOIN detalle_pedido dp USING (id_pedido)
     JOIN productos pr USING (id_producto)
WHERE p.id_pedido = 45;
-- precio_venta NO debe seguir a precio_catalogo_hoy.
-- Si el total de la API cambia al editar el catálogo, se está leyendo
-- productos.precio en lugar de detalle_pedido.precio_unitario.

-- CP-15, CP-16: las devoluciones aprobadas descuentan
SELECT p.id_pedido,
       SUM(dp.cantidad * dp.precio_unitario) AS bruto,
       COALESCE(SUM(CASE WHEN d.estado = 'APROBADA'
                        THEN d.cantidad_devuelta * dp.precio_unitario END), 0) AS devuelto
FROM pedidos p
     JOIN detalle_pedido dp USING (id_pedido)
     LEFT JOIN devoluciones d USING (id_detalle_pedido)
WHERE p.id_pedido = 47
GROUP BY p.id_pedido;
-- neto = bruto - devuelto. Solo las APROBADA cuentan.

-- CP-17: los cancelados fuera de la venta
SELECT p.estado, COUNT(DISTINCT p.id_pedido) AS pedidos,
       COALESCE(SUM(dp.cantidad * dp.precio_unitario), 0) AS total
FROM pedidos p LEFT JOIN detalle_pedido dp USING (id_pedido)
WHERE p.fecha_pedido BETWEEN '2026-09-01' AND '2026-09-30'
GROUP BY p.estado;

-- CP-20: ranking por unidades
SELECT pr.nombre,
       SUM(dp.cantidad) AS unidades,
       SUM(dp.cantidad * dp.precio_unitario) AS importe,
       COUNT(DISTINCT p.id_pedido) AS pedidos
FROM detalle_pedido dp
     JOIN productos pr USING (id_producto)
     JOIN pedidos p USING (id_pedido)
WHERE p.estado = 'ENTREGADO'
     AND p.fecha_pedido BETWEEN '2026-09-01' AND '2026-09-30'
GROUP BY pr.id_producto, pr.nombre
ORDER BY unidades DESC; -- por unidades, no por importe

-- CP-23: el índice de V2
EXPLAIN SELECT id_pedido FROM pedidos
WHERE fecha_pedido BETWEEN '2026-09-01' AND '2026-09-30' AND estado = 'ENTREGADO';
-- Se espera: key = idx_pedidos_fecha
```

4. Criterios de cierre

- ☐ Todos los casos ejecutados; los positivos pasan.
- ☐ Los negativos devuelven el HTTP y el `code` esperados.
- ☐ SEC-01 a SEC-05 pasan en todos los endpoints.
- ☐ Ningún endpoint supera 2 s (RNF-02).
- ☐ Cero defectos críticos o altos abiertos.
- ☐ Colección de Postman actualizada y versionada.

HU-26 · Eficiencia de producción

Especificación

Fase 8 · Fuente: `previo/Historias_de_Usuario.md`

Como administrador, **quiero** ver la eficiencia de producción (planificado vs. producido, teórico vs. real), **para que** evalúe el rendimiento de la panadería.

Depende de: [HU-22](#)

1. Criterios de aceptación

CA	Enunciado	Cómo se implementa
CA-01	Cada plan con sus productos: <code>cantidad_planificada</code> vs. <code>cantidad_producida</code>	Desde <code>detalle_plan_produccion</code>
CA-02	Consumo teórico vs. real por materia prima	Desde <code>consumo_materia_prima</code>
CA-03	Porcentaje de eficiencia general	⚠ Agregado, con protección contra división por cero
CA-04	Filtrar por rango de fechas de producción	<code>?fechaInicio=&fechaFin=</code> sobre <code>planes_produccion.fecha_produccion</code>

2. Endpoints

Método	Ruta	Descripción	Roles
GET	<code>/api/v1/reportes/eficiencia</code>	Reporte por período	ADMIN, PRODUCCION
GET	<code>/api/v1/reportes/eficiencia/{idPlan}</code>	Detalle de un plan	ADMIN, PRODUCCION
GET	<code>/api/v1/reportes/eficiencia/materias-primas</code>	Desviación acumulada por insumo	ADMIN, PRODUCCION, COMPRAS

Reglas transversales (paginación, formato de error, autenticación) en `00-base/04-CONTRATO-API.md`.

3. Reglas de negocio

R-01 · Son dos eficiencias distintas y hay que separarlas

CA-01 y CA-02 miden cosas diferentes, y mezclarlas produce un número sin significado:

Indicador	Fórmula	Qué dice
Cumplimiento del plan	<code>producido / planificado × 100</code>	¿Se fabricó lo que se prometió?
Rendimiento de materia prima	<code>teórica / real × 100</code>	¿Se gastó lo previsto por unidad?

Un plan puede tener 60% de cumplimiento (se hicieron 60 de 90 panes) y 100% de rendimiento (cada pan consumió exactamente lo que debía). Son dos problemas distintos: el primero es de planificación o de capacidad, el segundo es de proceso.

El reporte muestra ambas por separado y nunca las promedia entre sí.

R-02 · ⚠ La eficiencia de materia prima está sesgada por lo planificado

Consecuencia directa de HU-22 R-02: `cantidad_teorica` se calcula sobre la **cantidad planificada**, no sobre la producida.

Si se planificaron 90 panes y se hicieron 60, el teórico corresponde a 90 panes y el real a 60: la eficiencia sale artificialmente alta, como si se hubiera ahorrado materia prima.

Por eso el reporte expone una tercera métrica, la única comparable entre planes:

```
rendimiento_ajustado = (teórica × producido / planificado) / real × 100
```

Es decir, se ajusta el teórico a lo que realmente se produjo antes de comparar. Con 60 de 90 panes y consumo proporcional, esta métrica da 100% — que es la verdad.

Sin este ajuste, comparar la eficiencia de dos planes con distinto grado de cumplimiento no significa nada. Es la corrección más importante de este reporte.

R-03 • División por cero en tres lugares

Cálculo	Divisor	Cuándo es cero
Cumplimiento	cantidad_planificada	Nunca — CHECK > 0 en V2
Rendimiento	cantidad_real	Se registró consumo cero
Desviación	cantidad_teorica	Ingrediente fuera de receta (HU-22 R-08)

MySQL devuelve NULL al dividir por cero, sin error (§C-09). Los dos últimos casos son reales y se protegen con CASE WHEN ... THEN NULL ; el frontend muestra - .

Un NULL no se puede promediar como si fuera cero. Al agregar, se excluyen de la media y se informa cuántos registros quedaron fuera; si no, el promedio de eficiencia baja artificialmente cada vez que alguien registra un consumo en cero.

R-04 • La desviación es la métrica que la gente entiende

Repetido de HU-22 R-04 porque aquí es donde se consume:

```
desviación% = (real - teórica) / teórica × 100
```

Positiva = se gastó de más. Es directamente legible, a diferencia de una eficiencia del 200% que casi todo el mundo interpreta al revés.

El reporte encabeza con la desviación y ofrece la eficiencia como columna secundaria.

R-05 • Solo se reportan planes con consumo registrado

Un plan PLANIFICADO sin consumos no tiene nada que medir. Se incluyen los EN_PROCESO , COMPLETADO y CANCELADO que tengan al menos un consumo registrado.

Los cancelados se marcan aparte: su cumplimiento bajo es esperable y promediarlos con el resto distorsiona el indicador general.

R-06 • La desviación acumulada por materia prima es el hallazgo más útil

Agregando todos los planes del período por insumo se detecta el patrón: si la harina lleva un +7% sostenido durante un mes, o la receta está mal calibrada o hay desperdicio sistemático. Un plan aislado no revela eso; treinta sí.

Es el endpoint /eficiencia/materias-primas y probablemente lo más accionable del reporte.

R-07 • Los productos sin receta quedan fuera del análisis

Sin receta no hay consumo teórico (HU-21 R-06), así que no hay nada contra qué comparar. Se listan aparte para que el hueco sea visible, no se omiten en silencio.

4. Modelo de datos

Sin cambios de esquema. Consulta planes_produccion , detalle_plan_produccion , consumo_materia_prima , productos y materias_primas . V2 añadió idx_planes_fecha (fecha_produccion, estado) .

```
// GET /api/v1/reportes/eficiencia?fechaInicio=2026-09-01&fechaFin=2026-09-30
{
  "periodo": { "desde": "2026-09-01", "hasta": "2026-09-30" },
  "resumen": {
    "planesAnalizados": 12,
    "planesCancelados": 2,
    "cumplimientoPromedio": 87.40,
    "rendimientoPromedio": 94.20,
    "rendimientoAjustadoPromedio": 96.80,
    "registrosSinRendimiento": 3,
    "desviacionMateriaPrima": 5.80
  },
  "planes": [
    { "idPlan": 9, "fechaProduccion": "2026-09-18", "estado": "COMPLETADO",
      "responsable": "Ismael Batalla",
      "cumplimiento": 66.67,
      "rendimiento": 92.59,
      "rendimientoAjustado": 61.73,
      "productos": [
        { "producto": "Pan francés", "planificada": 90.00, "producida": 60.00,
          "cumplimiento": 66.67 }
      ],
      "consumos": [
        { "materiaPrima": "Harina de trigo", "teorica": 15.000, "real": 16.200,
          "desviacion": 8.00, "eficiencia": 92.59 },
        { "materiaPrima": "Sal", "teorica": 0.300, "real": 0.000,
          "desviacion": -100.00, "eficiencia": null }
      ]
    }
  ],
  "sinReceta": [
    { "producto": "Pan integral", "planes": 2,
      "nota": "Sin receta: no se puede calcular consumo teórico" }
  ]
}

// GET /api/v1/reportes/eficiencia/materias-primas?fechaInicio=2026-09-01&fechaFin=2026-09-30
[
  { "materiaPrima": "Harina de trigo", "unidad": "kg",
    "teoricaAcumulada": 180.000, "realAcumulada": 192.600,
    "desviacion": 7.00, "planes": 12,
    "tendencia": "CONSUMO_EXCESIVO" },
  { "materiaPrima": "Levadura", "unidad": "kg",
    "teoricaAcumulada": 1.800, "realAcumulada": 1.795,
    "desviacion": -0.28, "planes": 12,
    "tendencia": "NORMAL" }
]
```

`eficiencia: null` cuando `real = 0` (R-03). `rendimientoAjustado` es la métrica comparable entre planes (R-02).

5. Códigos de error

code	HTTP	Cuándo
RANGO_FECHAS_INVALIDO	400	<code>fechaInicio</code> posterior a <code>fechaFin</code>
PLAN_INEXISTENTE	404	El plan no existe
PLAN_SIN_CONSUMOS	404	El plan no tiene consumos registrados (R-05)

Los transversales (`VALIDACION_FALLIDA` , `NO_AUTENTICADO` , `SIN_PERMISO` , `RECURSO_NO_ENCONTRADO`) están en `00-base/04-CONTRATO-API.md` §5.

6. Fuera de alcance

Lo que **no** cubre esta historia y no debe implementarse aquí: se limita estrictamente a los criterios de aceptación listados arriba. Cualquier funcionalidad adicional se propone como historia nueva, no se agrega en silencio.

7. Definición de Terminado

Se aplica la DoD de `00-base/01-CONVENCIONES.md` §7.

8. Notas

Por qué este reporte es difícil de hacer bien

Tres trampas, y ninguna produce un error visible:

- 1. **Promediar NULL como cero** (R-03) hunde el indicador general cada vez que alguien registra un consumo en cero.
- 2. **Comparar eficiencias de planes con distinto cumplimiento** sin ajustar (R-02) lleva a conclusiones invertidas: el plan que menos produjo parece el más eficiente.
- 3. **Mezclar cumplimiento y rendimiento** en un solo “porcentaje de eficiencia” (R-01) produce una cifra que no responde a ninguna pregunta.

CA-03 pide “el porcentaje de eficiencia general”. Se entrega, pero acompañado de las métricas que lo hacen interpretable.

Diseño

1. Pantallas

Reporte de eficiencia

Eficiencia de producción

Período

01/09/2026 – 30/09/2026

CUMPLIMIENTO DEL PLAN

87,4 %

Se produjo el 87,4 % de lo planificado

12 planes · 2 cancel.

RENDIMIENTO AJUSTADO

96,8 %

Por cada unidad producida se usó un 3,2 % menos de lo previsto

DESVIACIÓN MAT. PRIMA

+5,8 % ●

Se gastó un 5,8 % más de materia prima de lo previsto

i 3 consumos sin rendimiento calculable (cantidad real en cero), excluidos del promedio.

DESVIACIÓN POR MATERIA PRIMA (acumulado del período)

Harina de trigo

+7,0 %

180,0 → 192,6 kg

● 12 planes

Mantequilla

+3,1 %

42,0 → 43,3 kg

● 12 planes

Levadura

-0,3 %

1,8 → 1,8 kg

● 12 planes

Azúcar

-4,2 %

60,0 → 57,5 kg

● 11 planes

PLANES DEL PERÍODO

PLAN	FECHA	RESPONSABLE	CUMPLIM.	RENDIM.	AJUSTADO	ESTADO
▼ #9	18/09/2026	Ismael Batalla	66,7 %	92,6 %	61,7 %	● Completado
└ Pan francés		planificado 90 und · producido 60 und			66,7 %	
└ Harina de trigo		teórica 15,000 real 16,200 +8,0 % ef. 92,6 %				
└ Sal		teórica 0,300 real 0,000 -100,0 % ef. - ①				
► #8	17/09/2026	Natalia Góngora	100,0 %	98,1 %	98,1 %	● Completado

△ PRODUCTOS SIN RECETA (fuera del análisis)

Pan integral – 2 planes. Sin receta no hay consumo teórico. [Crear receta →]

Cada tarjeta lleva una frase que explica el número. “96,8%” no significa nada por sí solo; “por cada unidad producida se usó un 3,2% menos de lo previsto” se entiende sin consultar la fórmula.

Las tres métricas están separadas (R-01): cumplimiento y rendimiento responden a preguntas distintas y nunca se promedian entre sí.

El - con ① en la fila de Sal es eficiencia: null (R-03). El aviso sobre los 3 consumos excluidos evita que el promedio parezca cubrir todo.

La barra de desviación por materia prima es lo más accionable del reporte (R-06): un +7% sostenido en harina durante doce planes no es ruido, es una receta mal calibrada o desperdicio sistemático.

Por qué el ajustado importa — el caso del plan #9

① Rendimiento vs. rendimiento ajustado

Plan #9: se planificaron 90 panes, se produjeron 60.

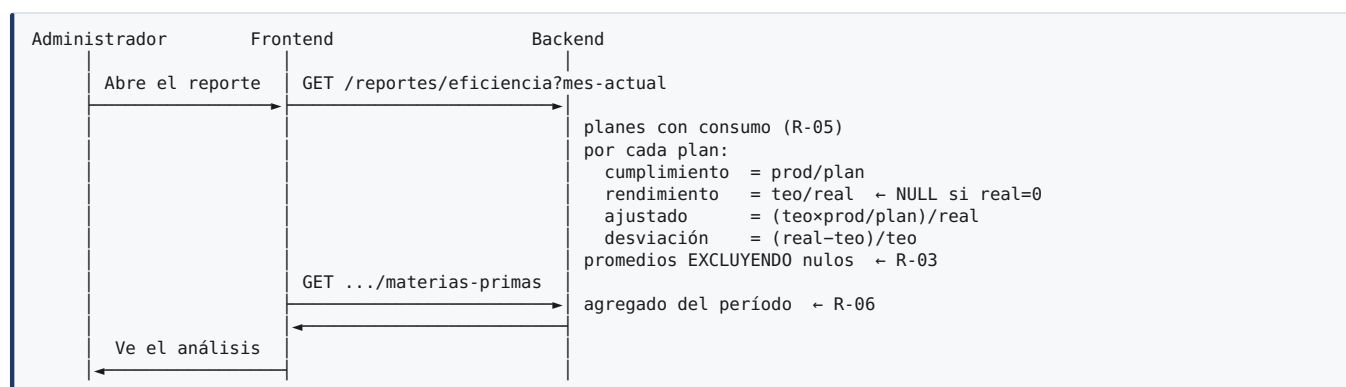
El consumo teórico (15 kg) corresponde a 90 panes, pero el consumo real (16,2 kg) corresponde a 60.

Rendimiento sin ajustar: 92,6 % ← compara peras con manzanas
Rendimiento ajustado: 61,7 % ← comparable entre planes

El ajustado revela que, por pan producido, se consumió bastante más harina de la que la receta indica.

Este tooltip explica R-02 en el punto donde importa. Sin el ajuste, el plan que peor cumplió aparecería como el más eficiente del período.

2. Flujo



3. Componentes Angular Material

Elemento	Componente
Tarjetas de métricas	<code>MatCard</code> con cifra, unidad y frase explicativa
Desviación por insumo	Barras divergentes (positivas y negativas desde el cero)
Tabla de planes	<code>MatTable</code> expandible
Métrica nula	– + <code>MatIcon</code> info con <code>MatTooltip</code>
Explicación del ajustado	<code>MatDialog</code> o <code>MatTooltip</code> extenso
Rango de fechas	<code>MatDateRangePicker</code>
Sin receta	<code>MatCard</code> ámbar con <code>RouterLink</code> a HU-12

Cargar la guía de visualización de datos del proyecto antes de escribir los gráficos. Las barras divergentes requieren una paleta que distinga exceso de defecto sin depender solo del color.

4. Validaciones en el formulario

Pantalla de consulta: **sin formularios de datos**. Solo el filtro de período:

Filtro	Regla
Rango de fechas de producción	<code>fechaInicio <= fechaFin</code>

Por defecto, **mes en curso**.

5. Estados de la pantalla

Estado	Qué se muestra
Cargando	Tarjetas en esqueleto
Sin planes en el período	“No hay planes con consumo registrado en el período”
Métrica no calculable	— con tooltip “No se puede calcular: consumo real en cero”
Consumos excluidos del promedio	i Aviso con el conteo
Productos sin receta	⚠ Sección aparte con enlace a HU-12
Solo planes cancelados	Aviso de que el promedio puede no ser representativo
Error	Mensaje + “Reintentar”

Nunca mostrar 0% donde el cálculo dio null (R-03): un cero inventado es peor que un hueco declarado, porque se promedia y se interpreta.

6. Accesibilidad y UX

- Todos los campos con `<mat-label>`; nunca solo *placeholder*.
- Navegación por teclado en orden lógico; `Enter` envía el formulario.
- Los mensajes de error del servidor se muestran **junto al campo** (`fieldErrors`).
- El botón de envío se deshabilita mientras la petición está en vuelo.
- Confirmación antes de descartar cambios sin guardar.
- Los tres estados obligatorios (cargando / vacío / error) según `00-base/05-ESTANDARES-QA.md` §7.

Tarea Backend

Solo lectura, pero es el reporte con más trampas de cálculo del proyecto. Reutiliza `CalculadoraEficienciaService` de HU-22.

1. Pasos

1. `EficienciaRepository`:
 - `planesConConsumo(fechaInicio, fechaFin)` — solo planes con al menos un consumo (R-05);
 - `detallePlan(idPlan)` — productos con planificado y producido;
 - `consumosPlan(idPlan)` — teórica y real por materia prima;
 - `desviacionPorMateriaPrima(fechaInicio, fechaFin)` — agregado del período (R-06);
 - `productosSinReceta(fechaInicio, fechaFin)` (R-07).
2. **Todos los cálculos con `CASE WHEN divisor = 0 THEN NULL`** (R-03).
3. `EficienciaService` con las tres métricas de R-01 y R-02:

```

    cumplimiento      = producida / planificada * 100;
    rendimiento        = real == 0 ? null : teorica / real * 100;
    rendimientoAjustado = real == 0 ? null
    : (teorica * producida / planificada) / real * 100; // R-02
    desviacion         = teorica == 0 ? null : (real - teorica) / teorica * 100;

```

Con `BigDecimal`, escala 4 en los intermedios y 2 en el resultado, `HALF_UP`.

4. **Al promediar, excluir los `null`** y contar cuántos se excluyeron (`registrosSinRendimiento`, R-03). **Nunca tratarlos como cero.**
5. Separar los planes cancelados del promedio general (R-05).
6. Validar `fechaInicio <= fechaFin`.
7. `@Transactional(readOnly = true)`. Sin endpoints de escritura.
8. Verificar con `EXPLAIN` el uso de `idx_planes_fecha`.

2. Archivos

```

backend/src/main/java/co/edu/corposucre/productionfood/reporte/
├── {EficienciaRepository,EficienciaService,EficienciaController}.java
└── dto/{EficienciaResponse,EficienciaPlanResponse,ResumenEficienciaResponse,
    DesviacionMateriaPrimaResponse}.java

```

3. Puntos de cuidado

- **null no es cero al promediar** (R-03). `AVG()` de SQL ya ignora los `NULL`, pero un promedio calculado en Java con un `orElse(0)` los incluye como ceros y hunde el indicador. Es el error más probable de esta historia.
- **Implementar `rendimientoAjustado`** (R-02). Sin él, el reporte lleva a la conclusión contraria: el plan que produjo la mitad aparece como el más eficiente.
- **No fusionar cumplimiento y rendimiento** en un solo número (R-01).
- **Reutilizar `CalculadoraEficienciaService` de HU-22**, no reescribir las fórmulas: si divergen, el dato del registro y el del reporte no coincidirán.
- Los planes cancelados se informan aparte (R-05).
- Los productos sin receta se listan, no se omiten (R-07).

4. Checklist de cierre

- ☐ Todos los CA implementados y verificados manualmente.
- ☐ Endpoints documentados en Swagger con ejemplos.
- ☐ `@PreAuthorize` en cada método, según la matriz de roles.
- ☐ Operaciones de escritura auditadas en bitácora (HU-04).
- ☐ Pruebas unitarias de las reglas con ramificación.
- ☐ Sin `System.out.println` ni código comentado.
- ☐ PR bajo ~250 líneas, revisado y aprobado.
- ☐ **Pruebas unitarias de las cuatro fórmulas con divisor cero.**
- ☐ **Prueba: un consumo con `real = 0` no baja el promedio general.**

Tarea Frontend

Reporte analítico. La presentación importa tanto como el cálculo: un número mal etiquetado lleva a la decisión equivocada.

1. Pasos

1. `eficiencia.model.ts` y `eficiencia.service.ts`.
2. **Cargar la guía de visualización de datos del proyecto antes de escribir cualquier gráfico**, para mantener la paleta coherente con HU-24 y HU-25.
3. `eficiencia-reporte.component`:
 - filtro de rango de fechas (por defecto, mes en curso);
 - tarjetas con las tres métricas **claramente etiquetadas** (R-01).
4. **Explicar cada métrica con un tooltip**: “Cumplimiento: se produjo el 87% de lo planificado” / “Rendimiento ajustado: por cada unidad producida se consumió un 3% menos de lo previsto”.
5. Tabla de planes con las tres columnas, expandible al detalle por producto y materia prima.
6. **Mostrar – donde la métrica sea `null`** (R-03), con tooltip explicativo. Nunca `0%`.
7. Indicar cuántos registros quedaron fuera del promedio (`registrosSinRendimiento`).
8. Gráfico de desviación por materia prima, ordenado por magnitud (R-06): es el hallazgo más accionable.
9. Sección aparte con los productos sin receta (R-07), con enlace a HU-12.
10. Ruta: ADMIN y PRODUCCION; el endpoint de materias primas también para COMPRAS.

2. Archivos

```
frontend/src/app/features/reporte/  
├── models/eficiencia.model.ts  
├── services/eficiencia.service.ts  
├── pages/eficiencia-reporte/  
└── components/{metricas-cards,desviacion-mp-chart,plan-eficiencia-tabla}/
```

3. Puntos de cuidado

- **Etiquetar las métricas con lo que significan, no con su fórmula.** “Eficiencia: 92,6%” no dice nada; “Se consumió un 8% más de materia prima de lo previsto” sí.
- **– para los nulos** (R-03), nunca `0%`: un cero inventado por el frontend es peor que un hueco honesto.
- **Mostrar el conteo de registros excluidos** del promedio: sin él, el promedio parece cubrir todo cuando no es así.
- El gráfico de desviación por insumo ordenado por magnitud pone arriba lo que hay que revisar (R-06).
- Los productos sin receta visibles: son un hueco de datos, no un cero.

4. Checklist de cierre

- ☐ Todos los CA cubiertos desde la interfaz.
- ☐ Los tres estados (cargando / vacío / error) implementados.
- ☐ Errores del servidor mostrados junto al campo correspondiente.
- ☐ Guard de rol aplicado en la ruta.
- ☐ Modelos TypeScript espejo de los DTO del backend.
- ☐ Probado en Chrome y Firefox a 1366×768.
- ☐ Sin `console.log` ni `any`.

Casos de Prueba

Formato y criterios en `00-base/05-ESTANDARES-QA.md`.

1. Casos funcionales

ID	CA	Descripción	Entrada	Resultado esperado	Tipo
CP-01	CA-01	Cumplimiento por producto	Planificado 90, producido 60	<code>cumplimiento: 66.67</code>	Positivo
CP-02	CA-01	Cumplimiento del 100%	Planificado 30, producido 30	<code>100.00</code>	Borde
CP-03	CA-01	Producción por encima del plan	Planificado 90, producido 100	<code>111.11</code> (HU-21 R-04)	Borde
CP-04	CA-01	Sin producción registrada	Planificado 90, producido 0	<code>cumplimiento: 0.00</code>	Borde
CP-05	CA-02	Teórico vs. real	Teórica 15, real 16,2	<code>desviacion: 8.00</code> , <code>eficiencia: 92.59</code>	Positivo
CP-06	CA-02	Rendimiento con real = 0	Teórica 15, real 0	<code>eficiencia: null</code> , no 0 ni error (R-03)	Integridad
CP-07	CA-02	Desviación con teórica = 0	Ingrediente fuera de receta	<code>desviacion: null</code> (R-03)	Integridad
CP-08	CA-03	Los nulos no se promedian como cero	3 consumos: 92%, 100% y <code>null</code>	Promedio = 96% , no 64%. <code>registrosSinRendimiento: 1</code> (R-03)	Integridad
CP-09	CA-03	Rendimiento ajustado	Teórica 15 (para 90), producido 60, real 10	<code>rendimientoAjustado: 100.00</code> (R-02)	Integridad
CP-10	CA-03	Ajustado vs. sin ajustar	Mismo caso que CP-09	<code>rendimiento: 150.00</code> pero <code>rendimientoAjustado: 100.00</code>	Integridad
CP-11	CA-03	Cumplimiento y rendimiento no se mezclan	Cumplimiento 66%, rendimiento 92%	Son dos campos distintos , sin promedio entre sí (R-01)	Integridad
CP-12	CA-04	Filtrar por rango	<code>?fechaInicio=2026-09-01&fechaFin=2026-09-30</code>	Solo planes con producción en el rango	Positivo
CP-13	CA-04	Rango inclusivo	Planes el 01/09 y el 30/09	Ambos incluidos	Borde
CP-14	CA-04	Rango invertido	<code>inicio > fin</code>	<code>400</code> <code>RANGO_FECHAS_INVALIDO</code>	Negativo
CP-15	—	Solo planes con consumo	Plan <code>PLANIFICADO</code> sin consumos	No aparece en el reporte (R-05)	Borde
CP-16	—	Planes cancelados aparte	Período con 2 planes cancelados	<code>planesCancelados: 2</code> , fuera del promedio general	Integridad
CP-17	—	Desviación acumulada por materia prima	Harina: teórica 180, real 192,6 en 12 planes	<code>desviacion: 7.00</code> , <code>tendencia: "CONSUMO_EXCESIVO"</code> (R-06)	Positivo
CP-18	—	Materia prima con desviación normal	Teórica 1,8 real 1,795	<code>tendencia: "NORMAL"</code>	Borde
CP-19	—	Productos sin receta listados	Plan con un producto sin receta	Aparece en <code>sinReceta[]</code> , no se omite (R-07)	Borde
CP-20	—	Período sin planes	Rango vacío	Cifras en 0 o <code>null</code> , sin error	Borde
CP-21	—	Detalle de un plan	<code>GET /reportes/eficiencia/9</code>	Productos y consumos del plan	Positivo
CP-22	—	Plan sin consumos	<code>GET /reportes/eficiencia/{id}</code> de un plan sin consumo	<code>404 PLAN_SIN_CONSUMOS</code>	Negativo
CP-23	—	⌚ Rendimiento	100 planes con 5 consumos cada uno	< 2 s (RNF-02)	Rendimiento
CP-24	—	Rol VENTAS no accede	<code>GET /reportes/eficiencia</code> con token VENTAS	<code>403 SIN_PERMISO</code>	Seguridad
CP-25	—	Rol COMPRAS accede a desviación de materias primas	<code>GET .../materias-primas</code> con token COMPRAS	<code>200</code>	Positivo

2. Batería de seguridad

Obligatoria en **cada endpoint** de esta historia:

ID	Caso	Esperado
SEC-01	Sin cabecera Authorization	401 NO_AUTENTICADO
SEC-02	Token malformado	401 NO_AUTENTICADO
SEC-03	Token expirado	401 NO_AUTENTICADO
SEC-04	Token de rol sin permiso	403 SIN_PERMISO
SEC-05	Token de usuario con estado = 0	403 USUARIO_INACTIVO

SEC-04 es el caso crítico. Si devuelve **200**, falta **@EnableMethodSecurity** y la autorización del sistema completo está desactivada.

CP-08 y CP-10 son los casos que distinguen un reporte útil de uno engañoso.

CP-08: si los **null** se promedian como cero, cada consumo registrado en cero hunde el indicador general. El reporte diría que la producción es ineficiente cuando en realidad hay un dato faltante.

CP-10: sin el ajuste de R-02, el plan que produjo 60 de 90 aparece con un rendimiento del 150%, es decir, como el más eficiente del período. La conclusión es exactamente la contraria a la realidad, y quien lea el reporte premiará el plan que peor cumplió.

3. Verificación en base de datos

```
-- CP-06 a CP-08: la división por cero y el promedio. El núcleo de esta historia.
SELECT c.id_consumo, mp.nombre, c.cantidad_teorica, c.cantidad_real,
CASE WHEN c.cantidad_real = 0 THEN NULL
ELSE ROUND(c.cantidad_teorica / c.cantidad_real * 100, 2) END AS rendimiento,
CASE WHEN c.cantidad_teorica = 0 THEN NULL
ELSE ROUND((c.cantidad_real - c.cantidad_teorica) / c.cantidad_teorica * 100, 2)
END AS desviacion
FROM consumo_materia_prima c
JOIN materias_primas mp USING (id_materia_prima)
WHERE c.id_detalle_plan = 21;

-- CP-08: AVG() de SQL ignora los NULL - el promedio debe coincidir con la API
SELECT ROUND(AVG(CASE WHEN cantidad_real = 0 THEN NULL
ELSE cantidad_teorica / cantidad_real * 100 END), 2) AS promedio,
COUNT(*) AS total_registros,
SUM(CASE WHEN cantidad_real = 0 THEN 1 ELSE 0 END) AS excluidos
FROM consumo_materia_prima;
-- Si la API devuelve un promedio MENOR que este, está tratando los NULL como cero.

-- CP-09, CP-10: el rendimiento ajustado (R-02)
SELECT dpp.cantidad_planificada, dpp.cantidad_producida,
c.cantidad_teorica, c.cantidad_real,
ROUND(c.cantidad_teorica / NULLIF(c.cantidad_real,0) * 100, 2) AS rendimiento_bruto,
ROUND((c.cantidad_teorica * dpp.cantidad_producida / dpp.cantidad_planificada)
/ NULLIF(c.cantidad_real,0) * 100, 2) AS rendimiento_ajustado
FROM consumo_materia_prima c
JOIN detalle_plan_produccion dpp USING (id_detalle_plan)
WHERE c.id_detalle_plan = 21;
-- Con planificado 90, producido 60, teórica 15 y real 10:
-- rendimiento_bruto = 150,00 ← sugiere un ahorro que no existe
-- rendimiento_ajustado = 100,00 ← la verdad: consumo proporcional correcto

-- CP-17: la desviación acumulada, el dato más accionable (R-06)
SELECT mp.nombre, u.abreviatura,
SUM(c.cantidad_teorica) AS teorica_total,
SUM(c.cantidad_real) AS real_total,
ROUND((SUM(c.cantidad_real) - SUM(c.cantidad_teorica))
/ NULLIF(SUM(c.cantidad_teorica),0) * 100, 2) AS desviacion_pct,
COUNT(DISTINCT dpp.id_plan) AS planes
FROM consumo_materia_prima c
JOIN detalle_plan_produccion dpp USING (id_detalle_plan)
JOIN planes_produccion pp USING (id_plan)
JOIN materias_primas mp USING (id_materia_prima)
JOIN unidades_medida u ON u.id_unidad = mp.id_unidad
WHERE pp.fecha_produccion BETWEEN '2026-09-01' AND '2026-09-30'
GROUP BY mp.id_materia_prima, mp.nombre, u.abreviatura
ORDER BY ABS(desviacion_pct) DESC;
-- Una desviación sostenida del +7% en harina indica receta mal calibrada
-- o desperdicio sistemático. Un plan aislado no lo revela; treinta sí.

-- CP-23: el índice de V2
EXPLAIN SELECT id_plan FROM planes_produccion
WHERE fecha_produccion BETWEEN '2026-09-01' AND '2026-09-30';
```

4. Criterios de cierre

- ☐ Todos los casos ejecutados; los positivos pasan.
- ☐ Los negativos devuelven el HTTP y el `code` esperados.
- ☐ SEC-01 a SEC-05 pasan en todos los endpoints.
- ☐ Ningún endpoint supera 2 s (RNF-02).
- ☐ Cero defectos críticos o altos abiertos.
- ☐ Colección de Postman actualizada y versionada.

HU-27 · Alerta de stock bajo

Especificación

Fase 8 · Fuente: `previo/Historias_de_Usuario.md`

Como administrador, **quiero** ver materias primas con stock por debajo del mínimo, **para que** reabastezca a tiempo.

Depende de: HU-10

1. Criterios de aceptación

CA	Enunciado	Cómo se implementa
CA-01	Listar todas las materias primas donde <code>cantidad_disponible < stock_minimo</code>	Vista <code>v_inventario_materia_prima</code> (V2), <code>estado_stock IN ('BAJO', 'AGOTADO')</code>
CA-02	Mostrar nombre, stock actual y stock mínimo	Campos de la vista
CA-03	Mostrar la diferencia (cuánto falta para alcanzar el mínimo)	Campo <code>faltante</code> — <code>GREATEST(minimo - disponible, 0)</code>
CA-04	Ordenar por urgencia (mayor diferencia primero)	<code>ORDER BY faltante DESC</code> por defecto

2. Endpoints

Método	Ruta	Descripción	Roles
GET	<code>/api/v1/inventario/alertas</code>	Materias primas bajo mínimo	ADMIN, PRODUCCION, COMPRAS, CONSULTA
GET	<code>/api/v1/inventario/alertas/resumen</code>	Conteo para el indicador global	Todos los roles autenticados
GET	<code>/api/v1/inventario/alertas/sugerencia-compra</code>	Propuesta de reposición agrupada por proveedor	ADMIN, COMPRAS

Reglas transversales (paginación, formato de error, autenticación) en `00-base/04-CONTRATO-API.md`.

3. Reglas de negocio

R-01 · La historia más simple de la fase, y la que más se usa

Una sola consulta sobre la vista creada en V2. Toda la complejidad está resuelta allí: el `LEFT JOIN`, el `COALESCE`, el `CASE` de `estado_stock` y el `GREATEST` del faltante.

Es también el reporte que se consulta a diario: sostiene el indicador global de la barra superior y la tarjeta del dashboard (HU-24 CA-03).

R-02 · “Por debajo del mínimo” es estricto (CA-01)

`cantidad_disponible < stock_minimo`. Con disponible **exactamente igual** al mínimo, **no hay alerta**: se está justo en el nivel definido como aceptable.

Es un borde que conviene tener claro porque se presta a interpretaciones: `<` no es `<=`.

R-03 · AGOTADO y BAJO son urgencias distintas

Nivel	Condición	Qué significa
AGOTADO	<code>disponible = 0</code>	Bloquea la producción. No se puede fabricar nada que lo use
BAJO	<code>0 < disponible < minimo</code>	Hay que reponer, pero aún se puede trabajar

AGOTADO siempre va primero, independientemente del faltante: una materia prima agotada con un faltante de 2 kg es más urgente que otra con 40 kg disponibles y un faltante de 60.

El orden real es: primero AGOTADO, luego BAJO por faltante descendente. CA-04 pide ordenar por diferencia; se cumple dentro de cada nivel.

R-04 · `stock_minimo = 0` significa “no alertar”

Una materia prima con mínimo cero nunca aparece como `BAJO` (HU-08 R-06). Es la forma de excluir del monitoreo los insumos que se compran bajo pedido.

Si además su disponible es cero, sí aparece como `AGOTADO`: no hay existencias.

R-05 · Las materias primas inactivas se excluyen por defecto

Una materia prima desactivada (HU-08 R-04) no se va a reponer. Alertar sobre ella genera ruido permanente.

Se excluyen salvo que se pida `?incluirInactivas=true`. Pero si tiene stock, sigue apareciendo en el inventario de HU-10: son dos preguntas distintas — “qué hay” y “qué hay que comprar”.

R-06 · La sugerencia de compra agrupa por proveedor

El endpoint `/sugerencia-compra` propone qué comprar y a quién, deduciendo el proveedor de la **última compra recibida** de cada materia prima.

La cantidad sugerida es el faltante más un margen del 20%, redondeado hacia arriba: comprar exactamente el faltante deja la materia prima justo en el mínimo, y al primer consumo vuelve a estar en alerta.

Es una sugerencia, no una compra automática: quien compra decide.

R-07 · Sin notificaciones automáticas

No hay correo ni notificaciones push: no está en los criterios de aceptación ni en el stack. La alerta se consulta activamente o se ve en el indicador global de la barra superior.

Se registra como mejora posible.

4. Modelo de datos

Sin cambios de esquema. Consulta la vista `v_inventario_materia_prima` creada en V2.

```
// GET /api/v1/inventario/alertas
{
  "generadoEn": "2026-09-18T10:42:33",
  "resumen": { "total": 3, "agotadas": 1, "bajas": 2 },
  "items": [
    { "idMateriaPrima": 7, "nombre": "Levadura", "unidad": "kg",
      "cantidadDisponible": 0.00, "stockMinimo": 5.00, "faltante": 5.00,
      "nivel": "AGOTADO", "costoUnitario": 180.00,
      "costoReposicion": 1080.00,
      "ultimaActualizacion": "2026-09-18T09:12:44" },
    { "idMateriaPrima": 12, "nombre": "Mantequilla", "unidad": "kg",
      "cantidadDisponible": 4.50, "stockMinimo": 20.00, "faltante": 15.50,
      "nivel": "BAJO", "costoUnitario": 18500.00,
      "costoReposicion": 344100.00,
      "ultimaActualizacion": "2026-09-17T14:20:10" },
    { "idMateriaPrima": 5, "nombre": "Harina de trigo", "unidad": "kg",
      "cantidadDisponible": 46.30, "stockMinimo": 50.00, "faltante": 3.70,
      "nivel": "BAJO", "costoUnitario": 3200.00,
      "costoReposicion": 14208.00,
      "ultimaActualizacion": "2026-09-18T10:22:41" }
  ],
  "costoReposicionTotal": 359388.00
}

// GET /api/v1/inventario/alertas/resumen
{ "total": 3, "agotadas": 1, "bajas": 2 }

// GET /api/v1/inventario/alertas/sugerencia-compra
[
  { "proveedor": { "idProveedor": 3, "nombre": "Harinas del Caribe S.A.S." },
    "items": [
      { "materiaPrima": "Harina de trigo", "faltante": 3.70,
        "cantidadSugerida": 4.44, "costoEstimado": 14208.00 }
    ],
    "totalEstimado": 14208.00 },
  { "proveedor": { "idProveedor": 5, "nombre": "Distribuidora Lácteos" },
    "items": [
      { "materiaPrima": "Mantequilla", "faltante": 15.50,
        "cantidadSugerida": 18.60, "costoEstimado": 344100.00 }
    ],
    "totalEstimado": 344100.00 }
]
```

El orden de `items` es `AGOTADO` primero, luego `BAJO` por faltante descendente (R-03). `costoReposicion` usa `materias_primas.costo_unitario`, que es un valor de referencia (HU-08 R-03).

5. Códigos de error

code	HTTP	Cuándo
RANGO_INVALIDO	400	Parámetro de filtro mal formado

Los transversales (`VALIDACION_FALLIDA`, `NO_AUTENTICADO`, `SIN_PERMISO`, `RECURSO_NO_ENCONTRADO`) están en `00-base/04-CONTRATO-API.md` §5.

6. Fuera de alcance

Lo que **no** cubre esta historia y no debe implementarse aquí: se limita estrictamente a los criterios de aceptación listados arriba. Cualquier funcionalidad adicional se propone como historia nueva, no se agrega en silencio.

7. Definición de Terminado

Se aplica la DoD de `00-base/01-CONVENCIONES.md` §7.

8. Notas

El cierre del ciclo de inventario

Historia	Papel
HU-08	Define el <code>stock_minimo</code> de cada materia prima
HU-13	Suma stock al recibir compras
HU-23	Resta stock al producir
HU-10	Muestra el estado actual
HU-27	Avisa cuándo actuar
HU-13	Y el ciclo vuelve a empezar con una compra

Es la historia que convierte el inventario de un registro pasivo en una herramienta operativa. Si HU-27 no muestra alertas cuando debería, el problema casi nunca está aquí: está en que alguna materia prima no tiene fila de inventario (HU-08 R-01) o en que las compras no están sumando (§C-03).

Diseño

1. Pantallas

Alertas de stock

Alertas de stock

☐ Incluir inactivas

3 en alerta

1 agotada

\$ 359.388
costo estimado de reposición

MATERIA PRIMA	DISPONIBLE	MÍNIMO	NIVEL	FALTA	REPOSICIÓN
Levadura	0,00 kg	5,00 kg	[]	5,00 kg	\$ 1.080
AGOTADA · bloquea la producción					
● Mantequilla	4,50 kg	20,00 kg	[]	15,50 kg	\$ 344.100
● Harina de trigo	46,30 kg	50,00 kg	[]	3,70 kg	\$ 14.208

[Ver sugerencia de compra →]

Las agotadas van primero aunque les falte menos (R-03): la levadura bloquea la producción hoy; a la mantequilla le falta más pero todavía permite trabajar.

La leyenda “bloquea la producción” bajo las agotadas explica por qué encabezan la lista.

Sugerencia de compra

← Sugerencia de compra

Agrupada por el proveedor de la última compra de cada insumo.
Las cantidades incluyen un margen del 20% sobre el faltante.

📦 Harinas del Caribe S.A.S.

Harina de trigo falta 3,70 kg → comprar 4,44 kg \$ 14.208
Total: \$ 14.208
[Crear compra a este prov.]

📦 Distribuidora Lácteos

Mantequilla falta 15,50 kg → comprar 18,60 kg \$ 344.100
Total: \$ 344.100
[Crear compra a este prov.]

⚠ Levadura no tiene compras previas registradas.
Seleccione un proveedor manualmente.

[Crear compra]

“Crear compra” prellena el formulario de HU-13 con el proveedor y los ítems sugeridos. Es lo que cierra el ciclo: de detectar el problema a resolverlo sin volver a teclear nada.

El margen del 20% (R-06) evita el efecto de comprar justo el faltante y volver a estar en alerta al primer consumo.

Indicador global

☰

ProductionFood

3 alertas

Carlos C. · COMPRAS

[🔊]

|

Visible desde cualquier pantalla. Rojo si hay alguna agotada, ámbar si solo hay bajas.

Sin alertas — una buena noticia

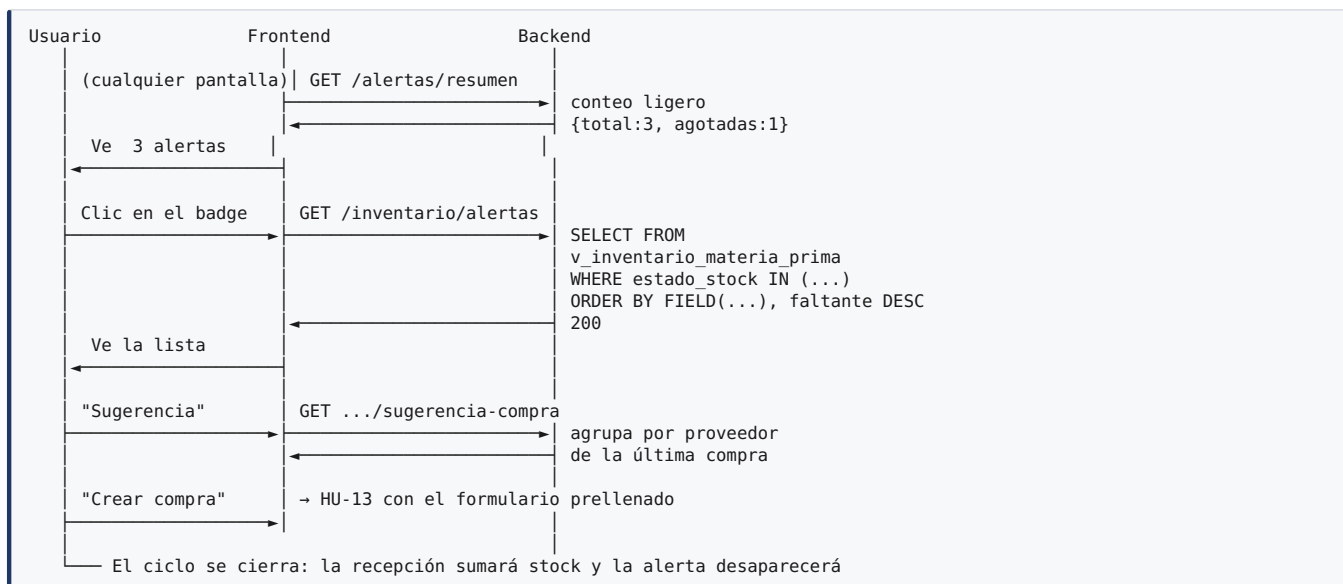
👉

Todas las materias primas están por encima de su stock mínimo

Última verificación: 18/09/2026 10:42

En verde y con un mensaje afirmativo. El mensaje genérico de “no se encontraron resultados” haría dudar de si la consulta funcionó.

2. Flujo



3. Componentes Angular Material

Elemento	Componente
Tarjetas de resumen	<code>MatCard</code>
Tabla	<code>MatTable</code> ordenada por urgencia
Nivel de stock	<code>MatProgressBar</code> con color
Nivel	<code>MatChip</code> — rojo AGOTADO , ámbar BAJO
Indicador global	<code>MatBadge</code> sobre <code>MatIcon</code> en <code>MatToolbar</code>
Sugerencia	<code>MatExpansionPanel</code> por proveedor
Crear compra	<code>MatRaisedButton</code> con navegación y estado prellenado
Estado vacío	<code>MatIcon</code> <code>check_circle</code> verde

4. Validaciones en el formulario

Pantalla de consulta: **sin formularios de datos**. Un solo filtro:

Filtro	Regla
Incluir inactivas	Booleano, desactivado por defecto (R-05)

El `stock_minimo` que define cuándo alertar se configura en HU-08, no aquí.

5. Estados de la pantalla

Estado	Qué se muestra
Cargando	<code>MatProgressBar</code> + tarjetas en esqueleto
Sin alertas	✔ Verde: "Todas las materias primas están por encima del mínimo"
Con alertas	Tabla ordenada, agotadas primero
Sin compras previas de un insumo	⚠ En la sugerencia, con opción de elegir proveedor
Error	Mensaje + "Reintentar"

El estado vacío es el caso deseable, y debe comunicarlo: verde, afirmativo y con la hora de la verificación. Un "sin resultados" genérico transmite lo contrario.

6. Accesibilidad y UX

- Todos los campos con `<mat-label>` ; nunca solo *placeholder*.
- Navegación por teclado en orden lógico; `Enter` envía el formulario.
- Los mensajes de error del servidor se muestran **junto al campo** (`fieldErrors`).
- El botón de envío se deshabilita mientras la petición está en vuelo.

- Confirmación antes de descartar cambios sin guardar.
- Los tres estados obligatorios (cargando / vacío / error) según `00-base/05-ESTANDARES-QA.md` §7.

Tarea Backend

La historia más simple del proyecto: una consulta sobre la vista de V2. El valor está en el ordenamiento por urgencia y en la sugerencia de compra.

1. Pasos

1. `AlertaStockRowMapper` sobre `v_inventario_materia_prima`.
2. `AlertaRepository.listarAlertas(incluirInactivas)`:

```
SELECT id_materia_prima, nombre, unidad, cantidad_disponible,
       stock_minimo, faltante, estado_stock, costo_unitario, fecha_actualizacion
FROM v_inventario_materia_prima
WHERE estado_stock IN ('BAJO', 'AGOTADO')
AND (:incluirInactivas OR estado = 1)
ORDER BY FIELD(estado_stock, 'AGOTADO', 'BAJO'), faltante DESC
```

El `FIELD()` implementa R-03: agotadas primero, luego por faltante.

3. `AlertaRepository.resumen()` — conteos para el indicador global.
4. `AlertaRepository.sugerenciaCompra()` (R-06):
 - por cada materia prima en alerta, buscar el proveedor de su **última compra RECIBIDA** (`ORDER BY c.fecha DESC LIMIT 1`);
 - `cantidadSugerida = faltante × 1.2`, redondeada hacia arriba a 2 decimales;
 - agrupar por proveedor.
5. `costoReposicion = faltante × costo_unitario`, con `BigDecimal`.
6. `AlertaService` con `@Transactional(readOnly = true)`.
7. `AlertaController` — solo `GET`. El endpoint `/resumen` abierto a todos los roles autenticados (alimenta el indicador global de HU-23).
8. **Sin caché**: el conteo alimenta un indicador que debe reflejar la realidad.

2. Archivos

```
backend/src/main/java/co/edu/corposucre/productionfood/inventario/
├── {AlertaRepository,AlertaService,AlertaController}.java
├── AlertaStockRowMapper.java
└── dto/{AlertaStockResponse,ResumenAlertasResponse,SugerenciaCompraResponse}.java
```

3. Puntos de cuidado

- `FIELD(estado_stock, 'AGOTADO', 'BAJO')` en el `ORDER BY` implementa R-03 en una sola expresión. CA-04 pide ordenar por diferencia; ese orden se aplica **dentro** de cada nivel, porque una materia prima agotada siempre es más urgente.
- `<, no <=` (R-02): con disponible igual al mínimo no hay alerta. La vista ya lo resuelve, pero conviene no reintroducir la comparación en el servicio.
- **Excluir las inactivas por defecto** (R-05): alertar sobre algo que no se va a reponer es ruido permanente.
- El endpoint `/resumen` debe ser muy ligero: se consulta desde todas las pantallas.
- Si la alerta no muestra nada cuando debería, revisar primero que todas las materias primas tengan fila de inventario (HU-08 R-01).

4. Checklist de cierre

- ☐ Todos los CA implementados y verificados manualmente.
- ☐ Endpoints documentados en Swagger con ejemplos.
- ☐ `@PreAuthorize` en cada método, según la matriz de roles.
- ☐ Operaciones de escritura auditadas en bitácora (HU-04).
- ☐ Pruebas unitarias de las reglas con ramificación.
- ☐ Sin `System.out.println` ni código comentado.

☐ PR bajo ~250 líneas, revisado y aprobado.

Tarea Frontend

Pantalla de alertas más el indicador global de la barra superior. Debe llevar a la acción: de la alerta a la compra en un clic.

1. Pasos

1. `alerta.model.ts` y `alerta.service.ts`.
2. `alertas-stock.component`:
 - tabla ordenada por urgencia: agotadas primero (R-03);
 - columnas: materia prima, disponible, mínimo, **falta**, nivel, costo de reposición.
3. **Barra visual de nivel** por fila: `disponible / minimo`, en rojo si es cero.
4. Tarjetas de resumen: total en alerta, agotadas, costo de reposición estimado.
5. `sugerencia-compra.component`: propuesta agrupada por proveedor, con botón **“Crear compra”** que prellena el formulario de HU-13 con esos ítems.
6. **Indicador global** en la barra superior (`MatBadge`), que consulta `/alertas/resumen` al cargar y al volver a la pestaña.
7. Filtro para incluir materias primas inactivas (R-05), desactivado por defecto.
8. Estado vacío en **verde**: no tener alertas es una buena noticia.
9. Ruta: ADMIN/PRODUCCION/COMPRAS/CONSULTA. La sugerencia de compra, ADMIN y COMPRAS.

2. Archivos

```
frontend/src/app/features/inventario/  
├── models/alerta.model.ts  
├── services/alerta.service.ts  
├── pages/{alertas-stock,sugerencia-compra}/  
├── components/nivel-stock-bar/  
frontend/src/app/layout/components/indicador-alertas/
```

3. Puntos de cuidado

- **El botón “Crear compra” es lo que cierra el ciclo.** Una alerta que obliga a apuntar en papel qué comprar y volver a teclearlo en otra pantalla se acaba ignorando.
- **El estado vacío va en verde con un mensaje positivo:** “Todas las materias primas están por encima del mínimo”. Usar el mensaje genérico de “sin resultados” hace dudar de si la consulta falló.
- **El indicador global no debe refrescarse agresivamente:** al cargar y al volver a la pestaña basta. Un sondeo cada pocos segundos multiplica las consultas sin aportar nada.
- Las agotadas deben distinguirse visualmente de las bajas (R-03): son urgencias distintas.
- El costo de reposición es una referencia (HU-08 R-03); indicarlo para que no se tome como cotización.

4. Checklist de cierre

- ☐ Todos los CA cubiertos desde la interfaz.
- ☐ Los tres estados (cargando / vacío / error) implementados.
- ☐ Errores del servidor mostrados junto al campo correspondiente.
- ☐ Guard de rol aplicado en la ruta.
- ☐ Modelos TypeScript espejo de los DTO del backend.
- ☐ Probado en Chrome y Firefox a 1366×768.
- ☐ Sin `console.log` ni `any`.

Casos de Prueba

Formato y criterios en `00-base/05-ESTANDARES-QA.md`.

1. Casos funcionales

ID	CA	Descripción	Entrada	Resultado esperado	Tipo
CP-01	CA-01	Listar alertas	2 bajo mínimo, 1 agotada	200 con 3 ítems	Positivo
CP-02	CA-01	Stock exactamente igual al mínimo	disponible 50, mínimo 50	No aparece — la comparación es < (R-02)	Borde
CP-03	CA-01	Una unidad por debajo	disponible 49,99, mínimo 50	Aparece con faltante: 0.01	Borde
CP-04	CA-01	Stock por encima	disponible 96, mínimo 50	No aparece	Positivo
CP-05	CA-02	Se muestran los tres datos	Revisar CP-01	Nombre, disponible y mínimo presentes	Positivo
CP-06	CA-03	Cálculo del faltante	disponible 46,30, mínimo 50	faltante: 3.70	Positivo
CP-07	CA-03	Faltante con stock en cero	disponible 0, mínimo 5	faltante: 5.00	Borde
CP-08	CA-04	Orden por urgencia	Agotada (falta 5) + baja (falta 40)	La agotada primero , aunque falte menos (R-03)	Integridad
CP-09	CA-04	Orden dentro del mismo nivel	Dos BAJO : faltan 15,5 y 3,7	La de 15,5 primero (CA-04)	Positivo
CP-10	—	stock_mínimo = 0 no genera alerta BAJO	mínimo 0, disponible 10	No aparece (R-04)	Borde
CP-11	—	Mínimo cero y disponible cero	mínimo 0, disponible 0	Aparece como AGOTADO (R-04)	Borde
CP-12	—	Las inactivas se excluyen	MP desactivada bajo mínimo	No aparece por defecto (R-05)	Integridad
CP-13	—	Incluir inactivas a petición	? <code>incluirInactivas=true</code>	Aparece	Positivo
CP-14	—	Materia prima sin fila de inventario	MP creada sin inventario (defecto HU-08)	Aparece como AGOTADO con disponible 0	Borde
CP-15	—	Costo de reposición	faltante 3,70 × costo 3.200	costoReposicion: 11840.00	Positivo
CP-16	—	Resumen para el indicador	<code>GET /alertas/resumen</code>	{total, agotadas, bajas} coherentes con el listado	Positivo
CP-17	—	Sugerencia de compra	<code>GET /alertas/sugerencia-compra</code>	Agrupado por proveedor de la última compra recibida (R-06)	Positivo
CP-18	—	Cantidad sugerida con margen	faltante 3,70	cantidadSugerida: 4.44 (+20%, R-06)	Positivo
CP-19	—	MP sin compras previas	MP nunca comprada, en alerta	Aparece sin proveedor asignado, sin error	Borde
CP-20	—	Sin alertas	Todo por encima del mínimo	items: [], total: 0 — sin error	Borde
CP-21	—	Coherencia con el dashboard	Comparar con HU-24 CA-03	Las mismas materias primas en ambos	Integridad
CP-22	—	Coherencia tras una producción	Registrar consumo que deje bajo mínimo (HU-23)	La MP aparece de inmediato en las alertas	Integridad
CP-23	—	⌚ Rendimiento	200 materias primas	< 2 s (RNF-02)	Rendimiento
CP-24	—	RoI CONSULTA puede ver alertas	<code>GET /alertas</code> con token CONSULTA	200	Positivo
CP-25	—	RoI VENTAS no accede a la sugerencia de compra	<code>GET .../sugerencia-compra</code> con token VENTAS	403 SIN_PERMISO	Seguridad
CP-26	—	Sin endpoints de escritura	<code>POST /inventario/alertas</code>	405 o 404	Seguridad

2. Batería de seguridad

Obligatoria en **cada endpoint** de esta historia:

ID	Caso	Esperado
SEC-01	Sin cabecera <code>Authorization</code>	<code>401 NO_AUTENTICADO</code>
SEC-02	Token malformado	<code>401 NO_AUTENTICADO</code>
SEC-03	Token expirado	<code>401 NO_AUTENTICADO</code>
SEC-04	Token de rol <code>sin</code> permiso	<code>403 SIN_PERMISO</code>
SEC-05	Token de usuario con <code>estado = 0</code>	<code>403 USUARIO_INACTIVO</code>

SEC-04 es el caso crítico. Si devuelve `200`, falta `@EnableMethodSecurity` y la autorización del sistema completo está desactivada.

CP-08 es el caso que distingue una lista útil de una lista ordenada. CA-04 pide ordenar por diferencia, y tomado literalmente pondría una materia prima con 40 kg disponibles y 60 de faltante por encima de una agotada a la que le faltan 5. Pero la agotada **bloquea la producción hoy**; la otra todavía permite trabajar. Por eso el nivel manda sobre el faltante (R-03), y el orden por diferencia se aplica dentro de cada nivel.

CP-14 y CP-22 son pruebas de coherencia del sistema completo. Si esta pantalla no muestra alertas cuando debería, el defecto rara vez está aquí: suele ser una materia prima sin fila de inventario (HU-08 R-01) o compras que no están sumando (§C-03).

3. Verificación en base de datos

```
-- CP-01 a CP-09: la consulta completa con su orden de urgencia
SELECT nombre, unidad, cantidad_disponible, stock_minimo, faltante, estado_stock
FROM v_inventario_materia_prima
WHERE estado_stock IN ('BAJO', 'AGOTADO') AND estado = 1
ORDER BY FIELD(estado_stock, 'AGOTADO', 'BAJO'), faltante DESC;
-- Las AGOTADO deben aparecer primero aunque su faltante sea menor (R-03)

-- CP-02 y CP-03: la comparación es estricta
SELECT nombre, cantidad_disponible, stock_minimo, estado_stock,
       cantidad_disponible < stock_minimo AS deberia_alertar
FROM v_inventario_materia_prima
WHERE cantidad_disponible = stock_minimo;
-- deberia_alertar = 0 : con disponible igual al mínimo NO hay alerta

-- CP-10, CP-11: el caso de stock_minimo = 0
SELECT nombre, cantidad_disponible, stock_minimo, estado_stock
FROM v_inventario_materia_prima WHERE stock_minimo = 0;
-- Con disponible > 0 → 'OK'. Con disponible = 0 → 'AGOTADO'.

-- CP-14: materias primas sin fila de inventario (defecto de HU-08 R-01)
SELECT mp.nombre FROM materias_primas mp
LEFT JOIN inventario i USING (id_materia_prima)
WHERE i.id_inventario IS NULL;
-- DEBE devolver 0 filas. Si devuelve alguna, esa materia prima aparecerá
-- siempre como AGOTADA aunque tenga existencias físicas.

-- CP-17: el proveedor de la última compra recibida (R-06)
SELECT dc.id_materia_prima, p.nombre AS proveedor, MAX(c.fecha) AS ultima_compra
FROM detalle_compra dc
JOIN compras c USING (id_compra)
JOIN proveedores p USING (id_proveedor)
WHERE c.estado = 'RECIBIDA'
GROUP BY dc.id_materia_prima, p.id_proveedor, p.nombre;

-- CP-21, CP-22: coherencia con el resto del sistema
-- El conteo de esta historia debe coincidir con la tarjeta del dashboard (HU-24 CA-03)
SELECT COUNT(*) AS en_alerta,
       SUM(estado_stock = 'AGOTADO') AS agotadas
FROM v_inventario_materia_prima
WHERE estado_stock IN ('BAJO', 'AGOTADO') AND estado = 1;
```

4. Criterios de cierre

- ☐ Todos los casos ejecutados; los positivos pasan.
- ☐ Los negativos devuelven el HTTP y el `code` esperados.
- ☐ SEC-01 a SEC-05 pasan en todos los endpoints.
- ☐ Ningún endpoint supera 2 s (RNF-02).
- ☐ Cero defectos críticos o altos abiertos.
- ☐ Colección de Postman actualizada y versionada.

Parte III · Anexos

Scripts de migración

V2_correcciones.sql

```
-- =====
-- V2_correcciones.sql
-- Correcciones al esquema base. Ver docs/00-base/02-CORRECCIONES-DB.md
-- Motor: MySQL 8.4 LTS en Amazon RDS
-- Idempotencia: este script NO es idempotente. Ejecutar una sola vez sobre V1.
--
-- SINTAXIS MySQL – dos diferencias frente a MariaDB que rompen el script si se ignoran:
-- 1. Las expresiones en DEFAULT van ENTRE PARENTESIS: DEFAULT (curdate())
-- Sin parentesis, MySQL devuelve un error de sintaxis.
-- Excepcion historica: CURRENT_TIMESTAMP en DATETIME/TIMESTAMP no los lleva.
-- 2. int(10) unsigned -> int unsigned (ancho obsoleto desde 8.0.17)
-- =====

START TRANSACTION;

-----
-- C-06 · estado con DEFAULT 1
-----
ALTER TABLE `usuarios`
  MODIFY `estado` tinyint(1) NOT NULL DEFAULT 1;

ALTER TABLE `materias_primas`
  MODIFY `estado` tinyint(1) NOT NULL DEFAULT 1;

ALTER TABLE `productos`
  MODIFY `estado` tinyint(1) NOT NULL DEFAULT 1;

ALTER TABLE `recetas`
  MODIFY `estado` tinyint(1) NOT NULL DEFAULT 1;

-----
-- C-07 · Fechas automáticas
-----
ALTER TABLE `pedidos`
  MODIFY `fecha_pedido` date NOT NULL DEFAULT (curdate());

ALTER TABLE `devoluciones`
  MODIFY `fecha_devolucion` date NOT NULL DEFAULT (curdate());

ALTER TABLE `inventario`
  MODIFY `fecha_actualizacion` datetime NOT NULL
  DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP;

-----
-- C-01 · precio_unitario en detalle_pedido
-- El DEFAULT 0 existe solo para permitir el ALTER sobre filas preexistentes.
-- Se retira acto seguido: la aplicación SIEMPRE envía el precio explícito.
-----
ALTER TABLE `detalle_pedido`
  ADD COLUMN `precio_unitario` decimal(10,2) NOT NULL DEFAULT 0.00 AFTER `cantidad`;

UPDATE `detalle_pedido` dp
  JOIN `productos` p ON p.id_producto = dp.id_producto
  SET dp.precio_unitario = p.precio
  WHERE dp.precio_unitario = 0.00;

ALTER TABLE `detalle_pedido`
  MODIFY `precio_unitario` decimal(10,2) NOT NULL;

-----
-- C-08 · cantidad_real obligatoria
-----
UPDATE `consumo_materia_prima` SET `cantidad_real` = 0.000 WHERE `cantidad_real` IS NULL;

ALTER TABLE `consumo_materia_prima`
  MODIFY `cantidad_real` decimal(10,3) NOT NULL;

-----
-- C-11 · bitacora.id_usuario nullable (NULL = acción del sistema)
-----
ALTER TABLE `bitacora`
  MODIFY `id_usuario` int unsigned NULL;

-----
-- C-05 y C-13 · UNIQUE en líneas de detalle
-- Si alguno falla por datos duplicados preexistentes, depurar antes de reintentar.
-----
ALTER TABLE `detalle_receta`
  ADD UNIQUE KEY `uk_detalle_receta` (`id_receta`, `id_materia_prima`);
```

```

ALTER TABLE `detalle_compra`
  ADD UNIQUE KEY `uk_detalle_compra` (`id_compra`, `id_materia_prima`);

ALTER TABLE `detalle_pedido`
  ADD UNIQUE KEY `uk_detalle_pedido` (`id_pedido`, `id_producto`);

ALTER TABLE `detalle_plan_produccion`
  ADD UNIQUE KEY `uk_detalle_plan` (`id_plan`, `id_producto`);

ALTER TABLE `consumo_materia_prima`
  ADD UNIQUE KEY `uk_consumo_mp` (`id_detalle_plan`, `id_materia_prima`);

-----
-- C-14 · CHECK de valores positivos
-----

ALTER TABLE `productos`
  ADD CONSTRAINT `ck_productos_precio` CHECK (`precio` >= 0);

ALTER TABLE `materias_primas`
  ADD CONSTRAINT `ck_mp_stock_minimo` CHECK (`stock_minimo` >= 0),
  ADD CONSTRAINT `ck_mp_costo` CHECK (`costo_unitario` >= 0);

ALTER TABLE `inventario`
  ADD CONSTRAINT `ck_inventario_no_negativo` CHECK (`cantidad_disponible` >= 0);

ALTER TABLE `inventario_producto_terminado`
  ADD CONSTRAINT `ck_inv_pt_no_negativo` CHECK (`cantidad_disponible` >= 0);

ALTER TABLE `detalle_compra`
  ADD CONSTRAINT `ck_dc_cantidad` CHECK (`cantidad` > 0),
  ADD CONSTRAINT `ck_dc_precio` CHECK (`precio_unitario` >= 0);

ALTER TABLE `detalle_pedido`
  ADD CONSTRAINT `ck_dp_cantidad` CHECK (`cantidad` > 0),
  ADD CONSTRAINT `ck_dp_precio` CHECK (`precio_unitario` >= 0);

ALTER TABLE `detalle_receta`
  ADD CONSTRAINT `ck_dr_cantidad` CHECK (`cantidad` > 0);

ALTER TABLE `detalle_plan_produccion`
  ADD CONSTRAINT `ck_dpp_planificada` CHECK (`cantidad_planificada` > 0),
  ADD CONSTRAINT `ck_dpp_producida` CHECK (`cantidad_producida` >= 0);

ALTER TABLE `consumo_materia_prima`
  ADD CONSTRAINT `ck_cmp_teorica` CHECK (`cantidad_teorica` >= 0),
  ADD CONSTRAINT `ck_cmp_real` CHECK (`cantidad_real` >= 0);

ALTER TABLE `devoluciones`
  ADD CONSTRAINT `ck_dev_cantidad` CHECK (`cantidad_devuelta` > 0);

ALTER TABLE `lotes`
  ADD CONSTRAINT `ck_lotes_cantidad` CHECK (`cantidad` > 0);

-----
-- C-10 · Integridad de lotes
-- Un lote es de un producto terminado O de una materia prima. Nunca ambos, nunca ninguno.
-----

ALTER TABLE `lotes`
  ADD CONSTRAINT `ck_lotes_exclusividad` CHECK (
    (`id_producto` IS NOT NULL AND `id_materia_prima` IS NULL)
    OR (`id_producto` IS NULL AND `id_materia_prima` IS NOT NULL)
  ),
  ADD CONSTRAINT `ck_lotes_fechas` CHECK (`fecha_vencimiento` >= `fecha_produccion`);

-----
-- C-12 · Estados canónicos
-----

UPDATE `compras` SET `estado` = upper(replace(`estado`, ' ', '_'));
UPDATE `pedidos` SET `estado` = upper(replace(`estado`, ' ', '_'));
UPDATE `devoluciones` SET `estado` = upper(replace(`estado`, ' ', '_'));
UPDATE `planes_produccion` SET `estado` = upper(replace(`estado`, ' ', '_'));

ALTER TABLE `compras`
  ADD CONSTRAINT `ck_compras_estado`
  CHECK (`estado` IN ('PENDIENTE', 'RECIBIDA', 'CANCELADA'));

ALTER TABLE `pedidos`
  ADD CONSTRAINT `ck_pedidos_estado`
  CHECK (`estado` IN ('PENDIENTE', 'EN_PREPARACION', 'ENTREGADO', 'CANCELADO'));

ALTER TABLE `devoluciones`
  ADD CONSTRAINT `ck_devoluciones_estado`
  CHECK (`estado` IN ('PENDIENTE', 'APROBADA', 'RECHAZADA'));

ALTER TABLE `planes_produccion`
  ADD CONSTRAINT `ck_planes_estado`
  CHECK (`estado` IN ('PLANIFICADO', 'EN_PROCESO', 'COMPLETADO', 'CANCELADO'));

-----
-- C-03 · Trazabilidad lote ↔ compra

```

-- Evita la doble suma al inventario cuando un lote nace de una compra ya recibida.

```
ALTER TABLE `lotes`
ADD COLUMN `id_detalle_compra` int unsigned DEFAULT NULL AFTER `id_materia_prima`,
ADD KEY `idx_lotes_detalle_compra` (`id_detalle_compra`),
ADD CONSTRAINT `lotes_ibfk_3` FOREIGN KEY (`id_detalle_compra`)
REFERENCES `detalle_compra` (`id_detalle_compra`) ON DELETE NO ACTION ON UPDATE NO ACTION;
```

-- C-04 · Kardex de materias primas

```
CREATE TABLE `movimientos_inventario_mp` (
  `id_movimiento` bigint unsigned NOT NULL AUTO_INCREMENT,
  `id_materia_prima` int unsigned NOT NULL,
  `tipo_movimiento` varchar(30) NOT NULL,
  `cantidad` decimal(10,3) NOT NULL,
  `saldo_anterior` decimal(10,2) NOT NULL,
  `saldo_posterior` decimal(10,2) NOT NULL,
  `tabla_origen` varchar(50) DEFAULT NULL,
  `id_origen` int unsigned DEFAULT NULL,
  `observacion` varchar(255) DEFAULT NULL,
  `id_usuario` int unsigned DEFAULT NULL,
  `fecha` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`id_movimiento`),
  KEY `idx_mov_mp_materia` (`id_materia_prima`, `fecha`),
  KEY `idx_mov_mp_origen` (`tabla_origen`, `id_origen`),
  CONSTRAINT `mov_mp_ibfk_1` FOREIGN KEY (`id_materia_prima`)
REFERENCES `materias_primas` (`id_materia_prima`),
  CONSTRAINT `mov_mp_ibfk_2` FOREIGN KEY (`id_usuario`)
REFERENCES `usuarios` (`id_usuario`),
  CONSTRAINT `ck_mov_mp_tipo` CHECK (`tipo_movimiento` IN (
    'ENTRADA_COMPRA', 'ENTRADA_LOTE', 'SALIDA_PRODUCCION',
    'AJUSTE_POSITIVO', 'AJUSTE_NEGATIVO', 'BAJA_VENCIMIENTO')),
  CONSTRAINT `ck_mov_mp_cantidad` CHECK (`cantidad` > 0)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

-- C-04 · Kardex de producto terminado (por lote)

```
CREATE TABLE `movimientos_inventario_pt` (
  `id_movimiento` bigint unsigned NOT NULL AUTO_INCREMENT,
  `id_lote` int unsigned NOT NULL,
  `tipo_movimiento` varchar(30) NOT NULL,
  `cantidad` decimal(10,2) NOT NULL,
  `saldo_anterior` decimal(10,2) NOT NULL,
  `saldo_posterior` decimal(10,2) NOT NULL,
  `tabla_origen` varchar(50) DEFAULT NULL,
  `id_origen` int unsigned DEFAULT NULL,
  `observacion` varchar(255) DEFAULT NULL,
  `id_usuario` int unsigned DEFAULT NULL,
  `fecha` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`id_movimiento`),
  KEY `idx_mov_pt_lote` (`id_lote`, `fecha`),
  KEY `idx_mov_pt_origen` (`tabla_origen`, `id_origen`),
  CONSTRAINT `mov_pt_ibfk_1` FOREIGN KEY (`id_lote`) REFERENCES `lotes` (`id_lote`),
  CONSTRAINT `mov_pt_ibfk_2` FOREIGN KEY (`id_usuario`) REFERENCES `usuarios` (`id_usuario`),
  CONSTRAINT `ck_mov_pt_tipo` CHECK (`tipo_movimiento` IN (
    'ENTRADA_PRODUCCION', 'SALIDA_VENTA', 'ENTRADA_DEVOLUCION',
    'AJUSTE_POSITIVO', 'AJUSTE_NEGATIVO', 'BAJA_VENCIMIENTO')),
  CONSTRAINT `ck_mov_pt_cantidad` CHECK (`cantidad` > 0)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

-- C-15 · Índices para los filtros exigidos por las HU

-- Nota: aceleran LIKE 'texto%' (prefijo), no LIKE '%texto%'.

```
ALTER TABLE `clientes` ADD KEY `idx_clientes_nombre` (`nombre`);
ALTER TABLE `proveedores` ADD KEY `idx_proveedores_nombre` (`nombre`);
ALTER TABLE `materias_primas` ADD KEY `idx_mp_nombre` (`nombre`);
ALTER TABLE `productos` ADD KEY `idx_productos_nombre` (`nombre`);
ALTER TABLE `usuarios` ADD KEY `idx_usuarios_nombre` (`nombre`);
ALTER TABLE `compras` ADD KEY `idx_compras_fecha` (`fecha`, `estado`);
ALTER TABLE `pedidos` ADD KEY `idx_pedidos_fecha` (`fecha_pedido`, `estado`);
ALTER TABLE `planes_produccion` ADD KEY `idx_planes_fecha` (`fecha_produccion`, `estado`);
ALTER TABLE `lotes` ADD KEY `idx_lotes_vencimiento` (`fecha_vencimiento`);
ALTER TABLE `bitacora` ADD KEY `idx_bitacora_tabla` (`tabla_afectada`, `fecha`);
ALTER TABLE `devoluciones` ADD KEY `idx_dev_estado` (`estado`, `fecha_devolucion`);
```

-- C-02 · Vista de stock de producto terminado agregado por producto

-- Excluye lotes vencidos: producto caducado no es stock vendible.

```
CREATE OR REPLACE SQL SECURITY INVOKER VIEW `v_stock_producto_terminado` AS
SELECT
  p.id_producto,
  p.nombre AS nombre_producto,
  p.precio,
  u.abreviatura AS unidad,
  COALESCE(SUM(ipt.cantidad_disponible), 0) AS cantidad_disponible,
  COUNT(DISTINCT l.id_lote) AS lotes_activos,
  MIN(l.fecha_vencimiento) AS proximo_vencimiento
```

```

FROM productos p
JOIN unidades_medida u ON u.id_unidad = p.id_unidad
LEFT JOIN lotes l
    ON l.id_producto = p.id_producto
    AND l.fecha_vencimiento >= curdate()
LEFT JOIN inventario_producto_terminado ipt
    ON ipt.id_lote = l.id_lote
    AND ipt.cantidad_disponible > 0
GROUP BY p.id_producto, p.nombre, p.precio, u.abreviatura;

-----
-- C-02 · Lotes disponibles en orden FEFO (First Expired, First Out)
-- El servicio de ventas consume de esta vista en orden para descontar stock.
-----

CREATE OR REPLACE SQL SECURITY INVOKER VIEW `v_lotes_disponibles_fefo` AS
SELECT
    l.id_producto,
    l.id_lote,
    l.codigo_lote,
    l.fecha_vencimiento,
    ipt.id_inv_producto,
    ipt.cantidad_disponible,
    DATEDIFF(l.fecha_vencimiento, curdate()) AS dias_para_vencer
FROM lotes l
JOIN inventario_producto_terminado ipt ON ipt.id_lote = l.id_lote
WHERE l.id_producto IS NOT NULL
    AND ipt.cantidad_disponible > 0
    AND l.fecha_vencimiento >= curdate();
-- NOTA: MySQL ignora el ORDER BY declarado dentro de una vista. El orden FEFO
-- se aplica en la consulta que la usa:
-- SELECT * FROM v_lotes_disponibles_fefo
-- WHERE id_producto = ? ORDER BY fecha_vencimiento ASC, id_lote ASC;

-----
-- HU-10 / HU-27 · Inventario de materia prima con estado de stock
-----

CREATE OR REPLACE SQL SECURITY INVOKER VIEW `v_inventario_materia_prima` AS
SELECT
    mp.id_materia_prima,
    mp.nombre,
    u.abreviatura AS unidad,
    mp.stock_minimo,
    mp.costo_unitario,
    mp.estado,
    COALESCE(i.cantidad_disponible, 0) AS cantidad_disponible,
    i.fecha_actualizacion,
    CASE
        WHEN COALESCE(i.cantidad_disponible, 0) = 0 THEN 'AGOTADO'
        WHEN COALESCE(i.cantidad_disponible, 0) < mp.stock_minimo THEN 'BAJO'
        ELSE 'OK'
    END AS estado_stock,
    GREATEST(mp.stock_minimo - COALESCE(i.cantidad_disponible, 0), 0) AS faltante
FROM materias_primas mp
JOIN unidades_medida u ON u.id_unidad = mp.id_unidad
LEFT JOIN inventario i ON i.id_materia_prima = mp.id_materia_prima;

COMMIT;

```

V3_datos_semilla.sql

```

-- =====
-- V3_datos_semilla.sql · MySQL 8.4 LTS (Amazon RDS)
-- Datos mínimos para que el sistema arranque. Ver 02-CORRECCIONES-DB.md §C-16
-- =====

START TRANSACTION;

-- -----
-- Roles – ver 03-MATRIZ-ROLES.md
-- Los ids son fijos y se referencian desde el código. No reordenar.
-- -----

INSERT INTO `roles` (`id_rol`, `nombre`, `descripcion`) VALUES
(1, 'ADMIN', 'Administrador del sistema. Acceso total.'),
(2, 'PRODUCCION', 'Encargado de producción. Planes, recetas, consumo e inventario.'),
(3, 'COMPRAS', 'Encargado de compras. Proveedores, compras y recepción de materia prima.'),
(4, 'VENTAS', 'Encargado de ventas. Clientes, pedidos y devoluciones.'),
(5, 'CONSULTA', 'Usuario de consulta. Solo lectura sobre los módulos autorizados.');
```

-- Unidades de medida – catálogo base de una panadería

```

-- -----
INSERT INTO `unidades_medida` (`nombre`, `abreviatura`) VALUES
('Kilogramo', 'kg'),
('Gramo', 'g'),
('Litro', 'L'),
('Mililitro', 'mL'),
('Unidad', 'und'),
('Docena', 'doc'),
('Paquete', 'paq'),
('Bolsa', 'bol'),
('Bandeja', 'bdj');
```

-- Administrador inicial

--

-- ▲ ADVERTENCIA DE SEGURIDAD

-- El hash corresponde a la contraseña 'Admin123*'. Es una credencial pública,

-- conocida por cualquiera que lea este repositorio.

--

-- OBLIGATORIO antes de exponer la aplicación fuera de localhost:

-- 1. Iniciar sesión con este usuario.

-- 2. Crear un administrador real con contraseña propia.

-- 3. Desactivar este usuario (estado = 0).

--

-- Está registrado como tarea de cierre en docs/HU-01/tarea-backend.md.

-- Para regenerar el hash:

-- new BCryptPasswordEncoder().encode("<contraseña>")

```

-- -----
INSERT INTO `usuarios` (`nombre`, `correo`, `password`, `estado`, `id_rol`) VALUES
('Administrador Inicial',
'admin@productionfood.local',
'$2a$10$N9qo8uL0ickgx2ZMRZoMyeIjZAgcf17p92ldGxad68LJZdL17lhWy',
1,
1);
```

-- Registro del acto de siembra en bitácora (id_usuario NULL = sistema, ver C-11)

```

-- -----
INSERT INTO `bitacora` (`id_usuario`, `accion`, `tabla_afectada`, `detalle`) VALUES
(NULL, 'SEED', 'usuarios', 'Creación del administrador inicial por script de semilla V3');
```

COMMIT;

V1_esquema_base.sql (el esquema original adaptado a MySQL 8.4) se omite de este anexo por extensión. Está en [docs/00-base/migraciones/](#), junto con el [db.sql](#) tal como se entregó, conservado en [docs/previo/](#).